

Navis N4

Public APIs

3.8.40



CONFIDENTIAL INFORMATION OF Navis LP

UNAUTHORIZED USE IS STRICTLY PROHIBITED.

N4 is a trademark of Navis LP

© Navis 1988 - 2024. All rights reserved.

This help has been made available as part of the license that has been granted to an authorized user of Navis LP software. Use of this documentation is subject to the terms and limitations of that license agreement. This documentation describes all functionality that can be licensed for this product. Not all functionality described in this documentation may be available to you depending on your license agreement. If you are not aware of the relevant terms of your license agreement, contact sales at Kaleris. This document is protected by copyright and by the licensee's agreement to maintain the confidentiality of the content of this document as provided in the license agreement for the software.

Liability Disclaimer

Kaleris takes steps to ensure that its published documentation is correct; however, errors do occur. Kaleris reserves the right to correct any such errors and disclaims liability resulting therefrom.

Kaleris

3460 Preston Ridge Rd, Suite 600

Alpharetta, GA 30005 USA

www.kaleris.com

Contents

Part 1 Universal Query API	16
1.1 Introduction	17
1.1.1 REST Protocol	18
1.2 Security	19
1.2.1 Basic Authentication	20
1.2.2 HTTPS	20
1.3 Using the Universal Query	21
1.3.1 Configuring the Query	22
1.3.2 Configuring the Default Query	22
1.3.3 Configuring an SNX Query	23
1.3.4 Configuring a Recap Query	24
1.3.4.1 Query results for three columns	25
1.3.4.2 Query results for two columns	25
1.3.4.3 Query results for a single column	26
1.3.5 Constructing the URL	26
1.3.6 Parameters	27
1.3.7 Executing the Query	28
1.3.8 Troubleshooting	28
1.4 Limiting Access to the Query Url	30
1.5 Configure the Universal Query for the First Time	31
1.5.1 Configure Tomcat Security	32
1.5.2 Enable Universal Query	32
1.5.3 Create an N4 User	32
1.5.4 Giving roles to the n4api user	33
1.5.5 Create a Simple Filter	33
1.5.6 Test the API in your browser	33
1.5.7 Universal Query Java Integration Example	34
1.6 Boolean Parameters in Universal Query	36
1.6.1 UI Filter Screen	37
1.6.2 Optional boolean predicates	37
1.6.3 Required boolean predicates	38
1.6.4 Either required and/or optional predicates	39
1.6.5 Universal Query	40
1.7 Static Code Lookup	41
1.7.1 How to look up for static code values	42
1.7.2 Unit Freight Kinds	42
1.7.3 Work Instruction Move Types	42

1.7.4 Unit Dray Statuses.....	42
1.7.5 Unit Categories.....	43
1.7.6 Unit Hazards and IMDG Classes.....	43
Part 2 Holds Permissions Update (HPU) API.....	46
2.1 HPU API.....	47
2.1.1 Logical business entities.....	48
2.1.2 Service Rules.....	48
2.1.3 HPU format.....	48
2.1.3.1 Allowed actions.....	50
2.1.4 Create a Unit on the Fly.....	51
2.1.5 Update Process & Return Error Messages.....	51
2.1.6 Security.....	53
Part 3 Incremental Container Update (ICU) API.....	54
3.1 ICU API.....	55
3.1.1 Configuration and Security Aspects.....	56
3.1.2 Message Structure.....	56
3.1.3 ICU Examples.....	57
3.1.4 Event Recording for ICU Updates.....	57
Part 4 Reefer Monitoring XML API.....	60
4.1 Reefer Monitoring API.....	61
4.1.1 Requirements.....	62
4.1.2 Supported Reefer Details.....	62
4.1.3 Reefer Messaging Flow.....	62
4.2 Inbound Messages from Reefer Monitoring System to N4.....	63
4.2.1 Querying Reefers.....	64
4.2.1.1 Querying specific reefer(s).....	64
4.2.1.2 Query All.....	66
4.2.1.3 Query for Reefers IDs only.....	66
4.2.1.4 Query for Reefers but exclude non move history, chargeable unit event, and guarantee table records.....	66
4.2.2 Updating Reefers.....	66
4.2.2.1 Single Reefer Update Message.....	66
4.2.2.2 Multiple Reefer Update Message.....	68
4.2.2.3 Groovy plug-ins available for pre and post inbound reefer message processing.....	70
4.3 Outbound Messages from N4 to Reefer Monitoring System.....	73
4.4 Testing Reefer Monitoring Web Services.....	73
Part 5 N4 Web Services.....	77
5.1 N4 Web Services: Introduction.....	78
5.1.1 Standards Compliance.....	79

5.1.2	Development Platforms and Web Service Toolkits.	79
5.1.3	Available Services.	79
5.2	Security and Session Management.	81
5.2.1	Topology Information.	82
5.2.2	Security Setup.	82
5.3	Argo Generic and Basic Web Service API.	83
5.3.1	Argo Generic and Basic Web Service API Implementation.	84
5.3.2	Argo Generic Web Service API Format.	84
5.4	Error Handling.	85
5.4.1	General Status.	86
5.4.2	Status Messages.	86
5.5	Development Guide.	87
5.5.1	API Specifications.	88
5.5.2	Testing the Generic Services.	88
5.5.3	Invoking Web Services Programmatically.	90
5.5.4	Related Topics.	92
5.5.5	Triggering Groovy via Web Services.	92
5.6	Argo Basic Service Client Example for .NET (ArgoServiceTester).	93
5.6.1	ArgoServiceTester skeleton application.	94
5.6.2	Calling the Argo Basic Webservice.	99
5.6.3	Java Client Integration Example.	105
5.6.3.1	How to Locate the Generic ArgoWebservice in java code.	106
5.6.3.2	How Client Invokes the Generic Service.	106
5.6.3.3	Sample Source Code of a Standalone (Java) Webservice Client.	107
5.6.3.4	Steps to run Java standalone Webservice Client.	109
5.7	Client Web Services Integration for .NET.	111
5.7.1	N4 Setup and Environment.	112
5.7.2	Setup and Environment for .NET.	112
5.7.2.1	Start and use Microsoft Visual Studio .NET to create your application.	113
5.7.2.2	Add Web Reference to N4 Web service in .NET Application.	113
5.7.2.3	Sample .NET C# Client Accessing ICU on N4.	115
5.7.2.4	Security - Authentication.	120
5.7.2.5	Code Snippets/Attachments.	121
5.7.2.6	Example .Net Gate Integration.	121
5.8	Expose User SNX schema to Web service.	122
5.8.1	SNX Import.	123
5.8.2	SNX Export.	124
5.8.3	Error cases.	126
5.8.4	Using MassUnitType.	127

5.9 N4 Compatible External Web Service.	128
5.9.1 Create a N4 external webservice in MS .NET (2008).	129
5.9.1.1 Create an N4 external web-service in C# with Visual Web Developer Express 2008.	129
5.9.2 Create a N4 external webservice in MS .NET (2010).	131
5.9.2.1 Create an N4 external web-service in C# with Visual Web Developer Express 2010.	131
5.10 Report Generation Web Service.	139
5.10.1 What does it do.	140
5.10.2 Supported Queries.	140
5.10.3 XML Schema.	142
5.10.4 How to test it.	143
5.10.4.1 Sample Java Client.	143
5.11 Invoking Outbound WebService from N4 which is compliant with N4 basic service.	146
5.11.1 Introduction.	147
5.11.2 Invocation.	147
5.11.2.1 Invoking the web service call.	148
Part 6 N4 REST API.	150
6.1 Overview of the N4 REST API Service.	151
6.2 Set up API access to N4.	152
6.3 Creating a REST API with Groovy.	152
6.3.1 Enable the Groovy message log.	153
6.3.2 Create the Groovy code.	154
6.3.2.1 JSON Processing.	154
6.3.2.2 Example Groovy.	155
6.3.2.3 Error handling.	157
6.3.3 Deploying the Groovy REST API.	158
6.3.4 Invoking the Groovy REST API	158
6.4 Security for the REST API.	160
6.4.1 Refreshing an access token after it expires.	160
6.4.2 REST Authentication API.	161
6.4.3 Use Cases for the REST Authentication API.	162
6.4.4 Check Privileges API.	165
6.4.5 Invoking N4 RESTful API.	166
6.5 CHE Current Position API	167
6.6 N4 Passthrough for EVP	169
6.7 Billing REST Endpoints.	170
6.8 Service Order API.	173

Part 7 Using SNX	177
7.1 What is SNX?	178
7.1.1 Introduction	179
7.1.2 Structure of an SNX document	179
7.1.3 Timestamps	180
7.1.4 Supported Entities	181
7.1.4.1 Reference Entities	181
7.1.4.2 Working Entities	182
7.2 Special Considerations: <unit> element	183
7.2.1 Data Import Mode vs. Production Mode	184
7.2.2 Data Import Mode	184
7.2.3 Production Mode	184
7.2.4 Some notes	185
7.2.5 Examples	185
7.2.5.1 Marrying the Chassis to a Container	187
7.3 Special Considerations: other Elements	189
7.3.1 vessel-visit, train-visit	190
7.3.2 user	190
7.3.3 trucking-company	190
7.3.4 booking-information	191
7.3.5 create-container-notice	192
7.3.6 delete-container-notice	193
7.3.7 flags	193
7.3.8 snx-export	194
7.4 Unit updates and transactions on Units with SNX	196
7.4.1 The <unit-routing> message	197
7.4.2 The <unit-hazards> message	198
7.4.3 The <unit-property-update> message	198
7.4.4 The <equipment> message	199
7.4.5 The <equipment-damages> message	199
7.4.6 The <unit-renumber> message	199
7.4.7 The <unit-return-to-storage> message	199
7.4.8 The <unit-assign-to-booking> message	200
7.4.9 The <unit-return-to-storage> message	200
7.4.10 The <unit-cancel-preadvise> message	200
7.5 Signaling Activities with SNX	202
7.5.1 Simple Unit movements within the facility	203
7.5.1.1 The 'is-correction-only' attribute	203
7.5.2 Arriving and Departing Units	203
7.5.2.1 Keep things Tidy	204

7.6	Initial data conversion via SNX.	205
7.6.1	Container vs. Embedded Equipment Element.	206
7.6.2	Import Process.	206
7.6.3	Deletion.	207
7.6.4	Transformation of Customer's XML Data Into SNX.	207
7.6.5	XML Troubleshooting Hints for First-time SNX/XML Customers.	208
7.7	Disabling XPSCache Cluster Loading for Data Migration.	209
7.8	Re-organization of the SNX unit element.	210
7.9	Event Processing Via JMS for N4.	215
7.9.1	JMS Configuration for Inbound and Outbound Queues.	216
7.9.1.1	Add JMS queues to the facilities.	216
7.9.1.2	Configure provider specific parameters.	217
7.9.1.3	Restart the JMS connections.	218
7.9.2	Configuring Notice Requests for the Outbound Queue.	219
7.9.3	Debugging and Error Handling.	220
7.9.4	Configure N4 Tomcat node to Weblogic JMS server.	221
7.10	ActiveMQ Configuration.	222
7.10.1	Setup ActiveMQ.	223
7.10.2	Monitor JMS Queues.	223
7.10.3	Configure JMS Queues.	224
7.10.4	Troubleshooting ActiveMQ.	225
7.10.5	ActiveMQ integration with .Net.	226
7.10.6	ActiveMQ Failover Configuration.	227
7.10.6.1	ActiveMQ Failover in a NutShell.	228
7.10.6.2	Failover Setup.	228
7.10.6.3	Preparing ActiveMQ for the same machine setup.	228
7.10.6.4	N4 JMS Restarting Connection.	228
7.10.6.5	Prepare ActiveMQ on multiple machines Setup.	229
7.10.7	JMS Failover Scenarios.	229
Part 8	Gate Workflow Customization.	232
8.1	Gate Workflow Customization Options.	233
8.1.1	Summary.	234
8.1.2	Overview of Gate Workflow Execution.	234
8.1.3	Summary of Customization Options.	234
8.1.4	Task Filters.	234
8.1.5	Tasks that Create Custom Error Conditions.	236
8.1.6	Task to Override the Next Stage Determination.	237
8.1.7	Groovy Code in the Customize Window.	237
8.1.8	Calling Groovy Plug-ins from the Customize Window.	238
8.1.9	Code Extensions Executed in the Workflow.	238

8.1.10	Code Extensions to Override Gate Tasks.	239
Part 9	Gate API Specification.	243
9.1	Gate API Specification Overview.	244
9.1.1	Levels of Gate Automation.	244
9.1.1.1	Total Automation.	244
9.1.1.2	Semi-automation.	244
9.1.1.3	Data Entry Assist.	244
9.1.2	General Process Flow.	245
9.1.2.1	Automated Gates.	245
9.1.2.2	Self-help Gates.	245
9.1.3	General API Structure.	246
9.1.3.1	Common Webservice API.	246
9.1.3.2	Accessing the Common Webservice API.	246
9.1.3.3	Gate API Message Naming Conventions.	247
9.1.3.4	XML Payload Formats.	247
9.1.3.5	XML Request Format.	247
9.1.3.6	XML Response Payload Format.	248
9.1.3.7	XML Response Format Configuration.	248
9.1.3.8	Response Format Configuration Settings.	248
	Process API's Provided by N4.	250
9.1.1	read-preadvise.	251
9.1.2	record-scan (truck only).	251
9.1.3	record-scan (truck + equipment).	251
9.1.4	request-visit-details.	252
9.1.5	list-transactions.	252
9.1.6	update-appointment.	252
9.1.7	request-trouble-transactions.	252
9.1.8	request-stage-details.	252
9.1.9	request-stage-details (visit/tran key).	252
9.1.10	update-truck-visit (scale-weight-kg).	252
9.1.11	create-truck-visit-appointment.	252
9.1.12	create-truck-visit-appointment(update truck position and door direction).	252
9.1.13	update-truck-visit-appointment.	252
9.1.14	cancel-truck-visit-appointment.	253
9.1.15	process-truck.	253
9.1.16	create-truck-visit.	259
9.1.17	notify-arrival.	259
9.1.18	request-pickup-appointments.	259
9.1.19	submit-transaction.	259
9.1.20	submit-multiple-transactions.	260
9.1.21	stage-done.	260

9.1.22	cancel-transaction.	260
9.1.23	cancel-truck-visit.	260
9.1.24	need-assistance.	261
9.1.25	create-appointment.	261
9.1.26	create-appointment.	261
9.1.27	update-appointment.	262
9.1.28	cancel-appointment.	262
Process API's Provided by Other Systems.		264
9.1.1	notify-resume.	265
9.1.2	open-gate-arm.	265
9.1.3	print-ticket.	265
9.1.4	truck-arrival.	265
9.1.5	truck-departure.	266
Driver Interaction.		267
9.1.1	Pickup Selection.	268
9.1.2	Dropoff Details.	268
Process Truck API Workflow.		269
9.1.1	Basic Scan Validation.	270
9.1.2	Process Pre-lodged Dropoffs.	270
9.1.3	reprint-most-recent-ticket.	270
9.1.4	Fail Unexpected Container.	271
9.1.5	Solicit Further Details.	271
9.1.6	Process Pre-lodged Pickups.	271
9.1.7	Solicit Appointment Selection.	271
9.1.8	Solicit Pickup Selection.	271
9.1.9	Trouble Processing.	271
Lane Monitor View.		273
Support for Custom API messages.		274
Including schema extension fields.		275
Part 10 Ship Bin Model.		276
10.1 Ship Bin Model Request.		277
10.1.1	Introduction.	278
10.1.2	Setup.	278
10.1.3	Example Request.	278
10.2 Import-Export Ship Bin Model.		286
10.2.1	Import Ship File.	287
10.2.2	View Bin Plan.	289
10.2.3	Export SNX File – Ship Bin Model.	289

Part 11 ASC Automated Equipment Database Interface (ECN4)	291
11.1 Introduction	292
11.1.1 Feature Summary	293
11.1.2 Assumptions	293
11.2 System Interactions	294
11.2.1 Transport Order Data Flow	295
11.2.1.1 Primary Interaction	295
11.2.1.2 Abort Interactions	299
11.2.1.3 Cancellations	304
11.2.1.4 Offline Operation of ASC and Inventory Synchronization	304
11.2.1.5 Road Truck Chassis Positions	305
11.2.1.6 Door Direction Changes	306
11.2.1.7 ECN4 Start Up and Recovery Process	306
11.2.1.8 ASC Status	306
11.2.1.9 ECN4 Transport Order Table (ASC_TRANSPORT_ORDERS)	307
11.2.1.10 ACCS Command Queue Table (ASC_COMMAND_QUEUE)	310
11.2.1.11 ACCS ASC Status Table Layout (STATUS)	313
11.2.2 Transfer Point Status Update	315
11.2.2.1 ECN4 Transfer Point Status Table (TRANSFER_POINT_STATUS)	315
11.2.2.2 Transfer Point Status Use Cases	321
11.2.3 Area Status Update	332
11.2.3.1 ACCS Area Status Table (AREA_STATUS)	332
11.2.3.2 User Interface Considerations	333
11.3 Non-Functional Specification	334
11.3.1 Technical Requirements	335
11.3.2 Performance and Scalability	335
Part 12 Automated Yard Crane Database Interface	336
12.1 Introduction	337
12.2 Data Modeling and Design	338
12.2.1 TOS Crane Transport Order Table (YC_TRANSPORT_ORDERS)	339
12.2.1.1 Container Door Direction	346
12.2.1.2 Bay Types	346
12.2.1.3 Road Truck Chassis Positions	346
12.2.2 Crane ECS Command Queue Table (YC_COMMAND_QUEUE)	348
12.2.2.1 Functional Columns	348
12.2.2.2 Informational Columns	352
12.2.3 Crane ECS Status Table Layout (STATUS)	355
12.2.4 Crane ECS Area Status Table (AREA_STATUS)	358
12.2.5 N4 and ECS Application Status (N4_APPLICATION_STATUS, ECS_APPLICATION_STATUS)	360
12.2.6 Truck Status (TRUCK_STATUS)	361

12.3 Default Workflows.	363
12.3.1 Transport Order primary interaction.	364
12.3.2 Abort Interactions.	367
12.3.2.1 Abort on Set Down (ABORT_01).	368
12.3.2.2 Abort on Pickup (ABORT_02).	372
12.3.2.3 Abort on Door direction (ABORT_03).	374
12.3.2.4 Abort due to Invalid command (ABORT_04).	374
12.3.2.5 Abort on Truck Mismatch (ABORT_05).	376
12.3.2.6 Abort on No Truck (ABORT_06).	381
12.3.3 Dispatched Moves Completed by Remote Operator.	385
12.3.4 Cancelations.	386
12.3.5 Offline Operation of Crane and Inventory Synchronization.	387
12.3.6 Crane Move Order.	389
12.3.6.1 TOS Initiated Crane Movement Order.	389
12.3.6.2 ECS Initiated Crane Movement Order.	390
12.4 Purge Yard Crane database tables.	392

Part 13 Automation Database Interface. 394

13.1 Overview.	395
13.1.1 Responsibilities.	397
13.1.1.1 Active Orders and Commands.	397
13.1.1.2 Providing time stamps.	397
13.1.1.3 Weight measurements.	397
13.1.1.4 Key Generation.	398
13.1.2 Notification Mechanism.	398
13.1.3 Synchronization Points.	399
13.1.4 Startup and Recovery Process.	399
13.2 Equipment Interaction.	401
13.2.1 Equipment Status.	402
13.2.1.1 Equipment Availability.	402
13.2.1.2 Equipment Failure.	402
13.2.1.3 Rack Suitability.	403
13.2.2 AGV Interaction.	403
13.2.2.1 Abstract AGV Interaction.	404
13.2.2.2 AGV Container Movement Interaction.	406
13.2.2.3 AGV Drive Orders.	408
13.2.3 ASC Interaction.	409
13.2.4 QC Interaction.	412
13.2.4.1 QC Container Movement Interaction.	413
13.2.4.2 Vessel Load Swapping.	417
13.2.4.3 Vessel Work Information.	418

13.2.5	Exceptions	419
13.2.5.1	AGV Exceptions	419
13.2.5.2	ASC Exceptions	424
13.2.5.3	Corrective Actions	427
13.3	Location Status	432
13.3.1	Area Status	433
13.3.2	Transfer Point Status	433
13.4	Database Interface Definition	435
13.4.1	Datatypes	436
13.4.2	N4 Orders	436
13.4.2.1	AGV_ORDERS	436
13.4.2.2	ASC_ORDERS	441
13.4.2.3	QC_ORDERS	444
13.4.2.4	TEST_ORDERS	446
13.4.3	N4 Status	447
13.4.3.1	AGV_CONTAINER_UPDATE	447
13.4.3.2	WORK_INSTRUCTION_STATUS	448
13.4.3.3	QC_WORK_QUEUE_STATUS	453
13.4.3.4	BERTH_STATUS	454
13.4.4	ECS Commands	455
13.4.4.1	AGV_COMMANDS	455
13.4.4.2	ASC_COMMANDS	458
13.4.4.3	QC_COMMANDS	463
13.4.4.4	TEST_COMMANDS	465
13.4.5	ESC Status	465
13.4.5.1	AGV_STATUS	465
13.4.5.2	ASC_STATUS	469
13.4.5.3	QC_STATUS	471
13.4.5.4	DRIVE_TIME_STATUS	471
13.4.5.5	WORK_QUEUE_BUFFER_STATUS	472
13.4.5.6	AREA_STATUS	474
13.4.5.7	TRANSFER_POINT_STATUS	475
13.4.6	Notification and Startup	476
13.4.6.1	TRIGGER_EVENT_STATUS	476
13.4.6.2	TRIGGER_STATUS_HANDLED_SPARCS (N4), TRIGGER_STATUS_HANDLED_ECS (ECS)	477
13.4.6.3	SPARCS_APPLICATION_STATUS (N4), ECS_APPLICATION_STATUS (ECS)	477
13.4.7	Datatypes	478
13.4.7.1	Locations	478
13.4.7.2	Example diagram for ASC block, waterside transfer point and waterside locations	481
13.4.7.3	Work Status	482
13.4.7.4	Technical status	482

13.4.7.5 Orientation.....	482
13.4.7.6 ASC origin or destination vehicle type.....	483
13.4.7.7 Position on truck chassis.....	483

Part 14 Crane Automation API Technical Specification..... 485

14.1 Crane Automation API.....	486
14.2 Crane Automation System (CAS).....	486
14.3 Communication protocol.....	486
14.4 Web service hosted by N4.....	487
14.5 Web service hosted by CAS.....	488
14.6 Common Functionality.....	490
14.7 Sample messages.....	490
14.8 Sample client implementation (Java).....	490
14.9 WSDL Definitions.....	491
Interface Specific Details.....	492
14.9.1 Carrier Geometry Request.....	493
14.9.1.1 Carrier Geometry Request.....	493
14.9.1.2 Sample Request Inputs.....	493
14.9.1.3 Sample Response Message for Rail Request (from N4).....	494
14.9.1.4 Ship Bin Model Structure.....	495
14.9.1.5 Message Description.....	497
14.9.1.6 20' Bays Paired for 40' Containers: No Container Lengths Allowed.....	503
14.9.2 Carrier Visit Update Message.....	504
14.9.2.1 Carrier Visit Update Message.....	504
14.9.2.2 Sample Message (from N4).....	504
14.9.2.3 Carrier Visit Update message prerequisite.....	504
14.9.3 Container List Request.....	505
14.9.3.1 Container List Request.....	505
14.9.3.2 Sample ContainerListRequest Message (to N4) for vessel.....	507
14.9.3.3 Sample ContainerListRequest Message (to N4) for rail.....	508
14.9.3.4 Sample ContainerListRequest Response (from N4) For Vessel.....	508
14.9.3.5 Sample ContainerListRequest Response (from N4) For Rail.....	511
14.9.4 Crane Work List Update Message.....	512
14.9.4.1 Crane Work List Update Message.....	512
14.9.4.2 Sample Message(from N4).....	512
14.9.4.3 Prerequisites for this message.....	513
14.9.4.4 Create Custom Event.....	514
14.9.4.5 Create a General Notice.....	514
14.9.4.6 For Rail Operations.....	514
14.9.4.7 Configuration to enable Crane Work List Update (CWLK) message.....	514

14.9.5	Load Swap Request Message	514
14.9.5.1	Load Swap Request	514
14.9.5.2	Sample Requests	515
14.9.6	Onboard Unit Update Message	516
14.9.6.1	Onboard Unit Update Message	516
14.9.6.2	Sample Message(from N4)	517
14.9.6.3	Prerequisites for this Message	518
14.9.6.4	Create Custom Event	518
14.9.6.5	Create a General Notice	518
14.9.7	QC Updates via CAS	518
14.9.7.1	Update QC technical status and location via CAS	518
14.9.7.2	Update QC position via CAS	519
14.9.8	Rail Messages in CAS API	520
14.9.8.1	IBCMA Rail Cone Status Update in CAS Interface	520
14.9.8.2	IBCMA Rail Cone Status Update in CAS By Platform	521
14.9.8.3	Rail Spotting interface in CAS API	521
14.9.9	Ready To Transfer Message	524
14.9.9.1	Sample Message (from N4)	524
14.9.9.2	Sample Message Sent In Response Unit Capture Identify	525
14.9.10	Unit Capture Messages	525
14.9.10.1	Rail Inspection - Rail Load	525
14.9.10.2	Image Messages	528
14.9.10.3	Identify Messages	529
14.9.10.4	Create Messages	530
14.9.10.5	Update Messages	531
14.9.10.6	Cannot Identify Messages	531
14.9.11	Unit Position Update Message	532
14.9.11.1	Unit Position Update Message	532
14.9.11.2	Sample Unit Position Update Message Request (from CAS to N4)	534
14.9.11.3	Sample Unit Position Update Message Response (From N4 to CAS)	535
14.9.12	Miscellaneous Notes	535
14.9.12.1	Position Notes	535
14.9.12.2	Error/Warning/Info message codes and text returned by N4	536
Index		539

Part 1

Universal Query API

This document is a developers' guide intended for Navis clients using the N4 Universal Query API.

In this Section:

Introduction	17
Security	19
Using the Universal Query	21
Limiting Access to the Query Url	30
Configure the Universal Query for the First Time	31
Boolean Parameters in Universal Query	36
Static Code Lookup	41

1.1 Introduction

The Universal Query API is a lightweight mechanism for specifying queries through the N4 UI and fetching the result set externally in XML format via a URL.

- Allows any N4 UI list view to be fetched as an XML document
- Intended for customers writing their own view applications
- Uses a simple REST protocol
- Strictly Read-Only

Different supported query modes are:

- **DEFAULT:** Returns values in view xml conforming to the Universal Query API
- **RECAP:** Returns an n-dimensional table conforming to the recap-table element in Universal Query API
- **SNX:** for SNX export enabled entities, these will be returned as SNX export records conforming to the snx.xsd

In This Section

[REST Protocol.](#) 18

1.1.1 REST Protocol

Representational State Transfer (REST) protocol is a simpler style of web service delivered via the http request with the request parameters embodied in the URL. REST is often preferred over SOAP-based web services for ease of integration.

1.2 Security

The Universal Query API is intended to be used internally inside the firewall between in-house systems and the security configuration recommended here reflects this need. By default, the Universal Query is turned off and requires the application setting `UNIVERSAL_QUERY_REST_ENABLE` to be enabled before access. When you enable the Universal Query, you must also configure the following security at the web server and the application server levels:

- Web server configuration to limit access to the /api service url.
- Basic Authentication Configuration
- Optionally configure HTTP over SSL

 A special purpose user account is recommended for Universal Query access. This account should be configured with view-only permissions. If you have multiple integrations, it is recommended you use different users.

In This Section

Basic Authentication.	20
HTTPS.	20

1.2.1 Basic Authentication

The N4 deployed war is configured to protect the URL for Universal Query API via http Basic Authentication. Within the application server(i.e. tomcat), you will have to configure a role named 'N4api' for the realm and assign a user with the role.

The application delivers with a default N4api user and you must log in the global scope to add roles to this user. However you can use whatever user you define as long as each user used for the Universal Query service is defined in N4 application with a matching password as specified in tomcat. Authentication happens twice, once for the application server and a second time inside the application.

If you have different integrations using the Universal Query service, you are recommended to create a separate user for each application. In this way you can manage visibility and security independently for each user.

For tomcat, tomcat-users.xml configuration may look like this:

```
<tomcat-users>
  <role rolename="tomcat"/>
  <role rolename="N4api"/>
  <user username="tomcat" password="xxxxxxx" roles="tomcat"/>
  <user username="N4api" password="xxxxxxx" roles="N4api"/>
</tomcat-users>
```

If you are running in clustered mode, tomcat-users.xml must be modified for each of the instance nodes. If you received a distribution of tomcat from Navis, this configuration may already be defined. You will want to change the passwords appropriately.

For Windows, the node appropriate tomcat-users.xml will be under the following directories:

- C:\Navis\tomcat\instances\node1\conf\tomcat-users.xml
- C:\Navis\tomcat\instances\node2\conf\tomcat-users.xml

Typing in a universal query url in a browser will pop open a authentication dialog box.

1.2.2 HTTPS

Securing the channel via HTTP over SSL is recommended if your company security policies require protecting against attack inside the firewall or if the REST access will be over a WAN. This step might not be necessary if you will access the Universal Query API inside the firewall and transmission inside the firewall is considered secure.

For apache configuration, please refer to apache documentation on forcing the clients to authenticate using a certificate for a url subset.

1.3 Using the Universal Query

The Universal Query is delivered disabled by default and requires [initial security configuration \(on page 31\)](#) at the server and application level before use.

After security has been properly configured, use of the Universal Query is achieved through these steps:

- 1. Configure and save a filter inside the desired N4 Table View

 As per the current N4 architecture, it is not possible to use the filter option for universal queries within child forms (for example, the Booking Items or the Order Items tables).

- 2. Construct the URL
- 3. Call the URL (If the port 8280 is not working, change the port to 9080 and try the URL).

In This Section

Configuring the Query.	22
Configuring the Default Query.	22
Configuring an SNX Query.	23
Configuring a Recap Query.	24
Constructing the URL.	26
Parameters.	27
Executing the Query.	28
Troubleshooting.	28

1.3.1 Configuring the Query

1. Define a filter with a unique private filter name. A private filter name is one in which no role has been defined and is visible only to that user.



As per the current N4 architecture, it is not possible to use the filter option for universal queries within child forms (for example, the Booking Items or the Order Items tables).

2. Specify columns and ordering needed
3. Specify predicates with or without parameters
4. Any change in the underlying query will automatically update API result set.
5. Don't forget to save the query.

1.3.2 Configuring the Default Query

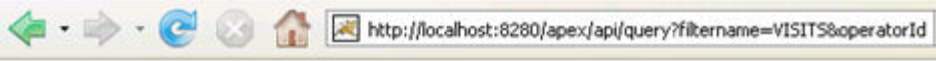
The default mode is intended for applications which will display the query results.

The screenshot shows a software interface titled 'VISITS'. It has a toolbar with buttons for 'Actions', 'Display', and a dropdown menu currently set to 'VISITS'. Below the toolbar is a table with the following data:

Visit	Vessel Name	Line	Service	ETA	ETD
AOA044A	AOTEA	MSK	JAPA	06-May-29 0200	06-May-29 1800
V1234567...	AOTEA	MSK	MEDS		
MMT001A	MSC EMPTIA	MSC	MSCE	07-Jan-01 0225	07-Jan-01 1825
MET005A	MSC EDITH	MSC	MSCS	06-May-20 0200	06-May-20 1800
MMI007A	MSC MARIA	MSC	MSCS	06-May-29 0200	06-May-29 1800
TNY007A	TENG YUN HE	COS	NASI	06-May-22 0200	06-May-22 1800

At the bottom right of the table area, there is a timer showing '0:16'.

The above saved filter will yield the following result:



```

- <query-response>
- <data-table filter="VISITS" count="16">
- <columns>
  <column>Visit</column>
  <column>Vessel Name</column>
  <column>Line</column>
  <column>Service</column>
  <column>ETA</column>
  <column>ETD</column>
</columns>
- <rows>
- <row primary-key="870">
  <field>JAIL999</field>
  <field>JAIL999</field>
  <field>TBA</field>
  <field>TBA</field>
  <field>2008-01-03 10:35</field>
  <field>2008-01-04 02:35</field>
  <field>870</field>
</row>
- <row primary-key="873">
  <field>MMT001A</field>
  <field>MSC EMPTIES</field>
  <field>MSC</field>
  <field>MSCE</field>
  <field>2006-12-31 05:25</field>
  <field>2006-12-31 21:25</field>
  <field>873</field>
</row>

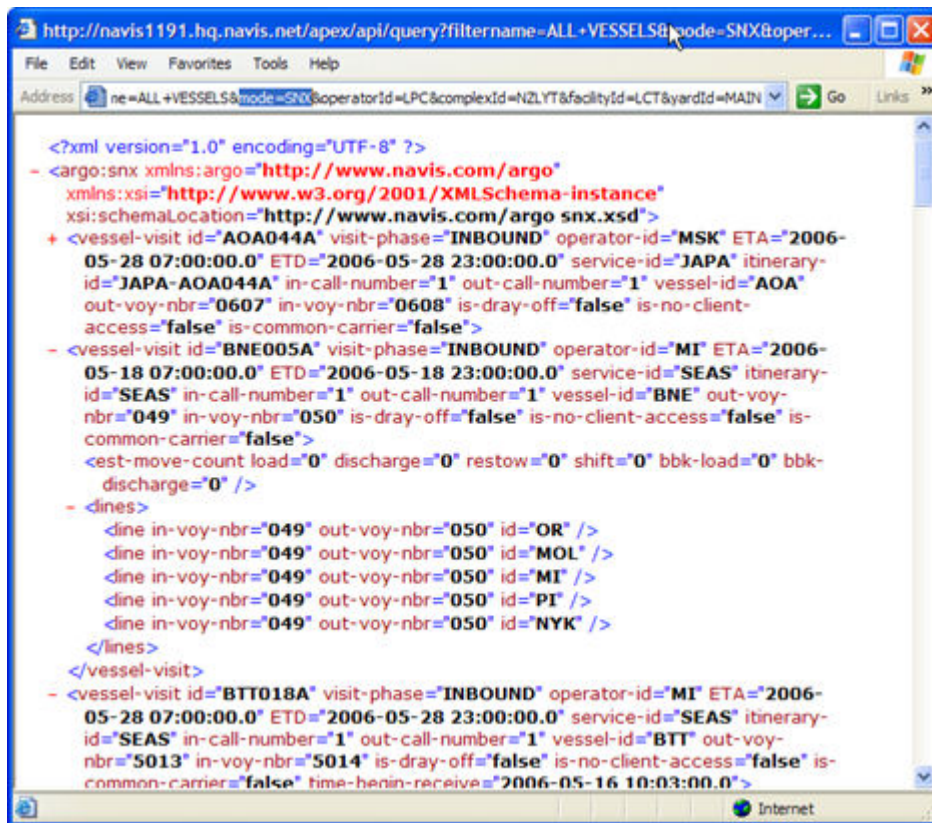
```

1.3.3 Configuring an SNX Query

A query can be only fetched by SNX mode if the underlying entity supports SNX export. You can check to see if your table view supports SNX export by looking for the "Export SNX File" action in the table action menu.

You configure a filter for SNX mode in the same way you configure a default query, except the columns selection is ignored as the query return will contain the entire SNX entity as seen below.

The Universal Query response in SNX mode will contain the exact same xml content that is returned when manually exporting an SNX file via the "Export SNX File" action.



For the Units table view, the SNX returned will include both the equipment and unit elements.

1.3.4 Configuring a Recap Query

The columns specified for a recap query will be the columns used for the group by function to obtain the counts. Query results will be returned in a recap-table filter element. A <bucket> element is the finest grained count. An array represents a subset of counts for a specific value and can contain buckets or more arrays. There is no limit to the number of columns selected, however, most common use cases will involve 1-3 columns. Totals are provided at every level. In the example below, we have a saved filter MTS IN YARD which shows the empties in the yard.

MTS IN YARD		
Equipment Operator	Category	Type Length
MSK	Export	20'
NYK	Import	40'
NYK	Import	40'
NYK	Import	20'
NYK	Import	20'
NYK	Import	20'
NYK	Import	20'

The URL for recap query is given below.

```
http://yourserver:9080/apex/api/query?
filtername=PNCVESSEL&mode=RECAPoperatorId=LPC&complexId=NZLYT&facilityId=LCT&yardId=MAIN
```

1.3.4.1 Query results for three columns

Selecting three columns above will result in a 3 dimensional recap-table below.

```
<query-response>
<recap-table filter="MTS IN YARD" total="142">
  <columns>
    <column>Equipment Operator</column>
    <column>Category</column>
    <column>Type Length</column>
  </columns>
  <array label="MOL" total="48">
    <array label="Import" total="48">
      <bucket label="20'" count="46" />
      <bucket label="40'" count="2" />
    </array>
  </array>
  <array label="MSK" total="15">
    <array label="Export" total="10">
      <bucket label="20'" count="10" />
    </array>
    <array label="Import" total="5">
      <bucket label="20'" count="5" />
    </array>
  </array>
  <array label="NYK" total="79">
    <array label="Export" total="51">
      <bucket label="40'" count="51" />
    </array>
    <array label="Import" total="28">
      <bucket label="20'" count="26" />
      <bucket label="40'" count="2" />
    </array>
  </array>
</recap-table>
</query-response>
```

The xml is equivalent to this more easily readable table:

Equipment Operator (Type Length) Category	MOL	MSK	NYK
Export-Total		10	51
Export(20')		10	
Export(40')			51
Import(Total)	48	5	28
Import(20')	46	5	26
Import(40')	2		2
Total	48	15	79

1.3.4.2 Query results for two columns

```
<query-response>
```

```
<recap-table filter="MTS IN YARD" total="142">
  <columns>
    <column>Equipment Operator</column>
    <column>Category</column>
  </columns>
  <array label="MOL" total="48">
    <bucket label="Import" count="48" />
  </array>
  <array label="MSK" total="15">
    <bucket label="Export" count="10" />
    <bucket label="Import" count="5" />
  </array>
  <array label="NYK" total="79">
    <bucket label="Export" count="51" />
    <bucket label="Import" count="28" />
  </array>
</recap-table>
</query-response>
```

The xml is equivalent to this more easily readable table:

Equipment Operator / Category	MOL	MSK	NYK
Export		10	51
Import	48	5	28
Total	48	15	79

1.3.4.3 Query results for a single column

Please note this functionality is currently broken as the UI tier is sending multiple sort columns for unshown columns.

```
<query-response>
<recap-table filter="MTS IN YARD" total="142">
  <columns>
    <column>Category</column>
  </columns>
  <bucket label="Export" count="61" />
  <bucket label="Import" count="81" />
</recap-table>
</query-response>
```

1.3.5 Constructing the URL

Example URL looks like this:

```
http://yourserver/apex/api/query?
filtername=PNCVESSEL&PARM_VISIT_REF=AOA044A&operatorId=LPC&complexId=NZLYT&facilityId=LCT&yar
```

- `filtername` specifies exact name defined in UI, use + or %20 notation for space characters within the filter name
- Parameter names are sent prefixed with `PARM_`. I.e. if you have a parameter named `VISIT_REF`, the REST parameter should be sent in as `PARM_VISIT_REF`. See below for supported parameter types.
- Universal Query API#mode

- scope coordinates specifying the view scope of the query
 - `operatorId`
 - `complexId`
 - `facilityId`
 - `yardId`
- `variform-id` specifies which List/Table view to apply filters. In N4, a filter is related to an entity, and as an entity it can be related to multiple List/Table views. The Universal Query needs to know the filter name as well as the List/Table view it is associated with.

Usage example: Apply the filter 'BLOCK' on variform-id=TABLE_YARD_BLOCK:

```
http://localhost:8280/apex/api/query?
filtername=BLOCK&operatorId=OPR1&complexId=CPX11&facilityId=FCY111&yardId=YRD1111&variform-
id=TABLE_YARD_BLOCKS
```

1.3.6 Parameters

Currently supported parameter types include:

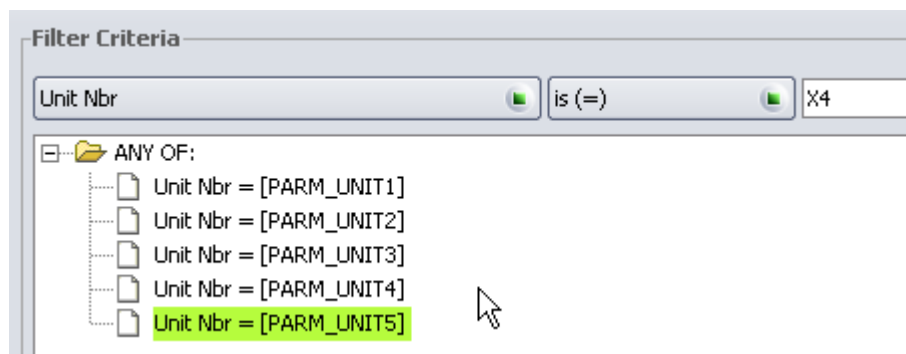
- String
- Number (Integer and Real)
- Dates - Depends on version.

The expected date format is what you have configured in your DATE_TIME_DISPLAY_FORMAT Setting and is the date format you see in the listviews.

- Reference data types as presented as pick lists. For example, truckers, shippers, line operators, and POD from the Unit filter screen are all supported.
- Static codes such as vessel visit phase, unit category, IMDG codes etc. Universal Query will expect the actual db value lookup which can be found from the static code lookup service.
- Relative dates in the filter criteria picked as list of values such as Today, Yesterday, < 10 days ago cannot be parameterized
- Booleans are parameterizable. See below.

Currently not supported are:

- Lists from the In predicate. In order to simulate the same functionality for a small handful of similar matches, the "OR" group can be used with optional parameters. Any parameters not sent in the URL, extra or clauses will not be added.



The Universal Query API can use comma separated values (CSV) for following parameter types:

- string
- integer (long)
- float (double)

1.3.7 Executing the Query

To test your query results, you can invoke the Universal Query by:

- Manual testing via browser by typing in the url
- Programmatic call - Most languages support the creation of http client
 - C#, HttpRequest and HttpResponseMessage classes. See Tutorial
 - java, it maybe be simplest to use the Jakarta Commons HttpClient class

In all cases, a valid user name and password in both the application server and the application must be supplied.

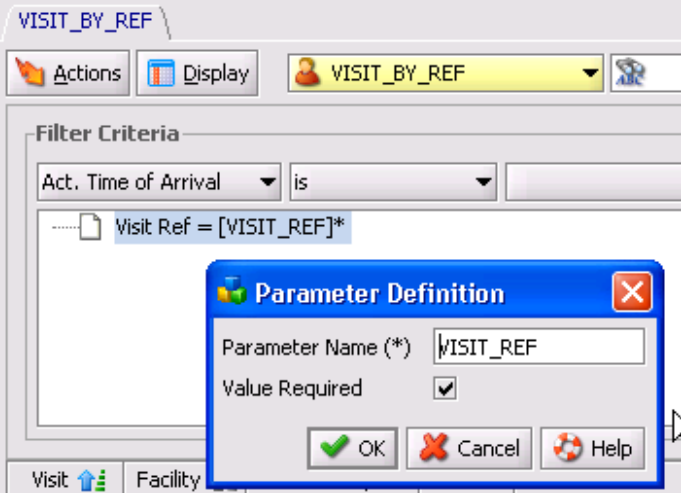
1.3.8 Troubleshooting

The error messages returned from the Universal Query service will be as verbose as possible. If a query request passes web server authentication, any errors from N4 will be sent as an error element:

```
<query-response>  
<error>User N4api does not have access to requested scope coordinates - LPC/  
NZLYT/-/-</error>  
</query-response>
```

Listed below are potential problems arising from the web server or servlet container even before your request is passed onto N4.

Problem/Message	Potential Causes
HTTP 401 Unauthorized Error Message	You must define a valid application server user and send the request with HTTP BASIC Authorization. The username and password could be wrong.

Problem/Message	Potential Causes
<pre><query-response> <error>Required query parameter for field Visit Ref with label VISIT_REF was not provided.</error> </query-response></pre>	If you have made a parameterized field required, you must send in a matching parameter as part of the URL.  By clearing the Value Required checkbox, you are making this criteria optional. If you do not send in any parameters, this criteria will not be added and all visits visible to the user scope will be returned.
<pre><query-response> <error>User N4api does not have access to requested scope coordinates - -/-/-/-.</error> </query-response></pre>	You must specify scope coordinates in the URL

1.4 Limiting Access to the Query Url

On your production instance, you will need to limit access to the Universal Query url by desired IP addresses. In case you configure your N4 system to be available outside the firewall to end customers, this will prevent them from using this API. This is a common security pattern for also limiting Web Service access to the outside world.

For apache configuration, you can use `mod_authz_host` with the `<Location>` directive. I.e.

```
<Location /apex/api/*>
  Order Deny,Allow
  Deny from all
  # where x.x.x represents the ip addresses you want to grant access to the universal
  query
  Allow from x.x.x
</Location>
```

Refer to apache documentation for more details.



If you are running a single tomcat instance for integration development, this step may be skipped.

1.5 Configure the Universal Query for the First Time

Using the Universal Query will need system admin to configure the security settings of your application server. Once the server configuration is complete, end user can configure the desired queries simply through the N4 application as a regular user. However- the system admin is encouraged to configure a simple query and validate against the browser to ensure Universal Query configuration has been correctly set.

In this Section:

In This Section

- [Configure Tomcat Security.](#) 32
- [Enable Universal Query.](#) 32
- [Create an N4 User.](#) 32
- [Giving roles to the n4api user.](#) 33
- [Create a Simple Filter.](#) 33
- [Test the API in your browser.](#) 33
- [Universal Query Java Integration Example.](#) 34

1.5.1 Configure Tomcat Security

The system administrator should add a tomcat user or confirm one has been shipped as outlined in the Basic Authentication Section.

1.5.2 Enable Universal Query

1. Log in as an admin user.
2. Go to Administration->Settings->Settings
3. Set UNIVERSAL_QUERY_REST_ENABLE setting to true at a global level

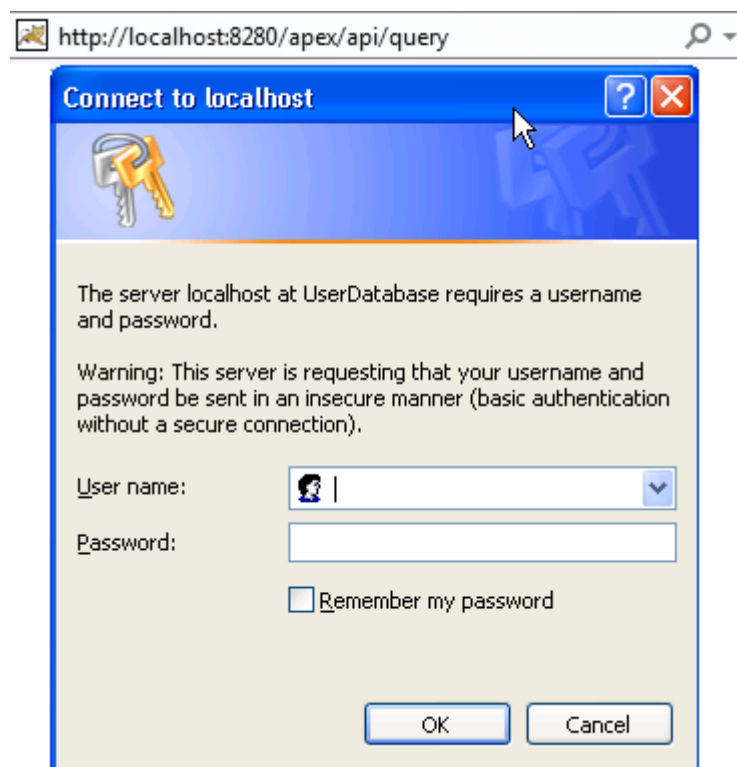
In addition, the TABLE_EXPORT_MAX_SIZE setting will limit the number of rows returned.

1.5.3 Create an N4 User

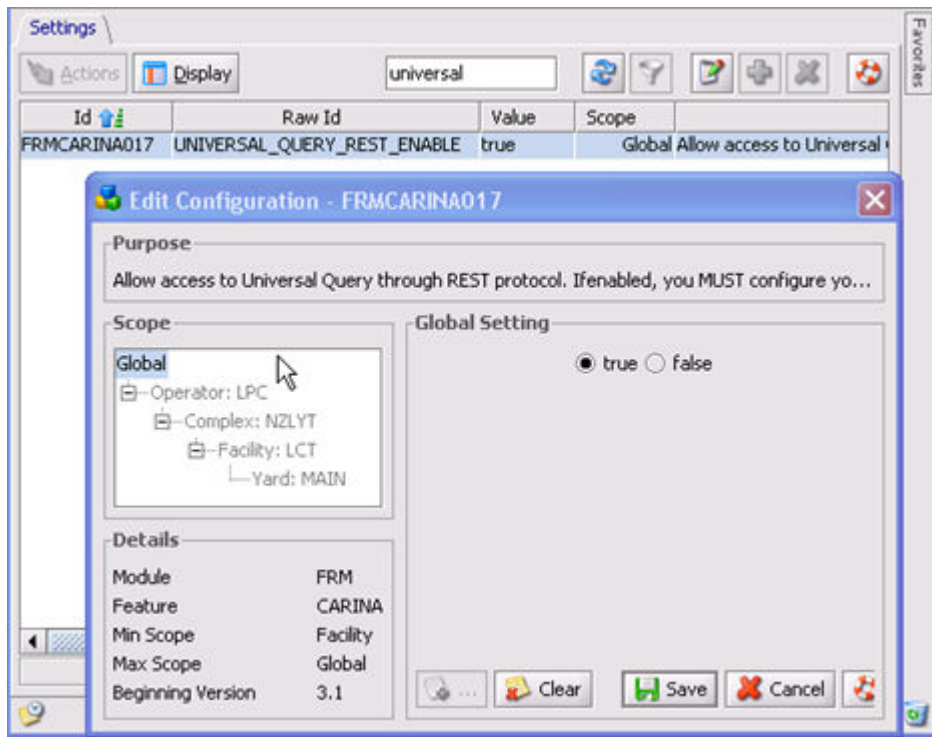
As an admin user, you will need to create a user which matches the name and password specified in tomcat. If you point your browser towards:

<http://localhost:8280/apex/api/query>

You will get a login screen from the browser:



Type in your username add password.



You should get the following error message:

```
<query-response>
  <error>User N4api does not have access to requested scope coordinates - -/-/-/-.</
error>
</query-response>
```

This means the tomcat user setting is correct.

Now you have to create an example query and sent in a request with the proper coordinates.

1.5.4 Giving roles to the n4api user

As shipped the n4api user is a special system user without any roles. You will need to log in to N4 as an admin user at the global level to be able to see the n4api user and give it some roles. This can be done via the users screen. If you login at a complex or facility scope you will not see the n4api user in the users list. If you do not do this when you log in as the n4api user you will not see very many menu options.

1.5.5 Create a Simple Filter

The universal query api can only see saved filters created by this n4api user. Filters created by other users will not be accessible. Login as the n4api user. Go to the Vessel Screen. Save the query as VESSEL_TEST.

1.5.6 Test the API in your browser

You want to type in something like:

```
http://localhost:8280/apex/api/query?
filtername=VESSEL_TEST&operatorId=LPC&complexId=NZLYT&facilityId=LCT
```

The query parameters should match your login parameters for the regular application.

The Universal Query is delivered disabled by default and requires initial security configuration at the server and application level before use.

1.5.7 Universal Query Java Integration Example

The below code shows a simple example to connect to the Universal query via an http client.

```
/*
 * Copyright (c) 2007 Navis LLC. All Rights Reserved.
 * $Id: $
 */
package com.Navis.sandbox.web;
import java.io.BufferedReader;
import java.io.DataInputStream;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLConnection;
/**
 * This class is a simple example to authenticate
 */
public class UniversalQueryExampleTest {
    private static void setBasicAuthentication(URLConnection connection,
        String username, String password) {
        // Build the string to be used for Basic Authentication <username>:<password>
        String userPassword = username + ":" + password;
        // Base64 encode the authentication string
        String encoding = new sun.misc.BASE64Encoder().encode(userPassword.getBytes());
        // Set Basic Authentication parameters
        connection.setRequestProperty("Authorization", "Basic " + encoding);
    }
    public static String getQueryXml(String username, String password, String
inFilterName) {
        StringBuffer sb = new StringBuffer();
        try {
            URL url = new URL("http://localhost:8280/apex/api/query?filtername=" +
inFilterName);
            HttpURLConnection con = (HttpURLConnection) url.openConnection();
            con.setRequestMethod("GET");
            setBasicAuthentication(con, username, password);
            con.connect();
            DataInputStream dis = new DataInputStream(con.getInputStream());
            String line = null;
            BufferedReader d = new BufferedReader(new InputStreamReader(dis));
            while ((line = d.readLine()) != null) {
                sb.append(line + "\n");
            }
            dis.close();
            con.disconnect();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return sb.toString();
    }
    public static void main(String[] inArgs) {
```

```
String xml = getQueryXml("N4api", "lookitup", "testfilter");  
System.out.println(xml);  
}  
}
```

1.6 Boolean Parameters in Universal Query

Starting 1.5.E, boolean fields set to the is true predicate is parameterizable. And since 2.0, the boolean predicate can be ignored completely if desired.

In This Section

- UI Filter Screen. 37
- Optional boolean predicates. 37
- Required boolean predicates. 38
- Either required and/or optional predicates. 39
- Universal Query. 40

1.6.1 UI Filter Screen

When you declare parameterized booleans as optional fields for a filter, when applying the filter, you will get a radio button set for specifying a true or false value in the Filter Parameter UI, and an Ignore button for ignoring the boolean predicate altogether. The widget will default to *Ignore* when displayed.

When you declare parameterized booleans as required fields for a filter, when applying the filter, you will get a checkbox for specifying a true or false value in the Filter Parameter UI. Even though the filter values are required, an unchecked box will by default mean a FALSE value has been filled out.

1.6.2 Optional boolean predicates

The screenshot shows the 'Units' tab of the 'Script Runner' application. The 'Filter Criteria' section shows a filter for 'Is OOG' with a radio button set set to 'YES (is true)'. Below this, a tree view shows 'ANY OF:' with two criteria: 'Hazardous? is true [HAZARDOUS]' and 'Is OOG is true [OGG]'. The 'Is OOG is true [OGG]' criterion is highlighted in green.

The data table below shows the results of the filter. The columns are: Unit Nbr, Type ISO, Is OOG, Hazardous?, Category, V-State, T-State, and Position. The table contains 20 rows of data.

Unit Nbr	Type ISO	Is OOG	Hazardous?	Category	V-State	T-State	Position
USNX0201249	F40M	•	•	Through	Departed	Departed	V-MKI078-14..
TTNU9315476	D40H		•	Import	Active	Inbound	V-MAT475-14..
TRIU0704450	F40		•	Import	Active	Yard	Y-SI-E1324
TOLU8798408	F40	•		Import	Active	Yard	Y-SI-D1714
TOLU8774345	F40	•					
TCNU9080933	D40H		•				
FSCU9649242	D40H		•				
FBLU4009020	D40		•				
CAXU5704330	F40	•					
AMZU4166035	F40	•					
AMFU4232292	F40		•				
MATU8824389	A40		•				
MATU8824120	A40		•	Import	Active	Yard	Y-SI-V118B2
MATU8823377	A40		•	Import	Active	Inbound	V-LUR609-04..
MATU8822853	A40		•	Import	Active	Yard	Y-SI-V222D2
MATU8822560	A40		•	Export	Departed	Departed	V-MAU689-08.
MATU8822302	A40		•	Import	Active	Yard	Y-SI-V122F3
MATU8820911	A40		•	Export	Departed	Departed	V-LHE218-031
MATU8802040	A40		•	Import	Active	Yard	Y-SI-V118F2

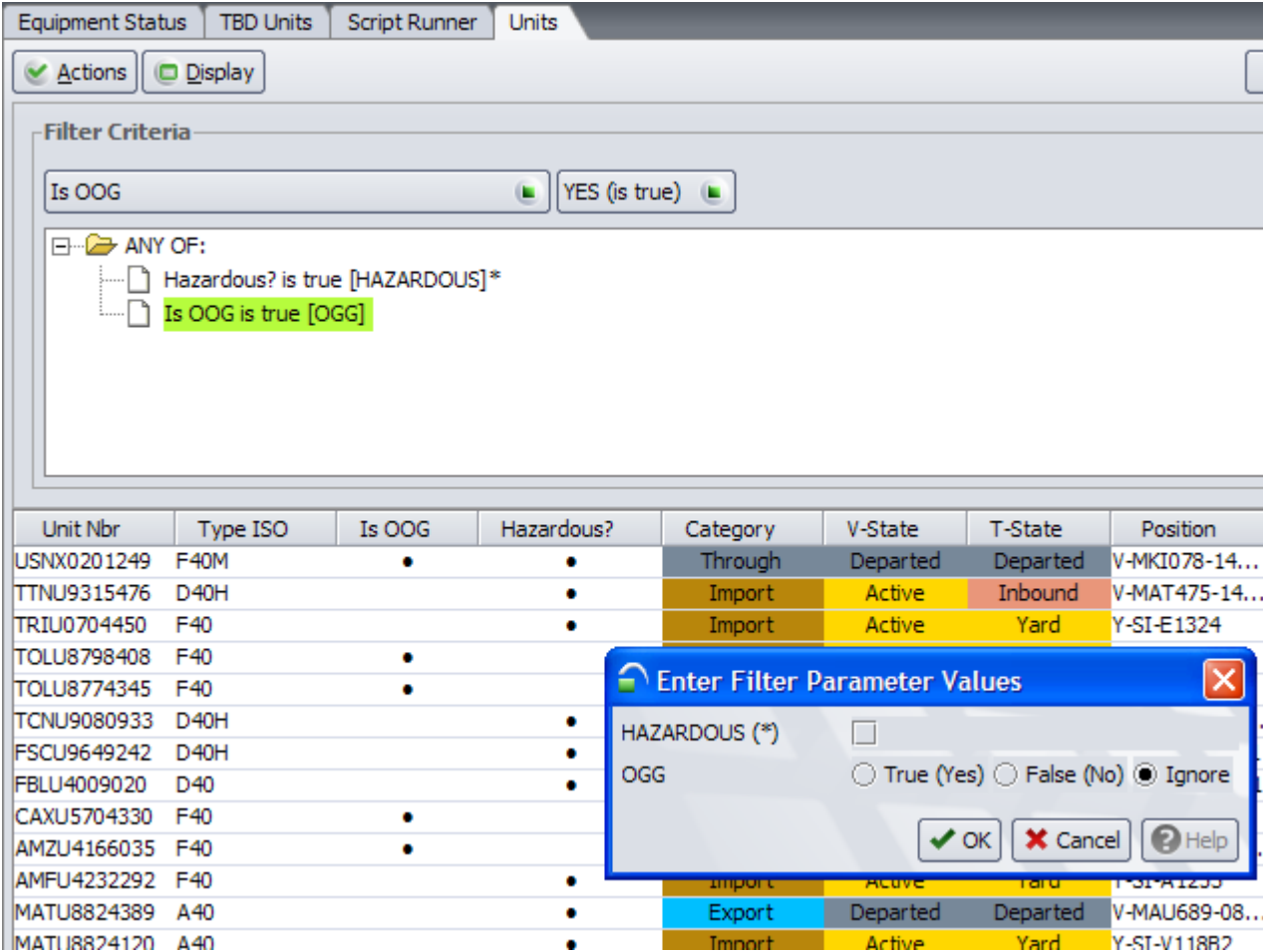
The 'Enter Filter Parameter Values' dialog box is open, showing the 'HAZARDOUS' and 'OGG' parameters. Both have radio buttons set to 'True (Yes)'. The 'OK' button is highlighted.

1.6.3 Required boolean predicates

The screenshot displays the Universal Query interface with the 'Units' tab selected. The 'Filter Criteria' section shows a filter for 'Is OOG' with the value 'YES (is true)'. Below this, a tree view shows 'ANY OF:' with two criteria: 'Hazardous? is true [HAZARDOUS]*' and 'Is OOG is true [OGG]*'. A table of units is visible below the filter criteria. A dialog box titled 'Enter Filter Para...' is open, showing checkboxes for 'HAZARDOUS (*)' and 'OGG (*)', both of which are checked. The dialog has 'OK', 'Cancel', and 'Help' buttons.

Unit Nbr	Type ISO	Is OOG	Hazardous?	Category	V-State
USNX0201249	F40M	•	•	Through	Departed
TTNU9315476	D40H		•	Import	Active
TRI0704450	F40		•	Import	Active
TOLU8798408	F40	•		Import	Active
TOLU8774345	F40	•			
TCNU9080933	D40H		•		
FSCU9649242	D40H		•		
FBLU4009020	D40		•		
CAXU5704330	F40	•			
AMZU4166035	F40	•			
AMFU4232292	F40		•		
MATU8824389	A40		•	Export	Departed
M&TI 18874120	A40		•	Import	Active

1.6.4 Either required and/or optional predicates



From the UI, if you specify both OOG and Hazardous as required parameters, you will be limited to the four choices specified in the table below.

Hazardous boolean	OOG boolean	Return Set
true	true	all hazardous and OOGs
true	false	hazardous plus non-OOGs
false	true	OOGs plus non-hazardous
false	false	neither hazardous nor OOG

If you want either OOG or Hazardous, you will specify the corresponding parameter(s) as optional.

Hazardous radio button (optional)	OOG radio button (optional)	Return Set
true	true	all hazardous and OOGs
true	false	hazardous plus non-OOGs

Hazardous radio button (optional)	OOG radio button (optional)	Return Set
true	ignore	all hazardous
false	true	OOGs plus non-hazardous
false	false	neither hazardous nor OOG
false	ignore	non-hazardous
ignore	true	all OOGs
ignore	false	non-OOGs

1.6.5 Universal Query

For the universal query, the third option of not specifying the parameter (ignore) will exclude the corresponding predicate as follows. For the example disjunction(or group) of isOOG OR is Hazardous, you will get the following 8 possibilities. N4 will accept the constants true and false for setting boolean parameter values.

Filter Parameter URL	Return Set
PARM_HAZARDOUS=true	Only hazardous containers
PARM_HAZARDOUS=false	All non-hazards, may contain OOGs or non-OOGs
PARM_OOG=true	Only OOGs
PARM_OOG=false	Non OOGs, may contain hazards and non-hazards
PARM_OOG=true&PARM_HAZARDOUS=true	OOG or hazardous , meaning union of all hazards and OOGs
PARM_OOG=true&PARM_HAZARDOUS=false	OOG or nonhazardous
PARM_OOG=false&PARM_HAZARDOUS=true	non OOG or hazardous
PARM_OOG=false&PARM_HAZARDOUS=false	neither hazardous nor OOG, returns everything

1.7 Static Code Lookup

In This Section

- How to look up for static code values. 42
- Unit Freight Kinds. 42
- Work Instruction Move Types. 42
- Unit Dray Statuses. 42
- Unit Categories. 43
- Unit Hazards and IMDG Classes. 43

1.7.1 How to look up for static code values

While doing custom development with N4, you may need to know the different values for static codes such as Vessel Visit Phase change or Freight Kind. For example with the Universal Query, you may need to know the codes to drive dynamic parameter values. You can find the static codes via custom code extension directly from the Metafield screen. This REST API may be used if you need to access the static codes programmatically in your integration.

Starting 1.5.E, N4 provides a new REST service for looking up static codes in N4. Like the IMDG class, static codes are codes that are NOT stored in the database but are fixed inside the java code in N4. The new codeLookup API will let you specify the reportable entity tag, and it will return xml of the description and the stored database code value. When you are writing integrations against the database, you will want to use the database code value for parameter matching instead of the description.

Parameters are required as follows:

- entity : name of the reportable entity, i.e. Unit, UnitHazard, VesselVisitDetails, WorkInstructions
- tag : valid reportable tag of the entity, i.e. FreightKind



You will still have to append the appropriate scope parameters to the URL request for which the login user has access. I.e. if the user has highest scope access of facility, you will add something like `operatorId=LPC&complexId=NZLYT&facilityId=LCT` to the end of the request.

1.7.2 Unit Freight Kinds

```
http://localhost:8280/apex/api/codeLookup?entity=Unit&tag=FreightKind
<codes entity="Unit" tag="FreightKind">
  <code dbvalue="MTY" desc="Empty"/>
  <code dbvalue="FCL" desc="FCL"/>
  <code dbvalue="LCL" desc="LCL"/>
  <code dbvalue="BBK" desc="B-bulk"/>
</codes>
```

1.7.3 Work Instruction Move Types

```
http://localhost:8280/apex/api/codeLookup?entity=WorkInstructions&tag=MoveType
<codes entity="WorkInstructions" tag="MoveType">
  <code dbvalue="RECV" desc="Receival" />
  <code dbvalue="DLVR" desc="Delivery" />
  <code dbvalue="DSCH" desc="Discharge" />
  <code dbvalue="LOAD" desc="Load" />
  <code dbvalue="RDSC" desc="Rail Disch" />
  <code dbvalue="RLOD" desc="Rail Load" />
  <code dbvalue="SHOB" desc="Shift O.B." />
  <code dbvalue="YARD" desc="Yard Move" />
  <code dbvalue="SHFT" desc="Yard Shift" />
  <code dbvalue="OTHR" desc="Other" />
</codes>
```

1.7.4 Unit Dray Statuses

```
http://localhost:8280/apex/api/codeLookup?entity=Unit&tag=DrayStatus
<codes entity="Unit" tag="DrayStatus">
```

```
<code dbvalue="FORWARD" desc="Forward to Loading Point" />
<code dbvalue="RETURN" desc="Return to shipper" />
<code dbvalue="DRAYIN" desc="Dray In" />
<code dbvalue="OFFSITE" desc="Dray Out and Back" />
<code dbvalue="TRANSFER" desc="Transfer to other Facility" />
</codes>
```

1.7.5 Unit Categories

```
http://localhost:8280/apex/api/codeLookup?entity=Unit&tag=Category
<codes entity="Unit" tag="Category">
  <code dbvalue="IMPRT" desc="Import" />
  <code dbvalue="EXPRT" desc="Export" />
  <code dbvalue="TRSHP" desc="Transship" />
  <code dbvalue="DMSTC" desc="Domestic" />
  <code dbvalue="STRGE" desc="Storage" />
  <code dbvalue="THRGH" desc="Through" />
</codes>
```

1.7.6 Unit Hazards and IMDG Classes

```
http://localhost:8280/apex/api/codeLookup?entity=UnitHazard&tag=HazardItemImdgClass
<codes entity="UnitHazard" tag="HazardItemImdgClass">
  <code dbvalue="1" desc="1 Explosives" />
  <code dbvalue="1.1" desc="1.1 Explosives" />
  <code dbvalue="1.2" desc="1.2 Explosives" />
  <code dbvalue="1.3" desc="1.3 Explosives" />
  <code dbvalue="1.4" desc="1.4 Explosives" />
  <code dbvalue="1.5" desc="1.5 Explosives" />
  <code dbvalue="1.6" desc="1.6 Explosives" />
  <code dbvalue="2" desc="2 Gases" />
  <code dbvalue="2.1" desc="2.1 Flammable Gases" />
  <code dbvalue="2.2" desc="2.2 Non-flammable Gases" />
  <code dbvalue="2.3" desc="2.3 Toxic Gases" />
  <code dbvalue="3" desc="3 Flammable Liquids" />
  <code dbvalue="3.1" desc="3.1 Flammable Liquids" />
  <code dbvalue="3.2" desc="3.2 Flammable Liquid" />
  <code dbvalue="3.3" desc="3.3 Flammable Liquid" />
  <code dbvalue="4.1" desc="4.1 Flammable Solids" />
  <code dbvalue="4.2" desc="4.2 Spontaneously Combustibles" />
  <code dbvalue="4.3" desc="4.3 Dangerous When Wet" />
  <code dbvalue="5.1" desc="5.1 Oxidizing Agents" />
  <code dbvalue="5.2" desc="5.2 Organic Peroxides" />
  <code dbvalue="6.1" desc="6.1 Toxic Substances" />
  <code dbvalue="6.2" desc="6.2 Infectious Substances" />
  <code dbvalue="7" desc="7 Radioactive Materials" />
  <code dbvalue="8" desc="8 Corrosives" />
  <code dbvalue="9" desc="9 Other Dangerous Materials" />
  <code dbvalue="X" desc="Unknown Hazardous Material" />
  <code dbvalue="1.1A" desc="1.1A Explosives" />
  <code dbvalue="1.1B" desc="1.1B Explosives" />
  <code dbvalue="1.1C" desc="1.1C Explosives" />
  <code dbvalue="1.1D" desc="1.1D Explosives" />
  <code dbvalue="1.1E" desc="1.1E Explosives" />
  <code dbvalue="1.1F" desc="1.1F Explosives" />
```

```

<code dbvalue="1.1G" desc="1.1G Explosives" />
<code dbvalue="1.1H" desc="1.1H Explosives" />
<code dbvalue="1.1J" desc="1.1J Explosives" />
<code dbvalue="1.1K" desc="1.1K Explosives" />
<code dbvalue="1.1L" desc="1.1L Explosives" />
<code dbvalue="1.1N" desc="1.1N Explosives" />
<code dbvalue="1.1S" desc="1.1S Explosives" />
<code dbvalue="1.2A" desc="1.2A Explosives" />
<code dbvalue="1.2B" desc="1.2B Explosives" />
<code dbvalue="1.2C" desc="1.2C Explosives" />
<code dbvalue="1.2D" desc="1.2D Explosives" />
<code dbvalue="1.2E" desc="1.2E Explosives" />
<code dbvalue="1.2F" desc="1.2F Explosives" />
<code dbvalue="1.2G" desc="1.2G Explosives" />
<code dbvalue="1.2H" desc="1.2H Explosives" />
<code dbvalue="1.2J" desc="1.2J Explosives" />
<code dbvalue="1.2K" desc="1.2K Explosives" />
<code dbvalue="1.2L" desc="1.2L Explosives" />
<code dbvalue="1.2N" desc="1.2N Explosives" />
<code dbvalue="1.2S" desc="1.2S Explosives" />
<code dbvalue="1.3A" desc="1.3A Explosives" />
<code dbvalue="1.3B" desc="1.3B Explosives" />
<code dbvalue="1.3C" desc="1.3C Explosives" />
<code dbvalue="1.3D" desc="1.3D Explosives" />
<code dbvalue="1.3E" desc="1.3E Explosives" />
<code dbvalue="1.3F" desc="1.3F Explosives" />
<code dbvalue="1.3G" desc="1.3G Explosives" />
<code dbvalue="1.3H" desc="1.3H Explosives" />
<code dbvalue="1.3J" desc="1.3J Explosives" />
<code dbvalue="1.3K" desc="1.3K Explosives" />
<code dbvalue="1.3L" desc="1.3L Explosives" />
<code dbvalue="1.3N" desc="1.3N Explosives" />
<code dbvalue="1.3S" desc="1.3S Explosives" />
<code dbvalue="1.4A" desc="1.4A Explosives" />
<code dbvalue="1.4B" desc="1.4B Explosives" />
<code dbvalue="1.4C" desc="1.4C Explosives" />
<code dbvalue="1.4D" desc="1.4D Explosives" />
<code dbvalue="1.4E" desc="1.4E Explosives" />
<code dbvalue="1.4F" desc="1.4F Explosives" />
<code dbvalue="1.4G" desc="1.4G Explosives" />
<code dbvalue="1.4H" desc="1.4H Explosives" />
<code dbvalue="1.4J" desc="1.4J Explosives" />
<code dbvalue="1.4K" desc="1.4K Explosives" />
<code dbvalue="1.4L" desc="1.4L Explosives" />
<code dbvalue="1.4N" desc="1.4N Explosives" />
<code dbvalue="1.4S" desc="1.4S Explosives" />
<code dbvalue="1.5A" desc="1.5A Explosives" />
<code dbvalue="1.5B" desc="1.5B Explosives" />
<code dbvalue="1.5C" desc="1.5C Explosives" />
<code dbvalue="1.5D" desc="1.5D Explosives" />
<code dbvalue="1.5E" desc="1.5E Explosives" />
<code dbvalue="1.5F" desc="1.5F Explosives" />
<code dbvalue="1.5G" desc="1.5G Explosives" />
<code dbvalue="1.5H" desc="1.5H Explosives" />
<code dbvalue="1.5J" desc="1.5J Explosives" />
<code dbvalue="1.5K" desc="1.5K Explosives" />

```



```
<code dbvalue="1.5L" desc="1.5L Explosives" />
<code dbvalue="1.5N" desc="1.5N Explosives" />
<code dbvalue="1.5S" desc="1.5S Explosives" />
<code dbvalue="1.6A" desc="1.6A Explosives" />
<code dbvalue="1.6B" desc="1.6B Explosives" />
<code dbvalue="1.6C" desc="1.6C Explosives" />
<code dbvalue="1.6D" desc="1.6D Explosives" />
<code dbvalue="1.6E" desc="1.6E Explosives" />
<code dbvalue="1.6F" desc="1.6F Explosives" />
<code dbvalue="1.6G" desc="1.6G Explosives" />
<code dbvalue="1.6H" desc="1.6H Explosives" />
<code dbvalue="1.6J" desc="1.6J Explosives" />
<code dbvalue="1.6K" desc="1.6K Explosives" />
<code dbvalue="1.6L" desc="1.6L Explosives" />
<code dbvalue="1.6N" desc="1.6N Explosives" />
<code dbvalue="1.6S" desc="1.6S Explosives" />
</codes>
```

Part 2

Holds Permissions Update (HPU) API

In this Section:

HPU API.....	47
--------------	----

2.1 HPU API

HPU, short for Holds-Permissions-Update allows an external system to update holds and permissions of logical business entities. HPU can be invoked through the N4 Generic Web Service or as an SNX message which can be delivered via JMS.

In This Section

- Logical business entities. 48
- Service Rules. 48
- HPU format. 48
- Create a Unit on the Fly. 51
- Update Process & Return Error Messages. 51
- Security. 53

2.1.1 Logical business entities

This term is technical and describes all the entities in the system that extend the interface LogicalEntity (can be Serviceable or Guardian) and that are therefore entities to which holds and permissions can be applied. Currently N4 supports the following business entities:

- Container*
- Chassis*
- Units*
- Accessory*
- Export Booking*
- Bill Of Lading

These Entities are Planned for Delivery in Future Releases pending client demand. Please contact your Navis Professional Services representative if you need any one of the below entities:

- Vessel Visit
- Vessel
- Train Visit
- Delivery Order
- Loadout Order
- (Yard)
- (EDI Session)

2.1.2 Service Rules

You MUST define a Service Rule for a hold to be effective. Even though a user can NOT manually add a hold to an entity through the UI without a linked service rule, the HPU API currently allows you to add any hold to an entity even if a service rule does not exist.

2.1.3 HPU format

The hpu xml message has two main parts:

- entities
- flags

The entities tag defines to which entity all the holds and permissions of the flags part should be applied. An 'entities' tag has one child element, either Containers, <chassis>, <units>, <accessories>, etc., which in turn can have one to multiple entity definitions.

Container vs. Unit holds

Container holds are on the actual container equipment and will be used rarely. Most gate operations will use Unit holds. In previous versions of 1.4, Container tag worked for unit holds. In versions later than 1.4.P, you must use the unit-identity element to apply a hold or a permission to the unit.

For containers (Equipment hold or permission)

```
<entities>
  Containers
    <ctr id="MATU2072710"/>
    <ctr id="MATU2356574"/>
  </containers>
</entities>
```

For chassis (Equipment hold or permission)

```
<entities>
  <chassis>
    <chs id="MATU3333333"/>
    <chs id="MATU4444444"/>
  </chassis>
</entities>
```

For accessories (Equipment hold or permission)

```
<entities>
  <accessories>
    <acc id="MATU7777777"/>
    <acc id="MATU8888888"/>
  </accessories>
</entities>
```

For units we need to use a unit-identity tag to define the correct target

```
<entities>
  <units>
    <unit>
      <unit-identity id="MATU2072710">
        <carrier direction="IB" qualifier="DECLARED" mode="VESSEL"
id="MNA136"/>
      </unit-identity>
      <unit-identity id="MATU5349023" />
    </unit>
  </units>
</entities>
```

For export bookings

```
<entities>
  <export_bookings>
    <bkg id="001" line="MSK"/>
    <bkg id="002" line="APL" vessel_visit="AA01"/>
  </export_bookings>
</entities>
```



The optional vessel_visit attribute: in case the line operator has unique booking ids by vessel, it is recommended to add the vessel visit id in order to make sure the system can uniquely identify the target export booking.

For bills of lading

```
<entities>
  <bills_of_lading>
    <bl nbr="BL001" line="MSK" carrier-visit="AA01"/>
  </bills_of_lading>
</entities>
```

```
<bl nbr="BL002" line="APL" carrier-visit="GEN_VESSEL"/>
</bills_of_lading>
</entities>
```

All three attributes, nbr, line, and carrier-visit are required attributes for the <bl> tag. If Bill of Lading is created with General Vessel reference then it <carrier-visit> tag should say "GEN_VESSEL".

The flags are defined within the flags tag and each flag requires the hold/permission id, as well as the action to be used when applying the flag:

```
<flags>
  <flag hold-perm-id="HOLD_1" action="ADD_HOLD"/>
  <flag hold-perm-id="HOLD_2" action="RELEASE_HOLD"/>
  <flag hold-perm-id="PERMISSION_1" action="GRANT_PERMISSION"/>
  <flag hold-perm-id="PERMISSION_2" action="CANCEL_PERMISSION"/>
</flags>
```

2.1.3.1 Allowed actions

Allowed actions are:

- ADD_HOLD
- RELEASE_HOLD
- GRANT_PERMISSION
- CANCEL_PERMISSION

The following code segment shows a complete valid HPU message for the bill of lading entity, where the hold, HOLD_1 is added to both the BLs, BL001 and BL002:

```
<hpu>
  <entities>
    <bills_of_lading>
      <bl nbr="BL001" line="Line1" carrier-visit="AA01"/>
      <bl nbr="BL002" line="Line2" carrier-visit="GEN_VESSEL"/>
    </bills_of_lading>
  </entities>
  <flags>
    <flag hold-perm-id="HOLD_1" action="ADD_HOLD"/>
  </flags>
</hpu>
```

If you use HPU to apply or release holds/permissions to a bill of lading, SPARCS N4 does not perform any Quantity related checks associated with the bill of lading.

Here is another example of a complete, valid HPU message for a unit:

```
<hpu>
  <entities>
    Containers
      <ctr id="MATU2072710"/>
    </containers>
  </entities>
  <flags>
    <flag hold-perm-id="SNX_ON_CTR" action="ADD_HOLD"/>
  </flags>
</hpu>
```

2.1.4 Create a Unit on the Fly

It is now possible to have an HPU message create a Unit 'on-the-fly'. This is useful for the situation where a Hold is sent for a Unit before the Unit has been created by other means.

In order to obtain this functionality, the caller must specify 'allow-create="true"' in the <hpu> element. Also, the <unit-identity> element must have a train child element (so that the system knows what kind of unit to create).

Here is an example:

```
<hpu create-allowed="true">
  <entities>
    <units>
      <unit id="MAEU7542981">
        <carrier mode="TRUCK" direction="IB"/>
      </unit>
    </units>
  </entities>
  <flags>
    <flag hold-perm-id="SNX_UNIT_HLD" action="ADD_HOLD"/>
  </flags>
</hpu>
```

Assuming the Unit MAEU7542981 does not exist in the system, this message will result one being created, with the specified hold attached. Due to the specification:

```
<carrier mode="TRUCK" direction="IB"/>
```

The created Unit will be an Export with TRUCK as the declared arrival mode and VESSEL as the declared departure mode. This Unit will be a 'measely unit', with unknown Operator and unknown Equipment Type. (This is the same result as when Units are created on-the-fly via CUSRES EDI.)"

2.1.5 Update Process & Return Error Messages

The message returned by HPU webservice have two parts:

A document level part that can have a status REJECTED, ACCEPTED, or WARNING

An entity level part that starts with a summary of how many entities were updated successfully, failed, or were not found, followed by sub-messages for each entity (e.g. unit).

The table below shows examples of possible errors.

Error Cause	return message
Container does not exist in system	<pre> <update-status> <document-update status="ACCEPTED" ></ document-update > <entity-updates amount-successful="0" amount-failed="0" amount-not-found="1"> <logical-entity-update logical- entity="UNIT" id="XXXXX2223331" status="ENTITY_NOT_FOUND" >Entities segment not valid: The 'unit' element has a unit-identity definition that is not valid: Unknown Equipment: 'XXXXX2223331'.</logical-entity-update> </entity-updates> </update-status> </pre>
Vessel visit does not exist in the system	<pre> <update-status> <document-update status="ACCEPTED" ></ document-update > <entity-updates amount-successful="0" amount-failed="0" amount-not-found="1"> <logical-entity-update logical- entity="UNIT" id="NYKU2223331" status="ENTITY_NOT_FOUND" >Entities segment not valid: The 'unit' element has a unit-identity definition that is not valid: The vessel visit defined can not be found! The vessel visit id is 'PRXX053A'.</logical-entity-update> </entity-updates> </update-status> </pre>
Hold does not exist in the system	<pre> <update-status> <document-update status="ACCEPTED" ></ document-update > <entity-updates amount-successful="0" amount-failed="0" amount-not-found="1"> <logical-entity-update logical- entity="UNIT" id="NYKU2223331" status="ENTITY_NOT_FOUND" >Entities segment not valid: The 'unit' element has a unit-identity definition that is not valid: The vessel visit defined can not be found! The vessel visit id is 'PRXX053A'.</logical-entity-update> </entity-updates> </update-status> </pre>

When multiple target entities (e.g. containers) are defined in the message, each container is identified and updated individually. This means that if one entity could not be identified or updated, a corresponding message is added to the return message and the system proceeds with the next target. Similarly, if one flag (hold/permission) was not correctly defined, a corresponding message will be added to the document level of the return message and this flag is ignored for all the targets. The system will go on to apply the next flag.

In the example below, two units are defined which results in one successful update and one failure,

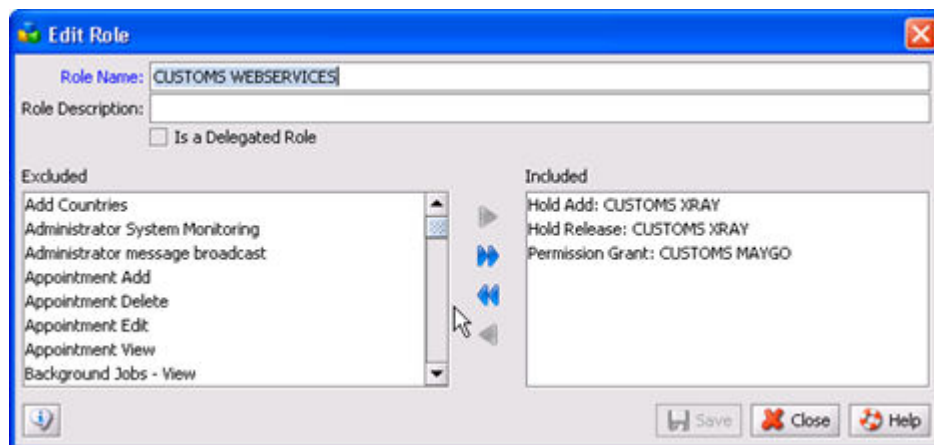

```
<hpu>
  <entities>
    <units>
      <unit-identity id="MATU5349023" />
      <unit-identity id="NOTTHERE2223331"/>
    </units>
  </entities>
  <flags>
    <flag hold-perm-id="LCT" action="ADD_HOLD"/>
  </flags>
</hpu>
```

The return message when sent via Webservices will return:

```
<update-status>
  <document-update status="ACCEPTED" ></document-update >
  <entity-updates amount-successful="1" amount-failed="0" amount-not-found="1">
    <logical-entity-update logical-entity="UNIT" id="MATU5349023"
status="ENTITY_NOT_FOUND" >Entities segment not valid: The 'unit' element has a unit-
identity definition that is not valid: Unknown Equipment: 'MATU5349023'.</logical-
entity-update>
    <logical-entity-update logical-entity="UNIT" id="NYKU2223331" status="SUCCESSFUL"
>Action 'Add Hold' for flag with id 'LCT' was successfully completed for entity.</
logical-entity-update>
  </entity-updates>
</update-status>
```

2.1.6 Security

Actions on holds and permissions are secured by the same privileges governing actions through the UI. The user called with the web service must be assigned roles which have the relevant privileges. In the example below, create a role for updating holds and permissions which should be assigned to the associated user making the webservice call.



If you want certain holds and permissions to be updated only via web services, grant the associated privileges only to your web service user.

Part 3

Incremental Container Update (ICU) API

In this Section:

ICU API.....	55
--------------	----

3.1 ICU API

The Incremental Container Update (ICU) API allows external systems to update the properties of existing Units in N4. You can invoke the ICU API either through the N4 Generic Web Service or as an SNX message delivered via JMS. ICU is a low-level API intended to be used by the terminal itself, not third parties.

To use ICU, the caller passes a small XML document that describes the Units to be updated and the changes to be made to them. ICU returns a status code and error messages when appropriate.

In This Section

Configuration and Security Aspects.	56
Message Structure.	56
ICU Examples.	57
Event Recording for ICU Updates.	57

3.1.1 Configuration and Security Aspects

ICU is a standard N4 webservice and doesn't need anything configured above and beyond what the standard N4 webservices require. You need an N4 user that calls the ICU API. As mentioned earlier, this service is expected to be used by the clients that know and care about the data integrity of the end (N4) system. The data posted through ICU doesn't go through all the usual validations that the UI based data does.

3.1.2 Message Structure

Formal XML schema definition is included in `snx.xsd` file which is present in the footprint of N4.

'units' element

In the 'units' element you specify the Units to be updated. Each Unit is identified in the sub-element 'unit'. A Unit is identified by its 'id', which in the case of a container Unit is the entire container number (includes the check-digit).

The 'type' attribute specifies how the provided id is to be used to locate the Unit. The specification `type="standard"` means that the provided id is the id of the primary equipment associated with the Unit. This is the recommended approach for identifying container or bare-chassis Units, and is the default if this element is omitted. See below for alternatives.

 Currently only containerized units are supported in ICU hence `type="CONTAINERIZED"` is the only valid choice. In addition to the Unit id, you can (optionally) specify a carrier with which the Unit is associated. The purpose of this is to disambiguate, and it is recommended. Typically, you will specify the declared inbound carrier for import Units, and the declared outbound carrier for export Units. If you do not specify a carrier when identifying a unit, N4 will search for the ACTIVE Unit with the given ID. If there is no ACTIVE Unit, then no Unit will be updated and an error will be reported.

'properties' element

In the 'properties' element you specify the properties that are to be updated. The specified properties will be updated for all Units identified in the 'units' element. Each property is identified by the 'tag' attribute. The allowed values of this attribute are the N4 standard External tags for the ICU Updateable Fields. The new value for the property is defined in the 'value' attribute.

For a complete list of External Tags for the available ICU Updateable Fields, see the *Navis N4: Reference Guide* for your version. You can also get these tags from the N4 online help or the **Reporting Entities** view in N4.

Alternative means of identifying units

To update a Break-bulk Unit you must specify `type="breakbulk"`. This tells N4 to find the Unit with Freight Kind Breakbulk with the provided id.

To update properties of a chassis which may or may not be mounted, specify `type="equipment"`. This tells N4 to find the ACTIVE Unit that uses the Equipment with the provided ID in any role (primary, carriage, etc.)

Events

You can specify an event id with your ICU message which will invoke any defined notice requests per unit defined.

 Event effects are not triggered when an event is applied to a unit via an SNX ICU message. However, the



notices are executed and Groovy code in the event notice could apply any updates the business scenerio requires.

3.1.3 ICU Examples

Below is an example ICU document that could be sent into N4:

```
<icu>
  <units>
    <unit-identity id="MATU8887773" type="CONTAINERIZED">
      <carrier direction="IB" qualifier="DECLARED"
        mode="VESSEL" id="RJP001"/>
    </unit-identity>
    <unit-identity id="MATU8882225" type="CONTAINERIZED">
      <carrier direction="OB" qualifier="ACTUAL"
        mode="VESSEL" id="NAW101"/>
    </unit-identity>
  </units>
  <properties>
    <property tag="SpecialStow" value="XYZ"/>
    <property tag="LineOperator" value="NYK"/>
  </properties>
</icu>
```

In this example, two basic container Units are to be updated such that their 'special stow' property is set to 'XYZ' and their line operator is set to 'NYK'.

3.1.4 Event Recording for ICU Updates

```
<icu>
  <units>
    <unit-identity id="NYKU4499874" type="CONTAINERIZED"/>
    <unit-identity id="NYKU6481129" type="CONTAINERIZED"/>
    <unit-identity id="NYKU6426380" type="CONTAINERIZED"/>
  </units>
  <properties>
    <property tag="GrossWeightKgValue" value="3400"/>
    <property tag="SealNbr1" value="TESTPNC"/>
  </properties>
  <event id="TEST EVENT" note="Event Note" time-event-applied="2007-07-23T11:23:24" user-
    id="icuUser" />
</icu>
```

As mentioned above, specifying an optional `event` tag in the ICU xml generates an audit trail of the ICU updates. The audit trail (events history) can be seen by going to N4 UI -> Unit Inspector -> Event History sub-panel. Following screen shot shows history for a test container that has been updated via ICU:

Unit Inspector for XXXU1234567

Container: 
Unit Nbr : [XXXU1234567](#)
Type ISO : XXX

Status:
T-State : Yard
Complex Position : Y-YRD1111
Last Move : 22 Jul 20:06
Fight Kind : FCL
Line Op : XXX
Weight (kg) : 3084 kg
Stow :
Booking Number : [XXXZZZ1](#)

Transit:
Category : Export
POD :
I/B Actual Visit : GEN_TRUCK
O/B Intend Visit : [AXX222](#)
Time In : 22 Jul 20:06
O/B Intend ETD :

Actions:  

Time	Description	Billable	Recorder	Notes	Changes
22 Jul 20:06	Testing Event Recording for ICU (CTR)		admin	Ctrl Event Note from Seal Numbers Update via ICU	null 4
22 Jul 20:06	Testing Event Recording for ICU (CTR)		admin	Ctrl Event Note from Seal Numbers Update via ICU	null 2
22 Jul 20:06	Testing Event Recording for ICU (CTR)		admin	Ctrl Event Note from Seal Numbers Update via ICU	null 1
22 Jul 20:06	Testing Event Recording for ICU (CTR)		admin	Ctrl Event Note from Seal Numbers Update via ICU	null 3
22 Jul 20:06	Activate				
22 Jul 20:05	Create				

Event Details

Event Type:
Event ID : **IcuCtrEventTestType**
Description : **Testing Event Recording for ICU (CTR)**
Billable : **No**

Event Info:
Target Entity : **XXXU1234567**
Container
Recorder : **admin**
Time : **22 Jul 20:06**
Qty :
Qty Unit :

Site Info:
Operator : **OPR1**
Facility : **FCY111**
Complex : **CPX11**
Yard : **YRD1111**

Notes: Ctrl Event Note from Seal Numbers Update via ICU

Field Changes:
Seal Nbr1 : null → 1

Note that in the following snapshot, event times are supplied by the caller:

Unit Inspector for XXXU1234567

Container: 
Unit Nbr : [XXXU1234567](#)
Type ISO : XXX

Status:
T-State : Yard
Complex Position : Y-YRD1111
Last Move : 07-Sep-07 1158
Fight Kind : FCL
Line Op : XXX
Weight (kg) : 3084 kg
Stow :
Booking Number : [XXXZZZ1](#)

Transit:
Category : Export
POD :
I/B Actual Visit : GEN_TRUCK
O/B Intend Visit : [AXX222](#)
Time In : 07-Sep-07 1158
O/B Intend ETD :

Actions:  

Time	Description	Billable	Recorder	Notes
07-Sep-07 1158	Activate		-system-	
07-Sep-07 1158	Create		-system-	test
07-Jul-23 1123	Testing Event Recording for ICU (CTR)		admin	Testing Event Recording for ICU
07-Jul-23 1123	Testing Event Recording for ICU (CTR)		admin	Testing Event Recording for ICU
07-Jul-23 1123	Testing Event Recording for ICU (CTR)		admin	Testing Event Recording for ICU
07-Jul-23 1123	Testing Event Recording for ICU (CTR)		admin	Testing Event Recording for ICU

As mentioned in the above discussion events are recorded when they are part of the XML. An ICU update will result in a 'Property Update' event by default (if ICU XML didn't have the event element). The events will look like the following snapshot:

Unit Inspector for XXXU1234567



Unit Nbr : [XXXU1234567](#)
Type ISO : XXX

Status

T-State : Yard
Complex Position : Y-YRD1111
Last Move : 07-Sep-08 0418
Fight Kind : FCL
Line Op : XXX
Weight (kg) : 3084 kg
Stow :
Booking Number : [XXXZZZ1](#)

Transit

Category : Export
POD :
I/B Actual Visit : GEN_TRUCK
O/B Intend Visit : [AXX222](#)
Time In : 07-Sep-08 0418
O/B Intend ETD :

Actions

Primary Equip. Routing Contents Damages Holds/Perms Storage **Event History** All Equipment Data Sources Bills of Lading Itinerary

Time	Description	Billable	Recorder	Notes
07-Sep-08 0419	Property Update		admin	Property Update
07-Sep-08 0418	Activate		-system-	
07-Sep-08 0418	Create		-system-	test

If the user specifies user-id attribute in the ICU xml then the user specified in this attribute's value is persisted in the database and can be seen in the Event History tab of the Unit Inspector in the Recorder column as shown below:

Unit Inspector for XXXU1234567



Unit Nbr : [XXXU1234567](#)
Type ISO : XXX

Status

T-State : Yard
Complex Position : Y-YRD1111
Last Move : 07-Sep-13 0323
Fight Kind : FCL
Line Op : XXX
Weight (kg) : 3084 kg
Stow :
Booking Number : [XXXZZZ1](#)

Transit

Category : Export
POD :
I/B Actual Visit : GEN_TRUCK
O/B Intend Visit : [AXX222](#)
Time In : 07-Sep-13 0323
O/B Intend ETD :

Actions

Primary Equip. Routing Contents Damages Holds/Perms Storage **Event History** All Equipment Data Sources Bills of Lading Itinerary

Time	Description	Billable	Recorder	Notes
07-Sep-13 0323	Activate		-system-	
07-Sep-13 0323	Create		-system-	test
07-Jul-23 1123	Testing Event Recording for ICU (UNIT)		snx:IcuTester	Testing Event Recording for ICU (UNIT) on Thu Sep 13 01:22:05 PDT 2
07-Jul-23 1123	Testing Event Recording for ICU (UNIT)		snx:IcuTester	Testing Event Recording for ICU (UNIT) on Thu Sep 13 01:22:05 PDT 2
07-Jul-23 1123	Testing Event Recording for ICU (UNIT)		snx:IcuTester	Testing Event Recording for ICU (UNIT) on Thu Sep 13 01:22:05 PDT 2
07-Jul-23 1123	Testing Event Recording for ICU (UNIT)		snx:IcuTester	Testing Event Recording for ICU (UNIT) on Thu Sep 13 01:22:05 PDT 2

Part 4

Reefer Monitoring XML API

In this Section:

- Reefer Monitoring API. 61
- Inbound Messages from Reefer Monitoring System to N4. 63
- Outbound Messages from N4 to Reefer Monitoring System. 73
- Testing Reefer Monitoring Web Services. 73

4.1 Reefer Monitoring API

This section describes the XML API for interfacing with a 3rd-party Reefer Monitoring System.

What is described here includes all the functionality currently supported. Anything not addressed in the spec (e.g. reefer deletion) would require customization.

In This Section

Requirements.	62
Supported Reefer Details.	62
Reefer Messaging Flow.	62

4.1.1 Requirements

Because this API is vendor-agnostic, there are no reefer-monitoring vendor requirements.

We recommend using Web Services since JMS has not been tested.

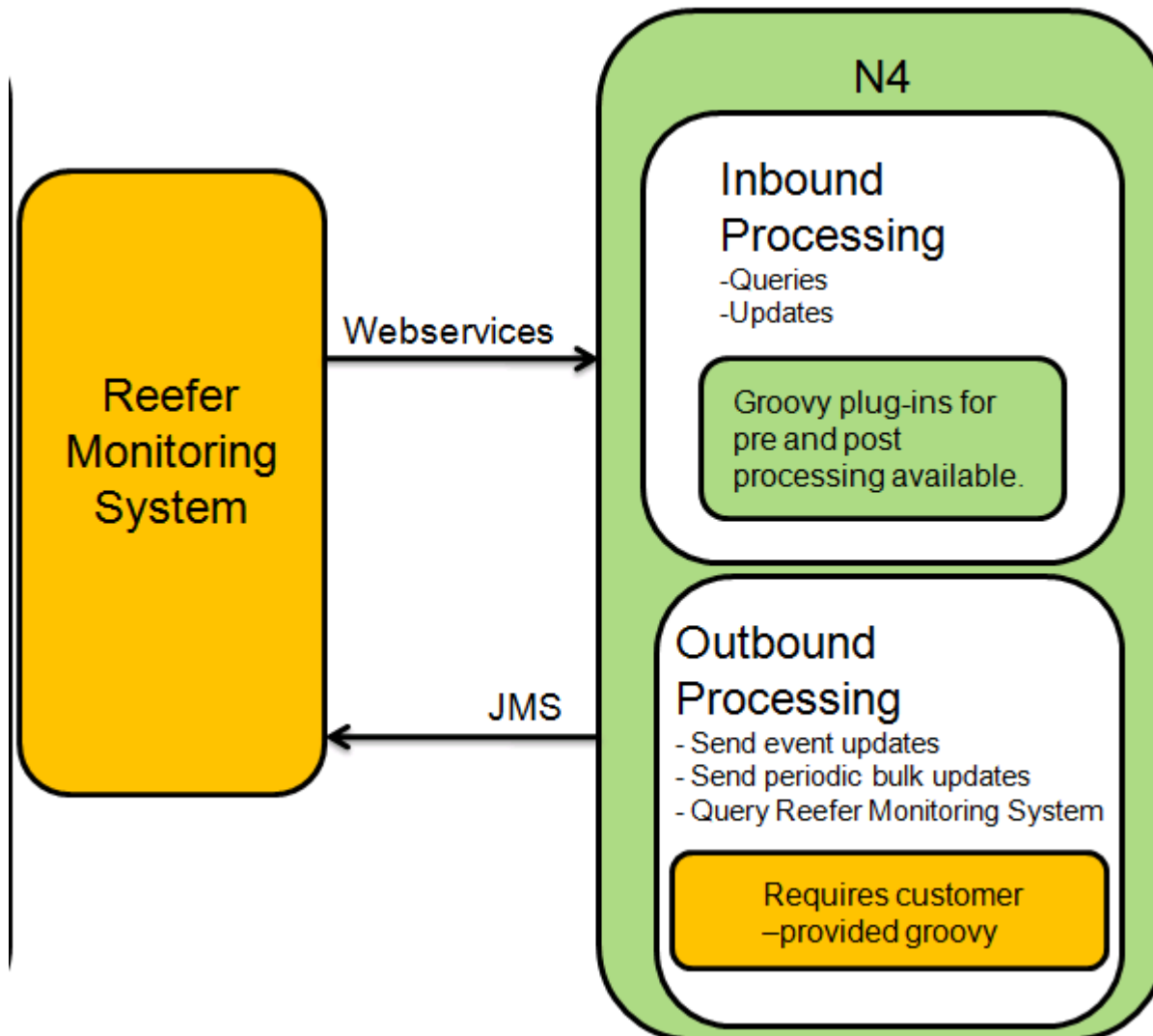
4.1.2 Supported Reefer Details

To see which reefer details are supported, refer to the following schema files, available by searching the N4 footprint at:

```
prompt[N4Root/sparcsn4/webapps/apex]% dir /s *reefer*.xsd
```

- `reefermonitoring.xsd`
- `ReeferMonitoringWebserviceRequest.xsd`
- `ReeferMonitoringWebserviceResponse.xsd`

4.1.3 Reefer Messaging Flow



4.2 Inbound Messages from Reefer Monitoring System to N4

In This Section

Querying Reefers.	64
Updating Reefers.	66

4.2.1 Querying Reefers

4.2.1.1 Querying specific reefer(s)

Use this message to retrieve information about one or many specified reefers. Note that you can only query on reefers that are in the yard.

Sample Request

```
<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <get-reefers>
    <unit-identity id="CRLU1413400"/>
  </get-reefers>
</argo:reefer-monitor>
```

Sample Response

The only response elements that are supported are the elements included in the response message below. If an optional element is not returned, it is because the value is null in the application.

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <get-reefers-response>
    <reefer-monitoring>
      <unit-identity id="CRLU1413400" type="CONTAINERIZED"/>
      <position loc-type="YARD" location="AGW" slot="111"/>
      <contents weight-kg="29900.0" weight-kg-yard-measured="-1.0" commodity-id="ACRYLIC
FIBRE" commodity-description="ACRYLIC FIBRE" shipper-id="0000010044" consignee-
id="0050979600" bl-nbr="5294437000" />
      <routing opl="OAK" pol="OAK" pol-name="United States" pod-1="HON" pod-1-
name="United States" destination="HON" designated-trucker="QMI">
        <carrier direction="IB" qualifier="DECLARED" mode="VESSEL" facility="HON"
id="MNA139" />
        <carrier direction="IB" qualifier="ACTUAL" mode="VESSEL" facility="HON"
id="MNA139" />
        <carrier direction="OB" qualifier="DECLARED" mode="TRUCK" facility="HON"
id="QMI" />
        <carrier direction="OB" qualifier="ACTUAL" mode="TRUCK" facility="HON" id="QMI" />
      </routing>
      <equipment eqid="CRLU1413400" type="45R1" created-by="camco" created-
date="2009-11-26T19:38:53" modified-by="camco"
        modified-date="2009-11-26T19:46:29" class="CTR" life-cycle-state="ACT"
role="PRIMARY">
        <physical has-wheels="N" height-mm="2895" is-insulated="N" is-permanently-out-
of-service="N" iso-group="RT" length-mm="12192"
          material="UNKNOWN" strength-code="FB" tare-weight-kg="4700.0" width-
mm="2438"/>
        <ownership owner="MAT" operator="MAT" />
        <reefer is-controlled-atmosphere="N" is-starvent="N" is-super-freeze="N" is-
temperature-controlled="Y" rfr-type="INTEG_AIR"/>
        <damages>
          <damage type="94" type-description="Dented in several Locations"
component="GENERAL" component-description="General Damages"
            severity="MINOR" reported-date="2009-11-26T19:57:30"/>
        </damages>
      </equipment>
    </reefer-monitoring>
  </get-reefers-response>
</argo-response>
```

```

    </damages>
  </equipment>
  <reefer temp-reqd-c="110.0" temp-min-c="10.0" temp-max-c="110.0" temp-display-
unit="C" o2-pct="100.0" co2-pct="0.01" humidity-pct="10.0"
    vent-required-value="10.0" vent-required-unit="PERCENTAGE" time-latest-on-
power="2012-01-07T11:42:56"
    time-monitor-1="2014-10-27T02:44:59" time-monitor-2="2013-06-17T10:14:58"
time-monitor-3="2005-09-12T01:55:02"
    time-monitor-4="2006-11-29T09:20:00" time-requested="2012-06-19T13:09:23" is-
power="N" wanted-is-power="N"/>
  <non-move-history>
    <event id="UNIT_CREATE" note="snx" time-event-applied="2014-02-13T09:34:46" user-
id="snx:-system-" is-billable="N" />
    <event id="UNIT_ACTIVATE" time-event-applied="2014-02-13T09:34:46" user-id="snx:-
system-" is-billable="N" />
    <event id="UNIT_SEAL" note="Seals Updated" time-event-applied="2014-02-13T09:34:46"
user-id="snx:-system-" is-billable="Y" related-entity-id="MATU5118075" related-entity-
class="Unit">
      <field-changes>
        <field-change id="unitForeignHostKey" new-value="MATU5118075" />
        <field-change id="unitVisitState" new-value="1ACTIVE" prev-value="2ADVISED" />
        <field-change id="unitCategory" new-value="IMPRT" prev-value="STRGE" />
        <field-change id="unitRequiresPower" new-value="true" prev-value="false" />
        <field-change id="unitGoodsAndCtrWtKg" new-value="25329.06" prev-
value="4084.375" />
        <field-change id="unitSealNbr1" new-value="816506" />
        <field-change id="unitActiveUfv" new-value="289" />
        <field-change id="unitPrimaryUe" new-value="289" />
        <field-change id="rtgOPL" new-value="6" />
        <field-change id="rtgPOL" new-value="6" />
        <field-change id="rtgPOD1" new-value="1" />
        <field-change id="rtgTruckingCompany" new-value="35" />
        <field-change id="unitTimeDenormalizeCalc" new-value="20140213113446PST" prev-
value="19691231160000PST" />
        <field-change id="unitTimeLastStateChange" new-value="20140213113446PST" prev-
value="20140213113446PST" />
        <field-change id="unitRemark" new-value="MEAT, INCLUDING POULTRY S-8/08/07" />
        <field-change id="unitFlexString03" new-value="$UTWAR" />
        <field-change id="unitIsCtrSealed" new-value="true" />
      </field-changes>
    </event>
    <event id="UNIT_PROPERTY_UPDATE" note="Sparcs Details Updated" time-event-
applied="2014-02-13T09:34:46" user-id="snx:-system-" is-billable="N">
      <field-changes>
        <field-change id="ufvTimeEcIn" new-value="20070812055632PDT" />
        <field-change id="ufvTimeIn" new-value="20070812055632PDT" />
      </field-changes>
    </event>
  </non-move-history>
  <reefer-recording-histories>
    <reefer-recording-history time-of-recording="2005-09-12T01:55:02" return-temp-
c="100.0" sensor1-tmp="10.0" sensor2-tmp="11.0"
      sensor3-tmp="12.0" sensor4-tmp="13.0" vent-setting="100.0" vent-
unit="PERCENTAGE" humidity-pct="100.0"
      o2-pct="100.0" co2-pct="100.0" rec-fault-code="N"
max_mtr_tmp="100.0" min_mtr_tmp="100.0"

```

```

        defrost-temp="100.0" tmp-set-pt="100.0" supply-tmp="100.0" fuel-
lvl="100.0" reefer-hours="100.0"
        remark="100.00" malfunc="Y" drains="Y" is-bulb-on="Y" fan="H" is-
alarm-on="Y"/>
    </reefer-recording-histories>
</reefer-monitoring>
</get-reefers-response>
</argo-response>

```

4.2.1.2 Query All

To get details for all reefers in the yard, use the following request:

```

<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <get-all-reefers-in-yard/>
</argo:reefer-monitor>

```

4.2.1.3 Query for Reefers IDs only

If you don't need full reefer IDs and you only want to see the reefers in the yard, use the following:

```

<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <get-all-reefers-in-yard id-only="Y"/>
</argo:reefer-monitor>

```

4.2.1.4 Query for Reefers but exclude non move history, chargeable unit event, and guarantee table records

To improve the performance of the Reefer query, you can include the "get-non-move-history" and "get-guarantee" attributes and set them to 'N' as follows:

```

<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <get-all-reefers-in-yard get-non-move-history="Y" get-guarantee='N' />
</argo:reefer-monitor>

```

4.2.2 Updating Reefers

You can update either a single reefer or multiple reefers in a message.

Note that under the reefer-recording-histories element, the update-mode is specified as "MERGE":

```

<reefer-recording-histories update-mode="MERGE">

```

This ensures that the update won't override the existing reefer history. If for whatever reason you do not want to keep the existing history, you can specify the element as follows:

```

<reefer-recording-histories update-mode="REPLACE">

```

4.2.2.1 Single Reefer Update Message

Use the following message to update details for a single reefer. If the reefer you're trying to update is not in the yard, you'll get an error.

```
<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <update-reefers>
    <!-- 1 or more repetitions: -->
    <reefer-monitoring>
      <unit-identity id="CUEU6434001"/>
      <position loc-type="YARD" location="X" slot="111" orientation="string" carrier-
id="string"/>
      <reefer
        temp-reqd-c="110.00"
        o2-pct="100.00"
        co2-pct="0.01"
        humidity-pct="10.00"
        vent-required-value="10.00"
        vent-required-unit="PERCENTAGE"
        temp-min-c="10.00"
        temp-max-c="110.00"
        temp-display-unit="C"
        is-power="N"
        wanted-is-power="N"
        time-latest-on-power="2012-01-07T11:42:56"
        time-monitor-1="2014-10-27T02:44:59"
        time-monitor-2="2013-06-17T10:14:58"
        time-monitor-3="2005-09-12T01:55:02-07:00"
        time-monitor-4="2006-11-29T09:20:00"
        is-alarm-on="Y"/>
      <reefer-recording-histories update-mode="MERGE">
        <!--1 or more repetitions: -->
        <reefer-recording-history
          time-of-recording="2005-09-12T01:55:02-07:00"
          return-temp-c="100.00"
          vent-setting="100.00"
          vent-unit="PERCENTAGE"
          humidity-pct="100.00"
          o2-pct="100.00"
          co2-pct="100.00"
          rec-fault-code="N"
          max_mtr_tmp="100.00"
          min_mtr_tmp="100.00"
          defrost-temp="100.00"
          tmp-set-pt="100.00"
          supply-tmp="100.00"
          fuel-lvl="100.00"
          reefer-hours="100.00"
          remark="100.00"
          sensor1-tmp="10.00"
          sensor2-tmp="11.00"
          sensor3-tmp="12.00"
          sensor4-tmp="13.00"
          is-alarm-on="Y"
          malfunc="Y"
          fan="H"
          drains="Y"
          is-bulb-on="Y"/>
        </reefer-recording-histories>
      </reefer-monitoring>
    </update-reefers>
```

```
</argo:reefer-monitor>
```

4.2.2.2 Multiple Reefer Update Message

To update multiple reefers simply add more <reefer-monitoring> tags into <update-reefers> tag, as shown below.

If any of the reefers you're trying to update are not in the yard, you'll get an error.

```
<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <update-reefers>
    <reefer-monitoring>
      <unit-identity id="CUEU6434001"/>
      .....
    </reefer-monitoring>
    <reefer-monitoring>
      <unit-identity id="QWERTY4001"/>
      .....
    </reefer-monitoring>
  </update-reefers>
</argo:reefer-monitor>
```

Example

```
<argo:reefer-monitor xmlns:argo="http://www.Navis.com/argo">
  <update-reefers>
    <!-- 1 or more repetitions: -->
    <reefer-monitoring>
      <unit-identity id="CUEU6434001"/>
      <position loc-type="YARD" location="X" slot="111" orientation="string" carrier-
id="string"/>
      <reefer
        temp-reqd-c="110.00"
        o2-pct="100.00"
        co2-pct="0.01"
        humidity-pct="10.00"
        vent-required-value="10.00"
        vent-required-unit="PERCENTAGE"
        temp-min-c="10.00"
        temp-max-c="110.00"
        temp-display-unit="C"
        is-power="N"
        wanted-is-power="N"
        time-latest-on-power="2012-01-07T11:42:56"
        time-monitor-1="2014-10-27T02:44:59"
        time-monitor-2="2013-06-17T10:14:58"
        time-monitor-3="2005-09-12T01:55:02-07:00"
        time-monitor-4="2006-11-29T09:20:00"
        is-alarm-on="Y"/>
      <reefer-recording-histories>
        <!--1 or more repetitions: -->
        <reefer-recording-history
          time-of-recording="2005-09-12T01:55:02-07:00"
          return-temp-c="100.00"
          vent-setting="100.00"
          vent-unit="PERCENTAGE"
```



```

        humidity-pct="100.00"
        o2-pct="100.00"
        co2-pct="100.00"
        rec-fault-code="N"
        max_mtr_tmp="100.00"
        min_mtr_tmp="100.00"
        defrost-temp="100.00"
        tmp-set-pt="100.00"
        supply-tmp="100.00"
        fuel-lvl="100.00"
        reefer-hours="100.00"
        remark="100.00"
        sensor1-tmp="10.00"
        sensor2-tmp="11.00"
        sensor3-tmp="12.00"
        sensor4-tmp="13.00"
        is-alarm-on="Y"
        malfunc="Y"
        fan="H"
        drains="Y"
        is-bulb-on="Y"/>
    </reefer-recording-histories>
</reefer-monitoring>
<reefer-monitoring>
    <unit-identity id="QWERTY4001"/>
    <position loc-type="YARD" location="X" slot="112" orientation="string"
carrier-id="string"/>
    <reefer
        temp-reqd-c="110.00"
        o2-pct="100.00"
        co2-pct="0.01"
        humidity-pct="10.00"
        vent-required-value="10.00"
        vent-required-unit="PERCENTAGE"
        temp-min-c="10.00"
        temp-max-c="110.00"
        temp-display-unit="C"
        is-power="N"
        wanted-is-power="N"
        time-latest-on-power="2012-01-07T11:42:56"
        time-monitor-1="2014-10-27T02:44:59"
        time-monitor-2="2013-06-17T10:14:58"
        time-monitor-3="2005-09-12T01:55:02-07:00"
        time-monitor-4="2006-11-29T09:20:00"
        is-alarm-on="Y"/>
    <reefer-recording-histories>
        <!--1 or more repetitions: -->
        <reefer-recording-history
            time-of-recording="2005-09-12T01:55:02-07:00"
            return-temp-c="100.00"
            vent-setting="100.00"
            vent-unit="PERCENTAGE"
            humidity-pct="100.00"
            o2-pct="100.00"
            co2-pct="100.00"
            rec-fault-code="N"

```

```

        max_mtr_tmp="100.00"
        min_mtr_tmp="100.00"
        defrost-temp="100.00"
        tmp-set-pt="100.00"
        supply-tmp="100.00"
        fuel-lvl="100.00"
        reefer-hours="100.00"
        remark="100.00"
        sensor1-tmp="10.00"
        sensor2-tmp="11.00"
        sensor3-tmp="12.00"
        sensor4-tmp="13.00"
        is-alarm-on="Y"
        malfunc="Y"
        fan="H"
        drains="Y"
        is-bulb-on="Y"/>
    </reefer-recording-histories>
</reefer-monitoring>
</update-reefers>
</argo:reefer-monitor>

```

4.2.2.3 Groovy plug-ins available for pre and post inbound reefer message processing

The following groovy plug-ins are available:

- **ReeferMonitoringWebServiceHandlerImplPreInvoke.groovy** - Allows user to inject code before an inbound query or update webservice message is called. This is for any action that does not require access to the N4 database, or an action that would need to be completed before the transaction. An example would be Groovy that fixes the XML message syntax to prevent the message from failing due to poor syntax.
- **ReeferMonitoringWebServiceHandlerImplPreInvokeInTx.groovy** - Allows user to inject code during a transaction before an inbound query or update webservice message is called. This would be for any action that requires accessing the transaction database. Validations are more safely done at this stage.
- **ReeferMonitoringWebServiceHandlerImplPostInvokeInTx.groovy** - Allows user to inject code during a transaction after an inbound query or update webservice message is called. At this stage, you can use groovy to impact the database. For example, you can use groovy here to undo a transaction under certain conditions.
- **ReeferMonitoringWebServiceHandlerImplPostInvoke.groovy** - Allows user to inject code after an inbound query or update webservice message is called. This would be for actions you would invoke after committing a database transaction.

ReeferMonitoringWebServiceHandlerImplPreInvoke

As a basis, you can use the following groovy snippet templates:

```

import java.util.Map;
import com.navis.apex.business.model.GroovyInjectionBase;
class ReeferMonitoringWebServiceHandlerImplPreInvoke extends GroovyInjectionBase {
    public String transform(String inXml) {
        log("transform method got called");
        return inXml;
    }
}

```

}

Exception/Transaction Handling

Exception will cause webservice invocation failure. The call will return before starting the transaction.

ReeferMonitoringWebServiceHandlerImplPreInvokeInTx

As a basis, you can use the following groovy snippet templates:

```
import java.util.Map;
import com.Navis.apex.business.model.GroovyInjectionBase;
class ReeferMonitoringWebServiceHandlerImplPreInvokeInTx extends GroovyInjectionBase {
    /**
     * This method is called just before invoking the N4 generic webservices handler.
     * @param inParameters contains the handler that is called.
     */
    public void preHandlerInvoke(Map inParameters) {
        log("preHandlerInvoke got called with :" + inParameters);
    }
}
```

Exception/Transaction Handling

Exception will cause webservice transaction to be rolledback.

ReeferMonitoringWebServiceHandlerImplPostInvoke

As a basis, you can use the following groovy snippet templates:

```
import java.util.Map;
import com.Navis.apex.business.model.GroovyInjectionBase;
class ReeferMonitoringWebServiceHandlerImplPostInvoke extends GroovyInjectionBase {
    /**
     * This method is called just after invoking the N4 generic webservices handler.
     * @param inParameters contains the handler that is called.
     */
    public void postHandlerInvoke(Map inParameters) {
        log("postHandlerInvoke (no tx) got called with :" + inParameters);
    }
}
```

Exception/Transaction Handling

Exception will cause webservice invocation failure. The call will return before starting the transaction.

ReeferMonitoringWebServiceHandlerImplPostInvokeInTx

As a basis, you can use the following groovy snippet templates:

```
import java.util.Map;
import com.Navis.apex.business.model.GroovyInjectionBase;
class ReeferMonitoringWebServiceHandlerImplPostInvokeInTx extends GroovyInjectionBase {
    /**
```

```
* This method is called just after invoking the N4 generic webservicess handler.  
* @param inParameters contains the handler that is called.  
*/  
public void postHandlerInvoke(Map inParameters) {  
    log("postHandlerInvoke got called with :" + inParameters);  
}  
}
```

Exception/Transaction Handling

Exception will cause webservice transaction to be rolled back.

4.3 Outbound Messages from N4 to Reefer Monitoring System

N4 does not provide out-of-the-box messages for sending reefer details to a 3rd-party system. Using the .xsd provided in the N4 footprint (see prompt[N4Root/sparcsn4/webapps/apex]% dir /s *reefer*.xsd), you will need to create the following outbound messages as needed.

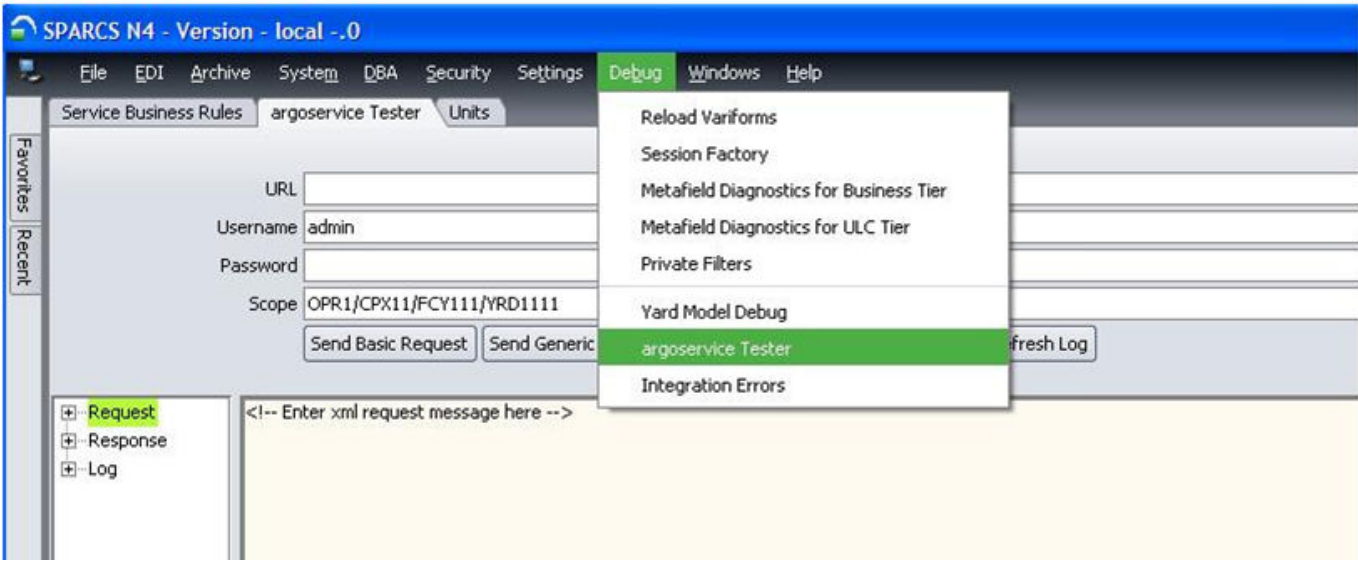
- **Single Reefer Update:** To notify the target monitoring system of reefer changes in N4.
- **Scheduled background jobs:** To periodically send the latest data for selected units.
- **Third-party queries:** To query the 3rd-party reefer monitoring system for latest data for selected units.

We recommend using outbound asynchronous JMS messaging triggered by those N4 events that impact reefer data, such as gate transactions, vessel discharges, reefer inspections, etc.

4.4 Testing Reefer Monitoring Web Services

To test your reefer monitor web service:

1. In N4, select the **Administration** → **Debug** → **argoservice Tester** option, as shown in the graphic below:



2. Paste your message in the request text field, as shown in the graphic below:

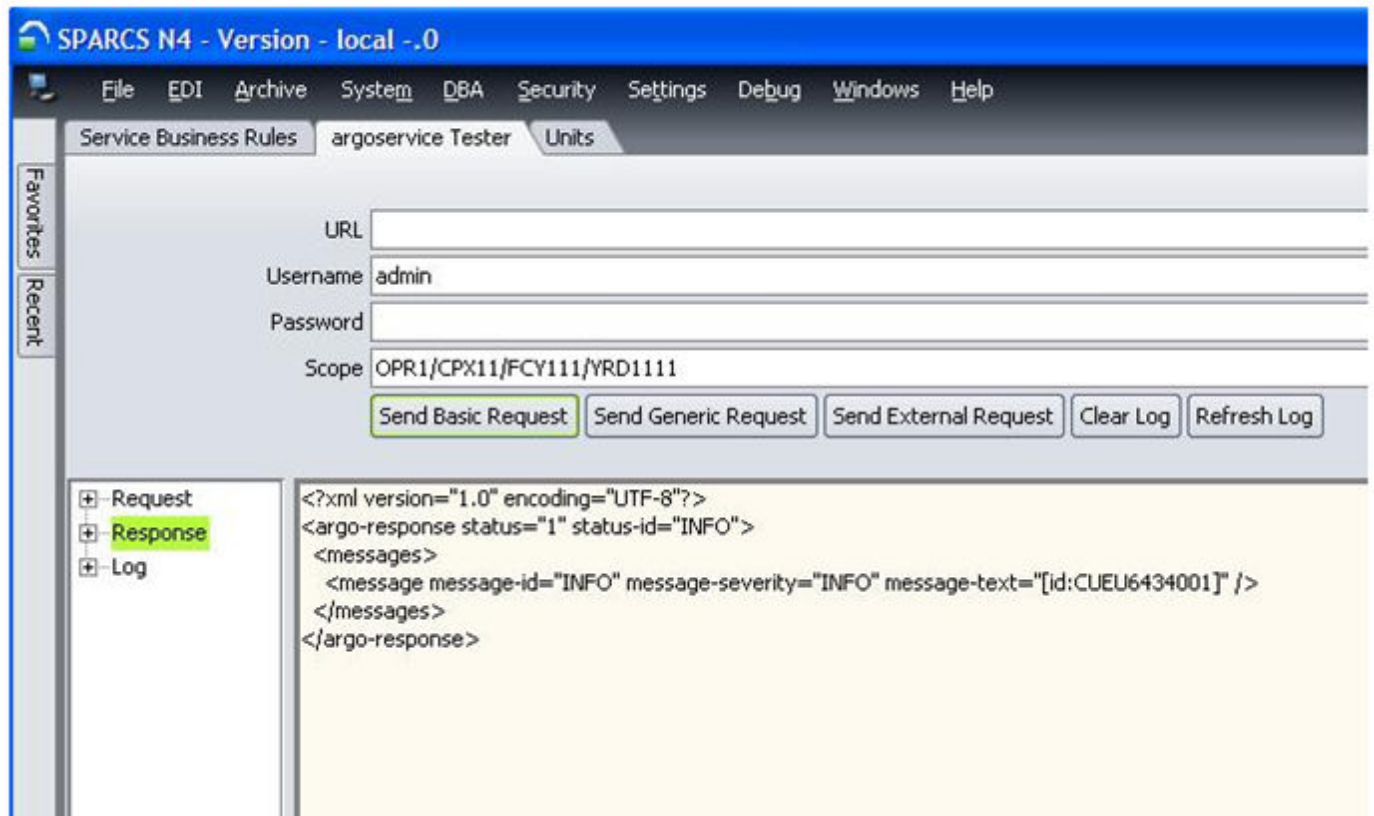
The screenshot shows the SPARCS N4 - Version - local -.0 application. The 'argoservice Tester' tab is active, displaying fields for URL, Username (admin), Password, and Scope (OPR1/CPX11/FCY111/YRD1111). Below these fields are buttons for 'Send Basic Request', 'Send Generic Request', 'Send External Request', 'Clear Log', and 'Refresh Log'. The 'Request' tab is selected in the left sidebar, showing an XML message in the main text area.

```
<argo:reefer-monitoring xmlns:argo="http://www.navis.com/argo">
  <update-reefers>
    <reefer-monitoring>
      <unit-identity id="CUEU6434001"/>
      <position loc-type="YARD" location="X" slot="111" orientation="string" carrier-id="string"/>

      <reefer
        temp-reqd-c="110.00"
        o2-pct="100.00"
        co2-pct="0.01"
        humidity-pct="10.00"
        vent-required-value="10.00"
        vent-required-unit="PERCENTAGE"
        temp-min-c="10.00"
        temp-max-c="110.00"
        temp-display-unit="C"
        is-power="N"
        wanted-is-power="N"
        time-latest-on-power="2012-01-07T11:42:56"
        time-monitor-1="2014-10-27T02:44:59"
        time-monitor-2="2013-06-17T10:14:58"
        time-monitor-3="2005-09-12T01:55:02-07:00"
        time-monitor-4="2006-11-29T09:20:00"/>

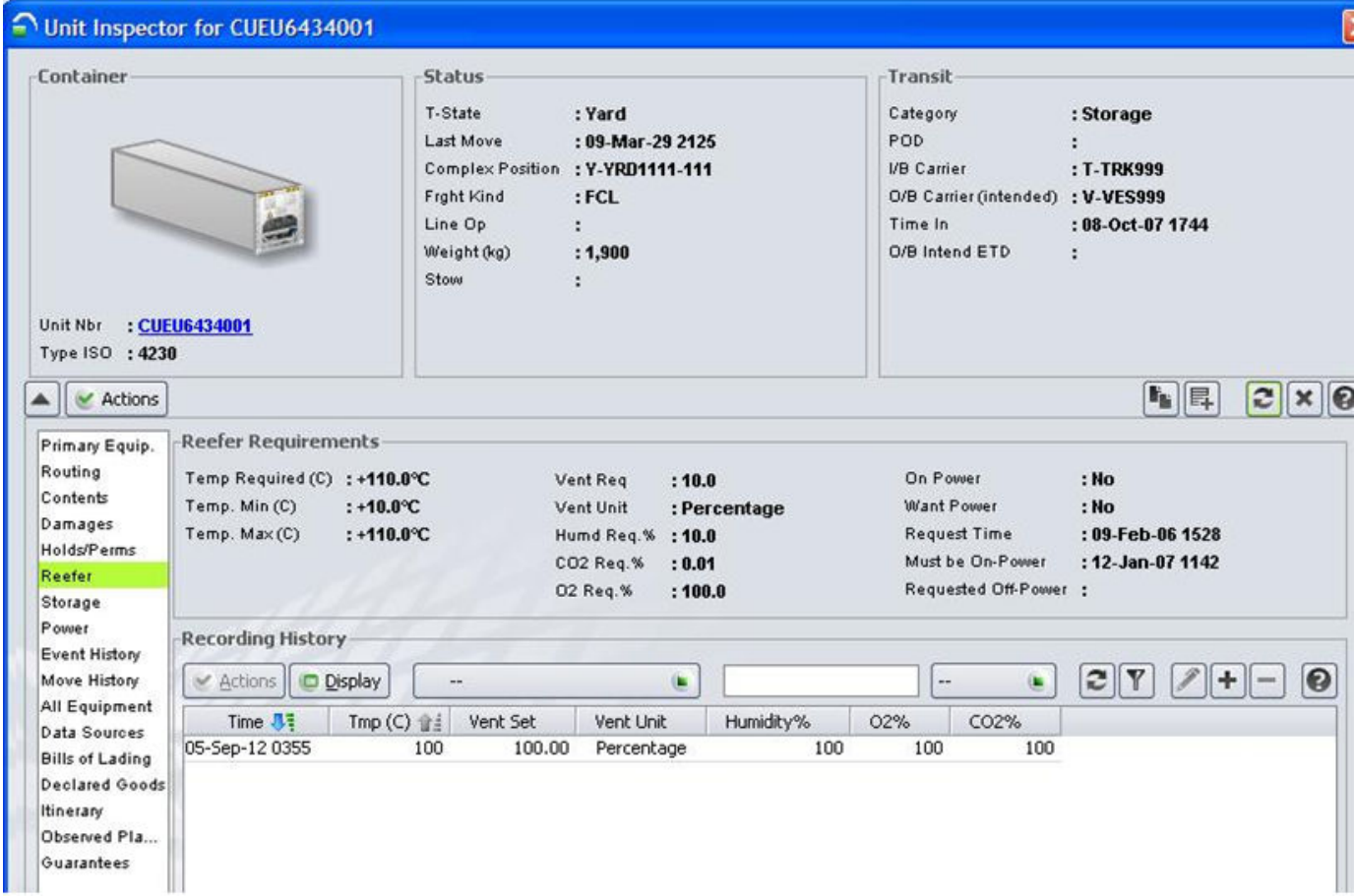
      <reefer-recording-histories>|
        <reefer-recording-history
          time-of-recording="2005-09-12T01:55:02-07:00"
          return-temp-c="100.00"
```

3. Click **Send Basic Request** and you should see web service response, as shown below:



If you do not see any error messages, it means that N4 processed your request successfully.

4. Use the **Unit Inspector** to verify your update, as shown below:



The screenshot shows the 'Unit Inspector for CUEU6434001' window. It is divided into several sections: 'Container' with a unit image and details (Unit Nbr: CUEU6434001, Type ISO: 4230); 'Status' with fields like T-State (Yard), Last Move (09-Mar-29 2125), Complex Position (Y-YRD1111-111), Frght Kind (FCL), Line Op, Weight (kg) (1,900), and Stow; 'Transit' with fields like Category (Storage), POD, I/B Carrier (T-TRK999), O/B Carrier (intended) (V-VES999), Time In (08-Oct-07 1744), and O/B Intend ETD; 'Reefer Requirements' with temperature and humidity settings; and 'Recording History' with a table of recent data points. A left-hand menu lists various inspection categories, with 'Reefer' currently selected.

Unit Inspector for CUEU6434001

Container

Unit Nbr : **CUEU6434001**
Type ISO : **4230**

Status

T-State : **Yard**
Last Move : **09-Mar-29 2125**
Complex Position : **Y-YRD1111-111**
Frght Kind : **FCL**
Line Op :
Weight (kg) : **1,900**
Stow :

Transit

Category : **Storage**
POD :
I/B Carrier : **T-TRK999**
O/B Carrier (intended) : **V-VES999**
Time In : **08-Oct-07 1744**
O/B Intend ETD :

Reefer Requirements

Temp Required (C) : +110.0°C	Vent Req : 10.0	On Power : No
Temp. Min (C) : +10.0°C	Vent Unit : Percentage	Want Power : No
Temp. Max (C) : +110.0°C	Humd Req.% : 10.0	Request Time : 09-Feb-06 1528
	CO2 Req.% : 0.01	Must be On-Power : 12-Jan-07 1142
	O2 Req.% : 100.0	Requested Off-Power :

Recording History

Time	Tmp (C)	Vent Set	Vent Unit	Humidity%	O2%	CO2%
05-Sep-12 0355	100	100.00	Percentage	100	100	100

Part 5

N4 Web Services

This section covers how customers can call N4 Groovy Web Services.

Navis will not support any Navis Web Services extensions that are built or maintained by non-Navis resources. Navis Partners are external to Navis and therefore any extensions they may create or maintain for these web services are also not supported.

In this Section:

N4 Web Services: Introduction.	78
Security and Session Management.	81
Argo Generic and Basic Web Service API.	83
Error Handling.	85
Development Guide.	87
Argo Basic Service Client Example for .NET (ArgoServiceTester).	93
Client Web Services Integration for .NET.	111
Expose User SNX schema to Web service.	122
N4 Compatible External Web Service.	128
Report Generation Web Service.	139
Invoking Outbound WebService from N4 which is compliant with N4 basic service.	146

5.1 N4 Web Services: Introduction

In This Section

Standards Compliance.	79
Development Platforms and Web Service Toolkits.	79
Available Services.	79

5.1.1 Standards Compliance

The general term Web Services refers to any mode of communication over http and can also encompass technologies such as XML-RPC and REST. When Navis refers to Web Services, we are specifically referring to Web Services delivered as a SOAP based message which conforms to a WSDL.

N4 Web Services APIs conform to the following standards:

Simple Object Access Protocol (SOAP) 1.1: For more information, visit <http://www.w3.org/TR/2000/NOTE-SOAP-20000508/>.

Web Service Description Language (WSDL) 1.1: For more information, visit <http://www.w3.org/TR/2001/NOTE-wsdl-20010315>.

5.1.2 Development Platforms and Web Service Toolkits

You can invoke N4 Web Service APIs with any SOAP development environment which support the above standards. Common examples include as Apache Axis/Java and Visual Studio .Net. Most development environments have toolkits to generate the Web Services API code. Using the WSDL, these toolkits generate the client stub classes in a target programming language. This document includes links to integration examples for:

Java client example using Apache Axis 1.3 (<http://ws.apache.org/axis/>) and Java Standard Edition Development Kit 6.0 (JDK 6.0).

.Net client example using Microsoft .Net v20

5.1.3 Available Services

N4 Web Services delivers three distinct style of API's:

- **generic** - gate, ICU, HPU which go through a single service called genericInvoke. All new API's will be developed in this style
- **basic**- introduced in 1.5.C, an alternative to the generic response has a simple string return type
- **static** - Billing API's for updating and proposing paid-thru date as well as the old versions of gate integration with separate business specific WSDLs

N4 ships with the following supported WSDLs:

WSDL	description
argo-service.wsdl	genericInvoke Web Service used for Gate, ICU, HPU.
argobasic-service.wsdl	basicInvoke Web Service used for Gate, ICU, HPU (alternative to genericInvoke)
inventory-service.wsdl	Billing API's for updating and proposing paid-thru date

Certain API's such as the Gate API are available only through Web Services while other services such as ICU and HPU are available via Web Services as well as any mechanism supported by SNX.

Additionally, N4 provides limited support for the following legacy APIs. New customers are strongly urged to integrate only with the above API's.

legacy WSDL	description
road-service.wsdl	old style gate APIs
orders-service.wsdl	load list update
services-service.wsdl	get Billable Service Event Types
edi-service.wsdl	vessel extract

5.2 Security and Session Management

For each operation, the third party client application submits a synchronous request to the Web Services API, waits for a response from the API, and processes the results. The Web Services APIs treat the third party client application as the authorized user. The APIs can only accept stateless requests from a third-party client application and do not preserve a session across consecutive requests. The client application must include a valid user name and password combination, and topology information with each request.

In This Section

Topology Information.	82
Security Setup.	82

5.2.1 Topology Information

N4 concurrently supports multiple operators within the same instance of the application. This feature requires that all requests that a client application sends to the Web Services APIs include the following topology information:

- **Operator:** Operator ID
- **Complex:** Complex ID - unique for the selected operator
- **Facility:** Facility ID - unique for the selected complex
- **Yard:** Yard ID -unique for the selected facility. May be optional pending the API.

5.2.2 Security Setup

The Web Services APIs use Basic Authentication to provide a security solution for the client application. To implement basic authentication:

- The client and server must be in a LAN.
- The client application must use Secure Sockets Layer (SSL) encrypted connections with 128-bit encryption for all requests it sends to the API.
- The developers must add a user name and password combination to the SOAP headers.
- The client toolkit must support basic authentication and must encrypt the user name and password combination before sending it to the API.

All Java and .Net Web Services toolkits support basic authentication. This is the preferred choice for client authentication.

5.3 Argo Generic and Basic Web Service API

In This Section

Argo Generic and Basic Web Service API Implementation.	84
Argo Generic Web Service API Format.	84

5.3.1 Argo Generic and Basic Web Service API Implementation

The Argo Generic Web Service API is a single entry-point webservice that supports multiple operations depending on the content of an xml message passed to it. The generic webservice is implemented in 2 forms:

- `argoservice/genericInvoke` which uses complex data types
- `argobasicservice/basicInvoke` a more simplified that uses only String datatypes.

Each of the implementations has its own WSDL with the single entry point declared. The parameters of the call for each are the same i.e. scope and xml message payload, though the datatypes differ. The specific action being performed and its data are specified in the xml message payload. Outbound (from N4's perspective) web-services, i.e. those which are implemented by external systems which are intended to be called from N4 should implement the `argobasicservice/basicInvoke` WSDL.

Using this approach not only prevents the proliferation of WSDL definitions, but also has the flexibility to provide downward compatibility as the process details evolve. Otherwise, if all the specific arguments were explicitly defined in numerous special purpose WSDL API definitions, the WSDL could not be modified independently by either party. Most Web Service interfaces do not support even the slightest change in a WSDL definition.

Since the details are only in the payload of the API call and response, extensive enhancements can be made to the schema of the payload document without causing backward compatibility issues. Typically attributes and elements can be added to the payload schema without effecting callers that are not providing or expecting these new document entries. When retrieving data from the payload the reader should anticipate that some values may not even be present if the caller is using a previous version of the payload schema.

5.3.2 Argo Generic Web Service API Format

All requests via the Argo Web Service API have the same format:

- The scope of the request, i.e. the terminal operator, complex, facility and yard
- User credentials
- Preferred language for returned messages
- XML payload for the specific application API being invoked

The response also has a common format:

- Any messages that have been generated during the call (info, warning or severe)
- A single status code indicating the worst level of any of the messages
- Results from the specific operation

Callers are strongly discouraged from acting upon any specific error message as the errors are subject to change. Instead, the caller should only be concerned if the API failed or succeeded and treat the messages as free format text to display and/or log.

5.4 Error Handling

For each request, the Web Services APIs return an XML document that describes the status of the request and contains a list of messages to describe the problems the request encountered, if any.

The client should only detect that at least one fatal error occurred and not be concerned with the specific errors that the API returns. Navis discourages constructing logic based on individual error keys.

In This Section

General Status.	86
Status Messages.	86

5.4.1 General Status

The Web Services API will return a general status value to the client application indicating the overall status of the API call. The following values indicate that the API returned:

- **Status 3:** At least one severe error
- **Status 2:** At least one warning, but no severe errors
- **Status 1:** At least one information only message, but no severe errors or even warnings
- **Status 0:** No messages and the API call was entirely successful

5.4.2 Status Messages

A Web Services API includes the following three components for each status message in the list of messages that it returns to the third-party client application:

- **Status:** 1, 2, or 3 indicating the severity of the message
- **Key:** Unique key assigned to the error message
- **Message:** Actual text of the error

5.5 Development Guide

You are recommended to read this section first before going into the example client integrations.

Navis will not support any Navis Web Services extensions that are built or maintained by non-Navis resources. Navis Partners are external to Navis and therefore any extensions they may create or maintain for these web services are also not supported.

In This Section

API Specifications.	88
Testing the Generic Services.	88
Invoking Web Services Programmatically.	90
Related Topics.	92
Triggering Groovy via Web Services.	92

5.5.1 API Specifications

```
GenericInvokeResponseWsType response = argoservice.genericInvoke
(ScopeCoordinateIdsWsType scopeCoordinates, String xmlPayload)
```

The `argoservice.genericInvoke` call uses the complex types `GenericInvokeResponseWsType` and `ScopeCoordinateIdsWsType` that are defined in the `argoservice.wsdl`. Messages, status and the xml response from the specific action are contained within `GenericInvokeResponseWsType`.

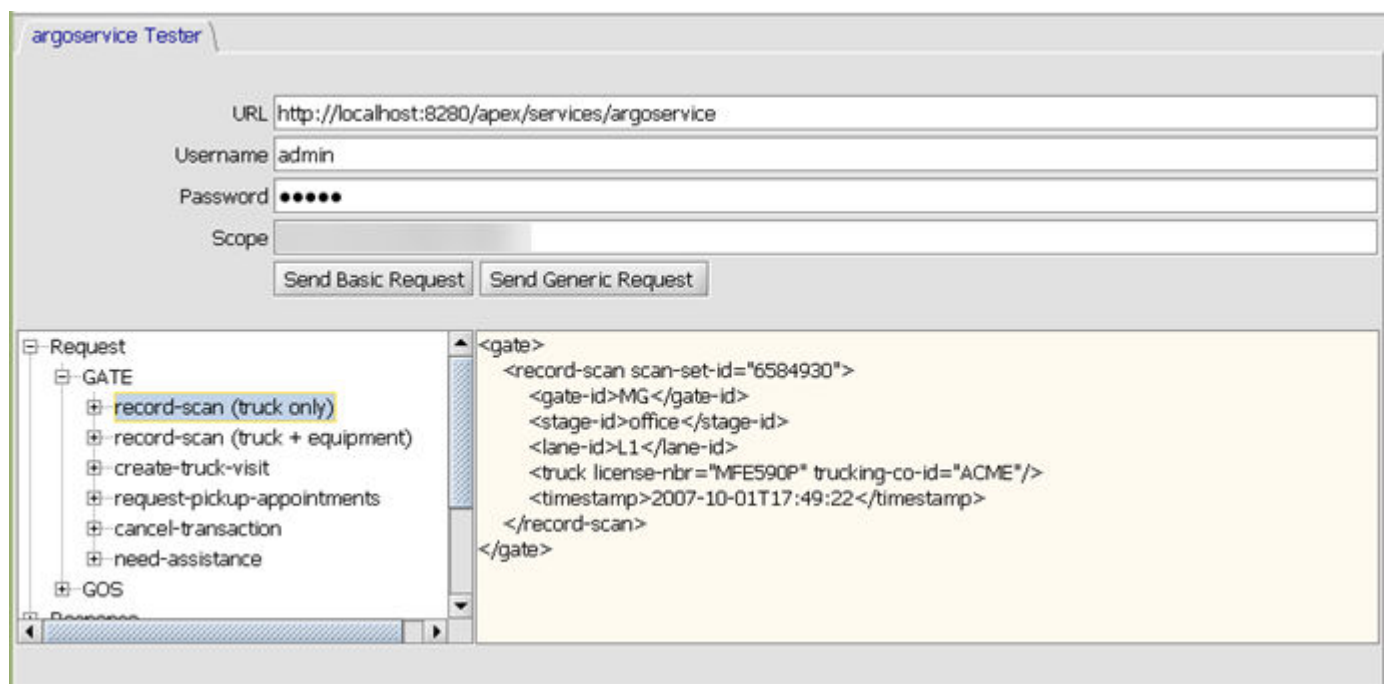
```
String response = argobasicservice.basicInvoke (String scopeCoordinates, String
xmlPayload)
```

The `argobasicservice.basicInvoke` call uses basic String datatypes only. The scope coordinates are specified as a single string with slash (/) separators between the scope components ie operator/complex/facility/yard. The response is an xml message containing the status, messages and the specific response from the action performed.

5.5.2 Testing the Generic Services

The Debug → `argoservice` tester feature in N4 allows both the `argoService.genericinvoke` and `argobasicService.basicinvoke` API's to be tested from within the application.

This allows a simple means of posting test xml payloads without the additional reliance on the client application that would otherwise be calling the webservice.



The fields at the top (URL, Username, Password, Scope) are used to specify the URL of the service to be tested, the authentication details and the scope (operator/complex/facility/yard). If the application is being run with an application server then the URL will default to the URL of the current application. The URL can either be amended to point to an external application which may or may not be another N4 instance or it can be left blank to perform an internal test. An internal test means that the request will go directly within the application, ie not via the webservice protocol and will use the credentials of the user currently logged in.

The tree on the left allows for entering of the request details and viewing the response. Clicking the Request node

allows the request xml message to be entered in the box on the right. Below the Request node are a set of sample messages for each service that can be used as is or edited as desired.

The buttons allow the request to be posted to either `argoservice.genericInvoke` or `argobasicservice.basicInvoke`.

Depending on the service requested the response node of the tree will be populated differently.

For `argobasicservice.basicInvoke` the response xml String will be displayed on the Response node. The example below shows a successful call with response details and no messages (the `<messages>` tag is empty).

The screenshot shows the 'argoservice Tester' application. At the top, there are input fields for 'URL' (http://localhost:8280/apex/services/argobasicservice), 'Username' (admin), 'Password' (masked with dots), and 'Scope'. Below these are two buttons: 'Send Basic Request' and 'Send Generic Request'. The 'Send Basic Request' button is highlighted with a yellow border. Below the buttons is a tree view on the left and a text area on the right. The tree view has a root node 'Request' which is expanded to show a 'GATE' node. The 'GATE' node is expanded to show several sub-nodes: 'record-scan (truck only)', 'record-scan (truck + equipment)', 'create-truck-visit', 'request-pickup-appointments', 'cancel-transaction', 'need-assistance', 'GOS', and 'Response'. The 'Response' node is selected and highlighted in blue. The text area on the right displays the XML response for the 'create-truck-visit' service. The XML is as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<argoResponse status="OK">
  <messages />
  <gate-response>
    <create-truck-visit-response>
      <truck-visit tv-key="2287" gos-tv-key="2287" next-stage-id="office" bat-nbr="F124" />
    </create-truck-visit-response>
  </gate-response>
</argoResponse>
```

For `argoservice.genericInvoke` the components of the complex response type are displayed in nodes below the response node.

The screenshot shows the 'argoservice Tester' window. At the top, there are input fields for 'URL' (http://localhost:8280/apex/services/argoservice), 'Username' (admin), 'Password' (masked with dots), and 'Scope'. Below these are two buttons: 'Send Basic Request' and 'Send Generic Request'. The main area is divided into two panes. The left pane shows a tree view of the request and response structure. The right pane displays the XML response.

Request Tree View:

- Request
 - GATE
 - record-scan (truck only)
 - record-scan (truck + equipment)
 - create-truck-visit
 - request-pickup-appointments
 - cancel-transaction
 - need-assistance
 - GOS
- Response
 - Results
 - Result[0] (highlighted)
 - Status
 - Payload

XML Response:

```
<gate-response> <create-truck-visit-response> <truck-visit tv-key="2287" gos-tv-key="2287" next-stage-id="office" bat-nbr="F124" /> </create-truck-visit-response> </gate-response>
```

Results such as an xml payload response are shown below the Results node for a successful call. The Status node shows the overall status of the request and any messages that have been returned. The Payload node shows an xml representation of the messages.

5.5.3 Invoking Web Services Programmatically

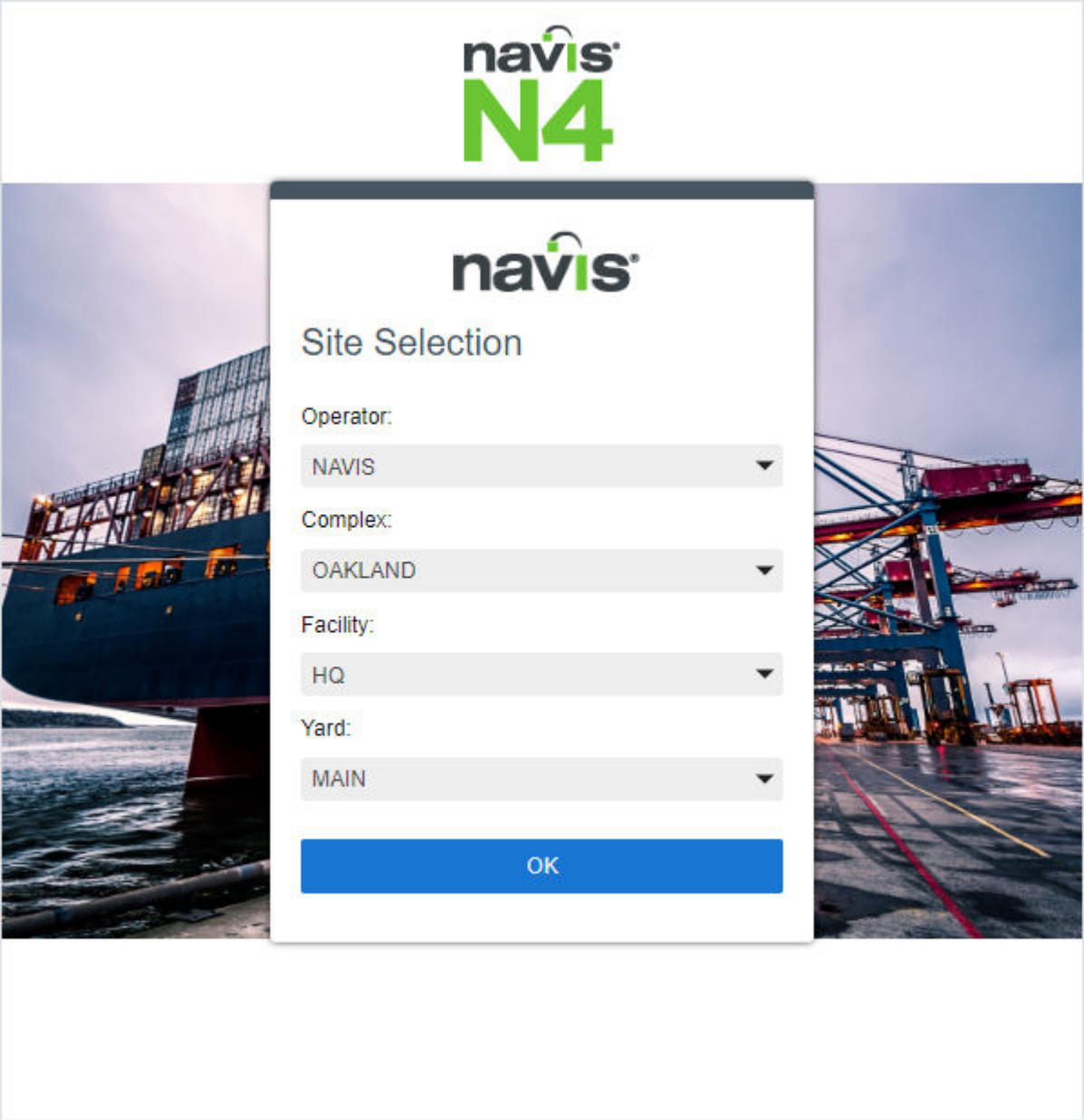
To invoke a N4 Web Service programmatically will require the following generalized steps and will vary widely depending on which platform is used.

To invoke a N4 Web Service programmatically

1. Deploy an instance of N4.
2. Make sure the target service is available by opening your browser to the services page which is found at <http://yourtestinstance/apex/services> which may be something like this:
<http://localhost:8280/apex/services>
3. Create a user with appropriate privileges to call the web service. Having a separate user for web services allows for better auditing and security configuration.

For instance, if you are calling the HPU API, this user must have the roles containing privileges to grant and release holds and permissions.
4. Generate the client stubs and artifact classes from the N4 WSDL of the service you intend to use. You may do this explicitly or in the case of .Net, do this simply by registering a Web Reference.

5. Write the code to invoke the service. This code may eventually be in a stand-alone client or part of another deployed servlet application but you are recommended to write your code first as a stand-alone client for easier development.
 - a. Create **Scope Coordinates** type. The easiest way to find out which ID's you need is to log into a N4 application which will present you with the IDs in the **Site Selection** form.



- b. Attach authentication credentials to the request header.
- c. Create the payload.
- d. Invoke.
- e. Look for the status in the `commonResponse` (`argoservice.genericInvoke`) or in the xml response (`argobasicservice.basicInvoke`).

As mentioned above, each request is stateless and hence must be authenticated independently each time.

If you are making an ICU/HPU request and are experiencing errors, you are recommended to use the SNX Importer to manually import the payload into the system.

5.5.4 Related Topics

You may also review:

- SNX
- Web Services vs. JMS

5.5.5 Triggering Groovy via Web Services

See Groovy Code Injection.

5.6 Argo Basic Service Client Example for .NET (ArgoServiceTester)

This describes a sample application built in C# that accesses the Argo Basic Webservice, and is the C# equivalent of the ArgoServiceTester in N4. The first section describes building the basic screen components and the second calling the webservice. If you are just interested in calling the the webservice, you can skip directly to section 2.

The example described was built with Microsoft Visual c# Express 2010 that you can download for free from Microsoft.

 The source code and the complete project files are included in the attachments.

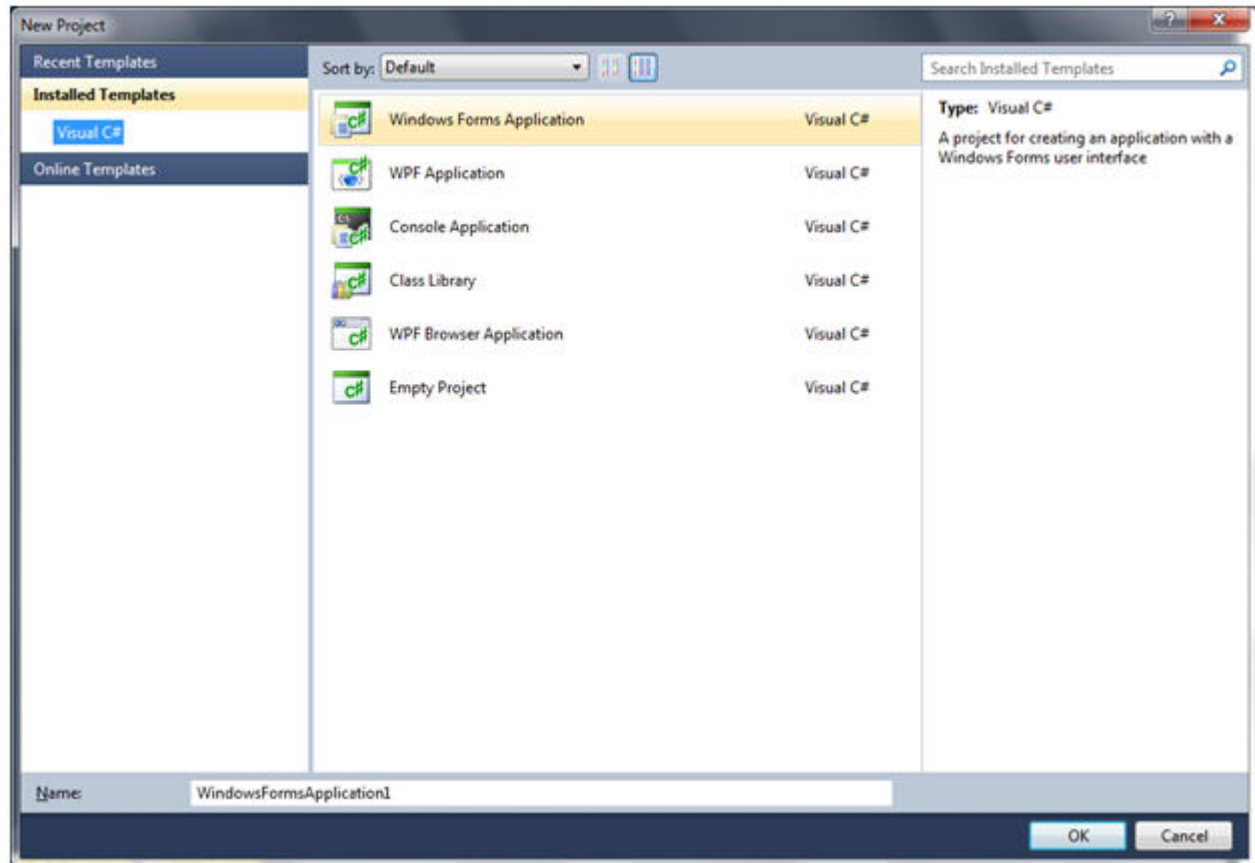
In This Section

ArgoServiceTester skeleton application.	94
Calling the Argo Basic Webservice.	99
Java Client Integration Example.	105

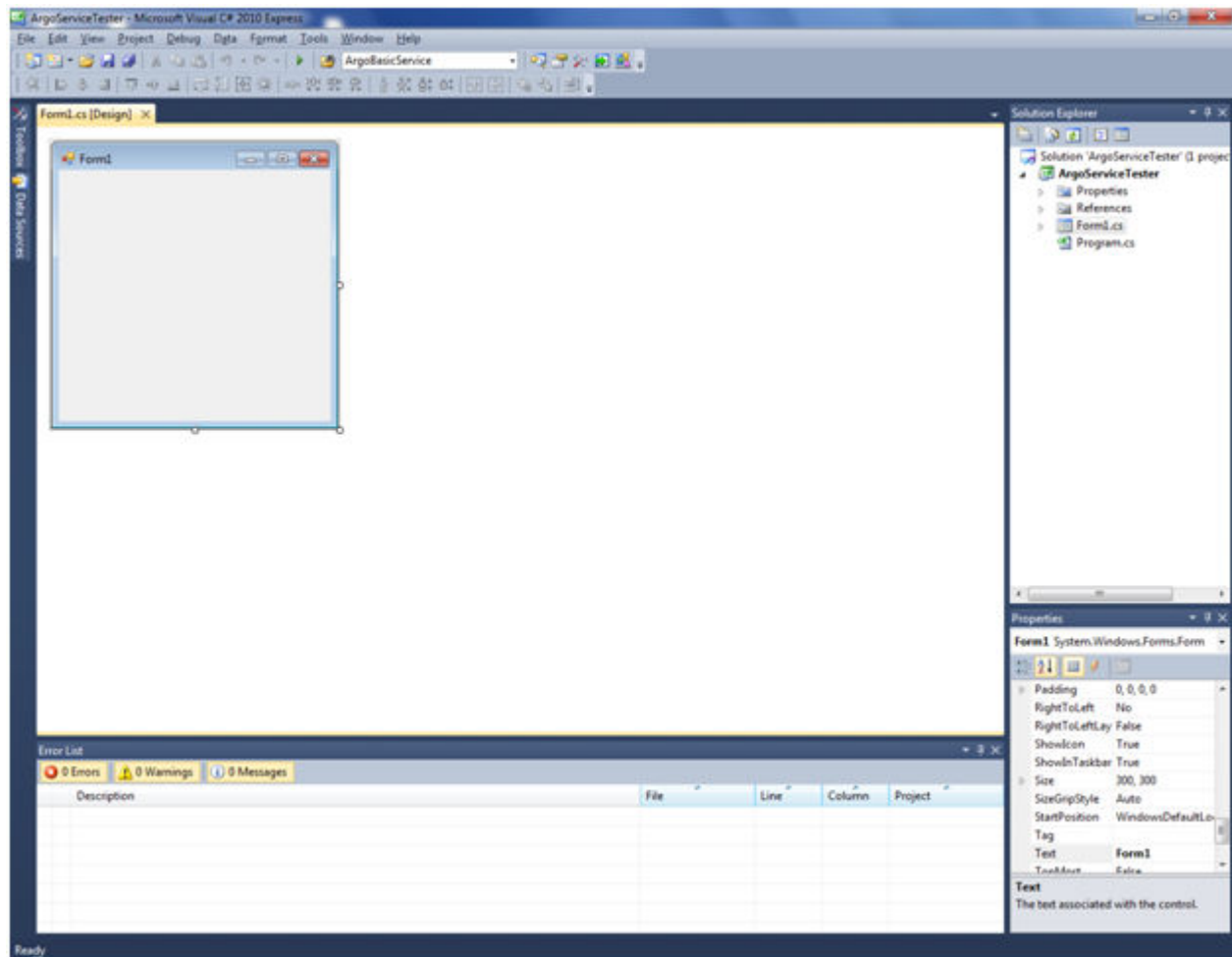
5.6.1 ArgoServiceTester skeleton application

This section covers the creation of the screen and performs a dummy test action on the clicking of the button.

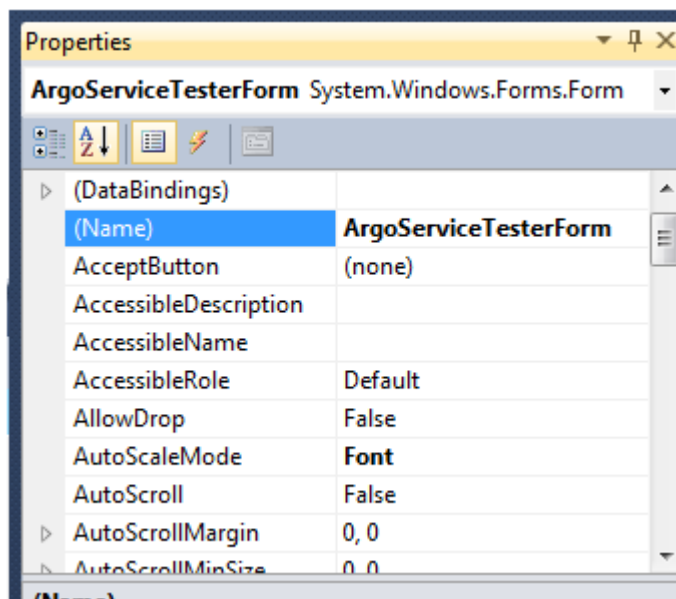
1. Select **New Project** and pick **Windows Forms Application**.



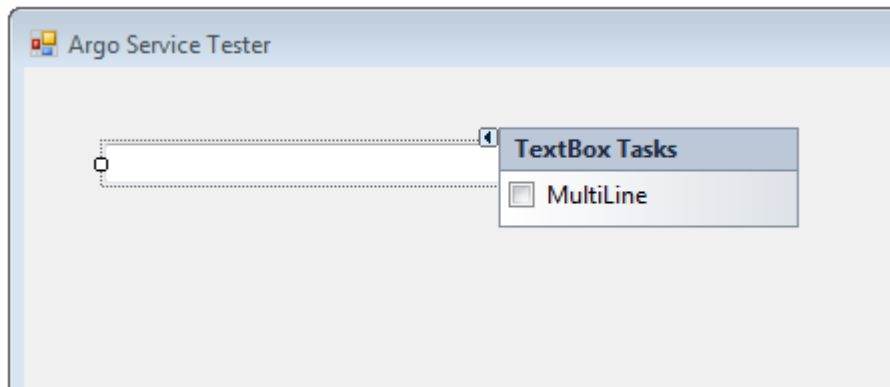
The following screen appears:



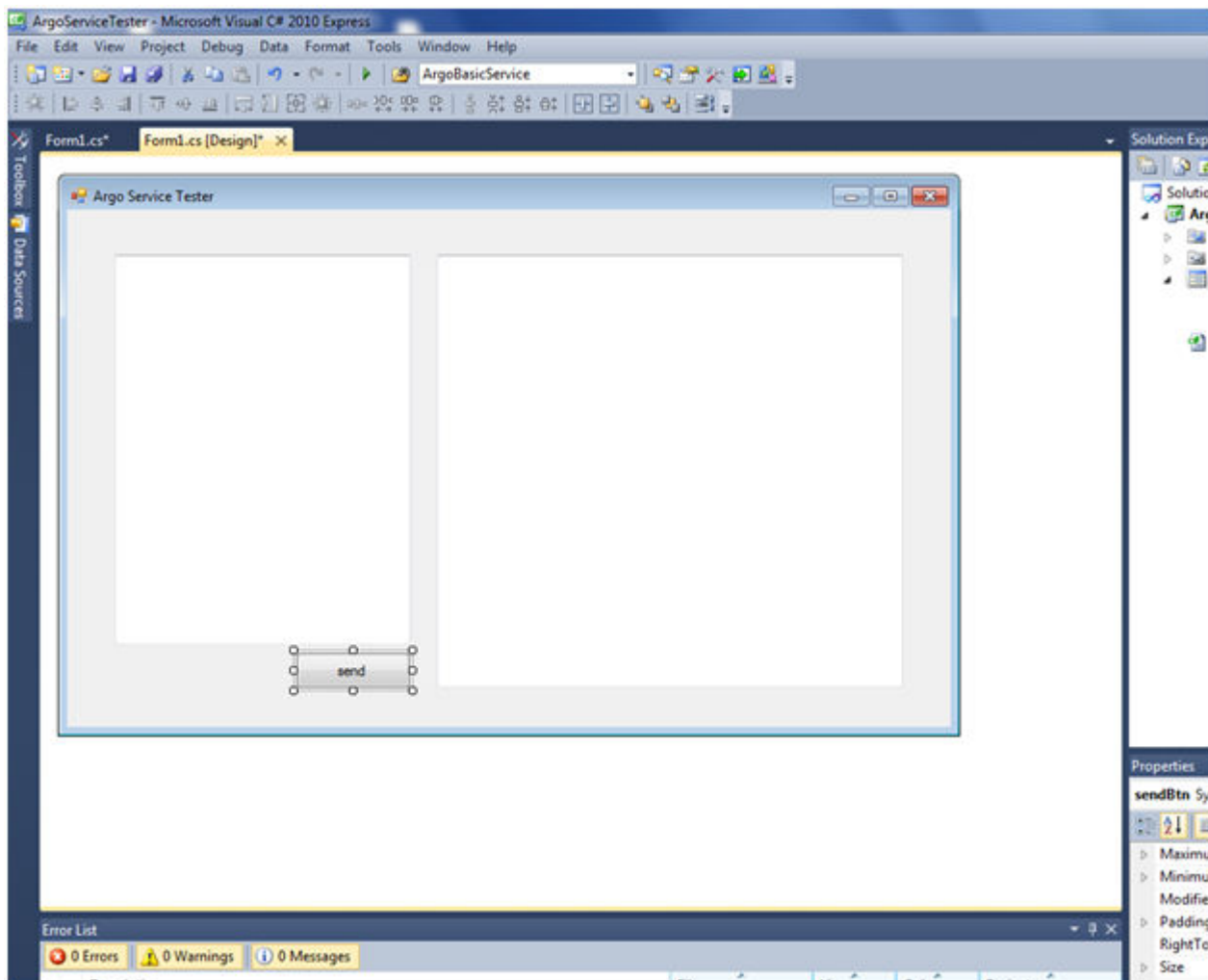
2. Save the project as **ArgoServiceTester**.
3. Right click on Form1 in the designer and on the properties sheet bottom right set Name to **ArgoServiceTesterForm** and Text to **Argo Service Tester**.
4. Right-click on **Form1.cs** in the **Solution Explorer** top right and rename it to **ArgoServiceTesterForm.cs**.



- From the **Toolbox**, add 2 text boxes – click the arrow to make them multi-line. Toolbox shows at the left of the designer. You can pick the toolbox at the top also (Standard Menu).



- Right-click each text box and set the (Name) properties to *requestXml* and *responseXml*. Place them side by side and expand them to make them bigger.
- Add a button, right click and set the (Name) property to *sendBtn* and the Text to *send*.



- Right-click the button and select **View Code**.

```
using System;
using System.Collections.Generic;
```

```
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace WindowsFormsApplication1
{
    public partial class ArgoServiceTesterForm : Form
    {
        public ArgoServiceTesterForm()
        {
            InitializeComponent();
        }
    }
}
```

9. Now we add an event to detect the button being clicked.

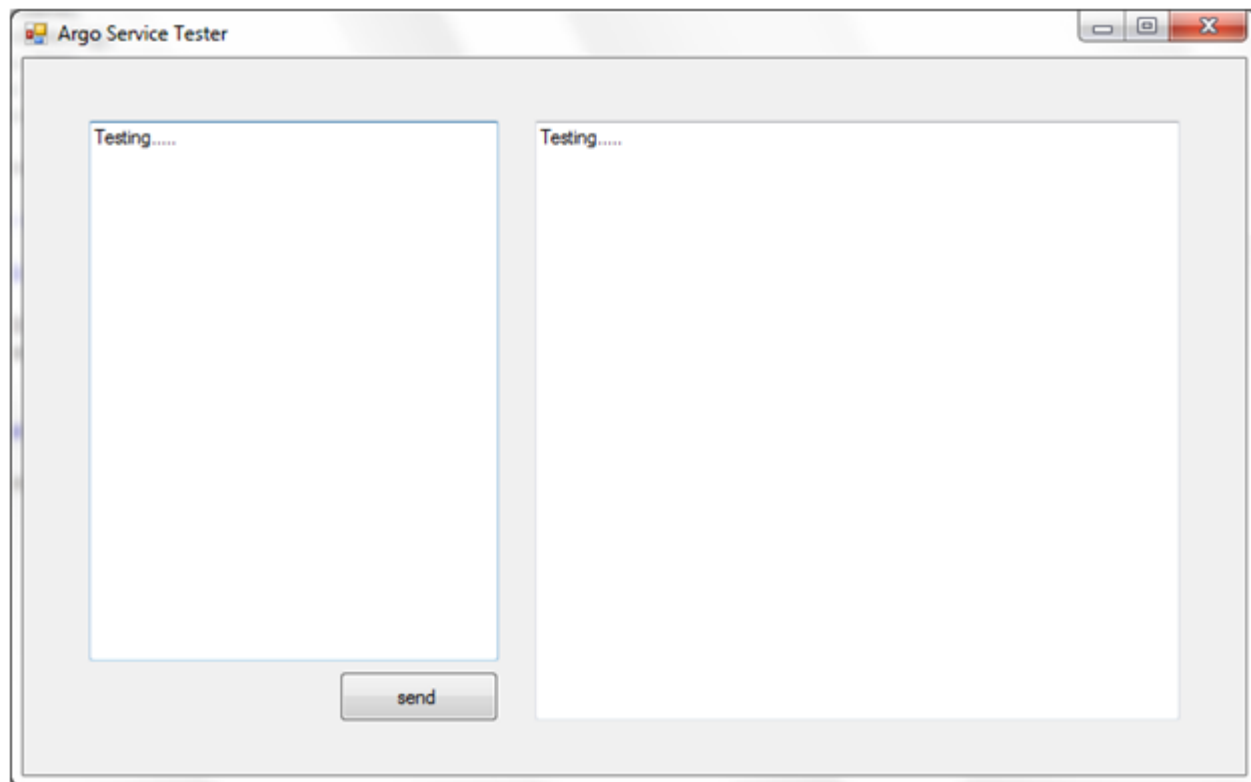
```
namespace WindowsFormsApplication1
{
    public partial class ArgoServiceTesterForm : Form
    {
        public ArgoServiceTesterForm()
        {
            InitializeComponent();
            sendBtn.Click += new EventHandler(this.sendBtn_Click);
        }
        private void sendBtn_Click(object sender, EventArgs e)
        {
            responseXml.Text = requestXml.Text;
        }
    }
}
```

```
namespace WindowsFormsApplication1
```

The line `sendBtn.Click += new EventHandler(this.sendBtn_Click);` registers the method `sendBtn_Click()` as the handler to be executed when the button is clicked.

For the moment we have a dummy line of code to copy the request field to the response field when the button is clicked.

10. Run the application.

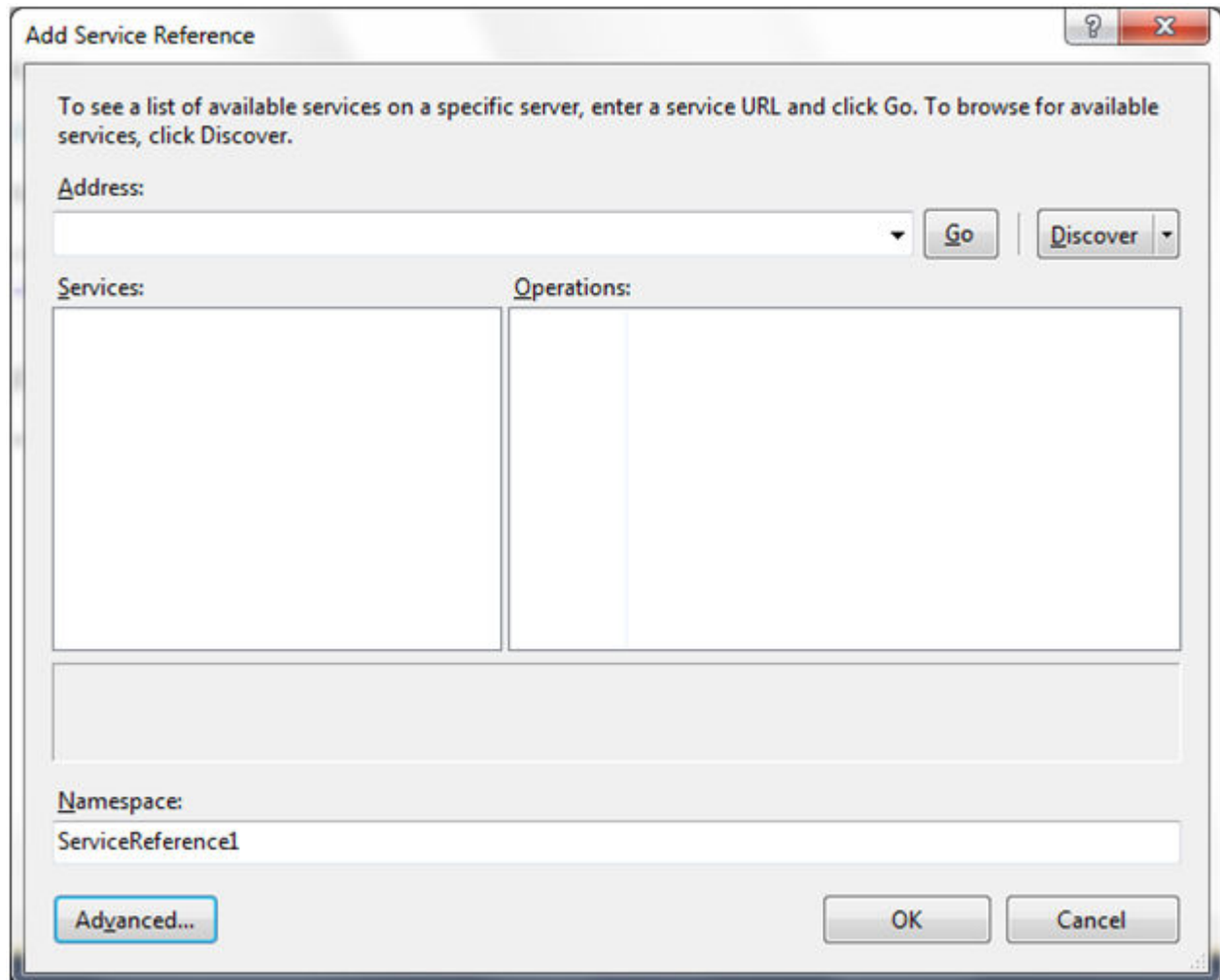


11. Enter something in the request field, click send and it should appear in the response field.

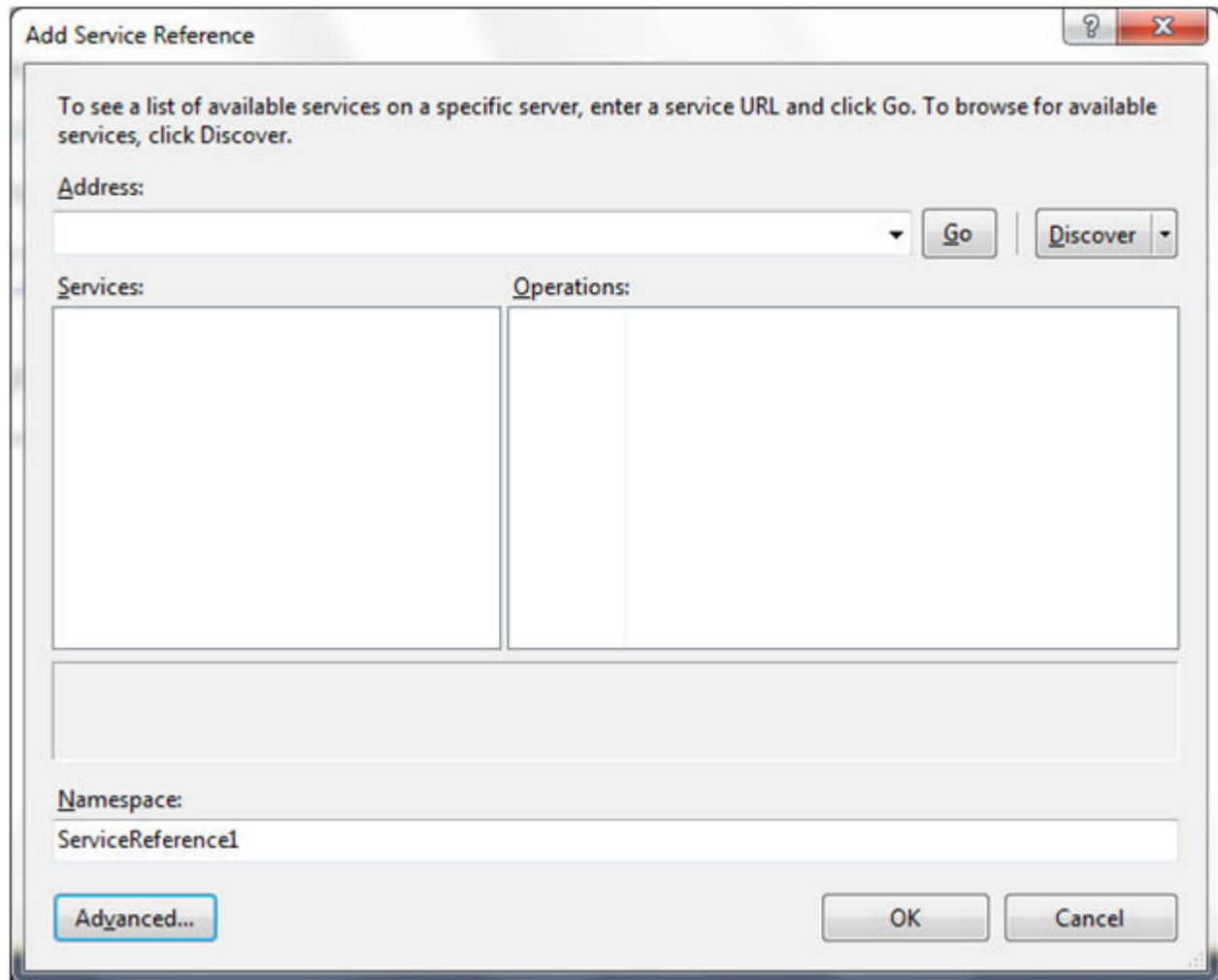
5.6.2 Calling the Argo Basic Webservice

To call the Argo Basic Webservice:

1. Select **Add Service Reference** from the **Project** menu.



2. Click on **Advanced**.



3. Click **Add Web Reference**.



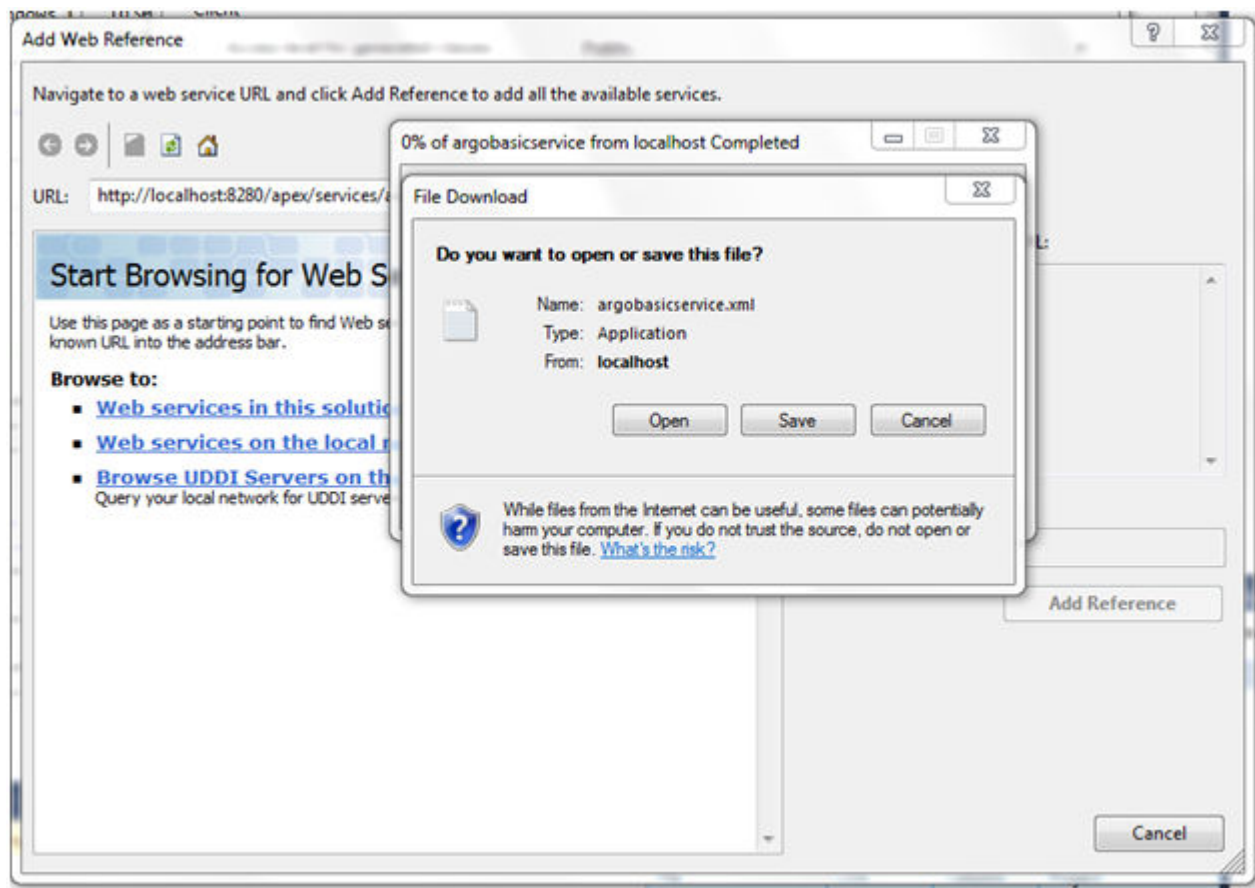
This step is necessary in order to create a webservice reference that will be compatible with the authentication mode used in N4.

4. In the url field enter the url of the `argobasicservice wsdl` on the N4 server:

`http://N4server:9080/apex/services/argobasicservice?wsdl`

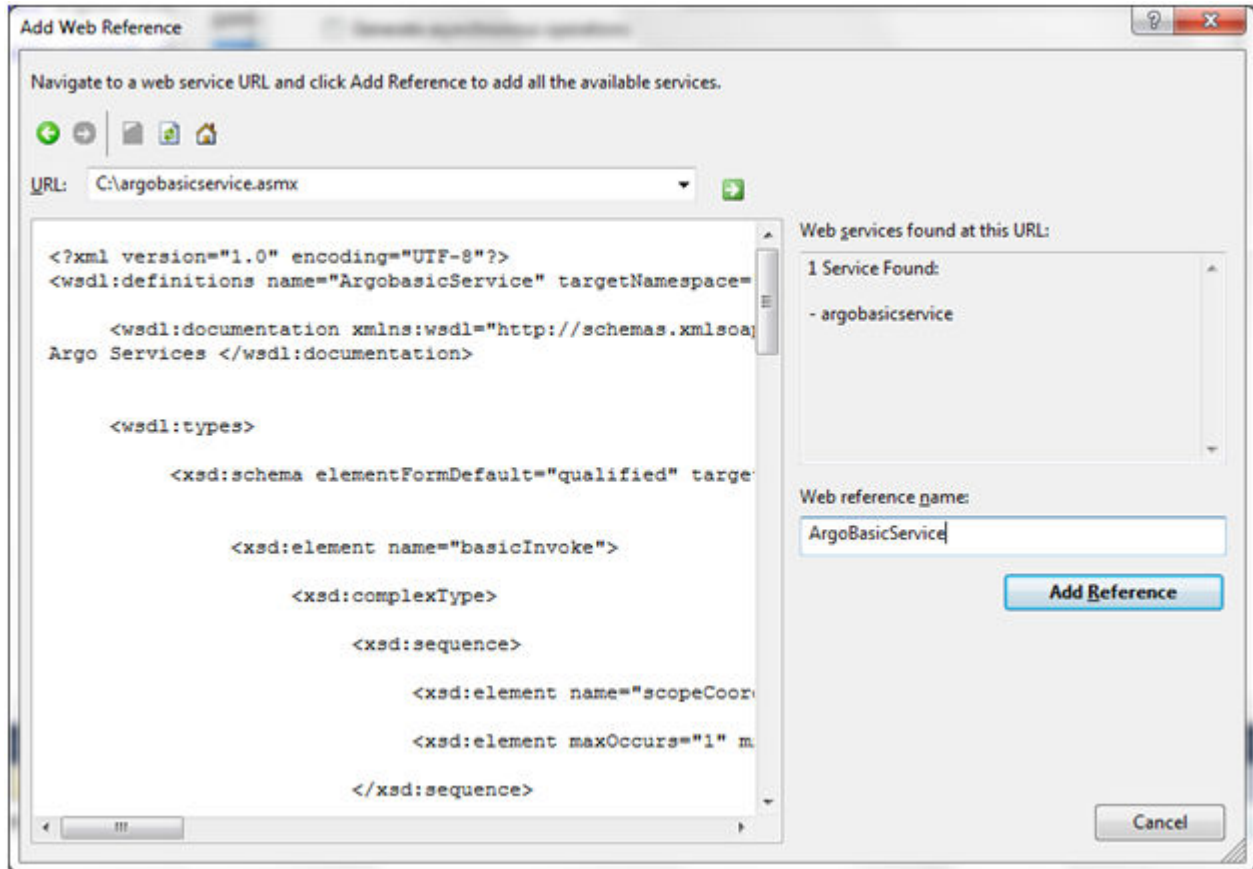
5. Click the arrow.

You may be asked to save the file `argobasicservice.xml`. If so, click save and save it in some temporary location as file type `.asmx` so `argobasicservice.asmx`.



6. Go back to the URL field, enter the path to the asmx file you just saved eg c:\argobasicservice.asmx and click the arrow again.

7. The service should now be found and the **Add Reference** button enabled.



8. Enter ArgoBasicService in the Web reference name field and click Add Reference.
9. Now that you have added the webservice reference, you need to go back to the button handling code and change it to call the webservice.
10. You need to add 3 sections:
- Variables for the username, password and scope coordinates.
 - A new method basicInvoke() to call the webservice.
 - A new class BasicAuthenticationWebservice that will supplement the built-in definition to allow for basic authentication via http headers.
11. Then you need to change the code in `sendBtn_Click` to call `basicInvoke()`.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace WindowsFormsApplication1
{
    public partial class ArgoServiceTesterForm : Form
    {
        String _username = "admin";
        String _password = "admin";
```

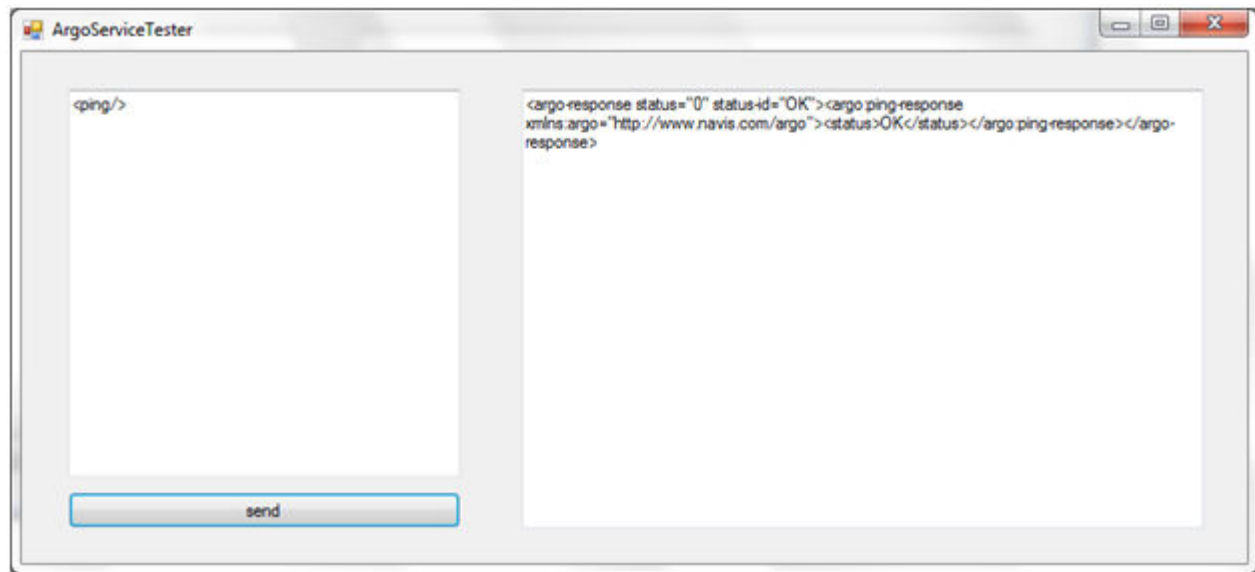
```

String _scopeCoordinateIds = "CTW/DEWVN/CTW/CTW1";
public ArgoServiceTesterForm()
{
    InitializeComponent();
    sendBtn.Click += new EventHandler(this.sendBtn_Click);
}
private void sendBtn_Click(object sender, EventArgs e)
{
    responseXml.Text = basicInvoke(_scopeCoordinateIds, requestXml.Text);
}
private String basicInvoke(String inScopeCoordinateIds, String inRequestXml)
{
    ArgoBasicService.basicInvoke wsParams = new ArgoBasicService.basicInvoke();
    wsParams.scopeCoordinateIds = _scopeCoordinateIds;
    wsParams.xmlDoc = inRequestXml;
    BasicAuthenticationWebservice ws = new
BasicAuthenticationWebservice(_username, _password);
    ws.Timeout = -1;    // disable the timeout
    ArgoBasicService.basicInvokeResponse wsResponse = ws.basicInvoke(wsParams);
    return wsResponse.basicInvokeResponse1;
}
/*
 * This class extends the generated webservice to support basic authentication
 * via the http headers
 */
class BasicAuthenticationWebservice : ArgoBasicService.ArgobasicService
{
    private String _b64Credentials;
    protected override System.Net.WebRequest GetWebRequest(Uri uri)
    {
        System.Net.HttpWebRequest httpRequest =
(System.Net.HttpWebRequest)base.GetWebRequest(uri);
        // set the authentication in the http header
        httpRequest.Headers.Add("Authorization", "Basic " + _b64Credentials);
        return (System.Net.WebRequest)httpRequest;
    }
    public BasicAuthenticationWebservice(String inUsername, String inPassword)
    {
        byte[] bCredentials = Encoding.ASCII.GetBytes(inUsername + ":" + inPassword);
        _b64Credentials = Convert.ToBase64String(bCredentials);
    }
}
}

```

Now when the send button is clicked the contents of the requestXml box will be sent to N4 and the results shown in the responseXml box.

The first message to try would be <ping/>, which should give the response as below.



Various other messages are available in the samples provided with the N4 version of the Argo Service Tester.

5.6.3 Java Client Integration Example

In this Section:

- How to Locate the Generic ArgoWebservice in java code. 106
- How Client Invokes the Generic Service. 106
- Sample Source Code of a Standalone (Java) Webservice Client. 107
- Steps to run Java standalone Webservice Client. 109

5.6.3.1 How to Locate the Generic ArgoWebservice in java code

The following code snippet shows what a typical Java web service client will need do to locate the service before invoking the service. Please note that these example steps are shown for only Java client using Apache's AXIS constructs. For other implementations on the client side, please consult appropriate documentation.

Following code snippet shows how a client will access ArgoWebservice that is deployed on the Tomcat running on localhost at port 8280. The authentication step is using the default admin user "admin" with password "admin". This will need to be changed if different credentials are setup on the server.

```
private static final String ARGO_SERVICE_URL = "http://localhost:8280/apex/services/argoservice";
private ArgoServicePort locateService() throws javax.xml.rpc.ServiceException,
MalformedURLException {
    ArgoServiceLocator service = new ArgoServiceLocator();
    ArgoServicePort port = service.getArgoServicePort(new URL(ARGO_SERVICE_URL));
    Stub stub = (Stub) port;
    stub._setProperty(Stub.USERNAME_PROPERTY, "admin");
    stub._setProperty(Stub.PASSWORD_PROPERTY, "admin");
    return port;
}
```

It is assumed that the client program has access to the right set of libraries (jars) in its CLASSPATH.

Please look at the end of this document for complete code listing of the client program and also for the JARs that AXIS client needs.

The above method is used in the following section to locate the service before invoking the method on the service. Please note that the steps mentioned here are same as for any other Navis webservice (e.g Inventory).

5.6.3.2 How Client Invokes the Generic Service

Following code snippet is showing the client side of the picture. As mentioned, to work with this web service client needs to package its request as an XML document. Client creates a serialized XML file that it passes to the server.

The server side creates a DOM tree from this XML, looks at the root tag and then delegates the actual business processing to the appropriate target bean. Assuming client wants to update one of the Unit's flex fields for the unit XXXU1234567 in the database, the client will need to have the following XML:

```
<icu>
<units>
  <unit-identity id="XXXU1234567" type=" CONTAINERIZED">
  </unit-identity>
</units>
<properties>
  <property tag="UnitFlexString01" value="UnitFlexFieldNewValue"/>
</properties>
<event id="MYEVENT" note="some note"/>
</icu>
```

Client will serialize the XML into string to send the above XML to the Incremental Update Service (ICU) as shown below :

```
ArgoServicePort port = locateService();
String icuXMLStr = "<icu>\n" +
```

```

    "<units>\n" +
    "    <unit-identity id=\"XXXU1234567\" type=\"CONTAINERIZED\"/>\n" +
    "</units>\n" +
    "<properties>\n" +
    "    <property tag=\"UnitFlexString01\" value=\"UnitFlexFieldNewValue\"/>\n" +
    "</properties>\n" +
    "<event id=\"MYEVENT\" user-id=\"icuUser\" time-event-applied=
\"2007-09-23T11:23:24\" note=\"some note\"/>\n"
    + "</icu>";
    GenericInvokeResponseWsType response = port.genericInvoke(getScopeCoordinateIds(),
    icuXMLStr);

```

It is the responsibility of the client(caller) to appropriately serialize It's XML document properly.

Please note that we are not eliminating the use of typed web services, this is just another way to avoid repetition of web service wiring/setup code where appropriate.

5.6.3.3 Sample Source Code of a Standalone (Java) Webservice Client

```

package com.navis.apex.business.webservice;
import javax.xml.rpc.Stub;
import java.net.URL;
import com.navis.argo.webservice.types.v1_0.GenericInvokeResponseWsType;
import com.navis.argo.webservice.types.v1_0.MessageCollectorType;
import com.navis.argo.webservice.types.v1_0.MessageType;
import com.navis.argo.webservice.types.v1_0.ResponseType;
import com.navis.argo.webservice.types.v1_0.ScopeCoordinateIdsWsType;
import com.navis.www.services.argo.service.ArgoServiceLocator;
import com.navis.www.services.argo.service.ArgoServicePort;
/**
 * Tests for generic argo webservice.
 */
public class GenericWebserviceSimpleInvocationTest {
    public GenericWebserviceSimpleInvocationTest() {
    }
    public static void main(String[] args) {
        new GenericWebserviceSimpleInvocationTest().callGenericWebservice();
    }
    public void callGenericWebservice() {
        try {
            // Specify the Operator, Complex, Facility, and Yard
            ScopeCoordinateIdsWsType scope = new ScopeCoordinateIdsWsType();
            scope.setOperatorId("OPR1");
            scope.setComplexId("CPX11");
            scope.setFacilityId("FCY111");
            scope.setYardId("YRD1111");
            // Identify the Web Services host
            ArgoServiceLocator service = new ArgoServiceLocator();
            ArgoServicePort port = service.getArgoServicePort(new
URL(ARGO_SERVICE_URL));
            Stub stub = (Stub) port;
            // Specify the User ID and the Password
            stub._setProperty(Stub.USERNAME_PROPERTY, "admin");
            stub._setProperty(Stub.PASSWORD_PROPERTY, "admin");

```

```

        //formulate the request in the form of an XML. The following XML has a
root tag of ICU which means
        // the call will be routed to the Incremental Container Update service.
        String icuXMLStr = "<icu>\n" +
            "    <units>\n" +
            "        <unit-identity id=\"XXXU1234567\" type=\"CONTAINERIZED\">\n" +
\n" +
            "            </unit-identity>\n" +
            "        </units>\n" +
            "    <properties>\n" +
            "        <property tag=\"UnitFlexString01\" value=\"NewFlexValue\n" +
\n"/>\n" +
            "    </properties>\n" +
            "</icu>";
        // simulate user's invocation of webservice. Note passing of xml string
object.

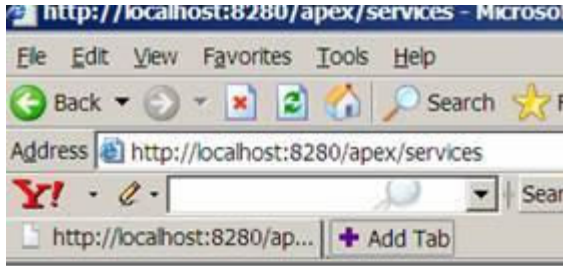
        // Call the Web Service
        GenericInvokeResponseWsType response = port.genericInvoke(scope,
icuXMLStr);
        // Interpret the API response
        ResponseType commonResponse = response.getCommonResponse();
        String status = commonResponse.getStatus();
        if (ERRORS.equals(status)) {
            // Error processing
            MessageCollectorType messageCollector =
                commonResponse.getMessageCollector();
            MessageType[] messages = messageCollector.getMessages();
            for (int i = 0; i < messages.length; i++) {
                MessageType message = messages[i];
                String msgSeverity = message.getSeverityLevel();
                String msgText = message.getMessage();
                // Display each message
            }
        } else { // normal processing
            System.out.println("successfully executed call to generic webservice");
            String xmlResults = response.getResponsePayload();
            if (xmlResults != null) {
                System.out.print("update status from ICU:\n" + xmlResults);
            }
        }
    }
    catch (Throwable throwable) {
        throwable.printStackTrace();
        System.err.println("error while executing webservice ");
    }
}

//Initialize the general status message strings
private static String OK = "0";
private static String INFO = "1";
private static String WARNINGS = "2";
private static String ERRORS = "3";
private static final String ARGO_SERVICE_URL = "http://localhost:8280/apex/
services/argoservice";
}

```


5.6.3.4 Steps to run Java standalone Webservice Client

1. Make sure Generic Argo Service is deployed properly



And now... Some Services

- ordersservice ([wsdl](#))
 - loadListUpdate
- roadservice ([wsdl](#))
 - resolveContainer
 - resolveChassis
 - resolveBooking
 - submitTransaction
 - createTruckVisit
 - readTruckVisit
 - submitStageDone
 - cancelTruckVisit
 - cancelTruckTransaction
 - callupService
 - callupServiceClient
 - deliveryOrderItemLov
 - deliveryOrderLov
 - truckingCompanyLov
- AdminService ([wsdl](#))
 - AdminService
- Version ([wsdl](#))
 - getVersion
- inventoryservice ([wsdl](#))
 - equipmentInquiry
 - updatePaidThruDay
 - proposePaidThruDay
 - genericInvoke
- argoservice ([wsdl](#))
 - genericInvoke

2. Run your client with appropriate Java archives in the CLASSPATH. Most of them you should be able to find in the footprint of N4 in `$N4InstallRoot\tomcat\webapps\apex\WEB-INF\lib`
3. `run_icu_client.bat c:\N4\latest\N4-marine-dev\tools\tomcat-5.5\webapps\apex\WEB-INF\lib`

Where contents of bat file are:

```
java -classpath C:\N4\latest\N4-marine-dev\apex\build\classes;%1\axis.jar;%1\commons-beanutils.jar;%1\commons-codec.jar;
```

```
%1\commons-collections.jar;%1\commons-dbcp.jar;%1\commons-digester.jar;%1\commons-  
discovery.jar;%1\commons-io.jar;  
%1\commons-lang.jar;%1\commons-logging.jar;%1\commons-net-1.0.0.jar;%1\commons-  
pool.jar;%1\commons-validator.jar;%1\dom4j.jar;  
%1\javamail.jar;%1\jaxen.jar;%1\jaxrpc.jar;%1\jcommon.jar;%1\jdom.jar;%1\N4-  
argo.jar;%1\n4-framework.jar;%1\resource.jar;  
%1\saa-j-impl.jar;%1\saa-j.jar;%1\s;%1\wsdl4j.jar;%1\xercesImpl.jar;%1\xml-apis.jar;%1\  
com.Navis.apex.business.webservice.GenericWebserviceSimpleInvocationTest
```

The program will output the following :

```
successfully executed call to generic webservice  
update status from ICU:  
<update-status>  
<document-update status="ACCEPTED"></document-update>  
<entity-updates amount-successful="0" amount-failed="0" amount-not-found="1">  
<unit-update unit-id="XXXU1234567" status="ENTITY_NOT_FOUND">No unit could be found  
that uses the CONTA  
INERIZED equipment with the id XXXU1234567 and that is defined for the declared  
carrier (if carrier is declared).  
</unit\<-  
update>  
</entity-updates>  
</update-status>
```

The classpath mentioned above is not fully optimized, there may be some extraneous jars that can be removed from the classpath.

5.7 Client Web Services Integration for .NET

In this Section:

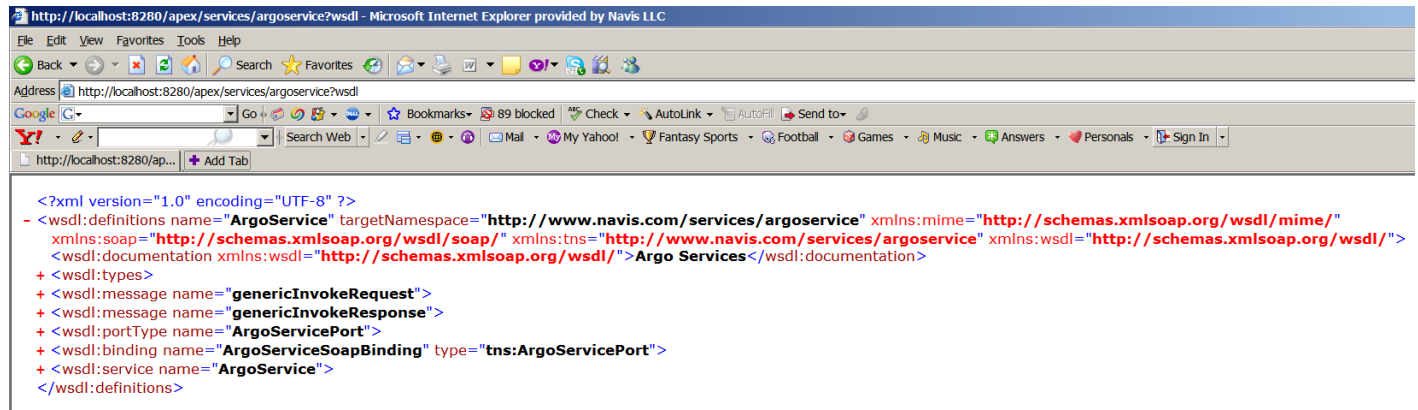
In This Section

N4 Setup and Environment.	112
Setup and Environment for .NET.	112

5.7.1 N4 Setup and Environment

N4 1.4.L (or higher) running JDK 1.4 or 1.5.x is needed to use this feature. Once on the right build, follow N4 installation and deployment instructions for Apex (web) application. Confirm your apex deployment has Generic Web service enabled and deployed by going to "http://localhost:8280/apex/services/" and seeing genericInvoke under argoservice listed. As we will see in the .NET sections below, we need the URL of the target N4 web service's WSDL. This can be done by going to the following URL adjusted with your host and port settings:

http://localhost:8280/apex/services/argoservice?wsdl



5.7.2 Setup and Environment for .NET

Following tools are installed on the client machine for runtime and development purposes:

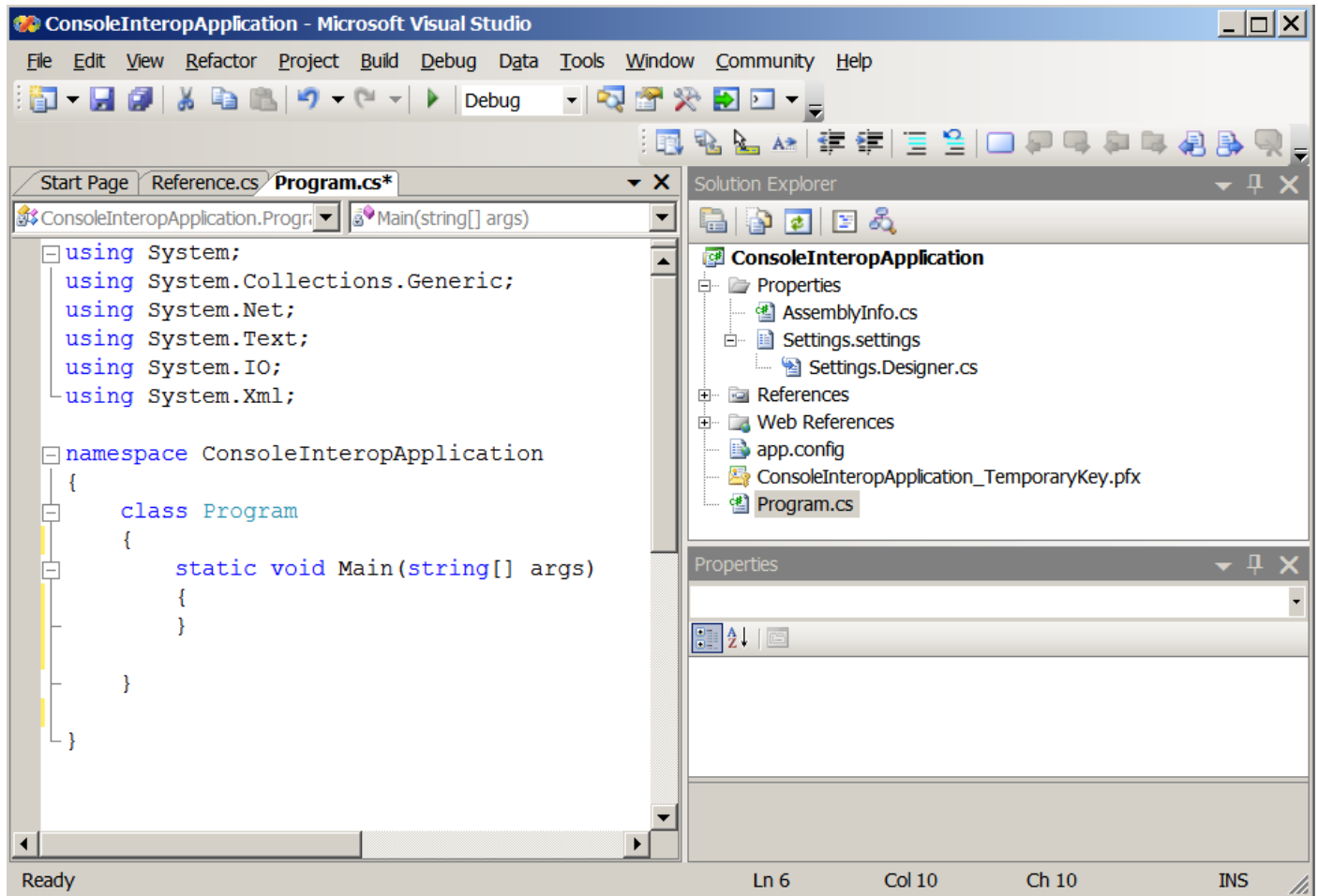
- Microsoft Visual Studio 2005
- Microsoft Visual C# 2005
- Microsoft .NET Framework v2.0 SDK tools

To invoke a deployed Web Service from a .NET client application, we use Microsoft Visual Studio .NET first to create an application add a Web Reference, specifying the WSDL (Web Service Description Language) of the deployed N4 Web Service, as described below. In Microsoft Visual Studio, adding a Web Reference leads to generation of web services client stubs and other code artifacts behind the scene. These generated artifacts are used by the .NET client during runtime.



This document describes one way to invoke a N4 Generic Web Service from a .NET client application. For the most current and detailed information about using Microsoft Visual Studio .NET to invoke Web Services, please consult the [Microsoft documentation](#).

5.7.2.1 Start and use Microsoft Visual Studio .NET to create your application.

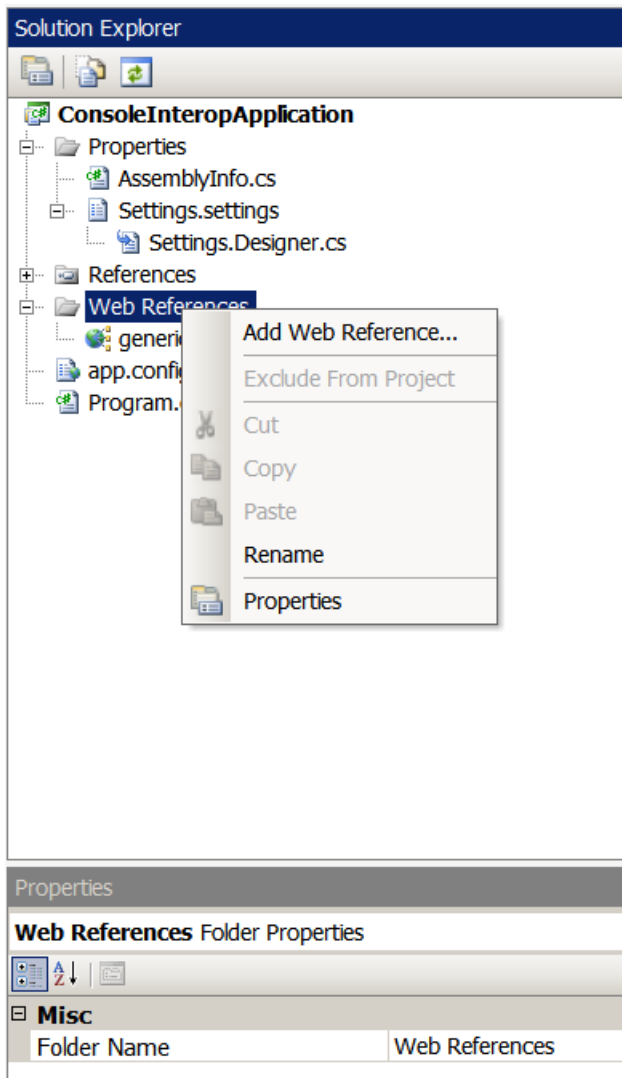


Application shown above is what is created by VS.NET when File --> New --> Project --> Visual C# --> Console Application option is chosen.

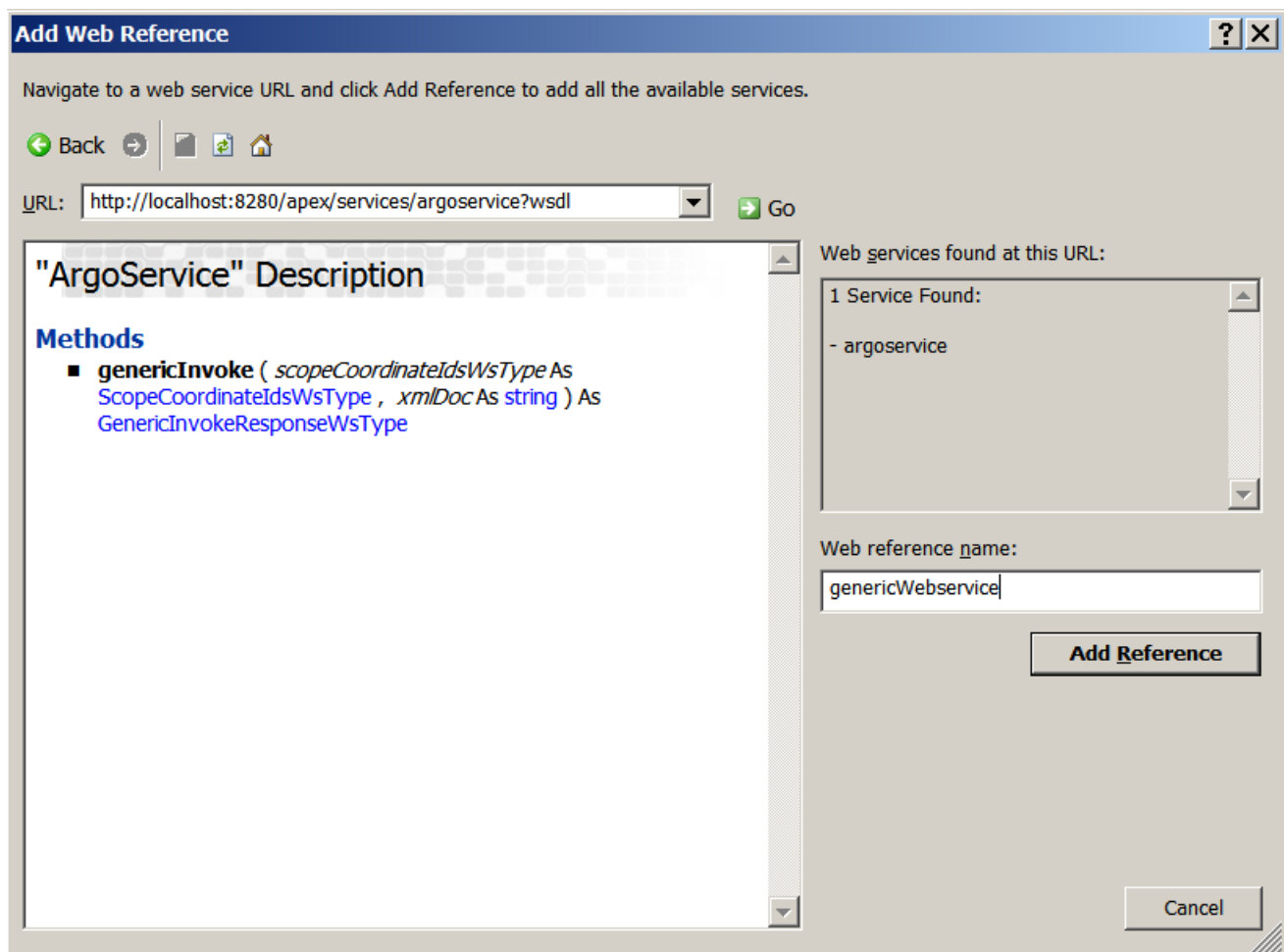
Appropriate logic needs to be added in the above template code for it to talk to the N4 web service. Complete code is pasted in the section below and is also attached with this document.

5.7.2.2 Add Web Reference to N4 Web service in .NET Application

To add web reference for a particular (target) N4 webservice, in the Solution Explorer in the right pane, right-click 'Web Reference' node and choose Add Web Reference.



The Solution Explorer Browser appears. Enter the WSDL of the deployed N4 Web Service in the Solution Explorer Browser. In our case the WDL URL is <http://localhost:8280/apex/services/argoservice?wsdl>. Once you paste the URL and press 'Go' and if the browser accepts the WSDL URL, the Add Reference button becomes active.



Click the Add Reference button. The N4 Web Service appears in the Solution Explorer. In your .NET client application component that will access N4 Web Service, such as a button, add Visual C# code to invoke a particular operation of the Web Service. Visual Studio .NET uses statement completion to help you write this code. The following Visual C# code excerpt shows a simple example:

```
ConsoleInteropApplication.genericWebservice.genericInvokeResponse response =
N4Webservice.genericInvoke(arg);
```

In the example, genericWebservice is the name of the Web Reference you added in preceding steps.

5.7.2.3 Sample .NET C# Client Accessing ICU on N4

Following code snippet is a complete stand-alone .NET (C#) client that invokes N4 ICU Webservice and writes the result on standard out. An equivalent Java client is available.

```
using System;
using System.Collections.Generic;
using System.Net;
using System.Text;
using System.IO;
using System.Xml;
namespace ConsoleInteropApplication
{
    class Program
    {
        private static String OK = "0";
```

```

private static String INFO = "1";
private static String WARNINGS = "2";
private static String ERRORS = "3";
static void Main(string[] args)
{
    genericWebService.genericInvoke arg = new
ConsoleInteropApplication.genericWebService.genericInvoke();
    // try to use one of the test data files that Java (JUnit) clients use.
    // Here we assume test container is pre-created on the server side.
    TextReader textReader = new StreamReader(@"C:\N4\latest\N4-marine-dev\argo\conf
\runtime\testdata\icuTestFlexFields.xml");
    arg.xmlDoc = textReader.ReadToEnd();
    //set the Scope coordinates
    arg.scopeCoordinateIdsWsType = new
ConsoleInteropApplication.genericWebService.ScopeCoordinateIdsWsType();
    arg.scopeCoordinateIdsWsType.operatorId = "OPR1";
    arg.scopeCoordinateIdsWsType.complexId = "CPX11";
    arg.scopeCoordinateIdsWsType.facilityId = "FCY111";
    arg.scopeCoordinateIdsWsType.yardId = "YRD1111";
    // Notes on Authentication
    // MS Documentation on the web and at various places on web search shows that
credentials are passed
    // to .NET based web services using the following code:
    // N4WebService.Credentials = new NetworkCredential("login", "password");
    // However, this approach doesn't end up populating appropriate ( username, and
password) properties
    // in the webservice context on the Java (N4 webservice site). Hence the call
gets authentication failure with
    // this approach.
    // An alternative approach to explicitly set these properties on the .NET client
side seems to do the
    // right thing.
    // following code is needed to pass user/pwd to the server.
    ExtendedGenericWebService N4WebService = new ExtendedGenericWebService();
    byte[] bcred = Encoding.ASCII.GetBytes("admin:admin");
    string b64cred = Convert.ToBase64String(bcred);
    N4WebService.SetRequestHeader("Authorization", "Basic " + b64cred);
    // default of 100 seconds is small for long transactions, -1 disables timeout.
    N4WebService.Timeout = -1;
    // actual call to the business/service method and look into the response objects
for status.
    ConsoleInteropApplication.genericWebService.genericInvokeResponse response =
N4WebService.genericInvoke(arg);
    ConsoleInteropApplication.genericWebService.ResponseType commonResponse =
response.genericInvokeResponse1.commonResponse;
    String status = commonResponse.Status;
    Console.WriteLine("N4 Webservice call returned with status = " + status);
    if (ERRORS.Equals(status))
    {
        ConsoleInteropApplication.genericWebService.MessageType[] messages =
            commonResponse.MessageCollector;
        for (int i = 0; i < messages.Length; i++)
        {
            ConsoleInteropApplication.genericWebService.MessageType message =
messages[i];
            String msgSeverity = message.SeverityLevel;

```



```

        String msgText = message.Message;
        // Display each message
        Console.WriteLine("update status from ICU:\n" + msgText);
    }
}
else
{
    Console.WriteLine("successfully executed call to generic webservice");
    String xmlResults = response.genericInvokeResponse1.responsePayload;
    if (xmlResults != null)
    {
        Console.WriteLine("update status from ICU:\n" + xmlResults);
        // xml can be written into the file, and following line is just to show that
aspect
        File.WriteAllText(@"c:\temp\results.xml", xmlResults);
        // An external XML can be read into .NET client using following code snippets
        XmlReaderSettings readerSettings = new XmlReaderSettings();
        readerSettings.IgnoreWhitespace = true;
        XmlReader reader = XmlReader.Create(@"c:\temp\results.xml", readerSettings);
    }
}
}
}
// Following class is needed to setup authentication credentials (user/pwd) in the
HttpRequest.
// Should we learn about any better way, then this can be replaced with any better
approach.
// Without this step, we get authentication failure with N4 webservices.
class ExtendedGenericWebService : genericWebService.ArgoService
{
    private String _headerName;
    private String _headerValue;
    protected override WebRequest GetWebRequest(Uri uri)
    {
        // call the base class to get the underlying WebRequest object
        HttpWebRequest req = (HttpWebRequest)base.GetWebRequest(uri);
        if (null != _headerName)
        {
            // set the header
            req.Headers.Add(_headerName, _headerValue);
        }
        // return the WebRequest to the caller
        return (WebRequest)req;
    }
    //-----
    // SetRequestHeader
    //
    // Sets the header name and value that GetWebRequest will add to the
    // underlying request used by the SOAP client when making the
    // we method call
    //-----
    public void SetRequestHeader(String headerName, String headerValue)
    {
        _headerName = headerName;
        _headerValue = headerValue;
    }
}

```

```
}
}
```

Where the following line is loading an ICU XML file from the disk (this file is attached in this doc) and is also pasted below:

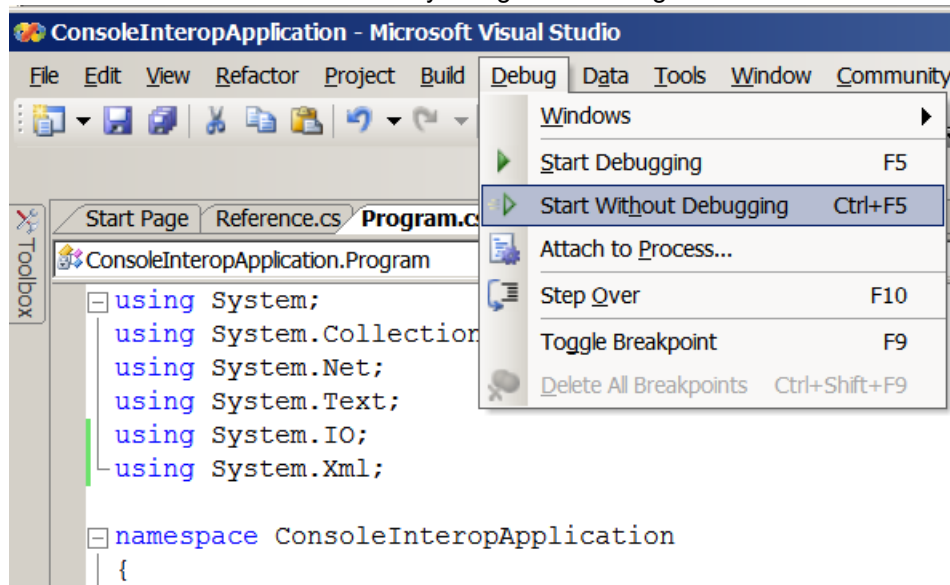
```
TextReader textReader = new StreamReader(@"C:\N4\latest\N4-marine-dev\argo\conf\runtime\testdata\icuTestFlexFields.xml");
```

Contents of the XML file being read are:

```
<icu>
  <units>
    <unit-identity id="XXXU1234567" type="CONTAINERIZED">
    </unit-identity>
  </units>
  <properties>
    <property tag="UnitFlexString01" value="UnitFlexString01Test"/>
    <property tag="UnitFlexString02" value="UnitFlexString02Test"/>
    <property tag="UnitFlexString03" value="UnitFlexString03Test"/>
    <property tag="UnitFlexString04" value="UnitFlexString04Test"/>
    <property tag="UfvFlexString01" value="UfvFlexString01Test"/>
    <property tag="UfvFlexString02" value="UfvFlexString02Test"/>
    <property tag="UfvFlexString03" value="UfvFlexString03Test"/>
    <property tag="UfvFlexString04" value="UfvFlexString04Test"/>
    <property tag="UfvFlexDate01" value="2007-07-24"/>
    <property tag="UfvFlexDate01" value="2007-07-23"/>
  </properties>
</icu>
```

Running the C# Client

The above client code can be run by doing the following:



The program output is shown below :

```
N4 Webservice call returned with status = 0
successfully executed call to generic webservice
update status from ICU:
<update-status>
```

```
<document-update status="ACCEPTED"/>
<entity-updates amount-successful="1" amount-failed="0" amount-not-found="0">
  <unit-update unit-id="XXXU1234567" status="SUCCESSFUL"/>
</entity-updates>
</update-status>
Press any key to continue . . .
```



Back end contains the unit(container) referenced in the ICU XML. If unit doesn't exist in the back end (N4) then you would get a warning message saying so.

To make sure end to end system work, your can go to Unit Inspector for the unit being updated in the above XML:

Unit Inspector for XXXU1234567

Unit Nbr : [XXXU1234567](#)
Type ISO : [XXX](#)

Status

T-State : **Yard**
Complex Position : **Y-YRD1111**
Last Move : **07-Sep-26 1308**
Frigh Kind : **FCL**
Line Op : **XXX**
Weight (kg) : **7654321 kg**
Stow :
Booking Number : [XXXZZZ1](#)

Transit

Category : **Export**
POD :
I/B Actual Visit : **GEN_TRUCK**
O/B Intend Visit : [AXX222](#)
Time In : **07-Sep-26 1308**
O/B Intend ETD :

Actions

Primary Equip.
Routing
Contents
Damages
Holds/Perms
Storage
Event History
All Equipment
Data Sources
Bills of Lading
Itinerary

Event History

Time	Description	Billable	Recorder
07-Sep-26 1313	Property Update		admin
07-Sep-26 1308	Activate		-system-
07-Sep-26 1308	Create		-system- test
07-Jul-23 1123	Testing Event Recording for ICU (UNIT)		snx:IcuTester Testing Ever

Event Details

Event Type

Event ID : **UNIT_PROPERTY_UPDATE**
Description : **Property Update**
Billable : **No**

Event Info

Target Entity : **XXXU1234567**
Unit
Recorder : **admin**
Time : **07-Sep-26 1313**
Qty :
Qty Unit :

Site Info

Operator : **OPR1**
Facility : **FCY111**
Complex : **CPX11**
Yard : **YRD1111**

Event Notes

Notes : **Property Update**

Field Changes

UFV Flex-04 : **null→UfvFlexString04Test**
Flex-03 : **null→UnitFlexString03Test**
Flex-01 : **null→UnitFlexString01Test**
UFV Flex-01 : **null→UfvFlexString01Test**
UFV Flex-03 : **null→UfvFlexString03Test**
ufvFlexDate01 : **null→20070722220000PDT**
Flex-04 : **null→UnitFlexString04Test**
Flex-02 : **null→UnitFlexString02Test**
UFV Flex-02 : **null→UfvFlexString02Test**

Close **Help**

5.7.2.4 Security - Authentication

As mentioned in the code comments above, there are various .NET web service client examples on the Internet that tend to use the following approach to supply credentials to the (.NET based web services) backend:

```
N4Webservice.Credentials = new NetworkCredential("login", "password");
```

Above code leads to an authentication failure. Following changes need to be done by the .NET client to be able to talk to N4 webservices (as all of them need username, and password)

Workaround to this issue is to subclass the generated webservice and instantiate that subclass. The subclass code is very generic and is pasted below:

```
// Following class is needed to setup authentication credentials (user/pwd) in the
HttpRequest.
// Should we learn about any better way, then this can be replaced with any better
approach.
// Without this step, we get authentication failure with N4 webservices.
class ExtendedGenericWebservice : genericWebservice.ArgoService
{
    private String _headerName;
    private String _headerValue;
    protected override WebRequest GetWebRequest(Uri uri)
    {
        // call the base class to get the underlying WebRequest object
        HttpWebRequest req = (HttpWebRequest)base.GetWebRequest(uri);
        if (null != _headerName)
        {
            // set the header
            req.Headers.Add(_headerName, _headerValue);
        }
        // return the WebRequest to the caller
        return (WebRequest)req;
    }
    //-----
    // SetRequestHeader
    //
    // Sets the header name and value that GetWebRequest will add to the
    // underlying request used by the SOAP client when making the
    // we method call
    //-----
    public void SetRequestHeader(String headerName, String headerValue)
    {
        _headerName = headerName;
        _headerValue = headerValue;
    }
}
```

Above code is used in the main class (shown above) thus:

```
// instantiate an instance of the subclassed service as shown below
ExtendedGenericWebservice N4Webservice = new ExtendedGenericWebservice();
// now populate the credentials
byte[] bcred = Encoding.ASCII.GetBytes("admin:admin");
string b64cred = Convert.ToBase64String(bcred);
N4Webservice.SetRequestHeader("Authorization", "Basic " + b64cred);
// actual call to business/service method and look into the response objects for
status.
```

```
ConsoleInteropApplication.genericWebservice.genericInvokeResponse response =  
N4Webservice.genericInvoke(arg);
```

5.7.2.5 Code Snippets/Attachments

A zip file attached to this document contains the .NET C# project and also an example ICU XML file that is referenced in the .NET project.

5.7.2.6 Example .Net Gate Integration

CSharpN4SParcsGateIntegration.zip contains a .Net project to make a gate API call. Running Program.cs will make two gate API calls, one successful and one failed with the following output.

N4 Gate API: Successful Example Call

```
Gate API STATUS:0-Successful  
PAYLOAD:  
<gate>  
<record-scan scan-set-id="54338">  
<gate-id>MG</gate-id>  
<stage-id>office</stage-id>  
<lane-id>LN.1</lane-id>  
<truck license-nbr="MFE590P" trucking-co-id="ACME"/>  
<timestamp>2007-10-01T17:49:22</timestamp>  
</record-scan>  
</gate>
```

N4 Gate API: Failed Example Call

```
Gate API STATUS:3-Severe errors occurred, check message.  
ERROR:Unknown gate NULLGATE  
PAYLOAD:  
<gate>  
<record-scan scan-set-id="54338">  
<gate-id>NULLGATE</gate-id>  
<stage-id>office</stage-id>  
<lane-id>LN.1</lane-id>  
<truck license-nbr="MFE590P" trucking-co-id="ACME"/>  
<timestamp>2007-10-01T17:49:22</timestamp>  
</record-scan>  
</gate>
```

5.8 Expose User SNX schema to Web service

This section covers both import and export of an entity to SNX using web services.

In This Section

- [SNX Import.](#) 123
- [SNX Export.](#) 124
- [Error cases.](#) 126
- [Using MassUnitType.](#) 127

5.8.1 SNX Import

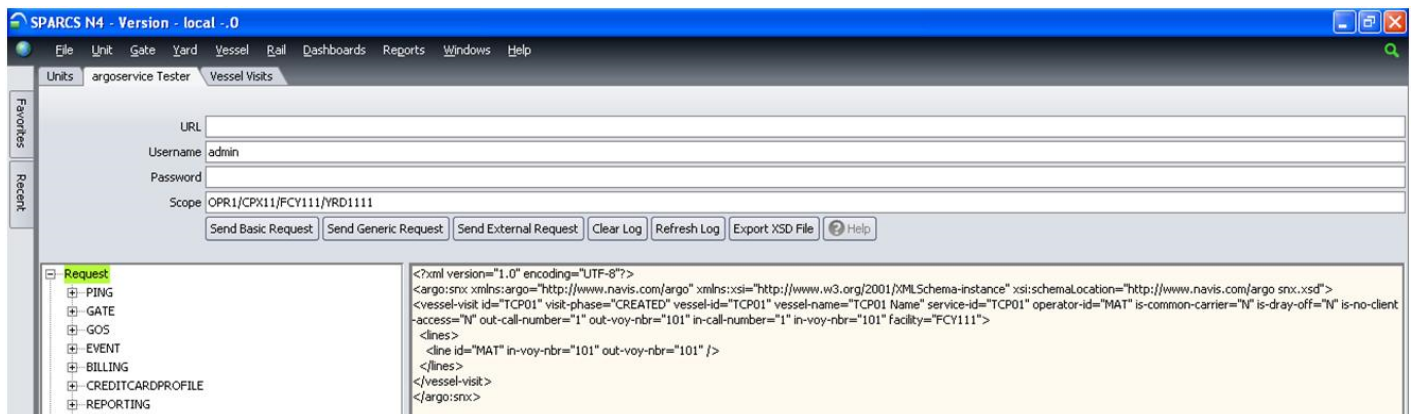
- The request element follows the same structure as the SNX and can contain multiple elements.
- If there are any processing errors during the import, the messages will be appended to the response.
- If the entity already exists, SNX webservice API will update it else the API will create it.
- The SNX webservice API delegates the import task to the same API that the UI calls so the result should be same as in UI.
- In case of multiple element import, suppose each element has an error and since these error messages will get appended to response, there may be a performance issue. This needs to be analyzed.

Example

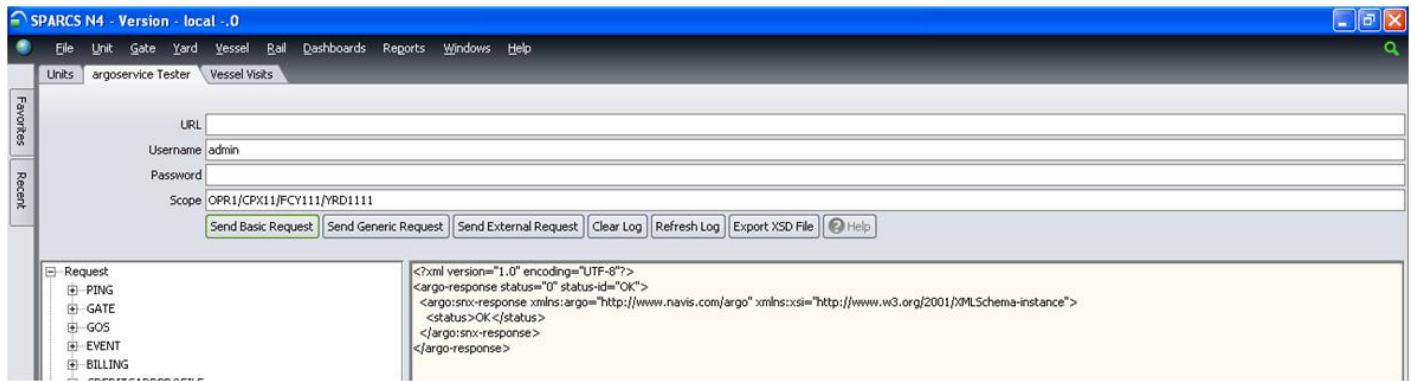
Suppose the user wants to import the following vessel-visit SNX element.

```
<?xml version="1.0" encoding="UTF-8"?>
<argo:snx xmlns:argo="http://www.Navis.com/argo" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xsi:schemaLocation="http://www.Navis.com/argo snx.xsd">
<vessel-visit id="TCP01" visit-phase="CREATED" vessel-id="TCP01" vessel-name="TCP01
Name" service-id="TCP01" operator-id="MAT" is-common-carrier="N" is-dray-off="N" is-no-
client-access="N" out-call-number="1" out-voy-nbr="101" in-call-number="1" in-voy-
nbr="101" facility="FCY111">
  <lines>
    <line id="MAT" in-voy-nbr="101" out-voy-nbr="101" />
  </lines>
</vessel-visit>
</argo:snx>
```

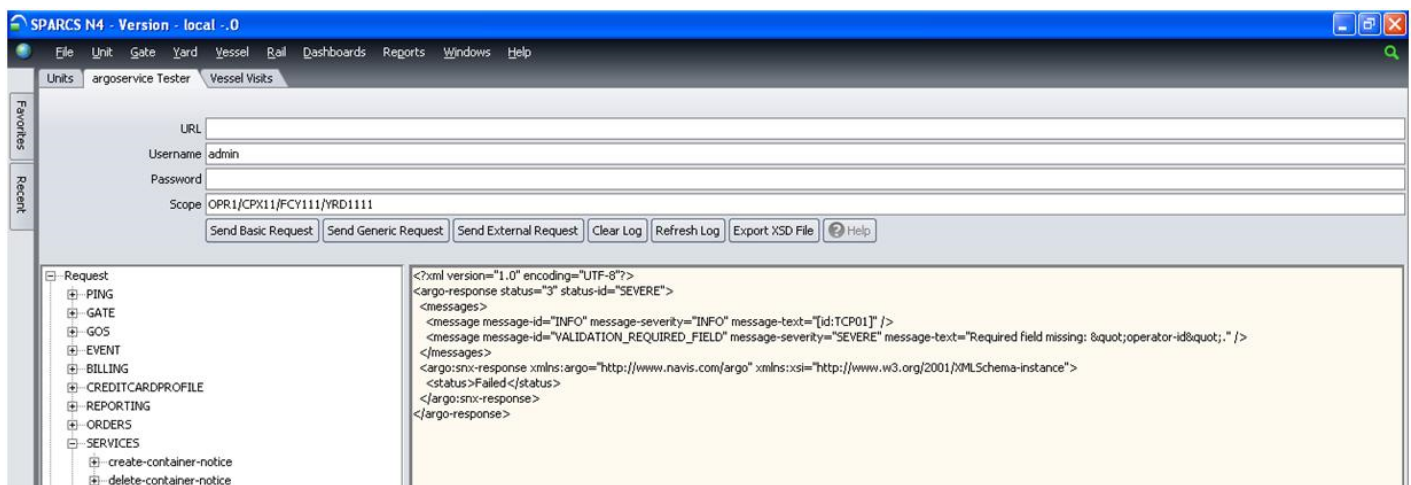
The above SNX element becomes the request element in the web service. See below the request element in argserviceTester.



On submitting, if the import is successful, the response element will have status code "0" and status-id "OK"



If the import fails, the response element will have status code "3" and status-id "SEVERE". The messages element will have the error messages.



SNX vessel-visit import (Create/update vessel visit using SNX via Web Services)

A new optional attribute validate-phase (= "true" or "false" -> default value) has been added to allow the client decide whether the importer should validate visit-phase or not. In case it should validate, the behavior will be exactly same as that of the UI.

```
<vessel-visit.....visit-phase="DEPARTED" validate-phase="true"...>
....
</vessel-visit>
```

5.8.2 SNX Export

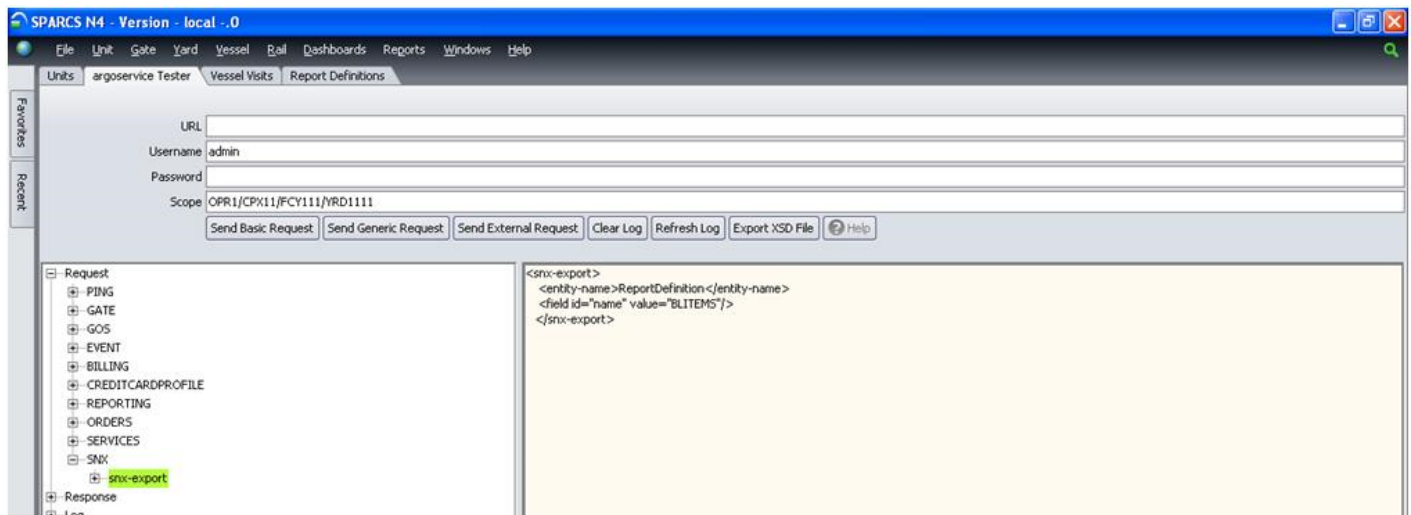
- The request element follows a generic structure so that later the request can be extended to all entities.

```
<snx-export>
<entity-name>ReportDefinition</entity-name>
<field id="name" value="ReportDefName"/>
</snx-export>
```

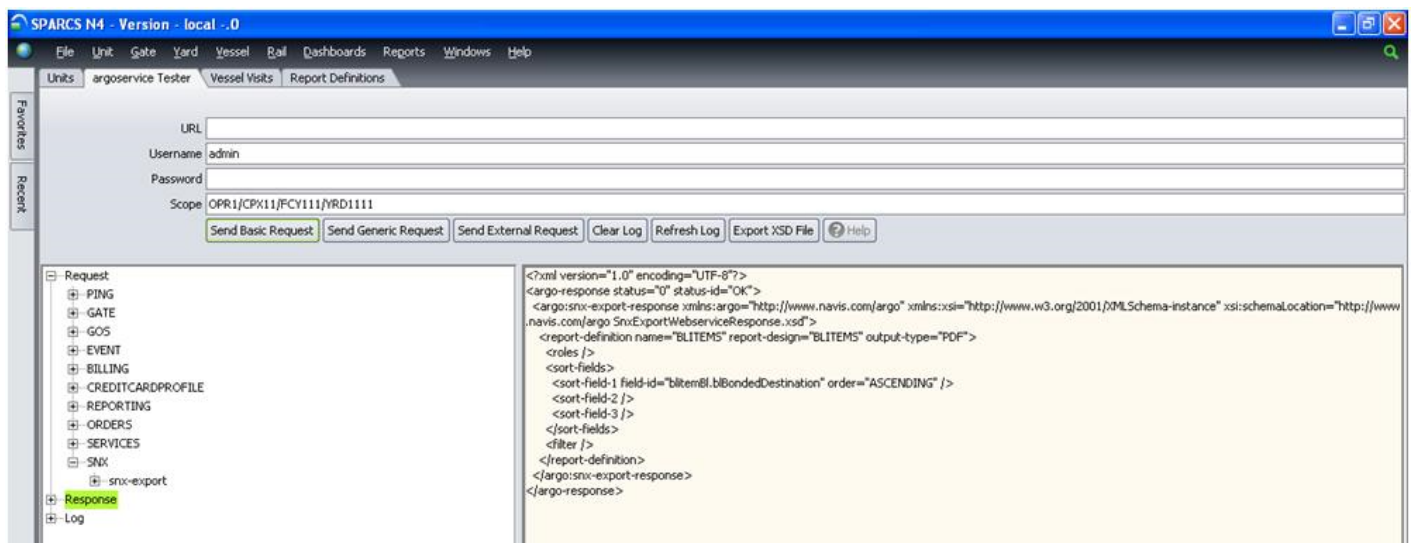
- The field element will be used to find the entity which needs to be exported.
- For now, it only supports ReportDefinition.
- <entity-name> will be the internal entity name that the service API is supporting. The id attribute contains the internal field name. In the case of report-definition, entity-name should be ReportDefinition and field ID "name".

Example

The request element in argoserviceTester:

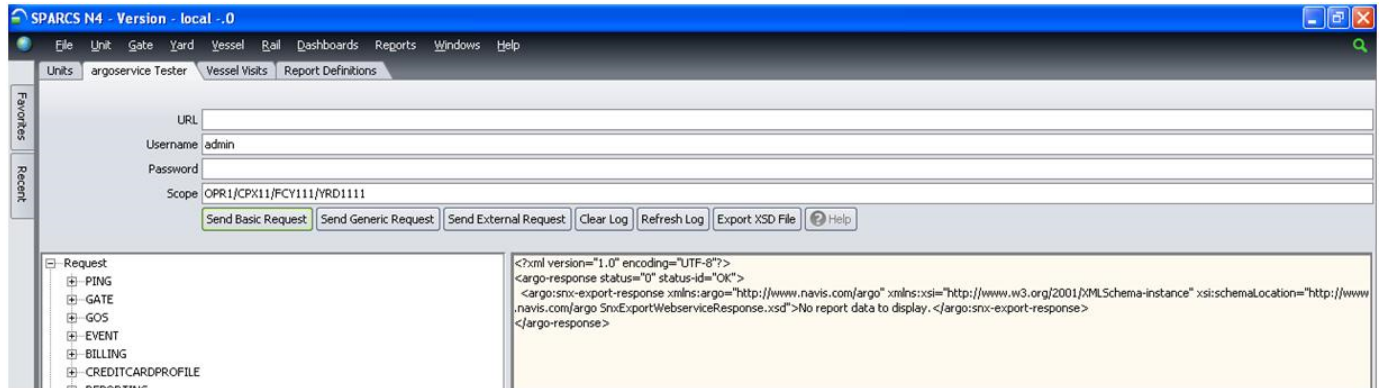


The response will return the exported SNX element. The response element will return OK status.



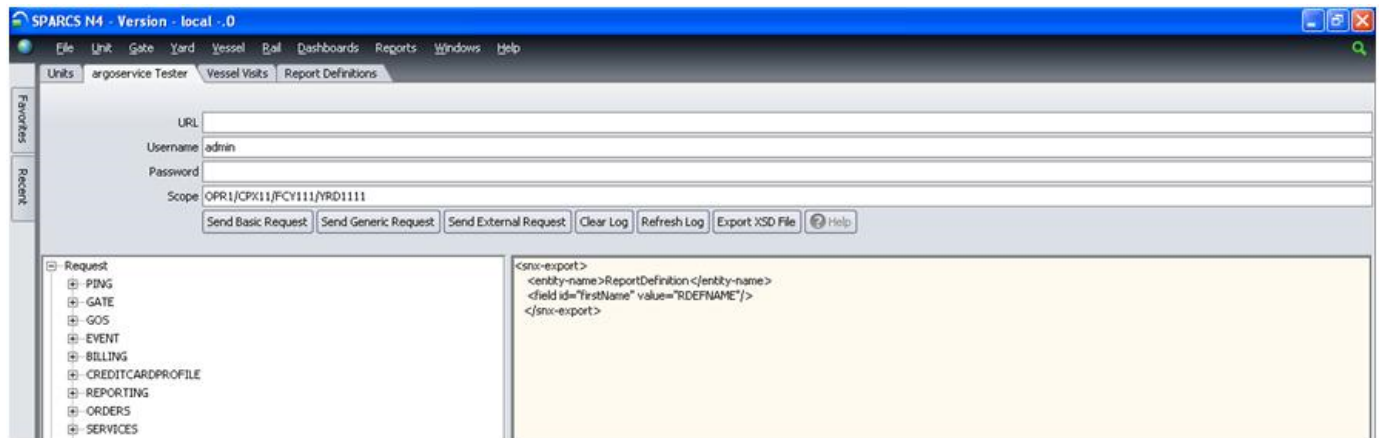
5.8.3 Error cases

- If the ReportDefinition requested doesn't exist in SN4.

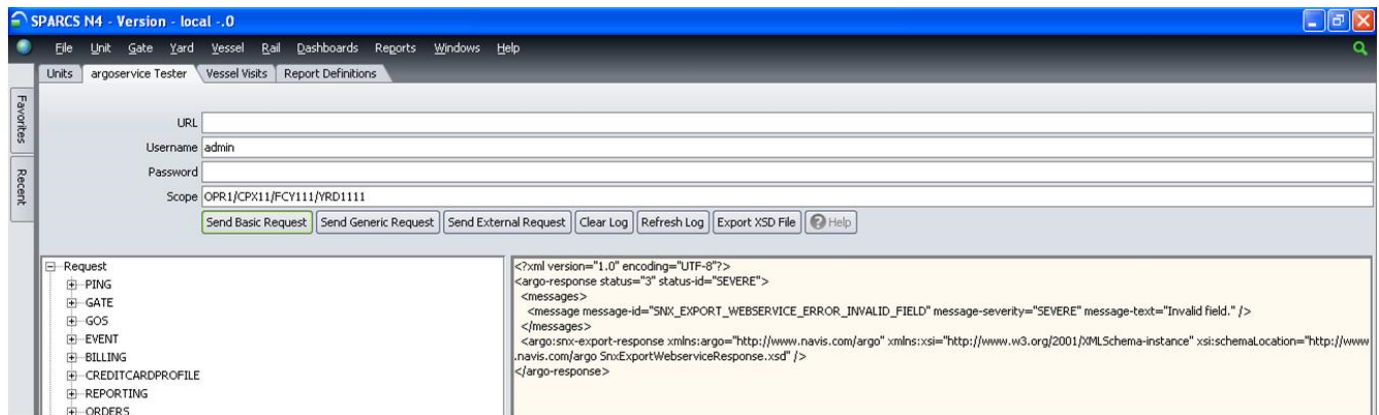


- If the field name is invalid.

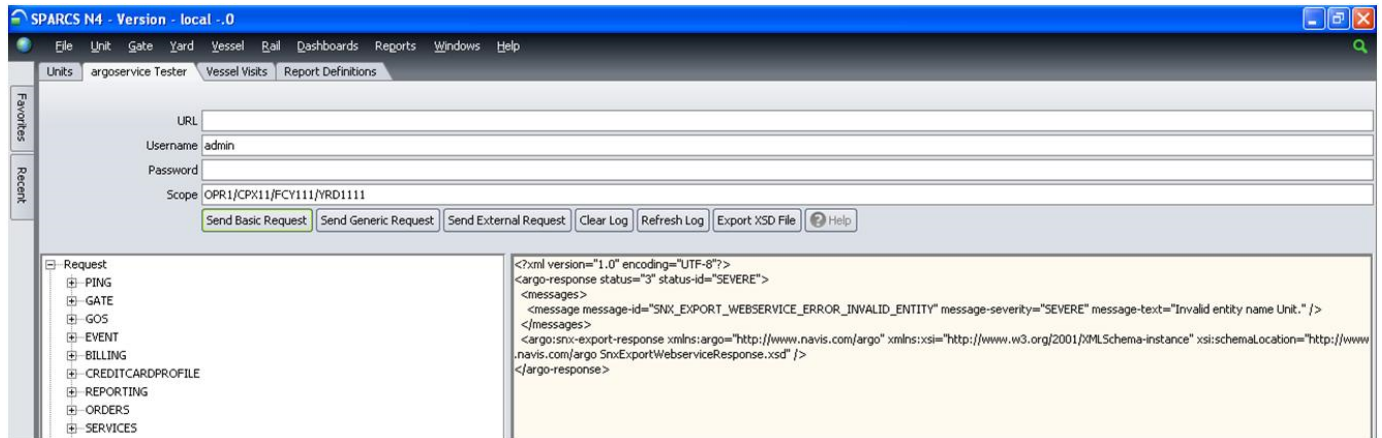
Request:



Response:



- If the entity-name is invalid.



5.8.4 Using MassUnitType

Any properties that take a mass unit of measure need be defined in the XML schema definition (XSD) using the MassUnitType.

For example,

```
<xs:complexType name="TruckType">
  <xs:all>
    <xs:element name="tare-weight" type="argo:MassUnitType" minOccurs="0"
maxOccurs="1"/>
    <xs:element name="safe-weight" type="argo:MassUnitType" minOccurs="0"
maxOccurs="1"/>
  </xs:all>
</xs:complexType>
```

A valid XML that conforms to the XSD above is then given as follows:

```
<truck>
  <tare-weight unit="kg">4000.88</tare-weight>
</truck>
```

5.9 N4 Compatible External Web Service

For some cases N4 needs to initiate calls to an external application for information. N4 has a mechanism built-in to make calls to a simple webservice with string parameters that exchanges xml messages. The content of the xml messages are specific to each installation.

The webservice API is closely modeled on the argobasic service (see [N4 Webservices \(on page 78\)](#)).

In This Section

Create a N4 external webservice in MS .NET (2008).	129
Create a N4 external webservice in MS .NET (2010).	131

5.9.1 Create a N4 external webservice in MS .NET (2008)

5.9.1.1 Create an N4 external web-service in C# with Visual Web Developer Express 2008

This example describes the creation of an N4 compatible web-service that N4 can use when making unsolicited calls to an external system in.

The webservice is modeled closely on the argobasicservice implemented in N4 for external requests. The basis of the webservice is that it has a simple string-only api that accepts xml messages and returns an xml response. This simplifies integrations and software upgrades because the webservice api never changes - only the message content evolves.

This example describes the creation of the webservice and verification with a simple fixed ping response that echoes back the request message. Other messages are described in the specific N4 module webservice documentation.

Microsoft Visual Web Developer Express 2008 (available free from Microsoft) was used with the webservice deployed to Internet Information Services (IIS).

In Visual Web Developer Express select New Web Site from the file menu.

Select ASP.NET Web Service and language C#.

In the Solution Explorer pane (top right) rename Service.cs and Service.asmx to n4Service.cs and n4Service.asmx respectively.

In the main window change the namespace from <http://tempuri.org/> to <http://argo.Navis.com/web-service/external>.

The following method was already created:

```
public string HelloWorld() {  
    return "Hello World";  
}
```

Change this to:

```
public string basicInvoke(stringscopeCoordinateIds, string xmlDoc) {  
    return "<gos-response>"  
        + "<ping-response>"  
        + "<scopeCoordinateIds>" + scopeCoordinateIds + "</scopeCoordinateIds>"  
        + "<request><![CDATA[" + xmlDoc + "]]></request>"  
        + "</ping-response>"  
        + "</gos-response>";  
}
```

Press CTRL-F5 to build and deploy the webservice. In the solution explorer click on the top entry of the tree (c:\files\n4Service in the screenshot below). The property sheet at the bottom shows the port on which the webservice is deployed (1561 in the screenshot).

Open a browser and go to <http://localhost:1561/n4Service>, replacing port as required. The following page should be displayed:

Click on the n4Service.asmx link.

Click on the basicInvoke link.

In the scopeCoordinateIds box enter the scope parameters for the site. These are terminal operator/complex/facility/yard (slash / separated, no spaces). See the argoservice tester section below for where this value is displayed in N4.

In the xmlDoc box enter <ping/>

Click Invoke.

The following xml response should be displayed:

```
<ping-response status="0">
  <scopeCoordinateIds>DPW/BEANT/AGW/AGW</scopeCoordinateIds>
  <request><![CDATA[ping]]></request>
</ping-response>
```

Note that the display of the response will vary by browser. Internet Explorer Firefox Chrome The elements <scopeCoordinateIds> and <request> show echo back the values of the 2 parameters entered.

Verification of the webservice from N4

In N4 open the argoservice Tester screen (Administration/Debug/argoservice Tester) Enter the URL of the service <http://localhost:1561/n4Service/n4Service.asmx>. The scope is pre-filled. Click on Request in the left pane and enter <ping/> on the right. Click Send External Request The response will be displayed in the right pane. Assuming this responds correctly then the mechanism of communication between N4 and the external system has been established. The webservice code can now be filled out to support whichever message content formats have been agreed.

Message content - Common Aspects

Although the message content specification varies by service there are some aspects that are common to all. The root element of the request identifies a grouping of related messages and equates to a module in N4. For example truck related messages which originate from the road module in N4 all have the root element <gos> to identify them as being for an external gate system. Any response should have the same root element as the request but with -response appended. Example: gos and gos-response. The element immediately below the root element identifies the message. As with the root element the response should return an element that is the message identifier passed in the request with -response appended. Example: request-truck-weight and request-truck-weight-response. (Note that the ping example above only had 1 level).

Status and Error Reporting

```
<gos-response>
  <messages>
    <message message-id="gos.truck_unknown"
      message-severity="SEVERE"
      message-text="Unknown truck RD-56-SK" />
  </messages>
  <request-truck-weight-response/>
</gos-response>
```

A status attribute should be included in the root element with the values 0 - OK or 3 - Error. Error messages can be returned by adding a <messages> element within the root element.

- message-id a code for the message
- message-severity a code indicating the severity of the message (INFO, WARNING, SEVERE)
- message-text the readable text of the message

Note that message response is also included where possible, even though its content may be empty. This helps to associate the response with the original request.

5.9.2 Create a N4 external webservice in MS .NET (2010)

5.9.2.1 Create an N4 external web-service in C# with Visual Web Developer Express 2010

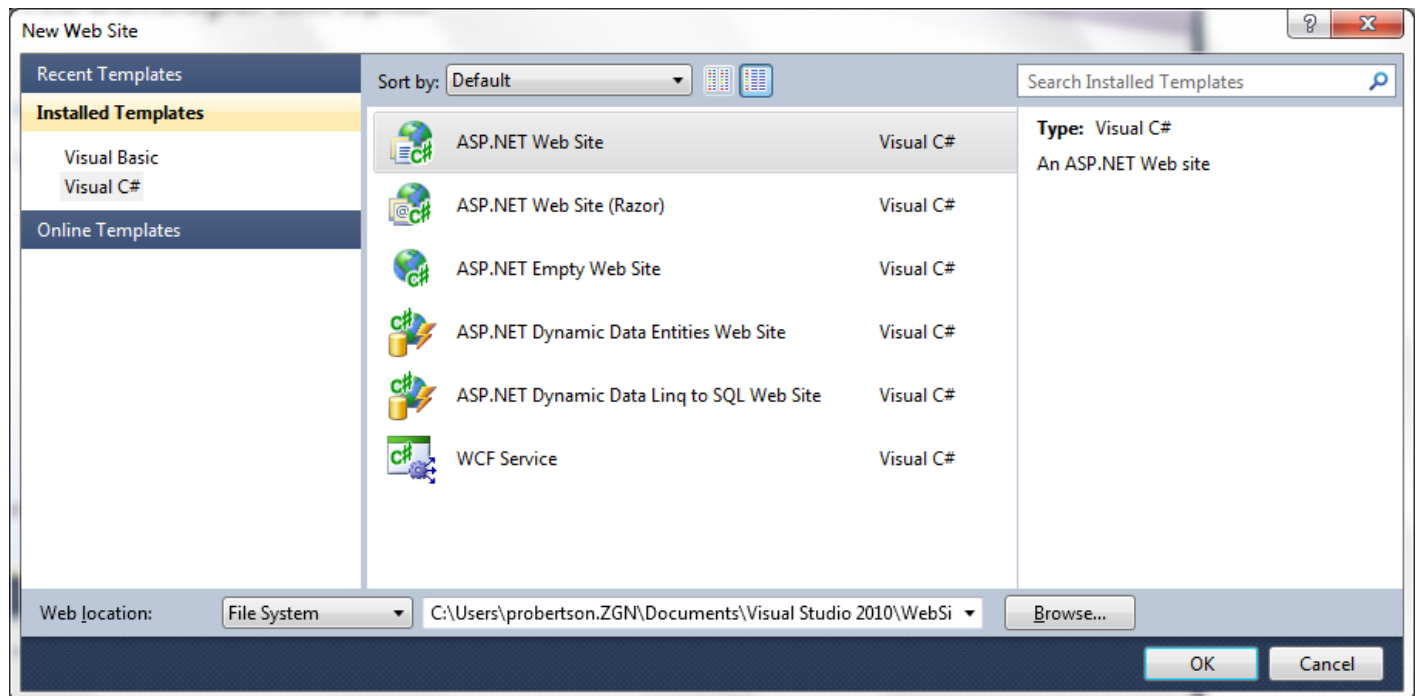
This example describes the creation of an N4 compatible web-service that N4 can use when making unsolicited calls to an external system.

The webservice is modeled closely on the `argobasicservice` implemented in N4 for external requests. The basis of the webservice is that it has a simple string-only api that accepts xml messages and returns an xml response. This simplifies integrations and software upgrades because the webservice api never changes - only the message content evolves.

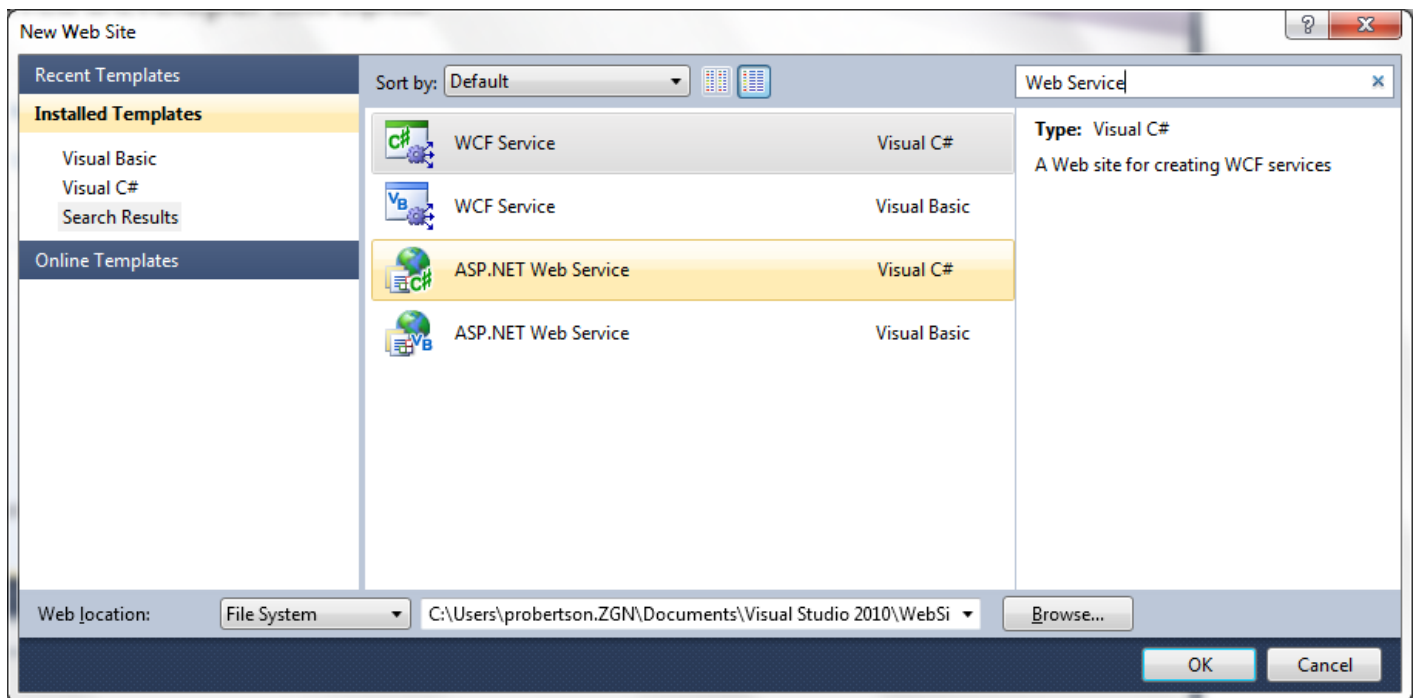
This example describes the creation of the webservice and verification with a simple fixed ping response that echoes back the request message. Other messages are described in the specific N4 module webservice documentation.

Microsoft Visual Web Developer Express 2010 (available free from Microsoft) was used with the webservice deployed to Internet Information Services (IIS).

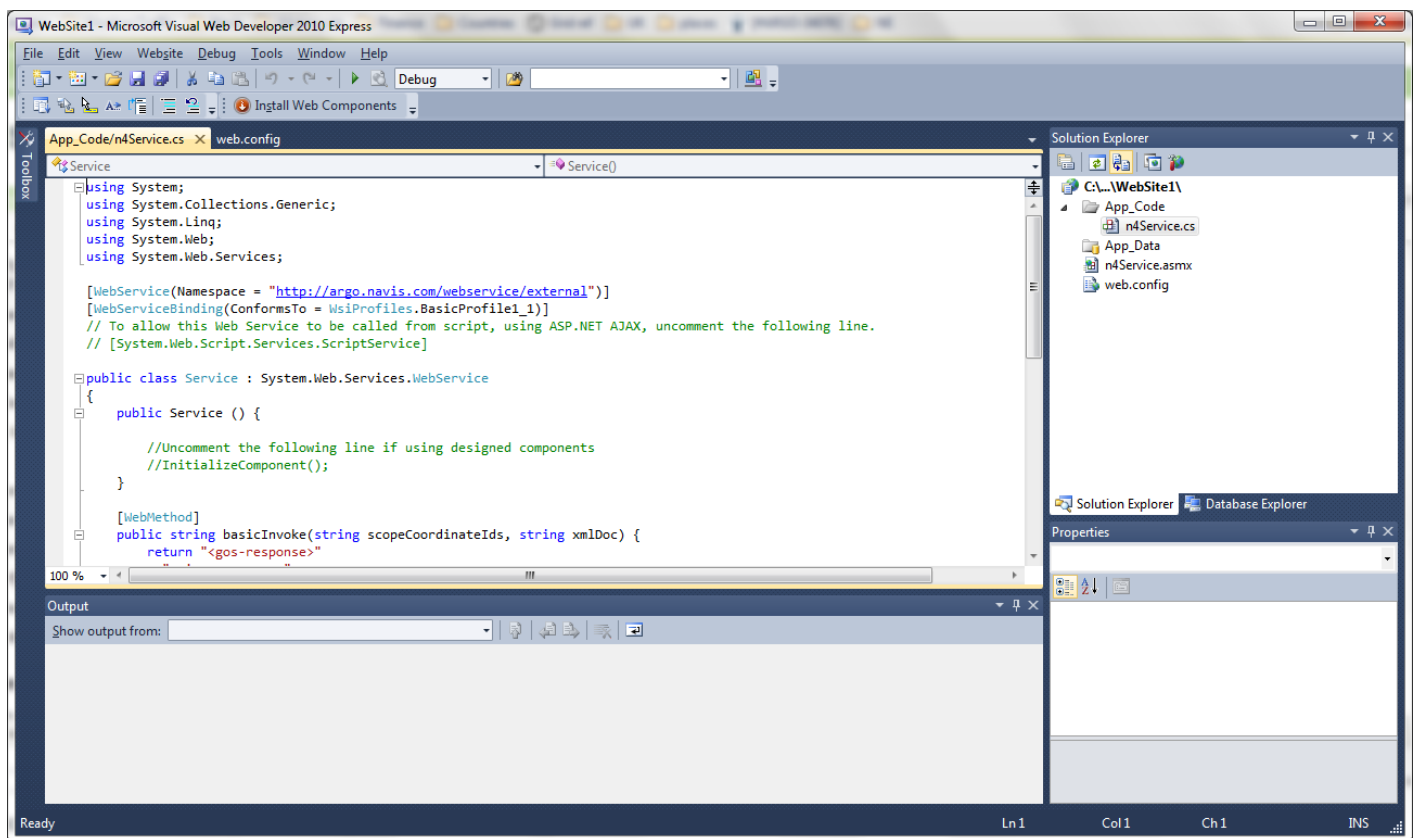
In Visual Web Developer Express select New Web Site from the file menu.



The type we want is ASP.NET Web Service Visual C#. This isn't installed by default - to find it enter Web Service in the search box.



Select ASP.NET Web Service Visual C#.



In the Solution Explorer pane (top right) rename Service.cs and Service.asmx to n4Service.cs and n4Service.asmx respectively.

In the main window change the namespace from <http://tempuri.org/> to <http://argo.Navis.com/webservice/external>.

In n4Service.cs the following method was already created.

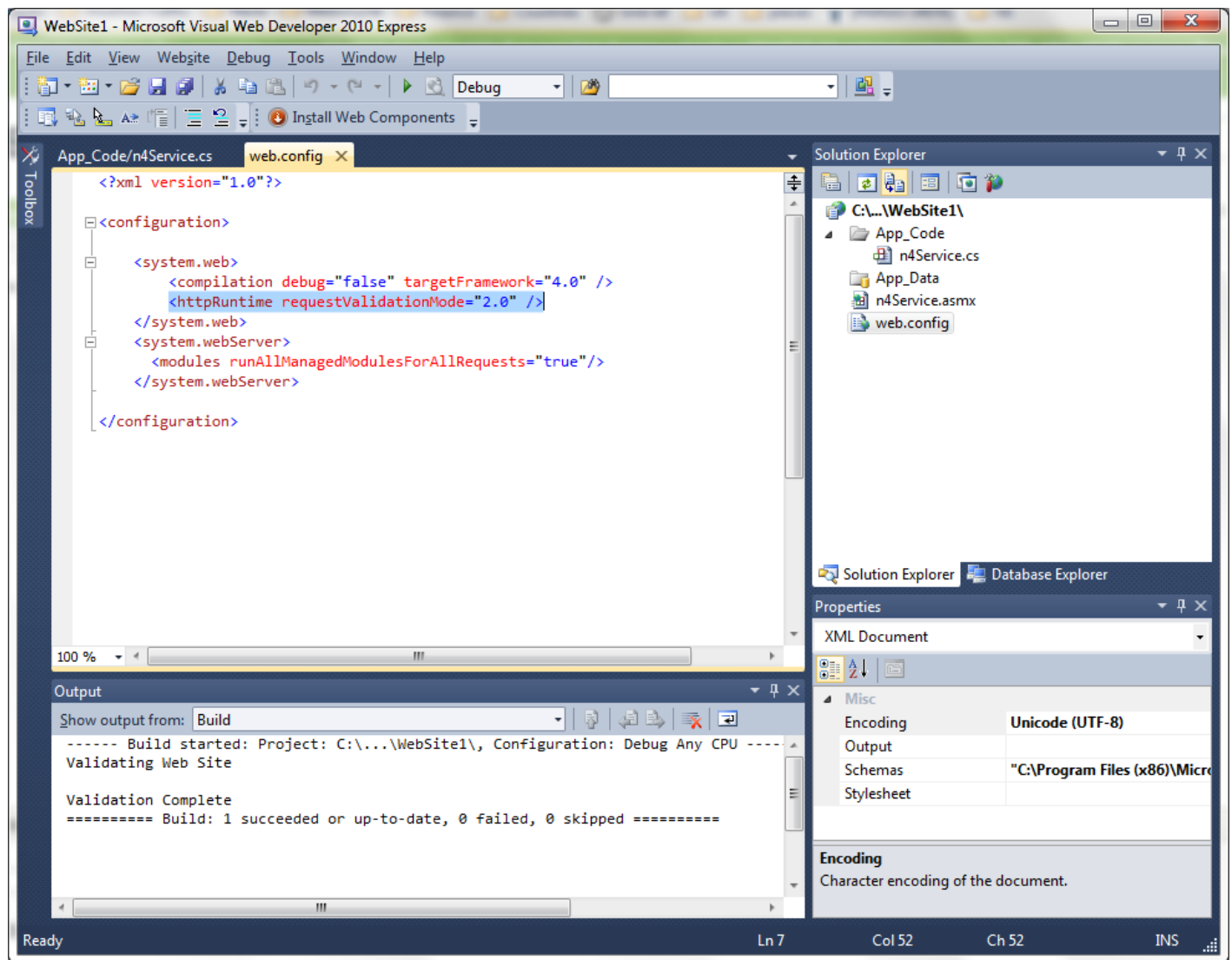
```
public string HelloWorld() {  
    return "Hello World";  
}
```

Change this to:

```
public string basicInvoke(string scopeCoordinateIds, string xmlDoc) {  
    return "<gos-response>"  
        + "<ping-response>"  
        + "<scopeCoordinateIds>" + scopeCoordinateIds + "</scopeCoordinateIds>"  
        + "<request><![CDATA[" + xmlDoc + "]]></request>"  
        + "</ping-response>"  
        + "</gos-response>";  
}
```

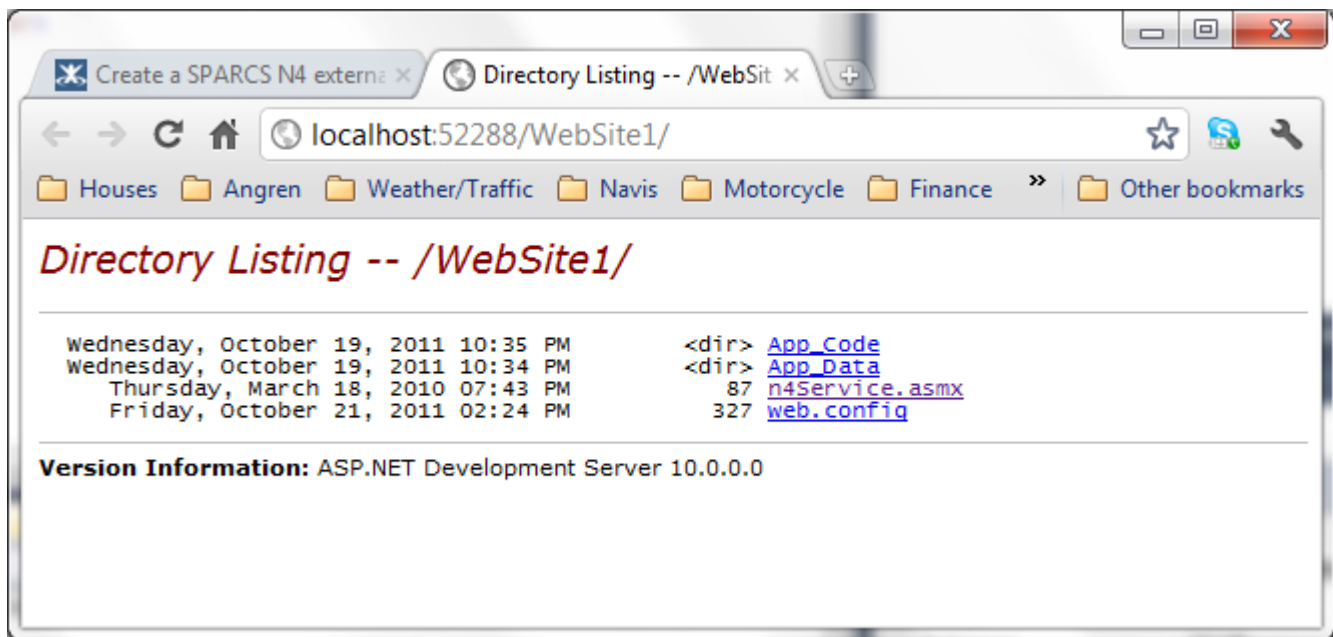
We will be passing xml to the xmlDoc parameter of the webservice however by default the <> characters will cause the webservice to give the following error:

```
System.Web.HttpRequestValidationException: A potentially dangerous Request.Form  
value was detected from the client (xmlDoc="<ping/>&quot;).  
To enable these characters double-click the web.config file in the solution explorer.
```



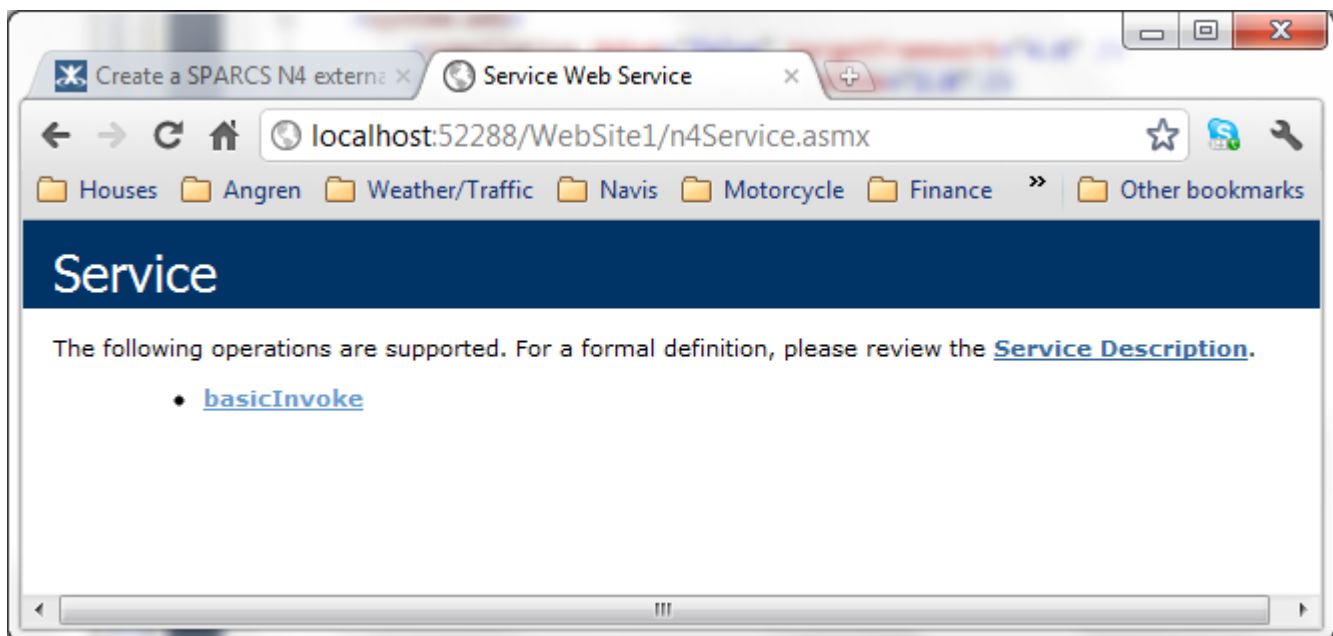
The highlighted line `<httpRuntime requestValidationMode="2.0" />` needs to be added. This will allow us to pass xml in the `xmlDoc` parameter.

Press CTRL-F5 to build and deploy the webservice - a browser window should open as shown below.

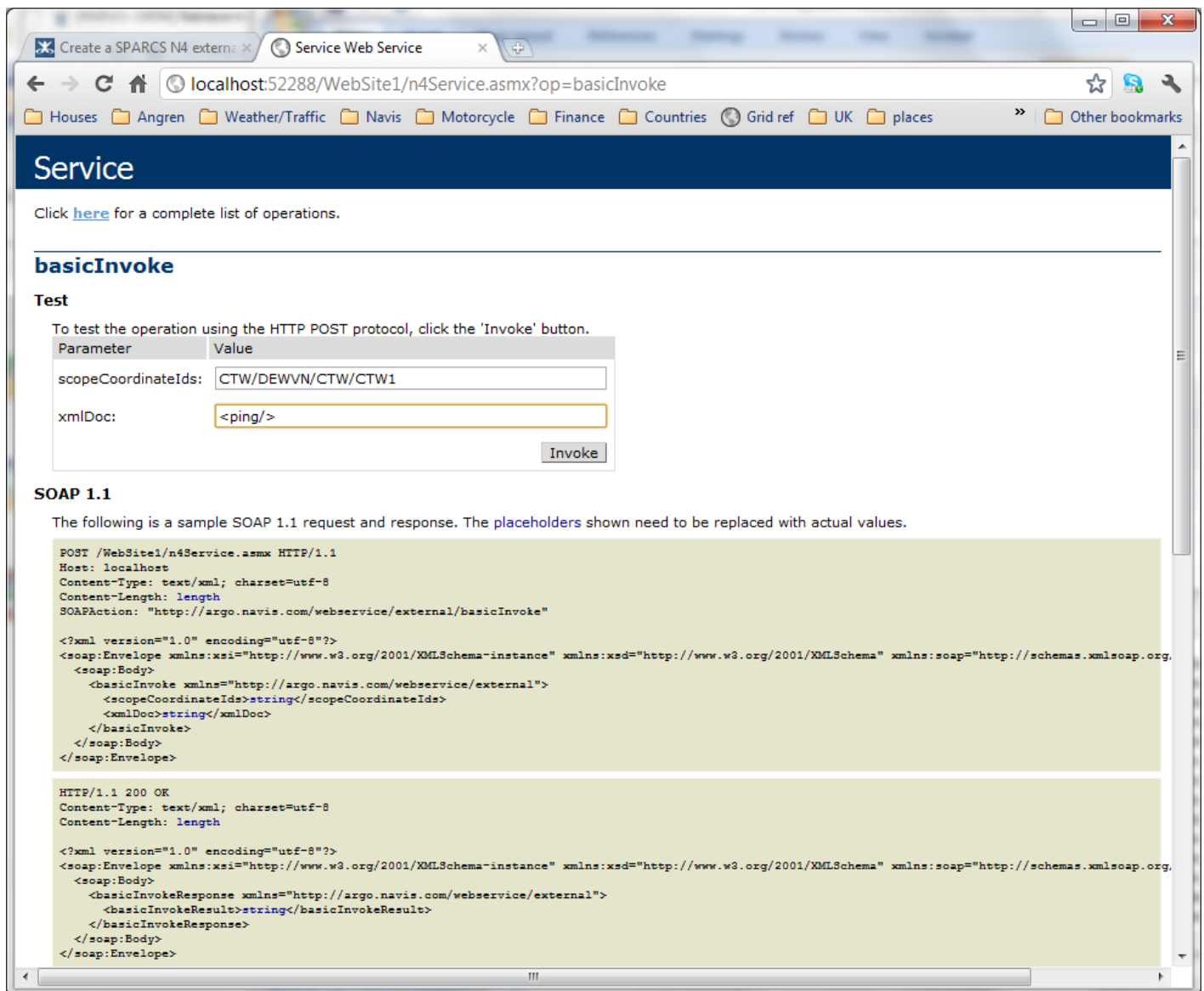


Take a note of the url show in the browser (Here it's <http://localhost:52288/WebSite1/>)

Click on the n4Service.asmx link.



Click on the basicInvoke link.

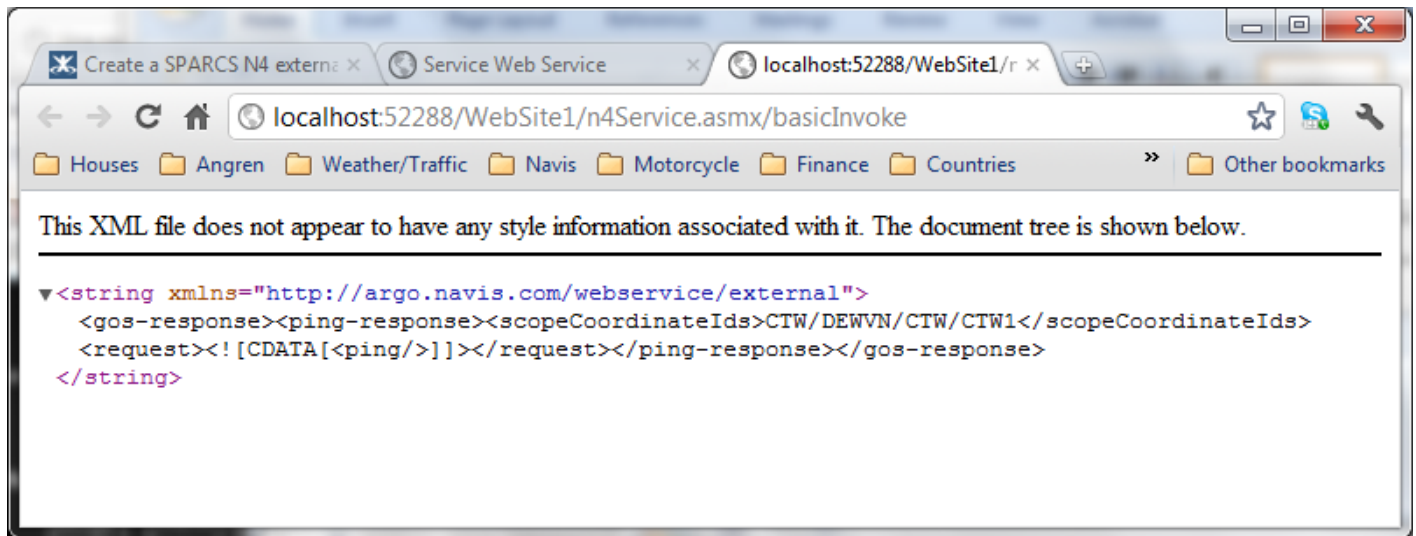


In the scopeCoordinateIds box enter the scope parameters for the site. These are terminal operator/complex/facility/yard (slash / separated, no spaces). See the argoservice tester section below for where this value is displayed in N4.

In the xmlDoc box enter <ping/>

Click Invoke.

The following xml response should be displayed:



Note that the display of the response will vary by browser.

Verification of the webservice from N4

In N4 open the argoservice Tester screen (Administration/Debug/argoservice Tester). Enter the URL of the service <http://localhost:52288/WebSite1>. The scope is pre-filled.

Click on Request in the left pane and enter `<ping/>` on the right. Click Send External Request

The response will be displayed in the right pane. Assuming this responds correctly then the mechanism of communication between N4 and the external system has been established. The webservice code can now be filled out to support whichever message content formats have been agreed.

Message content

Common Aspects

Although the message content specification varies by service there are some aspects that are common to all. The root element of the request identifies a grouping of related messages and equates to a module in N4. For example truck related messages which originate from the road module in N4 all have the route element `<gos>` to identify them as being for an external gate system. Any response should have the same root element as the request but with `-response` appended.

Example: `gos` and `gos-response`. The element immediately below the root element identifies the message.

As with the root element the response should return an element that is the message identifier passed in the request with `-response` appended.

Example: `request-truck-weight` and `request-truck-weight-response`. (Note that the ping example above only had 1 level).

```
<gos-response>
  <messages>
    <message message-id="gos.truck_unknown"
      message-severity="SEVERE"
      message-text="Unknown truck RD-56-SK" />
```

```
</messages>  
<request-truck-weight-response/>  
</gos-response>
```

Status and Error Reporting

A status attribute should be included in the root element with the values 0 - OK or 3 - Error. Error messages can be returned by adding a <messages> element within the root element.

- message-id a code for the message
- message-severity a code indicating the severity of the message (INFO, WARNING, SEVERE)
- message-text the readable text of the message

Note that message response is also included where possible, even though its content may be empty. This helps to associate the response with the original request.

5.10 Report Generation Web Service

In This Section

- [What does it do.](#) 140
- [Supported Queries.](#) 140
- [XML Schema.](#) 142
- [How to test it.](#) 143

5.10.1 What does it do

This webservice allows you to run pre-defined N4 reports. Report definitions must be already defined in N4.

Name	Report Type	Description	Design Name	Design Entity
ASC MOVE COUNT BY MO...	Reportable Entity	ASC Move Count by Move Type	ASC MOVE COUNT BY MO...	CheMoveStatistics
ASC Move Count Summary ...	Reportable Entity	ASC Move Count Summary Re...	ASC Move Count Summary ...	CheMoveStatistics
ASC PRODUCTIVITY REPORT	Reportable Entity	ASC Productivity Report	ASC PRODUCTIVITY REPORT	CheMoveStatistics
Average Daily Yard Utilization...	Reportable Entity	Average Daily Yard Utilization ...	Average Daily Yard Utilization...	YardBlockStatistics
CLERK ACTIVITY	Reportable Entity	Clerk Activity Report	CLERK ACTIVITY	TruckTransactionStage
External Trucks Report	Reportable Entity	External Trucks Report	External Trucks Report	TruckTransactionStage
Internal Prime Movers Report	Reportable Entity	Internal Prime Movers Report	Internal Prime Movers Report	CheMoveStatistics
MOVE COUNT BY POW BY ...	Reportable Entity	Move Count by Pow by CHE	MOVE COUNT BY POW BY ...	CheMoveStatistics
MOVING CHE PRODUCTIVL...	Reportable Entity	Moving che productivity analy...	MOVING CHE PRODUCTIVL...	CheTripStatistics
RTG/RMG Productivity Rep...	Reportable Entity	RTG/RMG Productivity Report	RTG/RMG Productivity Rep...	CheMoveStatistics
STRADDLE PRODUCTIVITY	Reportable Entity	Straddle Productivity	STRADDLE PRODUCTIVITY	CheMoveStatistics
TRUCK TURN TIME ANALYSIS	Reportable Entity	Truck Turn Time Analysis Report	TRUCK TURN TIME ANALYSIS	TruckVisitStageTransaction
UFV Move Count Report	Reportable Entity	UFV Move Count Report	UFV Move Count Report	Unit
Yard Reefer Plug Utilization	Reportable Entity	Yard Reefer Plug Utilization	Yard Reefer Plug Utilization	YardBlockStatistics
YARD REHANDLE	Reportable Entity	Yard Rehandle Report	YARD REHANDLE	UnitMoveEvent
Yard Stack Block Utilization	Reportable Entity	Yard Stack Block Utilization	Yard Stack Block Utilization	YardBlockStatistics

5.10.2 Supported Queries

Web service supports the following queries:

Universal query wrapper to get report definitions by filter name.

```
<report-def>
<universal-query filter="REP_DEF_DEMO"/>
</report-def>
```

For this request the webservice will return web definitions for the specified filter, like the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
<argo:argo xmlns:argo="http://www.Navis.com/argo" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xsi:schemaLocation="http://www.Navis.com/argo snx.xsd">
<report-definition name="A" report-design="RDNVM211211466" output-type="PDF">
<roles>
<role name="UNIT_ADMIN_ROLE" />
</roles>
<sort-fields>
<sort-field-1 field-id="arrivalCarrierVehicleSeq" order="ASCENDING" />
<sort-field-2 />
<sort-field-3 />
</sort-fields>
<filter />
</report-definition>
<report-definition name="UNITS_IN_YARD" report-design="UNITS IN YARD" output-
type="PDF" description="UNITS_IN_YARD" display-title="UNITS_IN_YARD">
<roles />
<sort-fields>
<sort-field-1 />
<sort-field-2 />
<sort-field-3 />
</sort-fields>
```



```

<filter>
<predicate verb="AND" order="1" negated="false" parm-type="NO_PARM">
<predicate verb="EQ" metafield="ufvUnit.unitLineOperator" order="0" negated="false"
parm-type="OPTIONAL_PARM" parm-label="Line Operator" parm-internal-
name="-476239859291268235" />
<predicate verb="EQ" metafield="ufvUnit.unitPrimaryUe.ueEquipment.eqEquipType"
order="1" negated="false" parm-type="OPTIONAL_PARM" parm-label="Equipment Type" parm-
internal-name="2968870044019877412" />
<predicate verb="TRUE" metafield="ufvUnit.unitRequiresPower" order="2" negated="false"
parm-type="REQUIRED_PARM" parm-label="Requires Power" parm-internal-
name="-8859221100133870547" />
<predicate verb="OR" order="3" negated="false" parm-type="NO_PARM">
<predicate verb="TRUE" metafield="ufvUnit.unitRequiresPower" order="0" negated="false"
parm-type="NO_PARM" />
<predicate verb="FALSE" metafield="ufvUnit.unitRequiresPower" order="1"
negated="false" parm-type="NO_PARM" />
</predicate>
</predicate>
</filter>
</report-definition>
</argo:argo>
</argo-response>

```

Get report parameters

```

<report-def>
  <get-report-parameters report-name="UNITS_IN_YARD"/>
</report-def>

```

On this request the webservice will return something like the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <parameters>
    <parameter name="Line Operator" is-required="N" />
    <parameter name="Equipment Type" is-required="N" />
    <parameter name="Requires Power" is-required="Y" />
  </parameters>
</argo-response>

```

Get Report URL

```

<report-def>
  <get-report-url report-name="UNITS_IN_YARD">
    <parameters>
      <parameter name="Requires Power" value="false" />
      <sort-field name="LineOperator" order="A" />
    </parameters>
  </get-report-url>
</report-def>

```

In response, the system returns a report URL, like the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <report-url>http://localhost:8280/apex/download?id=45</report-url>
</argo-response>

```

In response, the system also displays the missing parameters, if any, like the following:

```

<?xml version="1.0" encoding="UTF-8"?>

```

```
<argo-response status="2" status-id="WARNING">
<messages> <message message-id="MISSING_REQD_PARM" message-severity="WARNING" message-
text="Required parameter [DCB Voyage] is missing" message-detail="Required parameter
[DCB Voyage] is missing" />
</messages>
</argo-response>
```

Sort fields - Any field from reportable entity can be a sort field.

To support parameterized SQL reports, add the report-type attribute as in the example below. You need to add ":" after each parameter, as you can see in the example. Also, the sort field is not supported for SQL reports. (ARGO-160905)

```
<report-def>
<get-report-url report-name="SQLREPORT" report-type="SQL">
<parameters>
<parameter name="FreightKind:" value="FCL" />
<parameter name="Line:" value="POPEYE" />
</parameters>
</get-report-url>
</report-def>
```

5.10.3 XML Schema

XML schema that describes the above queries:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" xmlns:argo="http://
www.Navis.com/argo" targetNamespace="http://www.Navis.com/argo"
elementFormDefault="unqualified" attributeFormDefault="unqualified">
<xs:element name="report-def" type="report-defType" />
<xs:complexType name="report-defType">
<xs:choice>
<xs:element name="universal-query" type="universal-query-type" />
<xs:element name="get-report-parameters" type="get-report-parameters-type" />
<xs:element name="get-report-url" type="get-report-url-type" />
</xs:choice>
</xs:complexType>
<xs:complexType name="get-report-url-type">
<xs:attribute name="report-name" type="xs:string" />
<xs:sequence>
<xs:element name="parameters" type="parameters-type" />
</xs:sequence>
</xs:complexType>
<xs:complexType name="parameters-type">
<xs:sequence>
<xs:element name="parameter" type="parameter-type" />
<xs:element name="sort-field" type="sort-field-type" />
</xs:sequence>
</xs:complexType>
<xs:complexType name="sort-field-type">
<xs:attribute name="name" type="xs:string" />
<xs:attribute name="order" type="xs:string" />
</xs:complexType>
<xs:complexType name="parameter-type">
<xs:attribute name="name" type="xs:string" />
```

```

<xs:attribute name="value" type="xs:string" />
<xs:attribute name="is-required" type="xs:string" />
</xs:complexType>
<xs:complexType name="get-report-parameters-type">
  <xs:attribute name="report-name" type="xs:string" />
</xs:complexType>
<xs:complexType name="universal-query-type">
  <xs:attribute name="filter" type="xs:string" />
</xs:complexType>
</xs:schema>

```

5.10.4 How to test it

The above queries can be exercised from argoservice tester. The following illustrates an incorrect request:

Report Definitions | argoservice Tester

URL:

Username:

Password:

Scope:

Handler:

Buttons: Send Basic Request | Send Generic Request | Send External Request | Export XSD File | ? Help

Request: ☒ | Response: ☐

```

<report-def>
  <get-report-url report-name="CLERK ACTIVITY"/>
</report-def>

```

In response, the system displays the missing parameters:

Report Definitions | argoservice Tester

URL:

Username:

Password:

Scope:

Handler:

Buttons: Send Basic Request | Send Generic Request | Send External Request | Export XSD File | ? Help

Request: ☐ | Response: ☒

```

<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="2" status-id="WARNING">
  <messages>
    <message message-id="MISSING_REQD_PARM" message-severity="WARNING" message-text="Required parameter [StartTime, EndTime] is missing" message-detail="Required parameter [StartTime, EndTime] is missing" />
  </messages>
</argo-response>

```

5.10.4.1 Sample Java Client

The following is just a sample Java client to get report URL and to save it on disk:

```

public class GetReportWebserviceClient {
  private static final String ARGOSERVICE_URL = "http://localhost:8280/apex/services/argoservice";

```

```

private static String ERRORS = "3";
private static String fileName = "c:\\reportfile.pdf";
private static String xmlQuery = "<report-def>" +
    "<get-report-url report-name=\"UNITS_IN_YARD\">" +
    "<parameters>" +
    "<parameter name=\"Requires Power\" value=\"Y\"/>" +
    "<sort-field name=\"LineOperator\" order=\"A\"/>" +
    "</parameters>" +
    "</get-report-url>" +
    "</report-def>";

public static void main(String[] args) throws Exception {
    // sending webservice request
    System.out.println("Web service request:\n" + xmlQuery + "\n\n\n");
    GenericInvokeResponseWsType response = callGenericWebservice(xmlQuery);
    ResponseType commonResponse = response.getCommonResponse();
    if (commonResponse.getStatus().equals(ERRORS)) {
        System.err.println("Web service returned error:\n" +
commonResponse.getStatusDescription());
    }
    // getting report URL with webservice response
    String webserviceResponse = commonResponse.getQueryResults(0).getResult();
    System.out.println("Web service response:\n" + webserviceResponse);
    String reportUrl = webserviceResponse.substring(webserviceResponse.indexOf('>') +
1, webserviceResponse.lastIndexOf('<'));
    System.out.println("Downloading report file from " + reportUrl);
    // saving report to a file
    BufferedInputStream in = new BufferedInputStream(new URL(reportUrl).openStream());
    FileOutputStream out = new FileOutputStream(fileName);
    byte data[] = new byte[1024];
    int count = 0;
    while((count = in.read(data)) > 0) {
        out.write(data, 0, count);
    }
    out.close();
    in.close();
}

/**
 * Calls generic web service
 * @param inQueryXML
 * @return
 * @throws Exception
 */
public static GenericInvokeResponseWsType callGenericWebservice(String inQueryXML)
throws Exception {
    GenericInvokeResponseWsType response = null;
    ScopeCoordinateIdsWsType scope = new ScopeCoordinateIdsWsType();
    scope.setOperatorId("OPR1");
    scope.setComplexId("CPX11");
    scope.setFacilityId("FCY111");
    scope.setYardId("YRD1111");
    // Identify the Web Services host
    ArgoServiceLocator service = new ArgoServiceLocator();
    ArgoServicePort port = service.getArgoServicePort(new URL(ARGO_SERVICE_URL));
    Stub stub = (Stub) port;
    // Specify the User ID and the Password
    stub._setProperty(Stub.USERNAME_PROPERTY, "admin");
}

```

```
    stub._setProperty(Stub.PASSWORD_PROPERTY, "admin");  
    response = port.genericInvoke(scope, inQueryXML);  
    return response;  
}  
}
```

5.11 Invoking Outbound WebService from N4 which is compliant with N4 basic service

In This Section

Introduction.	147
Invocation.	147

5.11.1 Introduction

This document explains how to setup and invoke an outbound web service from N4 which is compliant with the N4 basic service. The compliance means that the the webservice API has the same signature of the N4 basic invoke, which is

```
public String basicInvoke(String inScope, String inXmlPayload)
```

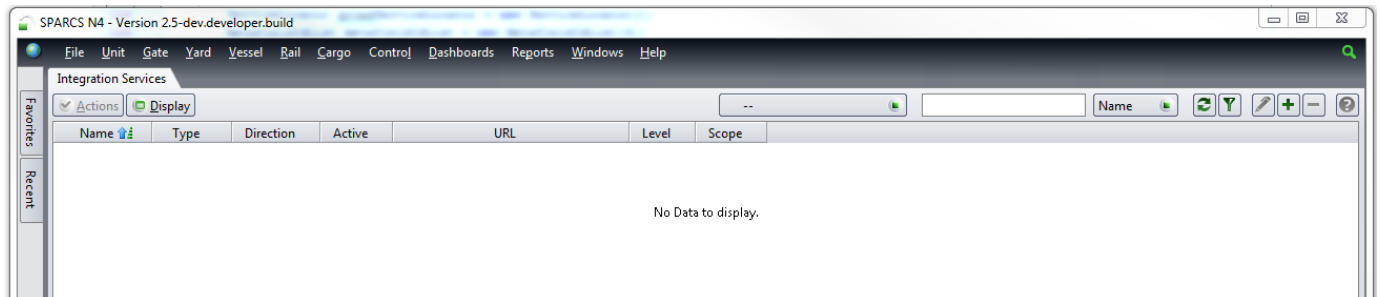
The main use case for this functionality is to invoke outbound web-services from Groovy as part of interfacing with the external systems. A good example of this Crane Automation System (CAS).

5.11.2 Invocation

The invocation of the outbound service requires setting up the outbound service in N4 and then invoking it from the Groovy code.

Steps to set up an outbound service in N4:

1. Go to the Integration services view in the N4 application.



2. Open a form for creating a new integration service by clicking the '+' button and enter the type as 'Web Service' and populate other fields and save the form.

The recommended way is to set up the web service at Global scope and handle scoping in the service.

5.11.2.1 Invoking the web service call

To invoke the web service call use the following code

```
String webServiceResponse = ArgoUtils.invokeExternalBasicService("CAS_OUTBOUND",  
xmlPayload);
```

The first argument is the name of the service as added in the form and the second argument is the payload. The scope is constructed from the logged in user by the API and web service is invoked.

The web services which are not N4 basic service API compliant can also be setup in the integration services screen, but in these cases the invocation API will not work and groovy code needs to be written to invoke the web service.

CONFIDENTIAL INFORMATION OF Navis LP

UNAUTHORIZED USE IS STRICTLY PROHIBITED.

N4 is a trademark of Navis LP

© Navis 1988 - 2024. All rights reserved.

This help has been made available as part of the license that has been granted to an authorized user of Navis LP software. Use of this documentation is subject to the terms and limitations of that license agreement. This documentation describes all functionality that can be licensed for this product. Not all functionality described in this documentation may be available to you depending on your license agreement. If you are not aware of the relevant terms of your license agreement, contact sales at Kaleris. This document is protected by copyright and by the licensee's agreement to maintain the confidentiality of the content of this document as provided in the license agreement for the software.

Liability Disclaimer

Kaleris takes steps to ensure that its published documentation is correct; however, errors do occur. Kaleris reserves the right to correct any such errors and disclaims liability resulting therefrom.

Kaleris

3460 Preston Ridge Rd, Suite 600

Alpharetta, GA 30005 USA

www.kaleris.com

Part 6

N4 REST API

This section covers use the N4 REST API.

In this Section:

Overview of the N4 REST API Service.	151
Set up API access to N4.	152
Creating a REST API with Groovy.	152
Security for the REST API.	160
CHE Current Position API	167
N4 Passthrough for EVP.	169
Billing REST Endpoints.	170
Service Order API.	173

6.1 Overview of the N4 REST API Service

N4 allows the extensibility and customization for external applications through N4 REST API service (n4restapiservice).

Through custom Code Extensions, you can create a REST API to N4. This allows an external application to call an API to get N4 fields and N4 layout dynamically to render them on its own screen. It can allow the application to add or remove fields, re-order the fields, and re-arrange the layout.

6.2 Set up API access to N4

The application using REST API is defined as an N4 user with a role that has the `EXTENSION_CODE_EXECUTION_VIA_REST` privilege that allows API execution. Having this privilege provides access for the application to execute any defined API available to the user's scope. You will set up the application's client ID as the N4 user.

To enable API access:

1. Enable the setting `FRMCARINA032 (CODE_EXTENSION_BY_REST_ENABLE)`.
 - a. In N4, open the **Settings** view (**Administration** → **Settings** → **Settings**).
 - b. Change the setting to **true**.
2. Define a user role that includes the privilege called **Execute Code Extensions via REST Protocol**.
 - a. Open the **Roles** view (**Administration** → **Security** → **Roles**).
 - b. Include **Execute Code Extensions via REST Protocol**.
 - c. Include **Code Extensions** if you are using Groovy code extensions.

Edit Role

Role Name:

Role Description:

☐ Is a Delegated Role

Owner Delegated Role:

Excluded [1725]

Search

Access to custom UI action
CUSTOM_CHANGE_OK_TOO_LATE_TO_OK on
variform=APT_TABLE_GATE_APPOINTMENTS
Access to custom UI action
CUSTOM_RAIL_SEND_OCR on variform=RAL005
Access to custom UI action
INV_FIX_DISCH_MISIDENTIFICATION
Access to custom UI action
INV_UNIT_RECAP_ALL_ACTION on
variform=TABLE_LIVE_UNITS
Access to custom UI action
INV_UNIT_RECAP_SELECT_ACTION on
variform=TABLE_LIVE_UNITS

Included [1]

Search

Execute Code Extensions via REST Protocol

6.3 Creating a REST API with Groovy

N4 can be accessed with REST APIs through the `n4restapiservice` service. This section describes how you can create a REST API with Groovy.

The N4 Groovy REST API supports media types such as XML, text, multipart data, files (streams), and JSON, when sending requests or receiving responses.

Prerequisites

Make sure that you have these privileges enabled in your user role in N4:

The following privileges control the ability to work with Groovy to customize N4.

ID	Name	Description	License Required	Option Name
GROOVY_PLUGIN_S	Admin - Groovy Plug-Ins	Create/Edit Groovy Plug-Ins	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GATE_BIZ_TASK_GROOVY	Gate Biz Task - Groovy	Add Groovy customization to a gate business task.	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GENERAL_NOTICES_GROOVY	General Notices - Groovy	Access the Groovy action in the General Notices	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GROOVY_JOB_ADD	Groovy Job - Add	Add Groovy Jobs	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GROOVY_JOB_DELETE	Groovy Job - Delete	Delete Groovy Jobs	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GROOVY_JOB_EDIT	Groovy Job - Edit	Edit Groovy Jobs	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GROOVY_JOB_VIEW	Groovy Job - View	View Groovy Jobs	EXTENSIONS	Advanced Configuration, Customization, and Integration Option
GROOVY_JOB_SCHEDULE	Groovy Jobs Actions - All	Access the schedule a groovy job, un-schedule a groovy job, or execute a groovy job actions in Groovy Jobs	EXTENSIONS	Advanced Configuration, Customization, and Integration Option

See **Groovy best practices** in the Navis N4: Setup, Maintenance, and System Diagnostics Guide.

6.3.1 Enable the Groovy message log

You can enable logging for custom Groovy API messages to appear in the **Web Service Logs** view. Set ARGOWEBSERVICE008 (WS_LOG_GROOVY) to **true**.

6.3.2 Create the Groovy code

You need to create the Groovy code as follows:

```
public class Reeferapi_v1 extends AbstractN4RestApiRequest {
    EN4RestApiRequest.IN4ApiPayload handleRestApiRequest(UserContext
inUserContext, MessageCollector inOutMessageCollector, Map inParameters) {
    ...
}
}
```

The *handleRestApiRequest* method is the entry point of the incoming REST requests.

The first thing that this method needs to do is map the incoming request *apiresource* to a method handler.

You may also need to invoke different handlers based on the REST verb, such as. GET, POST, PUT, DELETE.

```
String resourceParam = inParameters.get(INPARAM_RESOURCE);
String httpMethodParam = inParameters.get(INPARAM_HTTP_METHOD);
if ("GET".equalsIgnoreCase(httpMethodParam) &&
"reefer".equalsIgnoreCase(resourceParam)) {
    result = getReeferByID(inUserContext, inOutMessageCollector,
inParameters);
}
```

The handler method must return a response of type *IN4ApiPayload*.

All other parameters sent on the request are accessible from the parameters map.

The Groovy code will map this resource and the HTTP method (GET,PUT,POST,DELETE), to some feature functionality/method, that needs to be invoked. This effectively decouples the API resource name from the internal implementation. If at some point the internal implementation needs to change the public facing API can remain the same if possible, at least in terms of name changes.

At this time HTTP headers are not part of the parameter map.

6.3.2.1 JSON Processing

After running the API requests, data is returned from N4 as JSON payloads. The base class provides utility methods for turning entities into JSON payloads.

The returned *IN4ApiPayload* payload object will be turned into a JSON message like this:

```
{
  "content": {
    .....
  },
  "messages": [
    {
      "level": "INFO",
      "key": "INFO_NO_NOTIFICATION_CREATED",
      "message": "No Notification Created"
    }
  ]
}
```

Example

```
{
  "content": {
    "_key": "This is a GET method with: ",
    "_value": {
      "extensionName": "Testapi_v1",
      "http_method": "GET",
      "resource": "helloworld/0521140"
    }
  },
  "messages": []
}
```

6.3.2.2 Example Groovy

Fully functional example groovy that makes a bizFacade call, including a GET and a POST

Testapi_v1.groovy

```
/*
 * Copyright (c) 2021 Navis LLC. All Rights Reserved.
 *
 */
import com.google.common.collect.ImmutableList
import com.google.gson.annotations.SerializedName
import com.navis.external.framework.request.AbstractN4RestApiRequest
import com.navis.external.framework.request.EN4RestApiRequest
import com.navis.framework.portal.QueryUtils
import com.navis.framework.portal.UserContext
import com.navis.framework.portal.query.DomainQuery
import com.navis.framework.portal.query.PredicateFactory
import com.navis.framework.rest.RestConstants
import com.navis.framework.util.ValueHolder
import com.navis.framework.util.message.MessageCollector
import com.navis.framework.util.message.MessageLevel
import com.navis.inventory.InventoryEntity
import com.navis.inventory.business.api.UnitField
import com.navis.inventory.business.atoms.UnitVisitStateEnum
public class Testapi_v1 extends AbstractN4RestApiRequest {
  private static final String GET_ALL_ACTIVE_UFV = "units";
  private static final String GET_ACTIVE_UFV_BY_ID = "unit";
  @Override
  EN4RestApiRequest.IN4ApiPayload handleRestApiRequest(UserContext
inUserContext, MessageCollector inOutMessageCollector, Map inParameters) {
    EN4RestApiRequest.IN4ApiPayload result = null;
    String resourceParam = inParameters.get(INPARAM_RESOURCE);
    String httpMethodParam = inParameters.get(INPARAM_HTTP_METHOD);
    if (RestConstants.GET.equalsIgnoreCase(httpMethodParam) &&
GET_ALL_ACTIVE_UFV.equalsIgnoreCase(resourceParam)) {
      result = getAllActiveUFVHandler(inUserContext, inOutMessageCollector);
    } else {
      inOutMessageCollector.appendMessage(MessageLevel.SEVERE,
INVALID_REQUEST_MSG_KEY, null, { null });
      return AbstractN4RestApiRequest.N4ApiPayload.EMPTY_PAYLOAD;
    }
    return result;
  }
}
```

```

    private EN4RestApiRequest.IN4ApiPayload getAllActiveUFVHandler (UserContext
inUserContext,
                                final MessageCollector
inOutMessageCollector) {
    ImmutableList<ValueHolder> activeUnitValueHolders =
getUnitDomainQuery(inUserContext, inOutMessageCollector);
    List<UnitPayload> activeUnitList = new
AbstractN4RestApiRequest.JsonList();
    for (int i = 0; i < activeUnitValueHolders.size(); i++) {
        ValueHolder valueHolder = activeUnitValueHolders.get(i);
        UnitPayload unitPayload = new UnitPayload();
        unitPayload.set_id(valueHolder.getFieldValue (UnitField.UFV_UNIT_ID) as
String);

unitPayload.set_category(valueHolder.getFieldValue (UnitField.UFV_UNIT_CATEGORY)
as String);

unitPayload.set_lineOp(valueHolder.getFieldValue (UnitField.UFV_LINE_OPERATOR)
as String);
        activeUnitList.add(unitPayload);
    }
    return new AbstractN4RestApiRequest.N4ApiPayload(inUserContext,
inOutMessageCollector, activeUnitList);
}

public class UnitPayload {
    @SerializedName("id")
    private String _id;
    @SerializedName("category")
    private String _category;
    @SerializedName("line-op")
    private String _lineOp
    String get_id() {
        return _id
    }
    void set_id(String inId) {
        _id = inId
    }
    String get_category() {
        return _category
    }
    void set_category(String inCategory) {
        _category = inCategory
    }
    String get_lineOp() {
        return _lineOp
    }
    void set_lineOp(String inLineOp) {
        _lineOp = inLineOp
    }
}

    private ImmutableList<ValueHolder> getUnitDomainQuery (UserContext
inUserContext, MessageCollector inOutMessageCollector) {
        DomainQuery dq =
QueryUtils.createDomainQuery (InventoryEntity.UNIT_FACILITY_VISIT)
            .addDqPredicate (PredicateFactory.eq (UnitField.UFV_VISIT_STATE,
UnitVisitStateEnum.ACTIVE))

```



```

        .addDqField(UnitField.UFV_UNIT_CATEGORY)
        .addDqField(UnitField.UFV_UNIT_ID)
        .addDqField(UnitField.UFV_LINE_OPERATOR);
    return doScalarQuery(inUserContext, inOutMessageCollector, dq);
}
}

```

6.3.2.3 Error handling

Example 1:

```

[
  {
    "level": "SEVERE",
    "error":
"ERROR_CODE_EXTENSION_EXECUTION_FAILED:Parameters=NO_EXTENSION;",
    "message": "Custom code extensions failed to execute. Please contact
your administrator with the following error: NO_EXTENSION",
    "timestamp": 1612503001782,
    "status": 400
  }
]

```

Example 2:

```

[
  {
    "level": "SEVERE",
    "error":
"INVALID_REQUEST:Parameters=Reeferapi_v2$_handleRestApiRequest_closure1@64fc1
4f1;",
    "message": "INVALID_REQUEST",
    "timestamp": 1612503254636,
    "status": 400
  }
]

```

Example 3:

```

[
  {
    "level": "SEVERE",
    "error":
"INVALID_REQUEST:Parameters=Testapi_v1$_handleRestApiRequest_closure1@264cf2f
7;",
    "message": "INVALID_REQUEST",
    "timestamp": 1612863596102,
    "status": 400
  }
]

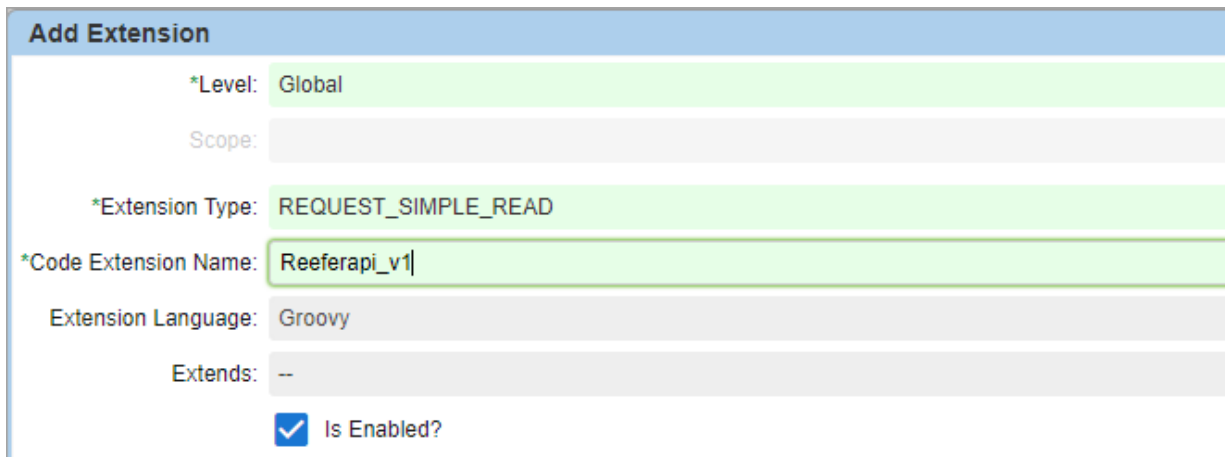
```

6.3.3 Deploying the Groovy REST API

When the Groovy script is ready to be deployed, add it to N4 in the **Code Extensions** view.

To deploy the REST API Groovy:

1. In N4, open the **Code Extensions** view.
2. Click the  Add button.
3. Enter the following information:
 - Level: Global
 - Extension Type: REQUEST_SIMPLE_READ
 - Code Extension Name: (Name of your Groovy script)
 - Extension Language: Groovy



4. Click Save.
5. Compile the script by choosing **Actions** → **Compile Code Extensions**.
N4 compiles the code extensions and displays the status and errors, if any, in the **Summary of Code Extensions Compilation** form.

6.3.4 Invoking the Groovy REST API

Get a Security Token

Before calling the REST API, you need to obtain a security token must be obtained from N4. This token must then be sent as an authorization header bearer type token on all subsequent requests. If the token expires the refresh token can be used to obtain a new one. See [Authentication for the REST API \(on page 160\)](#).

Call the REST API

The N4 service n4restapiservice allows you to access your Groovy REST API using a URL with the following format:

```
https://[hostname]:[port]/apex/n4restapiservice/[apiname]/[apiversion]/[apiresource]
```

Parameter	Description
<code>https://[hostname]:[port]</code>	This is the server serving the API
<code>/apex/n4restapiservice/reeferapi/v1</code>	This is the Base Path which is further broken down
<code>/apex</code>	The N4 web application
<code>/n4restapiservice</code>	The generic API servlet that serves the REST calls (GET, POST, PUT, DELETE).
<code>/[apiname]/[apiversion]</code>	The Groovy code extension which the n4restapiservice servlet changes into a Groovy code extension name by replacing the / with an underscore, and the first letter is capitalized. Example: <code>/reeferapi/v1</code> would map to a Groovy script called <code>Reeferapi_v1.groovy</code> . This will also allow two versions of an API to be deployed as two different Groovy files if necessary and both can be served at the same time.
<code>/[apiresource]</code>	The standard REST endpoint resource(s). Example: <code>/reefer/{reeferid}?myproperty=myvalue</code>

The following is an example of making a RESTful request using Postman.

`http://localhost:8280/apex/n4restapiservice/testapi/v1/units`

The screenshot shows the Postman interface with a GET request to `http://localhost:8280/apex/n4restapiservice/testapi/v1/units`. The response status is 200 OK, with a time of 287 ms and a size of 2.42 MB. The response body is displayed in JSON format, showing an array of units.

```

{
  "content": [
    {
      "id": "APZU3045179",
      "category": "UnitCategoryEnum[TRSHP]",
      "line-op": "2102"
    },
    {
      "id": "APZU3047090",
      "category": "UnitCategoryEnum[TRSHP]",
      "line-op": "2102"
    },
    {
      "id": "APZU3048202",
      "category": "UnitCategoryEnum[TRSHP]",
      "line-op": "2102"
    },
    {
      "id": "APZU4285370",
      "category": "UnitCategoryEnum[TRSHP]",
      "line-op": "2102"
    },
    {
      "id": "APZU4280721",
      "category": "UnitCategoryEnum[TRSHP]",
      "line-op": "2102"
    },
    ...
  ]
}

```

6.4 Security for the REST API

To access N4, you need to be authenticated as a user. The REST Authentication API supports basic or token authentication for user access.

- **Basic authentication:** API user credentials are supplied using standard basic authorization header and scope parameters are supplied as request parameters.
- **Navis Token authentication:** API user credentials are used to retrieve a token from N4, and this token is passed a *bearer* token to access an API in N4. This method is more secure.

A token is a piece of code sent from the API to the application that is requesting access. Access tokens provide authentication for the application to access an API. The client application is the bearer of the token and can use this token as credential when calling the N4 API. Bearer tokens provide authentication for the users who are making requests to the API.

Navis recommends that you use token authentication. When the REST Authentication API receives a login request, it generates a string of code which is the bearer token. The client sends this string in the Authorization header when making requests to the API.

Token expiration

N4 stores the tokens in the database. The token duration times can be overridden through the following Java system properties:

- **access.token.expiry** (default 30 minutes)
- **refresh.token.expiry** (default 60 minutes)

To override the default token duration:

1. On the N4 node, find the Tomcat service (for example, `n4clusterw.exe`). (The default location in Windows is `C:\Program Files\Navis\sparcsn4\bin.`)
2. Right-click on the file and select 'Run as administrator.'
3. Select the **Java** tab. Under **Java Options**, then add the following options to the JVM parameters with the value in minutes. For example, to change the access token duration to 60 minutes and the refresh access token to 90 minutes, add the following:

```
-Daccess.token.expiry=60  
-Drefresh.token.expiry=90
```

4. Click OK.
5. Restart the N4 service.

6.4.1 Refreshing an access token after it expires

While requesting an authentication endpoint for access token, you'll see the following details:

- `expires_in` - number of seconds remaining until the access token expires
- `refresh_token`

You can use the refresh token to get a new access token before the refresh token expires.

To refresh an access token, call the following URL with `response_type=token`.

Refresh token endpoint:

```
curl -v --location -X POST "https://SERVERNAME/n4/restAuthenticate" \
--header "Content-Type: application/x-www-form-urlencoded" \
--data-urlencode "response_type=token"
--data-urlencode
"refresh_token=eyJhbGciOiJIUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJhNDC4ZTI1MC0w"--data-urlencode "grant_type=refresh token"
```

The above endpoint produces same payload as authentication endpoint response.

6.4.2 REST Authentication API

The REST Authentication API provides N4 user authentication service.

When invoking RESTful services, Navis recommends that you use an SSL connection.

HTTP Response Codes

Response	Description
200	Success
401	Authentication failure
403	Required request parameters are missing

Supported response types and the various responses that are generated by this API:

response_type	Response (JSON payload)
scope (default type)	All allowed under user scope coordinates.
token	User access token
none	Empty JSON payload

Example: When `response_type=token` is sent in the request, It returns an empty JSON payload with HTTP status code.

URL

http://SERVERNAME:8280/apex/restAuthenticate

INPUT Parameters

Request Method: POST

Headers:

Content-Type: application/x-www-form-urlencoded

Request Parameters:

Parameter	Description	Required
username	N4 user that has the EXTENSION_CODE_EXECUTIO N_VIA_RESTprivilege	Required

Parameter	Description	Required
password	N4 password	Required
operatorId	N4 operator	Optional Required when logging in at yard level.
complexId	N4 complex	Optional Required when logging in at yard level.
facilityId	N4 facility	Optional Required when logging in at yard level.
yardId	N4 yard	Optional Required when logging in at yard level.
response_type	<scope token none> default: scope	Optional
refresh_token	refresh token	Optional This does not require user credentials or scope details.

6.4.3 Use Cases for the REST Authentication API

Use case 1: Fetch all the available scopes for a user in N4

`response_type=scope` [default]

- Successful authentication (returns 200 along with the complete JSON response in the Sample JSON Response section):

The screenshot shows a REST client interface for a REST API named 'RestAuth'. The request is a POST to 'http://localhost:8280/apex/restAuthenticate' with the following parameters:

- `username`: scott
- `password`: tiger
- `response_type`: none

The response is a 200 OK status with a time of 84ms and a size of 1003 B. The JSON response is as follows:

```

{
  "scope": {
    "operators": [
      {
        "complexes": [
          {
            "facilities": [
              {
                "yards": [
                  {
                    "gkey": 1,
                    "id": "VRD1111"
                  },
                  {
                    "gkey": 1,
                    "id": "FCV111"
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}

```

- Authentication failure (returns 403):

RestAuth Examples (0)

POST http://localhost:8280/apex/restAuthenticate Send Save

Params Authorization Headers (10) **Body** Pre-request Script Tests Cookies Code Comments (0)

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL BETA

KEY	VALUE	DESCRIPTION	*** Bulk Edit
☒ username	scott		
☒ password	lion		
☐ response_type	none		
Key	Value	Description	

Body Cookies (2) Headers (7) Test Results Status: 403 Forbidden Time: 20ms Size: 268 B Save Response

Pretty Raw Preview JSON 🔍

1 { }

- Sample JSON Response with user having access to multiple scopes:

```
{
  "scope": {
    "operators": {
      "complexes": {
        "facilities": {
          "yards": [
            { "gkey": 1, "id": "YRD1111" },
            { "gkey": 1, "id": "FCY111" },
            { "gkey": 3, "id": "YRD1121" },
            { "gkey": 3, "id": "FCY112" },
            { "gkey": 4, "id": "YRD1131" },
            { "gkey": 4, "id": "FCY113" },
            { "gkey": 2, "id": "YRD1112" },
            { "gkey": 5, "id": "FCY114" },
            { "gkey": 5, "id": "YRD1141" },
            { "gkey": 6, "id": "FCY115" },
            { "gkey": 9, "id": "NEK_FCY" },
            { "gkey": 10, "id": "NEK_RLY" },
            { "gkey": 1, "id": "CPX11" },
            { "gkey": 6, "id": "YRD1211" },
            { "gkey": 7, "id": "FCY121" },
            { "gkey": 2, "id": "CPX12" },
            { "gkey": 1, "id": "OPR1" },
            { "gkey": 8, "id": "YRD2112" },
            { "gkey": 8, "id": "FCY211" },
            { "gkey": 3, "id": "CPX21" },
            { "gkey": 2, "id": "OPR2" }
          ]
        }
      }
    }
  }
}
```

- Sample JSON Response with user having access to only one scope

```
{
  "scope": {
    "operators": [
      {
        "complexes": [
          {
            "facilities": [
              {
                "yards": [
                  { "gkey": 1, "id": "YRD1111" },
                  { "gkey": 1, "id": "FCY111" },
                  { "gkey": 1, "id": "CPX11" },
                  { "gkey": 1, "id": "OPR1" }
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}
```

Use case 2: Fetch a token to invoke a RESTful API in N4 (using user credentials)

response_type=token

- Successful authentication with access token in response:

The screenshot shows the RestAuth interface for a POST request to `http://localhost:8280/apex/restAuthenticate`. The request body is `x-www-form-urlencoded` with the following parameters:

KEY	VALUE	DESCRIPTION
☒ username	scott	
☒ password	tiger	
☒ operatorId	OPR1	
☒ complexId	CPX11	
☒ facilityId	FCY111	
☒ yardId	YRD1111	
☒ response_type	token	

The response status is **200 OK** with a time of **6ms** and size of **384 B**. The response body is JSON:

```
1 {
2   "accessToken": "166437ec-d35b-4cca-833a-573e48f5e46d",
3   "expiresIn": 1800,
4   "refreshToken": "cadd61b0-a407-45d1-b176-69bc35c045f6"
5 }
```

Use case 3: Fetch a new token to invoke a RESTful API in N4 (using the refresh token)

response_type=token (refresh_token flow)

- When the access token expires, use the refresh token to fetch the new access token:

The screenshot shows the RestAuth interface for a POST request to `http://localhost:8280/apex/restAuthenticate`. The request body is `x-www-form-urlencoded` with the following parameters:

KEY	VALUE	DESCRIPTION
☒ response_type	token	
☒ refresh_token	cadd61b0-a407-45d1-b176-69bc35c045f6	

The response status is **200 OK** with a time of **9ms** and size of **384 B**. The response body is JSON:

```
1 {
2   "accessToken": "4b064a27-9b97-4d13-aa8f-b69c20bfc58",
3   "expiresIn": 1800,
4   "refreshToken": "d394603b-9cff-439d-8e14-1c0f220a9b80"
5 }
```

Use case 4: Check a valid user in N4

`response_type=none`

- Successful authentication:

POST `http://localhost:8280/apex/restAuthenticate` Send Save

Params Authorization Headers (10) Body Pre-request Script Tests Cookies Code Comments (0)

none form-data x-www-form-urlencoded raw binary GraphQL BETA

KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒ username	scott			
☒ password	tiger			
☒ response_type	none			
Key	Value	Description		

Body Cookies (1) Headers (7) Test Results Status: 200 OK Time: 133ms Size: 261 B Save Response

Pretty Raw Preview JSON

1

6.4.4 Check Privileges API

The Check Privileges API validates if the user has the list of privileges sent in the payload. It returns a JSON object with details such as whether the user has corresponding privileges or not.

Access to this API requires valid user credentials (passed as either basic authentication or a bearer token).

HTTP Response Codes

Response	Description
200, along with JSON payload	Success
400	Invalid payload
401	Authentication failure

URL

`http://SERVERNAME:8280/apex/checkPrivileges`

INPUT Parameters

Request Method: POST

Headers

Content-Type: `application/json`

A valid token (basic or bearer) is required to invoke this service.

Sample Request

```
{
  "privileges": ["CUSTOM_NODE_MOBILE_CONFIG_ADMIN",
    "non_existent_privilege"]
}
```

Sample Response

```
{
  "privileges": {
    "CUSTOM_NODE_MOBILE_CONFIG_ADMIN": true,
    "non_existent_privilege": false
  }
}
```

6.4.5 Invoking N4 RESTful API

This section provides examples of how to invoke a RESTful API. The [Check Privileges API \(on page 165\)](#) is a RESTful API in N4, so you could follow similar steps to pass the credentials to invoke any RESTful API in N4.

Access to this RESTful API requires valid user credentials passed as either basic authentication credentials or bearer token.

URL

http://SERVERNAME:8280/apex/<a-restful-service>

INPUT Parameters

Pass the input parameters as required by the RESTful service.

Use case 1: How to invoke the RESTful API using basic authentication

When using the basic authentication method, scope parameters should be passed in the request each time an API is invoked.

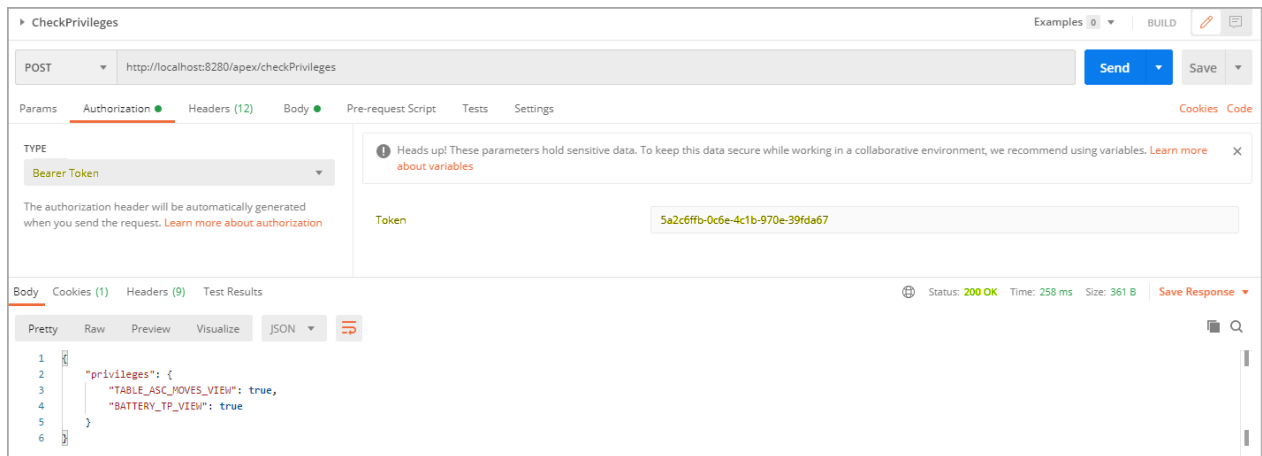
User credentials is passed with the basic authorization header:

The screenshot shows the N4 REST client interface for a POST request to `http://localhost:8280/apex/checkPrivileges`. The request is configured with Basic Authentication, with the username `admin` and a masked password. The response status is `200 OK` with a time of `128 ms` and a size of `311 B`. The response body is displayed in JSON format:

```
{
  "privileges": {}
}
```

Use case 2: How to invoke the RESTful API using an access token

The access token is passed as a bearer token:



6.5 CHE Current Position API

URL

<http://localhost:8280/apex/n4restapiservice/chetelemetry/v1/ches/currentposition>

INPUT Parameters

Request Method: POST

Headers:

Content-Type: application/xml

A valid token (basic or bearer) is required to invoke this service.

Authentication:

To access N4, you need to be authenticated as a user. This N4 REST API uses basic authentication for user access. API user credentials are supplied using standard basic authorization header and scope parameters are supplied as request parameters.

Request Parameters:

Parameter	Description
operatorId*	Name of the operator in topology
complexId*	Name of the complex in topology
facilityId*	Name of the facility in topology
yardId*	Name of the yard in topology

XML raw data format example to invoke CHE current position API.

```
<message type="2635" MSID="152" eventTime="2022-04-08 16:46:01.000">
```

```
<che CHID="R49">
```

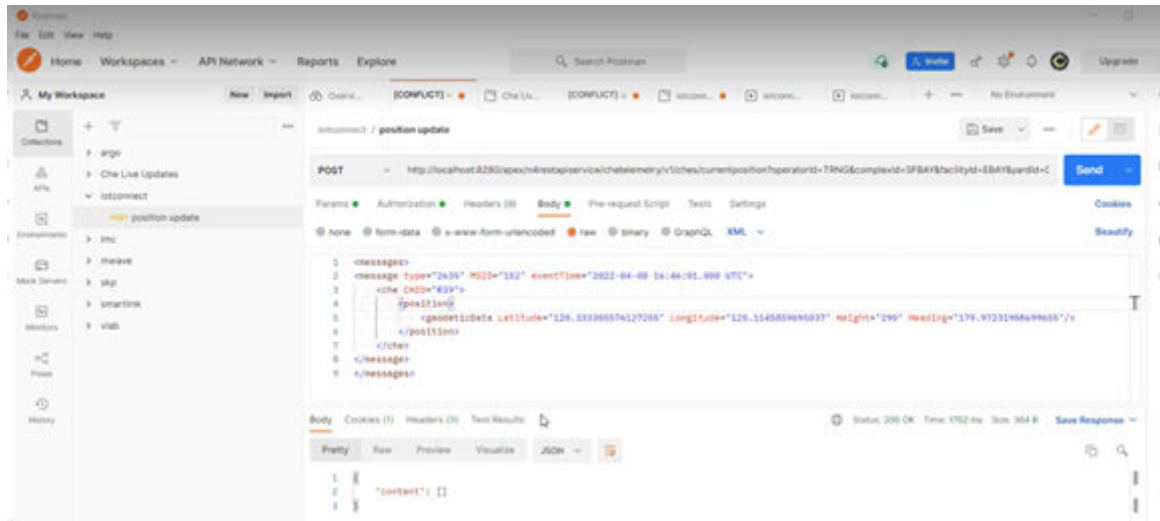
```
<position>
```

```
<geodeticData Latitude="35.333305576127255" Longitude="114.1145859695037"
Height="290" Heading="179.97231908699655"/>
```

```
</position>
```

```
</che>
```

```
</message>
```



Response: 200 - OK

To enable API access:

1. Enable the setting FRMCARINA032 (CODE_EXTENSION_BY_REST_ENABLE).
 - a. In N4, open the **Settings** view (**Administration -> Settings -> Settings**).
 - b. Change the setting to true.
2. Define a user role that includes the privileges called **Execute Code Extensions via REST Protocol** and **Che Telemetry**.
 - a. Open the **Roles** view (**Administration -> Security -> Roles**).
 - b. Include **Execute Code Extensions via REST Protocol**.
 - c. Include **Che Telemetry**.
 - d. Include **Code Extensions**.

2. Add headers, including the hyphen-separated scope keys. For example, Facility-ID, Operator-ID or Complex-ID.

POST http://localhost:8081/n4/n4restapiservice/billingapi/v1/release-payment-hold

Params Authorization Headers (14) Body Pre-request Script Tests Settings Cookies Code

Headers 10 hidden

KEY	VALUE	DESCRIPTION
Facility-ID	rwg	
Operator-ID	dpw	
Key	Value	Description

Body Cookies Headers (7) Test Results 200 OK 106 ms 261 B Save Response

Pretty Raw Preview Visualize

```
{"content":[]}
```

Then the valid response should be displayed.

3. In the Body section, select raw and add a valid JSON request.

POST http://localhost:8081/n4/n4restapiservice/billingapi/v1/update-payment-hold

Params Authorization Headers (14) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL Text

```
1 {
2   {
3     "entity": "ChargeableUnitEvent",
4     "gkeys": [11],
5     "data": [
6       {
7         "name": "payment-hold-action",
8         "type": "String",
9         "old": "null",
10        "new": "apply-payment-hold"
11      }
12    ]
13  },
14  {
15    "entity": "ChargeableUnitEvent",
16    "gkeys": [10],
17    "data": [
18      {
19        "name": "payment-hold-action",
20        "type": "String",
```

6.7 Billing REST Endpoints

The following are examples of using the Billing API.

Use case 1: Update Invoice status for both chargeable marine events and chargeable unit events

URL: <http://SERVERNAME:8081/n4/n4restapiservice/billingapi/v1/update-invoice-status>

Operation: POST

JSON raw data format example to invoke the update-invoice-status API.

```
[
  {
    "entity": "ChargeableUnitEvent",
    "gkeys": [10,16],
    "data": [
      {
        "name": "invoice-status",
        "type": "String",
        "old": "null",
        "new": "INVOICED"
      }
    ]
  },
  {
    "entity": "ChargeableMarineEvent",
    "gkeys": [1,3],
    "data": [
      {
        "name": "invoice-status",
        "type": "String",
        "old": "null",
        "new": "QUEUED"
      }
    ]
  }
]
```

The screenshot displays a REST client interface with a POST request to `http://localhost:8081/n4/n4restapiservice/billingapi/v1/update-invoice-status`. The request body is a JSON array of two objects, each representing an event with invoice status updates. The interface shows the 'Body' tab selected, displaying the raw JSON data. Below the request, the response is shown with a status of 200 OK, indicating a successful update.

POST `http://localhost:8081/n4/n4restapiservice/billingapi/v1/update-invoice-status` Send Save

Params Authorization Headers (14) Body Pre-request Script Tests Settings Cookies Code

none form-data x-www-form-urlencoded raw binary GraphQL Text

```
1 [
2   {
3     "entity": "ChargeableUnitEvent",
4     "gkeys": [1033,1034],
5     "data": [
6       {
7         "name": "invoice-status",
8         "type": "String",
9         "old": "QUEUED",
10        "new": "INVOICED"
11      }
12    ]
13  },
14  {
15    "entity": "ChargeableMarineEvent",
16    "gkeys": [1,3],
17    "data": [
18      {
19        "name": "invoice-status",
20        "type": "String",
21        "old": "null",
22        "new": "QUEUED"
23      }
24    ]
25  }
26 ]
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 681 ms Size: 303 B Save Response

Pretty Raw Preview Visualize

```
{
  "content": {
    "status": "SUCCESS",
    "messages": []
  }
}
```

Use case 2: Update delivery date for unit facility visit flex field API

URL: `http://SERVERNAME:8081/n4/n4restapiservice/billingapi/v1/update-delivery-date`

Operation: POST

JSON raw data format example to invoke update-delivery-date API.

(The delivery-date format is dd/mm/yyyy.)

```
[
{
  "entity": "UnitFacilityVisit",
  "gkeys": [9889],
  "data": [
    {
      "name": "delivery-date",
      "type": "String",
      "old": "null",
      "new": "01/02/2021"
    }
  ]
},
{
  "entity": "UnitFacilityVisit",
  "gkeys": [9889],
  "data": [
    {
      "name": "delivery-date",
      "type": "String",
      "old": "null",
      "new": "03/02/2021"
    }
  ]
}
]
```

POST http://localhost:8081/n4/n4restapiservice/billingapi/v1/update-delivery-date

Send Save

Params Authorization Headers (14) Body Pre-request Script Tests Settings Cookies Code

none form-data x-www-form-urlencoded raw binary GraphQL Text

```
1 [
2 {
3   "entity": "UnitFacilityVisit",
4   "gkeys": [9889],
5   "data": [
6     {
7       "name": "delivery-date",
8       "type": "String",
9       "old": "null",
10      "new": "01/02/2021"
11     }
12   ]
13 }
14 ]
```

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 1090 ms Size: 293 B Save Response

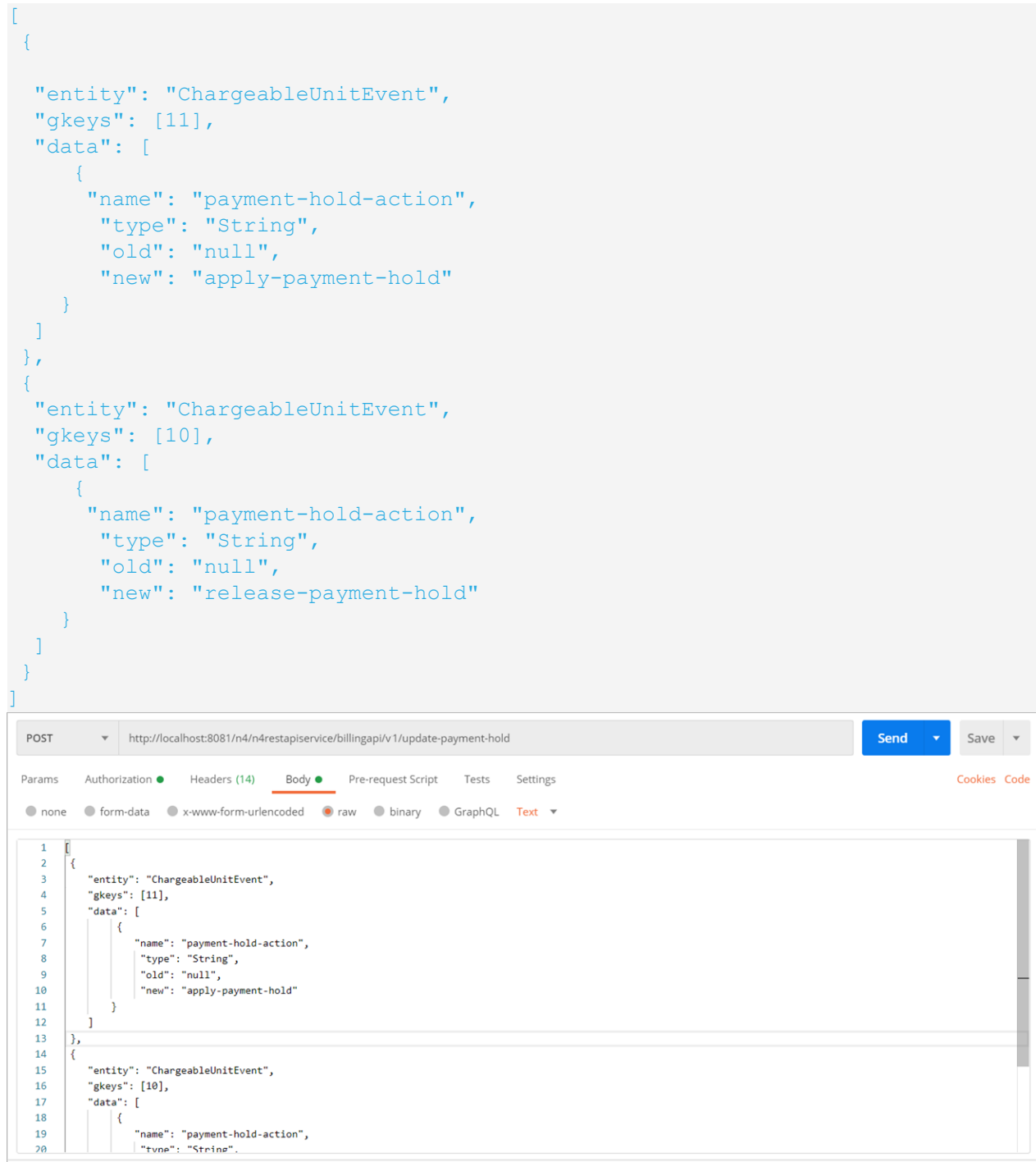
Pretty Raw Preview Visualize

Use Case 3: Update payment holds for unit API

URL: <http://SERVERNAME:8081/n4/n4restapiservice/billingapi/v1/update-payment-hold>

Operation: POST

JSON raw data format example to invoke update-payment-hold API



6.8 Service Order API

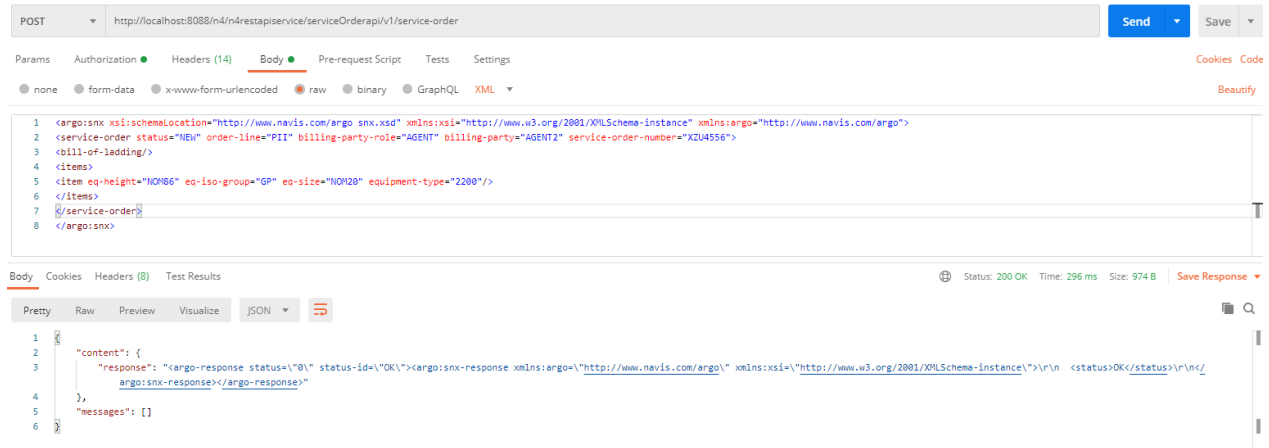
URL `http://SERVERNAME:8081/n4/n4restapiservice/serviceOrderapi/v1/service-order`

Operation: *POST*

Accepting XML raw data



Output response example



XML raw data format example to invoke service-order API

Service order snx

```
<argo:snx xsi:schemaLocation="http://www.Navis.com/argo snx.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:argo="http://
www.Navis.com/argo">
<service-order status="NEW" order-line="PII" billing-party-role="AGENT"
billing-party="AGENT2" service-order-number="XZU4556">
<bill-of-ladding/>
<items>
<item eq-height="NOM86" eq-iso-group="GP" eq-size="NOM20" equipment-
type="2200"/>
</items>
</service-order>
</argo:snx>
```

Accepting JSON raw data (Works with N4 3.7 and higher, except on 3.7.10)

POST ▼ http://localhost:8088/n4/n4restapiservice/serviceorderapi/v1/service-order Send ▼

Params Authorization Headers (15) **Body** Pre-request Script Tests Settings Cookies Beautify

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```

1 {
2   "service-orders": [
3     {
4       "service-order-number": "TEST128",
5       "billing-party": "815177926",
6       "billing-party-role": "HAULIER",
7       "order-line": "APL",
8       "start-date": "Feb 21, 2021 2:00:23 PM",
9       "completion-date": "Feb 28, 2021 7:24:23 PM",
10      "notes": "Created in serviceOrderapi_v1",
11      "status": "NEW"
12    },
13  ],
14 }

```

Body Cookies (1) Headers (7) Test Results Status: 200 OK Time: 942 ms Size: 460 B Save Response

Pretty Raw Preview Visualize XML ▼ screen

```

1 <argo-response status="0" status-id="OK">
2   <argo:snx-response xmlns:argo="http://www.navis.com/argo" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
3     <status>OK</status>
4   </argo:snx-response>

```

Runner

JSON raw data format example to invoke **service-order** API

Simple JSON raw data example

```

{
  "service-orders": [
    {
      "service-order-number": "TEST128",
      "billing-party": "815177926",
      "billing-party-role": "HAULIER",
      "order-line": "APL",
      "start-date": "Feb 21, 2021 2:00:23 PM",
      "completion-date": "Feb 28, 2021 7:24:23 PM",
      "notes": "Created in serviceOrderapi_v1",
      "status": "NEW"
    },
    {
      "service-order-number": "TEST129",
      "billing-party": "815177926",
      "billing-party-role": "HAULIER",
      "order-line": "APL",
      "start-date": "Mar 21, 2021 4:00:23 PM",
      "completion-date": "Mar 28, 2021 10:24:23 PM",
      "notes": "Created in serviceOrderapi_v1",
      "status": "NEW"
    },
    {
      "service-order-number": "TEST130",
      "billing-party": "815177926",
      "billing-party-role": "HAULIER",
      "order-line": "APL",
      "start-date": "Feb 1, 2021 6:00:23 PM",
      "completion-date": "Feb 12, 2021 4:24:23 PM",
      "notes": "Created in serviceOrderapi_v1",
      "status": "NEW"
    }
  ]
}

```

```
    "status": "NEW"
  }
]
}
```

Complex JSON raw data example

```
{
  "service-orders": [
    {
      "service-order-number": "SR05000",
      "billing-party": "AONE",
      "billing-party-role": "AGENT",
      "order-line": "LONE",
      "completion-date": "Feb 28, 2021 9:24:23 PM",
      "notes": "Created in serviceOrderapi_v1",
      "status": "NEW",
      "items": [
        {
          "eq-height": "NOM86",
          "eq-iso-group": "GP",
          "equipment-type": "2200",
          "eq-size": "NOM20",
          "service-types": [
            {
              "units": [
                {
                  "equip-nbr": "TEST1234567",
                  "unit-status": "NEW"
                },
                {
                  "equip-nbr": "TEST1234568",
                  "unit-status": "INPROGRESS"
                },
                {
                  "equip-nbr": "TEST1234569",
                  "unit-status": "COMPLETED"
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

Part 7

Using SNX

This section describes the SNX data import/export feature of N4.

In this Section:

What is SNX?	178
Special Considerations: <unit> element.	183
Special Considerations: other Elements.	189
Unit updates and transactions on Units with SNX.	196
Signaling Activities with SNX.	202
Initial data conversion via SNX.	205
Disabling XPSCache Cluster Loading for Data Migration.	209
Re-organization of the SNX unit element.	210
Event Processing Via JMS for N4.	215
ActiveMQ Configuration.	222

7.1 What is SNX?

In This Section

Introduction.	179
Structure of an SNX document.	179
Timestamps.	180
Supported Entities.	181

7.1.1 Introduction

SNX is an acronym for "N4 eXchange". SNX is a family of API's for interacting with N4 based on XML documents. SNX provides a means of importing (exporting) data into (out of) SN4, and also effecting certain transactions through messages into the system. In this document we will refer to an XML Document that conforms to the SNX schema as an "SNX Document". The main uses of SNX are envisaged to be:

- Initial data conversion of legacy system data into SN4 at system implementation.
- A low-level API for directly manipulating the state of SN4.
- A means to create test data for scripted test scenarios.
- Exporting subsets of data from live systems to synchronize foreign systems and for off-line analysis.

Both reference data (routing points, vessels, equipment, ...) and working data (units, vessel visits, bookings, ...) can be transmitted via SNX. Incoming SNX can be used to both create and update entities in SN4, and to perform certain key transactions on these entities.

There are three ways that SNX-formatted XML documents can be imported into SN4:

- SNX Data Importer: a function of N4 allowing an SNX document in the form of an xml text file to be uploaded into the system.
- JMS: SNX documents are sent in JMS messages to the configured JMS portal for a Facility.
- Web Services: SNX documents are sent as the payload of the SN4 generic WebService.

See also, [SNX Using WebServices \(on page 122\)](#).

7.1.2 Structure of an SNX document

A well-formed SNX document has "snx" as the top level element followed by one or more elements describing an entity within SN4. For example:

```
<?xml version="1.0" encoding="UTF-8"?>
<argo:snx xmlns:argo="http://www.Navis.com/ ... >
<routing-point id="AKL" transit-to-mode="VESSEL" ... />
<routing-point id="NSN" transit-to-mode="VESSEL" ... />
<carrier-itinerary id="NASI">
<...>
</carrier-itinerary>
<carrier-service id="NASI" carrier-mode="VESSEL" itinerary-id="NASI">
<...>
</carrier-service>
<vessel id="OOC" lloyds-idd="9159854" name="OOCL ACCLAIM" vessel-class="OOC" ... >
<...>
</vessel>
<vessel-visit id="07008" ... >
<...>
</vessel-visit>
<unit id="PSRU4513568" category="IMPORT" visit-state="ADVISED" freight-kind="FCL">
<...>
</unit>
<unit id="PSRU2858488" category="EXPORT" visit-state="YARD" freight-kind="FCL">
<...>
</unit>
```

```
</argo:snx>
```

Each top-level nested element (routing-point, carrier-service, unit, etc.) describes an SN4 entity instance of the type indicated by the element name. The structure of each of these nested elements is specified by an XML-schema, snx.xsd.

When sending SNX into N4 via JMS, you must include just one nested element in each snx document

The scope of an SNX Document (i.e. Yard, Facility, Complex and Operator) is not specified directly in the Document itself. Instead, SNX is always sent or received within the context of a Facility.

Many attributes within the SNX schema are optional. When updating an existing entity through SNX, the following rules apply to optional attributes:

- If an optional attribute is not supplied in the SNX, then the current value of that property in the entity being updated is left as-is.
- If an optional attribute is sent with an empty value (e.g. designated-trucker="") then that property of the entity is set to null.
- If an optional attribute is sent with a non-empty value (e.g. designated-trucker="ACME") then that property of the entity is updated to the value provided..

Certain SNX elements have collections of sub-elements to model entities that have child collections. In these cases, SNX is structured with a containing element that encapsulates the repeating element. For example, a unit may have multiple hazardous items within. The "hazards" element is the containing element:

```
<unit id="PSRU4513568" category="IMPORT" visit-state="ADVISED" freight-kind="FCL">
  <hazards>
    <hazard imdg="1.2" un="1203" tech-name="GASOLINE" page-number="13"/>
    <hazard imdg="4.1" un="6789" tech-name="PEROXIDE" page-number="45"/>
    <hazard imdg="6.1" un="5634" ltd-qty-flag="Y" package-type="BOX" .../>
  </hazards>
</unit>
```

When updating an existing entity that has child collections through SNX, the following rules apply:

- If the containing element is not supplied in the SNX, then the current child collection of the entity is left as-is.
- If the containing element is sent with no sub-elements (e.g. <hazards/>) then the child collection of the entity is cleared.
- If the containing element is sent with sub-elements then the entire child collection is replaced with the values provided.

A slight extension of these rules is supported for some child collections, such as the trucking-company line agreements. These entities have an optional attribute "update-mode" which adds this behavior:

- If update-mode is not specified or is set to "REPLACE", then the behavior is as noted above.
- If update-mode="MERGE" is specified, then the child SNX elements are used to update existing child entities in the model, or to create a new row when no matching child entity exists.
- If the containing element is sent with no sub-elements and update-mode="MERGE" is specified, the child collection is left as-is.

Some examples of using this are provided with reference to the trucking-company element below.

7.1.3 Timestamps

All date/time attributes in SNX messages should not include a timezone. The values should be sent in the local time of the facility in which the associated event is to occur (e.g. ETA is sent in the local time of the terminal at which the

vessel is arriving). If a timezone is included in the timestamp, the message will be rejected. Following is an example of a properly formed timestamp: eta="2007-09-12T11:12:00"

7.1.4 Supported Entities

The entities that may be imported via SNX specified in the xml schema file snx.xsd. The entities can be categorized into "reference entities" and "working entities". All entities can be specified in a single snx xml file, the schema defines an order for the entities to ensure that dependencies are satisfied (i.e. reference entities before working entities). The following sections identify the entities supported and the top level xml tag from where they are sourced.

7.1.4.1 Reference Entities

Reference entities are the entities at the fringes of the SN4 domain model. These are typically set up once, then changed or added to infrequently. They do not have a temporal lifecycle.

The following **Reference Entities** may be created through SNX.

Element Name	SN4 Entity	Scope	Notes
<routing-point>	RoutingPoint	user-defined	Defines locations that serve as discharge ports, load ports, destinations, etc.
<carrier-itinerary>	CarrierItinerary	user-defined	Defines a sequence of routing points that a carrier will visit.
<carrier-service>	CarrierService	user-defined	Defines a regular deployment of a carrier, includes a pro-forma itinerary.
<vessel>	Vessel	user-defined	Defines specific ship or barge.
<trucking-company>	TruckingCompany	user-defined	Defines trucking companies that dray containers to/from the facilities, including their restrictions and line agreements.
<truck-driver>	TruckDriver	user-defined	Defines truck drivers for a trucking company
<equipment>	Equipment	user-defined, usually global	Defines containers, chassis, accessories. Currently Equipment are only created through child-elements of the <unit> element.
<shipper-consignee>	Shipper	user-defined, usually global	Defines businesses that act as shippers or consignees.
<customer>	Customer	user-defined	Defines a customer in the Billing Application. (Line Operator, Trucking Company, etc.)

7.1.4.2 Working Entities

These are the principal **Operational Entities** of the system. They all have a time-limited lifecycle.

Element Name	SN4 Entity	Scope	Notes
<unit>	UnitFacilityVisit	Facility	Also results in creation of Unit, UnitEquipment, and related child entities.
<vessel-visit>	VesselVisit	Complex	The VesselVisit is scoped by Complex, although it has a Facility property indicating where the vessel is to call.
Booking	Booking	Complex	Defines Bookings and Booking Items.
<bill-of-lading>	BillOfLading	Complex	Defines the Goods within a Unit.

7.2 Special Considerations: <unit> element

The element name <unit> is used to create/update the key N4 entity **Unit**. As a review, a Unit is an item or items of inventory that constitutes as a single moveable object. Typically this will be a container but it also covers chassis, container(s) on a chassis, accessories and bundles. Included with the inventory unit are details of the visit of the unit to the facility including inbound and outbound carriers, equipment associated with the unit (eg physical details of the container), any damages recorded, details of the goods/contents, hazards and refrigeration requirements.

In This Section

Data Import Mode vs. Production Mode.	184
Data Import Mode.	184
Production Mode.	184
Some notes.	185
Examples.	185

7.2.1 Data Import Mode vs. Production Mode

There are two ways that SNX <unit> elements can be processed by N4:

- **Data Import Mode** - this is used for importing data into N4 to initialize the system. This mode *is not* intended to be used when the system is LIVE.
- **Production Mode** - this is used for advising the live, production system of Units that will be arriving at the Complex.

You instruct N4 which mode to use by inclusion/exclusion of the "transit-state" attribute. If the transit-state is defined, N4 uses Data Import Mode. If the transit-state is omitted, it uses Production Mode.

7.2.2 Data Import Mode

In Data Import Mode, N4 will create (or update an existing) Unit in the transit state specified in the SNX. This allows you to create ACTIVE, ADVISED and even DEPARTED/RETIRED Units. The logic that is used to determine whether to update an existing Unit or to create a new Unit is as follows:

```
If a unique-key is specified then
  Search for a Unit having this key
If no Unit is determined by the above then:
  If the transit-state is ADVISED then:
    try to find an ADVISED Unit using Equipment 'eqId'
  else if the transit-state is an ACTIVE state (INBOUND, EC-IN, YARD, EC-OUT, LOADED)
then:
  try to find an ACTIVE Unit using Equipment 'eqId'. If none found, then:
    try to find an ADVISED Unit using Equipment 'eqId'
If no Unit is determined by the above then:
  create a new Unit using the 'eqId', and put it into the specified transit-state.
```

7.2.3 Production Mode

In Production Mode, N4 will only accept SNX messages for Units that are *yet to arrive at the Complex*. You can not update Units that have been discharged or received in Production Mode. You can only create and update Units that are *still on their inbound carrier* in N4.

The logic is as follows:

If the SNX does not specify a position for the Unit aboard its Inbound Carrier then

```
Abort with error message
If a unique-key is specified then
  Search for a Unit having this key
If no Unit is determined by the above then:
  Search for a UnitFacilityVisit belonging to a Unit that is ACTIVE or ADVISED, having
the Declared Inbound Carrier
  specified in the SNX Message.
  If such a UFW exists, then
    If it is already in Transit State EC-IN or later (i.e., has been received or
discharged) then
      Do not update the Unit! Abort with error message
    Else
      Update this Unit (and its UFW) per the SNX
  Else
```

```

Create a new Unit and UFV per the SNX.
If the inbound carrier of this Unit is in phase CREATED
  Set the UFV Transit State to ADVISED, and the Unit Visit State ADVISED
Else (carrier is in phase INBOUND or later)
  If no other Unit is ACTIVE using the same Equipment as the new Unit
    Set the UFV Transit State to INBOUND, and the Unit Visit State ACTIVE
  Else (either the SNX is wrong, or the existing Unit is a phantom)
    Set the UFV Transit State to ADVISED, and the Unit Visit State ADVISED

```

7.2.4 Some notes

These are the fine points to be aware of when creating or updating a Unit via SNX:

- Currently only containerized Units can be created via SNX - no break bulk. 'Containerized' means that there is a primary equipment associated with the Unit and that equipment's ID is the Unit's ID. This includes individual containers (full or empty), containers on chassis, bare chassis, and bundles.
- When you create (or update) a Unit using SNX, you are creating both a Unit and a UnitFacilityVisit entity for that Unit at the Facility of the current context.
- When creating a new unit via SNX the primary equipment, freight kind, line operator and transit-state must be defined.
- When using SNX for initial data conversion, you can send transit-state "YARD" for a unit that is currently in the yard, and "DEPARTED" for historical Units.
- When updating an existing Unit via SNX, you can always specify a "unique-key" attribute to locate the existing record to be updated. The "unique-key" provides a means for external systems to uniquely identify units, disambiguating between the potential multiple Units using the same primary Equipment. **This overrides the logic described above for the two modes of use.**
- The CarrierVisits referenced in the train sub-elements **need not** be defined prior to importing a unit that references them.
- If the Unit will go on to another Facility in the Complex, you use the 'next-facility' attribute to specify the Facility which will handle it next. This will result in a second UnitFacilityVisit being created for the Unit. Also, if the Unit is on a barge or train between two Facilities, this attribute informs N4 that the unit is still ACTIVE, rather than DEPARTED. (This applies to Data Import Mode only.)
- You can attach hold and permission flags to any piece of Equipment comprising a Unit, or to the Goods of the Unit, with the <flags> element. In the case of a hold/permission on Equipment, the flag element must be defined as content of of equipment element to which the flags pertains.

7.2.5 Examples

Below is an example of a message to define a 'to-come' (Advised) export. As you normally will not know the specific truck that will bring the Unit, you can specify the generic truck for both the position and the inbound carrier as shown below.

```

<unit id="MATU3672143" category="EXPORT" transit-state="ADVISED" freight-kind="FCL"
line="MAT">
  <equipment eqid="MATU3672143" class="CTR" type="4200" role="PRIMARY" tare-
kg="3200.0" height-mm="2831" owner="POPEYE"/>
  <position loc-type="TRUCK"/>
  <routing opl="KAH" pol="HNL" pod-l="OAK" origin="MAKAWAO">
    <carrier direction="IB" qualifier="DECLARED" mode="TRUCK"/>
    <carrier direction="IB" qualifier="ACTUAL" mode="TRUCK"/>
    <carrier direction="OB" qualifier="DECLARED" mode="VESSEL" id="RJP056"/>
  </routing>
</unit>

```

```

    <carrier direction="OB" qualifier="ACTUAL" mode="VESSEL"/>
  </routing>
  <contents weight-kg="22000.0"/>
</unit>

```

Below is example of a message to define an incoming import. The position is a slot in the inbound carrier.

The example shows many of the optional elements and attributes in use.

```

<unit id="MATU2064601" transit-state="ADVISED" category="IMPORT" freight-kind="FCL"
line="MAT">
  <equipment eqid="MATU2064601" class="CTR" type="4200" role="PRIMARY" tare-
kg="1900.0" height-mm="2831"
    strength-code="AA" material="ALUMINUM" owner="ITT" operator="YBR">
    <damages>
      <damage type="BENT" component="DOOR" severity="MAJOR"/>
    </damages>
  </equipment>
  <position loc-type="VESSEL" location="RJP001" slot="010280"/>
  <routing pol="LAX" pod-1="KAH" pod-2="KHH" opl="HKG" origin="VENTURA"
destination="MAKAWAO"
    designated-trucker="ABC" carrier-service="USWC" group="HOT">
    <carrier direction="IB" qualifier="DECLARED" mode="VESSEL" id="RJP001"/>
    <carrier direction="IB" qualifier="ACTUAL" mode="VESSEL" id="RJP001"/>
    <carrier direction="OB" qualifier="DECLARED" mode="TRUCK"/>
    <carrier direction="OB" qualifier="ACTUAL" mode="VESSEL"/>
  </routing>
  <booking id="MAT6678435" line="MAT"/>
  <hazards>
    <hazard imdg="1.2" un="1203" tech-name="GASOLINE" page-number="13"/>
    <hazard imdg="4.1" un="6789" tech-name="PEROXIDE" page-number="45"/>
    <hazard imdg="6.1" un="5634" ltd-qty-flag="Y" package-type="BOX" inhalation-
zone="Z" explosive-class="B"
      page-number="32"
      flash-point="98.0" tech-name="TECH" proper-name="PROPER" ems-nbr="EMS"
mfag="MFAG"
      packing-group="II"
      haz-id-upper="UPPR" substance-lower="LWER" weight-kg="234.0" planner-
ref="PREF" quantity="5"
      move-method="MOVE"
      secondary-imo-1="1.4" secondary-imo-2="6.1" deck-restrictions="A" marine-
pollutants="Y"
      dc-lg-ref="LC" emergency-telephone="911"
      ems-response-guide-nbr="001" notes="-hazard notes-"/>
  </hazards>
  <reefer temp-reqd-c="10.0" temp-min-c="5.0" temp-max-c="15.0" temp-display-unit="F"
vent-required-value="5.0" vent-required-unit="PERCENTAGE" humidity-pct="65.0"
o2-pct="45.0"
    co2-pct="55.0"
    time-latest-on-power="2007-05-12T10:12:01" time-monitor-1="2007-05-12T11:12:00"
    time-monitor-2="2007-05-12T12:12:01"
    time-monitor-3="2007-05-12T13:12:01" time-monitor-4="2007-05-12T22:12:01"/>
  <oog left-cm="13" right-cm="14" top-cm="29" back-cm="15" front-cm="16"/>
  <handling special-stow="UDK" deck-requirement="EITHER" remark="OK" last-free-
day="2007-05-28"/>
  <contents weight-kg="11000.0" consignee-id="ROAD" consignee-name="RUNNER" shipper-
id="WILY" shipper-name="COYOTE"

```

```

        commodity-id="DRYF" commodity-name="Dry Fruits" bl-nbr="TX778866"/>
    <seals seal-1="SEAL00001"/>
    <flags/>
    <unit-etc requires-power="Y" dray-status="OFFSITE"/>
    <unit-flex unit-flex-1="FLEX1" unit-flex-2="FLEX2" unit-flex-3="FLEX3" unit-
flex-4="FLEX4" unit-flex-5="FLEX5"
        unit-flex-6="FLEX6" unit-flex-7="FLEX7" unit-flex-8="FLEX8" unit-
flex-9="FLEX9" unit-flex-10="FLEX10"/>
    <ufv-flex ufv-flex-date-1="2007-05-19T19:12:00" ufv-flex-
date-2="2007-05-20T19:12:00" ufv-flex-date-3="2007-05-21T19:12:00"
        ufv-flex-date-4="2007-05-22T19:12:00" ufv-flex-date-5="2007-05-23T19:12:00"
ufv-flex-date-6="2007-05-24T19:12:00"
        ufv-flex-date-7="2007-05-25T19:12:00" ufv-flex-date-8="2007-05-26T19:12:00"
        ufv-flex-1="UFVFLEX1" ufv-flex-2="UFVFLEX2" ufv-flex-3="UFVFLEX3" ufv-
flex-4="UFVFLEX4"/>
</unit>

```

7.2.5.1 Marrying the Chassis to a Container

When importing a unit which is a container married to a chassis, both equipments need to be listed inside the unit element with the container set with role="PRIMARY" and the chassis set with role="CARRIAGE". The equipments can be fully defined inside the unit or just referenced inside when the equipments already have been imported previously. The example shows the equipment defined outside the unit.

```

<container eqid="OUTS0000015" type="D40">
    <physical has-wheels="N" height-mm="2590" iso-group="GP" length-mm="12192" tare-
weight-kg="3035" width-mm="2438" />
    <ownership owner="MAT" operator="MAT" />
</container>
<chassis eqid="CHSS0000015" type="C40">
    <physical has-wheels="Y" height-mm="1260" iso-group="CH" length-mm="12192" tare-
weight-kg="892" width-mm="2438" />
    <ownership owner="MAT" operator="UNK" />
</chassis>
<unit id="OUTS0000015" category="IMPORT" transit-state="YARD" freight-kind="FCL"
unique-key="ChassisExample0000015" line="MAT">
    <equipment eqid="OUTS0000015" class="CTR" role="PRIMARY"/>
    <equipment eqid="CHSS0000015" class="CHS" role="CARRIAGE"/>
    <position loc-type="YARD" location="SI" slot="E2117" />
    <timestamps time-in="2008-08-13T21:40:38" time-last-move="2008-10-06T07:19:59" />
</unit>

```

Probably most sites will import all the chassis separately but define the container with the unit. The below example shows this split.

```

<chassis eqid="CHSS0000016" type="C40">
    <physical has-wheels="Y" height-mm="1260" iso-group="CH" length-mm="12192" tare-
weight-kg="892" width-mm="2438" />
    <ownership owner="MAT" operator="UNK" />
</chassis>
<unit id="OUTS0000016" category="IMPORT" transit-state="YARD" freight-kind="FCL"
unique-key="ChassisExample0000016"
line="MAT">
    <equipment eqid="OUTS0000016" class="CTR" type="D40" role="PRIMARY">
        <physical has-wheels="N" height-mm="2590" iso-group="GP" length-mm="12192" tare-
weight-kg="3035" width-mm="2438" />
        <ownership owner="MAT" operator="MAT" />
    </equipment>

```

```
</equipment>  
<equipment eqid="CHSS0000016" class="CHS" role="CARRIAGE"/>  
  <position loc-type="YARD" location="SI" slot="E2117" />  
  <timestamps time-in="2008-08-13T21:40:38" time-last-move="2008-10-06T07:19:59" />  
</unit>
```


7.3 Special Considerations: other Elements

In this section we briefly describe how some of the other complex entities of SN4 are modeled in SNX. We also over the fine points of using SNX to create (or update) them.

In This Section

vessel-visit, train-visit.	190
user.	190
trucking-company.	190
booking-information.	191
create-container-notice.	192
delete-container-notice.	193
flags.	193
snx-export.	194

7.3.1 vessel-visit, train-visit

An inventory unit arrives and departs on a carrier visit. A carrier visit is one visit of a carrier to the facility. Carriers could be vessels, trains or trucks. Details of the carrier visit can be imported for vessels and trains. Trucks are not included because the truck visit is short-lived and does not have a set of operations associated with it in the same way that trains and vessels do.

Although truck visits are not included, reference data associated with truck visits is included as it is assumed that this will be re-used in subsequent truck visits created by the N4 gate screens. Fine points to be aware of:

- You need not specify the {{ itinerary-id }} in the SNX. If you leave it unspecified, the itinerary of the designated Service (specified in {{ service-id }}) will be assumed to be the itinerary for the Vessel/Train.
- If the vessel or train will go on to another facility in the same complex (e.g. a barge at Matson) you must specify this with the 'next-facility' attribute. Here you provide the Id of the next facility that the carrier will visit.

7.3.2 user

The user password in SNX must already be encrypted. SNX does not encrypt/decrypt the password as it is imported or exported.

N4 uses MD5 (Message-Digest algorithm 5) to encrypt the password, Also we can encrypt the password manually using the below URL

Encrypt: <http://md5encryption.com/>;

Decrypt: <http://md5decryption.com/>

Example:

```
<userid="user3"password="5f4dcc3b5aa765d61d8327deb882cf99"first-name="SI User 3"last-name="Gate Clerk"operator="MATSON"complex="HAWAII"facility="HON"yard="SI"horizon-days="5">
```

7.3.3 trucking-company

This element will be used to both define trucking companies to SN4 and to update the status of the line agreements (i.e. permission of a trucking company to serve a shipping line). This is an example of a message that could be used to initially define a trucking company to SN4, including some line agreements:

```
<trucking-company id="ALOHA" name="Aloha Trucking Ltd." bic="000" scac="IIUU"
status="OK" insurance-expires="2007-12-31">
  <contact-info address-line-1="1 Kialoa Place" city="Honolulu" state="HA"
country="US" postal-code="888888" telephone="808-345-2222"/>
  <line-agreements update-mode="REPLACE">
    <line-agreement shipping-line="MAT" truck-line-status="OK" expiration-
date="2007-12-04"/>
    <line-agreement shipping-line="APL" truck-line-status="BANNED" expiration-
date="2007-12-15"/>
    <line-agreement shipping-line="NYK" truck-line-status="RCVONLY" expiration-
date="2020-01-01"/>
  </line-agreements>
</trucking-company>
```

This message would be used to simply change the date of an existing trucking company's insurance expiration:

```
<trucking-company id="ALOHA" insurance-expires="2007-09-15"/>
```

This message would be used to change the status of an existing trucking company's permission to service a particular line, leaving all other line agreements intact:

```
<trucking-company id="ALOHA">
  <line-agreements update-mode="MERGE">
    <line-agreement shipping-line="MAT" truck-line-status="BANNED" expiration-
date="2007-12-04"/>
  </line-agreements>
</trucking-company>
```

This message would be used to remove all line agreements for the trucking company:

```
<trucking-company id="ALOHA">
  <line-agreements update-mode="REPLACE"/>
</trucking-company>
```

7.3.4 booking-information

This element will be used to fetch the booking data. Additional to booking data the API will return containers assigned to the booking, containers reserved for the booking, containers delivered against the booking, and containers received against the booking.

Sample Request

```
<booking-information>
  <booking nbr="BKGFL " lineId="MAT" vessel-visit="SRVC"/>
</booking-information>
```

Element	Required?	Notes
Booking	Y	Nbr, line ID and vessel-visit will be used to identify the booking. 'Nbr' attribute which is the Booking nbr is the only mandatory field. If there are multiple bookings for a booking nbr, the response will return all the bookings found.

Sample Response

Response may contain single / multiple bookings. The booking-information-response element holds booking data in booking element. If multiple bookings are found there will be multiple booking elements.

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <argo:booking-information-response xmlns:argo="http://www.Navis.com/argo"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
www.Navis.com/argo OrdersWebserviceResponse.xsd">
  <argo:booking nbr="SPB1683211" status="FCL" full-return-location="FCL" hold-
partials="false" override-cutoff="false" prevent-type-subst="false">
    <argo:line>MAT</argo:line>
    <argo:vessel-visit>SPVL6832</argo:vessel-visit>
    <routing pod-1="LAX" pod-2="LYT" pol="OAK" />
    <argo:booking-item iso-group="GP" eq-height="NOM86" eq-material="UNKNOWN" eq-
size="NOM20" qty="1">
```

```
<argo:received-containers>
  <argo:container>SPLU3856832</argo:container>
</argo:received-containers>
</argo:booking-item>
</argo:booking>
</argo:booking-information-response>
</argo-response>
```

7.3.5 create-container-notice

This element will be used to create container notice. Multiples are not allowed.

Sample Request:

```
<create-container-notice>
ContainerSPLU6321972</container>
<event-type>UNIT_REROUTE</event-type>
<notification-action action-id="EMAIL" email-address="spabbala@zebra.com" email-
party="eqsEqOperator" sms-address="9962560326"/>
</create-container-notice>
```

Options for email-party are:

- eqsEqOperator
- eqsEqOwner
- unitLineOperator
- gdsConsigneeBzu
- gdsShipperBzu
- rtgTruckingCompany

Element	Required?	Notes
container	Y	Container ID
event-type	Y	Only unit based events are allowed.
notification-action	Y	"action-id" contains the action code. Allowed actions are "EMAIL", "SMS". If action is EMAIL then either email-address or email-party is mandatory attributes. If action is SMS then sms-address is mandatory.

Sample Response

create-container-notice-response holds the newly created notice request data.

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <argo:create-container-notice-response xmlns:argo="http://www.Navis.com/argo"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
www.Navis.com/argo ServicesWebserviceResponse.xsd">
    <create-container-notice container-nbr="SPLU6321972" />
  </argo:create-container-notice-response>
</argo-response>
```

7.3.6 delete-container-notice

delete-container-notice request deletes notices found based on provided input.

Sample Request

```
<delete-container-notice>
  ContainerFDLU09090810</container>
  <event-type>UNIT_REROUTE</event-type>
  <notification-action action-id="EMAIL" email-address="spabbala@zebra.com" email-
party="eqsEqOperator" />
</delete-container-notice>
```



Multiples are deleted if found.

Element	Required?	Notes
container	Y	Container ID
event-type	Y	Only unit based events are allowed.
notification-action	Y	Allowed actions are "EMAIL", "SMS".

Sample Response

notice element holds deleted notice request data. If multiple notice requests are deleted then there will be multiple notice elements, each notice element represents one notice request.

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <argo:delete-container-notice-response xmlns:argo="http://www.Navis.com/argo"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
www.Navis.com/argo ServicesWebserviceResponse.xsd">
    <argo:notice>
      <argo:container>SPLU6321972</argo:container>
      <argo:event-type>UNIT_REROUTE</argo:event-type>
      <argo:filter />
      <argo:notification-action action-id="EMAIL" email-address="spabbala@zebra.com"
email-party="unitPrimaryUe.ueEquipmentState.eqsEqOperator" />
    </argo:notice>
  </argo:delete-container-notice-response>
</argo-response>
```

7.3.7 flags

This element is used to import/export holds and permissions for an entity in N4, such as a container or vessel-visit. The elements that support the 'flags' element are: vessel-visit, train-visit, container, chassis, booking, bill-of-lading, equipment-receive-order, and equipment-delivery-order. The following is an example of the flags element used to include a booking hold and permission:

Example

```
<booking nbr="UNIT_TEST_02" line="XYZ" pol="OAK" pol-name="United States" pod-1="ADL"
eq-status="FCL" pod-2="ADL" ood="N" override-cutoff="N" hold-partials="N" prevent-type-
subst="N" category="EXPORT" created-by="-system-" created-date="2011-04-20T03:09:16"
modified-by="-system-" modified-date="2011-04-20T03:09:17" quantity="999">
  <carrier id="VSL004" facility="FCY111" mode="VESSEL" />
```

```
<flags>
  <hold id="BKG_HOLD" />
  <permission id="BKG_PRM" />
</flags>
</booking>
```

7.3.8 snx-export

Sample Request

```
<snx-export>
  <entity-name>ReportDefinition</entity-name>
  <field id="name" value="NFRM451 TEST"/>
</snx-export>
```

Element	Required?	Notes
entity-name	Y	entity-name should match our N4 entity name, but as of now we support only ReportDefinition. If user enters any other entity name he gets "Invalid entity name {0}." Error message.
field	Y	Field id will be metafield id and value is database field value.

Sample Response

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <argo:snx-export-response xmlns:argo="http://www.Navis.com/argo" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www.Navis.com/argo SnxExportWebserviceResponse.xsd">
    <report-definition name="CLERK ACTIVITY" report-design="CLERK ACTIVITY" output-type="RTF" reporting-entity="TruckTransactionStage" description="Clerk Activity Report">
      <roles />
      <sort-fields>
        <sort-field-1 />
        <sort-field-2 />
        <sort-field-3 />
      </sort-fields>
      <filter>
        <predicate verb="AND" order="1" negated="false" parm-type="NO_PARM">
          <predicate verb="OR" order="0" negated="false" parm-type="NO_PARM">
            <predicate verb="EQ" metafield="ttstageId" value="Inlanes" ui-value="Inlanes" order="0" negated="false" parm-type="NO_PARM" />
            <predicate verb="EQ" metafield="ttstageId" value="OB_Lanes" ui-value="OB_Lanes" order="1" negated="false" parm-type="NO_PARM" />
          </predicate>
          <predicate verb="EQ" metafield="tvstatGate.gateFacility" ui-value="PORTELIZABETH" order="1" negated="false" parm-type="NO_PARM" />
          <predicate verb="AND" order="2" negated="false" parm-type="NO_PARM">
            <predicate verb="AFTER" metafield="stageTransaction.tranTruckVisit.tvdtlsExitedYard" order="0" negated="false"
```

```
parm-type="REQUIRED_PARM" parm-label="StartTime" parm-internal-  
name="-7243169457693606849" />  
  <predicate verb="BEFORE"  
metafield="stageTransaction.tranTruckVisit.tvdtlsExitedYard" order="1" negated="false"  
parm-type="REQUIRED_PARM" parm-label="EndTime" parm-internal-  
name="8635505480694616634" />  
  </predicate>  
  </predicate>  
  </filter>  
  </report-definition>  
</argo:snx-export-response>  
</argo-response>
```

7.4 Unit updates and transactions on Units with SNX

Foreign systems are able to update the state of Units in a **live N4 system** with the following SNX messages:

Message	Purpose
<unit-routing>	update a Unit's routing, including carriers, pod, category, etc.
<unit-reefer>	update a Unit's reefer requirements
<unit-seals>	update the seals on the Unit
<unit-oog>	update the over-dimensions (out-of-gauge values) for a Unit
<unit-hazards>	update the hazardous cargo attributes of a Unit
<unit-property-update>	allows the update of miscellaneous properties of the Unit not covered in the foregoing messages.
<equipment-damages>	update the damages to a piece of equipment
<equipment>	update attributes of a a piece of equipment
<chassis>	update attributes of a chassis

Also, a number of common transactions involving Units can be effected via SNX messages as follows:

Message	Purpose
<unit-assign-to-booking>	assign the Unit to a new Booking
<unit-return-to-storage>	cancel the assignment of an empty Unit to a vessel (or train)
<unit-cancel-preadvise>	deletes an advised UFV, and its owning Unit if there is only one UFV for the Unit
<unit-renumber>	correct the container (or chassis) number of the container/ chassis of the Unit
<unit-return-to-storage>	De-assigns an empty from a specific outbound carrier

In This Section

The <unit-routing> message.	197
The <unit-hazards> message.	198
The <unit-property-update> message.	198
The <equipment> message.	199
The <equipment-damages> message.	199
The <unit-renumber> message.	199
The <unit-return-to-storage> message.	199
The <unit-assign-to-booking> message.	200
The <unit-return-to-storage> message.	200
The <unit-cancel-preadvise> message.	200

7.4.1 The <unit-routing> message

Send this message to update the outbound routing for a Unit. This includes the intended and declared outbound carriers, POD, destination, etc.

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated
routing	Y	describes carriers, POD, destinations, etc.
unit-etc	-	include to update category or dray-status
unit-flex	-	include to update any Unit flex-fields
ufv-flex	-	include to update any UnitFacilityVisit flex-fields
event	-	include to customize the event that is recorded

Optional Attributes

attribute	purpose
facility	to define the facility at which the update pertains

Use the *facility* attribute when updating the routing of a Unit that will visit more than one Facility in your Complex. This defines the Facility for which the routing information is to be interpreted and applied. This is only required when updating the Unit's arrival carrier, intended outbound carrier, or the UFV flex fields. If you do not provide it, SN4 will assume that the update applies at the Facility into which the SNX message was transmitted.

By default the <unit-routing> message will record a UNIT_ROUTING event in the event history. You can override this by providing the *event* element in the message. This allows you to post a different event, or to omit the event altogether.

An example <unit-routing> message:

```
<unit-routing facility="FCY111">
  <unit-identity id="MATU3989342" type="CONTAINERIZED">
    <carrier direction="OB" qualifier="DECLARED" mode="VESSEL" id="RJP070"/>
  </unit-identity>
  <routing pol="HNL" pod-1="SEA" destination="TACOMA">
    <carrier direction="OB" qualifier="DECLARED" mode="VESSEL" id="RJP071"/>
    <carrier direction="OB" qualifier="ACTUAL" mode="VESSEL" id="RJP071"/>
  </routing>
  <unit-etc category="DOMESTIC" dray-status=""/>
  <unit-flex unit-flex-1="JJS"/>
  <ufv-flex ufv-flex-1="XYZ" ufv-flex-date-1="2007-06-19T19:12:00"/>
</unit-routing>
```

7.4.2 The <unit-hazards> message

Send this message to update the hazardous contents of the Unit. The hazards will be **completely** replaced with the data provided in this message.

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated
hazards	Y	describes the items of hazardous cargo within the unit
unit-flex	-	include to update any Unit flex-fields
event	-	include to customize the event that is recorded

By default the <unit-hazards> message will not record an event in the event history. You can override this by providing the *event* element in the message.

7.4.3 The <unit-property-update> message

Send this message to update other miscellaneous properties of the Unit.

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated
position	-	include to update the slot, but not a change of location (yard to ship, etc.)
hazards	-	include to update hazardous contents
oog	-	include to update the over-dimensions
handling	-	include to update special-stow, deck-requirement, remark, last-free-day
contents	-	include to update commodity, weight, shipper, consignee, freight-kind, BL number
unit-etc	-	include to update category, dray-status, carrier-service, line, requires-power
unit-flex	-	include to update any Unit flex-fields
ufv-flex	-	include to update any UnitFacilityVisit flex-fields
event	-	include to customize the event that is recorded

If you include any UnitFacilityVisit (UFV) properties in this message such as the UFV flex fields, the **all non-departed UFV's** belonging to the Unit will be updated with the values that you provide.

By default this message will record a UNIT_PROPERTY_UPDATE event in the event history. You can override this by providing the *event* element in the message. This allows you to post a different event, or to omit the event altogether.

7.4.4 The <equipment> message

Send this message to update properties of a container or chassis. All Units using this equipment will reflect these changes.

As an example, send this message to update the equipment type and the offhire location for a container:

```
<equipment eqid="CTRU5050123" class="CTR" type="42V1" offhire-location="JON'S DEPOT"/>
```

7.4.5 The <equipment-damages> message

Send this message to define the damages to a container, chassis, gen-set, etc.

Example

```
<equipment-damages>
  <equipment-identity eqid="MATU3672143" class="CTR"/>
  <damages>
    <damage type="BENT" component="DOOR" severity="MAJOR"/>
    <damage type="WARPED" component="FLOOR" severity="MINOR"/>
  </damages>
</equipment-damages>
```

7.4.6 The <unit-renumber> message

Send this message to correct the equipment ID of the equipment used by the Unit.

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated
renumber	Y	describes whether the container or the chassis is to be renumbered, and provides the corrected number

By default the <unit-renumber> message will record either a UNIT_RENUMBER or a UNIT_CHECK_DIGIT event as appropriate. The logic used in renumbering is exactly the same as is used when the Renumber action is selected through the UI.

7.4.7 The <unit-return-to-storage> message

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated

If the unit is in a valid state for this action (unit's active unit facility visit is not null and it's transit state is in yard, freight kind is empty, no holds), the unit is updated to be a storage unit: This includes setting the Category to STORAGE,

clearing all the Routing information, updating all the OB Carriers to Generic Truck and clearing the unit equipment's EquipmentOrder reference (including order tally adjustment if needed). If the unit is not valid for this action a BizViolation is thrown.

Example

```
<!-- de-assign -->
<unit-return-to-storage>
  <unit-identity id="MATU6529342" type="CONTAINERIZED"/>
</unit-return-to-storage>
```

7.4.8 The <unit-assign-to-booking> message

Send this message to assign a Unit to an existing Booking

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated
booking	Y	identifies the booking to which the Unit should be assigned

This message is equivalent to the Unit Action "Assign to Booking" in the UI. The Booking element need only provide the booking number. The system will look for an existing Booking of this number belonging to the Line Operator of the Unit. You may also include the *line* attribute in the Booking element to be more explicit.

The Unit's routing fields will be updated to match the Booking **only if** the Line Operator is configured with the Booking Rule "Containers are rolled when Booking rolls".

A UNIT_ROLL event will be recorded for the Unit upon successful processing of this message, and tallies will be adjusted appropriately.

7.4.9 The <unit-return-to-storage> message

Send this message to clear the intended outbound carrier for an Empty Unit (e.g., to remove it from a vessel's load list).

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated

This message is equivalent to the Unit Action "Return to Storage" in the UI. It only may be applied to Empty Units. The Category for the Unit will be reset to STORAGE.

7.4.10 The <unit-cancel-preadvise> message

Send this message to delete an ADVISED UFV.

Synopsis

element	required?	notes
unit-identity	Y	identifies the unit to be updated

This message is equivalent to the Unit Action "Cancel Preadvise" in the UI. You should provide the *facility* attribute when you send this message to define the facility from which the UFV will be deleted. If not provided, the context Facility is assumed.

If the deleted UFV is the only UFV belonging to the Unit, the Unit will be deleted as well.

Example, showing use of the *facility* attribute:

```
<unit-cancel-preadvise facility="SI">
  <unit-identity id="MATU3989342" type="CONTAINERIZED">
    <carrier direction="OB" qualifier="DECLARED" mode="VESSEL" id="RJP070"/>
  </unit-identity>
</unit-cancel-preadvise>
```

7.5 Signaling Activities with SNX

You can use SNX to signal movement of Units, and to move Units in and out of the Facility. This is useful if N4 is being used to 'shadow' the operation of a Facility that it is not directly controlling. The TOS that is controlling the Facility can send these messages to N4 so that N4 is kept up-to-date.

The two SNX message elements that allow these transactions are

- <move> - signals the movement of a Unit
- <transit> - signals the arrival or departure of a CarrierVisit to the Facility

In This Section

Simple Unit movements within the facility.	203
Arriving and Departing Units.	203

7.5.1 Simple Unit movements within the facility

The <move> element can be used inform N4 about moves within the facility, like yard moves, and loading moves onto the vessel. In the <move> element you identify the unit that is to be moved with the <unit-identity> element, and specify the position that it is to be moved to with the <to-position> element. Here is an example of an SNX message to record a yard move:

```
<move>
  <unit-identity id="MATU2273247">
    <carrier direction="IB" qualifier="DECLARED" mode="VESSEL" id="MHI134"/>
  </unit-identity>
  <to-position loc-type="YARD" location="SI" slot="A326.1"/>
</move>
```

7.5.1.1 The 'is-correction-only' attribute

This attribute should be specified if the <move> message is being sent just to correct the information in N4, rather than to have N4 record an actual movement. When you specify 'is-correction-only="true"', the system will update the position of the Unit, but will not record a move in the Move History table.

7.5.2 Arriving and Departing Units

In order to arrive or depart a Unit from a Facility, you must perform two steps:

- Move the Unit onto a CarrierVisit (i.e., a visit of a vessel, truck, train, or railcar) with a <move> message.
- Arrive or depart the CarrierVisit with a <transit> message

Following is an example in which we deliver a container full of beer.

Step 1: Define the Unit (already in the yard)

```
<unit seal-1="3133385" shipper-name="MILLER BREWING COMPANY" consignee-name="PARADISE
BEVERAGES" consignee-id="0000010392"
time-in="2007-06-28T18:46:34.000" ufv-flex-date-3="2007-07-16T00:00:00.000" ufv-flex-
date-2="2007-06-29T00:00:00.000"
bl-nbr="3134547"
remark="BEER\!" weight-kg="23560.000" line="MAT" freight-kind="FCL" transit-
state="YARD" category="IMPORT"
unique-key="MATU2273247" id="MATU2273247">
  <equipment owner="MATU" strength-code="FA" height-mm="20472" tare-kg="3510.000"
role="PRIMARY" type="D40" class="CTR"
eqid="MATU2273247"/>
  <position slot="C13 33" location="YARD" loc-type="YARD"/>
  <routing destination="HON" opl="LAX" pod-1="HON" pol="LAX">
    <carrier id="MHI134" mode="VESSEL" qualifier="DECLARED" direction="IB"/>
    <carrier id="MHI134" mode="VESSEL" qualifier="ACTUAL" direction="IB"/>
    <carrier id="GEN_TRUCK" mode="TRUCK" qualifier="DECLARED" direction="OB"/>
    <carrier id="GEN_TRUCK" mode="TRUCK" qualifier="ACTUAL" direction="OB"/>
  </routing>
</unit>
```

Step 2: Move it onto a Truck (results in a DELIVER move in the move history)

```
<move>
  <unit-identity id="MATU2273247">
```

```
<carrier direction="IB" qualifier="DECLARED" mode="VESSEL" id="MHI134"/>
</unit-identity>
<to-position loc-type="TRUCK" location="JJS01" slot="1" orientation="FWD"/>
</move>
```

Step 3: Depart the Truck

This results in a DEPART event in the history, and the Unit becoming DEPARTED.

```
<transit direction="OB" mode="TRUCK" carrier-id="JJS01"
timestamp="2007-07-23T11:23:24"/>
```

To receive an Export into a Facility, you would start with an SNX to define the Unit, also defining its position on a particular truck. Then the truck is received into the Facility with the <transit> message, which in turn makes the Unit active. Then the Unit is placed into the yard with <move> message.

In summary, the sequence for an incoming unit is:

1. use <unit> to define the unit, if it is not yet known to the system..or if the unit is known, but its position is not currently on its inbound carrier, use <move>.
2. use <transit> to arrive the carrier that the unit is on.
3. use <move> to put the unit into the yard.

The sequence for an outgoing unit is:

1. use <move> to put the unit onto its outbound carrier.
2. use <transit> to depart the carrier that the unit is on.

If these sequences are followed, then N4 will record all appropriate event history automatically. The <move> element has extensive optional fields that allow specification of the CHE's that did the move, etc. just as they are passed from SPARCS. This allows N4's historr tables to be populated, enabling the statistics/BI functions of N4.

When you use the <transit> message to arrive a vessel or train, a background job is initiated that will transition each Unit on-board to the new state. This can take several minutes. You can not reliably perform discharge moves on inbound units until the job completes. (Because trucks take less time to process, arrival and departure of trucks runs in the foreground, so the same problem does not exist in the truck case.)

7.5.2.1 Keep things Tidy

You should inform N4 of the arrival and departure of empty CarrierVisits. For example, once a truck has arrived with a Unit, you should later tell us about the departure of that truck, even if it goes out bob-tail. Otherwise trucks accumulate in the Facility.

7.6 Initial data conversion via SNX

SNX enables the import of a customer's data into a new N4 instance from an xml file. The customer creates the xml file in the specified format from the data in their existing system then loads it into their N4 application via the SNX Data Importer (accessed via the menu "Administration>DBA>Import SNX File"). The SNX Data Importer covers the data items that have significant volume with additional smaller-volume items being entered manually.

 Basic reference data such as Routing Points should be completely imported to prevent cascading errors in Units. Any time you want to know what the SNX should look like, you can manually enter an entry and do an "Export SNX" action.

In This Section

Container vs. Embedded Equipment Element.	206
Import Process.	206
Deletion.	207
Transformation of Customer's XML Data Into SNX.	207
XML Troubleshooting Hints for First-time SNX/XML Customers.	208

7.6.1 Container vs. Embedded Equipment Element

With SNX, you can create containers through the system in two ways:

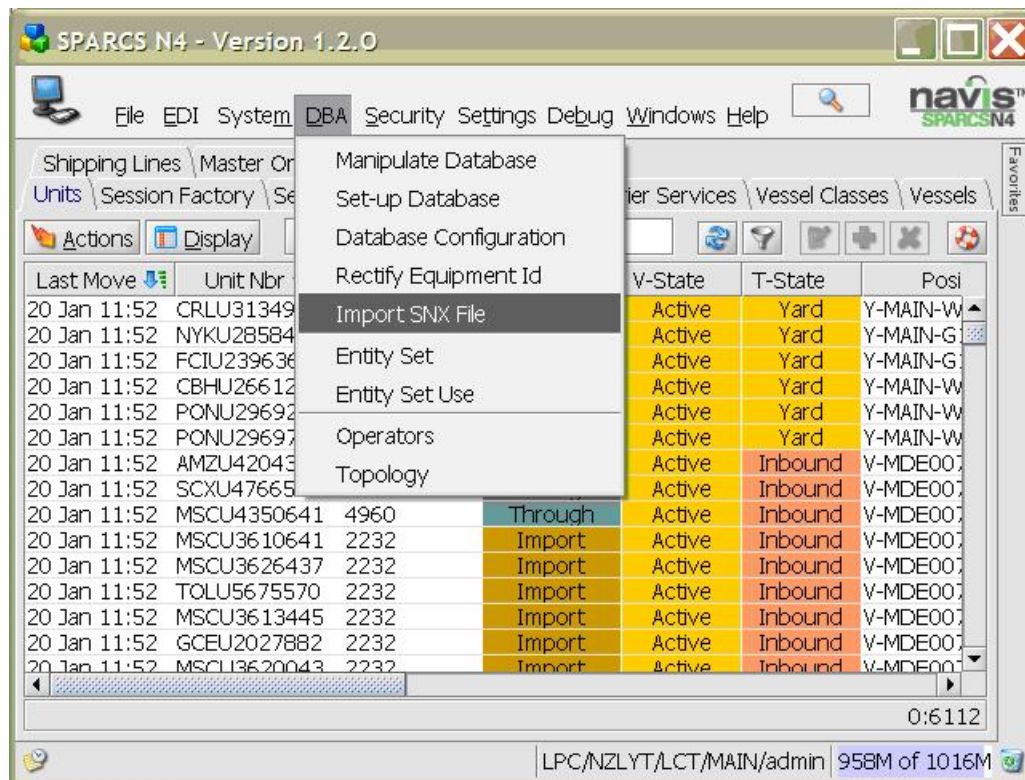
- use the embedded equipment element in the unit element which will cause a container in the N4 system to be created or updated
- use container elements, then reference them in the unit element

The container element in SNX is provided for clients who have an existing separate data model of containers. If your current system has a single table corresponding to the N4 unit and container, it may be more convenient to use the embedded equipment element.

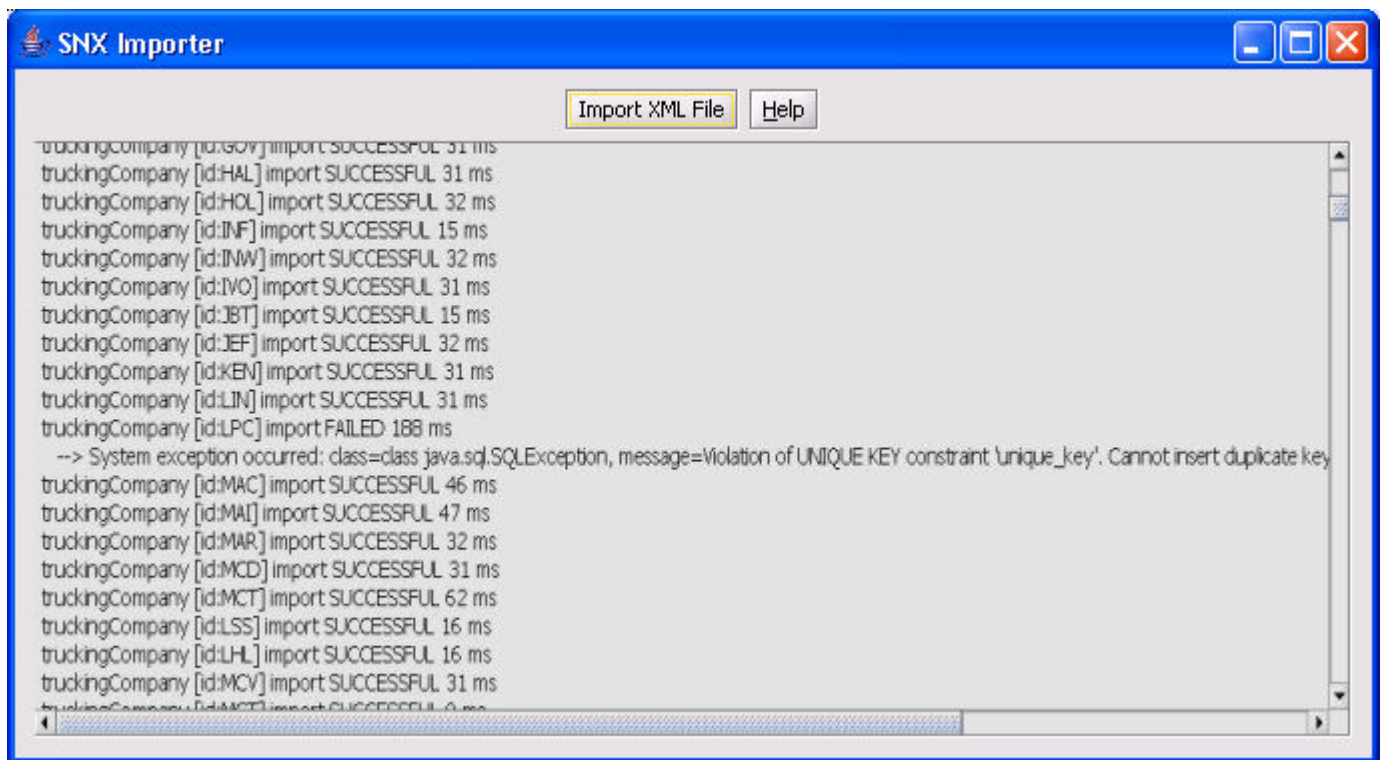
7.6.2 Import Process

The customer prepares an import xml file containing the units and the dependent entities according to the schema specification snx.xsd.

The file is imported via the Import SNX File utility on the Administration/DBA menu.



Clicking the Import XML File pops up a file-browser window to allow the user to select the xml file of data to be imported from their local machine. All entities are all treated as update/create. A typical usage scenario would be for a full database dump to be prepared and loaded. A much smaller file of changes since the initial load would then be used to bring the system up-to-date.



As each record is sent and processed by the server it is logged back to the user's screen as SUCCESSFUL or FAILED. Any failed records are also written to an xml file called {original filename}-errors.xml in the same folder as the original file together with the error message. This error file can be edited and reloaded.

7.6.3 Deletion

Often during the data migration process, you will want to delete the data and start over. Also some data especially the reference data cannot be permanently deleted from the UI as a delete action marks item as "obsolete" and takes row out of view. Deleting during the data migration process- depending on how much you delete is done mostly at the database level:

Deleting absolutely everything- you can drop all tables and restart the N4 which will recreate the schema.

Delete just the operational data-use N4 Sparcs SQL script deleting Units/Carrier Visits/Bills of Lading/Bookings/EDI Transactions/Gate Transactions

A good practice is to periodically take backups of your database at different stages so you can easily revert to a known stage.

The above delete actions are strictly for the data migration phase. You should not use any of the above for production.

Using the purge feature is not a practical way to delete targeted data as the purge process deletes in small increments over long periods of time.

7.6.4 Transformation of Customer's XML Data Into SNX

Some customers have existing tools that extract their production data in their own proprietary XML format.

Theoretically it is an XML to XML conversion and simple transformation may resolve it. However the solution is likely to be based on the complexity of the customer's XML.

- If it is a simple re-arrangement then an XSLT or other transformation engine/tools can be applied to bring this data into SNX format.
- If there is bigger mismatch between the customer's XML format and content then most likely some sort of logic will need to be written to massage that data and bring it into SNX format. If the customer is Java enabled and has reasonable Java expertise then they can use Java/JDOM to parse their XML, massage it, and then write it out into SNX format. Another approach is to write a little Groovy script to do the transformation. You can get everything you need in the one Groovy download. Groovy has direct support of XML in the language itself.

7.6.5 XML Troubleshooting Hints for First-time SNX/XML Customers

While working with first-time SNX customers we came across some issues that may be of use for the field engineers in future. Some customers are not XML-savvy and tend to use plain text editors (like Notepad) to edit their SNX files before they import it into N4. This leads to very simple structural and syntax errors that could have been caught by an XML editor and/or XML viewer. Following suggestions worked well (as an example with PICT):

Try to open SNX XML with XML viewer like Internet Explorer (IE) before the import process. If you can't open it most likely it has some basic syntax errors.

Try to open your XML in an XML-aware editor. XML editors are available as freeware and are also part of many products like IntelliJ, Eclipse, MS Visual Studio, etc. These editors make it simple to look/edit XML and also allow basic validation against SNX schema(snx.xsd). As we know we don't require/expect customer's SNX file to be fully XSD compliant, however, we do want the tags and attributes to have the right case and name. A good XML editor can catch these kind of issues. In 1.3 release of N4 we have (unfortunately) mixed naming convention for the tags and attributes. Some use Java variable-like syntax (e.g. <truckingCompany>) and others use traditional xml format (e.g. <shipper-consignee>). In case of PICT one of the most common errors in their XML has been misnamed tags differing in just the case with the correct one. In 1.4 version of N4 all the tags are renamed to be lower case with hyphen separating the multi-word tag names).

7.7 Disabling XPSCache Cluster Loading for Data Migration

When customers are importing data via SNX, the XPSCache clustering should be turned off to avoid extra processing.

The way objects are loaded into the Coherence cache is configured at the Yard level.

To modify the way this is done (e.g., disable / enable automatic loading):

1. Log in to N4 at the **Facility** level.
2. Open **Administration->DBA->Topology**.
3. Expand the tree so the **Yards** are displayed (they are children of the Facility).
4. Select the Yard to modify the cluster loading algorithm, right click, and select Edit from the popup.
5. To **DISABLE** automatic loading of the cluster, select **AWAIT_MANUAL_IMPORT** in the **XPS State** field and click **Save**.
6. Shutdown and restart Tomcat.

7.8 Re-organization of the SNX unit element

Starting with N4 v1.5.L, the SNX <unit> element has been re-organized into a cleaner structure. Most of the attributes that were originally defined to be part of the outermost element have been moved to new purpose-specific elements. This makes the <unit> message more coherent and the code that parses it more modular and testable.

For backward compatibility, the old format is still supported when importing data. However, N4 will export <unit> elements only in the new format going forward.

These are the attributes that moved, in alphabetical order:

attribute	owning element
bl-nbr	contents
commodity-id	contents
commodity-name	contents
consignee-id	contents
consignee-name	contents
deck-requirement	handling
departure-order-nbr	booking
dray-status	etc
last-free-day	handling
remark	handling
requires-power	etc
seal-1	seals
seal-2	seals
seal-3	seals
seal-4	seals
shipper-id	contents
shipper-name	contents
special-stow	handling
time-in	timestamps
time-load	timestamps
time-out	timestamps
ufv-flex-1	ufv-flex
ufv-flex-2	ufv-flex
ufv-flex-3	ufv-flex
ufv-flex-4	ufv-flex
ufv-flex-date-1	ufv-flex

attribute	owning element
ufv-flex-date-2	ufv-flex
ufv-flex-date-3	ufv-flex
ufv-flex-date-4	ufv-flex
ufv-flex-date-5	ufv-flex
ufv-flex-date-6	ufv-flex
ufv-flex-date-7	ufv-flex
ufv-flex-date-8	ufv-flex
unit-flex-1	unit-flex
unit-flex-10	unit-flex
unit-flex-2	unit-flex
unit-flex-3	unit-flex
unit-flex-4	unit-flex
unit-flex-5	unit-flex
unit-flex-6	unit-flex
unit-flex-7	unit-flex
unit-flex-8	unit-flex
unit-flex-9	unit-flex
weight-kg	contents

Here is the same information, organized by element:

attribute	owning element
id	unit
unique-key	unit
transit-state	unit
category	unit
freight-kind	unit
line	unit
time-in	timestamps
time-out	timestamps
time-load	timestamps
weight-kg	contents
bl-nbr	contents
commodity-id	contents
commodity-name	contents
consignee-id	contents

attribute	owning element
consignee-name	contents
shipper-id	contents
shipper-name	contents
departure-order-nbr	booking
deck-requirement	handling
special-stow	handling
last-free-day	handling
remark	handling
dray-status	etc
requires-power	etc
seal-1	seals
seal-2	seals
seal-3	seals
seal-4	seals
unit-flex-1	unit-flex
unit-flex-2	unit-flex
unit-flex-3	unit-flex
unit-flex-4	unit-flex
unit-flex-5	unit-flex
unit-flex-6	unit-flex
unit-flex-7	unit-flex
unit-flex-8	unit-flex
unit-flex-9	unit-flex
unit-flex-10	unit-flex
ufv-flex-1	ufv-flex
ufv-flex-2	ufv-flex
ufv-flex-3	ufv-flex
ufv-flex-4	ufv-flex
ufv-flex-date-1	ufv-flex
ufv-flex-date-2	ufv-flex
ufv-flex-date-3	ufv-flex
ufv-flex-date-4	ufv-flex
ufv-flex-date-5	ufv-flex
ufv-flex-date-6	ufv-flex
ufv-flex-date-7	ufv-flex

attribute	owning element
ufv-flex-date-8	ufv-flex

Here is an example of a fully endowed unit in the new format:

```
<unit id="MATU2064601" transit-state="ADVISED" category="IMPORT" freight-kind="FCL"
line="MAT">
  <equipment eqid="MATU2064601" class="CTR" type="4200" role="PRIMARY" tare-
kg="1900.0" height-mm="2831"
    strength-code="AA" material="ALUMINUM" owner="ITT" operator="YBR">
    <damages>
      <damage type="BENT" component="DOOR" severity="MAJOR"/>
    </damages>
  </equipment>
  <position loc-type="VESSEL" location="RJP001" slot="010280"/>
  <routing pol="LAX" pod-1="KAH" pod-2="KHH" opl="HKG" origin="VENTURA"
destination="MAKAWAO"
    designated-trucker="ABC" carrier-service="USWC" group="HOT">
    <carrier direction="IB" qualifier="DECLARED" mode="VESSEL" id="RJP001"/>
    <carrier direction="IB" qualifier="ACTUAL" mode="VESSEL" id="RJP001"/>
    <carrier direction="OB" qualifier="DECLARED" mode="TRUCK"/>
    <carrier direction="OB" qualifier="ACTUAL" mode="VESSEL"/>
  </routing>

  <booking id="MAT6678435" line="MAT"/>
  <hazards>
    <hazard imdg="1.2" un="1203" tech-name="GASOLINE" page-number="13"/>
    <hazard imdg="4.1" un="6789" tech-name="PEROXIDE" page-number="45"/>
    <hazard imdg="6.1" un="5634" ltd-qty-flag="Y" package-type="BOX" inhalation-
zone="Z" explosive-class="B"
      page-number="32"
      flash-point="98.0" tech-name="TECH" proper-name="PROPER" ems-nbr="EMS"
mfag="MFAG"
      packing-group="II"
      haz-id-upper="UPPR" substance-lower="LWER" weight-kg="234.0" planner-
ref="PREF" quantity="5"
      move-method="MOVE"
      secondary-imo-1="1.4" secondary-imo-2="6.1" deck-restrictions="A" marine-
pollutants="Y"
      dc-lg-ref="LC" emergency-telephone="911"
      ems-response-guide-nbr="001" notes="-hazard notes-"/>
  </hazards>
  <reefer temp-reqd-c="10.0" temp-min-c="5.0" temp-max-c="15.0" temp-display-unit="F"
vent-required-value="5.0" vent-required-unit="PERCENTAGE" humidity-pct="65.0"
o2-pct="45.0"
    co2-pct="55.0"
    time-latest-on-power="2007-05-12T10:12:01" time-monitor-1="2007-05-12T11:12:00"
    time-monitor-2="2007-05-12T12:12:01"
    time-monitor-3="2007-05-12T13:12:01" time-monitor-4="2007-05-12T22:12:01"/>
  <oog left-cm="13" right-cm="14" top-cm="29" back-cm="15" front-cm="16"/>
  <handling special-stow="UDK" deck-requirement="EITHER" remark="OK" last-free-
day="2007-05-28"/>
  <contents weight-kg="11000.0" consignee-id="ROAD" consignee-name="RUNNER" shipper-
id="WILY" shipper-name="COYOTE"
    commodity-id="DRYF" commodity-name="Dry Fruits" bl-nbr="TX778866"/>
</unit>
```

```
<seals seal-1="SEAL00001"/>
<flags/>
<unit-etc requires-power="Y" dray-status="OFFSITE"/>
<unit-flex unit-flex-1="FLEX1" unit-flex-2="FLEX2" unit-flex-3="FLEX3" unit-
flex-4="FLEX4" unit-flex-5="FLEX5"
    unit-flex-6="FLEX6" unit-flex-7="FLEX7" unit-flex-8="FLEX8" unit-
flex-9="FLEX9" unit-flex-10="FLEX10"/>
<ufv-flex ufv-flex-date-1="2007-05-19T19:12:00" ufv-flex-
date-2="2007-05-20T19:12:00" ufv-flex-date-3="2007-05-21T19:12:00"
    ufv-flex-date-4="2007-05-22T19:12:00" ufv-flex-date-5="2007-05-23T19:12:00"
ufv-flex-date-6="2007-05-24T19:12:00"
    ufv-flex-date-7="2007-05-25T19:12:00" ufv-flex-date-8="2007-05-26T19:12:00"
    ufv-flex-1="UFVFLEX1" ufv-flex-2="UFVFLEX2" ufv-flex-3="UFVFLEX3" ufv-
flex-4="UFVFLEX4"/>
</unit>
```

7.9 Event Processing Via JMS for N4

N4 supports both inbound and outbound JMS messages in SNX format. An external system can send SNX format messages into the N4 system via the configured inbound JMS queue. The only supported JMS queue provider in N4 2.2+ is ActiveMQ. You can configure N4 notice requests to send SNX messages via an outbound JMS queue when a specific event occurs.

In This Section

- [JMS Configuration for Inbound and Outbound Queues.](#) 216
- [Configuring Notice Requests for the Outbound Queue.](#) 219
- [Debugging and Error Handling.](#) 220
- [Configure N4 Tomcat node to Weblogic JMS server.](#) 221

7.9.1 JMS Configuration for Inbound and Outbound Queues

To configure SNX support in N4 the following 3 steps need to be completed:

Once the steps have been completed SN4 will be ready to process messages on the inbound queue.

7.9.1.1 Add JMS queues to the facilities

All JMS queues are configured at the facility level. There are two possible queues to configure for each facility:

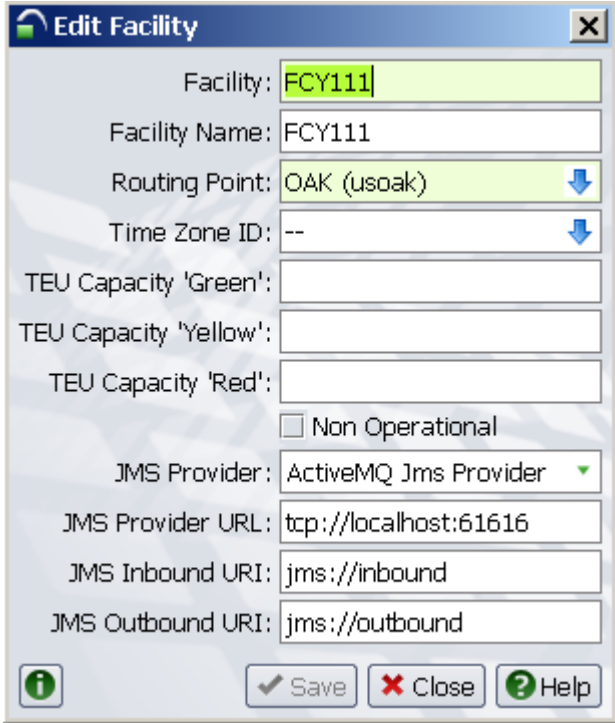
- Inbound JMS queue for real time processing of messages, called JMS Inbound URI.
- Outbound JMS queue used to send outbound messages as well as error message responses to inbound messages.

Both the inbound and the outbound queues are configured by default to run on the same JMS provider or broker instance. A separate provider can be used for a different facility, but the inbound and outbound queues for any particular facility will run on the same provider.

The JMS settings appear when editing the Facility as part of the Topology. A JMS provider must be chosen and then a valid URI must be added for the inbound and outbound queues. The queue names can be any valid URI names and the URI should start with the `jms://` prefix.

The typical JMS Provider URL will look as follows:

- For ActiveMQ (Used with Tomcat): `tcp://hostname:61616`



The screenshot shows the 'Edit Facility' dialog box with the following fields and values:

- Facility: **FCY111** (highlighted)
- Facility Name: FCY111
- Routing Point: OAK (usoak) (dropdown menu)
- Time Zone ID: -- (dropdown menu)
- TEU Capacity 'Green':
- TEU Capacity 'Yellow':
- TEU Capacity 'Red':
- ☐ Non Operational
- JMS Provider: ActiveMQ Jms Provider (dropdown menu)
- JMS Provider URL: tcp://localhost:61616
- JMS Inbound URI: jms://inbound
- JMS Outbound URI: jms://outbound

At the bottom, there are three buttons: **Save** (with a checkmark icon), **Close** (with a red X icon), and **Help** (with a question mark icon).

7.9.1.2 Configure provider specific parameters

Using ActiveMQ as a JMS provider (Add Link)

ActiveMQ is the default JMS provider shipped with the SN4 product. No further configuration is required once step 1 above is complete for the ActiveMQ case. Once step 3 below is completed, the JMS queues will be started up and listening on the specified ActiveMQ port as a TCP connection point. For more information on downloading and running the supported version of ActiveMQ, see .

JMX support has been enabled for the ActiveMQ provider and can be accessed using a JMX console such as JConsole. See <http://activemq.apache.org/jmx.html> for more information on how to connect using JMX.

This screenshot shows the JConsole for Java 1.6. The console shows a broker configuration that hosts the queues. Any queues configured on the facilities should be listed under the queues. One can use the console to send a test SNX message to test the configuration. The console also allows you to see how many messages have been processed and if there are any messages in the queue still waiting to be processed.

The screenshot shows the Java Monitoring & Management Console window. The title bar indicates the process is 'pid: 3528 run.jar start'. The 'Overview' tab is selected, showing a tree view on the left and a table of attribute values on the right.

Tree View (Left):

- JMImplementation
- com.sun.management
- java.lang
- java.util.logging
- org.apache.activemq
 - localhost
 - Broker
 - Connection
 - Connector
 - Queue
 - example.A
 - inbound
 - Attributes** (selected)
 - Operations
 - Subscription
 - Topic
- org.apache.camel

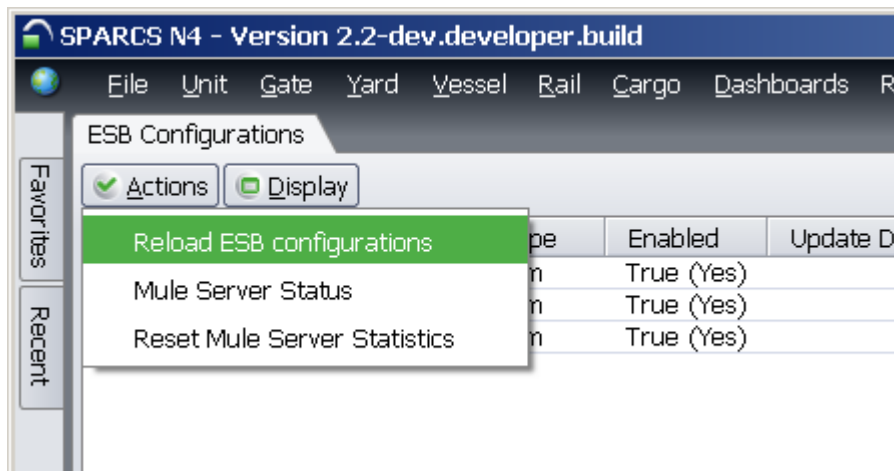
Attribute values (Right):

Name	Value
AverageEnqueueTime	0.0
BlockedProducerWarningInterval	30000
CacheEnabled	true
ConsumerCount	1
CursorFull	false
CursorMemoryUsage	0
CursorPercentUsage	0
DequeueCount	0
DispatchCount	0
EnqueueCount	0
ExpiredCount	0
InFlightCount	0
MaxAuditDepth	2048
MaxEnqueueTime	0
MaxPageSize	200
MaxProducersToAudit	64
MemoryLimit	1048576
MemoryPercentUsage	0
MemoryUsagePortion	0.0
MinEnqueueTime	0
Name	inbound
PrioritizedMessages	false
ProducerCount	0
ProducerFlowControl	true
QueueSize	0
SlowConsumerStrategy	
Subscriptions	javax.management.ObjectName[1]
UseCache	true

A 'Refresh' button is located at the bottom right of the attribute values table.

7.9.1.3 Restart the JMS connections

Once the queues have been configured you must restart the JMS connections within SN4 for your changes to take effect. Select the "Reload ESB Configurations" Action on the ESB Configurations table view.



If you do not see ESB Configurations view, make sure your user account has a role with the ESB_CONFIGURATIONS privilege.

7.9.2 Configuring Notice Requests for the Outbound Queue

Outbound SNX messages can be configured in the General Notices screen of the desired Notice Request.

Select the "SNX Message Via JMS" action to send an SNX message to the outbound queue. When a sent message is posted to JMS, it goes into the queue configured for the Facility in which the event occurred. If no queue is configured, this is considered an error and will be written to the system log. The client may then listen on the outbound queue for the messages.

7.9.3 Debugging and Error Handling

SNX message that can't be handled are forwarded to the outbound queue. The format is another XML element wrapping the error reason and the original SNX message. The format for this is described in the [snxError.xsd](#) file as follows. The payload is the original inbound message and the reason is the error message that occurred when trying to process the inbound message.

```
<?xml version="1.0" encoding="UTF-8" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://www.Navis.com/argo"
  xmlns:sch="snx.xsd"
  targetNamespace="http://www.Navis.com/argo"
  elementFormDefault="qualified">
  <xsd:element name="snx-error">
    <xsd:annotation>
      <xsd:documentation>
        Snx error element that goes into the error jms queue.
      </xsd:documentation>
    </xsd:annotation>
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="reason"/>
        <xsd:element name="payload">
```



```
<xsd:complexType>
  <xsd:sequence>
    <sch:snx/>
  </xsd:sequence>
</xsd:complexType>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
</xsd:schema>
```

If there is an error posting an outbound SNX message the SN4 logs will contain an error message along with the message. See the logging section on how to access the log files.

7.9.4 Configure N4 Tomcat node to Weblogic JMS server

It's possible to configure N4 node to connect to external weblogic jms server.

To setup during N4 tomcat deployment:

1. Copy weblogic.jar to the system where Tomcat is located. Don't copy to tomcat/common/lib or anywhere under tomcat's default classpath.

 Putting weblogic.jar in tomcat/common/lib is no good. Though Tomcat can pick up weblogic jms factories from that lib folder, but there are some standard ejb jms classes loaded already by then, which are not compatible with weblogic classes.

 Weblogic.jar may contains java classes that conflict with those in tomcat/common/lib (particularly javax.* package.) One may need to remove those conflicting classes from weblogic.jar

2. For tomcat started up from tomcat/bin/startup.bat, modify catalina.bat to put the weblogic.jar in front of any library in the CLASSPATH environment variable.
3. For tomcat started up from Windows Services window, modify the CLASSPATH under the Java tab in the Windows Service edit window (which is usually executed from tomcat/bin/tomcata.exe or tomcatb.exe)
4. Make sure the TCP port on the weblogic machine is open for connection. This is usually done through Controll Panel -> Windows Firewall -> Exceptions -> Add Port.
5. Make sure the jms queues names match on both Weblogic jms server and N4 topology facility nodes.

 The N4/Carina/Mule only works with weblogic.jar from Weblogic 9.X. It can't work with the jar from Weblogic 10+. It's ok for us to use the jar from Weblogic 9.X to receive/send messages from/to Weblogic 10 application server.

7.10 ActiveMQ Configuration

In This Section

Setup ActiveMQ.	223
Monitor JMS Queues.	223
Configure JMS Queues.	224
Troubleshooting ActiveMQ.	225
ActiveMQ integration with .Net.	226
ActiveMQ Failover Configuration.	227
JMS Failover Scenarios.	229

7.10.1 Setup ActiveMQ



These steps were documented with ActiveMQ version 4.1.1. For the current supported version of ActiveMQ, please check the *Navis N4 Application Environment Guide*.

To install ActiveMQ:

1. Download ActiveMQ 4.1.1 from <http://activemq.apache.org/download.html>
2. Unzip apache-activemq-4.1.1.zip to a directory of your choice, say C:\apache-activemq-4.1.1
3. ActiveMQ can run as it with default configurations. You simply run C:\apache-activemq-4.1.1\bin\activemq.bat

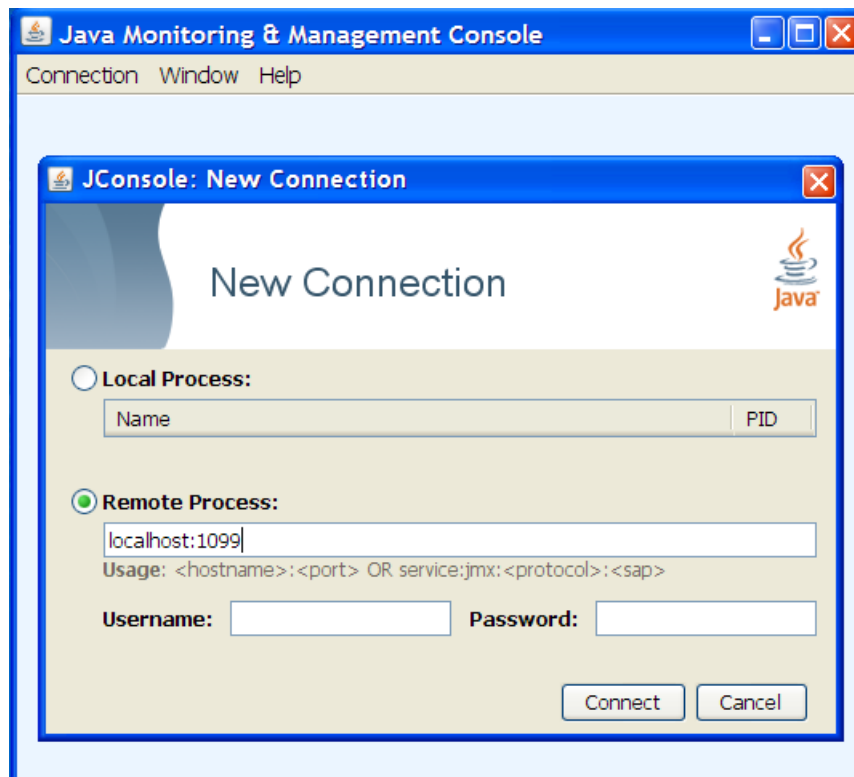
Now your ActiveMQ is ready for N4. You'll need to configure Facility in Topology in N4 to point the JMS provider URL to this ActiveMQ instance. For more information, see [Event Processing Via JMS for N4 \(on page 215\)](#).

7.10.2 Monitor JMS Queues

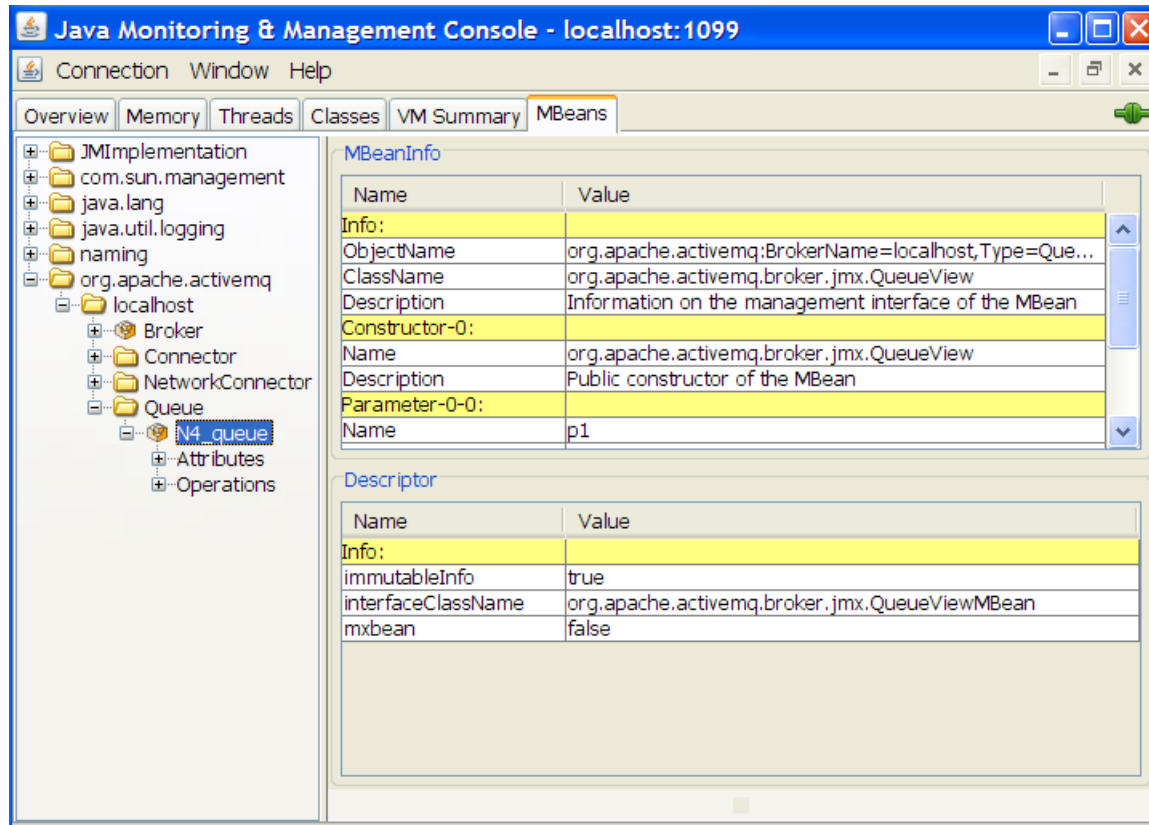
There is a web console tool that you can download from the ActiveMQ web site to help you configure and monitor the jms queue. The setup is relatively elaborate. The easiest way is to use the JConsole from JDK to configure ActiveMQ via JMX. JConsole is available from JDK 1.5 or above.

The steps are:

1. Start ActiveMQ by running bin\activemq.bat
2. Run jconsole.exe from your local JDK installed bin directory
3. Click on the "Remote Process" radio button and enter "localhost:1099" in the text field next to it.



- By default ActiveMQ configuration, Username and Password are not set. You leave them blank unless you've set up the security in ActiveMQ.
- Click on the "Connect" button.
- After the connection is successful, click on the "MBeans" tab. You'll see a tree list on the left panel and blank on the right panel.
- If you've configured N4 to connect to ActiveMQ, the queue name will appear in the right panel in the path: localhost -> Broker -> Queue -> (name of queue you configured in N4 topology)



Dynamic Queue Creation

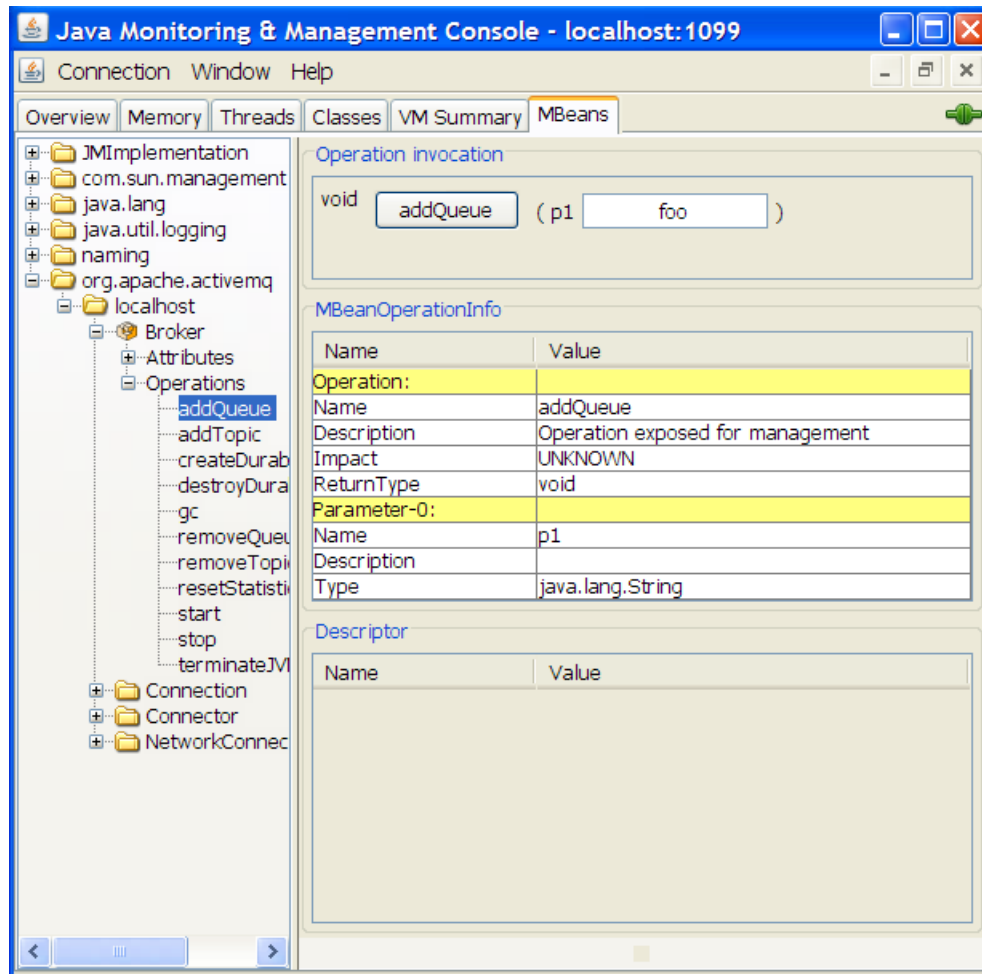
ActiveMQ allows the jms clients to dynamically create queues and topics. Thus, it's not necessary to pre-configure queues and topics in ActiveMQ. Rather, we configure the queue names in N4 topology first. Upon the connection from N4 to ActiveMQ, we let the connection to dynamically create the queues in ActiveMQ. **THIS IS NOT TRUE FOR WEBLOGIC JMS PROVIDER.** You do need to pre-configure queues and topics in Weblogic before N4 tries to connect.

7.10.3 Configure JMS Queues

In case you want to add and configure a new queue yourself, the steps are:

- Expand the tree node "org.apache.activemq" and its children along this path: localhost -> Broker -> Operations -> addQueue

- You can add a new queue to ActiveMQ by filling the queue name on the right panel and click on the "addQueue" button. The picture below shows adding a queue "foo".



- A new tree path "Queue ->foo" appears under the localhost tree node.
- Under the "Operations" tree node, you have buttons to monitor and manipulate the queue.

7.10.4 Troubleshooting ActiveMQ

To confirm that N4 has sent out all expected JMS messages, you can confirm the message count(QueueSize) in the Queue attributes screen.

The screenshot shows the J2SE 5.0 Monitoring & Management Console. The title bar reads "J2SE 5.0 Monitoring & Management Console: 2328@localhost". The "Connection" tab is selected, and the "MBeans" sub-tab is active. On the left, a tree view shows the hierarchy: Tree > org.apache.activemq > localhost > Broker. The right pane displays the "Attributes" tab for the selected MBean. It contains a table with the following data:

Name	Value
ConsumerCount	0
DequeueCount	919
DispatchCount	1277
EnqueueCount	925
MemoryLimit	9223372036854775807
MemoryPercentageUsed	0
Name	OutQ
QueueSize	1

A "Refresh" button is located at the bottom right of the console window.

7.10.5 ActiveMQ integration with .Net

For more information on NMS, see <http://activemq.apache.org/nms.html>.

7.10.6 ActiveMQ Failover Configuration

This document describes how to set up ActiveMQ instances for failover scenarios.

In this Section:

- ActiveMQ Failover in a NutShell. 228
- Failover Setup. 228
- Preparing ActiveMQ for the same machine setup. 228
- N4 JMS Restarting Connection. 228
- Prepare ActiveMQ on multiple machines Setup. 229

7.10.6.1 ActiveMQ Failover in a NutShell

In general, ActiveMQ provides the general mechanism of one running master instance and several waiting slaves. When the master instance is down, one of the slave instances takes over to become the new master. There are two ways of locking that can be used among the master and slaves to decide who can run: file system locking, database locking. To use the "high performance journal" feature in ActiveMQ, the file system locking is selected. Please see [here](#) for the comparison.

7.10.6.2 Failover Setup

There are two possible setups:

- The master and multiple slaves run on the same physical machine.
- The master and multiple slaves run on different physical machines.

The impact on N4 is different for each possible setup. In the case of running all activeMq instances on the same machine, N4 is talking to the same tcp URL. In the case of different machines, N4 needs to find the tcp URL of the next running master when the old master fails. The first case is simple and is described in this document. For the second case N4 needs to add the intelligence of finding and composing the tcp URL of the next running master. which is described on the last section.

7.10.6.3 Preparing ActiveMQ for the same machine setup

1. Download apache-activemq-4.1.1.zip
2. Unzip to a location, say C:\apache-activemq-4.1.1
3. Edit C:\apache-activemq-4.1.1\conf\activemq.xml with the following changes

```
<persistenceAdapter>
  <journalJDBC dataDirectory="${activemq.base}/activemq-data" dataSource="#mysql-ds" useDatabaseLock="false"/>
</persistenceAdapter>
<bean id="mysql-ds" class="org.apache.commons.dbcp.BasicDataSource" destroy-method="close">
  <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
  <property name="url" value="jdbc:mysql://localhost/activemq?relaxAutoCommit=true"/>
  <property name="username" value="mnr"/>
  <property name="password" value="mnrpass"/>
  <property name="poolPreparedStatements" value="true"/>
</bean>
```

4. Log in the mysql as root to create the table

```
create database activemq;
```

5. Copy the mysql database driver to C:\apache-activemq-4.1.1\lib\optional
6. Start ActiveMQ instances by running C:\apache-activemq-4.1.1\bin\activemq.bat **multiple times**. The first instance that can grab the file lock on file system will become the master. Other running instances will be slaves waiting to be the master.

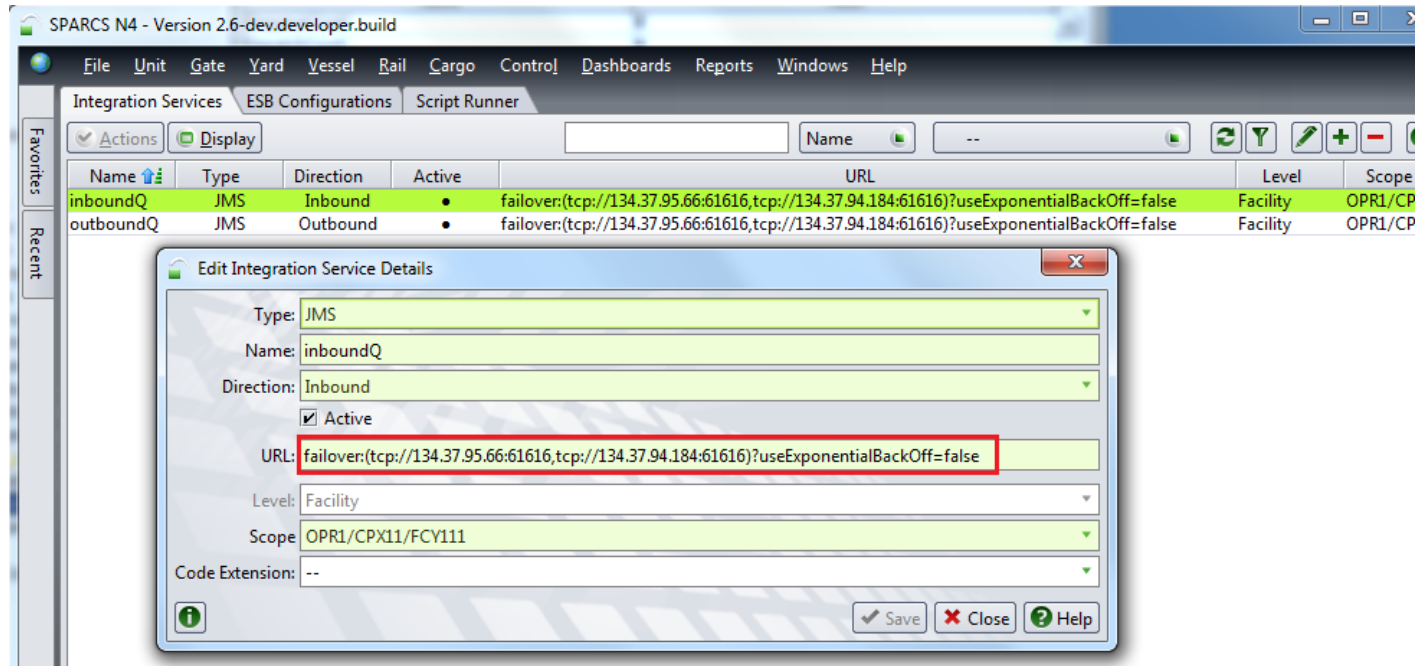
7.10.6.4 N4 JMS Restarting Connection

On the settings table view, there are two new settings: FAILOVER_RETRY_COUNT and FAILOVER_RETRY_FREQUENCY. They are taking effect when a the mule server is restarted. You can use this

setting to tell how many times and how often the mule server should try to re-establish JMS connections when the connections lost.

7.10.6.5 Prepare ActiveMQ on multiple machines Setup

The feature described here in N4 is the version 2.6.H. The Integration Services table view is where one configures the JMS connection information to connect to the external activemq instances. For the automatic failover setup, we use the failover protocol provided by the activemq. Please see below.



Note, once the JMS integration services are modified, one has to go to the **Integration Configurations** view and choose **Actions** → **Reload Integration** → **External JMS Configuration Context** to take the changes into effect.

For both the inbound and outbound integration services, we use the activemq failover scheme. For more available options, please see here.

<http://activemq.apache.org/failover-transport-reference.html>

All ActiveMQ instances (both master and multiple slaves) must be configured to lock either the same file system or the same database tables.

In case when none of the activemq instances are running, all outbound messages from N4 will be queued in memory on the outbound mule endpoint. When one of the activemq instances is connected back to N4, the outbound messages will be delivered over. In theory, for a prolonged activemq outage, this may potentially make the in-memory queue grow out-of-bound to cause out-of-memory in that node.

7.10.7 JMS Failover Scenarios

Scenario A

1. N4 Starts up, JMS server not available

N4 tries to connect to JMS server for ten times, once for few seconds, configurable in Settings

2. JMS Messages sent

Integration Errors view shows

```
External JMS server is not ready.:com.Navis.framework.esb.protocol.jms.JmsConnectorException:
External JMS server is not ready.
    at com.Navis.framework.esb.server.mule.ESBMuleServerManager.
getOutboundTransformer(ESBMuleServerManager.java:291)
    at com.Navis.argo.business.integration.jms.ArgoFacilityJmsManager.
sendJMSBytesMessage(ArgoFacilityJmsManager.java:36)
    at com.Navis.argo.business.api.GroovyApi.sendJMSBytesMessage(GroovyApi.java:55)
```

3. JMS Server brought up

If N4 finished retries, admin needs to manually **refresh** the JMS connections **on every node in the cluster**. If N4 were still in the middle of retry, it would connect to the JMS server automatically. (Currently there is no visual indication to tell whether N4 is in the middle of retry except in the Navis-apex.log.)

4. JMS Messages sent

Message is received in queue in ActiveMQ

5. JMS Server brought down

N4 starts to retry connection to JMS server for ten times, once for few seconds, configurable in Settings

6. JMS Message sent

```
Unexpected JMS connection problem:org.mule.umo.transformer.TransformerException:
null (java.lang.NullPointerException) (org.mule.umo.transformer.TransformerException)
    at org.mule.providers.jms.transformers.ObjectToJMSMessage.doTransform(ObjectToJMSMessage.java:54)
    at
com.Navis.argo.business.integration.jms.ArgoFacilityJmsManager.sendJMSBytesMessage(ArgoFacilityJmsManager.java:36)
```

7. JMS Server brought up on different server

N4 admin needs to update the topology to the new JMS server, and **refresh** the JMS connections **on every node in the cluster** if N4 finished retry.

8. JMS Messages sent

Message received in queue in ActiveMQ

Scenario B

1. JMS Server started up

2. N4 Startups up

N4 connected to JMS server, assuming the topology is configured correctly.

3. JMS Messages sent

Message is received in queue in ActiveMQ

4. Outbound changed

N4 admin needs to **refresh** the JMS connections **on every node in the cluster**.

5. JMS Messages sent

Message is received in the new outbound queue in ActiveMQ

6. JMS Server brought down

N4 starts to retry connection to JMS server for few times, once for few seconds, configurable in Settings

7. JMS Messages sent

Integration Errors view reports

```
Unexpected JMS connection problem:org.mule.umo.transformer.TransformerException:  
null (java.lang.NullPointerException) (org.mule.umo.transformer.TransformerException)  
    at org.mule.providers.jms.transformers.ObjectToJMSMessage.doTransform(ObjectToJMSMessage.java:54)  
    at  
com.Navis.argo.business.integration.jms.ArgoFacilityJmsManager.sendJMSBytesMessage(ArgoFacilityJmsManager.java:3  
    at com.Navis.argo.business.api.GroovyApi.sendJMSBytesMessage(GroovyApi.java:55)
```

8. 8. JMS Server brought up on different server

N4 admin needs to update the topology to the new JMS server, and **refresh** the JMS connections **on every node in the cluster** if N4 finished retry.

9. 9. JMS Messages sent

Message is recieved in queue in ActiveMQ

Questions

- When do the JMS connectors have to be manually refreshed?

Please see **refresh** in the above scenarios for the refresh points.

Part 8

Gate Workflow Customization

Gate Workflow Customization Options.	233
---	-----

8.1 Gate Workflow Customization Options

In This Section

- Summary. 234
- Overview of Gate Workflow Execution. 234
- Summary of Customization Options. 234
- Task Filters. 234
- Tasks that Create Custom Error Conditions. 236
- Task to Override the Next Stage Determination. 237
- Groovy Code in the Customize Window. 237
- Calling Groovy Plug-ins from the Customize Window. 238
- Code Extensions Executed in the Workflow. 238
- Code Extensions to Override Gate Tasks. 239

8.1.1 Summary

The page discusses various options for customizing the gate workflow processes.

8.1.2 Overview of Gate Workflow Execution

Gate workflows are a set of configurable tasks executed on a gate-related object as a result of some action in the gate process. A simple example is the set of gate tasks executed when a truck transaction is submitted at a stage. A more complex example is the set of tasks executed when a truck notifies its arrival at a stage – this results in the execution of 3 workflows each for a different timing point (see Gate Workflow Timing Points).

The workflow executes on a workflow context object (instance of TransactionAndVisitHolder) which holds the subject of the workflow (Truck Transaction, Truck Visit or Appointment) and other context information. Errors are collected in the workflow and depending on the action being performed may result in an error being returned to the caller or a trouble situation being recorded.

8.1.3 Summary of Customization Options

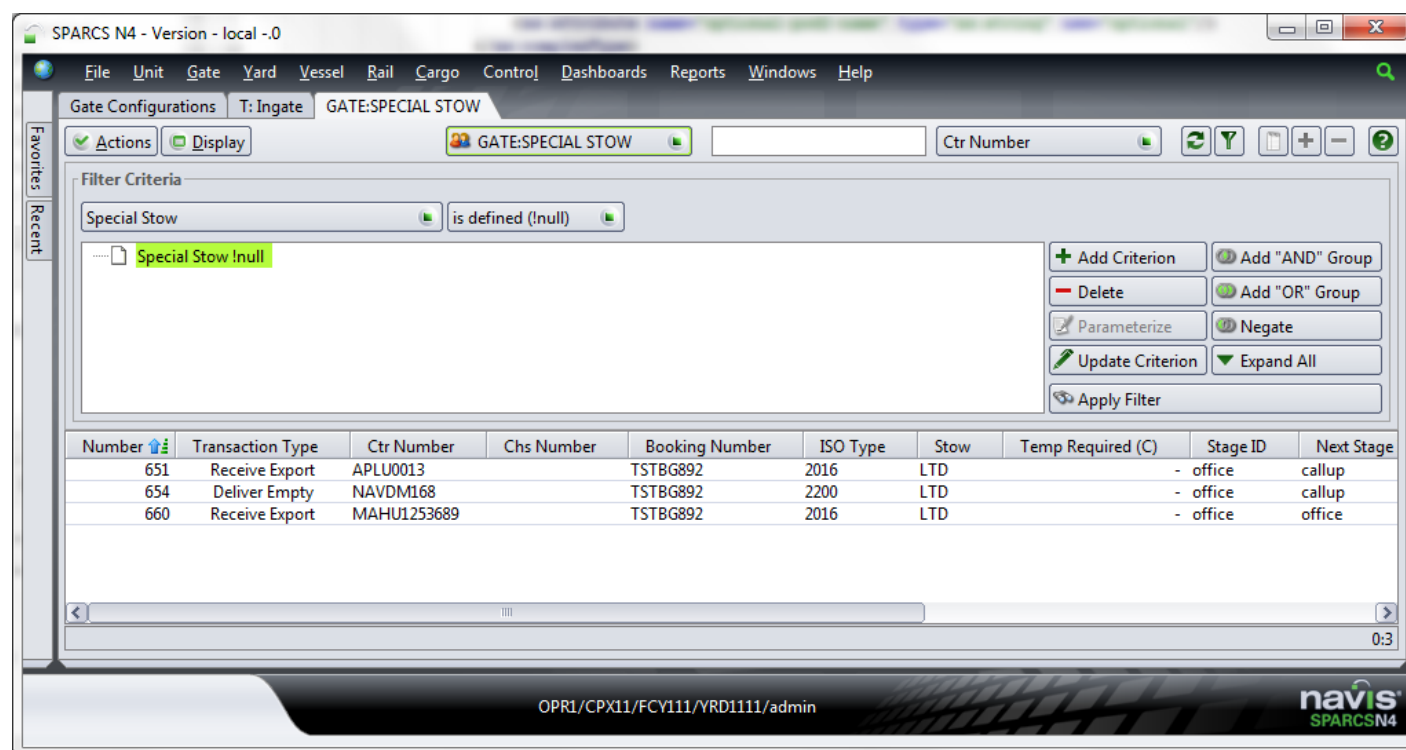
The gate workflow is inherently customizable via the inclusion or exclusion gate tasks, there is then the ability to further customize in varying degrees of complexity. Ranging from simplest to most complex these are:

- Dynamic inclusion or exclusion of tasks via filters
- Tasks that create custom error conditions
- Task to Override the next stage determination
- Groovy code defined in the gate task “customize” window, executed after the end of the task
- Groovy plug-ins called from the groovy code in the customize window
- Code extensions that are called at pre-defined trigger points in the workflow
- Code extensions that override the default implementation of a gate task, which may or may not include the default behavior and with the option for pre- or post-processing.

8.1.4 Task Filters

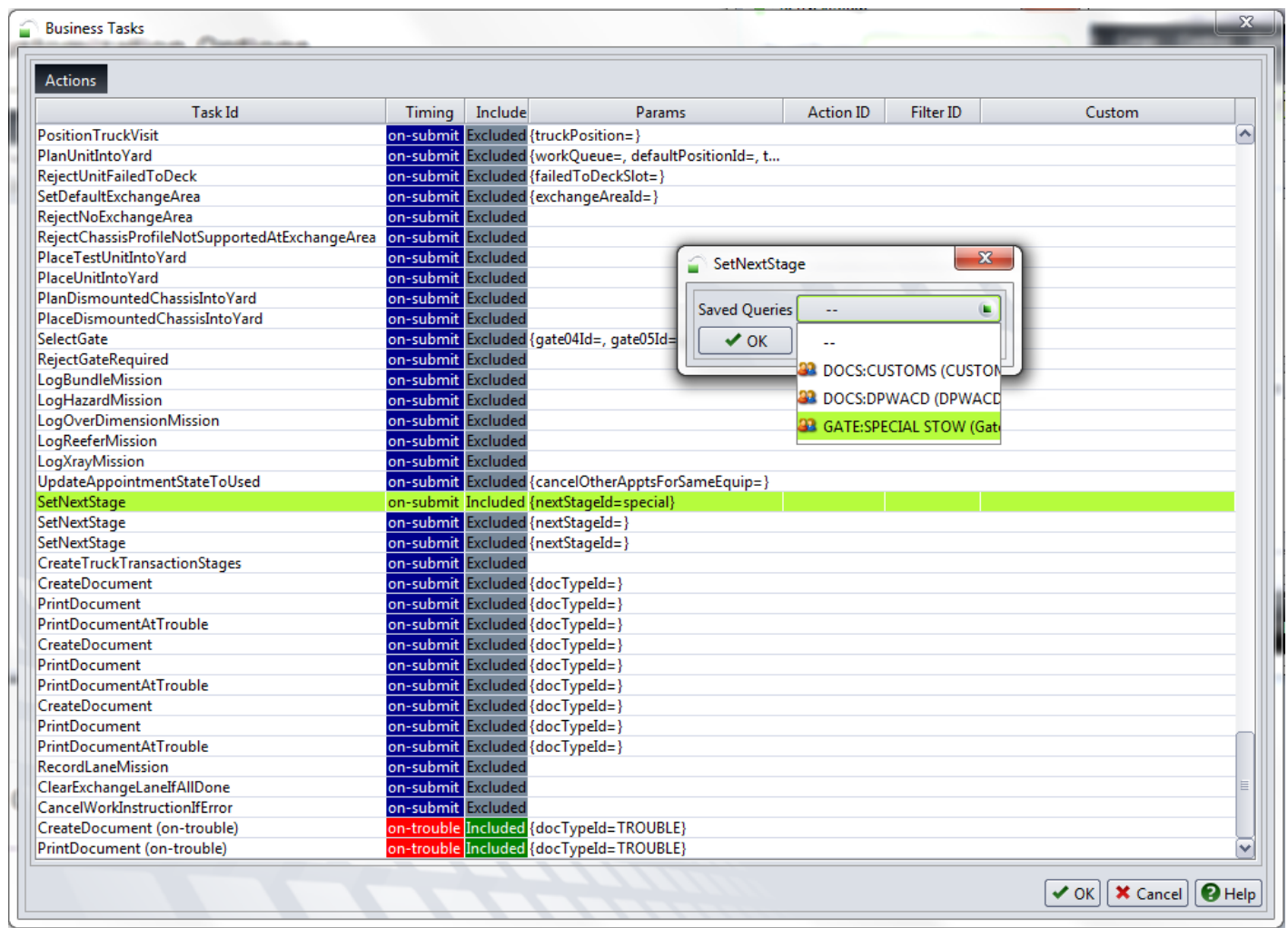
Filters can be used to dynamically include or exclude a gate task. Filters are defined on the truck transactions screen or truck visit details. They are in fact the same filters that are used to filter the screen views of the transactions or visit lists. Once a filter is defined on the transactions list screen it is available to be used to filter out gate tasks at the transaction level. Similarly filters defined on the truck visits list screen are available to filter out gate tasks at the truck visit level. The filter is assigned to the gate task on the gate tasks configuration screen where a drop-down list is presented with the available filters.

The screenshot shows a filter that selects only the transactions with values for special stow.



When a task is encountered in the workflow that has a filter set against it the task will only execute if the transaction (or visit) matches the criteria of the filter.

The screenshot below shows the application of the filter defined above to conditionally set the next stage for the transaction.



The big advantage of this method of customization is that the filter can easily be checked against the transaction data using the same filter on the list view.

The limitations of this method are that the filter conditions are restricted to those that are definable in the list view.

8.1.5 Tasks that Create Custom Error Conditions

The gate tasks `LogUserMessage` and `RejectConditionTrue` can be used to conditionally cause an error situation in the gate workflows. (See the help for details of these tasks).

LogUserMessage

`LogUserMessage` is an unconditional task that will force a message of the specified level. If this is `ERROR` then the workflow will fail. The trick is to use this task in conjunction with the filters described in the previous section to cause a conditional failure. Note that multiple declarations of this task can be used with different filters applied.

One possible value for the message level is `NONE`. In this case the task has no impact so it can be used as a place to attach groovy code where no other task behavior is desired.

Use of this task has the benefits and limitations of the filters described above.

RejectConditionTrue

RejectConditionTrue is more difficult to configure but is more powerful in that it can reference fields that aren't available in the transaction and visit filters. The parameters of the task specify a field name, comparison operator and a value. Fields on other entities can be referenced via the dot notation so to compare with the transit state on a unit facility visit the field id would be tranUnit.unitActiveUfv.ufvTransitState. The task also allows the creation of an "and" case by providing a 2nd set of field, operator and value. (An "or" case is always available by using multiple declarations of the task).

The disadvantage of RejectConditionTrue is that it requires knowledge of the field names in the data model.

8.1.6 Task to Override the Next Stage Determination

SetNextStage

The SetNextStage task can be used to force the transaction to a stage other than the next stage determined from the configuration. (Also see Help for this gate task).

Once a transaction has completed a stage the workflow will determine the next stage for the transaction. This will be the next stage in order of the defined stages at which that transaction type is declared. This can also be overridden for all transactions of a specific type by specifying the next stage in the gate configuration at the stage/transaction type level. Usually this is not necessary as the automatic determination is sufficient. To take an example of a gate configuration with the following stages - ingate, yard and outgate, if the transaction type TC (Through Container/ROT) is not defined at the yard stage then transactions of that type will automatically advance to outgate directly from the ingate without the need for a next stage specification.

The SetNextStage task allows for a finer tuning of the next stage determination when used in conjunction with filters. For example, if hazardous containers need to go to a special inspection area then a filter can be defined for hazardous containers and applied to the SetNextStage task which specifies that inspection stage.

8.1.7 Groovy Code in the Customize Window

Groovy code written in the customize window of the gate task configuration is executed within the following wrapper after execution of the built-in task. The code written is actually inserted at the point shown and then the intercept() method is called. The parameter inDao holds the TransactionAndVisitHolder which contains the TruckVisitDetails and the TruckTransaction (if executed in a transaction-level workflow). The parameter api is a pointer to the GroovyRoadApi class which has a number of helper methods (see JavaDoc).

```
import com.Navis.road.business.workflow.*;
import com.Navis.road.business.api.GroovyRoadApi;
public class Interceptor implements IGroovyRoadInterceptor {
    public void intercept(TransactionAndVisitHolder inDao, IUnitBean inUnitBean,
GroovyRoadApi api) {
        // *** CUSTOM CODE IS INSERTED HERE ***
    }
}
```

This scheme works well for small customizations but can become unwieldy for larger code segments. Because of the wrapper, there is no ability to include import statements so references outside the workflow package must include the full path name. Also as the code may be added for multiple transaction types it can lead to multiple copies of the same code that can lead to a management problem.

An advantage of this form of customization is that the groovy code is contained within the gate configuration xml file if it is exported.

8.1.8 Calling Groovy Plug-ins from the Customize Window

A helper method `api.getGroovyClassInstance()` is available to groovy code written in the customize window. This can be used to call centralized groovy modules defined in the groovy plug-ins screen. This overcomes the disadvantages of replicated code and unwieldy package references.

For example:

```
api.getGroovyClassInstance("MyGroovyClass").execute(inDao);
```

will call a method `execute(TransactionAndVisitHolder inDao)` defined in a groovy plug-in `MyGroovyClass` and pass in the `TransactionAndVisitHolder` from the gate workflow.

Note that referenced plug-ins are not included when the gate configuration is exported so all dependencies must be exported separately as `snx` files from the groovy plug-ins screen.

8.1.9 Code Extensions Executed in the Workflow

A code extension can be associated to a gate workflow by naming convention. When a gate workflow is executed it will search for a code extension named according to the following conventions:

- For truck visit level workflows (truck visit pre-stage, on-arrival, on-submit etc)

```
GateWorkflow{gateID}{stageId}{timingId}
```

For example truck arrival at the ingate stage of gate MAIN would be named `GateWorkflowMAINingateOnArrival`

- For transaction level workflows (submit-transaction)

```
GateWorkflow{gateID}{stageId}{transactionTypeId}
```

For example a receive export submitted at the ingate stage of gate MAIN would be named `GateWorkflowMAINingateRE`

The code extension should extend `AbstractGateWorkflowInterceptor` (implements `EGateWorkflowInterceptor`) and be of the type `GATE_WORKFLOW_INTERCEPTOR`.

The following methods are supported:

```
void preWorkflow(TransactionAndVisitHolder inWfCtx, List<CachedGateTask> inTasks);
```

executed immediately prior to the list of gate tasks. The tasks to be executed are passed as `inTasks` and this list may be modified

```
void postWorkflow(TransactionAndVisitHolder inWfCtx);
```

executed immediately following to the list of gate tasks.

Example

The code extensions below shows an example of workflow pre and post-processing with a Gate Workflow Interceptor.

The extension is of type `GATE_WORKFLOW_INTERCEPTOR` and will be automatically searched by name. The class is named `GateWorkflowTESTinRM` which indicates that it will be executed in the workflow for the gate called TEST, at the stage called "in" and for the transaction type RM.

This example gives special processing to trucks whose ID starts with "XX". Containers on XX trucks will not require inspection and any errors in processing the transaction will only be logged as warnings.

In the `preWorkflow()` method the list of business tasks to be executed is searched and the task `RejectInspectionNotFound` is excluded.

In the `postWorkflow()` method the message collector is checked for errors, if found they are demoted to warnings.

```
import com.Navis.external.road.AbstractGateWorkflowInterceptor
import com.Navis.external.road.EGateWorkflowInterceptor
import com.Navis.framework.util.TransactionParms
import com.Navis.framework.util.internationalization.UserMessage
import com.Navis.framework.util.message.MessageCollector
import com.Navis.framework.util.message.MessageLevel
import com.Navis.road.business.workflow.TransactionAndVisitHolder
import com.Navis.road.portal.configuration.CachedGateTask
public class GateWorkflowTESTinRM extends AbstractGateWorkflowInterceptor implements
EGateWorkflowInterceptor {
    public void preWorkflow(TransactionAndVisitHolder inWfCtx, List<CachedGateTask>
inTasks) {
        if (inWfCtx.getTv().getTvdtlsTruckId().startsWith("XX")) {
            CachedGateTask rejectInspectionNotFound = null;
            for (CachedGateTask task : inTasks) {
                if ("RejectInspectionNotFound".equalsIgnoreCase(task.getId())) {
                    rejectInspectionNotFound = task;
                }
            }
            if (rejectInspectionNotFound != null) {
                inTasks.remove(rejectInspectionNotFound);
            }
        }
    }
    public void postWorkflow(TransactionAndVisitHolder inWfCtx) {
        if (inWfCtx.getTv().getTvdtlsTruckId().startsWith("XX")) {
            MessageCollector mc = TransactionParms.getBoundParms().getMessageCollector();
            if (mc.hasError()) {
                for (UserMessage um : mc.getMessages()) {
                    if (MessageLevel.SEVERE.equals(um.getSeverity())) {
                        um.setSeverity(MessageLevel.WARNING);
                    }
                }
            }
        }
    }
}
```

8.1.10 Code Extensions to Override Gate Tasks

This feature allows a code extension to be called in place of the gate task by specifying the code extension name in place of the custom code.

The code extension should extend `AbstractGateTaskInterceptor` (implements interface `EGateTaskInterceptor`) and must be defined as type `GATE_TASK_INTERCEPTOR`.

`AbstractGateTaskInterceptor` provides default implementations for the following methods:

```
void preProcess(TransactionAndVisitHolder inWfCtx)
    default implementation: no action
```

```

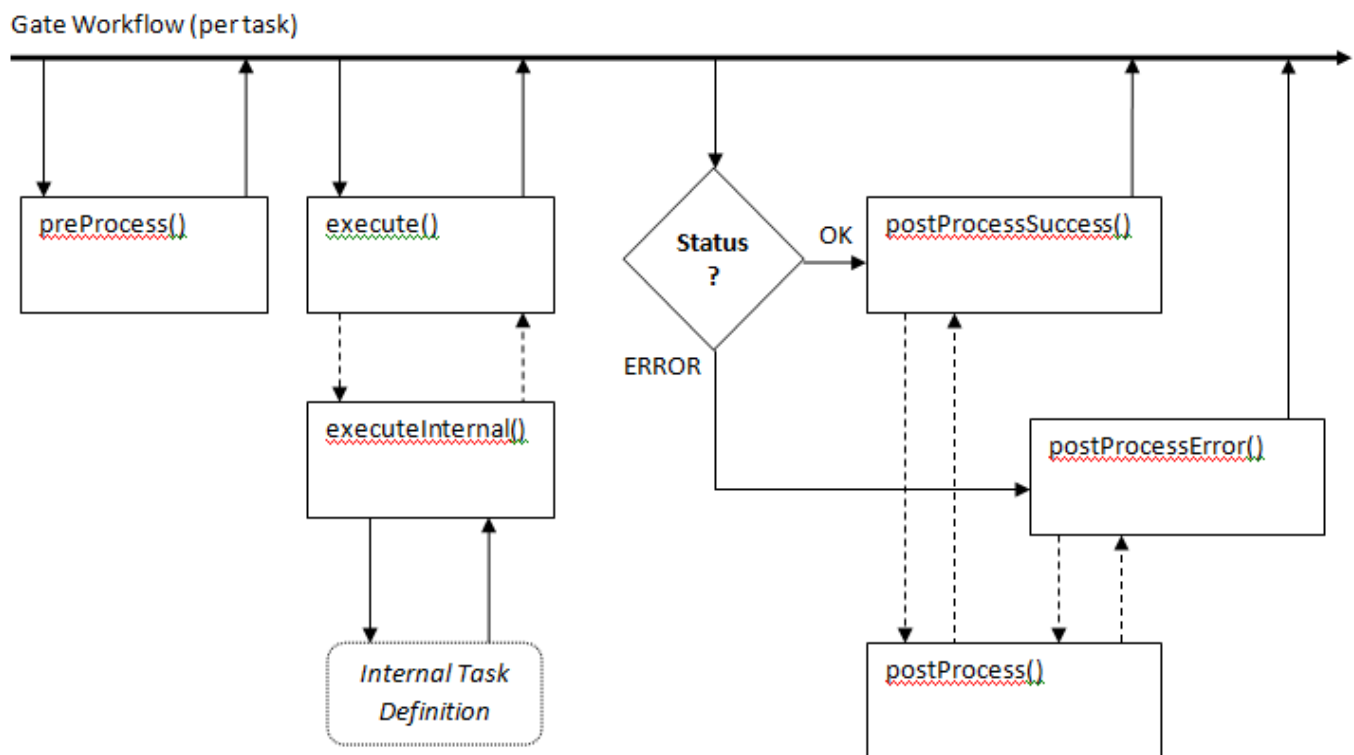
void execute(TransactionAndVisitHolder inWfCtx)
    default implementation: execute the internal definition of the gate task
void postProcess(TransactionAndVisitHolder inWfCtx)
    default implementation: no action
void postProcessSuccess(TransactionAndVisitHolder inWfCtx)
    default implmentation: call postProcess(inWfCtx)
void postProcessError(TransactionAndVisitHolder inWfCtx)
    default implementation: call postProcess(inWfCtx)

```

If a code extension is specified for a task and found then the workflow will execute in the following sequence - preProcess(), execute(), then either postProcessSuccess() or postProcessError() depending if any errors occurred during the execution of this task (ie not based on the overall status).

With no methods overridden the code extension will no effect other than to execute the internal definition of the gate task, ie it is the same as if it were not present.

The diagram below shows the interaction between the gate workflow and the methods of the code extension.



See the javadoc for AbstractGateTaskInterceptor for additional helper methods.

Example

The code extensions below shows an example of an over-ride for the task RejectDriverCardExpired. The code extension is of the type GATE_TASK_INTERCEPTOR, it is named here as GateTaskRejectDriverCardExpired but the name is not significant. The task is called by referencing its name in the Customize field of the task RejectDriverCardExpired as below.

Business Tasks							
Actions							
Task Id	Timing	Include	Params	Action ID	Filter ID	Custom	
AdvanceTruckToStage (pre-stage)	pre-stage	Excluded	{advanceFromStages=, advanceTra...				
CheckTruckOKAtStage	pre-stage	Included					
RejectTVAppointmentNbrRequired (pre-stage)	pre-stage	Excluded					
RejectTVAppointmentUnknown (pre-stage)	pre-stage	Excluded					
RejectTVAppointmentAlreadyUsed (pre-stage)	pre-stage	Excluded					
RejectTVAppointmentNotMatchingGate (pre-stag...	pre-stage	Excluded					
RejectTVAppointmentNotMatchingTruck (pre-sta...	pre-stage	Excluded					
RejectTVAppointmentNotMatchingTruckingCom...	pre-stage	Excluded					
RejectTVAppointmentNotMatchingDriver (pre-sta...	pre-stage	Excluded					
RejectTVAppointmentNotOnTime (pre-stage)	pre-stage	Excluded					
RejectWrongExchangeArea (pre-stage)	pre-stage	Excluded					
ReadGateAppointment (pre-stage)	pre-stage	Excluded					
RejectNoAppointmentForTruck (pre-stage)	pre-stage	Excluded					
RejectTruckBanned (pre-stage)	pre-stage	Excluded					
RejectDriverBanned (pre-stage)	pre-stage	Excluded					
RejectDriverCardExpired (pre-stage)	pre-stage	Included				GateTaskRejectDriverCardExpired	
RejectTruckingCompanyExpiredDriverAgreement ...	pre-stage	Excluded					
SetDefaultChassisProfile (pre-stage)	pre-stage	Excluded	{defaultChassisProfileId=}				
RejectChassisProfileNotSupported (pre-stage)	pre-stage	Excluded					
LoadIcerMessage (pre-stage)	pre-stage	Excluded	{userMessageNbr=, messageLevel=}				

In this example the `preProcess()`, `execute()` & 2 of the `postProcess()` methods have been overridden for illustrative purposes but it is possible to over-ride fewer methods.

Here the `preProcess()` method first checks to see if the card was an internal card and if so then it clears the truck driver details because for the purposes of this example we're saying that the internal cards are not driver-specific and hence we do not wish to record the driver or perform further checks.

The `execute()` method is first checking to see if the card ID starts with X. We're saying that X is an old series of cards which are no longer valid regardless of their original expiry date. For all other cards the regular `RejectDriverCardExpired` functionality is called via `executeInternal()`.

The method `postProcessError()`, called if the card failed the expiry check adds an info message to tell the driver he should make an appointment if he wants a new card.

The method `postProcessSuccess()`, called if the card expiry check passed, checks to see if the card will expire soon (within 60 days) and adds a warning message to the driver to renew at the help desk.

Note that message codes starting with `CUSTOM__` have been added here for the example and will need translations added to the application.

```
import com.Navis.argo.business.api.ArgoUtils
import com.Navis.external.road.AbstractGateTaskInterceptor
import com.Navis.external.road.EGateTaskInterceptor
import com.Navis.framework.util.DateUtil
import com.Navis.framework.util.internationalization.PropertyKey
import com.Navis.framework.util.internationalization.PropertyKeyFactory
import com.Navis.framework.util.message.MessageLevel
import com.Navis.road.business.model.TruckDriver
import com.Navis.road.business.model.TruckTransaction
import com.Navis.road.business.model.TruckVisitDetails
import com.Navis.road.business.util.RoadBizUtil
import com.Navis.road.business.workflow.TransactionAndVisitHolder
public class GateTaskRejectDriverCardExpired extends AbstractGateTaskInterceptor
implements EGateTaskInterceptor {
    // Internal cards aren't driver specific so erase the driver
    public void preProcess(TransactionAndVisitHolder inWfCtx) {
        TruckVisitDetails tvdttl = inWfCtx.getTv();
```

```

TruckDriver driver = tvdtmls.getTvdtmlsDriver();
if (driver != null && driver.getDriverIsInternal()) {
    tvdtmls.setTvdtmlsDriver(null);
}
}
// Old-series cards with X prefix have been withdrawn
public void execute(TransactionAndVisitHolder inWfCtx) {
    TruckVisitDetails tvdtmls = inWfCtx.getTv();
    TruckTransaction tran = inWfCtx.getTran();
    TruckDriver driver = tvdtmls.getTvdtmlsDriver();
    if (driver != null) {
        if (driver.getDriverCardId().startsWith("X")) {
            // error message for X cards
            PropertyKey CUSTOM__CARD_OLD_SERIES =
PropertyKeyFactory.valueOf("CUSTOM__CARD_OLD_SERIES");
            RoadBizUtil.getMessageCollector().appendMessage(MessageLevel.SEVERE,
CUSTOM__CARD_OLD_SERIES, null, null);
        }
        else {
            // otherwise the execute the regular RejectDriverCardExpired check
            executeInternal(inWfCtx);
        }
    }
}
// If it's already expired they need to make an appointment to renew
public void postProcessError(TransactionAndVisitHolder inWfCtx) {
    PropertyKey CUSTOM__RENEW_BY_APPOINTMENT =
PropertyKeyFactory.valueOf("CUSTOM__RENEW_BY_APPOINTMENT");
    RoadBizUtil.getMessageCollector().appendMessage(MessageLevel.INFO,
CUSTOM__RENEW_BY_APPOINTMENT, null, null);
}
// If it's OK but < 60 days to go then give them a warning
public void postProcessSuccess(TransactionAndVisitHolder inWfCtx) {
    TruckVisitDetails tvdtmls = inWfCtx.getTv();
    TruckDriver driver = tvdtmls.getTvdtmlsDriver();
    if (driver != null && driver.getDriverCardExpiration() != null) {
        if (ArgoUtils.timeNowMillis() + DateUtil.MILLIS_PER_DAY * 60 >
driver.getDriverCardExpiration().getTime()) {
            PropertyKey CUSTOM__RENEW_AT_HELPDESK =
PropertyKeyFactory.valueOf("CUSTOM__RENEW_AT_HELPDESK");
            RoadBizUtil.getMessageCollector().appendMessage(MessageLevel.WARNING,
CUSTOM__RENEW_AT_HELPDESK, null, null);
        }
    }
}
}
}
}

```

Part 9

Gate API Specification

In this Section:

Gate API Specification Overview.	244
Process API's Provided by N4.	250
Process API's Provided by Other Systems.	264
Driver Interaction.	267
Process Truck API Workflow.	269
Lane Monitor View.	273
Support for Custom API messages.	274
Including schema extension fields.	275

9.1 Gate API Specification Overview

This document describes how you can automate gate processing in N4 using Web Service API's. Most terminals use their own proprietary or third party Gate Operating System (GOS) software to manage and steer diverse and localized gate processes. Typically, the GOS comprises of customer-specific components, such as gate processes, Optical Character Recognition (OCR) Systems, RFID, kiosk screens and other specific interfaces.



N4 Web Services provide a critical link by leveraging various technologies used at the terminal to provide greater reliability and integration. The argoservice Tester view in N4, designed to be a development tool, allows you to test both basic and generic Web Service APIs within N4 without involving other Web Service resources. You can use the formal API's predefined in N4 to interact with your gate operating system (GOS). This licensed feature requires specific user privileges.

You can also use the argoservice Tester to parse and execute multi-line Groovy scripts consisting of XML elements. The argoservice Tester supports multiple Groovy commands, file inclusion, and variable substitution. The script processor parses the input XML elements in the request by substituting variable values and expanding included files. It then executes the resulting Groovy elements in the request, and provides live status reports as part of the response.

9.1.1 Levels of Gate Automation

Automating gate processing at your terminal depends upon the technology available at your site. A combination of these elements varies widely by installation and plays a major role in deciding the level of automation possible. The below image is a high level overview of a the process adhered by most terminals. This process can be automated at every stage and at each of these stages, GOS and N4 communicate with each before progressing further.

9.1.1.1 Total Automation

Using technology which identifies each container and the truck, if all drop-offs are pre-advised (or have appointments) and appointments are created for all pickups planned for the truck, the gate process upon the truck's arrival at the terminal can be entirely automated. Usually, for subsequent gate stages, just the technology alone is sufficient to entirely automate the processing.

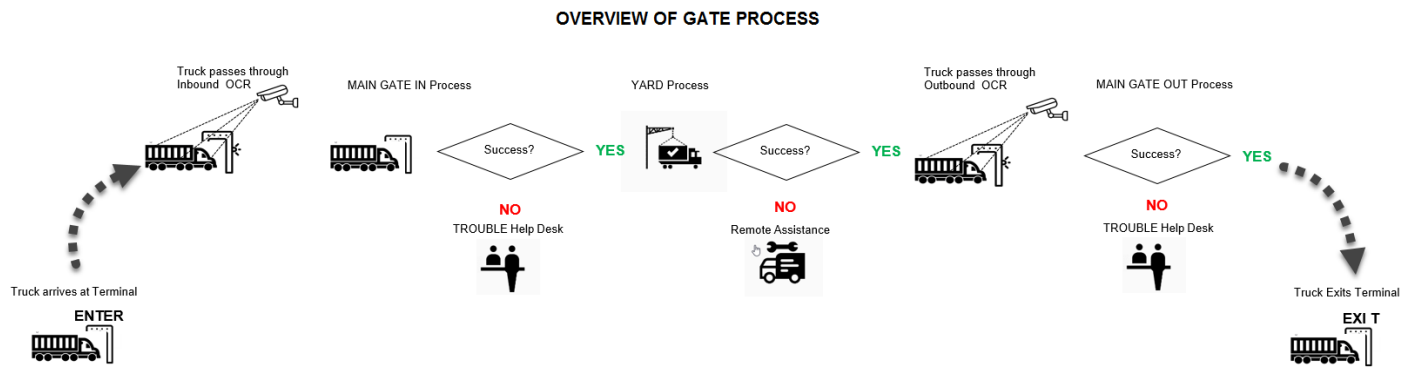
9.1.1.2 Semi-automation

Even with this technology, if pre-advised dropoffs or appointments and pickup appointments are not consistently provided, it is not possible to entirely automate the in-gate. Some intervention may be required, by either the driver or a clerk to provide any missing information.

9.1.1.3 Data Entry Assist

Some of this technology might be present with little prior information and therefore a clerk is always required to process the truck. At best, the technology is only used to assist in the clerk's data entry.

9.1.2 General Process Flow



9.1.2.1 Automated Gates

For gates that are entirely automated, the general interaction simply requires that the GOS send the scanned information to N4 via the *process-truck* API. Using the scanned information, and pre-lodged information may be sufficient to do entire processing for the truck.

However, even for a highly automated gate, determining the pickups may require some interaction from the driver to select the pickups. Ideally this will just be a simple matter of the driver selecting from a list of outstanding pickup appointments or entering specific container numbers. The N4 auto-gate workflow will be configured if appointments are used. Furthermore, N4 can be configured to automatically process any and all of the trucks pickup appointments if all the appointments can be reasonably preformed on one truck visit based on the max TEU parameter discussed later in this section.

If pickup appointments are supported, N4 will return a list of the possible pickup appointments in the response to the *process-truck* call. The GOS is expected to display this list to the driver and either call N4 with the driver's selection using the *submit-transaction* API or possibly the stage-done API if the driver has no further selections.

After each pickup appointment is performed, N4 will evaluate the remaining pickup appointments, and if so configured, N4 will automatically assume the pickup processing is complete. Also, a max TEU parameter can be set to indicate the pickup process is complete if selecting any further pickups would exceed this max TEU value. For example, assume max TEU is set to 2, and the driver has two 20' and one 40' appointment. If a 20' appointment is selected, only the remaining 20' pickup will be proposed to the driver as a possible second pickup, since choosing the remaining 40' pickup would exceed the max TEU setting.

If the work flow is configured to solicit containers numbers from the driver, N4 will not assume the pickup process is complete if there are no remaining pickup appointments. This way the GOS can solicit container numbers from the driver beyond any appointments and send these to N4 via the *submit-transaction* API. However, N4, if so configured, N4 will still enforce the max TEU setting regardless if the pickups where from an appointment or a specific container number entry.

9.1.2.2 Self-help Gates

For this style of gate, it is expected the driver will enter the information that would normally be provided by a dropoff appointment or pre-advised container. For this situation an entirely different workflow is provided. Instead of sending all the scans in one call, the GOS needs to construct the workflow from lower level APIs and stitch together the workflow and deal with unannounced drop-offs.

For each scanned container, the GOS should call the N4 *read-preadvise* API. N4 will return what is known about the

container from its master equipment inventory, pre-advised containers or a possible drop-off appointment. The GOS can then present this information to the driver to review and change present to N4 using the *submit-transaction* API.

Once the drop-offs have been processed, the GOS can request the pickups from the driver and send each selected container by calling the *submit-transaction* API. Finally, when the driver is finished the GOS notifies N4 using the stage-done API.

If pickup appointments are supported the GOS can optionally call the *request-pickup-appointments* API. Depending on the N4 auto-gate configuration as discussed in section 4.1, N4 might automatically process the trucks appointment or just return a list of the trucks outstanding pickup appointments. This way, the use of appointments can be encouraged even though the normal practice is for the driver to specify the container numbers for the pickups.

9.1.3 General API Structure

Web Services are used since they are widely accepted and supported by most platforms and programming environments. Each Web Service is defined using the Web Service Definition Language XML document referred to as a WSDL. A WSDL describes a set of API's including the calling arguments and expected response. In addition, the elements and attributes embedded in a WSDL can be further specified by including additional schemas to specify their structure.

To achieve a consistent process flow it is necessary at some points for N4 to initiate an action which is managed by the GOS. For instance the opening of a gate arm could happen from the GOS if all transactions completed successfully when it received a successful response, however if the clerk has to intervene using the N4 user interface then a call has to be made out from N4 to the GOS when the clerk completes the processing. To achieve this, the list of API's specified also include API's to be implemented on the GOS that N4 will call out to.

9.1.3.1 Common Webservice API

Due to the variety of interfaces required to support the numerous gate operations, a common WSDL with just one general purpose API is used to cover all these variations. The common WSDL API has a header structure that provides for the elements and attributes that are usually present in any request. Details that are specific to each individual process are carried in a common payload argument that is an XML document specific to the request. Outbound [ARGO:from N4's perspective] web-services, i.e. those which are implemented by the GOS or tracking systems are specified with the same WSDL.

Using this approach not only prevents the proliferations of WSDL definitions, but also has the flexibility to provide downward compatibility as the process details evolve. Otherwise, if all the specific arguments were explicitly defined in numerous special purpose WSDL API definitions, the WSDL could not be modified independently by either party. Most Web Service interfaces do not support even the slightest change in a WSDL definition.

Since the details are only in the payload of the API call and response, extensive enhancements can be made to the schema of the payload document without causing backward compatibility issues. Typically attributes and elements can be added to the payload schema without effecting callers that are not providing or expecting these new document entries. When retrieving data from the payload the reader should anticipate that some values may not even be present if the caller is using a previous version of the payload schema.

9.1.3.2 Accessing the Common Webservice API

The specification and instructions for accessing the common web-service API are detailed under [N4 Web Services](#) available in the N4 SDK.

All calls to the common Web Service API have the same general form:

- The scope of the request, i.e. the terminal operator, complex and facility.

- User credentials
- Preferred language for returned messages
- XML payload for the specific operation

The response also has a common format:

- Any messages that have been generated during the call (info, warning or severe)
- A single status code indicating the worst level of any of the messages
- Results from the specific operation



Callers are strongly discouraged from acting upon any specific error message as the errors are subject to change. Instead, the caller should only be concerned if the API failed or succeeded and treat the messages as free format text to display and/or log.

9.1.3.3 Gate API Message Naming Conventions

The gate API's are implemented using the argobasic webservice. Gate API calls are identified by the root element <gate> in the payload of the argobasic request. The individual API's are specified as the immediate children of the <gate> element.

The API's are defined in the files GateWebserviceRequest.xsd and the corresponding responses in GateWebserviceResponse.xsd. API's are named according to the action they perform. The include file gate-types.xsd defines business entities that are parameters to the requests or content of the responses.

The structure of the messages are defined in GateWebserviceRequest.xsd and GateWebserviceResponse.xsd down to the level of the business entities and the entities themselves in gate-types.xsd. (The term entities is used loosely because there may be more than one definition of the same N4 business entity in order to more tightly specify the API's).

Entities are defined as complex types in order to encourage re-use where possible.

Types are named (Entity)Type using mixed case (including first character uppercase) with no separators between words. eg ContainerType

Elements and attributes are named using lowercase with - (hyphen) separators between words. (Note that these form part of the published API definition so these should be agreed before publication). eg truck-transaction

The use of abbreviations is discouraged and should be restricted to obvious and low-value words. eg gate-id but truck-visit (not trk-visit).

9.1.3.4 XML Payload Formats

Whereas the interaction with the common Web Service is the same for a number of N4 modules, the contents of the xml payload and the results are specific to the application API being invoked. The formats of the XML payloads for the gate Web Service are covered later in this document. The process API's are described in the next section. The individual data elements are defined in the respective xsd's.

9.1.3.5 XML Request Format

The format of the xml request payload is defined in GateWebserviceRequest.xsd for calls to N4 and GosWebserviceRequest.xsd for calls to the GOS.

A sample message is shown below. The <gate> tag identifies that the payload is for the gate Web Service (GOS

implemented Web Service have a <gos> tag). <record-scan> indicates the name of the process API that is being invoked and within that is the data required for the action.

```
<gate>
  <record-scan scan-status="0">
    <gate-id>MAIN</gate-id>
    <stage-id>SECURITY</stage-id>
    <truck id="A56874" trucking-co-id="TC1"/>
    <truck-visit gos-tv-key="624093573762"/>
    <timestamp>2007-07-28T13:45:12</timestamp>
  </record-scan>
</gate>
```

9.1.3.6 XML Response Payload Format

N4 responds with payloads specified in GateWebserviceResponse.xsd, the GOS responses have no payload, only a status as there is no other information to return. The XML content is returned as the first query result within the common response of the common Web Service API response (see [N4 Web Services](#) in N4 SDK). For successful calls the details of the transaction, ticket or appointments are returned. The subsequent call from the GOS to N4 *submit-transaction* should contain only the field ID and the value pairs.

9.1.3.7 XML Response Format Configuration

In order to support conflicting requirements some response message configuration has been introduced. Configuration can be by application setting or by parameters in the request message. Setting based configuration is primarily used to maintain compatibility across application releases whereas parameterized configuration is used to tailor responses to a customer's specific requirement.

9.1.3.8 Response Format Configuration Settings

Element <truck-visit>

Prior to the 2.0 release all API calls that returned truck visits only returned a single truck visit. As of the 2.0 release API's were introduced that required the returning of multiple truck visits. The previous format of <truck-visit> and <truck-transactions> elements, each at the same level within the response does not handle multiple visits well as it requires the caller to work out which transactions are associated with which visit.

An alternative format for <truck-visit> has been introduced whereby the <truck-transactions> element is contained within the <truck-visit> element so the response indicates the relationship between truck-visit and transactions.

The configuration setting **ROADAPI001 TRUCK_TRANSACTION_RESPONSE_ATTACHMENT** controls the format with the follow values:

- TRUCK_TRANSACTION_RESPONSE_ATTACHMENT_RESPONSE: The <truck-visit> and <truck-transactions> elements are at the same level in the response. This is the default setting and is compatible with pre-2.0 releases.

Example:

```
<request-visit-details-response> <truck-visit tv-key="9043651" gos-tv-key="521" next-stage-id="IngateInsp"
status="OK"> <documents /> </truck-visit> <truck-transactions tv-key="9043651"> <truck-transaction tran-
key="9043654" tran-nbr="4594" tran-type="DOM" category="STRGE" freight-kind="MTY" next-stage-
id="Ingate" status="OK"> <container eqid="MHRU521.1" type="2200" line-id="UNK" has-documents="false" />
<documents /> </truck-transaction> </truck-transactions> </request-visit-details-response>
```

- TRUCK_TRANSACTION_RESPONSE_ATTACHMENT_TRUCK_VISIT: The <truck-transactions> element is contained within the <truck-visit> element.

Example:

```
<request-visit-details-response> <truck-visit tv-key="9043651" gos-tv-key="521" next-stage-id="IngateInsp"
status="OK"> <documents /> <truck-transactions tv-key="9043651"> <truck-transaction tran-key="9043654"
tran-nbr="4594" tran-type="DOM" category="STRGE" freight-kind="MTY" next-stage-id="Ingate" status="OK">
<container eqid="MHRU521.1" type="2200" line-id="UNK" has-documents="false" /> <documents /> </truck-
transaction> </truck-transactions> </truck-visit> </request-visit-details-response>
```

Notes

- The configuration setting applies to all API's that return a <truck-visit> and transactions. This enables the customer to use the same interpretation for all <truck-visit> responses.
- The attribute tv-key has also been added to the <truck-transactions> element. This allows a programmatic association between truck-visits and transactions if the pre-2.0 format is used.

Process API's Provided by N4

This section describes each API as named in the XSD's. API's implemented by the GOS are indicated as "(outbound)".

In This Section

read-preadvise.	251
record-scan (truck only).	251
record-scan (truck + equipment).	251
request-visit-details.	252
list-transactions.	252
update-appointment.	252
request-trouble-transactions.	252
request-stage-details.	252
request-stage-details (visit/tran key).	252
update-truck-visit (scale-weight-kg).	252
create-truck-visit-appointment.	252
create-truck-visit-appointment(update truck position and door direction).	252
update-truck-visit-appointment.	252
cancel-truck-visit-appointment.	253
process-truck.	253
create-truck-visit.	259
notify-arrival.	259
request-pickup-appointments.	259
submit-transaction.	259
submit-multiple-transactions.	260
stage-done.	260
cancel-transaction.	260
cancel-truck-visit.	260
need-assistance.	261
create-appointment.	261
create-appointment.	261
update-appointment.	262
cancel-appointment.	262

9.1.1 read-preadvise

Using this API for a self-help gate, the GOS container read the possible existing details for a drop that is either pre-advised or has an existing appointment.

Context Attributes: gate Id, stage id, lane id

Data Attributes: container id

Response: a detailed payload is returned containing the container type, export booking, gross weight, seal numbers, etc.

9.1.2 record-scan (truck only)

When the gate appointment is created manually or through API's, the container number may not be always available. In such cases, a appointment is created without the container number. Using this gate appointment, you can process a gate transaction using a record-scan API followed by a process-truck API.

Typically this API is used for a gate where a clerk does the actual processing and the scanned data is used to assist the data entry. The scan-set-id must be unique for all requests, except for cases where the intent is to update the existing scan information that was previously sent with a scan-set-id.

At this point in time N4 links the scanned container to the gate appointment if a match exists. In case of a mismatch, the appointment information is not integrated with the transaction record. N4 displays an error based on the gate business tasks configured in your terminal.

Context Attributes: gate Id, stage id, lane id, scan-set-id

Data Attributes: accessory ids, chassis Id, seals, scale weight, truck Identifier (or license nbr), bat nbr

Response: other than the common response, no payload is returned.

9.1.3 record-scan (truck + equipment)

When the gate appointment is created manually or through API's, the container number may not be always available. In such cases, a appointment is created without the container number. Using this gate appointment, you can process a gate transaction using a record-scan API followed by a process-truck API.

Typically this API is used for a gate where a clerk does the actual processing and the scanned data is used to assist the data entry. The scan-set-id must be unique for all requests, except for cases where the intent is to update the existing scan information that was previously sent with a scan-set-id.

At this point in time N4 links the scanned container to the gate appointment if a match exists. In case of a mismatch, the appointment information is not integrated with the transaction record. N4 displays an error based on the gate business tasks configured in your terminal.

Context Attributes: gate Id, stage id, lane id, scan-set-id

Data Attributes: container ids, container types, accessory ids, chassis Id, seals, scale weight, truck Identifier (or license nbr), bat nbr

Response: other than the common response, no payload is returned.

9.1.4 request-visit-details

This API is typically used at the outgate to fetch details of a truck visit by providing the truck license number.

9.1.5 list-transactions

The API is used to list all transactions associated with the line for the specified date. Details of the documents associated with the transactions are also listed in the response message.

9.1.6 update-appointment

This API is used to update appointment details such as the equipment type ISO, Freight Kind, etc.

9.1.7 request-trouble-transactions

Use this API to list out the trouble transactions associated with a trucking company. You can specify the begin date and the end date to list all trouble transactions for a specific period.

9.1.8 request-stage-details

Use this API request to get a list of transactions in a particular gate stage. For example, to list transactions in the ingate stage, post the API request with <gate-id>ingate</gate-id>.

9.1.9 request-stage-details (visit/tran key)

This API is similar to the request-stage-details API request. In addition, you can also provide the truck visit and truck transaction gkeys to get a list of transactions.

9.1.10 update-truck-visit (scale-weight-kg)

Use this API to update weight details. You must include the truck license number in the API to update the weight for the associate truck visit.

9.1.11 create-truck-visit-appointment

This API is used to create transaction appointments in N4.

9.1.12 create-truck-visit-appointment(update truck position and door direction)

In addition to creating a transaction appointment, truck position and door direction details are provided in the API so that the details are added to the newly created appointment.

9.1.13 update-truck-visit-appointment

This API is used to update transaction appointments in N4

9.1.14 cancel-truck-visit-appointment

This API is used to cancel transaction appointments in N4

9.1.15 process-truck

Using this API the processing of a truck can be initiated in N4. This API can include all the information scanned in the lane. So calling the "Record Scan" API prior to this call is not necessary. However the information from prior calls of "Record Scan" will be used to supplement the information sent in the "process truck" call.

Normally for a highly automated gate with appointments, the entire processing of the truck at the in-gate can be performed with this one call. Without appointment/pre-advise information, some further interaction with the driver or the assistance of a clerk may be required. This situation is discussed separately in a later section.

For other gate stages, such as the out-gate, or a straddle exchange lane, the entire processing can almost always be performed entirely from this one API call.

Context Attributes: gate Id, stage id, lane id

Data Attributes: Truck details, Chassis Profile details, Driver information details, Images, Scale weight information, Truck visit appointment number or Gate transaction appointment numbers, Equipment Scan information consisting of container scan, Accessory scan and Chassis scan, Image Details and scan set related details. Container scan has container accessory details, container damages details, container placard details, container VGM details (VGM weight, VGM weight unit and VGM entity) and container bundle(payload) details. Accessory scan has accessory damage details in addition to basic accessory info. Chassis scan has chassis accessory details, chassis damages details, chassis placard details and chassis bundle(payload) details.

Sample request and response message formats for process-truck gate api are shown below.

Request:

Sample Process Truck Message

```
<gate>
  <process-truck scan-status="0" scan-set-id="33333318" no-content="false">
    <gate-id>GEN_GATE</gate-id>
    <stage-id>ingate</stage-id>
    <lane-id>IGL1</lane-id>
    <truck license-nbr="SMPTL1755"/>
    <truck-visit gos-tv-key="12352" appointment-nbr="1314"/>
    <equipment>
      <container eqid="SMP01090115" type="2200" on-chassis-id="SMPC01090115" vgm-
weight="15000" vgm-weight-unit="lb">
        <accessory eqid="SMPA01090115" type="A01" owner-id="APL" operator-id="APL"/>
        <damages>
          <damage severity="Major" type="DENT" component-id="DOOR"/>
        </damages>
        <bundles>
          <bundle bundle-nbr="SMP01090215" eq-type="2200">
            <damages>
              <damage severity="Major" type="RUST" component-id="ROOF"/>
            </damages>
          </bundle>
        </bundles>
      </container>
```

```

    <chassis eqid="SMPC01090115" type="C20" is-owners="false"/>
  </equipment>
  <timestamp>2007-07-28T13:45:12</timestamp>
</process-truck>
</gate>

```

Response: For completely automated gates the ticket information is returned from a successful call. For less automated gates, information describing what additional data is required from the driver is returned.

Process-Truck API response

```

<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="0" status-id="OK">
  <argo:gate-response xmlns:argo="http://www.Navis.com/argo" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www.Navis.com/argo
GateWebserviceResponse.xsd">
    <process-truck-response>
      <truck-visit tv-key="8841" gos-tv-key="12352" appointment-nbr="1314" next-stage-
id="outgate" status="OK" gate-id="GEN_GATE" entered="2015-01-09T13:59:32">
        <trucking-co id="SPTC1" />
        <chassis-profile id="40" />
        <driver card-id="SMPDC001" driver-name="SATISH PAI" />
        <documents>
          <document doc-key="9213" type="ROUTE" no-of-copies="0">
            <content><![CDATA[<argo:Document xmlns:argo="http://www.Navis.com/
argo"><argo:docDescription><docName>ROUTE</docName><docType>55</docType></
argo:docDescription><argo:docBody><argo:truckVisit><tvdtlsLicNbr>SMPTL1755</
tvdtlsLicNbr><tvdtlsTruckId>SMPTID001</tvdtlsTruckId><tvdtlsDriverCardId>SMPDC001</
tvdtlsDriverCardId><tvdtlsDriverName>SATISH PAI</
tvdtlsDriverName><tvdtlsDriverLicenseNbr>DL10131101</
tvdtlsDriverLicenseNbr><tvdtlsTrkCompany>SPTC1</
tvdtlsTrkCompany><tvdtlsTrkCompanyName>SATISH PAI TRUCK CO1</
tvdtlsTrkCompanyName><tvdtlsTrkId>SMPTID001</tvdtlsTrkId><tvdtlsBatNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tvdtlsTvKey>8841</
tvdtlsTvKey><tvdtlsGosTvKey>12352</tvdtlsGosTvKey><tvdtlsTrkStartTime>2015-01-09
13:59:32</tvdtlsTrkStartTime><tvdtlsEntryLane><laneId>IGL1</laneId></
tvdtlsEntryLane><tvdtlsFlexString01 xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tvdtlsFlexString02 xsi:nil="true" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance"/><tvdtlsFlexString03 xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><argo:tvdtlsRequiredStages><stageId>ingate</
stageId><stageOrder>1</stageOrder><stageDescription>ingate</stageDescription></
argo:tvdtlsRequiredStages><tvdtlsAppointmentNbr>1314</
tvdtlsAppointmentNbr><tvdtlsAppointmentState>CREATED</
tvdtlsAppointmentState><argo:tvdtlsAnnouncements/></
argo:truckVisit><argo:trkTransaction><argo:tranCtrOperator><bizuId>APL</bizuId></
argo:tranCtrOperator><argo:tranUnit><unitId>SMP01090115</unitId></
argo:tranUnit><argo:tranFacility><fcyId>PORTELIZABETH</fcyId></
argo:tranFacility><argo:tranComplex><cpxId>USNWK</cpxId></
argo:tranComplex><argo:tranOperator><oprId>MAH</oprId></
argo:tranOperator><argo:tranCtrPosition><posLocId>Y-MAIN-FLD</posLocId></
argo:tranCtrPosition><argo:tranChsPosition><posLocId>MAIN</posLocId></
argo:tranChsPosition><tranNbr>4012</tranNbr><tranExchangeAreaId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranSubType>RM</
tranSubType><tranTruckerFriendlySubType>DOM</
tranTruckerFriendlySubType><tranStatus>COMPLETE</tranStatus><tranCreated>Jan 9, 2015
1:59 PM</tranCreated><tranCreator>admin</tranCreator><tranStageId>ingate</
tranStageId><tranNextStageId xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/

```

```

XMLSchema-instance"/><tranTrouble xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranTroubleStatus xsi:nil="true" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance"/><tranHadTrouble>false</tranHadTrouble><tranCancelCode
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranNotes
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranHandled>2015-01-09 13:59:48.0</tranHandled><tranTerminalId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranExportReleaseNbr
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranImportReleaseNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranAppointmentNbr>1313</tranAppointmentNbr><tranPinNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranTrkcId>SPTC1</
tranTrkcId><tranCtrTruckPosition xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranCtrDoorDirection xsi:nil="true" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance"/><tranCtrNbr>SMP01090115</tranCtrNbr><tranCtrOwnerId
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCtrFreightKind>MTY</tranCtrFreightKind><tranCtrTypeId>2200</
tranCtrTypeId><tranCtrIsDamaged>true</tranCtrIsDamaged><tranCtrSafeWeight>9000.0</
tranCtrSafeWeight><tranCtrTareWeight>1900.0</
tranCtrTareWeight><tranCtrGrossWeight>1900.0</tranCtrGrossWeight><tranCtrNbrAssigned
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCtrTicketPosId>FLD</tranCtrTicketPosId><tranCtrAccNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranCtrAccTypeId
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCtrAccFuelLevel xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranChsNbr>SMPC01090115</tranChsNbr><tranChsOwnerId>APL</
tranChsOwnerId><tranChsIsOwners>false</tranChsIsOwners><tranChsTypeId>C20</
tranChsTypeId><tranChsIsDamaged>false</tranChsIsDamaged><tranChsTareWeight>1900.0</
tranChsTareWeight><tranChsSafeWeight>2700.0</tranChsSafeWeight><tranChsLicenseNbr
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranChsNbrAssigned>SMPC01090115</tranChsNbrAssigned><tranChsAccNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranChsAccTypeId
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranChsAccFuelLevel xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranTradeId xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranOrigin xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranDestination xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranLineId>APL</tranLineId><tranShipper xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranConsignee xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranScAgent xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranEgoEqIsoGroup>GP</
tranEgoEqIsoGroup><tranEgoEqLength>NOM20</tranEgoEqLength><tranEgoEqHeight>NOM86</
tranEgoEqHeight><tranEgoNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranMilitaryVNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranMilitaryTcn xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranMaterial>UNKNOWN</tranMaterial><tranShandId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranShandId2 xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranShandId3 xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranGroupId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranIdoId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranCommodityCode
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCommodityDescription xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranVentRequired>null</tranVentRequired><tranVentSetting>null</
tranVentSetting><tranIsHazard>false</tranIsHazard><tranIsPlacarded>false</
tranIsPlacarded><tranIsPlacardedOk>false</tranIsPlacardedOk><tranIsOog>false</
tranIsOog><tranCtrIsSealed>false</tranCtrIsSealed><tranIsXrayRequired>false</

```

```

tranIsXrayRequired><argo:tranCtrDmg><dmgitemLocation xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><dmgitemSeverity>MAJOR</
dmgitemSeverity><dmgitemReported>2015-01-09T13:59:47.000-08:00</
dmgitemReported><dmgitemDescription xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
/XMLSchema-instance"/><dmgitemType>DENT</dmgitemType><dmgitemTypeDescription>DEnt in
Body</dmgitemTypeDescription><dmgitemComponentId>DOOR</
dmgitemComponentId><dmgitemComponentDescription>DOOR</dmgitemComponentDescription></
argo:tranCtrDmg><tranCscDate xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
/XMLSchema-instance"/><tranUnitCategory>UnitCategoryEnum[STRGE]</
tranUnitCategory><argo:tranRequiredStages><stageId>ingate</stageId><stageSequence>1</
stageSequence><stageDescription>ingate</stageDescription></
argo:tranRequiredStages><tranStartTime>2015-01-09 13:59:47</
tranStartTime><tranEndTime>2015-01-09 13:59:48</
tranEndTime><tranAppointmentState>CREATED</tranAppointmentState><argo:Messages
locale="en_US"/></argo:trkTransaction></argo:docBody><argo:Messages locale="en_US"/></
argo:Document>]]></content>
</document>
</documents>
</truck-visit>
<truck-transactions>
<truck-transaction tran-key="12935" tran-nbr="4012" tv-key="8841" tran-type="DOM"
category="STRGE" freight-kind="MTY" mission="FLD" block-id="FLD" gate-id="GEN_GATE"
appointment-nbr="1313" status="OK" is-hazard="false">
<container eqid="SMP01090115" type="2200" on-chassis-id="SMPC01090115" chs-is-
owners="false" is-sealed="false" gross-weight="1900.0" safe-weight="9000.0" tare-
weight="1900.0" line-id="APL" has-documents="false" planned-slot="FLD " block-id="FLD"
owner-id="APL" operator-id="APL">
<accessory />
<damages>
<damage severity="MAJOR" reported="2015-01-09T13:59:47" type="DENT" type-
description="DEnt in Body" component-id="DOOR" component-description="DOOR" />
</damages>
<routing />
</container>
<chassis eqid="SMPC01090115" is-owners="false" type="C20" safe-weight="2700.0" tare-
weight="1900.0" owner-id="APL" operator-id="APL">
<accessory />
</chassis>
<documents>
<document doc-key="9210" type="DROPOFF" no-of-copies="0">
<content><![CDATA[<argo:Document xmlns:argo="http://www.Navis.com/
argo"><argo:docDescription><docName>DROPOFF</docName><docType>5</docType></
argo:docDescription><argo:docBody><argo:truckVisit><tvdtlsLicNbr>SMPTL1755</
tvdtlsLicNbr><tvdtlsTruckId>SMPTID001</tvdtlsTruckId><tvdtlsDriverCardId>SMPDC001</
tvdtlsDriverCardId><tvdtlsDriverName>SATISH PAI</
tvdtlsDriverName><tvdtlsDriverLicenseNbr>DL10131101</
tvdtlsDriverLicenseNbr><tvdtlsTrkCompany>SPTC1</
tvdtlsTrkCompany><tvdtlsTrkCompanyName>SATISH PAI TRUCK CO1</
tvdtlsTrkCompanyName><tvdtlsTrkId>SMPTID001</tvdtlsTrkId><tvdtlsBatNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tvdtlsTvKey>8841</
tvdtlsTvKey><tvdtlsGosTvKey>12352</tvdtlsGosTvKey><tvdtlsTrkStartTime>2015-01-09
13:59:32</tvdtlsTrkStartTime><tvdtlsEntryLane><laneId>IGL1</laneId></
tvdtlsEntryLane><tvdtlsFlexString01 xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
/XMLSchema-instance"/><tvdtlsFlexString02 xsi:nil="true" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance"/><tvdtlsFlexString03 xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><argo:tvdtlsRequiredStages><stageId>ingate</

```

```

stageId><stageOrder>1</stageOrder><stageDescription>ingate</stageDescription></
argo:tvdtlsRequiredStages><tvdtlsAppointmentNbr>1314</
tvdtlsAppointmentNbr><tvdtlsAppointmentState>CREATED</
tvdtlsAppointmentState><argo:tvdtlsAnnouncements/></
argo:truckVisit><argo:trkTransaction><argo:tranCtrOperator><bizuId>APL</bizuId></
argo:tranCtrOperator><argo:tranUnit><unitId>SMP01090115</unitId></
argo:tranUnit><argo:tranFacility><fcyId>PORTELIZABETH</fcyId></
argo:tranFacility><argo:tranComplex><cpxId>USNWK</cpxId></
argo:tranComplex><argo:tranOperator><oprId>MAH</oprId></
argo:tranOperator><argo:tranCtrPosition><posLocId>Y-MAIN-FLD</posLocId></
argo:tranCtrPosition><argo:tranChsPosition><posLocId>MAIN</posLocId></
argo:tranChsPosition><tranNbr>4012</tranNbr><tranExchangeAreaId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranSubType>RM</
tranSubType><tranTruckerFriendlySubType>DOM</
tranTruckerFriendlySubType><tranStatus>COMPLETE</tranStatus><tranCreated>Jan 9, 2015
1:59 PM</tranCreated><tranCreator>admin</tranCreator><tranStageId>ingate</
tranStageId><tranNextStageId xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
/XMLSchema-instance"/><tranTrouble xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
/XMLSchema-instance"/><tranTroubleStatus xsi:nil="true" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance"/><tranHadTrouble>false</tranHadTrouble><tranCancelCode
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranNotes
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranTerminalId
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranExportReleaseNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranImportReleaseNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
/XMLSchema-instance"/><tranAppointmentNbr>1313</tranAppointmentNbr><tranPinNbr
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranTrkcId>SPTC1</tranTrkcId><tranCtrTruckPosition xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranCtrDoorDirection xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranCtrNbr>SMP01090115</
tranCtrNbr><tranCtrOwnerId xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranCtrFreightKind>MTY</tranCtrFreightKind><tranCtrTypeId>2200</
tranCtrTypeId><tranCtrIsDamaged>true</tranCtrIsDamaged><tranCtrSafeWeight>9000.0</
tranCtrSafeWeight><tranCtrTareWeight>1900.0</
tranCtrTareWeight><tranCtrGrossWeight>1900.0</tranCtrGrossWeight><tranCtrNbrAssigned
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCtrTicketPosId>FLD</tranCtrTicketPosId><tranCtrAccNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranCtrAccTypeId
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCtrAccFuelLevel xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranChsNbr>SMPC01090115</tranChsNbr><tranChsOwnerId>APL</
tranChsOwnerId><tranChsIsOwners>false</tranChsIsOwners><tranChsTypeId>C20</
tranChsTypeId><tranChsIsDamaged>false</tranChsIsDamaged><tranChsTareWeight>1900.0</
tranChsTareWeight><tranChsSafeWeight>2700.0</tranChsSafeWeight><tranChsLicenseNbr
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranChsNbrAssigned>SMPC01090115</tranChsNbrAssigned><tranChsAccNbr xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranChsAccTypeId
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranChsAccFuelLevel xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranTradeId xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranOrigin xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranDestination xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranLineId>APL</tranLineId><tranShipper xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranConsignee xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranScAgent xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><tranEgoEqIsoGroup>GP</

```

```

tranEgoEqIsoGroup><tranEgoEqLength>NOM20</tranEgoEqLength><tranEgoEqHeight>NOM86</
tranEgoEqHeight><tranEgoNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranMilitaryVNbr xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranMilitaryTcn xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranMaterial>UNKNOWN</tranMaterial><tranShandId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranShandId2 xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranShandId3 xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranGroupId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranIdoId xsi:nil="true"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/><tranCommodityCode
xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/
><tranCommodityDescription xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"/><tranVentRequired>null</tranVentRequired><tranVentSetting>null</
tranVentSetting><tranIsHazard>false</tranIsHazard><tranIsPlacarded>false</
tranIsPlacarded><tranIsPlacardedOk>false</tranIsPlacardedOk><tranIsOog>false</
tranIsOog><tranCtrIsSealed>false</tranCtrIsSealed><tranIsXrayRequired>false</
tranIsXrayRequired><argo:tranCtrDmg><dmgitemLocation xsi:nil="true" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"/><dmgitemSeverity>MAJOR</
dmgitemSeverity><dmgitemReported>2015-01-09T13:59:47.000-08:00</
dmgitemReported><dmgitemDescription xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><dmgitemType>DENT</dmgitemType><dmgitemTypeDescription>DEnt in
Body</dmgitemTypeDescription><dmgitemComponentId>DOOR</
dmgitemComponentId><dmgitemComponentDescription>DOOR</dmgitemComponentDescription></
argo:tranCtrDmg><tranCscDate xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"/><tranUnitCategory>UnitCategoryEnum[STRGE]</
tranUnitCategory><argo:tranRequiredStages><stageId>ingate</stageId><stageSequence>1</
stageSequence><stageDescription>ingate</stageDescription></
argo:tranRequiredStages><tranStartTime>2015-01-09 13:59:47</
tranStartTime><tranEndTime>2015-01-09 13:59:48</
tranEndTime><tranAppointmentState>CREATED</tranAppointmentState></
argo:trkTransaction></argo:docBody><argo:Messages locale="en_US"/></argo:Document>]]</
content>
</document>
</documents>
</truck-transaction>
</truck-transactions>
</process-truck-response>
</argo:gate-response>
</argo-response>

```

For cargo, you can use can include attributes like the BI number, BIItem Nbr, line -id and Item Quantity in the Container tag. For example,

```

<gate>
<process-truck do-stage-done="true">
<gate-id>MG</gate-id>
<stage-id>office</stage-id>
<lane-id>L1</lane-id>
<truck license-nbr="ATOL" />
<driver card-id="ATOD" />
<truck-visit gos-tv-key="112211"/>
<equipment>
<container eqid="CONC1223344" type="20BR" tran-type="DOE">
<cargo item-quantity="100">
<bill-of-lading nbr="ABL6" line-id="ATO" bl-item-nbr="ITEM"/>
</cargo>
</container>

```

```
</equipment>  
</process-truck>  
</gate>
```

9.1.16 create-truck-visit

This API can be called to create the truck visit only. In an environment where pickups are configured to be automatically assigned, a call to *process-truck* with no drop-off data would create the truck visit but would also continue to assign the pickups.

Context Attributes: gate id, stage id, lane id

Data Attributes: truck license nbr, truck id, bat nbr

Response: no specific response

9.1.17 notify-arrival

This API is called to notify N4 that a truck has arrived at a certain stage and that N4 should initiate processing of that stage. Typically this is used to notify arrival at an exchange area. N4 activates the work instructions and the stage will be completed once the moves have occurred.

Context Attributes: gate id, stage id, exchange area id, lane id

Data Attributes: truck license nbr, truck id, bat nbr

Response: no specific response

9.1.18 request-pickup-appointments

The GOS can call this API to initiate the automatic processing of a truck's pickup appointments. If the work flow is so configured, N4 will automatically process the truck's appointments if they do not exceed the maximum TEU's configured. Otherwise, if there are too many appointments, this API will just return a list of outstanding appointments.

Context Attributes: truck visit identifier

Data Attributes: none

Response: list of containers for pickup

9.1.19 submit-transaction

For semi-automated gates, information may be required from the truck driver to proceed. Typically for pickups this is often required where the driver must specify something to determine what pickup should be performed.

Context Attributes: gate id, stage id, lane id

Data Attributes: truck visit identifier, transaction type, optional truck transaction identifier along with either an appointment id, container id, pin number, container VGM details (VGM weight, VGM weight unit and VGM entity), order number (if selecting empty by booking/EDO) or equipment order item (previous call was select empty by booking/EDO and a list of order items was returned for driver selection), truck license number, truck transaction tran type, cargo item quantity, unit id, bill of lading number, bill item number.

Response: Normally the standard response with no payload. If an appointment number was submitted, a list of possible remaining appointments will be returned.

9.1.20 submit-multiple-transactions

For truck visits with multiple transactions a new API is required to submit multiple transactions from a given gate stage, gate stage permitting. This new API will submit multiple transactions for the truck visit after the truck visit is created using create-truck-visit API in the first gate stage and before the stage-done API for all remaining stages

Context Attributes: gate Id, stage id, lane id

Data Attributes: truck visit identifier, and multiple transactions element truck-transactions consisting of individual truck-transaction elements.

Response: Response for submit-multiple-transactions API consist of truck visit element followed by truck-transactions elements which contains individual truck-transaction response from each individual submit-transaction API.

9.1.21 stage-done

Further selections may be provided to the driver, especially for pickups. The GOS will use this API to indicate the driver has no further selections.

Context Attributes: gate Id, stage id, lane id

Data Attributes: truck visit identifier

Response: ticket details for all transactions

9.1.22 cancel-transaction

GOS will call this API to request N4 cancel a gate transaction. Typically this is used for a failing pickup transaction that the driver has decided to forego. User can specify either the transaction gkey directly or the container number in which case the latest transaction associated with this container-nbr will be cancelled.

Context Attributes: gate Id, stage id, lane id

Data Attributes: transaction key (returned in *submit-transaction* API response) or container number

Response: other than the common response no payload is returned.

9.1.23 cancel-truck-visit

GOS will call this API to request N4 cancel a gate truck visit. This is a new API required to cancel a bare truck visit with no transactions or all canceled transactions.

Context Attributes: gate Id, stage id, lane id

Data Attributes: truck visit key or gos truck visit key(returned in create-truck-visit API response), skip-tv-setting-validation

Response: For bare truck visits or truck visits with canceled transactions response consists of truck-visit element containing truck visit gkey, gos truck visit gkey and truck visit status. For truck visits with active transactions, response consists of truck-visit element followed by truck-transactions element containing individual active truck-transaction elements.

9.1.24 need-assistance

GOS will call this API to request N4 to mark this lane as requiring assistance.

Context Attributes: gate Id, stage id, lane id

Data Attributes: none

Response: other than the common response no payload is returned.

9.1.25 create-appointment

A third party system can call this API to create a container appointment in N4.

Context Arguments: gate Id

Data Attributes: appointment-date, appointment-time, truck trucking-co-id, tran-type, container, pin-nbr, Driver and Truck information, Booking information, Container information, appointment validation-only attribute, etc.

Response: Includes appointment number, that was generated by N4, along with appointment details.

Variation: The **<appointment>** element of the create-appointment API has a Boolean attribute, **validation-only**, that specifies whether the appointment request is for validation purpose only or for creating and saving an appointment. When a third party system uses the Boolean attribute to indicate that the appointment request is for validation purposes only, N4 does not save any appointment data; however, N4 does process the appointment stage workflow and if any errors are generated, N4 returns the same in the response.

For backward compatibility, the default value for the **validation-only** attribute is **false**. This means that if the **validation-only** attribute is missing OR the whole **<appointment>** element is missing then the appointment is always created (if no rules fail), as before.

Important points about the **validation-only** attribute:

Default setting of this attribute is **false**.

When you do not add the **validation-only** attribute, or, specify the **validation-only** attribute as **false**, or, do not specify the **<appointment>** element; N4 creates the appointment (if no rules fail), as before.

When the **validation-only** attribute is set to **true**, N4 does not create an appointment.

9.1.26 create-appointment

A third party system can call this API to create a cargo appointment in N4 for Receive Break-Bulk and Deliver Break-Bulk transaction types.

Context Arguments: gate Id

Data Attributes: appointment-date, tran-type, line-id, cargo item quantity, cargo weight, cargo volume, unit-id, bill of lading nbr, cargo delivery order nbr. The 'tran-type' tag for pick up break-bulk (DB Gate Transaction) is 'PUB' and 'DOB' for drop off break-bulk (RB Gate Transaction).

Response: Includes appointment number, that was generated by N4, along with appointment details.

Variation: The **<appointment>** element of the create-appointment API has a Boolean attribute, **validation-only**, that specifies whether the appointment request is for validation purpose only or for creating and saving an

appointment. When a third party system uses the Boolean attribute to indicate that the appointment request is for validation purposes only, N4 does not save any appointment data; however, N4 does process the appointment stage workflow and if any errors are generated, N4 returns the same in the response.

For backward compatibility, the default value for the **validation-only** attribute is **false**. This means that if the **validation-only** attribute is missing OR the whole **<appointment>** element is missing then the appointment is always created (if no rules fail), as before.

Important points about the **validation-only** attribute:

Default setting of this attribute is **false**.

When you do not add the **validation-only** attribute, or, specify the **validation-only** attribute as **false**, or, do not specify the **<appointment>** element; N4 creates the appointment (if no rules fail), as before.

When the **validation-only** attribute is set to **true**, N4 does not create an appointment.

Request:

Sample Create Cargo Appointment Message.

```
<gate>
  <create-appointment>
    <appointment-date>2018-09-22</appointment-date>
    <gate-id>APPT_CFG</gate-id>
    <tran-type>PUB</tran-type>
    <line-id>LGK</line-id>
    <cargo item-quantity="10.0" cargo-weight="1000.0" cargo-volume="10000.0" unit-
id="10000011-1-1">
      <bill-of-lading nbr="IMPBL1" bl-item-seq="2"/>
      <cargo-delivery-order nbr="5" />
    </cargo>
  </create-appointment>
</gate>
```

9.1.27 update-appointment

A third party system can call this API to update an existing container appointment in N4.

Context Arguments: gate Id

Data Attributes: appointment-date, appointment-time, truck trucking-co-id, tran-type, container, pin-nbr, Driver and Truck information, Booking information, Container information etc.

Response: Appointment details.

9.1.28 cancel-appointment

A third party system can call this API to cancel one or more existing container appointments in N4.

Context Arguments: None

Data Attributes: appointment-nbr or external-ref-nbr.

Response: Returns the same list of appointment-nbr or external-ref-nbr for each appointment along with the appointment state (Cancel).

Process API's Provided by Other Systems

This section describes API's to be implemented on the systems with which N4 communicates. N4 requires that API's be implemented in other systems (the GOS primarily) when it needs to initiate an action in the system as a result of a situation that only N4 knows about. An example of this is the completion of manual intervention in the gate process on a N4 screen by a clerk. The requirement to implement these services depends upon the particular installation.

In order to reduce the interface work and make use of the testing facilities in N4, the same WSDL is used for these services. No responses are required other than a status code and any error messages that N4 should display.

In This Section

notify-resume.	265
open-gate-arm.	265
print-ticket.	265
truck-arrival.	265
truck-departure.	266

9.1.1 notify-resume

N4 can call this API to indicate that a manual step has been completed the GOS can continue. This can be used to pass control back to the GOS after a clerk has intervened. For example a truck arrives and goes into trouble status because of some error. The response to the GOS indicates trouble but the GOS cannot continue as the truck is waiting for a trouble clerk to intervene. Once the trouble clerk has fixed the issue or has done all that he intends with it then control can be passed back to the GOS. The call will either be as a result of the stage being submitted successfully or the clerk selecting via a button on the gate screen.

Context Attributes: gate Id, stage Id, lane Id

Data Attributes: truck visit identifier, truck transactions, appointments

Response: status code and optionally error messages

9.1.2 open-gate-arm

N4 will call this API to request that the GOS open the gate arm. If the clerk has to manually process a gate stage in N4, then N4 needs to call out to open the gate arm when it detects that the clerk has finished.

Context Attributes: gate Id, stage id, lane id

Data Attributes: none

Response: status code and optionally error messages

9.1.3 print-ticket

N4 will call this API to request that the GOS print the specified ticket. Normally, the GOS can produce the ticket directly from the response to the *process-truck* API call. However, in some environments a clerk always intercedes and manually processes the truck entirely using the N4 Gate User Interface. Also, in some more automated environments, the clerk will sometimes intercede and manually process a truck with serious problems. In both these cases, the intercession can occur several minutes after the *process-truck* API call. The trigger for printing the ticket is the successful completion of a transaction and this event is in this case caused by an action in N4.

Context Attributes: gate Id, stage id, printer id

Data Attributes: ticket type, document data XML

Response: status code and optionally error messages

9.1.4 truck-arrival

This API is used to notify tracking systems of a truck arrival into the scope of coverage of the tracking system.

Context Attributes: gate Id, stage id, lane id

Data Attributes: container ids, container types, accessory ids, chassis Id, truck license nbr, truck id, tracking tag identifier

Response: status code and optionally error messages

9.1.5 truck-departure

This API is used to notify tracking systems of a truck departure from the scope of coverage of the tracking system.

Context Attributes: gate id, stage id, lane id

Data Attributes: truck license nbr, truck id, tracking tag identifier

Response: status code and optionally error messages

Driver Interaction

In This Section

Pickup Selection.	268
Dropoff Details.	268

9.1.1 Pickup Selection

Even in a highly automated gate, some interaction with the driver may be required. Typically this is needed to specify pickups. There are several variations for this interaction with the driver. The actual container id could be provided or a pin number might be used as an alias. Or, the driver could provide a group, bill of lading number, or even an export booking number for dray-offs. These entries would indicate any container within that group/BL/Booking is acceptable.

For picking up empties, a specific container could be entered, but it is more common to provide a booking or EDO number. If the booking/EDO has multiple types of equipment, just the number is not sufficient and N4 must respond with a list of order items for the driver to select from.

Since the interaction for empty pickups is somewhat different involving several different combinations of fields it may be necessary to initiate the entire interaction by first asking the driver the basic transaction type (pickup up an empty, bare chassis or full container). Where the term "full container" is sort of a misnomer here because the container could be a revenue empty container (empty import, empty export or transship drayoff). Once this determination is made the driver enters

- Pickup Full - appointment number or (container and/or pin) or (group or BL or booking)
- Pickup Empty - appointment number or container or order number

For the pickup Empty, if an order number is entered, when it is submitted N4 will return a list of order items from which the driver must select.

9.1.2 Dropoff Details

Normally in an automated gate just the OCR read alone is sufficient to process the gate transaction provided the container is pre-advised or an appointment has been lodged. However in the absence of this information, N4 has no idea what to do with the container.

To start with, the driver must indicate if the container is a storage empty or an export. For an empty storage the equipment type (if not scanned) and the controlling line operator must be provided. For an export container, the equipment type (if not scanned) and minimally the booking number must be provided by the driver. Beyond this information, for an export container the seal numbers and gross weight will probably also be needed.

- Dropoff Full - equipment type and booking number and possibly seals and gross weight
- Dropoff Empty - equipment type and line operator

Process Truck API Workflow

As the API at the in-gate invokes a complex process and varying process, a workflow must be specified that is constructed from a set of process steps appropriate for the individual customer's environment. Despite the variation between sites the steps, which may or may not be present are always in the same order. There is thus an underlying master workflow that can be configured by inclusion or exclusion of tasks.

In This Section

Basic Scan Validation.	270
Process Pre-lodged Dropoffs.	270
reprint-most-recent-ticket.	270
Fail Unexpected Container.	271
Solicit Further Details.	271
Process Pre-lodged Pickups.	271
Solicit Appointment Selection.	271
Solicit Pickup Selection.	271
Trouble Processing.	271

9.1.1 Basic Scan Validation

Before any attempt is made to do the complex business processing, basic validation can be configured to ensure and hopefully correct scan failures. This consists the following;

1. The RFID or OCR reader detected the presence of a value (e.g. a container number), but was unable to read the value. This is indicated by sending a value of "ERROR".
2. A value was returned but it seems suspicious. The validation here is verify specific and each validation can be configured to conform to the site's requirements
 - Truck ID or Truck License
 - Truck ID is null
 - Truck is unknown (not in the N4 truck database)
 - Truck has no appointments
 - Container ID
 - Container check digit validation fails
 - Container is not pre-advised or has no appointment
 - Container Type
 - Container type is unknown (not in the N4 container ISO code database)

When scan validation fails, no attempt is made to perform any further processing. The Gate Lane list view in N4 will indicate a problem has occurred in the lane. Gate clerks should monitor this list view. To intervene, the clerk can double click on the lane. A form will appear where the clerk can correct the data. Only then will the gate processing continue, but now using the corrected values.

9.1.2 Process Pre-lodged Dropoffs

This step processes possible pre-advised containers or appointments for each OCR container number scanned. This process is always performed even at facilities where pre-lodged information is seldom provided to encourage the use of appointments and/or pre-advised containers.

9.1.3 reprint-most-recent-ticket

This API allows you to reprint the most recent document from the gate transactions. This API accepts printer id, tran_gkey or tv_gkey (or gos-tv-key) in the Request API.

Request:

Sample Request API

```
<gos>
<reprint-most-recent-ticket>
<printer id="PTR1"/>
<truck-visit tv-key="9951" gos-tv-key="978022" no-of-copies="1" />
<truck-transaction tran-key="10358" no-of-copies="1"/>
</reprint-most-recent-ticket>
</gos>
```

- printer id="PTR1" is a mandatory field. If you enter an invalid printer id, N4 displays the error 'Unknown printer XXX'.
- no-of-copies is an optional field. Default value is 1.

- If you mention both the tv-key and tran-key in the Request API, tran-key takes priority and N4 prints the most recent document for the transaction.

9.1.4 Fail Unexpected Container

This optional step can be configured if it is expected that all container drop-offs must be pre-advised.

9.1.5 Solicit Further Details

With this optional step, for each container that was scanned that cannot be processed, the lack of key details is analyzed. If details are missing a dialog is initiated with the driver requesting the missing details. To achieve this, the container number is returned along with the unique truck visit key and a list of missing attributes. The GOS is then expected to solicit the missing information and call back the submit transaction API with the container number, truck visit key, the transaction type and other attributes.

The required fields are configured within this step. A specific transaction type can be configured if the gate only supports empty storage or only export drop-offs thus eliminating from the driver solicitation the transaction type and fields only appropriate for the other transaction type.

9.1.6 Process Pre-lodged Pickups

This step processes possible pickup appointments based on OCR truck identifier scanned. This process is always performed even at facilities where pre-lodged information is seldom provided to encourage the use of appointments.

If a truck visit appointment exists (not currently supported by N4) then only the trucks appointments for this truck visit are processed.

Otherwise, all unfulfilled pickup appointments for the truck for that day are analyzed. Based on a list of valid combinations of appointments that can be automatically processed a determination is made if we can assume the driver will do his remaining appointments on this trip. For example the list might indicate one 40' or two 20' containers. If this list is empty, no assumption is made. If all the appointments can be handled on this trip they are automatically processed.

9.1.7 Solicit Appointment Selection

This optional step either returns an appointment selection to the driver or assumes the driver is done if there are no further pickup appointments for the truck. In a manner similar to the other solicitation responses, the GOS is expected to obtain the selection from the driver and call the *submit-transaction* API with the appointment selection. The driver can also indicate he is done and the GOS should call the *submit-done* API.

9.1.8 Solicit Pickup Selection

With this optional step, the GOS solicits this information from the driver and calls the *submit-transaction* API. When the *submit-transaction* API is called, either the transaction will succeed or N4 will detect fields are still missing from the input. When the driver has finished selecting pickups, he indicates so to the GOS and it calls the *submit-done* API.

9.1.9 Trouble Processing

When a transaction is submitted the absence of key fields will be noted in the error messages returned. Once all the required information is provided the transaction might still fail for business reasons beyond the control of the driver. Depending on the combination of transactions processed a failure will usually terminate the interaction with the driver and mark the truck visit in trouble status.

If a business problem occurs for a dropoff, the processing for the remaining container OCR reads continues. Once they have all been processed, if a problem occurred on any one drop-off, the truck visit is marked in error and trouble processing is initiated. No attempt is made to solicit the driver's pickups.

If a problem occurs for a pickup transaction the interaction with the driver terminates immediately and the trouble processing is initiated.

Trouble processing can be configured in two ways. Either the driver is given a trouble ticket for each failing transaction and the driver is directed to the trouble desk, or the driver remains in the lane for clerk assistance. The clerk can choose to direct the driver to the trouble desk.

Lane Monitor View

The Lane Monitor view displays the status of the lanes defined for all the gates and exchange areas at your site. To view the scanned data for a truck in a gate lane use the Scan Sets view in N4.

You can use this view to:

- Monitor the progress of the trucks in the lanes
- Detect any trouble in a lane
- Correct scanned data, such as electronic scale, RFID, and OCR readings
- Manually process a truck
- Re-initiate automatic gate processing

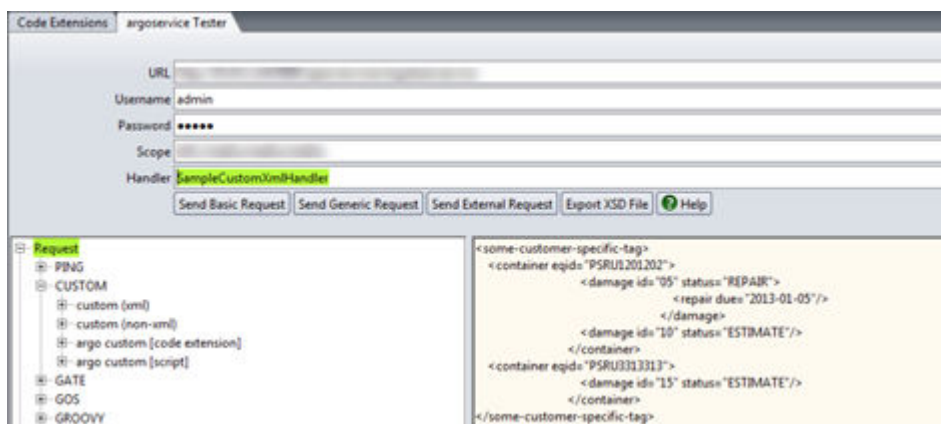
The Truck Status column indicates the status of the truck in the gate lane as follows:

- **Allocated:** The truck is assigned to a yard exchange lane.
- **Empty:** The gate lane is empty.
- **Processing:** A clerk is processing the gate transaction(s) for the truck in the gate lane.
- **Scan Failure:** An error is returned for a scanned item for a truck in the gate lane.
- **Trouble:** The truck in the gate lane has been sent to the trouble desk.
- **Waiting:** A truck is waiting in the gate lane.
- **Wrong Stage:** The truck in the gate lane is at the wrong gate stage.

Support for Custom API messages

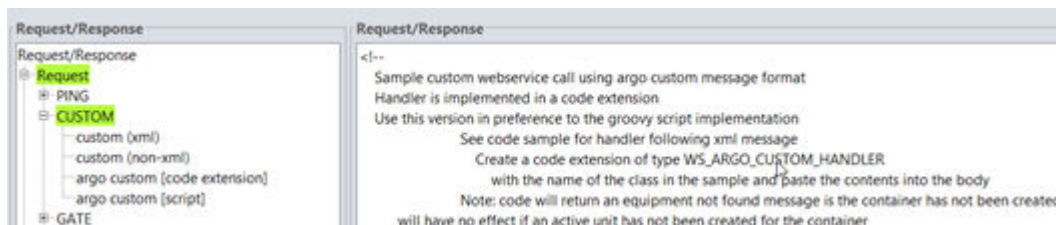
The gate webservice API supports API messages within the <gate> tags. However, if you want to define your own custom xml messages, you can do so by wrapping them within the This fix is for a specific configuration. If you did not request this fix, but want the functionality it enables, customization may be required. Please contact your Professional Services or Support representative for more information. tag.

In addition to this, N4 can also handle custom messages without the This fix is for a specific configuration. If you did not request this fix, but want the functionality it enables, customization may be required. Please contact your Professional Services or Support representative for more information. tag. For example, you can post any valid xml in any file format in order to override the existing groovy scripts. To do this, you must specify the name of the code extension that will process the message in the Handler field in the argoservice Tester view. Using this option, you can send a fully customized message without any N4 content requirements. In other words, you can send the output from a third party system to N4 without any amendments.



For example,

1. Create a Code Extension with Extension Type **WS_CUSTOM_HANDLER**.
2. In the Argoservice tester view enter the Username and Password.
3. Enter the Handler field (the Extension Type of the Code Extension).
4. Post the xml.



Sample messages and implementations are available in the Argoservice Tester view under Request > CUSTOM.

Including schema extension fields

N4 gives you the option to use **Dynamic Flex Fields** (DFF) in the create-appointment, update-appointment, create-truck-visit-appointment, and update-truck-visit-appointment API messages.

Given below is an example to create an appointment for a container and a truck visit in N4 using dynamic flex fields in the API. For information on dynamic flex fields, refer to the online help topic [Schema Extension Modification Registry](#) view.

Example: Create appointment

```
<gate>
<create-appointment>
<appointment-date>2017-09-15</appointment-date>
<gate-id>CTT-GATE-A</gate-id>
<tran-type>RM</tran-type>
<appointment>
<custom-flex-fields>
<custom-flex-field id="gapptCustomDFFTCMsgVersionNbr" value="6"/>
<custom-flex-field id="gapptCustomDFFTCQuayAccountNumber" value="9785455"/>
</custom-flex-fields>
</appointment>
<container eqid="PSRU1211212" type="2200" />
<external-ref-nbr>APPT123</external-ref-nbr>
</create-appointment>
</gate>
```

Part 10

Ship Bin Model

Ship Bin Model Request.	277
Import-Export Ship Bin Model.	286

10.1 Ship Bin Model Request

In This Section

Introduction.	278
Setup.	278
Example Request.	278

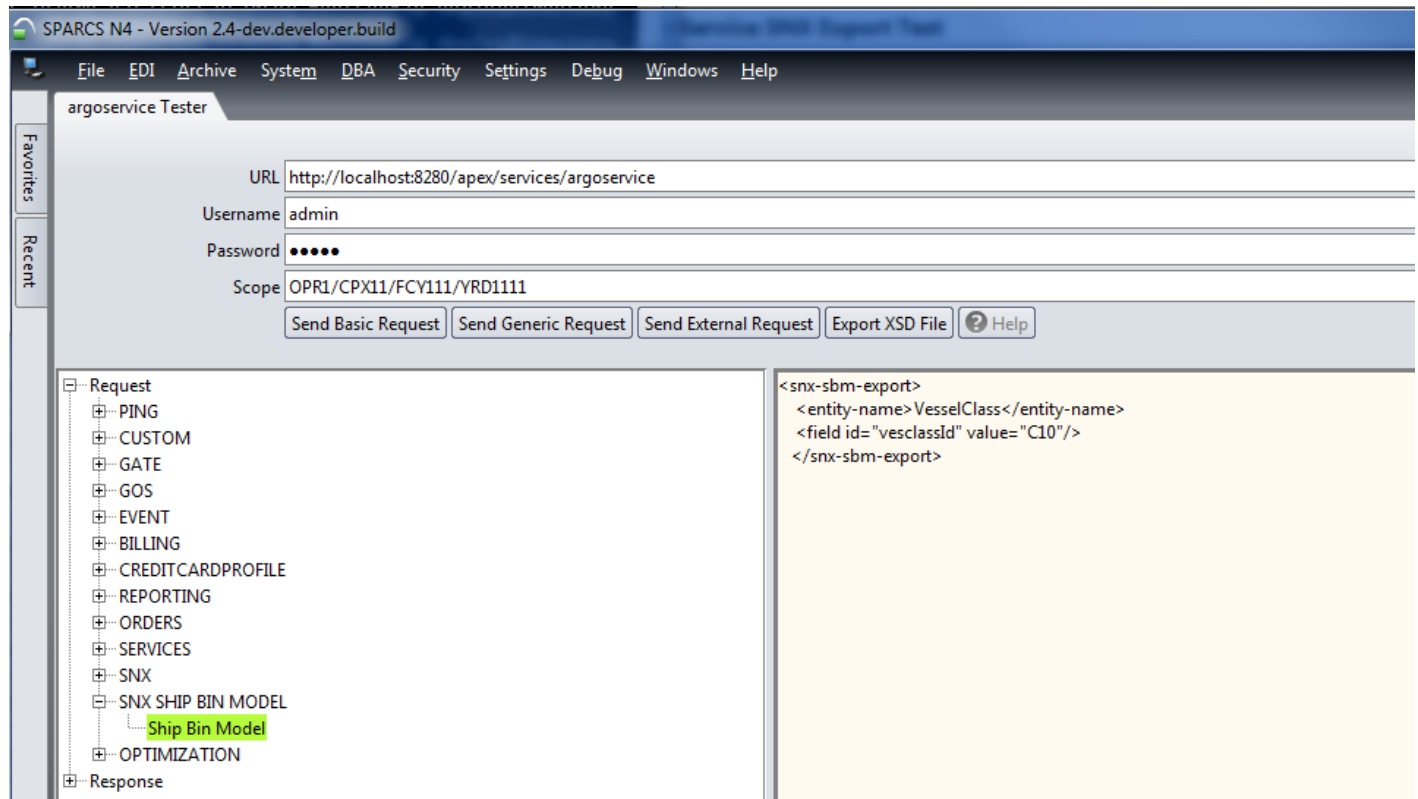
10.1.1 Introduction

This document describes how to request an SNX document that describes the ship bin model.

10.1.2 Setup

If ship bin model is not already loaded to Vessel Class object, go to vessel class screen and import it.

10.1.3 Example Request



snx-type.xsd for ship bin model.

```
<xs:complexType name="tSection">
  <xs:annotation>
    <xs:documentation>Section</xs:documentation>
  </xs:annotation>
  <xs:complexContent>
    <xs:extension base="argo:tBin">
      <xs:attribute name="conv-id-col" type="xs:long" use="optional"/>
      <xs:attribute name="row-index" type="xs:long" use="required"/>
      <xs:attribute name="row-paired-into" type="xs:long" use="optional"/>
      <xs:attribute name="row-paired-from" type="xs:long" use="optional"/>
      <xs:attribute name="col-table-offset" type="xs:long" use="optional"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:complexType name="tBlock">
```

```

<xs:annotation>
  <xs:documentation>Block</xs:documentation>
</xs:annotation>
<xs:complexContent>
  <xs:extension base="argo:tBin">
    <xs:attribute name="conv-id-row-std" type="xs:long" use="optional"/>
    <xs:attribute name="conv-id-row-alt" type="xs:long" use="optional"/>
    <xs:attribute name="conv-id-tier" type="xs:long" use="optional"/>
  </xs:extension>
</xs:complexContent>
</xs:complexType>
<xs:complexType name="tBin">
  <xs:annotation>
    <xs:documentation>Bin Model</xs:documentation>
  </xs:annotation>
  <xs:sequence>
    <xs:element name="bin-name-table" type="argo:tBinNameTable" form="unqualified"
minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="name" type="xs:string" use="required"/>
  <xs:attribute name="name-alt" type="xs:string" use="optional"/>
  <xs:attribute name="sub-type" use="optional">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="YDM">
          <xs:annotation>
            <xs:documentation>Yard Model</xs:documentation>
          </xs:annotation>
        </xs:enumeration>
        <xs:enumeration value="NSB">
          <xs:annotation>
            <xs:documentation>Non Stack Block</xs:documentation>
          </xs:annotation>
        </xs:enumeration>
        <xs:enumeration value="SBL">
          <xs:annotation>
            <xs:documentation>Stack Block</xs:documentation>
          </xs:annotation>
        </xs:enumeration>
        <xs:enumeration value="YSN">
          <xs:annotation>
            <xs:documentation>Yard Section</xs:documentation>
          </xs:annotation>
        </xs:enumeration>
        <xs:enumeration value="YST">
          <xs:annotation>
            <xs:documentation>Yard Stack</xs:documentation>
          </xs:annotation>
        </xs:enumeration>
        <xs:enumeration value="BTH">
          <xs:annotation>
            <xs:documentation>Berth Model</xs:documentation>
          </xs:annotation>
        </xs:enumeration>
        <xs:enumeration value="TPP">
          <xs:annotation>

```

```

        <xs:documentation>Track Plan</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="TPT">
      <xs:annotation>
        <xs:documentation>Rail Track</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="TPS">
      <xs:annotation>
        <xs:documentation>Track Spot</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="SHM">
      <xs:annotation>
        <xs:documentation>Ship Bin Model</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
  </xs:restriction>
</xs:simpleType>
</xs:attribute>
<xs:attribute name="type" use="optional">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="ABM_MODEL">
        <xs:annotation>
          <xs:documentation>Yard Model</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="ABM_BLOCK">
        <xs:annotation>
          <xs:documentation>Yard Block</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="ABM_SECTION">
        <xs:annotation>
          <xs:documentation>Yard Section</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="ABM_STACK">
        <xs:annotation>
          <xs:documentation>Yard Stack</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="TRACK_PLAN">
        <xs:annotation>
          <xs:documentation>Track Plan</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="TRACK_TRACK">
        <xs:annotation>
          <xs:documentation>Rail Track</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="TRACK_SPOT">
        <xs:annotation>

```

```

        <xs:documentation>Track Spot</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="BERTH_BERTH">
      <xs:annotation>
        <xs:documentation>Berth</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
  </xs:restriction>
</xs:simpleType>
</xs:attribute>
<xs:attribute name="level" type="xs:long" use="optional"/>
<xs:attribute name="long-name" type="xs:string" use="optional"/>
<xs:attribute name="parent-name" type="xs:string" use="required"/>
<xs:attribute name="geo-anchor-latitude" type="xs:double" use="optional"/>
<xs:attribute name="geo-anchor-longitude" type="xs:double" use="optional"/>
<xs:attribute name="geo-anchor-orientation" type="xs:double" use="optional"/>
<xs:attribute name="center-line" type="xs:string" use="optional"/>
<xs:attribute name="polygon" type="xs:string" use="optional"/>
<xs:attribute name="vertical-min" type="xs:double" use="optional"/>
<xs:attribute name="vertical-max" type="xs:double" use="optional"/>
<xs:attribute name="z-index-min" type="xs:long" use="optional"/>
<xs:attribute name="z-index-max" type="xs:long" use="optional"/>
<xs:attribute name="life-cycle-state" type="argo:tLifeCycleStateType"
use="optional"/>
</xs:complexType>
<xs:complexType name="tStack">
  <xs:annotation>
    <xs:documentation>Stack</xs:documentation>
  </xs:annotation>
  <xs:complexContent>
    <xs:extension base="argo:tBin">
      <xs:attribute name="col-index" type="xs:long" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:complexType name="tShipModel">
  <xs:annotation>
    <xs:documentation>Ship Model</xs:documentation>
  </xs:annotation>
  <xs:complexContent>
    <xs:extension base="argo:tBin">
      <xs:attribute name="lcg-ref" type="argo:tLcgRef" use="optional">
        <xs:annotation>
          <xs:documentation>Referance of longitudinal center of gravity(lcg).</
xs:documentation>
        </xs:annotation>
      </xs:attribute>
      <xs:attribute name="lcg-mode" type="argo:tLcgMode" use="optional">
        <xs:annotation>
          <xs:documentation>Mode of longitudinal center of gravity(lcg).</
xs:documentation>
        </xs:annotation>
      </xs:attribute>
      <xs:attribute name="tcg-mode" type="argo:tTcgMode" use="optional">
        <xs:annotation>

```

```

        <xs:documentation>Mode of transverse center of gravity(tcg).</
xs:documentation>
    </xs:annotation>
</xs:attribute>
<xs:attribute name="model-name" type="xs:string" use="optional">
    <xs:annotation>
        <xs:documentation>Name of ship bin model.</xs:documentation>
    </xs:annotation>
</xs:attribute>
</xs:extension>
</xs:complexContent>
</xs:complexType>
<xs:complexType name="tCellSection">
    <xs:annotation>
        <xs:documentation>Cell Section</xs:documentation>
    </xs:annotation>
    <xs:complexContent>
        <xs:extension base="argo:tSection">
            <xs:attribute name="basic-lengths-allowed"
type="argo:tSectionLengthsAllowedList" use="optional">
                <xs:annotation>
                    <xs:documentation>Only containers of lengths specified will be allowed be
stowed in this section.</xs:documentation>
                </xs:annotation>
            </xs:attribute>
            <xs:attribute name="csn-conv-id-tier" type="xs:long" use="optional">
                <xs:annotation>
                    <xs:documentation>Specifies the naming table that defines the tier labels
for cell sections.</xs:documentation>
                </xs:annotation>
            </xs:attribute>
            <xs:attribute name="lcg-20" type="xs:double" use="optional">
                <xs:annotation>
                    <xs:documentation>LCG 20 standard (20' containers).</xs:documentation>
                </xs:annotation>
            </xs:attribute>
            <xs:attribute name="lcg-40" type="xs:double" use="optional">
                <xs:annotation>
                    <xs:documentation>LCG 40 alternate (40' containers).</xs:documentation>
                </xs:annotation>
            </xs:attribute>
            <xs:attribute name="lcg-45" type="xs:double" use="optional">
                <xs:annotation>
                    <xs:documentation>LCG 45 alternate (45' containers).</xs:documentation>
                </xs:annotation>
            </xs:attribute>
        </xs:extension>
    </xs:complexContent>
</xs:complexType>
<xs:complexType name="tCellStack">
    <xs:annotation>
        <xs:documentation>Cell Stack</xs:documentation>
    </xs:annotation>
    <xs:complexContent>
        <xs:extension base="argo:tStack">
            <xs:attribute name="lcg-20" type="xs:double" use="optional">

```

```

    <xs:annotation>
      <xs:documentation>LCG 20 standard (20' containers).</xs:documentation>
    </xs:annotation>
  </xs:attribute>
  <xs:attribute name="lcg-40" type="xs:double" use="optional">
    <xs:annotation>
      <xs:documentation>LCG 40 alternate (40' containers).</xs:documentation>
    </xs:annotation>
  </xs:attribute>
  <xs:attribute name="lcg-45" type="xs:double" use="optional">
    <xs:annotation>
      <xs:documentation>LCG 45 alternate (45' containers).</xs:documentation>
    </xs:annotation>
  </xs:attribute>
  <xs:attribute name="tcg" type="xs:double" use="optional">
    <xs:annotation>
      <xs:documentation>Transverse center of gravity.</xs:documentation>
    </xs:annotation>
  </xs:attribute>
</xs:extension>
</xs:complexContent>
</xs:complexType>
<xs:complexType name="tShipBlock">
  <xs:annotation>
    <xs:documentation>Ship Block</xs:documentation>
  </xs:annotation>
  <xs:complexContent>
    <xs:extension base="argo:tBlock"/>
  </xs:complexContent>
</xs:complexType>
<xs:simpleType name="tLcgRef">
  <xs:restriction base="xs:string">
    <xs:enumeration value="MIDSHIPS">
      <xs:annotation>
        <xs:documentation>Midships</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="AFT_PERPENDICULAR">
      <xs:annotation>
        <xs:documentation>Aft Perpendicular</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="FWD_PERPENDICULAR">
      <xs:annotation>
        <xs:documentation>Fwd Perpendicular</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
  </xs:restriction>
</xs:simpleType>
<xs:simpleType name="tLcgMode">
  <xs:restriction base="xs:string">
    <xs:enumeration value="KNOWN">
      <xs:annotation>
        <xs:documentation>Known</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
  </xs:restriction>
</xs:simpleType>

```

```

    <xs:enumeration value="UNKNOWN">
      <xs:annotation>
        <xs:documentation>Unknown</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
  </xs:restriction>
</xs:simpleType>
<xs:simpleType name="tTcgMode">
  <xs:restriction base="xs:string">
    <xs:enumeration value="KNOWN">
      <xs:annotation>
        <xs:documentation>Known</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
    <xs:enumeration value="UNKNOWN">
      <xs:annotation>
        <xs:documentation>Unknown</xs:documentation>
      </xs:annotation>
    </xs:enumeration>
  </xs:restriction>
</xs:simpleType>

```

Notes

- lcg-ref can be MIDSHIPS, AFT_PERPENDICULAR, or FWD_PERPENDICULAR. LCG values are always positive in the forward direction. Attached [ship](#) and [SNX](#) files show examples of different reference and orientation cases for LCGs if anyone is confused.
- TCG values are always positive in the starboard (to the right when facing forward on the ship) direction. The origin in the transverse (TCG) direction is always on the ship's centerline (middle of ship in transverse direction).
- lcg-mode and tcg-mode can be KNOWN or ESTIMATED. An ESTIMATED value indicates that the cgs are not to be considered accurate.
- Length dimensions are in centimeters.
- z-index-min and z-index-max are min and max tiers, respectively.
- The bottom-most slot on the ship is tier 1. Some below-deck ship stacks do not go all the way to the bottom of the vessel's hull due to the ship's flare (or hull's curve). I.e., often center stacks have a min index of 1, but outboard ones might have a min index of 5, for example.
- col-index and row-index are logical index values. E.g., logical row 17 might have a user name of 33 for 20' container stowage.
- col-index of 0 is only needed if a bay has a center stack.
- Starboard side column index values are always odd and port side column index values are always even.
- conv-id-row-alt, conv-id-row-std and conv-id-tier point to naming tables at the top of the message. std names are for 20' rows and alt names are for 40' rows. These tables take the logical index of the row, col, or tier to map to string names for those elements in the model.
- row-paired-from and row-paired-into are about 40' containers extending into other rows. For example, 40' containers stowed in row 17 may have row-paired-into set to 18 if they extend into the row aft of 17 (row 18).

Clarifying, row names are assigned based on the logical index of the cell-section AND the length of the container (20 or 40/45). When displaying the ship stowage views, 20' names (conv-id-row-std) are used. The 40' row name is shown alongside the 20' name if 40s are allowed in that row based on info in the basic-lengths-allowed field. The XPS Client UI has good examples of this.

E.g.,

```
<bin-name-table table-index="1" table-name="C1020">
...
  <bin-name logical-position="17" user-name="33" />
  <bin-name logical-position="18" user-name="35" />
...
</bin-name-table table-index="2" table-name="C1040">
...
  <bin-name logical-position="17" user-name="34" />
</bin-name-table>
...
<ship-block name="A" parent-name="C10" conv-id-row-alt="2" conv-id-row-std="1" />
...
<cell-section
  name="A033" parent-name="A" conv-id-tier="6" row-index="17"
  row-paired-from="0" row-paired-into="18" conv-id-col="4" lcg-20="-257.0"
  lcg-40="-640.0" lcg-45="-640.0" basic-lengths-allowed="[20, 40]" />
```

The above example shows that logical row 17 in cell-section A033 would have a user row name of "33" for 20' containers and a user row name of "34" for 40' containers. Note that the name field on each bin is de-normalized value composed of all relevant position name elements (i.e., block, 20' row, column) .

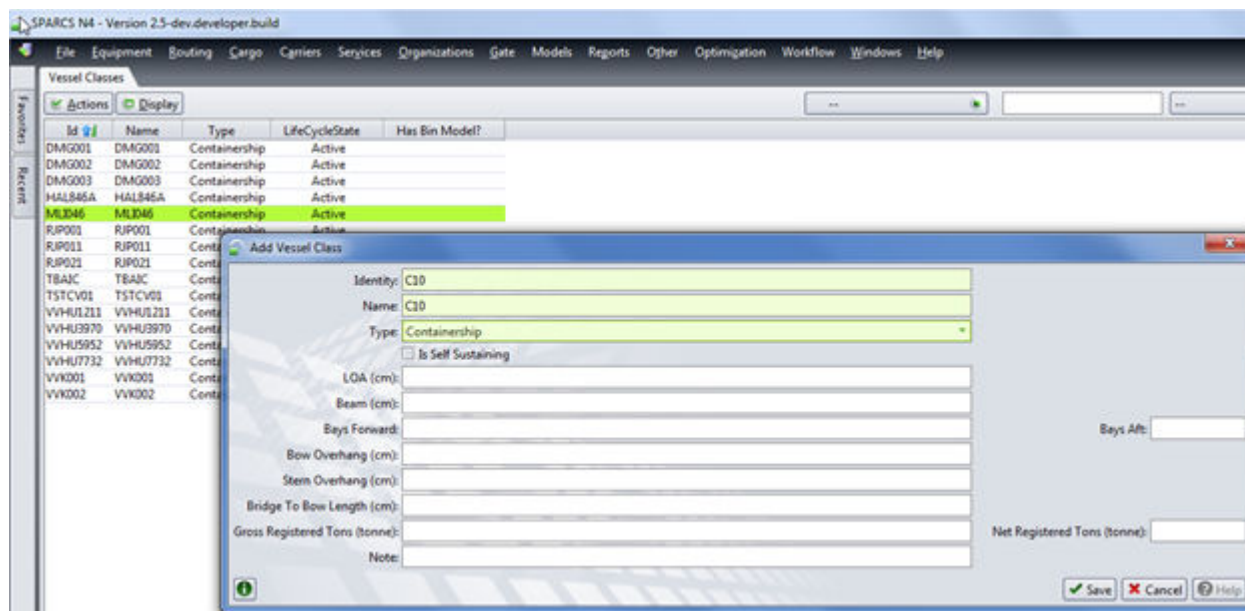
10.2 Import-Export Ship Bin Model

In This Section

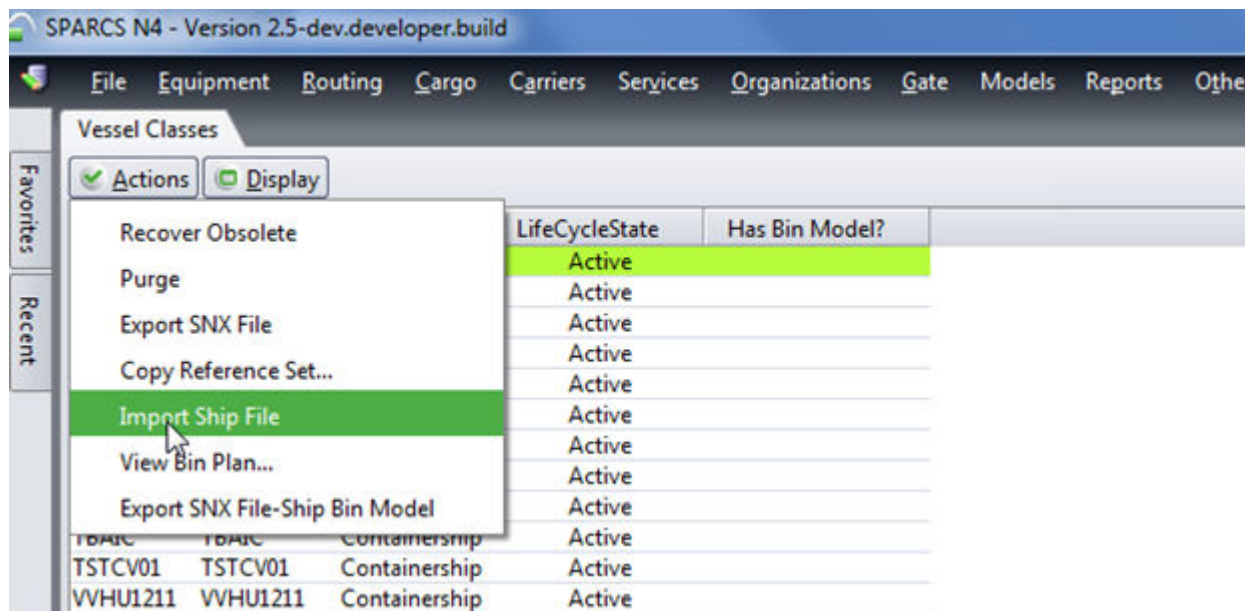
- [Import Ship File.](#) 287
- [View Bin Plan.](#) 289
- [Export SNX File – Ship Bin Model.](#) 289

10.2.1 Import Ship File

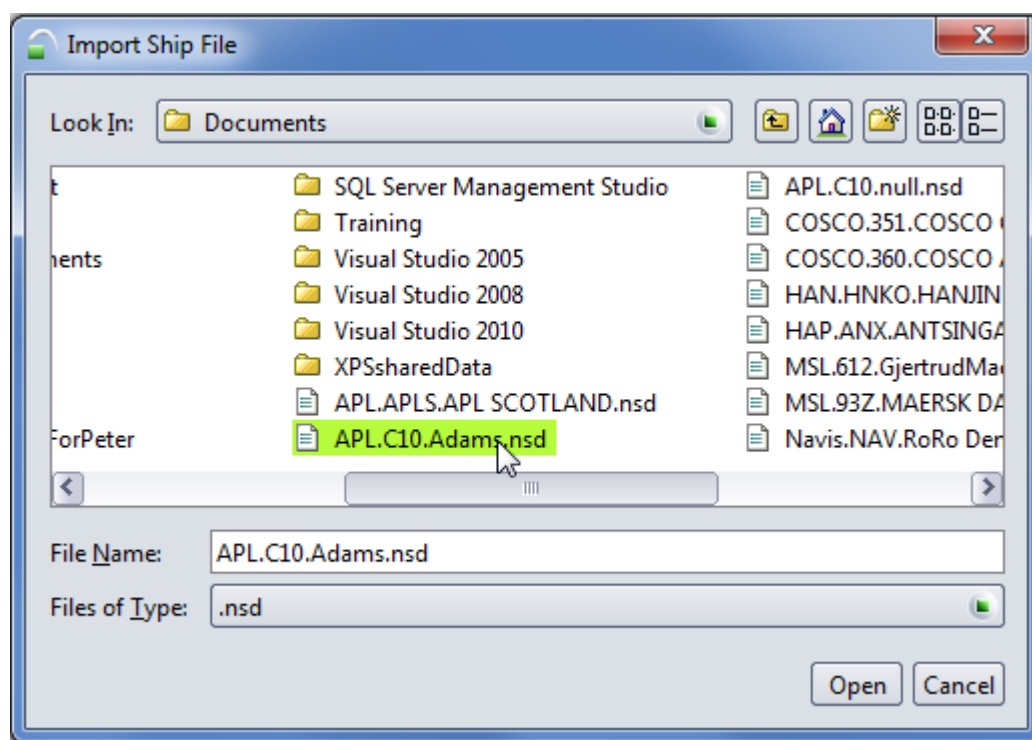
For the user interface, use the **Configuration** model and choose the **Vessel Classes** under the **Carriers** menu item. Highlight a vessel class, or add one if there is not any for the ship bin model file.



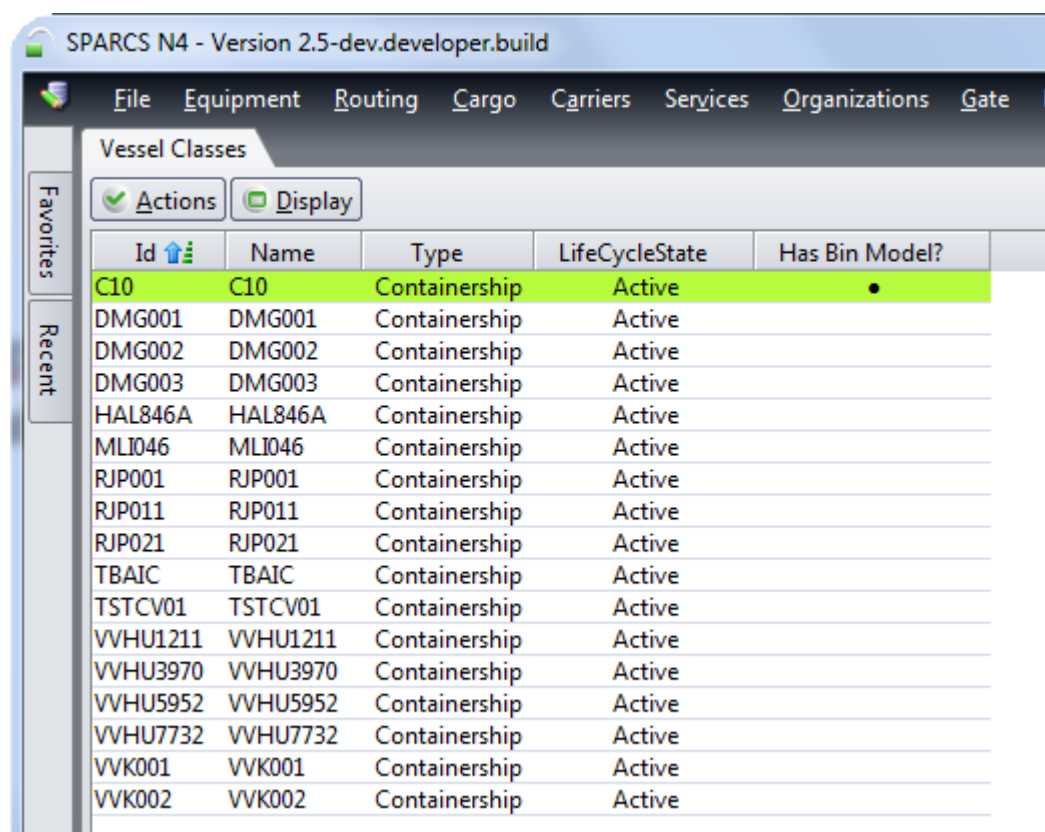
After highlighting vessel class, select **Import Ship File** under **Actions** menu item.



A dialog box named **Import Ship File** will open. Choose a valid binary ship file with the extension **.nsd** that you want to import. If the file's version is under 114 or over 999 -127 is current ship file version as of 11/17/2011-, the system could not continue to import the XPS's model into N4. The binary ship files are created by the ShipEditor application and used by Sparcs XPS. We need ship bins in the N4, therefore we need to process this ship file into N4 application.



After finishing import the ship bin file into N4, the field "Has Bin Model" has a dot now and it means the vessel class has ship bin model data imported.



A vessel class should be created in the N4 database before importing a file (vessel class name and ship file class names should be same. E.g. If the ship file name is APL.C10.Adams.nsd, the class name should be C10).

10.2.2 View Bin Plan

Bin model list can be seen in the form under Vessel Classes' Actions menu item. Highlight one vessel class to see its bin model, then select **View Bin Model** under **Actions**.

Identity: C10

Actions Display

Name	Tcg	Lcg	Deck	Row	Column
A00101	123.000	10,840.000	A	001	01
A00102	-123.000	10,840.000	A	001	02
A00103	369.000	10,840.000	A	001	03
A00104	-369.000	10,840.000	A	001	04
A00105	614.000	10,840.000	A	001	05
A00106	-614.000	10,840.000	A	001	06
A00107	860.000	10,840.000	A	001	07
A00108	-860.000	10,840.000	A	001	08
A00109	1,106.000	10,840.000	A	001	09
A00110	-1,106.000	10,840.000	A	001	10
A00111	1,352.000	10,840.000	A	001	11
A00112	-1,352.000	10,840.000	A	001	12
A00501	123.000	9,600.000	A	005	01
A00502	-123.000	9,600.000	A	005	02
A00503	369.000	9,600.000	A	005	03
A00504	-369.000	9,600.000	A	005	04
A00505	614.000	9,600.000	A	005	05
A00506	-614.000	9,600.000	A	005	06
A00507	860.000	9,600.000	A	005	07
A00508	-860.000	9,600.000	A	005	08
A00509	1,106.000	9,600.000	A	005	09
A00510	-1,106.000	9,600.000	A	005	10
A00511	1,351.000	9,600.000	A	005	11
A00512	-1,351.000	9,600.000	A	005	12
A00513	1,597.000	9,600.000	A	005	13
A00514	-1,597.000	9,600.000	A	005	14
A00901	123.000	8,467.000	A	009	01
A00902	-123.000	8,467.000	A	009	02
A00903	369.000	8,467.000	A	009	03
A00904	-369.000	8,467.000	A	009	04
A00905	614.000	8,467.000	A	009	05
A00906	-614.000	8,467.000	A	009	06
A00907	860.000	8,467.000	A	009	07
A00908	-860.000	8,467.000	A	009	08
A00909	1,106.000	8,467.000	A	009	09

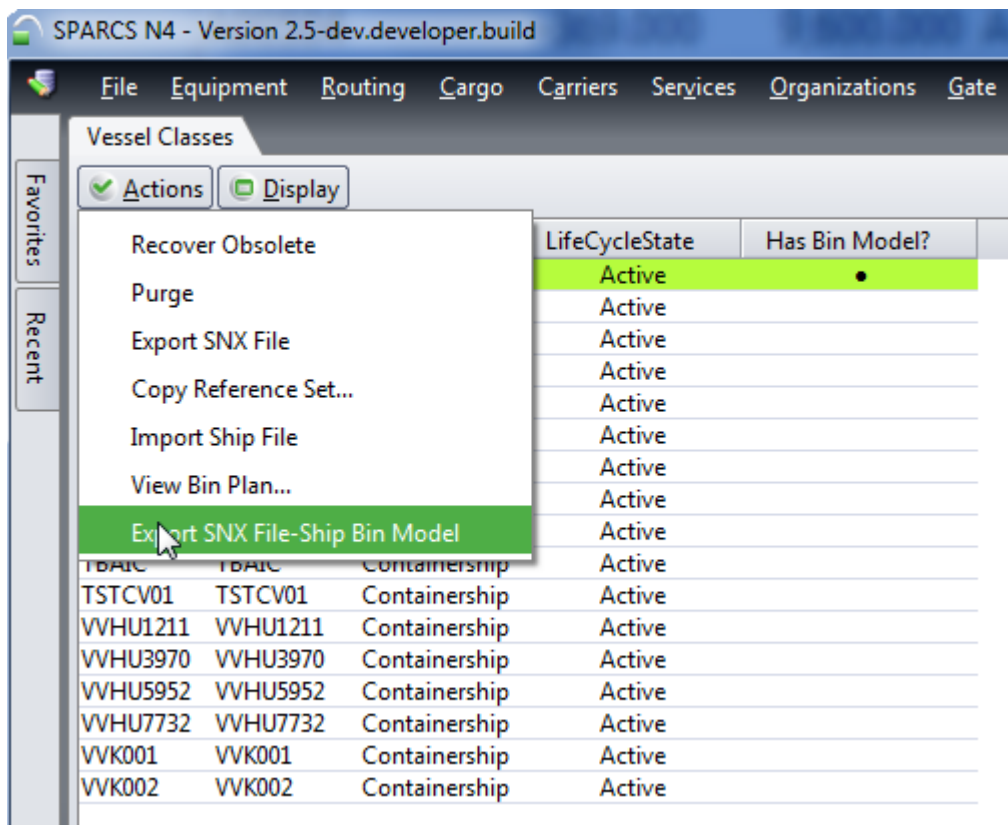
Vessel Bin:

0:501

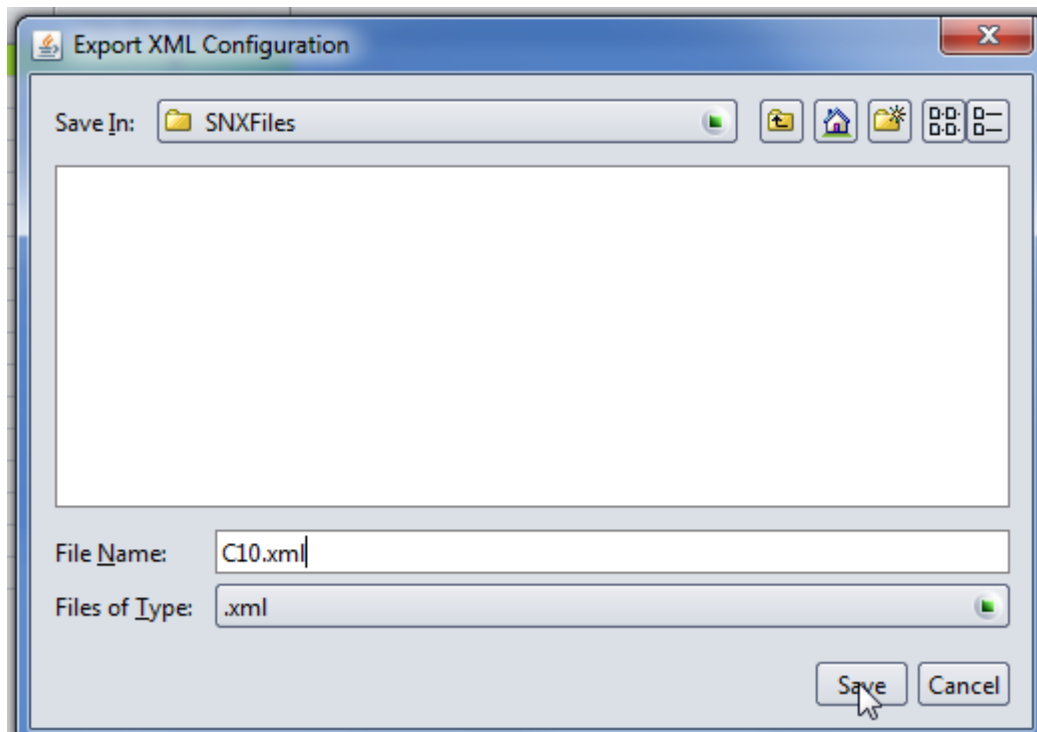
Save Close Help

10.2.3 Export SNX File – Ship Bin Model

Highlight a vessel class it should have ship bin model imported. Select **Export SNX File – Ship Bin Model** under **Actions** menu item.



A dialog box named **Export SNX Configuration** will open. Choose a location to save ship bin model snx file.



Part 11

ASC Automated Equipment Database Interface (ECN4)

Introduction.	292
System Interactions.	294
Non-Functional Specification.	334

11.1 Introduction

In This Section

Feature Summary.	293
Assumptions.	293

11.1.1 Feature Summary

This document describes the interface between ECN4 and the Automated Crane Control System. The purpose of this Automated Equipment Interface is to provide a means for ECN4 to communicate container movement instructions to an automated stacking cranes (ASC) controlled by a third party Automated Crane Control System (ACCS) and for the ACCS to communicate container movement in response to an instruction or as a result of manual override.

Transport orders and ASC status are the two primary types of information communicated through the interface. The current inventory in the transfer points and stack or transfer zone slot availability are secondary and optional. The exchange of this information is accomplished through database table updates. This mechanism has a number of attractive qualities that make it suitable for communicating service requests as well as status updates:

- Message exchange is persistent and has historical depth.
- Field based structure eliminates parsing. In addition referential integrity and field usage rules improve quality of message structure.
- One writer rule is easily implemented. Database security can insure integrity of stream.
- Participants can easily develop and validate adherence to the standard. Responsibility and accountability is compartmental.
- Key generation for uniqueness and sequencing is intrinsic to the package
- Trigger mechanisms provide reasonable process integration - polled activity is possible if responsiveness allows it.
- Very easy for third party software to monitor the link and perform forensic analysis of issues.
- Potential use of standard replication mechanisms to support redundancy.

In this approach, two tables are used to store the associated service requests and responses. The service request table is named `ASC_TRANSPORT_ORDERS`. The response table is named `ASC_COMMAND_QUEUE`.

Each application is responsible for managing its respective tables. When ECN4 wants to communicate a transport order to the ACCS, it writes a new row in its database table with the transport order information. Each transport order in the database table is uniquely identified by a database generated sequential key value. This allows for implicit ordering and is used to provide integrity across both applications, i.e. for each unique transport order a matching transport order is present in the ACCS command queue table. When the ASC is ready to execute its next movement, the ACCS application reads the next pending transport order from the ECN4 managed transport order table and places the order in its own command queue table with a corresponding database generated receipt key. ECN4 uses the receipt key to determine that the transport order has been accepted and is now in progress.

11.1.2 Assumptions

- The database being described here is not part of the Navis N4 system. It is assumed that this database is installed, configured, and maintained by the customer.
- If bombcarts (terminal owned chassis) or cassettes will be de-coupled and parked in ASC transfer points then both prime route for terminal tractors and specific chassis tracking must be licensed. The bombcarts or cassettes are uniquely identified and tracked terminal managed equipment.

11.2 System Interactions

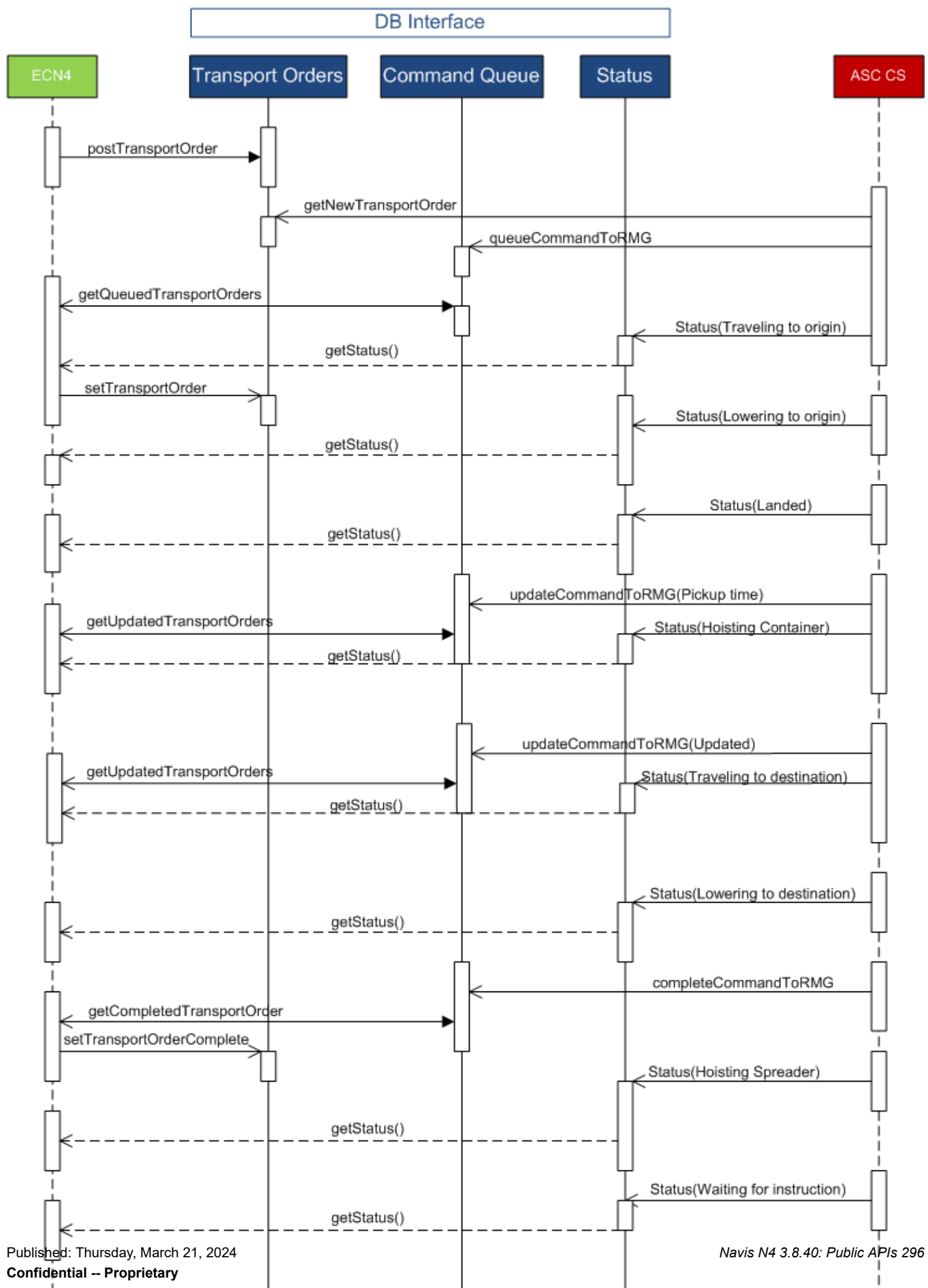
In This Section

Transport Order Data Flow.	295
Transfer Point Status Update.	315
Area Status Update.	332

11.2.1 Transport Order Data Flow

11.2.1.1 Primary Interaction

The interaction diagram below shows how ECN4 and the ACCS work together to dispatch and complete a transport order.



This use case elaborates on the interaction.

Step	Action
1	ECN4 issues a new transport order to the Transport Order table. The record includes: • ASC_TRANSPORT_ORDER.TO_GKEY= unique database generated sequencing field • ASC_TRANSPORT_ORDER.CQ_GKEY= NULL • ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED • ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order • ASC_TRANSPORT_ORDER.CREATED= current time stamp • ASC_TRANSPORT_ORDER.UPDATED= current time stamp
2	The ASC CS queries the Transport Order table for new transport orders, validates the information, issues the command to the ACCS, and issues a command to the Command Queue table. The Status table must be updated as well in this step. The Command Queue table and Status table are updated as below: • ASC_COMMAND_QUEUE.CQ_GKEY= unique database generated sequencing field • ASC_COMMAND_QUEUE.TO_GKEY= unique id of the transport order issued by TOS for the particular transport order. • ASC_COMMAND_QUEUE.CQ_STATUS= WORKING • ASC_COMMAND_QUEUE.CHE_ID= ID of ASC being issued the transport order • ASC_COMMAND_QUEUE.CREATED=current time stamp • ASC_COMMAND_QUEUE.UPDATED= current time stamp • STATUS.TO_GKEY= transport order currently being executed • STATUS.CQ_GKEY= command currently being executed • STATUS.WORK_STATUS= WORKING • STATUS.SEQUENCE_STATUS= TRAVELING TO ORIGIN • STATUS.UPDATED= current time stamp

Step	Action
3	ECN4 polls the Command Queue table for records where the CQ_GKEY is NOT set in the ECN4 Transport Orders table and the command key CQ_GKEY is present in the Command Queue table. ECN4 updates the status of the CQ_GKEY field in its Transport Order table and the TO_STATUS to WORKING
4	Once the ASC has arrived at the move origin, ACCS updates the Status table: STATUS.SEQUENCE_STATUS= LOWERING TO ORIGIN STATUS.UPDATED= current time stamp
5	When the crane lands the spreader on the container the ACCS updates the Status table STATUS.SEQUENCE_STATUS= LANDED STATUS.UPDATED= current time stamp
6	The crane locks the twistlocks and starts hoisting the container. The ACCS updates the Command Queue table with the pickup time and Status table with the new sequence: ASC_COMMAND_QUEUE.CQ_STATUS= WORKING ASC_COMMAND_QUEUE.PICKUP_TIME= current time stamp ASC_COMMAND_QUEUE.UPDATED= current time stamp STATUS.SEQUENCE_STATUS= HOISTING CONTAINER STATUS.UPDATED=current time stamp ECN4 updates the move stage and job step state to CARRY_UNDERWAY
7	The crane completes hoisting and starts traveling to the destination. The ACCS updates the Command Queue and Status tables with the new sequence: ASC_COMMAND_QUEUE.CQ_STATUS= WORKING ASC_COMMAND_QUEUE.UPDATED= current time stamp STATUS.SEQUENCE_STATUS= TRAVELING TO DESTINATION STATUS.UPDATED= current time stamp
8	Once the ASC has arrived at the move destination, ACCS updates the Status table: STATUS.SEQUENCE_STATUS= LOWERING TO DESTINATION STATUS.UPDATED= current time stamp

Step	Action
9	Once the ASC has arrived at the move destination, ACCS updates the Status table: STATUS.SEQUENCE_STATUS= COMPLETE STATUS.UPDATED= current time stamp This update would include the final destination and other completion attributes - all in ONE I/O.
10	ECN4 polls the Command Queue table using CQ_GKEY as a link. It requests all rows where CQ_STATUS = "COMPLETE" in the Command Queue Table and NOT set in the ECN4 Transport Order Table.
11	ECN4 marks the transport order as "COMPLETE" in the ECN4 Transport Order Table under TO_STATUS .
12	When the crane is lifting the spreader, the Status table must be updated with the corresponding sequence: STATUS.SEQUENCE_STATUS= HOISTING SPREADER STATUS.UPDATED= current time stamp
13	The move has been completed and ACCS must initialize the crane for ECN4 to know that the crane is IDLE and ready to be dispatched to execute a new transport order. STATUS.TO_GKEY= 0 STATUS.CQ_GKEY= 0 STATUS.WORK_STATUS= IDLE

11.2.1.2 Abort Interactions

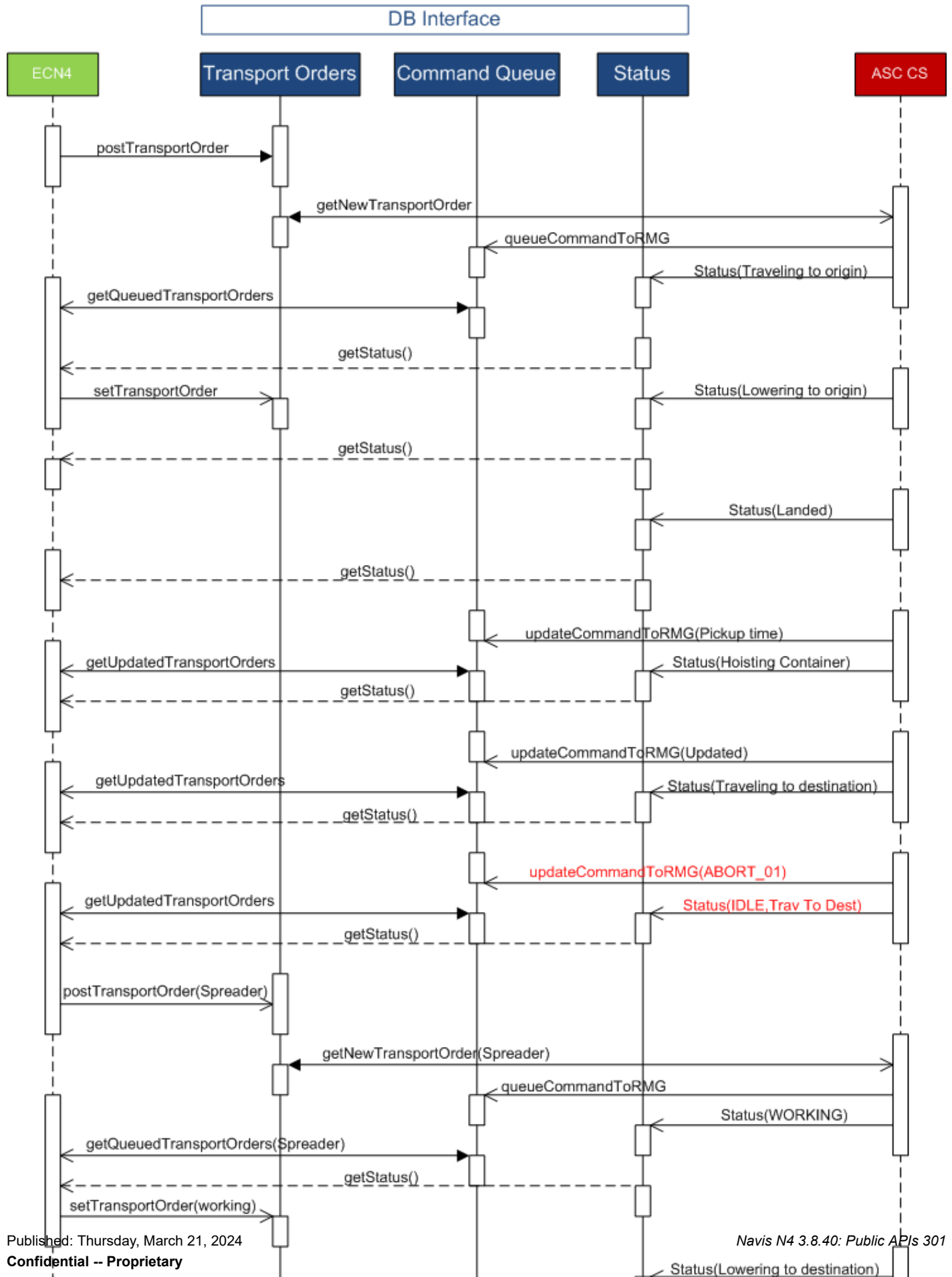
Aborts can only be performed by the ACCS application after the transport order has been placed in the ASCs order queue. In the event of an abort, the status of the command to the ASC will be set to 'ABORT_xx' where "xx" is the abort error code (See table below). Based upon the abort error code, ECN4 will cancel any subsequently issued transport orders and re-issue a transport order as appropriate, e.g. Abort on set down - ECN4 picks a new location, Abort on pick-up - ECN4 dispatches a new container move, etc.

Abort code	Description	Action
ABORT_01	Abort on set down	1. ECN4 sets the TO_STATUS = "ABORT_01" 2. ECN4 issues a new transport order to place the container in a new location. 3. New transport order has an ORIG_BAY_TYPE = "SPREADER".
ABORT_02	Abort on pick-up	1. ECN4 sets the TO_STATUS = "ABORT_02" 2. ECN4 resets the work instruction and places an "Bypassed" on the move. 3. ECN4 issues a new transport order to move a different container.

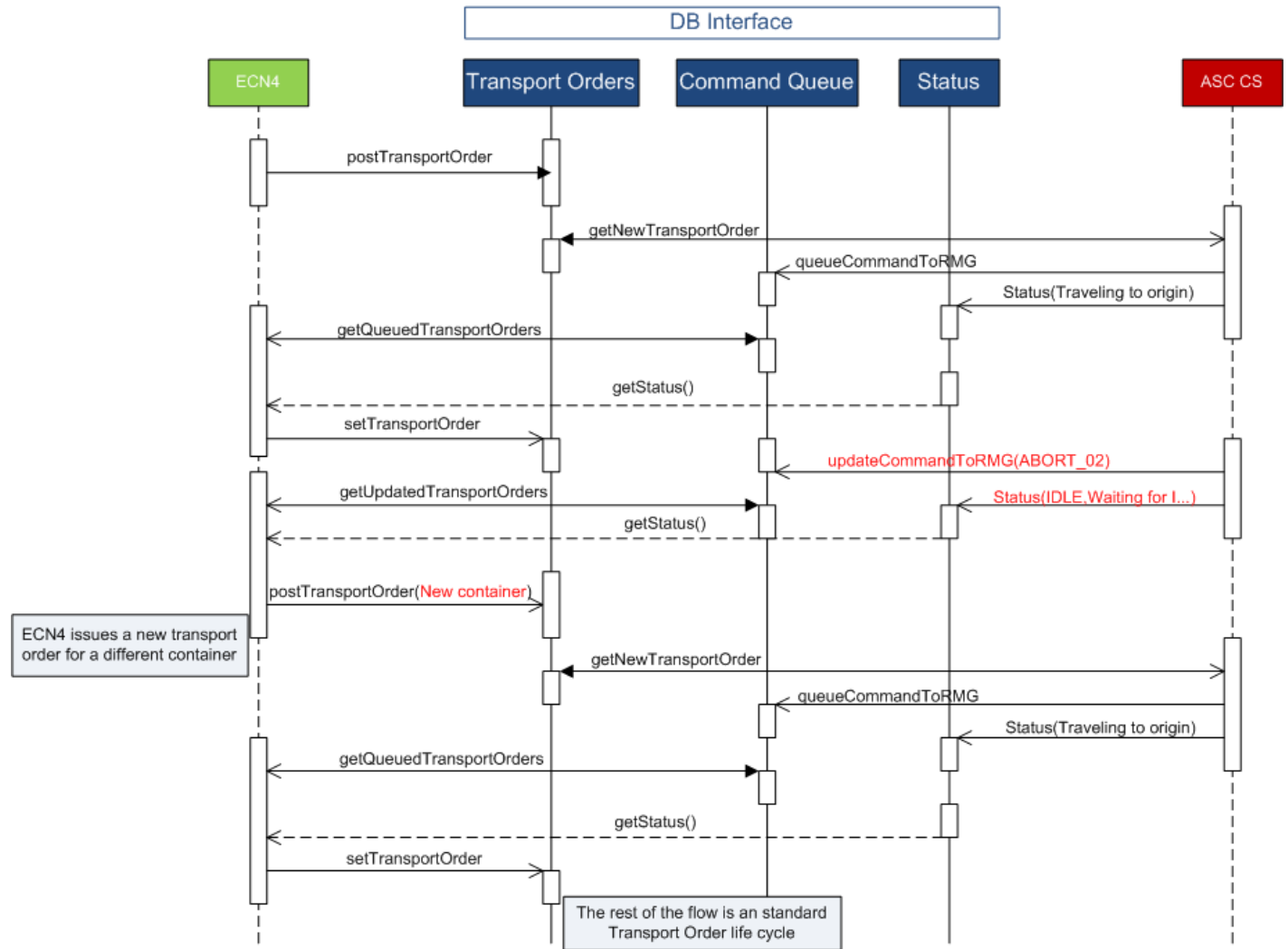
Abort code	Description	Action
ABORT_03	Abort due to Door Direction	1. ECN4 sets the TO_STATUS = "ABORT_03" 2. ECN4 issues a new transport order to place the container in a new location within the ASC stack. 3. New transport order has an ORIG_BAY_TYPE = "SPREADER". 4. ECN4 schedules a UTR to move to the stack to facilitate a door direction change.
ABORT_04	Abort due to invalid command to ASC	1. ECN4 sets the TO_STATUS = "ABORT_04" 2. ECN4 resets the work instruction and places an "Bypassed" on the move. 3. ECN4 issues a new transport order to move a different container.

These are the default abort use case implementations in ECN4. The interface provides extension points to allow alternative implementations as needed.

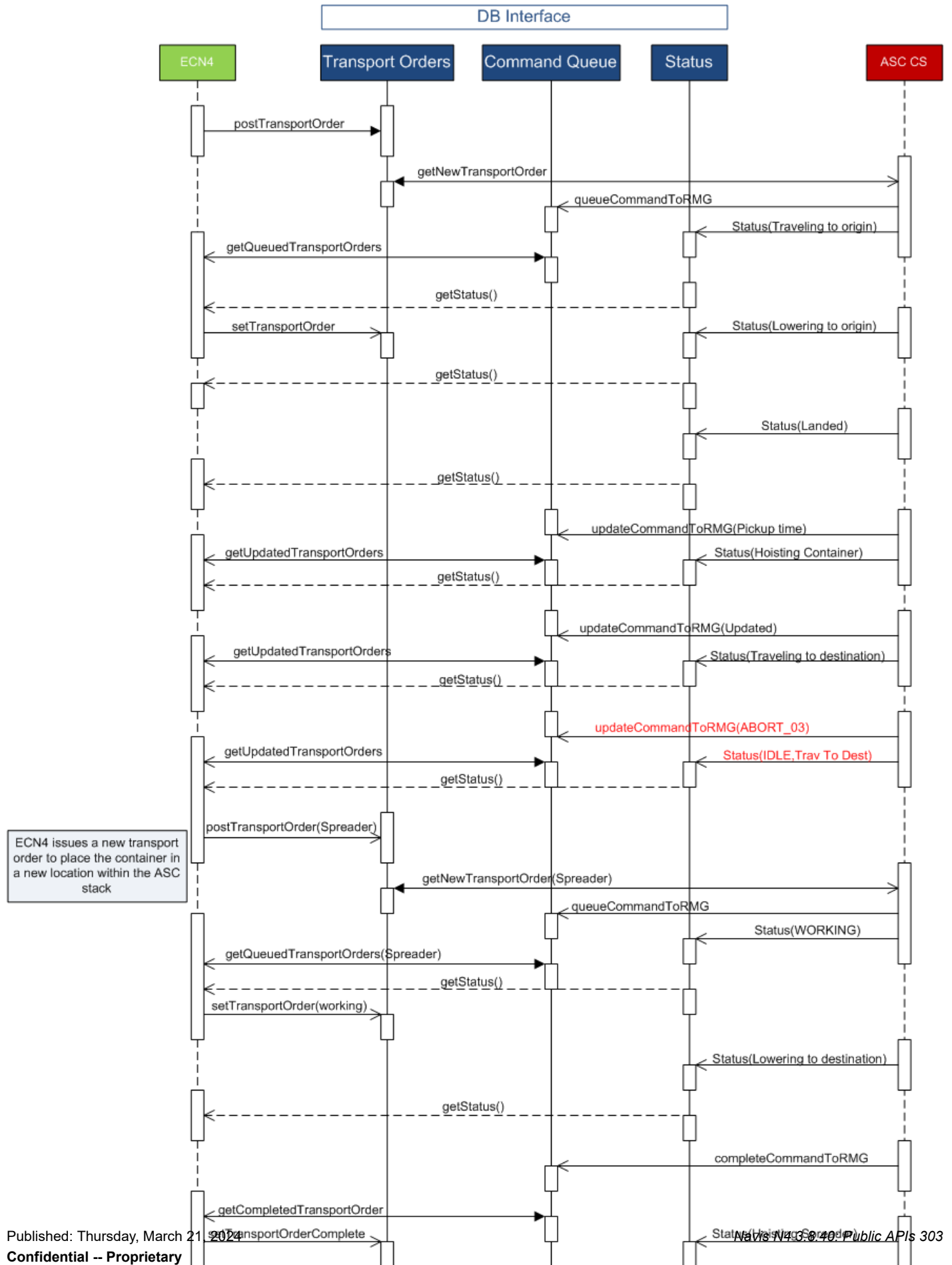
Abort_01 Interaction Diagram

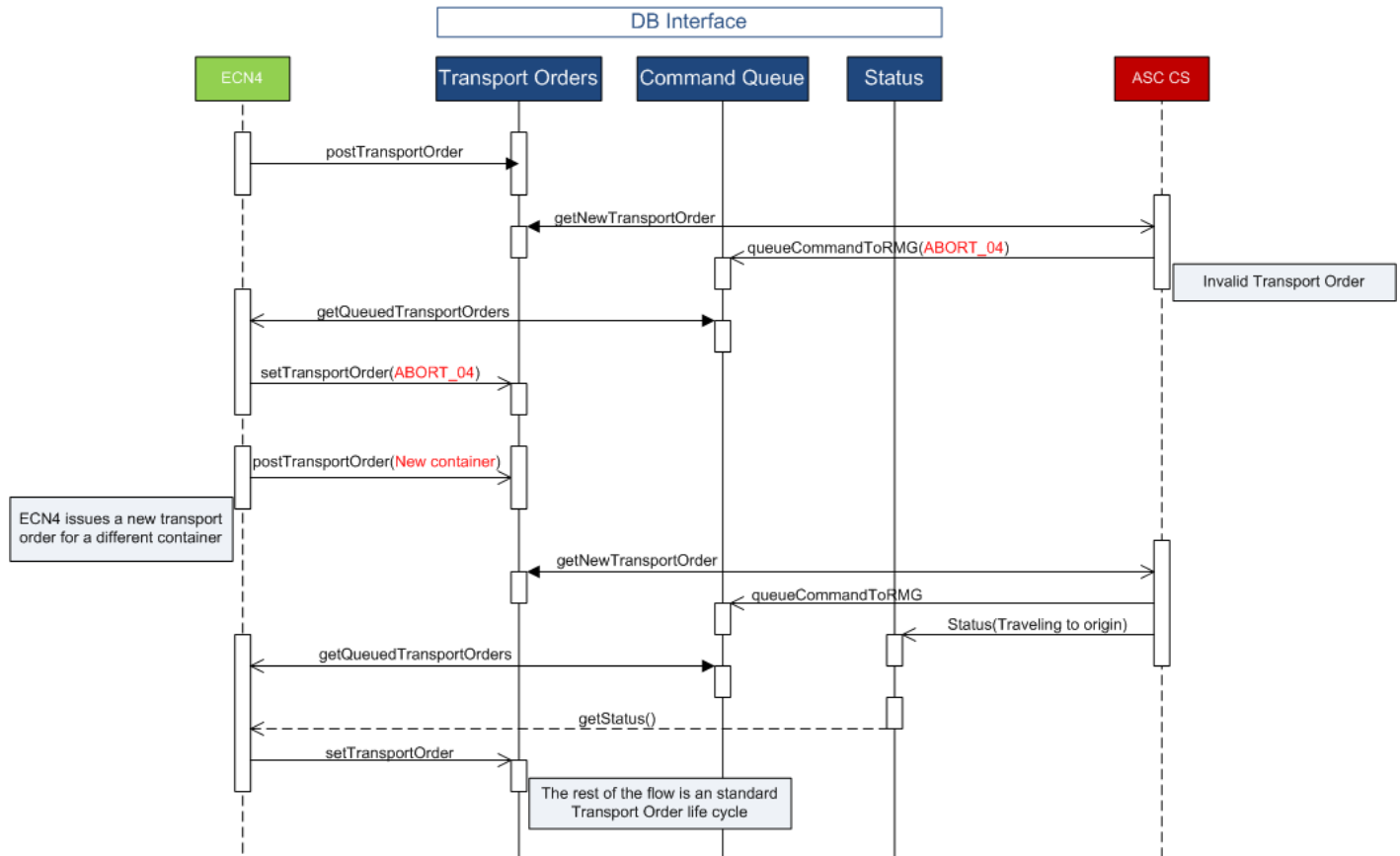


Abort_02 Interaction Diagram



Abort_03 Interaction Diagram



Abort_04 Interaction Diagram

11.2.1.3 Cancellations

ECN4 may need to cancel a move which has been entered into the Transport Order Table. Typically this is due to a user reverting a work instruction that was just dispatched. Only orders which have NOT been started may be canceled, consequently ECN4 will only update the **TO_STATUS** to "CANCEL" if the status equals "ENTERED".

Note that there is a potential race condition related to the actual start of the order and the intent to cancel. The presence of the order in the CQ table will override attempts to cancel, and the order will complete as normal.

11.2.1.4 Offline Operation of ASC and Inventory Synchronization

In exception cases ASC operations take place without the direction of ECN4. There are three types of "non-TOS" crane operations, manual (a.k.a. un-commanded) moves, ACCS-initiated moves, and TOS-initiated then manually completed (a.k.a. half-commanded) moves. Manual moves are initiated by the remote operator. In this case ACCS will report these moves as a series of `ASC_COMMAND_QUEUE` table entries. A series of moves is required since the ACCS cannot determine the origin and destination in advance so the series is typically a move from a location to the spreader and a subsequent move from the spreader to a location. An ACCS-initiated move is similar to a TOS move in that it is "commanded" and has a planned source and destination.

In the manual and ACCS-initiated cases, the ACCS records all container movements to the Command Queue table without `TO_GKEYs`. ECN4 will interpret these sequences either in real-time, or when it reconnects to the database. ECN4 shall "apply" these movements by inserting corresponding Transport Orders. Then the ACCS must update the `TO_GKEY` in the Command Queue table from the corresponding Transport Orders.

A TOS initiated and manually completed move occurs when the ACCS switches to manual mode and remote operator either cancels the move, triggering an abort_01, but then completes the move manually, or simply completes the move in manual mode without cancelling. In the case where the operator cancels the move and then completes manually:

- ACCS sets the move's current Command Queue record's CQ_STATUS to "ABORT_01"
- ECN4 sets the move's Transport Order record's TO_STATUS to "ABORT_01"
- The ACCS creates a new Command Queue record for the half-commanded move with the container ID, origin at the spreader and destination where the operator set the container.
- ECN4 "applies" the completed move by creating a Transport Order for the spreader > set down move and updates the inventory with the location specified by the ACCS.
- The ACCS updates the Command Queue for the half-commanded move with the TO_GKEY from the transport order created by ECN4.

In the case where the operator does not cancel the move and completes manually:

- The ACCS updates Command Queue record, setting the destination to the location where the container was set and sets CQ_STATUS to "COMPLETE"
- ECN4 updates Transport Order, setting the destination to the location where the container was set, sets TO_STATUS to "COMPLETE" and updates the inventory with the location specified by the ACCS.

When connecting to the database ECN4 will check the status of the ASC_COMMAND_QUEUE. If there are "un-applied" records in the COMMAND_QUEUE table ECN4 will "apply" the offline moves before issuing any new transport orders.

The ACCS must check the status of the ASC_COMMAND_QUEUE before inserting a new command queue record and update the TO_GKEYs for any command queue records that have been "applied" by ECN4.

The ACCS may also use the COMMAND_QUEUE table to record crane positioning moves, for example when moving to a maintenance location. These moves are initiated by the ACCS, and may or may not involve a container. Moves that do NOT involve a container will have a container number field with NULL value.

11.2.1.5 Road Truck Chassis Positions

Containers destined for the land side transfer zone may arrive or depart on one or two road truck chassis. The **ORIG_POS_TRUCK** and **DEST_POS_TRUCK** shall be used to indicate the "Position on Truck".

- Position "1" will be the forward 20' position of the first chassis after the cab (side that connects to truck)
- Position "2" will be the middle position of the first chassis, if chassis is 2 TEU chassis, and may contain a 20', 40' or 45' container
- Position "3" will be the aft 20' position of the first chassis, if chassis is 2 TEU chassis, or the forward 20' position on a chassis following a 1 TEU chassis
- Position "4" will be middle position on a 2 TEU chassis following a 1 TEU chassis and may contain a 20', 40' or 45' container
- Position "5" will be the forward 20' position on a chassis following a 2 TEU chassis, the aft 20' position on a 2 TEU chassis following a 1 TEU chassis or the aft position on a 1.5 TEU chassis
- Position "6" will be the middle position on a 2 TEU chassis following a 2 TEU chassis and may contain a 20', 40' or 45' container
- Position "7" will be the aft 20' position on the chassis following a 2 TEU chassis

11.2.1.6 Door Direction Changes

In some cases, containers being transferred out of the stack will have the incorrect door direction. In such cases, one of two actions must be taken.

- If the container cannot be placed on the chassis, the **TO_STATUS** should be set to 'ABORT_03' (see Abort case above). This will trigger ECN4 to generate the instructions necessary to change the door direction. In this case that means a work instruction to place the container back into the ASC stack. Another work instruction is from the ASC stack to the FLIP queue. This will then trigger a yard truck to come to the land side transfer zone to receive the container. Once the container is on the yard truck that CHE goes to the FEL where it is moved from the internal truck to the external truck.

Extensions?

The behavior for door direction changes are handled as an ECN4 extension. A customer can choose to have that extension customized to fit their own use.

Who identifies the door direction being a problem?

It is crucial to understand that the identification of a door direction issue is not within the scope of ECN4. This is something that comes either as a function of the ACCS, or more likely, as a result of a remote operator involvement. The above codes are how these issues are communicated to ECN4.

11.2.1.7 ECN4 Start Up and Recovery Process

Step	Action
1	ECN4 starts up
2	If Transfer Points Status table is in use the status of each transfer point is initialized with current status as recorded by ECN4
3	If the Area Status table is in the ECN4 polls the Area Status table and updates the stack status in the TOS
4	ECN4 polls the Command Queue and ASC Status tables and sequentially processes each retrieved row.

11.2.1.8 ASC Status

ECN4 requires certain status information about ASC be shared by the ACCS application in order to appropriately update the work instruction in N4, as described in the primary interaction and abort interactions, as well as to display the state of the ASC to the user. The ACCS maintains ASC status information about the CHE in the STATUS table. The advantage of this approach is that it does not require direct communication between the ACCS application and ECN4; ECN4 simply reads from this status table in order to determine the current status of each CHE within the ACCS application.

Each ASC on the terminal will be allocated ONE row in the STATUS table. The ACCS will be responsible for updating the status information in the table periodically, or upon completion of an event. The ACCS software will always use SQL UPDATE commands - NEVER INSERT commands. Rows will be pre-allocated by an offline process to insure the integrity of the table.

 The ACCS will also add a row with CHE_ID = 'Navis', ONLINE_STATUS = 'AUTO' , WORK_STATUS ='IDLE' and SEQUENCE_STATUS ='WAITING FOR INSTRUCTION'.

 The TOS will update this row on regular intervals. The update to this row acts as a heartbeat indicator for the ACCS.

Step	Action
1	ACCS posts update to the ASC Status Table
2	ECN4 periodically queries the ASC Status Table and retrieves the current status of the ASC.
3	ECN4 updates the ASC working status, online status, job step state and the last known position of the ASC.

11.2.1.9 ECN4 Transport Order Table (ASC_TRANSPORT_ORDERS)

Column Name	Data Type	Description	Example	Default
TO_GKEY	Number(14)	ECN4 transport order sequence. Primary key, Indexed field - no duplicates - automatically generated by database.	41272147385345 (MAX)	DB GENERATED
CQ_GKEY	Number(14)	ACCS transport order sequence Indexed field	43812147385345 (MAX)	NULL
TO_STATUS	String(20)	Status of transport order as issued, complete or aborted	"ENTERED" "WORKING" "COMPLETE" "ABORT_01" "ABORT_02" "ABORT_03" "ABORT_04" "CANCEL"	NULL
CHE_ID	String(7)	ID of ASC being issued the transport order	"ASC03"	
CONTAINER_ID	String(12)	ID of container to be moved Indexed field	"MSKU1234567"	NULL
PTA	Data/Time	Planned time of arrival	25-MAY-2005 12:00:05	NULL
CONTAINER_LENGTH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Length of container	610 1219 1372	NULL
CONTAINER_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Height of container	244 259 279 290	NULL
CONTAINER_ISO	String (4)	ISO code of container	"4300"	NULL

Column Name	Data Type	Description	Example	Default
CONTAINER_DOOR_DIRECTION	String(3)	Container door direction 'FWD' is normal door direction for the block. 'AFT' is door direction is reverse 'NA' system does not care about door direction. Default = 'NA'	"FWD" "AFT" "NA"	"NA"
ORIG_BLOCK	String (6)	Origin block of the container	"02"	NULL
ORIG_BAY_TYPE	String (10)	Bay type Note: SPREADER is a valid ORIG, but is NOT a valid DEST.	'RMG' 'WATERSIDE' 'LANDSIDE' 'SPREADER'	NULL
ORIG_BAY	String (3)	Origin row of container, formatted as defined in SPARCS yard model.	"020" "20"	NULL
ORIG_CELL	String (3)	Origin column of container, formatted as defined in SPARCS yard model.	"01"	NULL
ORIG_TIER	String (1)	Origin tier of container Default = '0' (not specified)	"3"	"0"
ORIG_FLOOR_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Highest point of stack above the ground	777	NULL
ORIG_VEHICLE_TYPE	String (4)	Type of vehicle Default="NONE"	"OTR" "UTR" "NONE"	"NONE"
ORIG_POS_TRUCK	String (2)	Location on truck chassis, see section on Road Truck Chassis Positions. Default = "0"	"0" "2" "3"	"0"
DEST_BLOCK	String (6)	Destination block of the container	"02"	NULL
DEST_BAY_TYPE	String (10)	Bay type Note: SPREADER is a valid ORIG but NOT a valid DEST	"RMG" "WATERSIDE" "LANDSIDE"	NULL
DEST_BAY	String (3)	Destination row of container, formatted as defined in SPARCS yard model.	"020"	NULL

Column Name	Data Type	Description	Example	Default
DEST_CELL	String (3)	Destination column of container, formatted as defined in SPARCS yard model.	"001"	NULL
DEST_TIER	String (1)	Destination tier of container Default = '0' (not specified)	"3"	"0"
DEST_FLOOR_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Highest point of stack above the ground	777	NULL
DEST_VEHICLE_TYPE	String (4)	Type of vehicle Default=NONE	"OTR" "UTR" "NONE"	"NONE"
DEST_POS_TRUCK	String (2)	Location on truck chassis, see section on Road Truck Chassis Positions Default = 0	"0" "1" "2"	"0"
CREATED	Date/Time	Creation time of record	25-MAY-2005 11:30:05	current time stamp
UPDATED	Date/Time	Time of last update	25-MAY-2005 11:32:45	NULL
MOVE_TYPE	String (35)	Type of order issued by N4. Used for reporting.	"Unknown" "Transfer_In" "Transfer_Out" "Intrastack" "Split_Transfer_In" "Split_Transfer_Out" "Split_Intrastack" "Rehandle" "Preposition" "Housekeeping"	NULL

Column Name	Data Type	Description	Example	Default
ACK	DATETIME	The time ECN4 observed the corresponding update. On the next poll cycle, ECN4 will query for updates from that timestamp. This is different from UPDATED, the time ECN4 updated the TO row after processing an update. This field is for internal ECN4 use only.	25-MAY-2005 11:32:45	NULL

Data Types

- Data Type Number (I4) designates Integer - 4 byte which has a maximum value of 2147385345
- Data Type String (xx) designates a character string of maximum length xx
- Unsigned 2-byte integer range: 0 to 65,535 which equals 2¹⁶ – 1

11.2.1.10 ACCS Command Queue Table (ASC_COMMAND_QUEUE)

Column Name	Data Type	Description	Example	Default
CQ_GKEY	Number(I4)	ACCS transport order sequence Primary key, Indexed field - no duplicates - automatically generated by database.	738 // 2147385345 (MAX)	DB GENERATED
TO_GKEY	Number(I4)	ECN4 transport order sequence. Indexed field	863 2147385345 (MAX)	NULL
CQ_STATUS	String(20)	Status of transport order as issued, complete or aborted	"ENTERED" "WORKING" "COMPLETE" "ABORT_01" "ABORT_02" "ABORT_03" "ABORT_04" "CANCEL" (echoed)	NULL

Column Name	Data Type	Description	Example	Default
CHE_ID	String(7)	ID of ASC being issued the transport order	"RMG03"	NULL
CONTAINER_ID	String(12)	ID of container to be moved Indexed field	MSKU1234567	NULL
ATA	Data/Time	Actual time of arrival	25-MAY-2005 12:00:05	NULL
CONTAINER_LENGTH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Length of container	610 1219 1372	NULL
CONTAINER_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Height of container	244 259 279 290	NULL
CONTAINER_ISO	String (4)	ISO code of container	"4300"	NULL
CONTAINER_DOOR_DIRECTION	String(3)	Container door direction 'FWD' is normal door direction for the block. 'AFT' is door direction is reverse 'NA' system does not care about door direction. Default = 'NA'	"FWD" "AFT" "NA"	"NA"
ORIG_BLOCK	String (6)	Origin block of the container	"02"	NULL
ORIG_BAY_TYPE	String (10)	Bay type Note: SPREADER is a valid ORIG, but is NOT a valid DEST.	'RMG' 'WATERSIDE' 'LANDSIDE' 'SPREADER'	NULL
ORIG_BAY	String (3)	Origin row of container, formatted as defined in SPARCS yard model. .	"020"	NULL
ORIG_CELL	String (3)	Origin column of container, formatted as defined in SPARCS yard model.	"001"	NULL
ORIG_TIER	String (1)	Origin tier of container Default = '0' (not specified)	"3"	"0"
ORIG_FLOOR_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Highest point of stack above the ground	777	NULL

Column Name	Data Type	Description	Example	Default
ORIG_VEHICLE_TYPE	String (4)	Type of vehicle Default=NONE	"OTR" "UTR" "NONE"	"NONE"
ORIG_POS_TRUCK	String (2)	Location on truck chassis, see section on Road Truck Chassis Positions. Default = 0	"0" "1" "2"	"0"
DEST_BLOCK	String (6)	Destination block of the container	"02"	NULL
DEST_BAY_TYPE	String (10)	Bay type Note: SPREADER is a valid ORIG but NOT a valid DEST	"RMG" "WATERSIDE" "LANDSIDE"	NULL
DEST_BAY	String (3)	Destination row of container, formatted as defined in SPARCS yard model.	"020"	NULL
DEST_CELL	String (3)	Destination column of container, formatted as defined in SPARCS yard model.	"001"	NULL
DEST_TIER	String (1)	Destination tier of container Default = '0' (not specified)	"3"	"0"
DEST_FLOOR_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Highest point of stack above the ground	777	NULL
DEST_VEHICLE_TYPE	String (4)	Type of vehicle Default="NONE"	"OTR" "UTR" "NONE"	"NONE"
DEST_POS_TRUCK	String (2)	Location on truck chassis, see section on Road Truck Chassis Positions. Default = "0"	"0" "1" "2"	"0"
PRIORITY	String (1)	Command execution priority. Not set by ECN4	1 - Lowest with increasing priority with ascending values	NULL
CREATED	Date/Time	Creation time of record	25-MAY-2005 11:30:05	current time stamp
UPDATED	Date/Time	Time of last update	25-MAY-2005 11:32:45	NULL
OPERATOR1	String (15)	First operator name	"POTTER"	NULL

Column Name	Data Type	Description	Example	Default
OPERATOR2	String (15)	Second operator name	"SNAPE"	NULL
SUPERUSER	BIT (1)	Indicates super user operated on command	1 - yes 0 - no	NULL
PICKUP_TIME	Date/Time	Date and time container picked up with spreader	25-MAY-2005 11:31:22	NULL
MEASURED_WEIGHT	String (6)	Measured weight in a unit value that is configured on the CHE level. Typical values would be pounds or kilograms Value is applied to the yard measured weight attribute of the container	"75000"	NULL
WAITFOROPR	Unsigned 2-byte integer or Int (SQL Server 2008)	Time waiting for operator (sec)	10	NULL
WAITFORWS	Unsigned 2-byte integer or Int (SQL Server 2008)	Time waiting for waterside to be clear (sec)	11	NULL
WAITFORCRN	Unsigned 2-byte integer or Int (SQL Server 2008)	Time waiting for other crane to clear the area(sec)	12	NULL

Data Types

- Data Type Number (l4) designates Integer - 4 byte which has a maximum value of 2147385345
- Data Type String (xx) designates a character string of maximum length xx
- Container_ID of NONE is used when the crane is being positioned.
- Unsigned 2-byte integer range: 0 to 65,535 which equals 216 – 1

11.2.1.11 ACCS ASC Status Table Layout (STATUS)

Column Name	Data Type	Description	Example	Default
CHE_ID	String(7)	ID of ASC being issued the transport order	RMG03	NOT NULL

Column Name	Data Type	Description	Example	Default
ONLINE_STATUS	String(10)	Online status of ASC	"AUTO" "MANUAL" "PENDING" "YARD" "OFF"	NULL
WORK_STATUS	String(20)	Work status of ASC	"WORKING" "IDLE" "FAULT"	NULL
TO_GKEY	Number(I4)	Transport order currently being executed Set to zero when WORK_STATUS is IDLE	863 2147385345 (MAX)	NULL
CQ_GKEY	Number(I4)	Command currently being executed Set to zero when WORK_STATUS is IDLE	738 // 2147385345 (MAX)	NULL
LOC_BLOCK	String (6)	Current block of the ASC	"02"	NULL
LOC_BAY_TYPE	String (10)	Current bay type location of ASC	"RMG" "WATERSIDE" "LANDSIDE"	NULL
LOC_BAY	String (3)	Current bay of the ASC, formatted as defined in SPARCS yard model.	"020" "20"	NULL
UPDATED	Date/Time	Time of last update	25-MAY-2005 11:32:45	NULL
CONTAINER_1	String (12)	Current container 1 (ASC use only) Indexed field Set to an empty string (string = "") when WORK_STATUS is IDLE	"MSKU1234567"	NULL
CONTAINER_2	String (12)	Current container 2 (ASC use only) Indexed field - Blank if only 1 container is being operated on Set to an empty string (string = "") when WORK_STATUS is IDLE	"MSKU357193"	NULL

Column Name	Data Type	Description	Example	Default
SEQUENCE_STATU S	String (50)	Additional details of action being carried out by ASC	WAITING FOR INSTRUCTION WAITING CRANE TO CLEAR AREA TRAVELING TO ORIGIN LOWERING TO ORIGIN LANDED HOISTING CONTAINER TRAVELING TO DESTINATION LOWERING TO DESTINATION WAITING TO ENTER TRANSFER AREA MOVING AWAY FROM OTHER CRANE HOISTING SPREADER MANUAL	NULL

11.2.2 Transfer Point Status Update

11.2.2.1 ECN4 Transfer Point Status Table (TRANSFER_POINT_STATUS)

This table is updated by ECN4 and contains the current inventory for all waterside and landside transfer points. There are transfer point records in the table for each slot defined in the yard file. This interface supports up to 4 containers on 2 road truck chassis for road transactions in a single landside transfer point as these may be defined as 80'+ wheeled slots capable of handling a 2 x 40' trailer road truck configuration.

The following constraints apply to **waterside** transfer point status:

- Only a single container will occupy a waterside transfer point. The transfer points are defined as cells for either 20' or 40' containers, i.e. a 40' row would have cells 01 and 03 for 20' containers and 02 for a 40' container so that a waterside lane/row would have 3 entries in the table.
- Use cases for chassis/cassettes in waterside transfer points are out of scope for 2.5.**

The following constraints apply to **landside** transfer point inventory:

- Multiple containers in a transfer point only occur for road transactions. In this case the entire road truck, containing up to 4 containers and 2 road truck chassis, is assigned to the same transfer point.
- Only one terminal managed chassis/cassette will occupy a transfer point. If the LS TZ is capable of holding more than one chassis/cassette then multiple rows should be defined. This would be similar to the waterside transfer points where each lane has multiple slots, where each slot is occupied by one chassis/cassette.
- Only one container can be mounted on a chassis/cassette.
- Use cases for bundled chassis or stacked cassettes in transfer points are not supported.

The trigger points for updating the Transfer Point Status table are as follows:

- When ECN4 is started up, ECN4 initializes all transfer points with the current status.
- The transfer points for a single ASC can be re-initialized by the user resetting the ASC (RESET CHE action in XPS client).
- A transfer point status record is updated whenever a container lift/set, chassis park/pull or road truck arrival is recorded in the transfer point.
 - When a container is removed from the transfer point by an ASC the CONTAINER attributes are reset to their defaults (NULL)
 - When a container is removed from the transfer point by shuttle, terminal truck or road truck, the transfer point STATUS is updated to indicate that the containers have been removed but the CONTAINER, CHASSIS and TRK_CHASSIS attributes remain to provide a reference to the ACCS of the removed container, chassis or road truck.
- For road truck transactions, when all transactions for the road truck for the ASC are completed it is assumed that the road truck departs and the transfer point is now unoccupied (status = 0). There is no event recorded in the TOS to indicate that a road truck has left the transfer point.

Please refer to the use cases below for further details on transfer point status table updates.

Creation and use of this table is optional. Whether ECN4 writes to the TRANSFER_POINT_STATUS table is controlled via the TRANSFER_POINT_STATUS_ENABLED setting.

Column Name	Data Type	Description	Example	Default
BLOCK	String (6)	ASC Module Name	"02"	NULL
BAY_TYPE	String (10)	[RMG, WATERSIDE, LANDSIDE, SPREADER]. For this interface only WATERSIDE and LANDSIDE transfer point locations will be updated.	"WATERSIDE"	NULL
BAY	String (3)	Row name, formatted as defined in SPARCS yard model.	"002" "02"	NULL
CELL	String (3)	Column name. In TZ this is the lane name. Formatted as defined in SPARCS yard model.	"001" "00A" "A"	NULL
TIER	String (1)	Tier name	"1" "2" "A"	NULL
VEHICLE_TYPE	String (4)	Type of vehicle, if any, used for pull or park in wheeled slot	"OTR" "UTR"	NULL

Column Name	Data Type	Description	Example	Default
STATUS	Unsigned 2-byte integer or Int (SQL Server 2008)	0 – Unoccupied 1 – Occupied by container 2 - Occupied by bare chassis/cassette 3 - Occupied by container on chassis/cassette 4 - Occupied by road truck	1	NULL
CREATED	Date/Time	Record created time	25-MAY-2005 11:32:45	current time stamp
UPDATED	Date/Time	Record updated time	25-MAY-2005 11:32:45	NULL
CHE_ID	String (7)	ID of ASC, shuttle carrier or terminal tractor that completed last lift/pull (for unoccupied slot) or set/park (for occupied slot)	"RMG01" "SHC03" "TT04" "2ABC1234"	NULL
CONTAINER1_ID	String (12)	Container number e.g. HLXU1234567	"HLXU1234567"	NULL
CONTAINER1_ISO	String (4)	Container ISO code	"2200"	NULL
CONTAINER1_TRK_POS	String(2)	For landside transfer points, the position of the container on a road truck's chassis.	"1" "2" "3" "4" "5" "6" "7"	NULL
CONTAINER1_LENGTH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical length of container in centimeters	610 1219 1379	NULL
CONTAINER1_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical height of container in centimeters	244 259 279 290	NULL
CONTAINER1_DOOR_DIRECTION	String (3)	Container door direction 'FWD' is normal door direction for the block. 'AFT' is door direction is reverse 'NA' system does not care about door direction.	"FWD" "AFT" "NA"	NULL

Column Name	Data Type	Description	Example	Default
CONTAINER1_WEIGHT_KG	Unsigned 2-byte integer or Int (SQL Server 2008)	Unit gross weight in kg.	23000	NULL
CONTAINER2_ID	String (12)	Container number e.g. HLXU1234567	"HLXU1234567"	NULL
CONTAINER2_ISO	String (4)	Container ISO code	"2200"	NULL
CONTAINER2_TRK_POS	String(2)	For landside transfer points, the position of the container on a road truck's chassis.	"1" "2" "3" "4" "5" "6" "7"	NULL
CONTAINER2_LENGTH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical length of container in centimeters	610 1219 1379	NULL
CONTAINER2_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical height of container in centimeters	244 259 279 290	NULL
CONTAINER2_DOOR_DIRECTION	String (3)	Container door direction 'FWD' is normal door direction for the block. 'AFT' is door direction is reverse 'NA' system does not care about door direction.	"FWD" "AFT" "NA"	NULL
CONTAINER2_WEIGHT_KG	Unsigned 2-byte integer or Int (SQL Server 2008)	Unit gross weight in kg.	23000	NULL
CONTAINER3_ID	String (12)	Container number e.g. HLXU1234567	"HLXU1234567"	NULL
CONTAINER3_ISO	String (4)	Container ISO code	"2200"	NULL
CONTAINER3_TRK_POS	String(2)	For landside transfer points, the position of the container on a road truck's chassis.	"1" "2" "3" "4" "5" "6" "7"	NULL
CONTAINER3_LENGTH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical length of container in centimeters	610 1219 1379	NULL

Column Name	Data Type	Description	Example	Default
CONTAINER3_HEIG HT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical height of container in centimeters	244 259 279 290	NULL
CONTAINER3_DOO R_DIRECTION	String (3)	Container door direction 'FWD' is normal door direction for the block. 'AFT' is door direction is reverse 'NA' system does not care about door direction.	"FWD" "AFT" "NA"	NULL
CONTAINER3_WEIG HT_KG	Unsigned 2-byte integer or Int (SQL Server 2008)	Unit gross weight in kg.	23000	NULL
CONTAINER4_ID	String (12)	Container number e.g. HLXU1234567	"HLXU1234567"	NULL
CONTAINER4_ISO	String (4)	Container ISO code	"2200"	NULL
CONTAINER4_TRK_ POS	String(2)	For landside transfer points, the position of the container on a road truck's chassis	"1" "2" "3" "4" "5" "6" "7"	NULL
CONTAINER4 LENG TH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical length of container in centimeters	610 1219 1379	NULL
CONTAINER4_HEIG HT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical height of container in centimeters	244 259 279 290	NULL
CONTAINER4_DOO R_DIRECTION	String (3)	Container door direction 'FWD' is normal door direction for the block. 'AFT' is door direction is reverse 'NA' system does not care about door direction.	"FWD" "AFT" "NA"	NULL
CONTAINER4_WEIG HT_KG	Unsigned 2-byte integer or Int (SQL Server 2008)	Unit gross weight in kg.	23000	NULL

Column Name	Data Type	Description	Example	Default
CONTAINER4_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical height of container in centimeters	244 259 279 290	NULL
CHASSIS1_ID	String (12)	ID of terminal managed chassis or cassette, either bare or married to the container	"APLZ1234567"	NULL
CHASSIS1_LENGTH_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical length of terminal managed chassis/cassette in centimeters	762 1372	NULL
CHASSIS1_HEIGHT_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Physical height of terminal managed chassis/cassette in centimeters	106	NULL
TRK_CHASSIS_1_TEU	Unsigned 2-byte integer or Int (SQL Server 2008)	TEU capacity of first road truck chassis directly behind the cab	2	NULL
TRK_CHASSIS_1_CRANE	BIT (1) or BIT (SQL Server 2008)	0 – Side loading cranes not present on road truck chassis 1 – Side loading cranes present on road truck chassis	1	NULL
TRK_CHASSIS_2_TEU	Unsigned 2-byte integer or Int (SQL Server 2008)	TEU capacity of second road truck chassis	2	NULL
TRK_CHASSIS_2_CRANE	BIT (1) or BIT (SQL Server 2008)	0 – Side loading cranes not present on road truck chassis 1 – Side loading cranes present on road truck chassis	0	NULL
TRK_CHASSIS_LENGTH_OVERALL_CM	Unsigned 2-byte integer or Int (SQL Server 2008)	Length overall of road truck chassis combination in centimeters	2560	NULL

Data Types

- Data Type Number (I4) designates Integer - 4 byte which has a maximum value of 2147385345
- Data Type String (xx) designates a character string of maximum length xx
- Container_ID of NONE is used when the crane is being positioned.
- Unsigned 2-byte integer range: 0 to 65,535 which equals $2^{16} - 1$

11.2.2.2 Transfer Point Status Use Cases

In these use cases, for brevity and clarity, only key attributes of the transfer point status tables will be listed in each step. Also, if the expected value of an attribute is the default value, then the attribute is not listed. If the value of an attribute is unchanged from one step to the next, it is not listed in the later step(s).

Intra Stack Move

- An intra stack move, where the container never enters a transfer zone, doesn't update the transfer point status table.

Waterside Inbound Move

1. A shuttle is dispatched to fetch a container from a non-ASC stack. Shuttle carrier picks up the container and drives to the waterside transfer zone.
2. Once the shuttle is at the WSTZ, it sets down the container and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 1
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= Shuttle ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
3. To dispatch an ASC to fetch the container, ECN4 issues a new transport order to execute the move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
4. Refer to steps 2 to 7 from Transport Order Life Cycle.
5. When the ASC is at the WSTZ, it picks up the container and ECN4 updates the status in the transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL
6. Refer to steps 8 to 13 from Transport Order Life Cycle.

Waterside Outbound Move

1. The ASC is dispatched to fetch a container residing within the ASC Stack. ECN4 issues a new transport order to execute the move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field

- ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
2. Refer to steps 2 to 11 from Transport Order Life Cycle.
 3. After the ASC has set down the container at the transfer zone, ECN4 updates the transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 1
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
 4. A shuttle is dispatched to fetch the container from the WSTZ.
 5. Once the shuttle is at the WSTZ, it picks up the container and ECN4 updates transfer point status table. Container information remains in the table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= Shuttle ID

Waterside Inter Stack Move

1. The ASC is dispatched to fetch a container residing within the ASC Stack. ECN4 issues a new transport order to execute the first move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
2. Refer to steps 2 to 11 from Transport Order Life Cycle.
3. After the ASC has set down the container at the transfer zone, ECN4 updates the transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 1
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
4. A shuttle is dispatched to fetch the container from the origin WSTZ.
5. Once the shuttle is at the WSTZ, it picks up the container and ECN4 updates transfer point status table. Container information remains in the table:
 - TRANSFER_POINT_STATUS.STATUS= 0

- TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= Shuttle ID
6. The shuttle carrier drives to the destination transfer zone. Once the shuttle is at the destination WSTZ, it sets down the container and ECN4 updates transfer point status table:
- TRANSFER_POINT_STATUS.STATUS= 1
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= Shuttle ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
7. The ASC is dispatched to fetch the container. ECN4 issues a new transport order to execute the second move. The record includes:
- ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
8. Refer to steps 2 to 7 from Transport Order Life Cycle.
9. When the ASC is at the WSTZ, it picks up the container and ECN4 updates the status in the transfer point status table:
- TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL
10. Refer to steps 8 to 13 from Transport Order Life Cycle.

Landside OTR Receive Export

1. Truck arrives at the LSTZ and notifies arrival.
2. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= OTR
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= container position on truck
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_CRANE= As per the spec.

- TRANSFER_POINT_STATUS.TRK_CHASSIS_2_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_LENGTH_OA_CM= As per the spec.
3. ECN4 issues a new transport order to execute the move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
 4. Refer to steps 2 to 7 from Transport Order Life Cycle.
 5. After the ASC lifts the container, there are no remaining transactions. The TOS assumes the trucks departs and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= NULL
 6. Refer to steps 8 to 13 from Transport Order Life Cycle.

Landside OTR Receive 2 Exports

1. Truck arrives at the LSTZ and notifies arrival.
2. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= OTR
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= container position on truck
 - TRANSFER_POINT_STATUS.CONTAINER2_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER2_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER2_TRK_POS= container position on truck
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_CRANE= As per the spec.

- TRANSFER_POINT_STATUS.TRK_CHASSIS_LENGTH_OA_CM= As per the spec.
3. ECN4 issues a new transport order to execute the first move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
 4. Refer to steps 2 to 7 from Transport Order Life Cycle.
 5. After the ASC lifts the container, in this case CONTAINER2 was lifted, the truck has one remaining receipt. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER2_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER2_ISO= NULL
 - TRANSFER_POINT_STATUS.CONTAINER2_TRK_POS= NULL
 6. Refer to steps 8 to 13 from Transport Order Life Cycle.
 7. ECN4 issues a new transport order to execute the second move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
 8. Refer to steps 2 to 7 from Transport Order Life Cycle.
 9. After the ASC lifts the container there are no transactions remaining. The TOS assumes the truck departs and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= NULL
 10. Refer to steps 8 to 13 from Transport Order Life Cycle.

Landside OTR Deliver Import

1. Truck arrives at the LSTZ and notifies arrival.

2. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= OTR
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_LENGTH_OA_CM= As per the spec.
3. ECN4 issues a new transport order to execute the move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
4. Refer to steps 2 to 11 from Transport Order Life Cycle
5. After the ASC sets the container, there are no remaining transactions. The TOS assumes the trucks departs and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= container position on truck

Landside OTR Deliver 2 Imports

1. Truck arrives at the LSTZ and notifies arrival.
2. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= OTR
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_CRANE= As per the spec.

- TRANSFER_POINT_STATUS.TRK_CHASSIS_LENGTH_OA_CM= As per the spec.
3. ECN4 issues a new transport order to execute the first move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
 4. Refer to steps 2 to 11 from Transport Order Life Cycle
 5. After the ASC completes the first container onto the truck there is one remaining delivery. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= container position on truck
 6. ECN4 issues a new transport order to execute the second move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
 7. Refer to steps 2 to 11 from Transport Order Life Cycle
 8. After the ASC sets the container there are no remaining transactions. The TOS assumes the truck departs and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER2_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER2_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER2_TRK_POS= container position on truck

Landside OTR Receive Export & Deliver Import

1. Truck arrives at the LSTZ and notifies arrival.
2. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4

- TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= OTR
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= receival container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= receival container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= receival container position on truck
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_LENGTH_OA_CM= As per the spec.
3. ECN4 issues a new transport order to execute the receival move. The record includes:
- ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
4. Refer to steps 2 to 7 from Transport Order Life Cycle.
5. After the ASC lifts the receival ECN4 updates transfer point status table:
- TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= NULL
6. Refer to steps 8 to 13 from Transport Order Life Cycle.
7. ECN4 issues a new transport order to execute the delivery move. The record includes:
- ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
8. Refer to steps 2 to 11 from Transport Order Life Cycle.
9. After the ASC sets the container, there are no remaining transactions. The TOS assumes the trucks departs and ECN4 updates transfer point status table:
- TRANSFER_POINT_STATUS.STATUS= 0

- TRANSFER_POINT_STATUS.UPDATED= current time stamp
- TRANSFER_POINT_STATUS.CHE_ID= truck license
- TRANSFER_POINT_STATUS.CONTAINER1_ID= delivery container number
- TRANSFER_POINT_STATUS.CONTAINER1_ISO= delivery container ISO code
- TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= delivery container position on truck

Landside OTR Receive Export & Deliver Import w/ Remain on Truck Container

1. Truck arrives at the LSTZ and notifies arrival.
2. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= OTR
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= receival container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= receival container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= receival container position on truck
 - TRANSFER_POINT_STATUS.CONTAINER2_ID= remain on truck container number
 - TRANSFER_POINT_STATUS.CONTAINER2_ISO= remain on truck container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER2_TRK_POS= remain on truck container position on truck
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_1_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_TEU= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_2_CRANE= As per the spec.
 - TRANSFER_POINT_STATUS.TRK_CHASSIS_LENGTH_OA_CM= As per the spec.
3. ECN4 issues a new transport order to execute the receival move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
4. Refer to steps 2 to 7 from Transport Order Life Cycle.
5. After the ASC lifts the receival ECN4 updates transfer point status table.
 - TRANSFER_POINT_STATUS.STATUS= 4
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL

- TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= NULL
6. Refer to steps 8 to 13 from Transport Order Life Cycle.
 7. ECN4 issues a new transport order to execute the delivery move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
 8. Refer to steps 2 to 11 from Transport Order Life Cycle.
 9. After the ASC sets the container, there are no remaining transactions. The TOS assumes the trucks departs and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.CHE_ID= truck license
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= delivery container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= delivery container ISO code
 - TRANSFER_POINT_STATUS.CONTAINER1_TRK_POS= delivery container position on truck

Landside UTR Inbound Move (Wheeled Area to ASC)

1. The planned container resides on a cassette and both are already married. A UTR is dispatched to fetch cassette and container.
2. The UTR arrives at the LSTZ, parks in a lane and decouples from the cassette. Container and cassette reside at the LSTZ. UTR is decoupled and IDLE.
3. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 3
 - TRANSFER_POINT_STATUS.UPDATED= current timestamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= UTR
 - TRANSFER_POINT_STATUS.CHE_ID= TT ID
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
 - TRANSFER_POINT_STATUS.CHASSIS1_ID= cassette ID
 - TRANSFER_POINT_STATUS.CHASSIS1_LENGTH_CM = cassette length
 - TRANSFER_POINT_STATUS.CHASSIS1_HEIGHT_CM = cassette height
4. ECN4 issues a new transport order to execute the move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order

- ASC_TRANSPORT_ORDER.CREATED= current timestamp
 - ASC_TRANSPORT_ORDER.UPDATED= current timestamp
5. Refer to steps 2 to 7 from Transport Order Life Cycle.
 6. After the ASC lifts the container, the cassette remains in the LSTZ and ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 2
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.UPDATED= current timestamp
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= NULL
 - TRANSFER_POINT_STATUS.CONTAINER1_ISO= NULL
 7. Refer to steps 8 to 13 from Transport Order Life Cycle.

Landside UTR Outbound Move (ASC to Wheeled Area)

1. A UTR is dispatched to fetch an available cassette.
2. The UTR carries the cassette to the LSTZ, decouples and leaves. Cassette resides at the TP, UTR becomes IDLE and available for another job.
3. ECN4 updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 2
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= UTR
 - TRANSFER_POINT_STATUS.CHE_ID= TT ID
 - TRANSFER_POINT_STATUS.CHASSIS1_ID= cassette ID
 - TRANSFER_POINT_STATUS.CHASSIS1_LENGTH_CM = cassette length
 - TRANSFER_POINT_STATUS.CHASSIS1_HEIGHT_CM = cassette height
 - ECN4 issues a new transport order to execute the move. The record includes:
 - ASC_TRANSPORT_ORDER.TO_GKEY= Unique database generated sequencing field
 - ASC_TRANSPORT_ORDER.CQ_GKEY= NULL
 - ASC_TRANSPORT_ORDER.TO_STATUS= ENTERED
 - ASC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order
 - ASC_TRANSPORT_ORDER.CREATED= current time stamp
 - ASC_TRANSPORT_ORDER.UPDATED= current time stamp
4. Refer to steps 2 to 11 from Transport Order Life Cycle.
5. After the ASC places the container on the cassette, ECN4 marries container and cassette and updates transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 3
 - TRANSFER_POINT_STATUS.CHE_ID= ASC ID
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp
 - TRANSFER_POINT_STATUS.VEHICLE_TYPE= UTR
 - TRANSFER_POINT_STATUS.CONTAINER1_ID= container number

- TRANSFER_POINT_STATUS.CONTAINER1_ISO= container ISO code
6. A UTR is dispatched to fetch cassette and container from LSTZ.
 7. The UTR arrives at the transfer point and pulls the cassette and container. The TP is now unoccupied and ECN4 updates the transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.CHE_ID= TT ID
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp

Pull Bare Cassette from LSTZ

1. A UTR is dispatched to fetch bare cassette from LSTZ.
2. The UTR arrives at the transfer point and pulls the cassette. The TP is now unoccupied and ECN4 updates the transfer point status table:
 - TRANSFER_POINT_STATUS.STATUS= 0
 - TRANSFER_POINT_STATUS.CHE_ID= TT ID
 - TRANSFER_POINT_STATUS.UPDATED= current time stamp

11.2.3 Area Status Update

11.2.3.1 ACCS Area Status Table (AREA_STATUS)

This table is used to show the availability of the reefer area or transfer area (LSTP or WSTP). The ACCS writes this table. The table will be populated only with entries for transfer points and yard stacks where availability is managed. For example, the table may only contain entries for cells in the reefer bays within the ASC block if these are the only locations that are temporarily blocked by the ACCS. Entries are cell-specific so if a range of cells, for example an entire row in the block, or a lane in a transfer point, change status then all cells corresponding to that row or lane must be updated.

N4 will set stack status to "Temp Blocked" for the unavailable states and clear stack status for the available states. The differentiation between the available and unavailable states recorded in the table is for reporting and monitoring purposes.

Any re-conciliation between the N4 yard model and entries for transfer points and yard stacks in table is handled by ACCS provider. AREA_STATUS entries not found in the N4 yard model would result in error messages in ECN4 logs.

Creation an use of this table is optional. Whether ECN4 reads the AREA_STATUS table is controlled via the AREA_STATUS_ENABLED setting.

Column Name	Data Type	Description	Example	Default
BLOCK	String (6)	ASC Module Name	"02"	NULL
BAY_TYPE	String (10)	[RMG, WATERSIDE, LANDSIDE, SPREADER]. For this interface only RMG, WATERSIDE, LANDSIDE are possibilities.	"WATERSIDE"	NULL

Column Name	Data Type	Description	Example	Default
BAY	String (3)	Row name, formatted as defined in SPARCS yard model.	"002" "02"	NULL
CELL	String (3)	Column name. In TZ this is the lane name. Formatted as defined in SPARCS yard model.	"001" "A"	NULL
AVAILABLE	Number (I2)	0 - Unavailable 1 - Available 2 - Unavailable and SC occupied (only applies to WS transfer zone) 3 - Available and SC occupied (only applies to WS transfer zone) 4 - Available and ASC has claimed or working in lane (applies to both WS and LS transfer zones)	0 1 2 3 4	NULL
CREATED	Date/Time	Record created time	25-MAY-2005 11:32:45	current time stamp
UPDATED	Date/Time	Record updated time	25-MAY-2005 11:32:45	NULL

11.2.3.2 User Interface Considerations

There is no user interface into the database tables within the TOS. Typically a UI is supplied by the database itself.

11.3 Non-Functional Specification

In This Section

Technical Requirements. 335

Performance and Scalability. 335

11.3.1 Technical Requirements

An external database is required for this Interface. That database must be either of the following:

- Microsoft SQLServer

Please review the N4 Application Environment Guide to determine the currently supported versions of the above databases.

11.3.2 Performance and Scalability

There are a few tiers of performance for consideration. The first tier is the sampling rate of the database itself. The second tier is the expected updates that result from the sampling.

The ASC_COMMAND_QUEUE and ASC_TRANSPORT_ORDERS table will have a sampling rate of 3 to 5 seconds. For the both tables the updates will come about once every 60 to 240 seconds for each ASC.

The STATUS table will also have a sampling rate of 3 to 5 seconds. The STATUS table will have from 15 to 200 records where each record will be updated on an average of once every 5 to 10 seconds. The primary update triggers will be location information of the ASC.

The TRANSFER_POINT_STATUS table will also have a sampling rate of 3 to 5 seconds. The TRANSFER_POINT_STATUS table will have from 150 to 500 records where each record will be updated on an average of once every 60 to 240 seconds. The update triggers will be shuttle carrier drops/picks, ASC drops/picks, UTR pull/park and road truck arrival.

The AREA_STATUS table will also have a sampling rate of 3 to 5 seconds. The AREA_STATUS table will have from 200 to 1000 records where each record will be updated on an average of once every 5 to 10 seconds. The primary update triggers will be location information of the ASC.

In all scenarios the updating of work instruction, CHE state and stack status information in N4 should take less than 3 seconds after the database record has been queried.

Part 12

Automated Yard Crane Database Interface

In this Section:

Introduction.	337
Data Modeling and Design.	338
Default Workflows.	363
Purge Yard Crane database tables.	392

12.1 Introduction

The following sections specify the Automated Yard Crane Database Interface available with Navis N4 3.1 and later. This interface allows communication between N4 and the yard crane equipment control system (ECS) for cantilever automated rail mounted gantry cranes (C-ARMGs) and automated rubber tire gantry (A-RTG) cranes working with terminal tractors and road trucks. It is based on an earlier database interface implemented by ECN4. Deviations between the two interfaces are indicated throughout this specification by the Info boxes that list **values introduced in N4 3.1** and **values not supported in N4 3.1**.



The database described here is not part of the N4 system. It is an external database that is installed, configured, and maintained by the customer.



You can create the Automated Yard Crane Database in either Microsoft SQLServer or Oracle.



Please review the N4 Application Environment Guide to determine the currently supported versions of the above databases.

12.2 Data Modeling and Design

This database represents the third version of a database interface for Navis products to connect to semi-automated control systems. The first version is the AutoCheODBC Interface defined in SPARCS 3.6 in 2006 for ASC Control Systems to connect to the Radio Server. The second version is the ASC Automated Equipment Database Interface, first introduced in N4 2.3, that allows ASC Control Systems to connect to ECN4. This third version is an extension of the ECN4 interface, that allows existing ECN4 customers to use with minimal changes to their ECS interface. In the database table definitions given in this section, there is a **Version Introduced** column that will specify what version of N4 where interface changes have been made. When versions of 3.1+ are given, it is understood this is in the "third" version, the Automated Yard Crane Database Interface for ECS connecting to N4, and when the versions are 2.6-, it is understood this is the "second" version, the ASC Automated Equipment Database Interface in ECN4.

In This Section

TOS Crane Transport Order Table (YC_TRANSPORT_ORDERS).....	339
Crane ECS Command Queue Table (YC_COMMAND_QUEUE).....	348
Crane ECS Status Table Layout (STATUS).....	355
Crane ECS Area Status Table (AREA_STATUS).....	358
N4 and ECS Application Status (N4_APPLICATION_STATUS, ECS_APPLICATION_STATUS).....	360
Truck Status (TRUCK_STATUS).....	361

12.2.1 TOS Crane Transport Order Table (YC_TRANSPORT_ORDERS)

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
TO_GKEY	bigint	number(19,0)	TOS transport order sequence. Primary key, Indexed, Unique - Automatically generated by database.	2.3	100	DB GENERATED
CQ_GKEY	bigint	number(19,0)	ECS command order sequence Indexed field	2.3	100	NULL
TO_VERSION	int	number(10,0)	Synchronization point number. Incremented when N4 updates the transport order.	3.1	2	NULL
CQ_VERSION	int	number(10,0)	Synchronization point number. Incremented with CQ_VERSION from the YC_COMMAND_QU EUE last queried by N4. Can be NULL when the Transport Order is created.	3.1	2	NULL
TO_STATUS	varchar(20)	varchar(20)	Status of command order N4 sets the following status types: "ENTERED" "CANCEL" In addition, the following status types are synchronized from the Command Queue table for N4-initiated operations: "WORKING" "COMPLETE" Values introduced in N4 3.1 The following status types are synchronized from ECS initiated operations: "ABORT" "MAINTENANCE" Values not supported in N4 3.1 "ABORT_01" "ABORT_02" "ABORT_03" "ABORT_04"	2.3	"COMPLETE"	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
PROBLEM_DESCRIPTION	varchar(4000)	varchar(4000)	Free text description of a problem on the N4 side	3.1		NULL
CHE_ID	varchar(7)	varchar(7)	ID of crane being issued the transport order	2.3	"RMG03"	
PTA	datetime	timestamp	Planned time of arrival In N4 3.1: For ASCs only. For other CHE Kinds the value is set to NULL.	2.3	11-MAY-2014 12:00:05	NULL
CONTAINER_ID	varchar(12)	varchar(12)	ID of container to be moved Indexed field	2.3	"MSKU1234567"	NULL
CONTAINER_LENGTH_CM	numeric	number(18,2)	Length of container, in centimeters	2.3	610	NULL
CONTAINER_HEIGHT_CM	numeric	number(18,2)	Height of container, in centimeters	2.3	244	NULL
CONTAINER_ISO	varchar(4)	varchar(4)	ISO code of container	2.3	"42G1"	NULL
CONTAINER_DOOR_DIRECTION	varchar(10)	varchar(10)	Container door direction (for more information, see section 2.1.1) 'HIGH_ROW' 'LOW_ROW' 'UNKNOWN' Values not supported in N4 3.1 'FWD' 'AFT' 'NA' N4 3.1 sets value to "UNKNOWN", typically semi Auto sites do not track this information. Please check with PM for more information.	2.3	"FWD"	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
ORIG_BLOCK	varchar(6)	varchar(6)	Origin block of the container	2.3	"A02"	NULL
ORIG_BAY_TYPE	varchar(10)	varchar(10)	Origin bay type (for more information, see section 2.1.2) "SPREADER" "RMG" Values introduced in N4 3.1 "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL" Values not supported in N4 3.1 N4 3.1 does not support the following values: "WATERSIDE" "LANDSIDE"	2.3	"RMG"	NULL
ORIG_BAY	varchar(3)	varchar(3)	Origin row of container, formatted as defined in the N4 yard model	2.3	"20"	NULL
ORIG_CELL	varchar(3)	varchar(3)	Origin column of container, formatted as defined in the N4 yard model	2.3	"01"	NULL
ORIG_TIER	varchar(1)	varchar(1)	Origin tier of container	2.3	"3"	NULL
ORIG_FLOOR_HEIGHT_CM	numeric	number(18,2)	Origin highest point of stack above the ground N4 3.1 does not use this field. This value may be set to 0 by TOS.	2.3	777	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
ORIG_VEHICLE_TYPE	varchar(4)	varchar(4)	Origin type of vehicle, which can be any of the following: "OTR" - Over-the-road truck "UTR" - Internal truck, coupled operations Values introduced in N4 3.1 "CHS" - Internal truck, de-coupled operations "UNK" - Unknown vehicle type Values not supported in N4 3.1 "None"	2.3	"UTR"	NULL
ORIG_POS_TRUCK	varchar(2)	varchar(2)	Origin container location on truck chassis (for more information, see section 2.1.3)	2.3	"3"	NULL
ORIG_CENTER_TO_CHS_BACK_CM	numeric	number(18,2)	Distance, in centimeters, from the center of the container to the end of the chassis The gate system receives this value and transmits it to the ECS without any changes.	3.1	4500	NULL
PLANNED_DEST_BLOCK	varchar(6)	varchar(6)	Destination block of the container	2.3	"A02"	NULL
PLANNED_DEST_BAY_TYPE	varchar(10)	varchar(10)	Destination bay type (for more information, see section 2.1.2) "SPREADER" "RMG" Values introduced in N4 3.1 "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL" Values not supported in N4 3.1 "WATERSIDE" "LANDSIDE"	2.3	"RMG"	NULL
PLANNED_DEST_BAY	varchar(3)	varchar(3)	Planned Destination row of container, formatted as defined in the N4 yard model	2.3	"20"	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
PLANNED_DEST_CELL	varchar(3)	varchar(3)	Planned Destination column of container, formatted as defined in the N4 yard model	2.3	"01"	NULL
PLANNED_DEST_TIER	varchar(1)	varchar(1)	Planned Destination tier of container AutoRTG specific default value introduced in N4 3.8: For outbound AutoRTG moves - even if tier information is missing from the WI's planned location or appointment lane - the default value will be "1".	2.3	"3"	NULL
PLANNED_DEST_FLOOR_HEIGHT_CM	numeric	number(18,2)	Planned Destination highest point of stack above the ground N4 3.1 does not use this field. This value may be set to 0 by TOS.	2.3	777	NULL
PLANNED_DEST_VEHICLE_TYPE	varchar(4)	varchar(4)	Planned Destination type of vehicle "OTR" - Over-the-road truck "UTR" - Internal truck, coupled operations Values introduced in N4 3.1 "CHS" - Internal truck, de-coupled operations "UNK" - Unknown vehicle type Values not supported in N4 3.1 "NONE"	2.3	"OTR"	NULL
PLANNED_DEST_POS_TRUCK	varchar(2)	varchar(2)	Planned Destination container location on truck chassis (for more information, see section 2.1.1)	2.3	"3"	NULL
PLANNED_DEST_CENTER_TO_CHASSIS_BACK_CM	numeric	number(18,2)	Planned Distance, in centimeters, from the center of the container to the end of the chassis The gate system receives this value and transmits it to the ECS without any changes.	3.1	4500	NULL
CREATED	datetime	timestamp	Creation time of record	2.3	11-MAY-2014 12:00:05	Current time stamp

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
UPDATED	datetime	timestamp	Time of last update	2.3	11-MAY-2014 12:00:05	NULL
MOVE_TYPE	varchar(35)	varchar(35)	Type of order issued by N4 (used for reporting purposes only). "Unknown" "Transfer_In" "Transfer_Out" "Intrastack" "Split_Transfer_In" "Split_Transfer_Out" "Split_Intrastack" "Rehandle" "Preposition" "Housekeeping" Values introduced in N4 3.1 "ASC_ORDERED" "Reposition"	2.3	"Transfer_In"	NULL
WA_REF	bigint	number(19,0)	Work Assignment reference For internal use only	3.1	-102435566567	NULL
CONTAINER_WI_REF	bigint	number(19,0)	First container Work Instruction reference For internal use only	3.1	-102435566567	NULL
UFV_REF	bigint	number(19,0)	UFV reference For internal use only	3.1	-102435566567	NULL
CHASSIS1_ID	varchar(12)	varchar(12)	ID of terminal managed chassis or cassette, either bare or married to the container	3.1	"APLZ1234567"	NULL
CHASSIS1_LENGTH_CM	numeric	number(18,2)	Physical length of terminal managed chassis/cassette, in centimeters	3.1	762	NULL
CHASSIS1_HEIGHT_CM	numeric	number(18,2)	Physical height of terminal managed chassis/cassette, in centimeters	3.1	106	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
TRANSFER_ALLOWED	bit	char(1)	Indicates that for a given transport order, the anticipated truck has arrived and that the yard crane can either lift or drop the container onto/from the UTR/OTR 0 - Truck has not arrived 1 - Truck has arrived The TRANSFER_ALLOWED column is used only by sites that use passive RFID for Terminal Trucks (TT) and Cantilever (side-loading) ASCs. This is not applicable to the front-loading ASC sites or the sites that use the N4 Automation ASC Scheduler.	3.8	1	NULL
TRK_ID	varchar(12)	varchar(12)	Road truck identifier, typically a truck license	3.1	CA-2ABC1234	
TRK_CHASSIS_1_TEU	int	number(10,0)	TEU capacity of first road truck chassis directly behind the cab	3.1	2	NULL
TRK_CHASSIS_1_CRANE	bit	char(1)	Indicates whether side-loading cranes are present on road truck chassis, where: 0 – Side loading cranes not present on road truck chassis 1 – Side loading cranes present on road truck chassis	3.1	1	NULL
TRK_CHASSIS_2_TEU	int	number(10,0)	TEU capacity of second road truck chassis	3.1	2	NULL
TRK_CHASSIS_2_CRANE	bit	char(1)	Indicates whether side-loading cranes are present on road truck chassis, where: 0 – Side loading cranes not present on road truck chassis 1 – Side loading cranes present on road truck chassis	3.1	0	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
TRK_CHASSIS_LEN GTH_OA_CM	numeric	number(18,2)	Overall length of road truck chassis combination, in centimeters	3.1	2560	NULL
UTR_ID	varchar(7)	varchar(7)	The UTR identifier, for the internal truck from the WI associated with the transport order.	3.4	TT005	NULL

12.2.1.1 Container Door Direction

The container door direction is defined as follows:

- "HIGH_ROW": The container doors are facing in the direction of increasing row (bay) number.
- "LOW_ROW": The container doors are facing in the direction of decreasing row (bay) number.
- "UNKNOWN": The container door direction is unknown.

12.2.1.2 Bay Types

The origin and destination bay types, ORIG_BAY_TYPE, DEST_BAY_TYPE, LOC_BAY_TYPE and BAY_TYPE, are defined as follows:

- "SPREADER": The position is on the crane's spreader.
- "RMG": The position is within the yard block stacks where containers are stored.
- "HIGH_ROW": The position is in the transfer zone adjacent to the bay with the highest row (bay) number. Used with end-loading ASCs.
- "LOW_ROW": The position is in the transfer zone adjacent to the bay with the lowest row (bay) number. Used with end-loading ASCs.
- "HIGH_COL": The position is in the transfer zone adjacent to the stack with the highest column (stack) number. Used with cantilever RMGs and RTGs.
- "LOW_COL": The position is in the transfer zone adjacent to the stack with the lowest column (stack) number. Used with cantilever RMGs and RTGs.

12.2.1.3 Road Truck Chassis Positions

Containers destined for the landside transfer zone may arrive or depart on one or two road truck chassis.

The **ORIG_POS_TRUCK** and **DEST_POS_TRUCK** shall be used to indicate the "Position on Truck". The below describes the numbering scheme N4 uses to identify the position on truck. Only numbers 1-7 are applicable for this interface.

N4 supports positions 1 to 11 on up to three 40' chassis for a truck. The numbering scheme for the supported positions has the following rules:

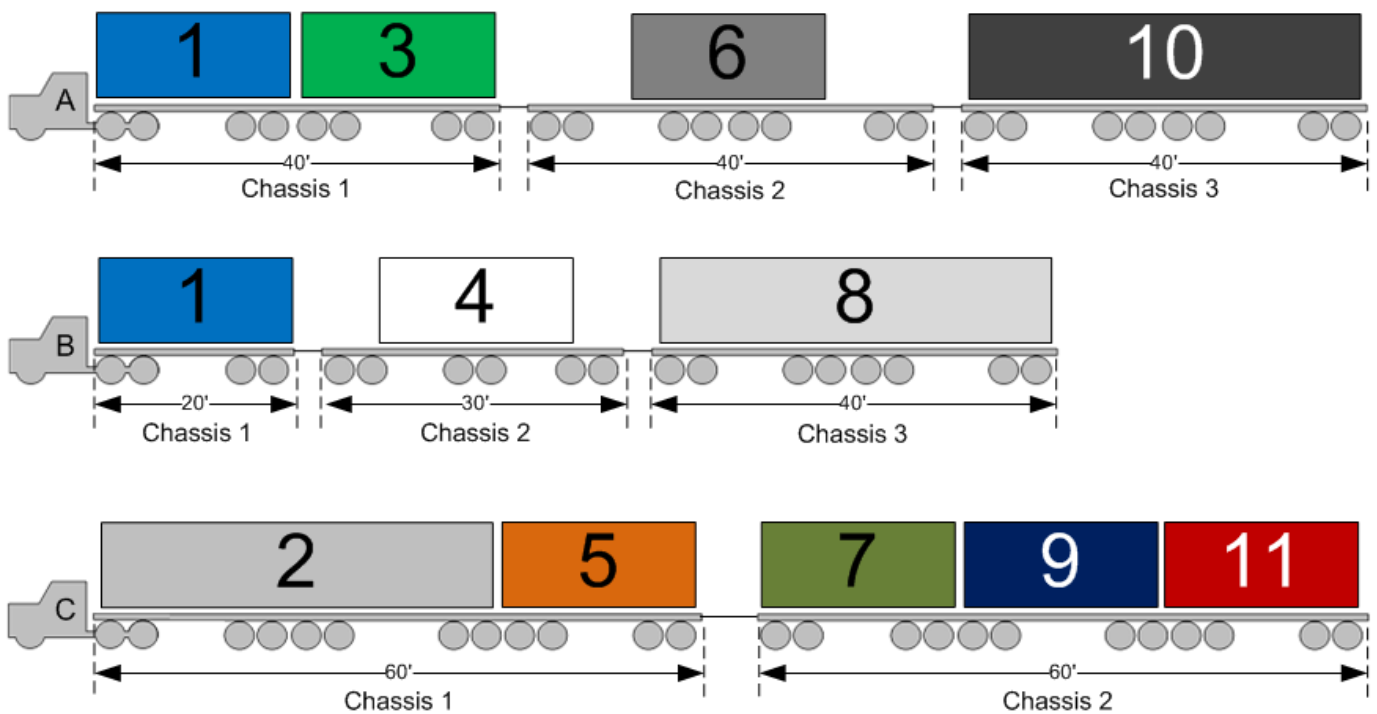
- 20' positions are odd numbers starting behind the truck cab at position 1 and counting up for each 20' of chassis, such as 3, 5, and so on.

- 40' positions are comprised of two 20' positions and are the even number that falls between the two 20' positions. For example, 40' position 2 takes up the same space as 20' positions 1 and 3.
- 40' containers are always in even positions. However, 20' containers can be placed in either odd positions or in an even position when centered on a 30' chassis or inside a 40' position.
- Because the first and last 20' position on each chassis is in an odd position, you must skip an even position when moving from one chassis to the next. For example, a truck with two 40' chassis has odd 20' positions 1 and 3 on the first chassis and 5 and 7 on the second. Therefore, the even 40' positions are position 2 on the first chassis and position 6 on the second, skipping position 4.
- A 30' chassis is treated similar to a 40' chassis in that it also has three possible positions. Because you can place a 20' container in a fore, centered, or aft position, it has the same three possible positions as a 40' chassis even though it cannot carry a 40' container.

For example, a truck with one 40' chassis has the following possible positions:

- A 20' container closest to the truck cab is in position 1.
- A 40' container or a 20' container centered on the chassis is in position 2.
- A 20' container farthest on the chassis is in position 3.

The following illustration displays three example truck configurations and the positions on the containers on each truck.



In the above illustration:

- Truck A has three 40' chassis. The first 20' container behind the truck cab is in position 1, and the second 20' container is in position 3. Position 4 is skipped when moving from the first to the second chassis. The 20' container on the second chassis is in position 6 because it is centered on the chassis. If this container was instead fore or aft on this chassis, it would be in position 5 or 7, respectively. Position 9 is skipped when moving from the second to the third chassis. Therefore, the 40' container on the third chassis is in position 10.
- Truck B has a 20' chassis followed by a 30' chassis and then a 40' chassis. The first 20' container behind the truck cab is in position 1. Position 2 is skipped when moving from the first to the second chassis. The 20' container on the

second chassis is in position 4 because it is centered on the chassis. If this container was instead fore or aft on the chassis, it would have been in position 3 or 5, respectively. Position 6 is skipped when moving from the second to the third chassis. Therefore, the 40' container on the third chassis is in position 8.

- Truck C has two 60' chassis. The first 40' container behind the truck cab is in position 2. The 20' container on the first chassis is in position 5 because the previous 40' container uses up positions 1 and 3. Position 6 is skipped when moving from the first to the second chassis. Therefore, the three 20' containers on the second chassis are in positions 7, 9, and 11. If the third chassis instead had two 20' containers that were centered in 40' positions, they would be in positions 8 and 10.

12.2.2 Crane ECS Command Queue Table (YC_COMMAND_QUEUE)

From the point of view of N4, this table has been broken into two sections: one that describes **Functional Columns** where N4 makes use of the information and another called **Informational Columns**, where N4 does not make use of the information, but users may find this information helpful when auditing their database.

12.2.2.1 Functional Columns

These columns are used by N4 and required by the interface:

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
CQ_GKEY	bigint	NUMBER(19,0)	ECS command order sequence Primary key, Indexed, Unique - automatically generated by database.	2.3	100	DB GENERATED
TO_GKEY	bigint	NUMBER(19,0)	TOS transport order sequence. Indexed field	2.3	100	NULL
CQ_VERSION	int	NUMBER(10,0)	Synchronization point number. Incremented when the ECS updates the Command Queue record.	3.1	2	1
TO_VERSION	int	NUMBER(10,0)	Synchronization point number. Incremented with TO_VERSION from YC_TRANSPORT_ORDER last queried by ECS. Can be NULL when the Command Queue record is created.	3.1	2	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
CQ_STATUS	varchar(20)	varchar(20)	Status of command order The following status types are set for N4- or ECS-initiated operations: "ENTERED" "WORKING" "COMPLETE" The following status are synchronized from transport orders for N4-initiated operations: "CANCEL" Values introduced in N4 3.1 The following status types are set for ECS-initiated operations: "ABORT" "MAINTENANCE" Values not supported in N4 3.1 "ABORT_01" "ABORT_02" "ABORT_03" "ABORT_04"	2.3	"COMPLETE"	NULL
PROBLEM_CODE	varchar(25)	varchar(25)	Describes the reason if the order was rejected, aborted, or delayed by the ECS; otherwise null. The standard abort code definitions are: "ABORT_01" "ABORT_02" "ABORT_03" "ABORT_04"	3.1	"ABORT_01"	NULL
CHE_ID	varchar(7)	varchar(7)	ID of CHE being issued the transport order	2.3	"RMG03"	NULL
CONTAINER_ID	varchar(12)	varchar(12)	ID of container to be moved Indexed field	2.3	MSKU1234567	NULL
CONTAINER_LENGTH_CM	numeric	NUMBER(18,2)	Length of container, in centimeters	2.3	610	NULL
CONTAINER_HEIGHT_CM	numeric	NUMBER(18,2)	Height of container, in centimeters	2.3	244	NULL
CONTAINER_ISO	varchar(4)	varchar(4)	ISO code of container	2.3	"42G1"	NULL
ORIG_BLOCK	varchar(6)	varchar(6)	Origin block of the container	2.3	"02"	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
ORIG_BAY_TYPE	varchar(10)	varchar(10)	Origin bay type (for more information, see section 2.1.2), which can be any of the following: "SPREADER" "RMG" Values introduced in N4 3.1 "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL" Values not supported in N4 3.1 "WATERSIDE" "LANDSIDE"	2.3	"RMG"	NULL
ORIG_BAY	varchar(3)	varchar(3)	Origin row of container, formatted as defined in the N4 yard model.	2.3	"20"	NULL
ORIG_CELL	varchar(3)	varchar(3)	Origin column of container, formatted as defined in the N4 yard model.	2.3	"01"	NULL
ORIG_TIER	varchar(1)	varchar(1)	Origin tier of container	2.3	"3"	NULL
ORIG_VEHICLE_TYPE	varchar(4)	varchar(4)	Origin type of vehicle, which can be any of the following: "OTR" - Over road truck "UTR" - Internal truck, coupled operations Values introduced in N4 3.1 "CHS" - Internal truck, de-coupled operations "UNK" - Unknown vehicle type Values not supported in N4 3.1 "NONE"	2.3	"UTR"	NULL
ORIG_POS_TRUCK	varchar(2)	varchar(2)	Origin container location on truck chassis (for more information, see section 2.1.3).	2.3	"3"	NULL
ORIG_CENTER_TO_CHS_BACK_CM	numeric	NUMBER(18,2)	Distance, in centimeters, from center of container to end of chassis The gate system receives this value, which is then echoed from N4 to the ECS without any changes.	3.1	4500	NULL
ACTUAL_DEST_BLOCK	varchar(6)	varchar(6)	Actual destination block of the container	2.3	"A02"	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
ACTUAL_DEST_BAY_TYPE	varchar(10)	varchar(10)	Actual destination bay type (for more information, see section 2.1.2), which can be any of the following: "SPREADER" "RMG" Values introduced in N4 3.1 "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL" Values not supported in N4 3.1 "WATERSIDE" "LANDSIDE"	2.3	"RMG"	NULL
ACTUAL_DEST_BAY	varchar(3)	varchar(3)	Actual destination row of container, formatted as defined in the N4 yard model	2.3	"20"	NULL
ACTUAL_DEST_CELL	varchar(3)	varchar(3)	Actual destination column of container, formatted as defined in the N4 yard model	2.3	"01"	NULL
ACTUAL_DEST_TIER	varchar(1)	varchar(1)	Actual destination tier of container	2.3	"3"	NULL
ACTUAL_DEST_VEHICLE_TYPE	varchar(4)	varchar(4)	Actual destination vehicle type, which can be any of the following "OTR" - Over-the-road truck "UTR" - Internal truck, coupled operations Values introduced in N4 3.1 "CHS" - Internal truck, de-coupled operations "UNK" - Unknown vehicle type Values not supported in N4 3.1 "NONE"	2.3	"OTR"	NULL
ACTUAL_DEST_POS_TRUCK	varchar(2)	varchar(2)	Actual destination location of the container on the truck chassis (for more information, see section 2.1.3)	2.3	"3"	NULL
ACTUAL_DEST_CENTER_TO_CHS_BACK_CM	numeric	NUMBER(18,2)	Actual distance, in centimeters, from center of container to end of chassis The gate system receives this value, which is then echoed from N4 to the ECS without any changes.	3.1	4500	NULL
CREATED	datetime	timestamp	Time the record was created	2.3	11-MAY-2014 12:00:05	current time stamp
UPDATED	datetime	timestamp	Time of last update	2.3	11-MAY-2014 12:00:05	NULL

Column Name	SQL Server Data Type	Oracle Data Types	Description	Version Introduced	Example	Default
PICKUP_TIME	datetime	timestamp	Date and time of container pickup	2.3	11-MAY-2014 12:00:05	NULL
SETDOWN_TIME	datetime	timestamp	Date and time of container setdown	3.1	11-MAY-2014 12:00:05	NULL
TRANSFER_POINT_ENTRY_TIME	datetime	timestamp	Date and time when the CHE entered the transfer point area	3.1	11-MAY-2014 12:00:05	NULL
TRANSFER_POINT_EXIT_TIME	datetime	timestamp	Date and time when the CHE exited the transfer point area	3.1	11-MAY-2014 12:00:05	NULL
MEASURED_WEIGHT_KG	varchar(6)	varchar(6)	Measured weight unit, in kg	2.3	"75000"	NULL
MAINTENANCE_CODE	varchar(25)	VARCHAR(100)	The standard maintenance code definitions, which can be any of the following: SERVICE PARKING GENERAL N4 3.1 does not use this field.	3.1	PARKING	NULL
UTR_ID	varchar(7)	VARCHAR(7)	The UTR identifier, for the internal truck from the WI associated with the command order.	3.4	TT005	NULL

12.2.2.2 Informational Columns

The following columns are informational or only used by the ECS and are not required by the interface. Other columns can be added as well, though they will not be queried or otherwise processed by N4.

Column Name	SQL Server Data Type	Description	Version Introduced	Example	Default
ORIG_FLOOR_HEIGHT_CM	numeric	Origin highest point of stack above the ground N4 3.1 does not use this field. This value may be set to 0 by TOS.	2.3	777	NULL
ACTUAL_DEST_FLOOR_HEIGHT_CM	numeric	Highest point of stack above the ground of the actual destination N4 3.1 does not use this field. This value may be set to 0 by TOS.	2.3	777	NULL

Column Name	SQL Server Data Type	Description	Version Introduced	Example	Default
CONTAINER_DOOR_DIRECTION	varchar(10)	Container door direction (for more information, see section 2.1.1), which can be any of the following: 'HIGH_ROW' 'LOW_ROW' 'UNKNOWN' Values not supported in N4 3.1 'FWD' 'AFT' 'NA'	2.3	"FWD"	NULL
PROBLEM_DESCRIPTION	varchar(4000)	Free text description for detailed description of a problem on the ECS side	3.1		NULL
ATA	datetime	Actual time of arrival	2.3	11-MAY-2014 12:00:05	NULL
PRIORITY	varchar(1)	Command execution priority. For ECS internal use only	1 - Lowest with increasing priority with ascending values	1	NULL
OPERATOR1	varchar(15)	First operator name For ECS internal use only	2.3	"SMITH"	NULL
OPERATOR2	varchar(15)	Second operator name For ECS internal use only	2.3	"JONES"	NULL
SUPERUSER	bit	Indicates super user operated on command For ECS internal use only	2.3	1 - yes 0 - no	NULL
WAITFOROPR	int	Time waiting for operator (sec) For ECS internal use only	2.3	10	NULL

Column Name	SQL Server Data Type	Description	Version Introduced	Example	Default
WAITFORWS	int	Time waiting for waterside to be clear (sec) For ECS internal use only	2.3	11	NULL
WAITFORCRN	int	Time waiting for other crane to clear the area(sec) For ECS internal use only	2.3	12	NULL
CHASSIS1_ID	varchar(12)	ID of terminal managed chassis or cassette, either bare or married to the container	2.5	"APLZ1234567"	NULL
CHASSIS1_LENGTH_CM	numeric	Physical length of terminal-managed chassis/cassette, in centimeters	2.5	762	NULL
CHASSIS1_HEIGHT_CM	numeric	Physical height of terminal-managed chassis/cassette, in centimeters	2.5	106	NULL
TRK_ID	varchar(12)	Road truck identifier, typically a truck license	3.1	CA-2ABC1234	
TRK_CHASSIS_1_TEU	int	TEU capacity of first road truck chassis directly behind the cab	3.1	2	NULL
TRK_CHASSIS_1_CRANE	bit	0 – Side loading cranes not present on road truck chassis 1 – Side loading cranes present on road truck chassis	3.1	1	NULL
TRK_CHASSIS_2_TEU	int	TEU capacity of second road truck chassis	3.1	2	NULL

Column Name	SQL Server Data Type	Description	Version Introduced	Example	Default
TRK_CHASSIS_2_CRANE	bit	Indicates whether side-loading cranes are present on road truck chassis, where: 0 – Side loading cranes not present on road truck chassis 1 – Side loading cranes present on road truck chassis	3.1	0	NULL
TRK_CHASSIS_LENGTH_OA_CM	numeric	Overall length of road truck chassis combination, in centimeters	3.1	2560	NULL
MAINTENANCE_CODE_DESCRIPTION	varchar(100)	Maintenance code description (for reporting purposes only)  N4 3.1 does not use this field.	3.1	The crane is currently being parked	NULL

12.2.3 Crane ECS Status Table Layout (STATUS)

N4 requires that certain status information about the crane be shared by the crane ECS in order to:

- Appropriately update the work instruction in N4, as described in the primary interaction and abort interactions.
- Update and display the state of the crane in N4 and to the user.

The ECS maintains crane status information in the STATUS table. The advantage of this approach is that it does not require direct communication between the ECS application and N4; N4 simply reads from this status table to determine the current status of each CHE within the ECS application. Each automated yard crane in the terminal is allocated ONE row in the STATUS table. The ECS is responsible for updating the status information in the table periodically, or upon completion of an event. The ECS software always uses SQL UPDATE commands (*never* INSERT commands). Rows are pre-allocated by an offline process to insure the integrity of the table.

About ONLINE_STATUS values

For backward compatibility, this interface supports ONLINE_STATUS values that were previously used by other vendors in the legacy version of this interface. Going forward, the expectation is that **only** AUTO, MANUAL, and OFFLINE will be used. One vendor uses YARD for uncommanded moves, but MANUAL is sufficient. The status PENDING indicates that the crane is pending unavailable and should not be scheduled for further work.

The TOS CheStatus Mapping table shows how these status values map to the CHE Status in N4. For N4, only status values of WORKING, OUTOFSERVICE, and UNAVAILABLE are relevant. N4 only schedules and dispatches moves

automatically when the status is WORKING. When the remote operator is in control, the mode should be MANUAL (not AUTO), and the crane is not considered available for auto dispatching. The sequencer may however still sequence the moves for cranes that are in Manual Mode.

Column Name	SQL Server Data Type	Oracle	Description	Version Introduced	Example	Default
CHE_ID	varchar(7)	varchar(7)	ID of Crane being issued the transport order Primary key Indexed field	2.3	RMG03	NOT NULL
ONLINE_STATUS	varchar(11)	varchar(11)	Crane's online status, which can be any of the following: "AUTO" - Auto Mode "MANUAL" - Manual Mode "PENDING" - Scheduled to go Offline "YARD" - Waiting to reconcile un-commanded yard moves "OFFLINE" - Offline Values introduced in N4 3.1 "MAINTENANCE" - Maintenance Mode	2.3	"AUTO"	NULL
WORK_STATUS	varchar(20)	varchar(20)	Crane's work status, which can be any of the following: "WORKING" - Crane is working on a job "IDLE" - Crane is in idle state and available for dispatch "FAULT" - Crane may have a hardware fault and is not available for dispatch	2.3	"WORKING"	NULL
TO_GKEY	int	NUMBER(10,0)	Transport order currently being executed	2.3	100	NULL
CQ_GKEY	int	NUMBER(10,0)	Command order currently being executed	2.3	100	NULL
LOC_BLOCK	varchar(6)	varchar(6)	Current block of the crane	2.3	"02"	NULL

Column Name	SQL Server Data Type	Oracle	Description	Version Introduced	Example	Default
LOC_BAY_TYPE	varchar(10)	varchar(10)	Current bay type location of crane (for more information, see section 1.2), which can be any of the following: "RMG" Values introduced in N4 3.1 "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL" Values not supported in N4 3.1 "WATERSIDE" "LANDSIDE"	2.3	"RMG"	NULL
LOC_BAY	varchar(3)	varchar(3)	Crane's current bay	2.3	"20"	NULL
UPDATED	datetime	timestamp	Time of last update	2.3	11-MAY-2014 12:00:05	NULL
CONTAINER_1	varchar(12)	varchar(12)	Current container 1 Set to NULL when WORK_STATUS is IDLE. Indexed field Note: N4 3.1 does not use this field.	2.3	"MSKU1234567"	NULL
SEQUENCE_STATUSES	varchar(50)	varchar(50)	Additional details of action being carried out by crane, which can be any of the following: "WAITING_FOR_INSTRUCTION" "WAITING_CRANE_TO_CLEAR_AREA" "TRAVELING_TO_ORIGIN" "LOWERING_TO_ORIGIN" "LANDED" "HOISTING_CONTAINER" "TRAVELING_TO_DESTINATION" "LOWERING_TO_DESTINATION" "WAITING_TO_ENTER_TRANSFER_AREA" "MOVING_AWAY_FROM_OTHER_CRANE" "HOISTING_SPREADER" "MANUAL"	2.3	"WAITING_FOR_INSTRUCTION"	NULL

Column Name	SQL Server Data Type	Oracle	Description	Version Introduced	Example	Default
ACTIVE	bit	char(1)	Indicates whether this CHE is active in the ECS model, where: 0 - not active 1 - active Note: N4 3.1 does not use this field.	3.1	1	NULL

Step	Action
1	ECS posts update to the STATUS table
2	N4 periodically queries the STATUS table and retrieves the current status of the yard crane.
3	N4 updates the yard crane's working status, online status, job step state. and last known position.

The following table shows the mapping of STATUS.ONLINE_STATUS and STATUS.WORK_STATUS against Che.CheStatus in TOS.

STATUS.ONLINE_STATU S	STATUS.WORK_STATUS	TOS Che.CheStatus	COMMENT
AUTO	WORKING	WORKING	
AUTO	IDLE	WORKING	
AUTO	FAULT	OUTOFSERVICE	Ignores ONLINE_STATUS
MANUAL	WORKING	MANUAL	
MANUAL	IDLE	MANUAL	
MANUAL	FAULT	OUTOFSERVICE	Ignores ONLINE_STATUS
PENDING	WORKING	UNAVAILABLE	
PENDING	IDLE	UNAVAILABLE	
PENDING	FAULT	OUTOFSERVICE	Ignores ONLINE_STATUS
YARD	WORKING	UNAVAILABLE	
YARD	IDLE	UNAVAILABLE	
YARD	FAULT	OUTOFSERVICE	Ignores ONLINE_STATUS
MAINTENANCE	WORKING	UNAVAILABLE	
MAINTENANCE	IDLE	UNAVAILABLE	
MAINTENANCE	FAULT	OUTOFSERVICE	Ignores ONLINE_STATUS
OFF	WORKING	UNAVAILABLE	
OFF	IDLE	UNAVAILABLE	
OFF	FAULT	OUTOFSERVICE	Ignores ONLINE_STATUS

12.2.4 Crane ECS Area Status Table (AREA_STATUS)

The ECS writes to this table to show the availability of the reefer area or transfer area. The table is populated only with

entries for transfer points and yard stacks where availability is managed. For example, the table may only contain entries for cells in the reefer bays within the yard crane block if these are the only locations that are temporarily blocked by the ECS. Entries are cell-specific: if a range of cells, for example an entire row in the block or a lane in a transfer point, change status, then all cells corresponding to that row or lane must be updated.

N4 sets the stack status to "Men Working" for 0-Available, ""Temp Blocked" for the states ('2', or '5') and clears the stack status for the available states ('1', '3', '4'). The differentiation between the available and unavailable states recorded in the table is for reporting and monitoring purposes.

N4 is not responsible for ensuring operational safety. It merely uses the status provided by the ECS to schedule and optimize moves. The recommended operational procedures for sending men to work in an automated stack are to set the necessary safety restrictions in the ECS. Do not set these as restrictions in N4 because it is vital that only one system is responsible for enforcing operational safety, and that must be the ECS.

Any re-conciliation between the N4 yard model and entries for transfer points and yard stacks in the table is handled by the ECS provider. AREA_STATUS entries not found in the N4 yard model result in error messages in N4 logs and in integration errors.

Column Name	SQL Server Data Type	Oracle	Description	Version Introduced	Example	Default
BLOCK	varchar(6)	varchar(6)	Module Block Name	2.5	"02"	NULL
BAY_TYPE	varchar(10)	varchar(10)	"RMG" - Bay location is in module Values introduced in N4 3.1 "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL"	2.5	"RMG"	NULL
BAY	varchar(3)	varchar(3)	Row name	2.5	"02"	NULL
CELL	varchar(3)	varchar(3)	Column name In transfer zones, this is a lane name.	2.5	"01"	NULL

Column Name	SQL Server Data Type	Oracle	Description	Version Introduced	Example	Default
STATUS	int	NUMBER(10)	The area status, which can be any of the following: 0 - Unavailable 1 - Available 2 - Unavailable and CHE occupied 3 - Available and CHE occupied 4 - Available and RMG has claimed or working in lane Values introduced in N4 3.1 5 - Available but occupied by reefer mechanic	2.5	1	NULL
CREATED	datetime	timestamp	Time the record was created	2.5	12-MAY-2014 12:00:05	current time stamp
UPDATED	datetime	timestamp	Time the record was updated	2.5	12-MAY-2014 12:00:05	NULL
ACTIVE	bit	char(1)	Indicates whether this location is still active in the ECS yard model.	3.1	1	NULL

12.2.5 N4 and ECS Application Status (N4_APPLICATION_STATUS, ECS_APPLICATION_STATUS)

Both N4 and the ECS publish their status in their respective tables: N4_APPLICATION_STATUS table for N4 and ECS_APPLICATION_STATUS table for the ECS. This is a keep-alive mechanism so that either application can determine whether the other application is up and responsive. The heartbeat update rate is configurable in the application that sends the update. The recommended values are between 15 seconds for production and 60 seconds for testing.

Column Name	SQL Server Data Type	Oracle	Description	Version Introduced	Example	Default
SYSTEM_ID	varchar(10)	varchar(10)	System name	3.1	"N4" "ECS"	NULL
START_TIME	datetime	timestamp	Time the application was started, to be updated on each application start	3.1	12-MAY-2014 12:00:05	NULL
REFERENCE_TIME	datetime	timestamp	To determine a possible difference in the actual time and the time of the system (such as with historical data sets)	3.1	12-MAY-2014 12:00:05	NULL
STATUS	varchar(10)	varchar(10)	The application status, which can be any of the following: STARTING - Initialization and bootstrap UP - Initialized and running SHUTDOWN - Graceful shutdown	3.1	UP	NULL
UPDATED	datetime	timestamp	Heartbeat update	3.1	12-MAY-2014 12:00:05	NULL

During application startup, if the last heartbeat received is more than the heartbeat duration configured and the status is not SHUTDOWN, this is taken as an indication for a non-graceful shutdown.

The ECS can define a similar action or decide to work in manual mode if it detects a stale heartbeat in N4_APPLICATION_STATUS.



The ECS_APPLICATION_STATUS table serves informational purposes only. N4 ignores this table. This may change in future releases.

12.2.6 Truck Status (TRUCK_STATUS)

This table is used to support the ability for the RTG Dispatcher to dispatch RTGs when a truck arrives in the TZ – as opposed to explicitly being at the TP or yard stack - typically for terminals that use radio frequency identification (RFID) readers for identifying trucks in cantilever lanes. This applies to both internal and external trucks.

Column Name	SQL Server Data Type	Description	Version Introduced	Example	Default
CHE_ID	varchar (7)	ID of the AYC involved in the move.	3.8	"RTG008"	NULL
CREATED	datetime	When the record was created.	3.8	12-MAY-2014 12:00:05	CURRENT_TIMESTAMP
LOC_BAY	varchar (3)	Row name.	3.8	"02"	NULL
LOC_BLOCK	varchar (6)	Module block name.	3.8	"02"	NULL
LOC_TZ	varchar (6)	Bay location is in module "HIGH_ROW" "LOW_ROW" "HIGH_COL" "LOW_COL"	3.8	"HIGH_ROW"	NULL
TRUCK_ID	varchar (12)	ID of the internal or external truck involved in the move.	3.8	"TT024"	NULL
UPDATED	datetime	When the record was last updated.	3.8	12-MAY-2014 12:00:05	NULL

12.3 Default Workflows

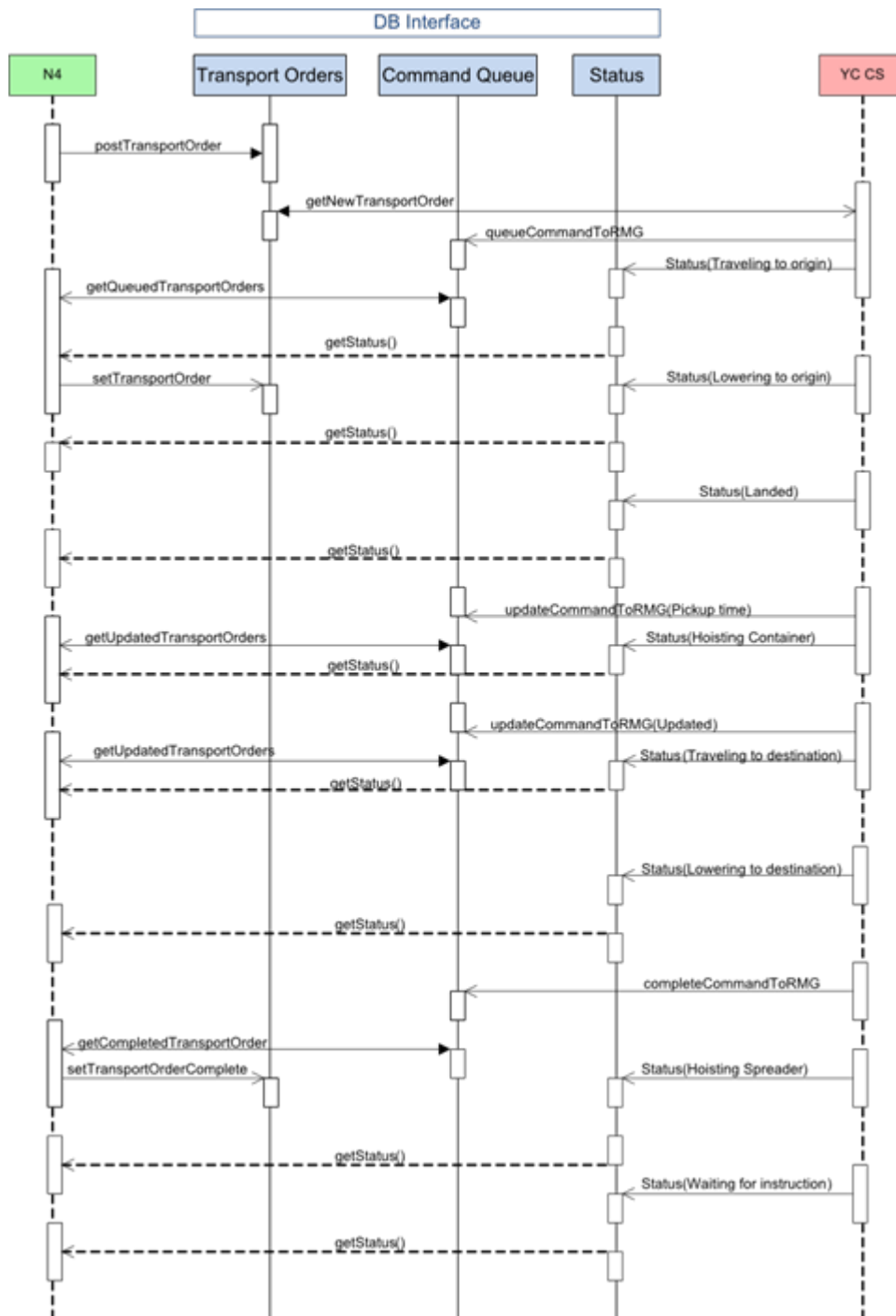
This section describes the default workflows available with N4. These workflows correspond to the workflows for the Automated Yard Cranes (ASCs, C-ARMGs, A-RTGs, etc). It is possible to customize these flows, for instance to handle a special abort code, with customized logic. For more information on Automated Yard Crane Database interaction workflows, see the *Navis N4 Automation Monitoring and Troubleshooting Guide*.

In This Section

- [Transport Order primary interaction.](#) 364
- [Abort Interactions.](#) 367
- [Dispatched Moves Completed by Remote Operator.](#) 385
- [Cancelations.](#) 386
- [Offline Operation of Crane and Inventory Synchronization.](#) 387
- [Crane Move Order.](#) 389

12.3.1 Transport Order primary interaction

The interaction diagram below shows how TOS and the ECS work together to dispatch and complete a full cycle transport order:



This use case elaborates on the interaction.

Step	Action
1	TOS issues a new transport order to the Transport Order table. The record includes: YC_TRANSPORT_ORDER.TO_GKEY= unique database generated sequencing field YC_TRANSPORT_ORDER.CQ_GKEY= NULL YC_TRANSPORT_ORDER.TO_STATUS= ENTERED YC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order YC_TRANSPORT_ORDER.CREATED= current time stamp YC_TRANSPORT_ORDER.UPDATED= current time stamp
2	The ECS queries the Transport Order table for new transport orders, validates the information, issues the command to the ECS, and issues a command to the Command Queue table. The Status table must be updated as well in this step. The Command Queue table and Status table are updated as below: YC_COMMAND_QUEUE.CQ_GKEY= unique database generated sequencing field YC_COMMAND_QUEUE.TO_GKEY= unique id of the transport order issued by TOS for the particular transport order. YC_COMMAND_QUEUE.CQ_STATUS= WORKING YC_COMMAND_QUEUE.CHE_ID= ID of ASC being issued the transport order YC_COMMAND_QUEUE.CREATED=current time stamp YC_COMMAND_QUEUE.UPDATED= current time stamp STATUS.TO_GKEY= transport order currently being executed STATUS.CQ_GKEY= command currently being executed STATUS.WORK_STATUS= WORKING STATUS.SEQUENCE_STATUS= TRAVELING TO ORIGIN STATUS.UPDATED= current time stamp
3	N4 polls the Command Queue table for records where the CQ_GKEY is NOT set in the N4 Transport Orders table and the command key CQ_GKEY is present in the Command Queue table. N4 updates the status of the CQ_GKEY field in its Transport Order table and the TO_STATUS to WORKING

Step	Action
4	Once the YC has arrived at the move origin, ECS updates the Status table: STATUS.SEQUENCE_STATUS= LOWERING TO ORIGIN STATUS.UPDATED= current time stamp
5	When the YC lands the spreader on the container the ECS updates the Status table STATUS.SEQUENCE_STATUS= LANDED STATUS.UPDATED= current time stamp
6	The YC locks the twistlocks and starts hoisting the container. The ECS updates the Command Queue table with the pickup time and Status table with the new sequence: YC_COMMAND_QUEUE.CQ_STATUS= WORKING YC_COMMAND_QUEUE.PICKUP_TIME= current time stamp YC_COMMAND_QUEUE.UPDATED= current time stamp STATUS.SEQUENCE_STATUS= HOISTING CONTAINER STATUS.UPDATED=current time stamp N4 updates the move stage and job step state to CARRY_UNDERWAY
7	The crane completes hoisting and starts traveling to the destination. The ECS updates the Command Queue and Status tables with the new sequence: YC_COMMAND_QUEUE.CQ_STATUS= WORKING YC_COMMAND_QUEUE.UPDATED= current time stamp STATUS.SEQUENCE_STATUS= TRAVELING TO DESTINATION STATUS.UPDATED= current time stamp
8	Once the YC has arrived at the move destination, ECS updates the Status table: STATUS.SEQUENCE_STATUS= LOWERING TO DESTINATION STATUS.UPDATED= current time stamp
9	Once the YC has arrived at the move destination, ECS updates the Status table: STATUS.SEQUENCE_STATUS= COMPLETE STATUS.UPDATED= current time stamp This update would include the final destination and other completion attributes - all in ONE I/O.
10	N4 polls the Command Queue table using CQ_GKEY as a link. It requests all rows where CQ_STATUS = "COMPLETE" in the Command Queue Table and NOT set in the N4 Transport Order Table.

Step	Action
11	N4 marks the transport order as "COMPLETE" in the N4 Transport Order Table under TO_STATUS .
12	When the YC is lifting the spreader, the Status table must be updated with the corresponding sequence: STATUS.SEQUENCE_STATUS= HOISTING SPREADER STATUS.UPDATED= current time stamp
13	The move has been completed and ECS must initialize the YC status. This tells N4 that the YC is IDLE and ready to be dispatched to execute a new transport order. STATUS.TO_GKEY= 0 STATUS.CQ_GKEY= 0 STATUS.WORK_STATUS= IDLE

12.3.2 Abort Interactions

Abort scenarios occur when a crane cannot complete a dispatched order. The N4 TOS supports 4 abort scenarios "out of the box", but additional abort scenarios can be supported as needed through system extension.

In the event of an abort, the CQ_STATUS of the crane command will be set to "ABORT" and PROBLEM_CODE is set to "ABORT_xx" where "xx" is the abort error code (see summary table below). Based upon the abort error code, N4 will cancel any subsequently issued transport orders and re-issue a transport order as appropriate, e.g. Abort on set down - TOS picks a new location, Abort on pick-up - TOS dispatches a new container move, etc.

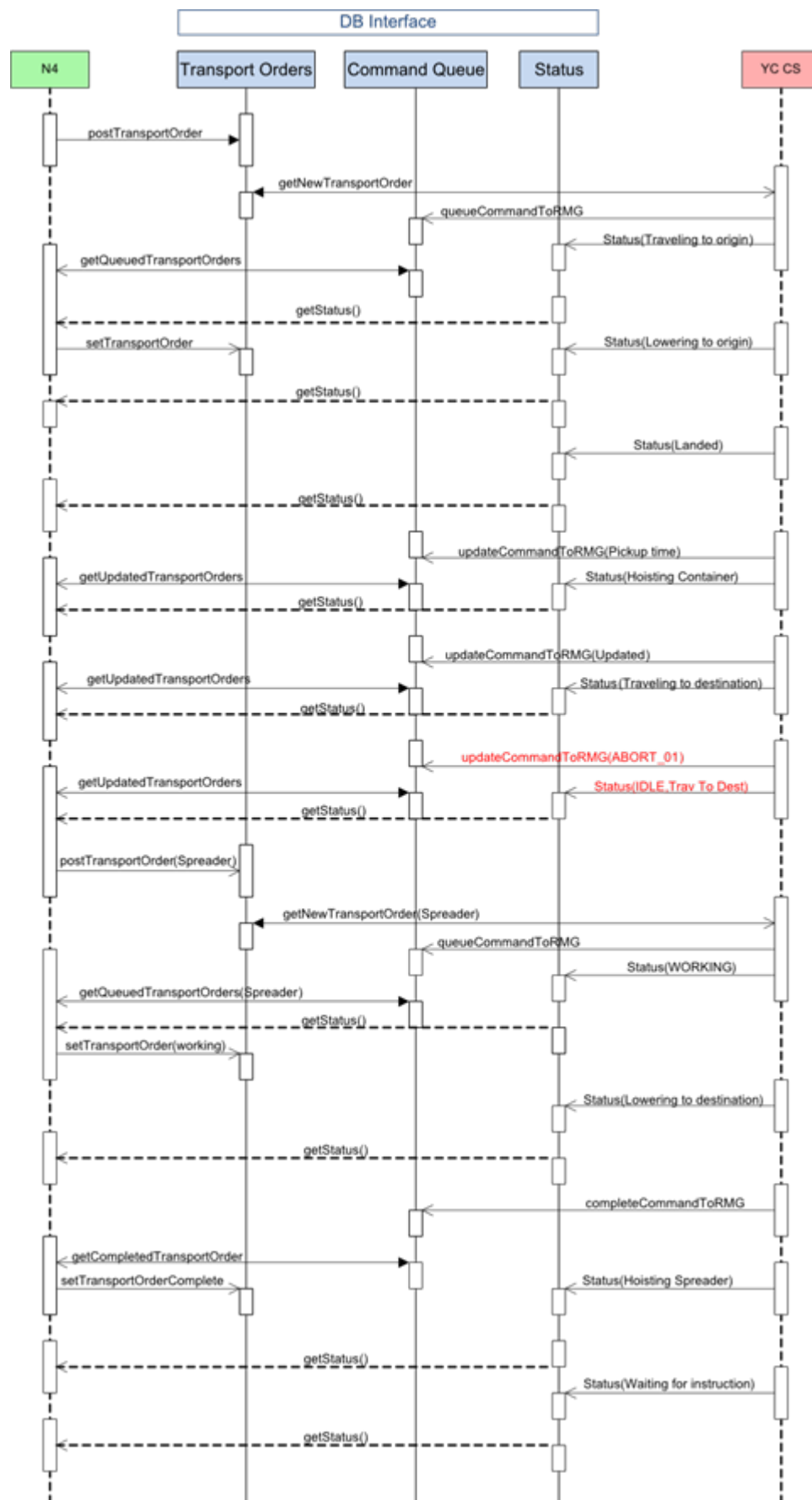
Abort code	Description	Action
ABORT_01	Abort on set down	N4 sets the TO_STATUS = "ABORT", PROBLEM_CODE = "ABORT_01" N4 issues a new transport order to place the container in a new location. New transport order has an ORIG_BAY_TYPE = "SPREADER".
ABORT_02	Abort on pick-up	N4 sets the TO_STATUS = "ABORT", PROBLEM_CODE = "ABORT_02" N4 resets the work instruction move stage to "planned" and "bypass" the dispatch stage. N4 temp blocks the origin position. N4 issues a new transport order to move a different container.

Abort code	Description	Action
ABORT_03	Abort due to Door Direction	N4 sets the TO_STATUS = "ABORT" , PROBLEM_CODE = "ABORT_03" Note: In release 3.1, Navis has removed this to create additional instructions to change the door directions, opting instead for handling ABORT_03 as an ABORT_02 (abort on pickup) if the container is not yet lifted, or as an ABORT_01 (abort on set down) if the container is being carried by the automated yard crane.
ABORT_04	Abort due to invalid command to ASC	N4 sets the TO_STATUS = "ABORT" , PROBLEM_CODE = "ABORT_04" N4 treats this as an ABORT_01 if the container was already lifted or as an ABORT_02 if the container was not yet lifted.
ABORT_05	Abort on Truck Mismatch	N4 sets the TO_STATUS = "ABORT" , PROBLEM_CODE = "ABORT_05"
ABORT_06	Abort on No Truck	N4 sets the TO_STATUS = "ABORT" , PROBLEM_CODE = "ABORT_06"

These are the default abort use case implementations in N4. The interface provides extension points to allow alternative implementations as needed.

12.3.2.1 Abort on Set Down (ABORT_01)

The interaction diagram below shows typical abort on set workflow.



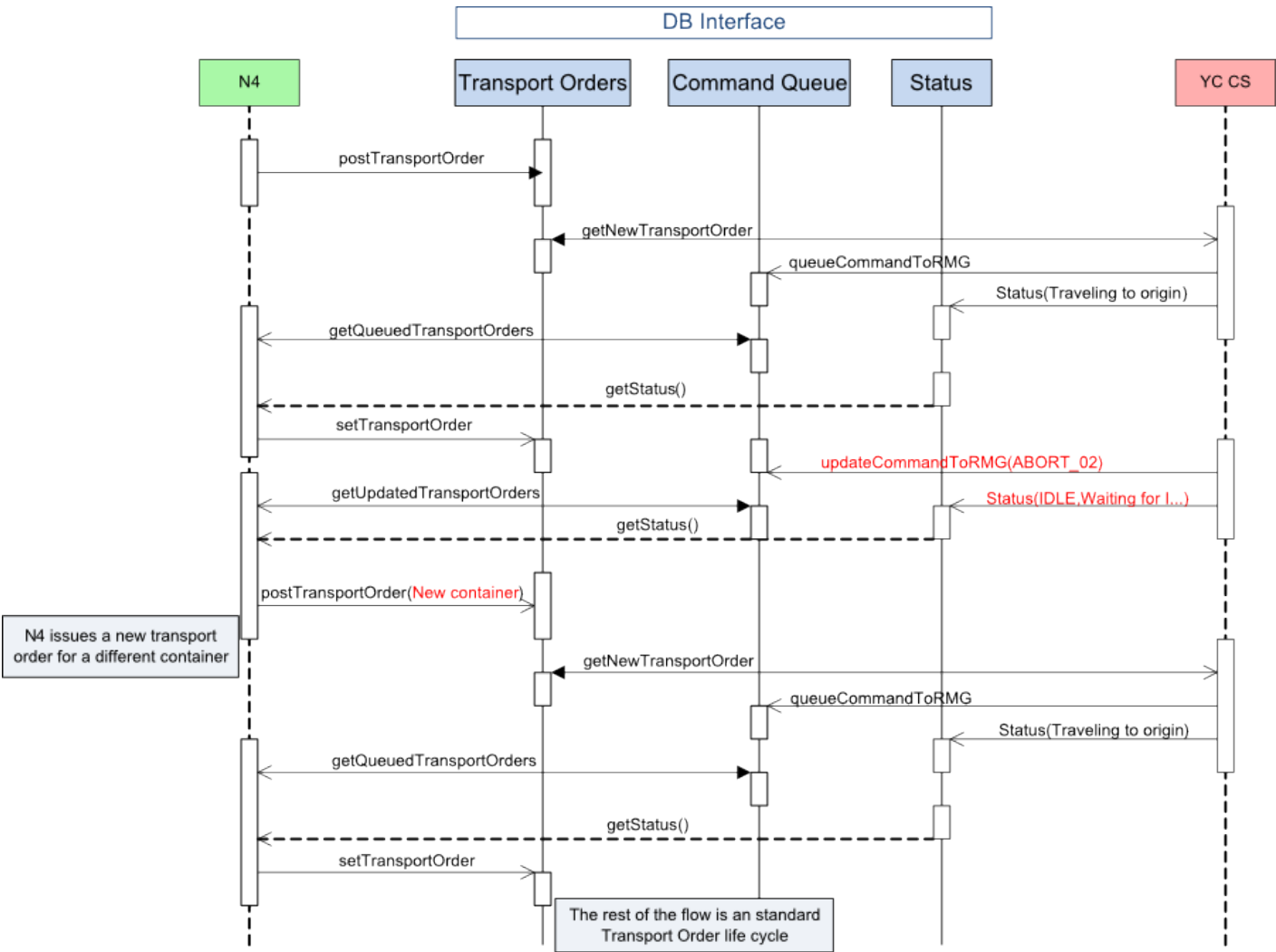
Item	Description
Level	User Goal
Trigger	Crane ECS sets CQ_STATUS = 'ABORT' and PROBLEM_CODE = 'ABORT_01'
Actors	N4 and ECS
Scenario	Crane Remains in Automatic Mode The crane is executing a job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ECS is not able to set the container in the destination specified by TO1 ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_01' and CQ.UPDATED = <i>current time stamp</i> N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_01' and UPDATED = <i>current time stamp</i> N4 issues a new Transport Order to place a container in a new location. The new Transport Order has an ORIG_BAY_TYPE = 'SPREADER'

Item	Description
Extension 1	Operator Switches Crane to Manual Mode, Completes Move to New Location The crane is executing a job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ECS is not able to set the container in the destination specified by TO1 Operator switches control to manual ECS sets ST.ONLINE_STATUS = "MANUAL" ECS sets CQ1.CQ_STATUS = "ABORT", CQ1.PROBLEM_CODE = "ABORT_01" and CQ1.UPDATED = <i>current time stamp</i> N4 updates the current Transport Order, sets TO1.TO_STATUS = 'ABORT', TO1.PROBLEM_CODE = 'ABORT_01' and TO1.UPDATED = <i>current time stamp</i> Operator use remote table to control crane and sets the container down in a new location ECS creates CQ2 to record the manual override (uncommanded move), sets CQ2.CONTAINER_ID = <i>container id</i>, CQ2.ORIG_BAY_TYPE = "SPREADER", CQ2.CQ_STATUS = "COMPLETE" and destination fields for the location where the container was set by the operator. Operator returns control to automatic ECS sets ST.ONLINE_STATUS = "AUTO" N4 creates TO2 with TO2.ORIG_BAY_TYPE = "SPREADER". N4 reconciles TO2 with destination fields specified in CQ2, sets TO2.TO_STATUS = "COMPLETE", updates the current position of the container as specified in CQ2 and completes the work instruction

Item	Description
Extension 2	Operator Switches Crane to Manual Mode, Returns Container to Origin Location The crane is executing a job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ECS is not able to set the container in the destination specified by TO1 Operator switches control to manual ECS sets ST.ONLINE_STATUS = "MANUAL" ECS sets CQ1.CQ_STATUS = "ABORT", CQ1.PROBLEM_CODE = "ABORT_01" and CQ1.UPDATED = <i>current time stamp</i> N4 updates the current Transport Order, sets TO1.TO_STATUS = 'ABORT', TO1.PROBLEM_CODE = 'ABORT_01' and TO1.UPDATED = <i>current time stamp</i> Operator use remote table to control crane and sets the container down in a new location ECS creates CQ2 to record the manual override (uncommanded move), sets CQ2.CONTAINER_ID = <i>container id</i>, CQ2.ORIG_BAY_TYPE = "SPREADER", CQ2.CQ_STATUS = "COMPLETE" and destination fields for the location where the container was set by the operator. Operator returns control to automatic ECS sets ST.ONLINE_STATUS = "AUTO" N4 creates TO2 with TO2.ORIG_BAY_TYPE = "SPREADER". N4 reconciles TO2 with destination fields specified in CQ2, sets TO2.TO_STATUS = "COMPLETE", resets the current position of the container and resets the dispatched work instruction to move stage = "Planned" and dispatch state = "Bypass"

12.3.2.2 Abort on Pickup (ABORT_02)

The interaction diagram below shows typical abort on pickup workflow.



Item	Description
Level	User Goal
Trigger	ECS set CQ_STATUS = 'ABORT' and PROBLEM_CODE = 'ABORT_02'
Actors	N4 and ECS

Item	Description
Scenario	The crane is executing a job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ECS is not able to pickup the container from it's origin location ECS sets CQ1.CQ_STATUS = 'ABORT', CQ1.PROBLEM_CODE = 'ABORT_02' and CQ1.UPDATED = <i>current timestamp</i> N4 updates the current Transport Order, sets TO1.TO_STATUS = 'ABORT', TO1.PROBLEM_CODE = 'ABORT_02' and TO1.UPDATED = <i>current time stamp</i> N4 resets the dispatched work instruction to move stage = "Planned" and dispatch state = "Bypass" N4 issues a new Transport Order (TO2) to move a different container.
Extensions	None

12.3.2.3 Abort on Door direction (ABORT_03)

In the ECN4 ASC Automated Equipment Database Interface, ABORT_03 cases were handled as follows:

In some cases, containers being transferred out of the stack will have the incorrect door direction. In such cases, one of two actions must be taken. If the container cannot be placed on the chassis, the **TO_STATUS** should be set to 'ABORT_03'. This will trigger TOS to generate the instructions necessary to change the door direction. In this case that means a work instruction to place the container back into the YC stack. Another work instruction is generated from the YC stack to the FLIP queue. This will then trigger a yard truck to come to the land side transfer zone to receive the container. Once the container is on the yard truck the truck goes to "flip" location where a top pick will move it from the internal truck to the external truck with the desired door direction.

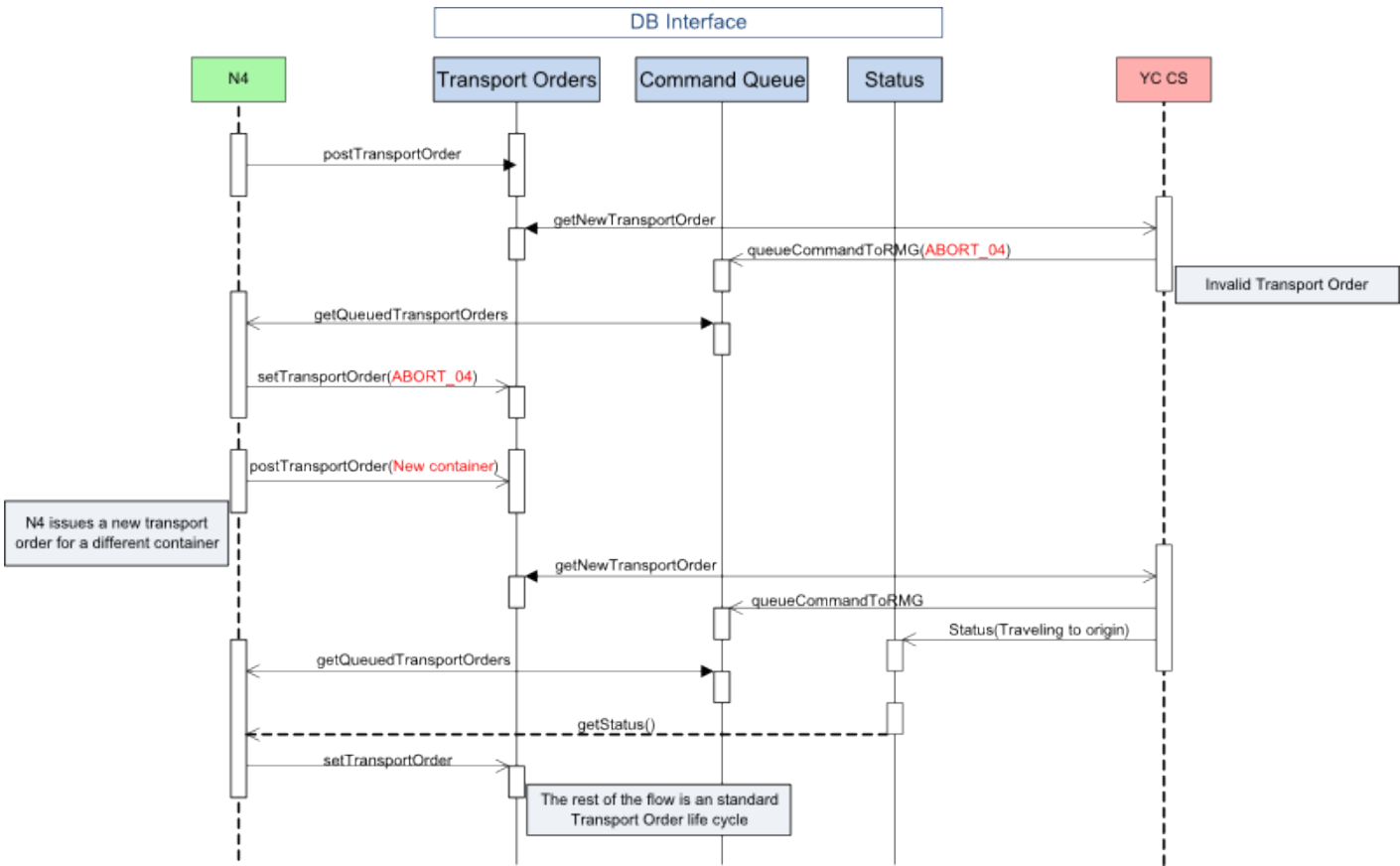
In release 3.1, Navis has removed this functionality, opting instead for handling ABORT_03 as an ABORT_02 (abort on pickup) if the container is not yet lifted, or as an ABORT_01 (abort on set down) if the container is being carried by the automated yard crane.

Who identifies the door direction being a problem ?

It is crucial to understand that the identification of a door direction issue is not within the scope of N4. This is something that comes either as a function of the ECS, or more likely, as a result of a remote operator involvement. The above codes are how these issues are communicated to N4.

12.3.2.4 Abort due to Invalid command (ABORT_04)

N4 handles ABORT_04 as an ABORT_02 (abort on pickup if the container is not yet lifted, or as an ABORT_01 (abort on set down) if the container is being carried by the automated yard crane. The interaction diagram below shows typical abort due to invalid command workflow.



Item	Description
Level	User Goal
Trigger	Crane ECS sets CQ_STATUS = 'ABORT' and PROBLEM_CODE = 'ABORT_04'
Actors	N4 and ECS
Scenario	N4 inserts Transport Order (TO1) with status "ENTERED" ECS or Remote Operator is not able to take any action as part of workflow. ECS inserts Command Queue (CQ1) with CQ1.CQ_STATUS = 'ABORT', CQ1.PROBLEM_CODE = 'ABORT_04' and CQ1.UPDATED = <i>current timestamp</i> N4 updates the current Transport Order, sets TO1.TO_STATUS = 'ABORT', TO1.PROBLEM_CODE = 'ABORT_04' and TO1.UPDATED = <i>current time stamp</i> N4 resets the dispatched work instruction to move stage = "Planned" and dispatch state = "Bypass" N4 issues a new Transport Order (TO2) to move a different container.

Item	Description
Extension 1	Crane is executing a Job and Container on Spreader The crane is currently executing a job and hoisting a container on spreader ECS or Remote Operator is not able to take any action as part of workflow ECS inserts Command Queue (CQ1) with CQ1.CQ_STATUS = 'ABORT', CQ1.PROBLEM_CODE = 'ABORT_04' and CQ1.UPDATED = <i>current timestamp</i> N4 updates the current Transport Order, sets TO1.TO_STATUS = 'ABORT', TO1.PROBLEM_CODE = 'ABORT_04' TO1.UPDATED = <i>current time stamp</i> N4 issues a new Transport Order to place a container in a new location. The new Transport Order has an ORIG_BAY_TYPE = 'SPREADER'

12.3.2.5 Abort on Truck Mismatch (ABORT_05)

The interaction diagram below shows typical abort on truck mismatch workflows.

Item	Description
Level	User Goal
Trigger	Crane ECS sets CQ_STATUS = 'ABORT' and PROBLEM_CODE = 'ABORT_05'
Actors	N4 and ECS
Prerequisite	CONTROLAUTOMATION029 set to TRANSFER_ZONE (Passive RFID solution)

Scenario	ECS issues ABORT (ABORT_05) due to Wrong Truck at Transfer Point (TP) before lift Note: Applicable for both inbound move and outbound move (before lift) • Truck for the move entered into Transfer Zone • The crane is dispatched by N4 • The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" • ECS reads different truck at the Transfer Point (TP) than the truck ID specified in TO1 • ECS updates the TRUCK_STATUS table with the new truck at Transfer Point • N4 updates the new truck to be available in the TP • ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_05' and CQ.UPDATED = current timestamp • N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_05' and UPDATED = current timestamp • Note: ▪ WI for RTG is not bypassed ▪ Transfer Point will not be temp blocked ▪ For 5 seconds after abort 05 WA is last updated, dispatcher will only allow dispatches to the same row.
----------	--

Extension 1	Outbound move - ECS issues ABORT (ABORT_05) due to Wrong Truck (not swappable) at Transfer Point (TP) - After Lift • Truck for the move entered into Transfer Zone ▪ The crane is dispatched by N4 ▪ The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ▪ The crane lifted the container for the outbound move from yard position and is in the crane's spreader ▪ ECS reads different truck (UTR in different pool / different OTR) at the Transfer Point (TP) than the truck ID specified in TO1 ▪ ECS updates the TRUCK_STATUS table with the new truck at Transfer Point ▪ N4 updates the new truck to be available in the TP ▪ ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_05' and CQ.UPDATED = current timestamp ▪ N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_05' and UPDATED = current timestamp ▪ N4 creates a new WI in Transfer Queue and issues a new Transport Order (Laden redirect move) to place the container in the same location where it was picked from. The new Transport Order has an ORIG_BAY_TYPE = 'SPREADER' ▪ ECS completes the laden redirect move ▪ Container position should be Yard Position
-------------	--

Extension 2	Outbound move - ECS issues ABORT (ABORT_05) due to Wrong Truck (swappable) at Transfer Point (TP) - After Lift • Truck for the move entered into Transfer Zone ▪ The crane is dispatched by N4 ▪ The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ▪ The crane lifted the outbound move from yard position and container is in the crane's spreader ▪ ECS reads different truck (same pool) at the Transfer Point (TP) than the truck ID specified in TO1 ▪ ECS updates the TRUCK_STATUS table with the new truck at Transfer Point ▪ N4 updates the new truck to be available in the TP ▪ ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_05' and CQ.UPDATED = current timestamp ▪ N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_05' and UPDATED = current timestamp ▪ N4 swaps the move to the new truck at TP and issues a new Transport Order (Laden redirect move) to the same TP position after swap. The new Transport Order has an ORIG_BAY_TYPE = 'SPREADER' ▪ ECS completes the laden redirect move to the new truck at the same TP.
-------------	---

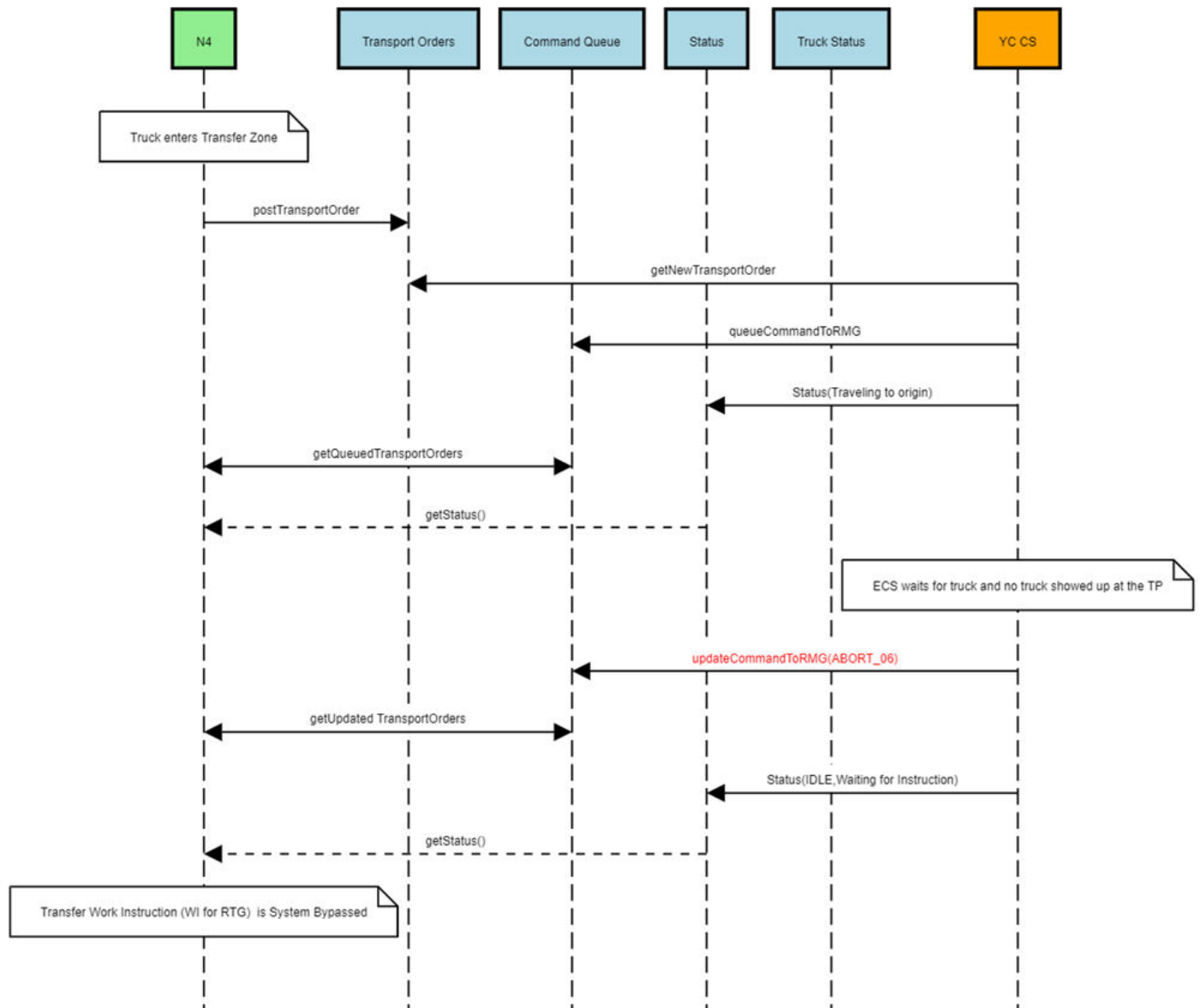
Extension 3	Outbound move - ECS issues ABORT (ABORT_05) due to Wrong Truck (swappable) at Transfer Point (TP) - After Lift (Truck Status updated after ABORT) ▪ Truck for the move entered into Transfer Zone ▪ The crane is dispatched by N4 ▪ The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ▪ The crane lifted the outbound move from yard position and container is in the crane's spreader ▪ ECS reads different truck (different pool) at the Transfer Point (TP) than the truck ID specified in TO1 ▪ ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_05' and CQ.UPDATED = current time stamp ▪ N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_05' and UPDATED = current time stamp ▪ ECS updates the TRUCK_STATUS table with the new truck at Transfer Point (within 5 seconds) ▪ N4 updates the truck to be available in the TP ▪ N4 issues a new Transport Order (Laden redirect move) to the same TP position after swap. The new Transport Order has an ORIG_BAY_TYPE = 'SPREADER' ▪ ECS completes the laden redirect move to the new truck at the same TP.
-------------	--

Extension 4	ECS issues ABORT (ABORT_05) due to Wrong Truck at Transfer Point (TP) before lift (no TRUCK_STATUS update) • Truck for the move entered into Transfer Zone ▪ The crane is dispatched by N4 ▪ The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" ▪ ECS reads different truck at the Transfer Point (TP) than the truck ID specified in TO1 ▪ ECS did not update the TRUCK_STATUS table with the new truck at Transfer Point ▪ ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_05' and CQ.UPDATED = current time stamp ▪ N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_05' and UPDATED = current timestamp ▪ Note: ◦ For 5 seconds after abort 05 WA is last updated, dispatcher will NOT allow any dispatches from the same row i.e. if there is another job in different row, that is dispatched immediately
-------------	---

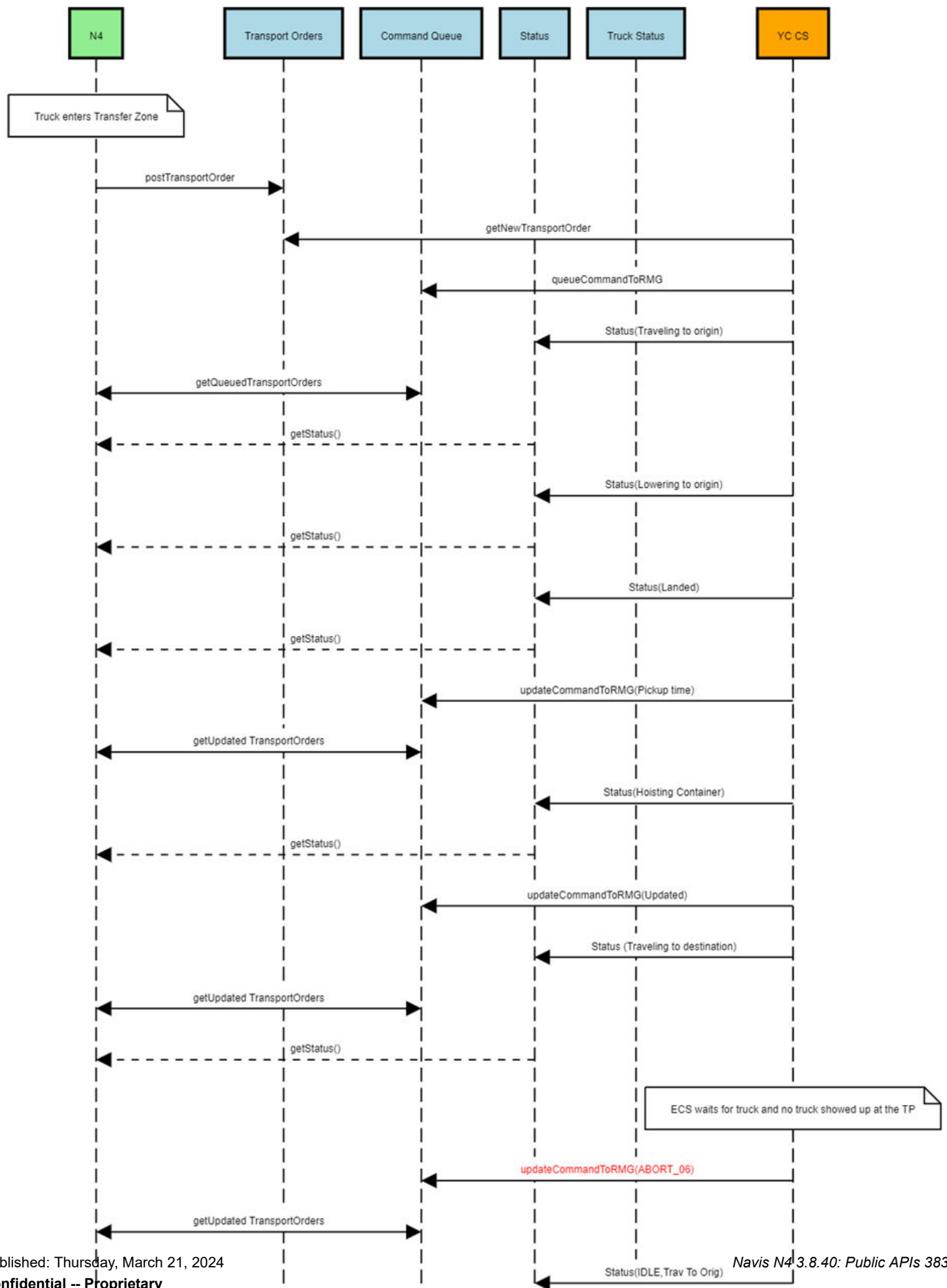
12.3.2.6 Abort on No Truck (ABORT_06)

The interaction diagram below shows typical abort on no truck workflows.

ECS issues ABORT (ABORT_06) due to No Truck at Transfer Point (TP)



Outbound move - ECS issues ABORT (ABORT_06) due to No Truck at Transfer Point (TP) - After Lift



Item	Description
Level	User Goal
Trigger	Crane ECS sets CQ_STATUS = 'ABORT' and PROBLEM_CODE = 'ABORT_06'
Actors	N4 and ECS
Prerequisite	CONTROLAUTOMATION029 set to TRANSFER_ZONE (Passive RFID solution)
Scenario	ECS issues ABORT (ABORT_06) due to No Truck at Transfer Point (TP) Note: Applicable for both inbound move and outbound move (before lift) - Truck for the move entered into Transfer Zone - The crane is dispatched by N4 - The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" - ECS waits for the truck and no truck showed up at the Transfer Point (TP) - ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_06' and CQ.UPDATED = current timestamp - N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_06' and UPDATED = current timestamp - Transfer Work Instruction (WI for RTG) is System Bypassed Note: Neither Transfer point or Stack position is temp blocked.

Extension 1	Outbound move - ECS issues ABORT (ABORT_05) due to Wrong Truck (not swappable) at Transfer Point (TP) - After Lift - Truck for the move entered into Transfer Zone - The crane is dispatched by N4 - The crane is executing the job dispatched by N4 and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" - The crane lifted the container for the outbound move from yard position and is in the crane's spreader - ECS reads different truck (UTR in different pool / different OTR) at the Transfer Point (TP) than the truck ID specified in TO1 - ECS updates the TRUCK_STATUS table with the new truck at Transfer Point - N4 updates the new truck to be available in the TP - ECS sets CQ.CQ_STATUS = 'ABORT', CQ.PROBLEM_CODE = 'ABORT_05' and CQ.UPDATED = current timestamp - N4 updates the current Transport Order, sets TO_STATUS = 'ABORT', PROBLEM_CODE = 'ABORT_05' and UPDATED = current timestamp - N4 creates a new WI in Transfer Queue and issues a new Transport Order (Laden redirect move) to place the container in the same location where it was picked from. The new Transport Order has an ORIG_BAY_TYPE = 'SPREADER' - ECS completes the laden redirect move - Container position should be Yard Position
-------------	--

12.3.3 Dispatched Moves Completed by Remote Operator

While the crane is executing a move that has been dispatched by the TOS, a remote operator can switch the crane out of automatic mode and manually complete the move to another location , including setting the container back down in it's original location. Unlike the abort scenarios listed above, in these cases no abort status is communicated to the TOS, the dispatched job is simply overridden.

Item	Description
Level	User Goal
Trigger	Crane ECS remote operator sets crane to MANUAL mode while job is in progress
Actor	N4 and ECS

Item	Description
Scenario	Operator Switches Crane to Manual Mode, Completes Move to New Location The crane is executing a job dispatched by the TOS and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" Operator switches control to manual ECS sets ST.ONLINE_STATUS = "MANUAL" Operator use remote table to control crane and sets the container down in a new location. ECS updates the current Command Queue (CQ1) to CQ1.CQ_STATUS = "COMPLETE" and destination fields for the location where the container was set by the operator. Operator returns control to automatic ECS sets ST.ONLINE_STATUS = "AUTO" N4 reconciles TO1 with destination fields specified in CQ1, sets TO1.TO_STATUS = "COMPLETE", updates the current position of the container as specified in CQ1 and completes the work instruction
Extension 1	Operator Switches Crane to Manual Mode, Returns Container to Origin Location The crane is executing a job dispatched by the TOS and the current Transport Order (TO1) and Command Queue (CQ1) records have status "WORKING" Operator switches control to manual ECS sets ST.ONLINE_STATUS = "MANUAL" Operator use remote table to control crane and sets the container down to origin location ECS updates the current Command Queue (CQ1) to CQ1.CQ_STATUS = "COMPLETE" and destination fields for the origin location where the container was set by the operator. Operator returns control to automatic ECS sets ST.ONLINE_STATUS = "AUTO" N4 reconciles TO1 with destination fields specified in CQ1, sets TO1.TO_STATUS = "COMPLETE", resets the current position of the container at origin location and resets the dispatched work instruction to move stage = "Planned" and dispatch state = "Bypass"

12.3.4 Cancellations

The N4 TOS may need to cancel a move which has been entered into the Transport Order Table. Typically this is due to a user reverting a work instruction that was just dispatched.

Note that there is a potential race condition related to the actual start of the order and the intent to cancel. The presence of the order in the CQ table will override attempts to cancel, and the order will complete as normal.

Item	Description
Level	User Goal
Preconditions	- N4 has inserted Transport Order (TO1) with status "ENTERED" - ECS has <i>not</i> inserted a Command Queue record corresponding to TO1
Trigger	N4 user reverts work instruction corresponding to TO1
Actors	N4
Scenario	N4 updates the Transport Order, sets TO1.TO_STATUS = 'CANCEL' and TO1.UPDATED = <i>current time stamp</i> ECS ignores Transport Order with TO1.TO_STATUS='CANCEL'
Extension 1	ECS has inserted a Command Queue record (CQ1) corresponding to TO1 - N4 user reverts work instruction corresponding to TO1 - ECS completes Transport Order TO1, Container Inventory may not be updated.

12.3.5 Offline Operation of Crane and Inventory Synchronization

Crane operations can take place without the direction of the N4 TOS. There are two types of "non-TOS" crane operations, manual (a.k.a. un-commanded) moves and crane ECS-initiated moves. Manual moves are initiated by the remote operator. In this case the crane ECS will report these moves as a series of YC_COMMAND_QUEUE table entries. A series of moves is required since the crane ECS cannot determine the origin and destination in advance so the series is typically a move from a location to the spreader and a subsequent move from the spreader to a location. A crane ECS-initiated move is similar to an N4-initiated move in that it is "commanded" and has a planned source and destination.

In the manual and ECS-initiated cases, the ECS records all container movements to the Command Queue table without TO_GKEYs. N4 will interpret these sequences either in real-time, or if offline, when it reconnects to the database. N4 shall "apply" these movements by inserting corresponding the Transport Orders. Then the crane ECS must update the TO_GKEY in the Command Queue table from the corresponding Transport Orders. All of the command queue records must be reconciled with transport orders before N4 will issue any new transport orders. The ECS must check the status of the YC_COMMAND_QUEUE before inserting a new command queue record and update the TO_GKEYs for any command queue records that have been "applied" by TOS.

Item	Description
Level	User Goal

Item	Description
Preconditions	Crane STATUS.ONLINE_STATUS = "OFFLINE" ECS operator moves one or more containers. For each container moved ECS inserts 2 records into the Command Queue table: CQ1 from origin location to a destination where DEST_BAY_TYPE= "SPREADER", CONTAINER_ID = <i>container id</i>, CQ1.TO_GKEY = <i>Null</i> and CQ1.CQ_STATUS = "COMPLETE" CQ2 from origin where ORIG_BAY_TYPE = "SPREADER" to destination location, CONTAINER_ID = <i>container id</i>, CQ2.TO_GKEY = <i>Null</i> and CQ2.CQ_STATUS = "COMPLETE"
Trigger	N4 bootstrap or Crane operating in MANUAL mode.
Actors	N4 and ECS
Main Scenario	N4 connects to database interface and for each Command Queue entry pair where TO_GKEY = <i>Null</i> and CQ_STATUS = "COMPLETE": Validates the container at the origin location from CQ1 Updates container position based on the destination recorded in CQ2 Inserts Transport Order TO1 to reconcile against CQ1 Inserts Transport Order TO2 to reconcile against CQ2
Extension 1	ECS Sets Container ID = <i>Null</i>, N4 Resolves Container from Origin N4 connects to database interface and for each Command Queue entry pair where TO_GKEY = <i>Null</i> and CQ_STATUS = "COMPLETE": Determines the container from the origin location from CQ1 Updates container position based on the destination recorded in CQ2 Inserts Transport Order TO1 to reconcile against CQ1 Inserts Transport Order TO2 to reconcile against CQ2
Extension 2	ECS Sets Container ID = <i>Null</i>, N4 Cannot Resolve Container from Origin N4 connects to database interface and for each Command Queue entry pair where TO_GKEY = <i>Null</i> and CQ_STATUS = "COMPLETE": Cannot determine the container from the origin location from CQ1 Logs ERROR
Extension 3	ECS sets CQ_STATUS = 'ENTERED' or CQ_STATUS = 'WORKING' N4 will ignore these records.
Extension 4	ECS sets CQ_STATUS = 'ABORT' N4 Inserts Transport Order TO to reconcile against CQ, no updates in TOS

Item	Description
Extension 5	ECS Sets Container ID = Null, CQ_STATUS ='ABORT' N4 Inserts Transport Order TO to reconcile against CQ, no updates in TOS Logs massive amounts of ERRORS
Extension 6	ECS sets CQ_STATUS ='MAINTENANCE' N4 Inserts Transport Order TO to reconcile against CQ, no updates in TOS

12.3.6 Crane Move Order

In release 3.1, a TOS initiated crane movement order has been added to allow for the repositioning of an automated crane for impending work. The "reposition" workflow controls the sending of this move order and it is used by the C-ARMG Scheduler in side loading crane operations to send an automated crane to the bay of an impending outbound move if the crane is IDLE and not near the outbound bay.

12.3.6.1 TOS Initiated Crane Movement Order

The TOS can issue a move order, a crane movement without a container, to a crane in order to preposition it for impending work. The following workflow describes the sequence of events for a move order.

Item	Description
1	TOS issues a new transport order to the Transport Order table. The record includes: YC_TRANSPORT_ORDER.TO_GKEY= unique database generated sequencing field YC_TRANSPORT_ORDER.CQ_GKEY= NULL YC_TRANSPORT_ORDER.TO_STATUS= ENTERED YC_TRANSPORT_ORDER.CHE_ID= ID of ASC being issued the transport order YC_TRANSPORT_ORDER.CREATED= current time stamp YC_TRANSPORT_ORDER.UPDATED= current time stamp The container info is Null, the position destination is entered. (Is origin needed?)

Item	Description
2	The ECS queries the Transport Order table for new transport orders, validates the information, issues the command to the ECS, and issues a command to the Command Queue table. The Status table must be updated as well in this step. The Command Queue table and Status table are updated as below: YC_COMMAND_QUEUE.CQ_GKEY= unique database generated sequencing field YC_COMMAND_QUEUE.TO_GKEY= unique id of the transport order issued by TOS for the particular transport order. YC_COMMAND_QUEUE.CQ_STATUS= WORKING YC_COMMAND_QUEUE.CHE_ID= ID of ASC being issued the transport order YC_COMMAND_QUEUE.CREATED=current time stamp YC_COMMAND_QUEUE.UPDATED= current time stamp STATUS.TO_GKEY= transport order currently being executed STATUS.CQ_GKEY= command currently being executed STATUS.WORK_STATUS= WORKING STATUS.SEQUENCE_STATUS= TRAVELING TO DESTINATION STATUS.UPDATED= current time stamp
3	N4 polls the Command Queue table for records where the CQ_GKEY is NOT set the TO_STATUS to WORKING in corresponding Transport Order
4	N4 marks the transport order as "COMPLETE" in the N4 Transport Order Table under TO_STATUS. YC_COMMAND_QUEUE.CQ_STATUS= COMPLETE YC_COMMAND_QUEUE.UPDATED= current time stamp
5	The move has been completed and ECS must initialize the YC status. This tells N4 that the YC is IDLE and ready to be dispatched to execute a new transport order. STATUS.TO_GKEY= 0 STATUS.CQ_GKEY= 0 STATUS.SEQUENCE_STATUS= WAITING FOR INSTRUCTION STATUS.WORK_STATUS= IDLE STATUS.UPDATED= current time stamp

12.3.6.2 ECS Initiated Crane Movement Order

The ECS can issue a move order, a crane movement without a container, to a crane occasionally for various reasons. The following workflow describes the sequence of events for a move order.

Item	Description
Level	User Goal
Scope	ECS
Trigger	ECS set appropriate MAINTENANCE_CODE , MAINTENANCE_CODE_DESCRIPTION and CQ_STATUS = 'MAINTENANCE'.
Actor	ECS operator issues a crane move order to a location in module
Scenario	ECS sets ONLINE_STATUS = 'MAINTENANCE' in STATUS table ECS inserts a new row in the command queue table MAINTENANCE_CODE MAINTENANCE_CODE_DESCRIPTION CQ_STATUS = 'MAINTENANCE' Origin fields Destination fields TOS makes crane unavailable for automatic dispatch TOS echos newly inserts new order in transport order table
Extensions	None

12.4 Purge Yard Crane database tables

You can purge data from the yard crane related tables in the Equipment Control Interface (ECI), for both SQL Server and Oracle databases. Purging is the process of deleting all the obsolete data (data which is no longer required) from the tables. When the purging process runs for the yard crane database, all the records in the related tables are first archived into another table to secure the data for future references, and then purged.

Tables in YC ECI DB

Table Name	Parameters considered	is Archived?	Archived table name
N4_APPLICATION_STATU S	All entries are purged	Yes	ARCHIVE_N4_APPLICATION_STATU S
ECS_APPLICATION_STAT US	All entries are purged	Yes	ARCHIVE_ECS_APPLICATION_STAT US
STATUS	All entries are purged	Yes	ARCHIVE_STATUS
AREA_STATUS	CREATED date	Yes	ARCHIVE_AREA_STATUS
TRUCK_STATUS	CREATED date	Yes	ARCHIVE_TRUCK_STATU S
YC_COMMAND_QUEUE	CREATED date	Yes	ARCHIVE_YC_COMMAND _QUEUE
YC_TRANSPORT_ORDER S	CREATED date	Yes	ARCHIVE_YC_TRANSPOR T_ORDERS

The purging process involves the following steps:

1. Identification – during this process, records to be purged from the yard crane tables are identified and the gkey of that entry is stored in a temporary table. This gkey is later used to archive and purge the records. If the user wants to use their own logic (purging on basis of parameters other than created time) they can change the logic only during this phase and nowhere else.
2. Archiving - all the obsolete records in the yard crane tables are archived based on 'CREATED' date. However, records in the "N4_APPLICATION_STATUS", "ECS_APPLICATION_STATUS", and "STATUS" tables are archived irrespective of the created date.
3. Purging - all the obsolete records in the yard crane tables are purged based on 'CREATED' date. However, records in the "N4_APPLICATION_STATUS", "ECS_APPLICATION_STATUS", and "STATUS" tables are purged irrespective of the created date.

Setting up the purge job

1. Unzip the 'purge.zip' file. This file contains all the required files and is included as part of the **N4 SDK** zip file available from the **Navis Customer Portal**.

The purge.zip file has a folder named 'purge' with the following sub folders:

purge-oracle – includes the readme.txt file and all script files for purging.

purge-sqlserver – includes the readme.txt file and all script files for purging.

2. Review the readme files to get an overview of the purge process.
3. Configure the SqlServer Agent Job/Oracle database background job to run this procedure(purgeJob) in the background on regular intervals.



Implementations using native database tools (such as the reference implementation, which uses stored



procedures) may often be conveniently deployed using standard database job scheduling mechanisms (such as SQL Server's SQL Server Agent or Oracle Enterprise Scheduler).

Purge tables information

PURGE_DIAGNOSTICS table

This table stores details about each purge run. Using this table you can retrieve all the records of the tables at that particular time. This table has the following columns:

purge_id - this is a unique ID that is associated with every run - this ID can be used to get all the entries of the table at the time it was archived (mentioned below) - This ID is matched with the **obs_id** in the archive table.

archived - this is the time when the archiving job began.

PURGE_PARAMETER table

This table has some parameters which help you to configure the purge job.

expiredEntriesAgeInDays - Every time the "purgeJob" script is run, it will purge the records in the YC_TRANSPORT_ORDERS, YC_COMMAND_QUEUE, AREA_STATUS, and TRUCK_STATUS tables that are 10 days old. Records in the records in the N4_APPLICATION_STATUS, ECS_APPLICATION_STATUS, and STATUS tables are not purged. The default value is 10 days..

purgeAllTables - If the value is set as 1, the script will purge all records in the N4_APPLICATION_STATUS, ECS_APPLICATION_STATUS, and STATUS tables. Records in the YC_TRANSPORT_ORDERS, YC_COMMAND_QUEUE, AREA_STATUS, and TRUCK_STATUS tables are purged based on the value defined for the expiredEntriesAgeInDays parameter.

Archive table

All the archive tables have the same parameters as the original tables, but also have two new columns - "archived" (the time when a record is archived) and "obs_id" (unique id associated with every run - this is equivalent to the **purge_id** in the PURGE_DIAGNOSTICS table).

Part 13

Automation Database Interface

In this Section:

Overview	395
Equipment Interaction	401
Location Status	432
Database Interface Definition	435

13.1 Overview

The Automation Database (AutoDB) Interface allows for N4 to integration to Equipment Control Systems (ECS) involved in the operation of an automated terminal. N4 provides instructions for movements for automated equipment by writing orders to this database. The ECS reads the orders and manages the execution of commands for the equipment to carry out the orders.

This mechanism has a number of attractive qualities that make it suitable for communicating this type of service request:

- Message exchange is persistent and has historical depth.
- Field based structure eliminates parsing. In addition referential integrity and field usage rules improve quality of message structure.
- One writer rule is easily implemented. Database security can insure integrity of stream.
- Participants can easily develop and validate adherence to the standard. Responsibility and accountability is compartmental.
- Key generation for uniqueness and sequencing is intrinsic.
- Very easy for third party software to monitor the link and perform forensic analysis of issues.
- Potential use of standard replication mechanisms to support redundancy.
- Each application is responsible for managing its respective table.

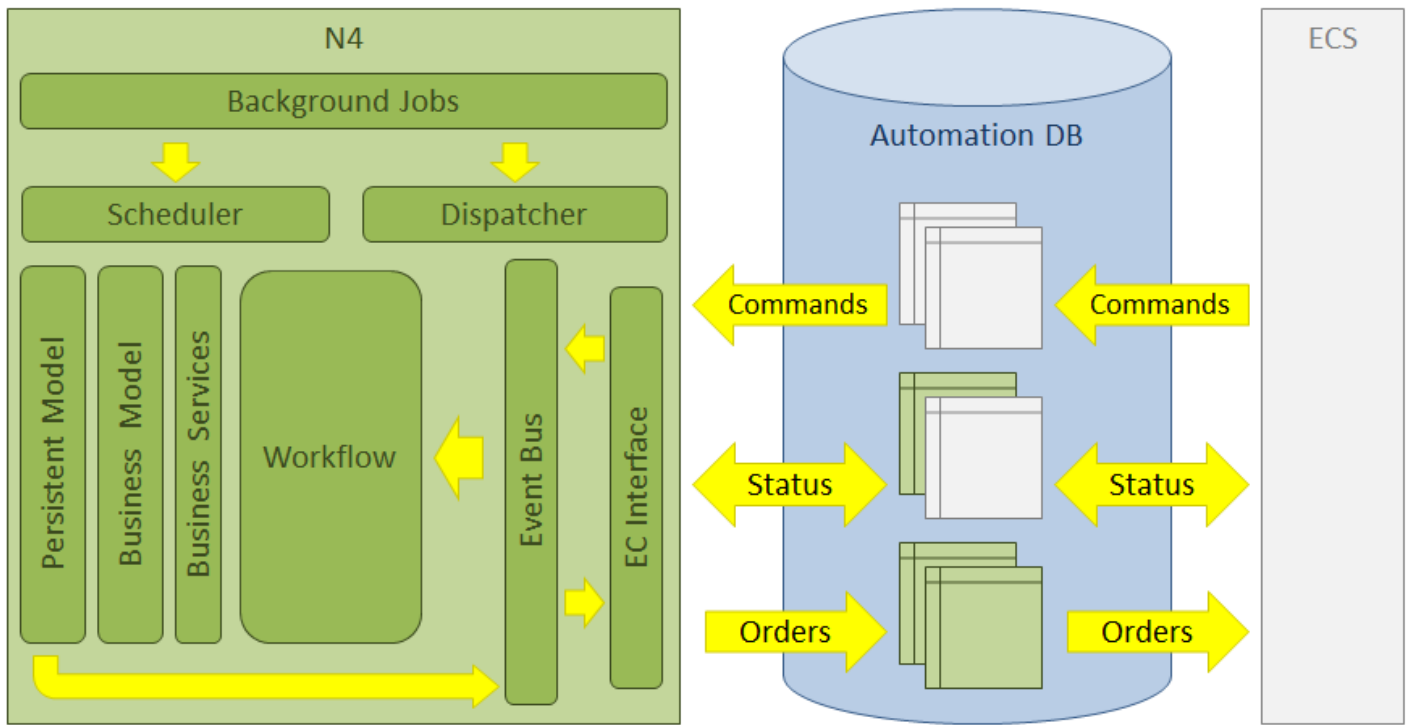
The database described here is not part of the N4 system. It is an external database that is installed, configured, and maintained by the customer.

The Automation Database must be either of the following:

Microsoft SQLServer

Oracle

Please review the TOS Application Environment Guide to determine the currently supported versions of the above databases.



In This Section

Responsibilities.	397
Notification Mechanism.	398
Synchronization Points.	399
Startup and Recovery Process.	399

13.1.1 Responsibilities

Database tables belong to either N4 or ECS, with the owner of the table as the only application allowed to write to that table. For database names and columns the upper case is always used. Both systems operate in the same time zone, which is normally the timezone of the container terminal facility.

To prevent unnecessary updates, both systems write to the database table only if there is a change in data. A notification mechanism is implemented by the TRIGGER_EVENT_STATUS table to prevent the need for applications to poll all the database tables.

When N4 generates an instruction it adds a row of data to the appropriate ORDERS table. Typically there is only one instruction in the table waiting to be executed, though there can be more than one. For all container movement cases it is assumed that before N4 inserts a record in AGV/ASC _ORDERS or before the ECS inserts a record in QC_COMMANDS, N4 will provide information about the container and the work instruction in the table WORK_INSTRUCTION_STATUS.

The ECS maintains claims in the transfer point to avoid collisions and respect safety height restrictions of the CHE. N4 does not use the claim information for managing transfer points as this information is usually immediate and N4 is using its internal job projections to determine when in the future CHEs may be interacting in a particular transfer point.

13.1.1.1 Active Orders and Commands

ORDERS and COMMANDS records having one or more of the following characteristics are considered "Active", meaning additional updates and processing is expected.

An "Active" ORDERS or COMMANDS record:

- has a STATUS value of ENTERED or WORKING; or
- has a null value in the GKEY field which is supposed to link it to the partner's record (e.g. COMMANDS_GKEY for ORDERS records or ORDERS_GKEY for COMMANDS records); or
- has a STATUS value differing from that of the partner's record.
- each side (N4, ECS) will respond to an insertion or an update of the partner's "Active" record by inserting or updating its record such that it eventually becomes inactive; and
- once a COMMANDS or ORDERS record becomes inactive, there should be no action causing it to become "Active" again.

13.1.1.2 Providing time stamps

The ECS must provide timestamps when ASCs or AGVs enter or leave transfer points, and for AGVs this includes the parallel buffer. When an AGV enters the parallel buffer again, the timestamp is overwritten.

13.1.1.3 Weight measurements

The ECS can report the weight of containers via updating the measurements: field MEASURED_WEIGHT_KG in QC_COMMANDS and, ASC_COMMANDS. Note that entries in these tables represent a "live cycle" of the container in the crane system, which may involve more than one lift and set. the ECS will update the field MEASURED_WEIGHT_KG whenever a measurement from a crane comes in. A new valid measurement will override a previous value. the ECS shall not replace a non-null value and set it to Null based on new input from the cranes.

For ASCs, this updates the yard-measured-weight. For the QC the interface only supports individually setting the

container weight for containers. For multiple lift, this can be done by the Crane Automation System (CAS) Identify message which supports yard-gross-weight and yard-measured-weight.

13.1.1.4 Key Generation

The database should be configured to automatically generate the following values:

- ORDER_GKEY for AGV_ORDERS, ASC_ORDERS, QC_ORDERS, TEST_ORDERS
- COMMAND_GKEY for AGV_COMMANDS, ASC_COMMANDS, QC_COMMANDS, TEST_COMMANDS

13.1.2 Notification Mechanism

AutoDB provides a notification mechanism via the TRIGGER_EVENT_STATUS table. Whenever a record is inserted or updated a configured database trigger updates the record for the corresponding action, for example, AGV_ORDERS_INSERTED or AGV_ORDERS_UPDATED. To keep track of new notifications, a notification record gets a new incremental sequence attribute, provided by the database itself. Sequence attributes for the last handled notifications are kept in tables TRIGGER_EVENT_STATUS_HANDLED_N4 and TRIGGER_EVENT_STATUS_HANDLED_the ECS, updated correspondingly by N4 and the ECS.

N4 and the ECS will poll the TRIGGER_EVENT_STATUS to determine when to read updates from the other tables. When N4 or the ECS reads the newly inserted or updated records, it will update the corresponding record in the TRIGGER_EVENT_STATUS_HANDLED table for the respective application, for example N4 will update the record for AGV_ORDERS_UPDATED to the corresponding SEQ handled in the TRIGGER_EVENT_STATUS.

Due to the size and frequency of access, TRIGGER_EVENT_STATUS_* tables remain completely in the database cache-memory, which allows for high frequent evaluation with minimal use of load on either db-server or db-client side. The TRIGGER_EVENT_STATUS_* tables allow multiple clients to evaluate the same notification in parallel. N4 polls this table about every 6 seconds.

Below is the true state of the availability flow in the data base, but for the remainder of this document the triggering mechanism for updates will be omitted. Additionally many Read actions will be omitted with the notes for N4 or the ECS attached to the Update or Insert of the record that the other system would read.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_STATUS	CHE_ID, OPERATIONAL_STA TUS = AUTOMATIC	ECS informs N4 that the AGV is available for orders.
Database	Update	TRIGGER_EVENT_S TATUS	UPDATED, SEQ, NAME = AGV_STATUS_UPDA TE	The database trigger updates the record for the AGV_STATUS_UPDA TE.
N4	Read	TRIGGER_EVENT_S TATUS	UPDATED, SEQ, NAME = AGV_STATUS_UPDA TE	N4 polls the TRIGGER_EVENT_S TATUS table.
N4	Read	AGV_STATUS	CHE_ID, OPERATIONAL_STA TUS = AUTOMATIC	N4 updates the CHE status to WORKING in N4.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	TRIGGER_STATUS_ HANDLED_SPARCS	UPDATED, SEQ, NAME = AGV_STATUS_UPDA TE	N4 acknowledges that the AGV_STATUS event has been handled.

13.1.3 Synchronization Points

The tables AGV_ORDERS, AGV_COMMANDS, ASC_ORDERS, ASC_COMMANDS, QC_ORDERS, QC_COMMANDS have columns called ORDER_VERSION and COMMAND_VERSION. Those columns are used to indicate process parity between the order and command entries. When an update is made to the order the value of ORDER_VERSION is incremented by 1, and similarly when an update is made to the command the value of COMMAND_VERSION is incremented by 1. To acknowledge that the update has been processed, the reader shall update its respective table to set the ORDER_VERSION/COMMAND_VERSION equal to the writer's value.

Although the maximum value of the ORDER_VERSION/COMMAND_VERSION columns is large, care should be taken not to create situations where the two systems endlessly loop in synchronization increments without any other significant data change.

13.1.4 Startup and Recovery Process

In the event of N4 startup after any period of downtime, N4 is responsible for reading and auditing the COMMANDS tables for each CHE in order to update container movements that have taken place during the downtime and to establish the current order state of each CHE.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	SPARCS_APPLICATION_STATUS	STATUS = Starting, START_TIME = <now>, INTERFACE_VERSION	N4 updates its application status when starting.
N4	Read	AGV_STATUS ASC_STATUS QC_STATUS	All fields	N4 updates CHE Status.
N4	Read	AGV_COMMANDS ASC_COMMANDS QC_COMMANDS	All fields	N4 reconciles inventory for unsolicited commands.
N4	Update	AGV_ORDERS ASC_ORDERS QC_ORDERS	All fields	N4 completes WA based on completed commands.
N4	Update	SPARCS_APPLICATION_STATUS	STATUS = Up, UPDATED = <now>	N4 updates its own application status

The ECS shall try to keep the ECS controlled tables up to date, regardless of the N4 status. Likewise, N4 shall maintain the QC_WORK_QUEUE_STATUS, WORK_INSTRUCTION_STATUS, BERTH_STATUS regardless of the the ECS application status. N4 shall dispatch and create AGV and ASC ORDERS with the latest equipment STATUS it has read.

Either system will consider the other system to be up when:

- Its respective application table STATUS (SPARCS_APPLICATION_STATUS or ECS_APPLICATION_STATUS) is Up
- Heartbeat (UPDATED) is within the last 30 seconds

In all other cases the other system is considered down.

When the ECS restarts, it must abort all new orders that it reads from the ASC_ORDERS and AGV_ORDERS table by marking the orders in progress as Aborted with problem type ECS_RESTART.

Moves performed by the ASC while the ECS was down must be reported in the same way unsolicited moves are reported. As the ECS connects again with QCs, ASC and AGVs, their status are reported in the tables QC_STATUS, AGV_STATUS and ASC_STATUS respectively.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	ECS_APPLICATION_STATUS	STATUS = Starting, START_TIME = <now>, INTERFACE_VERSION	The ECS updates its application status when starting.
ECS	Read	AGV_COMMANDS ASC_COMMANDS		The ECS reads the new ASC and AGV orders.
ECS	Update	AGV_COMMANDS ASC_COMMANDS	All fields	The ECS aborts all new orders.
ECS	Update	AGV_STATUS ASC_STATUS QC_STATUS	All fields	The ECS updates CHE Status
ECS	Update	ECS_APPLICATION_STATUS	STATUS = Up, UPDATED = <now>	The ECS updates its own application status

13.2 Equipment Interaction

In This Section

- Equipment Status. 402
- AGV Interaction. 403
- ASC Interaction. 409
- QC Interaction. 412
- Exceptions. 419

13.2.1 Equipment Status

The ECS maintains two tables to inform N4 of the current status of the equipment: the AGV_STATUS and ASC_STATUS tables. Each CHE has a reserved row within its CHE type table and each update for a single CHE updates only that record. N4 uses the OPERATIONAL_STATUS and TECHNICAL_STATUS to determine equipment availability for N4 scheduling. The ECS must provide an updated timestamp in the UPDATED column for every update to this table. The LOCATION column is updated in the AGV_STATUS table, even if the AGV has an OPERATIONAL_STATUS of OFFLINE. The same is also true of the ASC_STATUS LOCATION for ASCs. Equipment are available to N4 for dispatch when the OPERATIONAL_STATUS is AUTOMATIC and the TECHNICAL_STATUS is GREEN OR YELLOW.

13.2.1.1 Equipment Availability

The ECS informs N4 of the availability of the AGV. Additionally the ECS may update the LOCATION of the AGV in the AGV_STATUS record and N4 will update the AGV last known position.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_STATUS	CHE_ID, OPERATIONAL_STA TUS = AUTOMATIC, TECHNICAL_STATU S <> RED	ECS informs N4 that the AGV is available for orders. N4 updates the AGV status to WORKING in N4.
System	Action	Database Table	Relevant Fields	Notes
ECS	Update	ASC_STATUS	CHE_ID, OPERATIONAL_STA TUS = AUTOMATIC, TECHNICAL_STATU S <> RED	ECS informs N4 that the ASC is available for orders. N4 updates the ASC status to WORKING in N4.

When an AGV or ASC are no longer available for new orders, the ECS informs N4 by updating the row in the corresponding STATUS table to set the OPERATIONAL_STATUS to OFFLINE. This follows the same data communication flow as above, with N4 updating the status of the CHE to UNAVAILABLE.

When an ASC is parking, the ECS updates the OPERATIONAL_STATUS to OFFLINE and the LOCATION to the park location in the ASC_STATUS table.

13.2.1.2 Equipment Failure

When an AGV or ASC breaks down when performing an order:

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_STATUS	CHE_ID, TECHNICAL_STATU S = RED	ECS informs N4 that the AGV has an equipment failure. N4 updates the AGV technical status to RED.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	ASC_STATUS	CHE_ID, TECHNICAL_STATU S = RED	ECS informs N4 that the ASC has an equipment failure. N4 updates the ASC technical status to RED.

CHEs with a RED technical status will not be dispatched by N4. Users may try to reset the equipment in the ECS which may abort the current command.

13.2.1.3 Rack Suitability

N4 notifies the ECS of containers that are suitable for placement on racks by updating the RACK_SUITABLE column to YES in the WORK_INSTRUCTION_STATUS table. There is a RackSuitabilityValidationHandler that is extensible to determine what types of containers are suitable. There are dispatch validations for AGV to rack jobs, they must be lift AGV.

13.2.2 AGV Interaction

For AGV interactions N4 writes to the AGV_ORDERS, AGV_CONTAINER_UPDATE, and WORK_INSTRUCTION_STATUS tables, while the ECS writes to the AGV_COMMANDS, AGV_STATUS, DRIVE_TIME_STATUS, and TRANSFER_POINT_STATUS tables.

Figure 2 below shows the data transferred between N4 – ECS for AGV orders.

AGV Automation Database Design

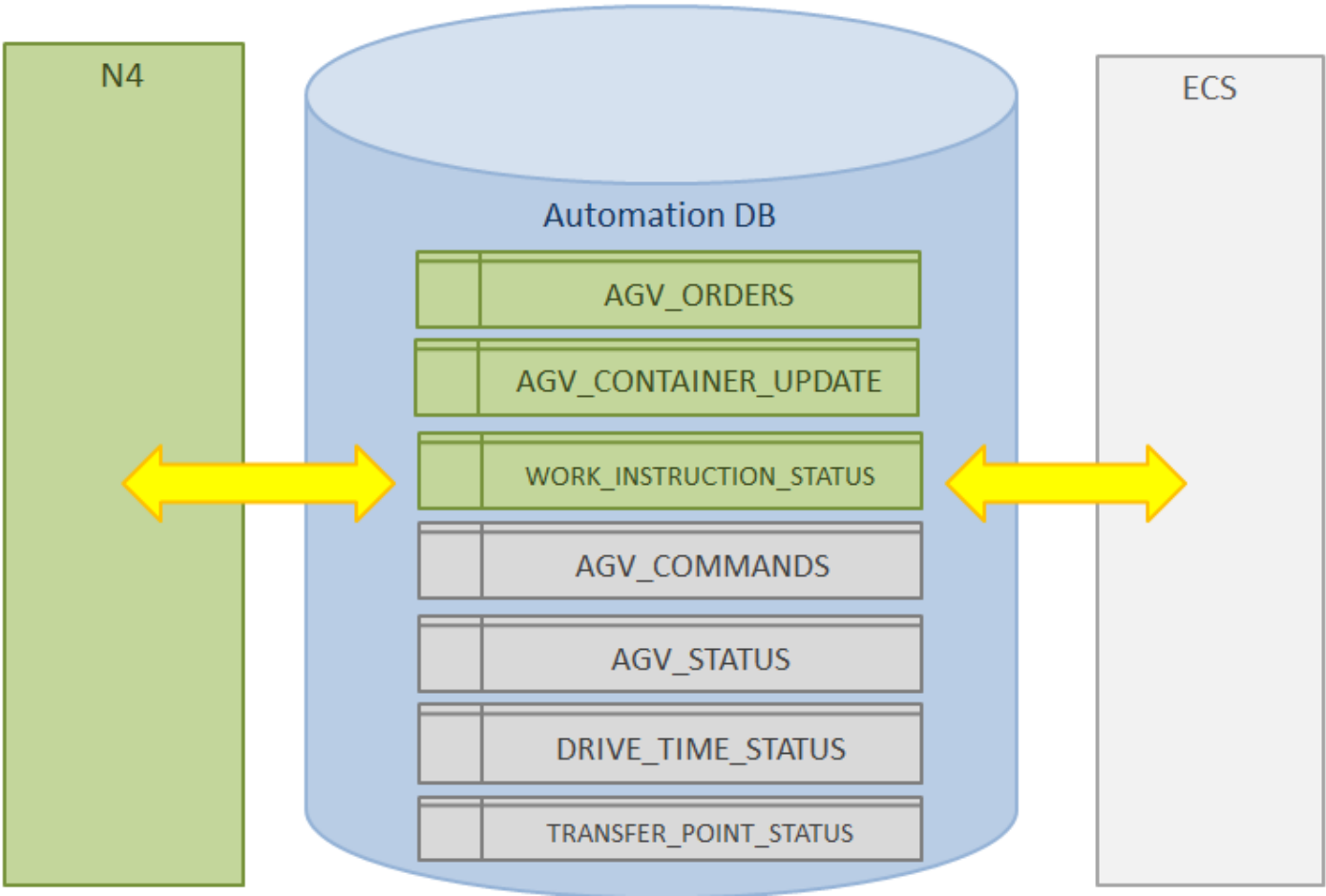


Figure 2: The N4-ECS data exchange for AGV.

13.2.2.1 Abstract AGV Interaction

The normal data flow for a container-related order is below:

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	AGV_ORDERS	CHE_ID, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the AGV to go to the PLANNED_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	AGV_COMMANDS	CHE_ID, STATUS = WORKING, ACTUAL_DESTINATION	The ECS validates the newly inserted AGV order. The ECS creates a new command. N4 updates the state of the AGV and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	AGV_COMMANDS	STATUS = COMPLETE, FINAL_LOCATION_1	The ECS completes the command. The COMMAND contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins) of the physical location of the container.
N4	Update	AGV_ORDERS	STATUS = COMPLETE	N4 updates the outstanding order to COMPLETE. N4 updates state in N4 related to this order being complete (move stages, inventory, etc.). N4 updates the AGV to IDLE and it is considered for scheduling and dispatching.

The above sequence is the general interaction between the ECS and N4. N4 dispatches half-cycle orders to the AGV, meaning for a particular container move, two half-cycle orders are needed to move the container: one order to direct the AGV to receive the container and another order to deliver the container.

Notes

- The ECS may update the status of the AGV in the AGV_STATUS table during command execution, such as LOCATION, BATTERY_STATE, UPDATED, etc.
- If this is a container-related order, N4 provides information about the related container in table WORK_INSTRUCTION_STATUS. As a general rule when a column in a database refers to a work instruction, N4 makes sure that the corresponding entry in the WORK_INSTRUCTION_STATUS table exists.

13.2.2.2 AGV Container Movement Interaction

The below interaction shows two half-cycle orders of the AGV to transport a container:

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, STATUS = ENTERED, AGV_ORDER_TYPE = RECEIVE, PLANNED_DESTINA TION	N4 dispatches an order to the AGV to go to the PLANNED_DESTINATION to RECEIVE CONTAINER_ID_1.
ECS	Insert	AGV_COMMANDS	CHE_ID, CONTAINER_ID_1, STATUS = WORKING, FINAL_LOCATION_1, ACTUAL_DESTINATI ON	The ECS validates the newly inserted AGV_ORDER. The ECS creates a new command. The COMMAND always contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins). N4 updates the state of the AGV and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	AGV_COMMANDS	STATUS = COMPLETE, CONTAINER_ID_1, FINAL_LOCATION_1	The AGV command is completed when the AGV is laden with CONTAINER_ID_1.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	AGV_ORDERS	STATUS = COMPLETE	N4 updates the container to be carry underway on the AGV. N4 updates the outstanding order to COMPLETE. The AGV is IDLE and available for laden dispatch, the ECS updates the AGV status to indicate there is no current order or command being executed by this AGV.
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, AGV_ORDER_TYPE = DELIVER, PLANNED_DESTINATION	N4 dispatches an order to the AGV to DELIVER CONTAINER_ID_1 to the PLANNED_DESTINATION.
ECS	Insert	AGV_COMMANDS	CHE_ID, CONTAINER_ID_1, STATUS = WORKING, FINAL_LOCATION_1, ACTUAL_DESTINATION	The ECS validates the newly inserted AGV_ORDER. The ECS creates a new command. The COMMAND always contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins). N4 updates the state of the AGV and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	AGV_COMMANDS	STATUS = COMPLETE, CONTAINER_ID_1, FINAL_LOCATION_1	The AGV command is completed when the AGV has delivered the container.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	AGV_ORDERS	STATUS = COMPLETE	N4 updates state in N4 related to the order being complete such as inventorying CONTAINER_ID_1 to FINAL_LOCATION_1. N4 updates the outstanding order to COMPLETE.

Notes

- When the AGV is receiving vessel discharge containers the CONTAINER_ID fields are empty, as the AGV is sent to the QC, and the QC will determine when the AGV has all containers and has completed its receive order.
- Orders that include container transfer will have WORK_QUEUE information provided in the AGV_ORDERS record.
- For twins this sequence of insert/updates will be repeated two times if an ASC is lifting the twins into the ASC stack from the AGV. These twins may be from two different blocks and involve two ASCs each lifting one of the twin pair.
- For orders where the AGV is receiving a container from the ASC, both the AGV and ASC commands COMPLETE when the container is placed on the AGV.
- The ECS may convert a planned rack transfer into a direct interchange if during the execution the AGV arrives in time, such that the ASC may set the container directly onto the AGV. When this happens the ECS will update the ACTUAL_DESTINATION of the ASC_COMMAND and AGV_COMMAND to notify N4 of this change, for example, updating the ACTUAL_DESTINATION from WSRACK-A to WSTP-C.
- At times AGVs may arrive late and N4 may swap jobs between the AGVs to avoid delays. N4 will notify the ECS of this by updating the destination of orders when the type is WSTP, for example, updating the PLANNED_DESTINATION in the AGV_ORDERS from WSTP.A to WSTP.B. The ECS acknowledges this by updating the AGV_COMMANDS PLANNED_DESTINATION to the updated location. The ECS always reports the ACTUAL_DESTINATION in the AGV_COMMANDS and this will reflect if the ECS was able to divert the AGV to the new location. If the ECS is unable to direct the AGV to the new location, the ACTUAL_DESTINATION will remain WSTP.A.
- After QC-AGV container exchange the ECS will clear the AGV from the QC area, even though there may not be a new order for the AGV. To do this, the ECS sends the AGV to the end of the QC cluster, updating the LOCATION of the AGV record in the AGV_STATUS table from QC TP to Quay.

13.2.2.3 AGV Drive Orders

When N4 issues drive orders to the AGV, N4 will insert an order into the AGV_ORDERS table with an empty CONTAINER_ID_1 and CONTAINER_WI_REF_1 of 0. The ECS may direct AGVs without orders from N4 for purposes of prepositioning or door turns, however N4 will evict AGVs if they are blocking a transfer point by issuing a REPOSITION drive order.

The below table describes the supported drive orders:

Order Type	Purpose
REPOSITION	N4 issues reposition orders to position the AGV to a new WSTP location during AGV eviction.

Order Type	Purpose
MAINTENANCE	When N4 users direct AGV to the maintenance area, N4 issues a maintenance order. The PLANNED_DESTINATION of the order will be the defined maintenance area in the yard. During maintenance, the ECS may update the AGV_STATUS LOCATION to NOT_ON_TERMINAL, which N4 will read and update the AGV's last known position to NULL.
X-RAY	N4 gives an order to the AGV to carry a container to an X-Ray Station. This is similar to a charging station. For X-ray orders the ECS completes the command when the AGV has performed the scan. N4 will complete the X-RAY order, though the AGV may still be driving, and will issue a new order to the AGV.
RECHARGE	N4 gives an order to the AGV to go to a charging station. The AGV may be taken offline while it is charging and its OPERATIONAL_STATUS in the AGV_STATUS table may be updated to OFFLINE. If AGVs can be charged in transfer zones and ECS provides charging state in TRANSFER_POINT_STATUS for transfer points which can charge, there should be no transfer zone associations defined in N4 with the group name RECHARGE. N4 will find possible destinations itself using information in TRANSFER_POINT_STATUS.

13.2.3 ASC Interaction

For ASC interactions N4 writes to the ASC_ORDERS and WORK_INSTRUCTION_STATUS tables, while the ECS writes to the ASC_COMMANDS, ASC_STATUS, and AREA_STATUS tables.

Figure 3 below shows the data transferred between N4 – ECS for ASC orders.

ASC Automation Database Design

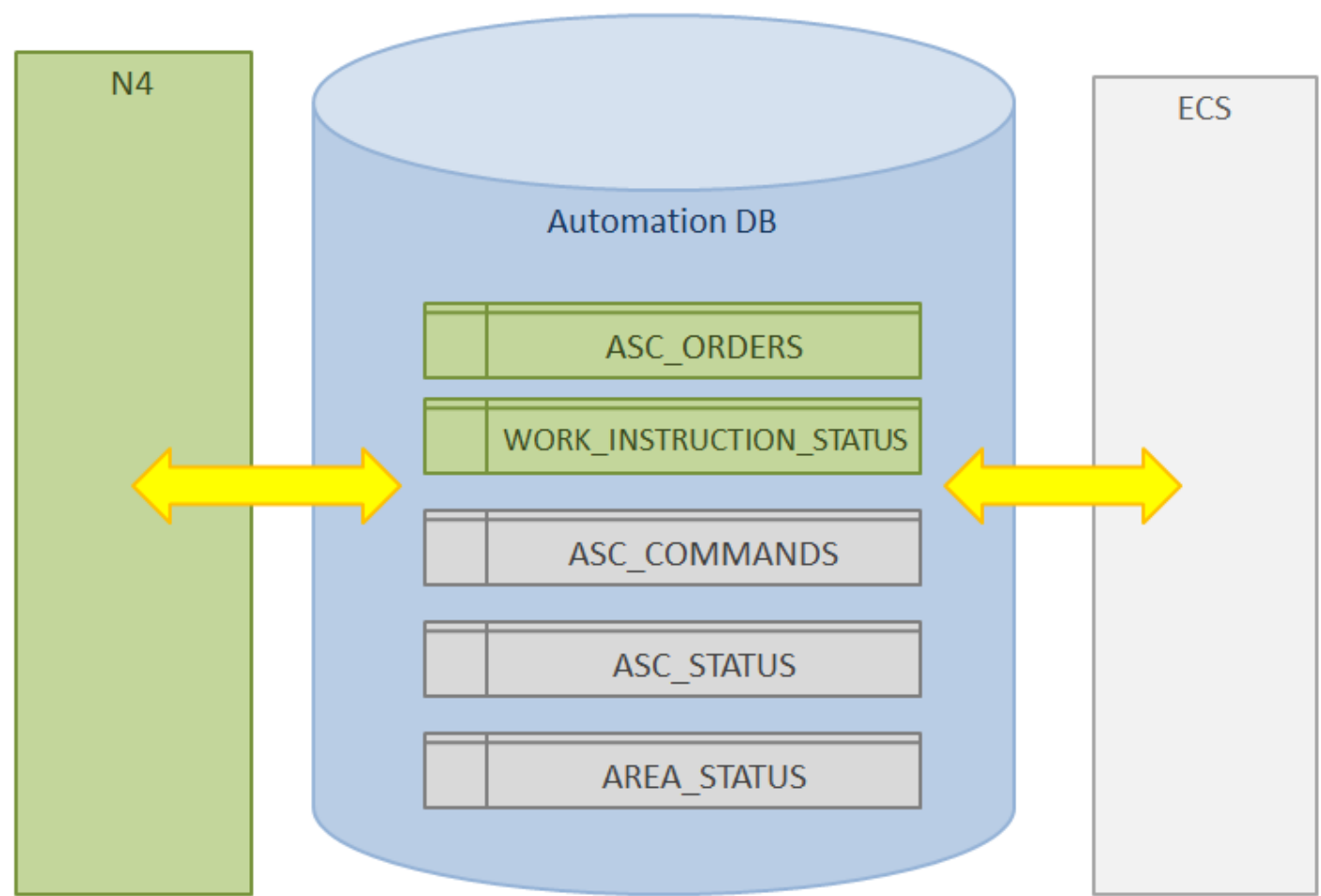


Figure 3: The N4-ECS data exchange for ASC.

ASC Container Movement Interaction

The below interaction shows a container movement for an ASC:

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	ASC_ORDERS	CHE_ID, CONTAINER_ID, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the ASC to go to move the CONTAINER_ID to the PLANNED_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	ASC_COMMANDS	CHE_ID, CONTAINER_ID, STATUS = WORKING, ACTUAL_DESTINATION	The ECS creates a new command. N4 updates the state of the ASC and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	ASC_COMMANDS	ORDER_PROGRESS	The ECS reports the ORDER_PROGRESS of the ASC. There may be multiple updates of the ORDER_PROGRESS.
ECS	Update	ASC_COMMANDS	TRANSFER_POINT_ENTRY	For a inbound move (from TZ to ASC Stowage), the ECS reports the time the ASC has entered the TZ. Once the TRANSFER_POINT_ENTRY field is populated, this field does not change.
ECS	Update	ASC_COMMANDS	LIFT_TIME	The ASC has lifted the container.
ECS	Update	ASC_COMMANDS	TRANSFER_POINT_ENTRY	For an outbound move (from ASC Stowage to TZ), the ECS reports the time the ASC has entered the TZ. Once the TRANSFER_POINT_ENTRY field is populated, this field does not change.
ECS	Update	ASC_COMMANDS	SET_TIME	The ECS may update the SET_TIME prior to completion of the job as job completion is usually considered once the spreader is hoisted.
ECS	Update	ASC_COMMANDS	STATUS = COMPLETE, CONTAINER_ID, ACTUAL_DESTINATION	The ASC command is completed. N4 inventories the container to the ACTUAL_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	ASC_ORDERS	STATUS = COMPLETE	N4 updates the outstanding order to COMPLETE. The ASC is IDLE and available for another dispatch, repeating this sequence.

Notes

- The ECS may update the status of the ASC in the ASC_STATUS table during command execution.
- The ECS may update the ORDER_PROGRESS of the command in the ASC_COMMANDS table, which N4 will read and update the Order Progress on the ASC's work assignment visible in the N4. The expected sequence of ORDER_PROGRESS updates is:
 - DRIVING_TO_PICK
 - LIFTING_AFTER_PICK
 - DRIVING_TO_DEPOSIT
 - LIFTING_AFTER_DEPOSIT
- N4 uses the ORDER_PROGRESS information to provide information to users. N4 only uses the LIFT_TIME to transition the ASC workflow to laden and the SET_TIME to transition the ASC to unladen.
- N4 may change the destination of an ASC order in progress by updating the PLANNED_DESTINATION of the order in the ASC_ORDERS table, and the ECS may acknowledge this update by updating the ACTUAL_DESTINATION of the command in the ASC_COMMANDS table.=
- For orders where the AGV is receiving a container from the ASC, both the AGV and ASC commands COMPLETE when the container is placed on the AGV.
- For inbound and outbound moves, the ECS should report a time in the TRANSFER_POINT_ENTRY field. The TRANSFER_POINT_ENTRY field should be populated just after the ASC has entered the TZ - either unladen or laden depending on the type of move. Once set, this field value should not change for any inbound or outbound move. For any intra-stack moves, the field should only ever be populated with a NULL value. The field is used by N4 to update the ASC's Job Step State to 'Laden At Row' for outbound moves and 'Unladen At Row' for inbound moves. N4 uses this Job Step State to make decisions about ASC and AGV interactions.
- No logic in N4 currently depends on or uses the TRANSFER_POINT_EXIT field value - as of N4 v3.4.

13.2.4 QC Interaction

The QC Interaction differs from the AGV and ASC interactions in that QC movements are not N4 directed but rather ECS initiated. N4 provides the ECS with the planned vessel load and discharge jobs for the next hour represented by a set of records in the WORK_INSTRUCTION_STATUS table, along with the QC_WORK_QUEUE_STATUS and BERTH_STATUS tables. The ECS updates the order progress of the QC as it handles containers. N4 provides the next hour of information in the WORK_INSTRUCTION_STATUS table, but if the QC handles a container that was not present, then N4 will insert a record into the WI Status for this.

Figure 4 below shows the data transferred between N4 – ECS for QC orders:

QC Automation Database Design

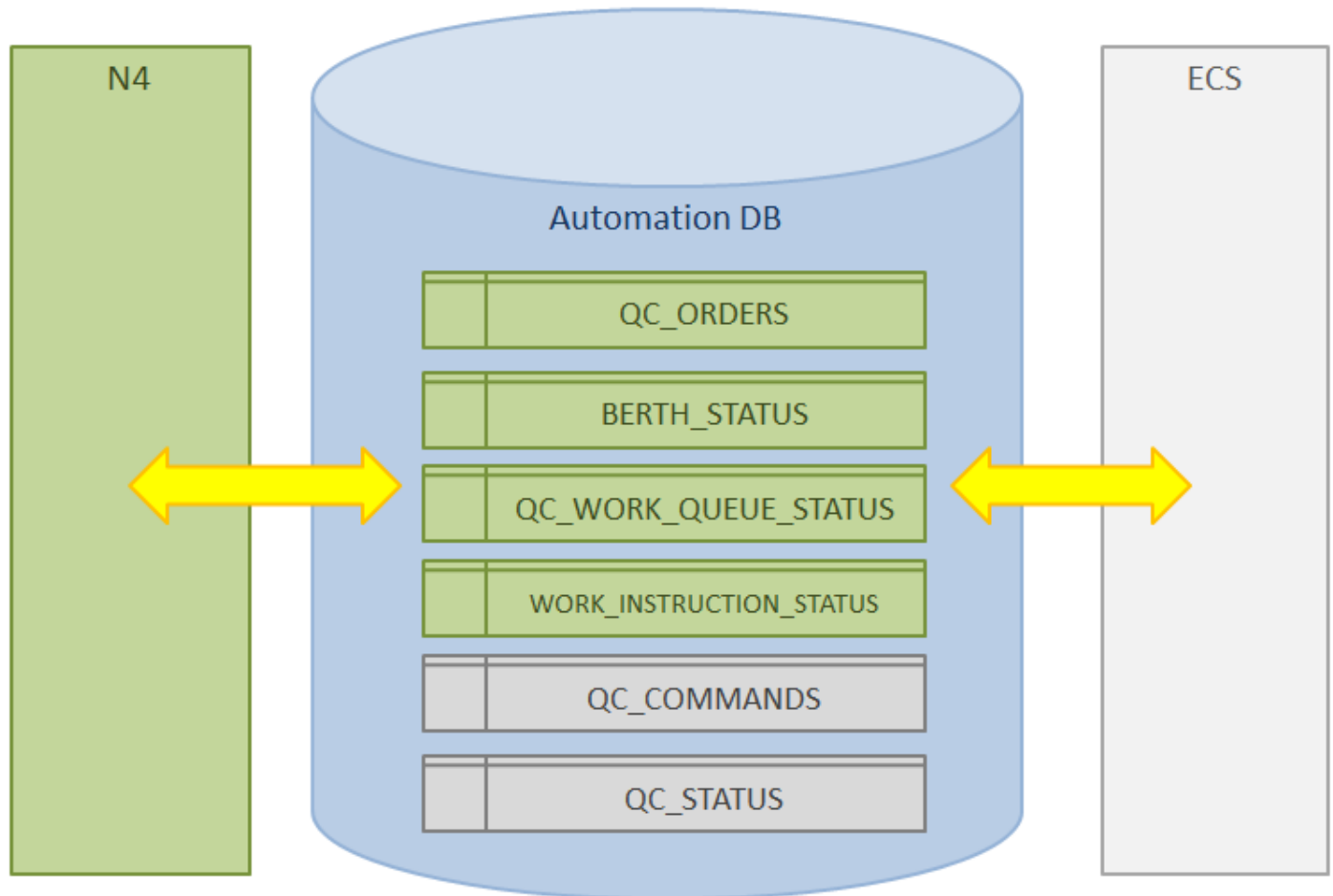


Figure 4: The N4-ECS data exchange for QC.

13.2.4.1 QC Container Movement Interaction

Below is the QC container movement interaction move, which must stop on the platform for identification, before being set to its final location. This is a typical discharge flow:

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	QC_COMMANDS	QC_ID, REFERENCE_ID, STATUS = WORKING, WORK_QUEUE, CONTAINER_ORIG_ LOCATION	The QC lifts a container. The ECS generates a REFERENCE_ID for the container. For container movements, where the container has not yet been identified, this reference will be used to reconcile container information upon identification.
ECS	Update	QC_COMMANDS	QC_ID, CONTAINER_LAST_ LOCATION, CONTAINER_CURR_ LOCATION	The QC sets the container onto the platform while it awaits identification.
N4	Insert	QC_ORDERS	QC_ID, REFERENCE_ID, STATUS = WORKING	N4 creates a corresponding order in the QC_ORDERS table with the same REFERENCE_ID from the command. Prior to container identification, there is no container information in the order.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	QC_ORDERS	QC_ID, REFERENCE_ID, CONTAINER_ID, DOOR_DIRECTION_ AT_QC, CONTAINER_WI_RE F, WA_REF, OCR_STATUS, TRANSFER_ALLOW ED	N4 receives container identification (either from an OCR system or manual entry) and updates the container information. N4 updates the TRANSFER_ALLOWED column of the order to notify the ECS that the transfer of this container is allowed per configured business rules. The time period between OCR trigger and the report to N4 may be short (when the container is known and the read is successful) or long (when a user has to process the images). The ECS can now reconcile the container in the command using the REFERENCE_ID.
ECS	Update	QC_COMMANDS	QC_ID, REFERENCE_ID, STATUS = WORKING, WORK_QUEUE, CONTAINER_ORIG_ LOCATION	The QC lifts this container.
N4	Update	QC_ORDERS	QC_ID, REFERENCE_ID, STATUS = WORKING	N4 creates a corresponding order for the QC.
N4	Update	QC_ORDERS	CONTAINER_ID, DOOR_DIRECTION_ AT_QC, CONTAINER_WI_RE F, WA_REF, TRANSFER_ALLOW ED	The container has been identified previously, N4 can match the corresponding container information in the order record.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	QC_COMMANDS	STATUS = COMPLETE, CONTAINER_ID, CONTAINER_CURR_LOCATION	The QC sets the container to its completion location: VESSEL, APRON, AGV, TRUCK, or TRUCK_AT_QC, apron, on a TT trailer, or on an AGV, from the WORK_INSTRUCTION_STATUS table CONTAINER_NEXT_QC_LOC_TYPE.
N4	Update	QC_ORDERS	STATUS = COMPLETE	N4 completes the corresponding order for the QC, completing the QC's work assignment. N4 inventories the container to the set location.

Notes

- If the QC is lifting two containers, then two QC_COMMANDS are entered for the lift, one for each container, and both records will share the same TRANSACTION_REF.
- The ECS updates the AGV command record in the AGV_COMMANDS table to STATUS=COMPLETE after all expected containers have been transferred between QC and AGV - in other words, any time after AGV_STATUS updates for the container(s) have been issued. It is also assumed the status is updated to COMPLETE after the QC_REFERENCE_ID column is filled as this is used to correlate data to identify the containers being handled.
- The OCR process will identify the following relevant attributes of the containers in the context of the N4-the ECS interface: CONTAINER_ID, CONTAINER_WI_REF, and DOOR_DIRECTION_AT_QC. The container attributes may come incrementally from the OCR system to N4. N4 will amend the information in the relevant QC_ORDERS entry once all three columns are known. The ECS may choose to first collect all the relevant attributes before updating the AGV load status. As a result partial information may be known to N4, but is then not yet visible to the ECS users. In the state that not all above attributes are known, the ECS is not ready to receive orders for an AGV where the container attributes are needed, so for instance for a deliver order. In case of twin loaded AGVs, the statement is true for both containers. the ECS will accept park orders for such an AGV. In this case the ECS may move the AGV away from the QC to await an order from N4 to deliver the container.
- If a container not intended for discharge is landed onto an AGV, then N4 will create the unit and work instruction. The WI Status table will have a record for this work instruction after identification. Then this is planned to EC, and treated as a normal discharge, requiring terminal staff to replan this to the Apron or vessel if necessary.
- When the QC moves a container, the ECS provides the updated position in the command. For vessel positions, the tier is not necessary, N4 will infer the tier based on the vessel inventory.
- When a container is on the platform, the ECS must track which records in WI_STATUS belong to which discharged containers on the platform. The NEXT_QC_LOC_TYPE can be determined directly from the WI_STATUS record. Further, the ECS needs to decide if and when a trolley can pick up this container and where it should move to. Subsequently the ECS needs to decide when the QC gets clearance to finalize this order.
- In case of tandem and quad loading, the ECS needs to know on which platform position the container should be placed. The ECS must infer the position on the platform from the intended positions on

RELATIVE_POS_ON_CARRIER and DOOR_DIRECTION_ON_CARRIER. N4 does not provide this mapping to platform positions, as this is the responsibility of the ECS.

- N4 will insert an order record anytime after the ECS has inserted a command for the QC, this may happen before the QC has set the container or after depending on the timing of the container movement.
- For vessel load moves, the ECS will provide the PRESUMED_CONTAINER_ID in the QC_COMMANDS.

13.2.4.2 Vessel Load Swapping

N4 supports the ECS performing vessel load swapping. This type of swapping can provide operational benefit by keeping the operation moving even if containers in earlier load sequence are not yet present in the parallel buffer. Sequencing matters only for AGV delivery orders to QC for load moves only.

The data involved in swapping decisions is in the WORK_INSTRUCTION_STATUS table: CONTAINER_WEIGHT_KG, LOGICAL_PREDECESSOR, PHYSICAL_PREDECESSOR, CONTAINER_STOW_FACTOR, and CONTAINER_WEIGHT_MARGIN_KG.

The container marked in the picture below is the physical predecessor to the container above and logical predecessor to the container on the right assuming loading is from left to right in the picture. The logical and physical predecessors define which containers and which AGV, shall be next at the QC. N4 provides the PHYSICAL_PREDECESSOR to ensure it containers that must be loaded first are respected by the ECS when bringing AGV out of order to the QC, for instance a container on tier 2 must have a container on tier 1 placed on the vessel first.

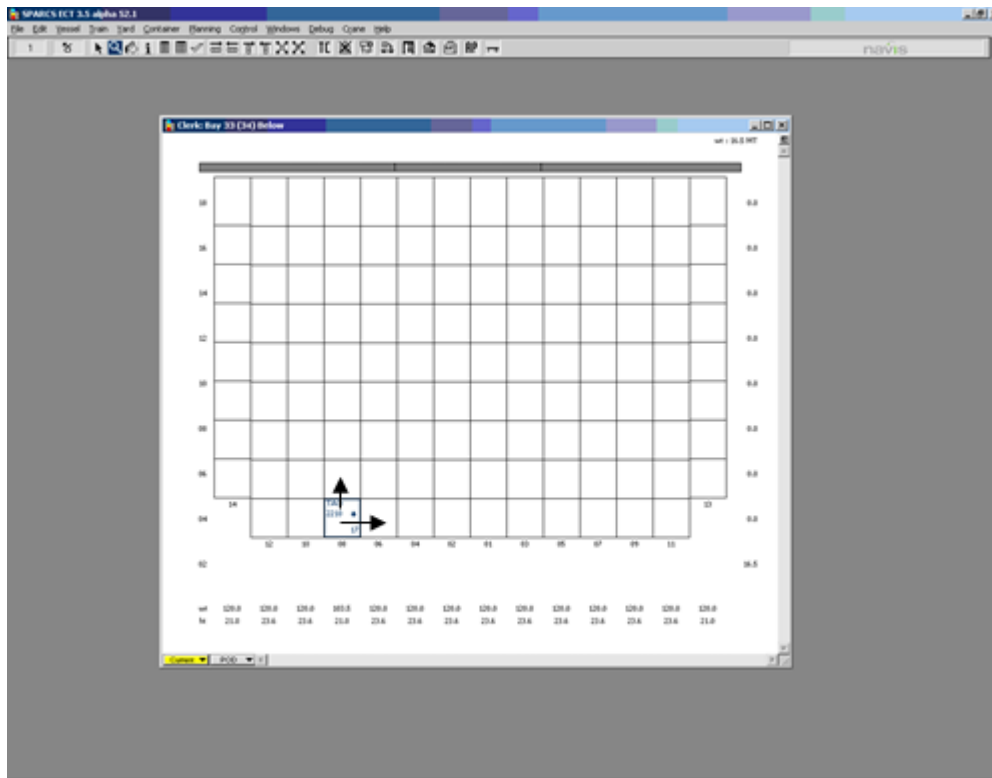


Figure 5: Illustration of physical and logical predecessors

Container 1 has stow factor CONTAINER_STOW_FACTOR1, weight CONTAINER_WEIGHT_KG1, and tolerance CONTAINER_WEIGHT_MARGIN_KG1 and Container 2 has stow factor CONTAINER_STOW_FACTOR2, weight

CONTAINER_WEIGHT_KG2, and Tolerance CONTAINER_WEIGHT_MARGIN_KG2, the containers can be swapped if:

- $\text{CONTAINER_STOW_FACTOR1} = \text{CONTAINER_STOW_FACTOR2}$
- $\text{CONTAINER_WEIGHT_KG1} - \text{CONTAINER_WEIGHT_MARGIN_KG1} < \text{CONTAINER_WEIGHT_KG2} < \text{CONTAINER_WEIGHT_KG1} + \text{CONTAINER_WEIGHT_MARGIN_KG1}$
- $\text{CONTAINER_WEIGHT_KG2} - \text{CONTAINER_WEIGHT_MARGIN_KG2} < \text{CONTAINER_WEIGHT_KG1} < \text{CONTAINER_WEIGHT_KG2} + \text{CONTAINER_WEIGHT_MARGIN_KG2}$

13.2.4.3 Vessel Work Information

When a vessel arrives or leaves, N4 updates the vessel positions and queue locations. Below are examples of updates N4 will write when a vessel is being actively worked and N4 will continue to update the records in the BERTH_STATUS and QC_WORK_QUEUE_STATUS.

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	BERTH_STATUS	VESSEL_VISIT_PHASE = ARRIVED	Vessel is at the quay.
N4	Update	BERTH_STATUS	VESSEL_VISIT_PHASE = WORKING	Work on the vessel starts.
N4	Update	BERTH_STATUS	BOW_BOLLARD, BOW_BOLLARD_OFFSET_CM STERN_BOLLARD, STERN_BOLLARD_OFFSET_CM	Vessel position update by N4
N4	Update	QC_WORK_QUEUE_STATUS	QC_BOLLARD, QC_BOLLARD_OFFSET_CM	Vessel ready, still at the quay.
N4	Update	BERTH_STATUS	VESSEL_VISIT_PHASE = COMPLETE	Vessel complete, still at quay
N4	Update	BERTH_STATUS	VESSEL_VISIT_PHASE = DEPARTED	Vessel leaves the quay

Notes

- If the vessel visit phase in N4 changes from Arrived or Working to Inbound or Cancelled, N4 will update the VESSEL_VISIT_PHASE to DEPARTED.

N4 will maintain the table QC_WORK_QUEUE_STATUS to reflect the planning of the different work queues. Below are examples of updates N4 will write as the work queue is being worked:

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	QC_WORK_QUEUE_STATUS	All fields, WORK_QUEUE=A, WQ_SEQ = 1, WQ_STATUS = WORKING	A new queue has come into the time horizon.

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	QC_WORK_QUEUE_STATUS	All fields, WORK_QUEUE=B, WQ_SEQ = 2, WQ_STATUS = WORKING	A new queue has come into the time horizon.
N4	Update	QC_WORK_QUEUE_STATUS	WORK_QUEUE=A, WQ_STATUS = EMPTY, WQ_SEQ=1	The last container of work queue A is placed on the AGV or vessel.

Notes

- A queue is considered finished when the last container is placed on AGV for a discharge queue, or when the last container is placed on the vessel for a load queue, or when all the remaining moves in the queue have the status BYPASS in N4.

13.2.5 Exceptions

The ECS updates the STATUS = REJECTED or STATUS = ABORTED in the AGV_COMMANDS or ASC_COMMANDS table to indicate an error to N4. The ECS should reject orders that it cannot accept and abort commands when unrecoverable issues are encountered during the execution of an order. Rejected and Aborted orders are finalized states that require no additional action from N4 on the same order. N4 will issue new orders to the equipment.

Additional information is sent to N4 by updating the PROBLEM_CODE in the AGV_COMMANDS or the ECS_PROBLEM_TYPE in the ASC_COMMANDS. N4 exception handling can be customized by overwriting the handlers for these exception codes.

13.2.5.1 AGV Exceptions

PROBLEM_CODE	Description	STATUS
VALIDATION_ERROR	Generic error for ECS unable to accept the order.	REJECTED

PROBLEM_CODE	Description	STATUS
AGV_NOT_AVAILABLE	The ECS cannot accept the AGV order, because the AGV is non-existing, unavailable in N4, or not in operation. N4 will suspend auto dispatch on this AGV if this error is encountered while the AGV is laden. N4 does not bypass the WI, such that it remains dispatchable to another AGV. Laden abort of this needs to be investigated by operational staff and may involve sending staff out to the yard to service the AGV. As the AGV is offline, nothing more can be done from N4 other than fixing the state of the WI and/or container.	REJECTED
DESTINATION_UNREACHABLE	A change in the topology has made the destination unreachable. This is the result of an area becoming unavailable, via temp block or men working, and no alternate route being found. If the AGV was laden and received this area, N4 will replan to a new location and laden dispatch the move. If the move cannot be redispach, then the WI is bypassed and auto dispatch is suspended on this AGV. A move that cannot be re-dispatched, is a move that cannot have a new viable plan. Probably loads cannot be redispached as they need to proceed to the same QC.	ABORTED
EXECUTION_FAILED	Generic code for ASC orders that cannot be executed.	ABORTED

AGV Rejection

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the AGV to go to move the CONTAINER_ID_1 to the PLANNED_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_COMMANDS	STATUS = REJECTED, PROBLEM_CODE, PROBLEM_DESCRIPTION = <Free Text>	The ECS validates and rejects the newly inserted AGV_ORDERS record.
N4	Read	AGV_COMMANDS	STATUS = REJECTED, PROBLEM_CODE	N4 bypasses this WI.

AGV Unladen Abort

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, STATUS = ENTERED, AGV_ORDER_TYPE = RECEIVE, PLANNED_DESTINATION	N4 dispatches an order to the AGV to go to the PLANNED_DESTINATION to RECEIVE CONTAINER_ID_1.
ECS	Insert	AGV_COMMANDS	CHE_ID, CONTAINER_ID_1, STATUS = WORKING, FINAL_LOCATION_1, ACTUAL_DESTINATION	The ECS validates the newly inserted AGV_ORDER. The ECS creates a new command. The COMMAND always contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins). N4 updates the state of the AGV and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	AGV_COMMANDS	STATUS = ABORTED, PROBLEM_CODE, PROBLEM_DESCRIPTION = <Free Text>	The AGV command is aborted.

System	Action	Database Table	Relevant Fields	Notes
N4	Update	AGV_ORDERS	STATUS = ABORTED	N4 bypasses this WI, letting a dispatcher take corrective action. N4 finalizes this WI, the ASC is now available for scheduling and dispatch consideration.
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, STATUS = ENTERED, AGV_ORDER_TYPE = RECEIVE, PLANNED_DESTINATION	N4 dispatches a new order to the AGV.

AGV Laden Abort

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, AGV_ORDER_TYPE = DELIVER, PLANNED_DESTINATION	N4 dispatches an order to the AGV to DELIVER CONTAINER_ID_1 to the PLANNED_DESTINATION.
ECS	Insert	AGV_COMMANDS	CHE_ID, CONTAINER_ID_1, STATUS = WORKING, FINAL_LOCATION_1, ACTUAL_DESTINATION	The ECS validates the newly inserted AGV_ORDER. The ECS creates a new command. The COMMAND always contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins). N4 updates the state of the AGV and instruction in N4 to indicate the dispatch has been accepted by the ECS.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_COMMANDS	STATUS = ABORTED, PROBLEM_CODE, PROBLEM_DESCRIPTION = <Free Text>	The AGV command is aborted
N4	Update	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, AGV_ORDER_TYPE = DELIVER, PLANNED_DESTINATION	If the PROBLEM_CODE is DESTINATION_UNREACHABLE N4 will dispatch a new order to the AGV to deliver the container to a new destination. For all other PROBLEM_CODE types, a user must manually resolve this situation.

AGV Delays

The ECS may notify N4 of AGV delays by updating the PROBLEM_DESCRIPTION to one of two delay causes:

- WAITING_FOR_ROUTE: The area is blocked and the ECS must find an alternative route for the AGV.
- WAITING_FOR_SEQUENCE: The AGV must wait in the parallel buffer because the AGV cannot yet deliver the load to the QC (out of order vessel loads cannot swap).

Below is the data exchange when an AGV is rerouted because of a long term blocked area:

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	AGV_ORDERS	CHE_ID, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the AGV to go to the PLANNED_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	AGV_COMMANDS	CHE_ID, STATUS = WORKING, FINAL_LOCATION_1, ACTUAL_DESTINATION	The ECS validates the newly inserted AGV_ORDER. The ECS creates a new command. The COMMAND always contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins). N4 updates the state of the AGV and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	AGV_COMMANDS	STATUS = WORKING, PROBLEM_DESCRIPTION = WAITING_FOR_ROUTE, ETA=NULL	The ECS is finding an alternative route for the AGV around an obstacle.
ECS	Update	AGV_COMMANDS	STATUS = WORKING, PROBLEM_DESCRIPTION = NULL, ETA <> NULL	The ECS has found a new route and updated the ETA.
ECS	Update	AGV_COMMANDS	STATUS = COMPLETE	The ECS completes the command.

13.2.5.2 ASC Exceptions

From the fields LIFT_TIME and SET_TIME N4 can derive the container in spreader state in case the order was aborted.

The CRANE_ABORT_CODE can be used to denote different abort cases where the PROBLEM_TYPE is the same, allowing N4 groovy customizations based on different identified cases.

ECS_PROBLEM_TYPE	Description	STATUS
ASC_NOT_AVAILABLE	The ECS cannot accept the AGV order, because the AGV is non-existing, unavailable in N4, or not in operation.	REJECTED
VALIDATION_ERROR	Generic code for ASC order that cannot be accepted.	REJECTED

ECS_PROBLEM_TYPE	Description	STATUS
ABORTED_BY_OPERATOR	This is a user generated error when the ASC is in semi-auto mode.	ABORTED
BLOCKMAP_ERROR	This is the result of inventory synchronization issue, meaning the ECS does not detect a container here. N4 will temp block the origin or destination if this error is received unladen or laden.	ABORTED
REEFER_ON_POWER	Can be used to issue disconnect orders from N4 via groovy customizations.	ABORTED
VALIDATION_ERROR	Generic code for ASC orders that cannot be executed.	ABORTED
ECS_RESTART	ECS Restart, outstanding orders are aborted when the ECS comes back online.	ABORTED

ASC Rejection

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	ASC_ORDERS	CHE_ID, CONTAINER_ID, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the ASC to go to move the CONTAINER_ID to the PLANNED_DESTINATION.
ECS	Update	ASC_COMMANDS	STATUS = REJECTED, CRANE_ABORT_CODE, CRANE_ABORT_DESCRIPTION = <Free Text>	The ECS validates and rejects the newly inserted ASC_ORDERS record. N4 bypasses this WI.

ASC Unladen Abort

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	ASC_ORDERS	CHE_ID, CONTAINER_ID, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the ASC to go to move the CONTAINER_ID to the PLANNED_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	ASC_COMMANDS	CHE_ID, CONTAINER_ID, STATUS = WORKING, ACTUAL_DESTINATION	The ECS validates the newly inserted ASC_ORDERS record. The ECS creates a new command. N4 updates the state of the ASC and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	ASC_COMMANDS	STATUS = ABORTED, CRANE_ABORT_CODE, CRANE_ABORT_DESCRIPTION = <Free Text>, LIFT_TIME = NULL	ECS aborts the order. N4 infers an unladen abort as the LIFT_TIME is still NULL. N4 bypasses this WI, letting a dispatcher take corrective action.
N4	Update	ASC_ORDERS	STATUS = ABORTED	N4 finalizes this WI, the ASC is now available for scheduling and dispatch consideration.
N4	Insert	ASC_ORDERS	STATUS=ENTERED	N4 will dispatch a new order for a different container to the ASC.

ASC Laden Abort

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	ASC_ORDERS	CHE_ID, CONTAINER_ID, STATUS = ENTERED, PLANNED_DESTINATION	N4 dispatches an order to the ASC to go to move the CONTAINER_ID to the PLANNED_DESTINATION.

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	ASC_COMMANDS	CHE_ID, CONTAINER_ID, STATUS = WORKING, ACTUAL_DESTINATION	The ECS validates the newly inserted ASC_ORDER. The ECS creates a new command. N4 updates the state of the ASC and instruction in N4 to indicate the dispatch has been accepted by the ECS.
ECS	Update	ASC_COMMANDS	LIFT_TIME	The ASC has lifted the container. N4 updates the work instruction associated with the container to carry underway.
ECS	Update	ASC_COMMANDS	STATUS = ABORTED, CRANE_ABORT_CODE, CRANE_ABORT_DESCRIPTION = <Free Text>	ECS aborts the order. N4 infers a laden abort as the LIFT_TIME is not NULL.
N4	Update	ASC_ORDERS	STATUS = ABORTED	N4 finalizes this order and the ASC work assignment, the ASC is now available for scheduling and dispatch consideration.
N4	Insert	ASC_ORDERS	CHE_ID, CONTAINER_ID, STATUS = ENTERED, ORIGIN = SPREADER, PLANNED_DESTINATION	N4 will dispatch a new order for the ASC to deposit the container from its spreader to a new destination.

13.2.5.3 Corrective Actions

N4 supports various corrective actions for ASC and AGVs:

- cancellations for ASC and AGV
- unsolicited ASC moves
- AGV state corrections
- AGV diverts.

Cancellations

N4 may request the ECS to cancel issued orders by updating the order status to CANCEL. When the ECS honors this cancellation the command STATUS is updated to CANCEL. The threshold for when to respect a cancellation from N4 is defined by the ECS vendor. Often if the order is about to complete, the ECS cannot honor the cancel request, as it is past the cancellation threshold.

AGV Cancellation

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_COMMANDS	CHE_ID, STATUS = WORKING	The AGV is executing the command.
N4	Update	AGV_ORDERS	STATUS = CANCEL	N4 cancels the order.
ECS	Update	AGV_COMMANDS	STATUS = CANCEL	The ECS cancels the command. N4 finalizes the AGV work assignment.

ASC Cancellation

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	ASC_COMMANDS	CHE_ID, STATUS = WORKING	The ASC is executing the command.
N4	Update	ASC_ORDERS	STATUS = CANCEL	N4 cancels the order.
ECS	Update	ASC_COMMANDS	STATUS = CANCEL	The ECS cancels the command. N4 finalizes the AGV work assignment.

Notes

- N4 users should cancel AGV orders involved in container transfer when an ASC order is canceled, else the AGV will wait indefinitely.

Unsolicited Moves

The ECS may perform unsolicited moves when the crane is on or offline, meaning ASC operations may take place without the direction of N4. In this circumstance, the ECS application shall record all container movements to the ASC_COMMANDS table without ORDER_GKEYs. ASC Unsolicited scenarios are supported by N4, and no exceptions should be thrown. N4 will create an ASC Unsolicited move. Any inventory update will be ignored as the container is not being moved to a new stowage position.

Note that Unsolicited AGV commands are not supported in the Automation Database. If a container has been placed on an AGV or removed from it using means beyond Automation Database interface (for example, a straddle repositioned container(s) from a broken AGV onto a different AGV), N4 will inform the ECS by inserting a new row in AGV_CONTAINER_UPDATE.

If the ASC picks up a container without a corresponding order from N4, the ECS creates an entry in the ASC_COMMANDS table, with the location data, container data if available. N4 creates a matching ASC_ORDERS entry and inventories the container.

Inbound and Outbound ASC Unsolicited moves are supported. However, an AGV may not be displayed unladen or

laden following the move. That is, the ASC will complete the move to or from the AGV, but the AGV will not take further action. This is not true for TTs and OTRs. If an ASC drops a container on a TT or OTR, then the TT will proceed laden or the OTR will proceed to start to Gate Out. It is the same for inbound moves when a TT or OTR is involved.

System	Action	Database Table	Relevant Fields	Notes
ECS	Insert	ASC_COMMANDS	CHE_ID, STATUS = WORKING, CONTAINER_ID, ORDER_GKEY = NULL, ORDER_VERSION = NULL, LIFT_TIME = <pickup time>	The ASC lifts a container without an order from N4.
N4	Insert	ASC_ORDERS	Status=WORKING, CHE_ID, COMMAND_GKEY, ORDER_GKEY, ORDER_VERSION	N4 inserts an order and creates a work assignment for the ASC.
ECS	Update	ASC_COMMANDS	STATUS = COMPLETE, CHE_ID, ACTUAL_DESTINATION = <set location>, SET_TIME	The ASC sets the container.
N4	Update	ASC_ORDERS	STATUS = COMPLETE, ACTUAL_DESTINATION = <set location>	N4 finalizes the AGV work assignment. N4 inventories the container to the set location.

Following an ASC Abort and the ASC is OFFLINE a remote operator may set down a container without an order from N4::

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	ASC_COMMANDS	Status=ABORTED, Che_id, Container_Id Lift time = <time of pickup>, Set time = NULL	The ECS aborts the order leaving CONTAINER_ID on the SPREADER.
ECS	Insert	ASC_COMMANDS	Status=COMPLETE, Che_id, Origin = ASCSPREADER.<asc cid>, Actual destination = <location of setdown>, Lift time = NULL, Set time = <time of setdown>, Transfer point entry NULL, Transfer point exit = NULL	N4 inserts an order and creates a work assignment for the ASC.

System	Action	Database Table	Relevant Fields	Notes
N4	Insert	ASC_ORDERS	STATUS = COMPLETE, CHE_ID, ACTUAL_DESTINATION = <set location>, SET_TIME	N4 finalizes the AGV work assignment. N4 inventories the container to the set location.

AGV State Correction

Container <Container id> removed from AGV outside of the ECS. An N4 user is correcting this state by updating the container's position in the yard.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_STATUS	CHE_ID, CONTAINER_ID1 = <Container ID>	AGV is currently laden
N4	Insert	AGV_CONTAINER_UPDATE	CHE_ID, CONTAINER_ID = <Container ID> CONTAINER_STATE = UNLOADED	The position of the container updates in N4 to a location <> the AGV
ECS	Update	AGV_STATUS	CHE_ID, CONTAINER_ID1 = Null	AGV is currently unladen

Notes

- The ECS will remove the container based on the CHE_ID, irrespective of the location on the AGV.

Container <Container id> placed on AGV with id <AGV id> outside of the ECS. An N4 user is correcting this state by updating the AGV to laden in N4.

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_STATUS	CHE_ID, CONTAINER_ID1 = Null	AGV is currently unladen
N4	Insert	AGV_CONTAINER_UPDATE	CHE_ID, CONTAINER_ID = <Container ID> CONTAINER_STATE = LOADED, LOCATION = on AGV	N4 user places container onto the AGV
ECS	Update	AGV_STATUS	CHE_ID, CONTAINER_ID1 = <Container ID>	AGV is currently laden

AGV Divert

Deliver Load Out of Sequence

When an AGV is waiting in the parallel buffer to deliver a load container, N4 users can request the ECS to ignore the vessel load swap sequence restrictions:

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_STATUS	CHE_ID, LOCATION = <Parallel Buffer>	N4 dispatches an order to the ASC to go to move the CONTAINER_ID to the PLANNED_DESTINATION.
ECS	Update	AGV_COMMANDS	STATUS = WORKING	The AGV is currently waiting at the parallel buffer with a container.
N4	Update	AGV_ORDERS	CHE_ID, BYPASS_SEQUENCE = YES	N4 directs the AGV to the QC out of sequence.

Deliver to a Stowage Block

In the event that a container has been landed on an AGV, but the AGV needs to be diverted to a holding area. Users achieve this by resetting the AGV, updating the container's plan to this holding area, and laden dispatching the AGV. Once the AGV arrives at this holding area, which may be a reachstacker block with a defined TP, a reachstacker or other manual CHE may remove the container from the AGV. This container removal from the AGV follows the AGV State Correction interaction.

To the AutoDB interface, this process to divert an AGV to a stowage block appears as an order cancel followed by a new order:

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AGV_COMMANDS	CHE_ID, CONTAINER_ID1 = <Container ID>, STATUS = WORKING	The AGV is currently laden executing a delivery command.
N4	Update	AGV_ORDERS	STATUS = CANCEL	N4 cancels the order.
ECS	Update	AGV_COMMANDS	STATUS = CANCEL	The ECS cancels the command. N4 finalizes the AGV work assignment.
N4	Insert	AGV_ORDERS	CHE_ID, CONTAINER_ID_1, STATUS = ENTERED, AGV_ORDER_TYPE = RECEIVE, PLANNED_DESTINATION	N4 dispatches a new order to the AGV to the stowage block.

13.3 Location Status

The ECS informs N4 about vehicle types allowed and charging status in the TRANSFER_POINT_STATUS table. The AREA_STATUS table provides availability for dispatch to yard areas, including transfer points.

In This Section

Area Status.	433
Transfer Point Status.	433

13.3.1 Area Status

ASC crane systems offer the option to block parts of the yard block or the transfer points. If so, they report the blocked positions to the ECS. the ECS in turn will inform N4. The information is per individual yard slot or TP. N4 should not send new orders to the areas with an unavailable status.

The ECS notifies N4 when TP are made unavailable by setting the AREA for the TP in the AREA_STATUS table to UNAVAILABLE. When this happens N4 will not send a new AGV or ASC order to the TP.

In between the ECS setting the unavailable status and N4 processing that information a new AGV order may be sent. In that case the AGV will not be sent to the destination and the order will be delayed with the PROBLEM_CODE in AGV_COMMANDS set to WAITING_FOR_ROUTE. After some time, the ECS will update this to DESTINATION_UNREACHABLE and initiate N4 abort processing.

The ECS allows current N4 directed AGV/ASC orders complete and leave the TP. If there is an idle AGV without an order on the TP, N4 will send it away. The ECS will send away AGV that are under ECS control from this TP.

Location Unavailable

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AREA_STATUS	AREA, STATUS = 0	AREA notes which slot in the ASC block or ASC TP is unavailable.

Location Available

System	Action	Database Table	Relevant Fields	Notes
ECS	Update	AREA_STATUS	AREA, STATUS = 1	AREA notes which slot in the ASC block or ASC TP is unavailable.

13.3.2 Transfer Point Status

The TRANSFER_POINT_STATUS table reports status for the following types of locations, though not all of these are locations where container transfers may take place:

- WSTP: Waterside transfer point for direct interchange
- WSRACK: Waterside transfer points for rack interchange
- LSTP: Landside transfer points for direct interchange
- LSRACK: Landside transfer points for rack interchange
- CLTP: Cantilever transfer point for direct interchange
- CLRACK: Cantilever transfer point for rack interchange
- MAINT: Maintenance points by individual lane
- RECH: Battery exchange lanes
- X-RAY: X-Ray stations by individual lane
- YARD: Stowage blocks used for transfer to manual equipment like reach stackers

N4 only consumes the LANE_TYPE and IS_CHARGING information in the TRANSFER_POINT_STATUS table. The ECS may use this table to provide visibility to users of the transfer point usage via the CLAIM_OWNER and ASSIGNABLE when a transfer point is being used as an intermediate location for prepositioning or a door turn, however N4 does not consume the ASSIGNABLE information for determining availability as this information should be transient and not on the same scale as the time frames in which N4 plans.

The LANE_TYPE column in the TRANSFER_POINT_STATUS table determines the moves allowed in this transfer point. To ensure consistent move kinds allowed by N4 the LANE_TYPE value should default to "TOS". If other values for LANE_TYPE are used then the move kinds allowed will be overridden:

LANE_TYPE	Move Kinds Allowed
UTR	YARD, RLOD, RDSCH
OTR	RECV, DLVR
AGV	YARD, LOAD, DSCH
TOS	N4 controls access via Transfer Point Usage Move Kinds Allowed.

13.4 Database Interface Definition

In This Section

- Datatypes. 436
- N4 Orders. 436
- N4 Status. 447
- ECS Commands. 455
- ESC Status. 465
- Notification and Startup. 476
- Datatypes. 478

13.4.1 Datatypes

This interface definition uses the following data types:

Data Type	SQL Server	Oracle	Notes
Number(I2)	smallint	SMALLINT	Signed 16 bit integer
Number(I4)	int	INTEGER	Signed 32 bit integer
Number(I8)	bigint	LONGINTEGER	Signed 64 bit integer
Float(I8)	float(53)	FLOAT(53)	Floating point 64 bit number with a double precision
String(n)	varchar(n)	VARCHAR2(n)	String with a maximum length n
Date	datetime	DATE	Date columns have a NULL value when there is not meaningful data For example when it represents the estimated time of arrival but the destination becomes unreachable.

13.4.2 N4 Orders

13.4.2.1 AGV_ORDERS

Column Name	Data Type	Description
ORDER_GKEY	Number(I8)	Order sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 863
COMMAND_GKEY	Number(I8)	Command sequence. Indexed field. Can be NULL when order is created, until ECS response. Example: 863
ORDER_VERSION	Number(I4)	Synchronization point number. Should increment with each AGV_ORDERS update. Default = 1. Example: 1

Column Name	Data Type	Description
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each AGV_COMMANDS update. Can be NULL when order is created, until ECS response. Example: 1
CHE_ID	String(7)	ID of AGV being issued the order. Indexed field. Example: AGV03
STATUS	See AGV/ASC order status	AGV order status Example: ENTERED
PROBLEM_DESCRIPTION	String(1000)	Free text description of a problem on N4 side
PLANNED_DESTINATION	See Container and equipment locations	Location where AGV should arrive for container transfer or for other purpose (if CONTAINER_ID_1 is blank). The 40 ft location name value is always used for this, because an AGV occupies a 40 ft location, Can be NULL if AGV_ORDER_TYPE = NEAREST_PARKING.
AGV_ORDER_TYPE	String(25)	AGV Order type String(25) Values: REPOSITION, DELIVER, RECEIVE, DUAL_CYCLE, RECHARGE, MAINTENANCE, X-RAY, NEAREST_PARKING. DUAL_CYCLE this implies a full cycle order, but is only used for the deliver part. That will prevent the AGV from departing automatically after the delivery has been made. The receive part of the order still requires its own RECEIVE order. N4 does not support NEAREST_PARKING or DUAL_CYCLE values at this time. These are reserved for future use NEAREST_PARKING is used when N4 wants ECS to send the AGV to the nearest parking location.
WORK_QUEUE	String(60)	Work queue name. Can be NULL for orders which do not involve container transfer.

Column Name	Data Type	Description
POS_ON_AGV_1	String(1)	The (planned/current) position of CONTAINER_ID_1 on the AGV. Valid value if available will be populated for sites using auto trucks. 'U' will be populated for sites using AGVs. Possible value for this field will be F ("front"), R ("rear"), C ("center") or U ("unknown"). Used by the ECS to decide the driving direction to the block and QC. A value is populated only for receival order from a block, and for delivery order to the block and the QC. When the order is to receive a container - the position on the CHE the container is planned to be placed on the AGV. When the order is to receive a container - the actual position of the containers on the AGV. Positions on the AutoTrucks follow the same convention as UTRs (i.e. uses 1, 2 or 3), but since AGV_ORDERS table is used for both AGVs and Auto Trucks, in this table we decided to use the convention used for AGVs F ("front"), R ("rear"), C ("center") or U ("unknown").

Column Name	Data Type	Description
POS_ON_AGV_2	String(1)	The (planned/current) position of CONTAINER_ID_2 on the AGV. Valid value if available will be populated for sites using auto trucks. 'U' will be populated for sites using AGVs. Possible value for this field will be F ("front"), R ("rear"), C ("center") or U ("unknown"). Used by the ECS to decide the driving direction to the block and QC. A value is populated only for receipt order from a block, and for delivery order to the block and the QC. When the order is to receive a container - the position on the CHE the container is planned to be placed on the AGV. When the order is to receive a container - the actual position of the containers on the AGV. Positions on the AutoTrucks follow the same convention as UTRs (i.e. uses 1, 2 or 3), but since AGV_ORDERS table is used for both AGVs and Auto Trucks, in this table we decided to use the convention used for AGVs F ("front"), R ("rear"), C ("center") or U ("unknown").
CONTAINER_ID_1	String(12)	ID of container to be moved. Indexed field. If CONTAINER_ID_2 also has a value, CONTAINER_ID_1 has lower sequence in the queue compared to CONTAINER_ID_2. Is NULL for vessel discharge receipt orders or a drive order. Example: MSKU1234567
CONTAINER_WI_REF_1	Number(18)	Matches CONTAINER_WI_REF in WORK_INSTRUCTION_STATUS. Is NULL for vessel discharge receipt orders or a drive order. Example: 2001

Column Name	Data Type	Description
DEST_DOOR_1	See Orientation	The CONTAINER_ID_1's door direction at the AGV order destination. When the order is to receive a container - the container's door direction at the current location; when the order it to deliver a container - the container's door direction at the location to which it is planned. Can be UNKNOWN only for - vessel discharge receive orders and - rail discharge receive orders via direct TP, in which case it must be updated with a valid door direction before or at the moment the order status changes to COMPLETE. Can be N/A in orders to an X-RAY station unless the door direction is required. Null if there is no container involved.
CONTAINER_ID_2	String(12)	Same as for CONTAINER_ID_1 above. Is NULL if the AGV is dispatched only for one container or a drive order. Example: MSKU8901234
CONTAINER_WI_REF_2	Number(18)	Same as for CONTAINER_WI_REF_1 above. Is NULL if the AGV is dispatched only for one container or a drive order. Example: 2002
DEST_DOOR_2	See Orientation	Same as for DEST_DOOR_1 above, applied to CONTAINER_ID_2 . Is NULL if the AGV is dispatched only one container or a drive order.
WA_REF	Number(18)	N4 internal work assignment reference. Indexed field. May not be null; at this time, unsolicited AGV orders are not supported.
BYPASS_SEQUENCE	String(5)	YES/NO (default). Used when N4 user diverts the AGV to bring a vessel load to the QC out of sequence. Used for LOAD moves, otherwise null.

Column Name	Data Type	Description
PTA	Date	Even though this column is named "Planned Time of Arrival", this name is misleading as PTA is actually populated with the Work Instruction EMT value. Can be NULL if the AGV is not delivering a container to the QC.
CREATED	Date	Creation time of record
UPDATED	Date	Time of last update

13.4.2.2 ASC_ORDERS

Column Name	Data Type	Description
ORDER_GKEY	Number(18)	Order sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 738
COMMAND_GKEY	Number(18)	Command sequence Indexed field. N4 will write the gkey after the first time it processes a newly inserted Command record. The status of the Command doesn't matter - could be ENTERED or WORKING. Can be NULL when an order is created, until ECS response. Example: 863
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each ASC_ORDERS update. Default=1 Example: 1
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each ASC_COMMANDS update. Can be NULL when an order is created, until ECS response. Example: 1
CHE_ID	String(7)	ID of ASC being issued the transport order Indexed field Example: RMG03
STATUS	See AGV/ASC order status	ASC order status Example: ENTERED

Column Name	Data Type	Description
PROBLEM_DESCRIPTION	String(1000)	Free text description of a problem on N4 side. Is NULL when there is no N4 problem.
CONTAINER_ID	String(12)	ID of container to be moved Indexed field Example: MSKU1234567
CONTAINER_ID_2	String(12)	ID of the second twinned container to be moved Indexed field NULL when a single container is being moved - i.e. not a twin move. Example: MSKU1234567
CONTAINER_WI_REF	Number(18)	Matches CONTAINER_WI_REF in WORK_INSTRUCTION_STATUS. Null if there is no work instruction. Example: 2001
CONTAINER_WI_REF_2	Number(18)	Matches CONTAINER_WI_REF in WORK_INSTRUCTION_STATUS. Null if there is no work instruction. Only applies to twin moves. Example: 2002
ORIGIN	See Container and equipment locations	Location where ASC should get container represented by CONTAINER_ID, including cases when ASC order starts with a container in the spreader.
ORIGIN_2	See Container and equipment locations	Location where ASC should get the second container of a twin move represented by CONTAINER_ID_2, including cases when ASC order starts with a container in the spreader.
ORIG_FLOOR_HEIGHT_CM	Number(12)	Highest point of stack above the ground. Informational only. N4 does not write any value for this field. Can be NULL. Example: 777
ORIG_FLOOR_HEIGHT_CM_2	Number(12)	Highest point of stack above the ground. Informational only. N4 does not write any value for this field. Only applies to twin moves. Can be NULL. Example: 778
ORIG_VEHICLE_TYPE	See ASC origin or destination vehicle type	

Column Name	Data Type	Description
ORIG_POS_TRUCK	See Position on truck chassis	Is NULL if origin is not on a truck. Example:1
ORIG_POS_TRUCK_2	See Position on truck chassis	Is NULL if origin is not on a truck. Only applies to twin moves. Example:3
ORIG_CENTER_TO_CHS_BACK_MM	Number(18)	This value is received from the gate system and transmitted to ECS without any changes. Is NULL if origin is not on a truck. Example:4500
ORIG_CENTER_TO_CHS_BACK_MM_2	Number(18)	This value is received from the gate system and transmitted to ECS without any changes. Is NULL if origin is not on a truck. Only applies to twin moves. Example:4500
PLANNED_DESTINATION	See Container and equipment locations	Location where ASC should deliver container represented by CONTAINER_ID, or where ASC should relocate if CONTAINER_ID is blank. Can be NULL if the ASC order is a response to an unsolicited ASC command.
PLANNED_DESTINATION_2	See Container and equipment locations	Location where ASC should deliver container represented by CONTAINER_ID_2, or where ASC should relocate if CONTAINER_ID_2 is blank. Can be NULL if the ASC order is a response to an unsolicited ASC command. Only applies to twin moves.
DEST_FLOOR_HEIGHT_CM	Number(12)	Highest point of stack above the ground. Informational only. N4 does not write any value for this field. Can be NULL. Example: 777
DEST_FLOOR_HEIGHT_CM_2	Number(12)	Highest point of stack above the ground. Informational only. N4 does not write any value for this field. Only applies to twin moves. Can be NULL. Example: 777

Column Name	Data Type	Description
DEST_VEHICLE_TYPE	See ASC origin or destination vehicle type	
DEST_POS_TRUCK	See Position on truck chassis	Is NULL if destination is not on a truck. Example: 1
DEST_POS_TRUCK_2	See Position on truck chassis	Is NULL if destination is not on a truck. Only applies to twin moves. Example: 3
DEST_CENTER_TO_CHS_BACK_MM	Number(18)	This value is received from the gate system and transmitted to ECS without any changes. Is NULL if destination is not on a truck. Example:4500
DEST_CENTER_TO_CHS_BACK_MM_2	Number(18)	This value is received from the gate system and transmitted to ECS without any changes. Is NULL if destination is not on a truck. Only applies to twin moves. Example:4500
WA_REF	Number(18)	SPARCS internal work assignment reference. Indexed field.
DOOR_DIRECTION_IN_YARD	See Orientation	Door direction when the container is in yard block. Can be NULL if no container is involved in the order.
DOOR_DIRECTION_IN_YARD_2	See Orientation	Door direction when the container is in yard block. Only applies to twin moves. Can be NULL if no container is involved in the order.
PLANNED_COMPLETION_TIME	Date	Time when N4 expects the order to be completed. Can be NULL.
CREATED	Date	Creation time of record
UPDATED	Date	Time of last update

13.4.2.3 QC_ORDERS

N4 will not reject incorrect QC commands.

If a QC command is incorrect, a QC order for it will not be created.

Column Name	Data Type (SQL Server/Oracle)	Description
ORDER_GKEY	Number(18)	Order sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 738
COMMAND_GKEY	Number(18)	Command sequence. Indexed field. Can be NULL. Example: 863
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each QC_ORDERS update. Default=1.
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each QC_COMMANDS update. Can be NULL.
QC_ID	String(7)	Quay crane ID Example: QC25
CONTAINER_ID	String(12)	ID of container identified after hatch clerk/OCR reading. CONTAINER_ID, DOOR_DIRECTION_AT_QC and CONTAINER_WI_REF are updated together when identified. Is NULL when the container is not yet identified Example: MSKU1234567
DOOR_DIRECTION_AT_QC	See Orientation	Door direction at QC of container identified after hatch clerk/OCR reading. When identified, updated together with CONTAINER_ID and CONTAINER_WI_REF. For discharges cannot be UNKNOWN. For loads ECS accepts any not null value after identification, including UNKNOWN. Is NULL when the container is not yet identified.
CONTAINER_WI_REF	Number(18)	Matches CONTAINER_WI_REF in WORK_INSTRUCTION_STATUS. CONTAINER_ID, DOOR_DIRECTION_AT_QC and CONTAINER_WI_REF are updated together when identified. Is NULL when the container is not yet identified. Example: 2001

Column Name	Data Type (SQL Server/Oracle)	Description
WA_REF	Number(18)	SPARCS internal work assignment reference Indexed field, Default=0. Can be NULL.
STATUS	String(10)	Order status Values: WORKING, COMPLETE Example: WORKING
PROBLEM_DESCRIPTION	String(1000)	Free text description of a problem on N4 side Is NULL when there is no problem.
OCR_STATUS	String(25)	OCR transaction status for display purposes in ECS GUI. The ECS need not consume the field value. This field is used internally by N4. Can be NULL.
TRANSFER_ALLOWED	String(10)	Informs ECS that that according to business rules container transfer has been allowed, Although UNKNOWN is a valid value, N4 never writes UNKNOWN to any QC_ORDERS record Values: YES, NO, UNKNOWN. Example: NO
CREATED	Date	Record creation time
UPDATED	Date	Time of last update

13.4.2.4 TEST_ORDERS

This table is not used for production systems and can be used for testing.

Column Name	Data Type	Description
ORDER_GKEY	Number(18)	Order sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 738
COMMAND_GKEY	Number(18)	Command sequence Indexed field Can be NULL. Example: 863
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each TEST_ORDERS update. Default = 1 Example: 1

Column Name	Data Type	Description
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each TEST_COMMANDS update. Default = 1 Can be NULL. Example: 1
TEST_ORDER	String(80)	Order name Example: BeamAGV
STATUS	String(10)	Order status. Values - ENTERED, WORKING, COMPLETE, REJECTED, ABORTED Example: ENTERED
PARAMETERS	String(200)	Optional Can be NULL.
PROBLEM_DESCRIPTION	String(100)	Possible N4 problem related to this order. Is NULL when there is no problem in N4.
CREATED	Date	Record created time
UPDATED	Date	Time of last update

13.4.3 N4 Status

13.4.3.1 AGV_CONTAINER_UPDATE

N4 informs the ECS when a container was placed on an AGV or removed from an AGV not using AGV_ORDERS/COMMANDS.

An example use case - XPS client user performs an action like "Place container onto CHE".

The purpose is to let ECS know when a container is placed on AGV or removed from it outside of the ECS interface. In this example, a straddle removed a container from a broken AGV and places it on a different AGV.

What is the N4-ECS interaction for this table?

- Container Loaded Interaction
 - AGV Status, AGV->No Container
 - N4 Inserts Record,
 - AGV Status, AGV->Container
- Container Unloaded Interaction
- AGV Status, AGV->Container
- N4 Inserts Record,
- AGV Status, AGV->No Container

Will N4 ever delete records from this table?

- ECS or N4 should never delete records from this table. It is recommended that entries in these tables be purged when the corresponding vehicle no longer exists or is decommissioned.

Column Name	Data Type SQL Server Oracle	Description
CHE_ID	String(7)	AGV ID. Primary key. Indexed field - no duplicates. Example: AGV03
CONTAINER_ID	String(12)	ID of container placed on AGV or removed from AGV Example: MSKU1234567
CONTAINER_STATE	String(8)	LOADED, UNLOADED Example: LOADED
LOCATION	See Container and equipment locations	Container location on AGV Example: AGV.501.F.F
UPDATED	Date	Time of last update

13.4.3.2 WORK_INSTRUCTION_STATUS

This table contains information about container properties and work instructions in active work queues. AGV/ASC/QC_ORDERS tables have columns with references to this table.

For QC work N4 will provide information for the moves during the next hour per point of work. Twin lifted containers are considered as 1 move with a separate QC order for each of them.

This table can have several records for the same container, but should not have more than one vessel-related work instruction for the same container for a load or discharge queue.

Column Name	Data Type SQL Server Oracle	Description
CONTAINER_ID	String(12)	ID of container to be moved Indexed field Example: MSKU1234567
CONTAINER_WI_REF	Number(18)	Work instruction reference. AGV/ASC/QC_ORDERS tables have columns with references to this table. Primary key. Example: 111
CONTAINER_LENGTH_CM	Number(12)	Container length in centimeters.
CONTAINER_HEIGHT_CM	Number(12)	Container height in centimeters.
CONTAINER_WEIGHT_KG	Number(14)	Container weight in kilograms. Example: 25000

Column Name	Data Type SQL Server Oracle	Description
CONTAINER_ISO	String(4)	Container ISO Example: 4300
IS_TANK	String(5)	YES, NO Indication whether the container is a tank Example: YES
RACK_SUITABLE	String(5)	YES, NO Whether container can be transferred via rack Example: YES
HOLD	String(10)	Hold on the work instruction. Values: NONE, BYPASSED, SUSPENDED Example: NONE
WORK_QUEUE	String(60)	Work queue name. Mandatory for discharge and load queues, optional for other queue types. Updated to null if the work instruction has been reverted in N4. Can be NULL.
POINT_OF_WORK	String(30)	Point of work name. Mandatory for discharge and load queues, optional for other queue types. Can be NULL.
MOVE_KIND	String(5)	RECV, DLVR, DSCH, LOAD, RDSC, RLOD, SHOB, YARD Example: LOAD
MOVE_STAGE	String(25)	NONE, PLANNED, FETCH_UNDERWAY, CARRY_READY, CARRY_UNDERWAY, CARRY_COMPLETE, PUT_UNDERWAY, PUT_COMPLETE, COMPLETE NONE is used only if the work instruction has been reverted in N4. Example: PLANNED
ORDER_SEQ	Number(14)	Place of the work instruction in work queue. Can be NULL for non-vessel moves. Example: 1

Column Name	Data Type SQL Server Oracle	Description
LOGICAL_PREDECESSOR	String(12)	Container ID for the vessel load work instruction planned for execution directly prior to the execution of this work instruction from within the same work queue. Makes it easier for ECS, because gaps are possible in sequence values. For example, 1, 2, 3, 5, 8. Used for vessel loads only, otherwise null. Set to NULL if the logical_predecessor has been loaded
PHYSICAL_PREDECESSOR	String(12)	Container ID for the vessel load work instruction which must be physically completed prior to completion of this work instruction. Used for vessel loads only, otherwise null. If there is more than one physical predecessor, the one with the higher tier and highest sequence value should be chosen. Set to NULL if the physical_predecessor has been loaded
CONTAINER_STOW_FACTOR	String(60)	Stow factor. Can be NULL for non vessel loads. Example: 4300/HKG
CONTAINER_WEIGHT_MARGIN_KG	Number(14)	Container weight margin in kilograms. Used for vessel load swapping validation. Can be NULL for non vessel loads. Example: 2500
LIFT_REFERENCE	Number(18)	Common reference number for all containers planned in a same lift action by the QC's main trolley to indicate twin lift, tandem, triple and quad lift. Can be NULL when the QC is single lifting. Example: 5467
CARRY_REFERENCE	Number(18)	Common reference number for all containers planned on the same transport. Can be NULL when the QC is single lifting. Example: 3456

Column Name	Data Type SQL Server Oracle	Description
RELATIVE_POS_ON_CARRIER	(See "QC hoist and spreader" in Container and equipment locations)	Used for vessel discharges and loads which are not single to indicate position of the container in relation to the other participating container(s). For singles the value is also provided, with the values corresponding to the central position in both the hoist and the spreader. Lift Reference and Equipment Length can distinguish singles, twins and tandem 40s. To distinguish tandem 20s from twins must use RELATIVE_POS_ON_CARRIER. LIFT_REFERENCE and CARRY_REFERENCE also can used for the same purpose Can be NULL for non-vessel moves.
ORIGIN_CARRIER_SLOT	See Container and equipment locations	The origin carrier slot. Can be NULL for non DSCH or SHOB moves.
DESTINATION_CARRIER_SLOT	See Container and equipment locations	The destination carrier slot. Can be NULL for non LOAD or SHOB moves.
DOOR_DIRECTION_ON_CARRIER	See Orientation	Door direction on vessel. When provided for loads, cannot be UNKNOWN or NA. Can be NULL for non LOAD or DSCH moves.
OFFSET_TO_BAY_CENTER_CM	Number(12)	Offset to the center of the bay relative to the berth location provided for the queue (as bollard + offset). Provided only for work instructions in a vessel discharge or a load queue, otherwise null. 0 = aligned with the centre of the bay. Negative value = offset in the LowBollard direction. Positive value = offset in the HighBollard direction. Can be NULL. Example: -310

Column Name	Data Type SQL Server Oracle	Description
CONTAINER_QC_LOC_TYPE	String(30) Values: AGV, APRON, TRUCK, TRUCK_AT_QC, VESSEL	Used only for vessel QC related work instructions, otherwise null. Populated as follows: VESSEL - the unit's last known position location type is vessel APRON - the unit's last known position is the apron AGV - none of the above applies and the work instruction is dispatched to an AGV TRUCK or TRUCK_AT_QC - none of the above applies and the work instruction is dispatched to a truck. In that case TRUCK_AT_QC is used if the WI is a load and the truck is currently at QC (its job steps is 'At Ship To Drop'). Otherwise TRUCK is used Example: AGV
CONTAINER_NEXT_QC_LOC_TYPE	String(30) Values: AGV, APRON, TRUCK, TRUCK_AT_QC, VESSEL	Used only for vessel QC related work instructions, otherwise null. Populated as follows: If the work instruction is a load - VESSEL - if the WI is planned to vessel, otherwise APRON If the work instruction is a shift-on-board - VESSEL If the work instruction is a discharge - APRON - if the WI is planned to apron TRUCK - if the work instruction is NOT planned to an ASC block or to a TZ associated with an ASC block or if the work instruction failed to deck and the equipment type is "??6%" (flatrack), or "??P%" (flatrack collapsible), or "L5G1" (45ft high-cube general), otherwise AGV TRUCK_AT_QC is not used in any scenario Example: AGV

Column Name	Data Type SQL Server Oracle	Description
HAS_TOP_RAILS	String(7)	YES, NO, NA, UNKNOWN Example: YES
HAS_BOTTOM_RAILS	String(7)	YES, NO, NA, UNKNOWN Example: NO
UPDATED	Date	Time of last update

13.4.3.3 QC_WORK_QUEUE_STATUS

Describes active QC load and discharge queues within a configurable horizon (1 hour by default). Table WORK_INSTRUCTION_STATUS provides information about work instructions in those queues.

Column Name	Data Type	Description
WORK_QUEUE	String(60)	Name of work queue, Primary key, no duplicates
MOVE_KIND	String(10)	LOAD, DSCH, DUAL_CYCLE Move kind of the majority of work instructions in queue. N4 does not support the DUAL_CYCLE value. In addition, Dual Cycling Work Queues (WQ) are not supported. Example: LOAD
ABOVE_BELOW	String(1)	Values: A, B Above or below deck Example: A
QC_ID	String(7)	Quay crane ID
VESSEL_VISIT	String(30)	Vessel visit reference
VESSEL_BAY	String(12)	Bay on the vessel corresponding to the work queue. For unconfigured vessels this is "0". For a work queue with twins this value will match that of 40's WQ,
QC_BOLLARD	String(7)	Name of bollard closest to QC. The bollard closest to the center line of the main spreader and following offset correspond to the vessel bay position on the quay derived from vessel quay position & vessel layout. Can be NULL for an unconfigured queue. Example: B003

Column Name	Data Type	Description
QC_BOLLARD_OFFSET_CM	Number(14)	Bollard offset in centimeters. Can be NULL for an unconfigured queue. Example: -1230
WQ_SEQ	float(53) FLOAT(53)	Place of the work queue within the POW. Value should be a unique number per POW. Represents the order of the WQ relative to others assigned to the same POW.
CONFIGURATION	String(15)	CONFIGURED/UNCONFIGURED Indication if the queue is configured or unconfigured. If a queue is UNCONFIGURED, as mentioned above, QC_BOLLARD and QC_BOLLARD_OFFSET_CM field values will be NULL. Example: CONFIGURED
START_TIME	Date	Queue start time. The field values will be the earliest and latest WI EMT for those in the Work Queue; not the start or end time of the Work Shift. The START_TIME might change periodically based upon the EMTs - e.g. splitting Work Queues or adding/removing Vessel Projections.
END_TIME	Date	Queue end time. The field values will be the earliest and latest WI EMT for those in the Work Queue; not the start or end time of the Work Shift.
WQ_STATUS	String(7)	WORKING (there are containers to be handled at QC in the queue), EMPTY (there are none) WQ_STATUS will have a value of 'EMPTY' when: No EMTs for WIs WQ is deactivated No Vessel Visit associated with the WQ No uncompleted WIs in the WQ
UPDATED	Date	Time of last update

13.4.3.4 BERTH_STATUS

Describes arriving vessels and their position on the quay.

Column Name	Data Type SQL Server Oracle	Description
VESSEL_VISIT	String(30)	Unique reference for a single visit of a vessel. Primary key
VESSEL_CALL_SIGN	String(12)	Call sign of the vessel
VESSEL_NAME	String(32)	Name of the vessel
BOW_BOLLARD	String(7)	Name of bollard closest to vessel bow. Example: B002
BOW_BOLLARD_OFFSET_CM	Number(14)	Bollard offset in centimeters. Example: 1110
STERN_BOLLARD	String(7)	Name of bollard closest to vessel bow. Example: B111
STERN_BOLLARD_OFFSET_CM	Number(14)	Bollard offset in centimeters. Example: -2220
VESSEL_CLASSIFICATION	String(7)	DEEPSEA, FEEDER, BARGE
VESSEL_VISIT_PHASE	String(8)	ARRIVED, WORKING, COMPLETE, DEPARTED
DISTANCE_FROM_DOCK_CM	Number(18)	Distance from dock in centimeters. This column was added in N4 3.2.
UPDATED	Date	Time of last update

13.4.4 ECS Commands

13.4.4.1 AGV_COMMANDS

Column Name	Data Type SQL Server Oracle	Description
COMMAND_GKEY	Number(18)	Command sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 863
ORDER_GKEY	Number(18)	Order sequence. Indexed field. Can be NULL if an unsolicited command is created, until N4 response. Example: 738

Column Name	Data Type SQL Server Oracle	Description
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each AGV_COMMANDS update. Default = 1. Example: 1
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each AGV_ORDERS update. Can be NULL if an unsolicited command is created, until N4 response. Example: 1
CHE_ID	String(7)	AGV ID
STATUS	String(10)	AGV order status Values - ENTERED, WORKING, COMPLETE, REJECTED, CANCELED, ABORTED. Example: ENTERED
PROBLEM_CODE	String(25)	If the order was rejected, aborted or delayed (by ECS), describes the reason: Rejected - AGV_NOT_AVAILABLE, VALIDATION_ERROR, ECS_RESTART, ROUTE_REJECTED Aborted - DESTINATION_UNREACHABLE, EXECUTION_FAILED, ECS_RESTART, DESTINATION_INVALID Delayed - WAITING_FOR_ROUTE, WAITING_FOR_SEQUENCE Can be NULL if there is no ECS problem during job execution. Note that DESTINATION_UNREACHABLE is the only code when N4 will react - if DEST_UNREACHABLE is returned, then AGV is laden dispatched to a different location when carrying a container Example: ECS_RESTART Can be NULL.
PROBLEM_DESCRIPTION	String(100)	Free text for ECS to give details. Can be NULL.

Column Name	Data Type SQL Server Oracle	Description
ACTUAL_DESTINATION	See Container and equipment locations	Filled in by ECS. Matches the value of PLANNED_DESTINATION in AGV_ORDERS, unless ECS selects a different destination. Does not have slot and tier. Represents the final location for the AGV as a result of the order. The 40 ft location name value is always used for this, because an AGV occupies a 40 ft location, Can be NULL.
QC_REFERENCE_ID_1	String(32)	Reference ID generated by the QC system. Equal to REFERENCE_ID in corresponding QC_COMMAND. Should be filled in before STATUS column changes to COMPLETE. Can be NULL for non DSCH moves. Example: #QC31-12345678
FINAL_LOCATION_1	See Container and equipment locations	If the AGV receives or delivers AGV_ORDERS.CONTAINER_ID_1 from/to an ASC or an RGC, this column represents the said container's location in the ASC/RGC transfer zone. Unlike values for AGV destination, this value should include <slot>. Can be NULL when not applicable. Example: WSTP.W009.B.02.1
QC_REFERENCE_ID_2	String(32)	See description for QC_REFERENCE_ID_1 above. Can be NULL for non DSCH moves. Example: #QC31-12345678
FINAL_LOCATION_2	See Container and equipment locations	Same as for FINAL_LOCATION_1 above, applied to AGV_ORDERS.CONTAINER_ID_2. Can be NULL when not applicable. Example: WSTP.W009.B.02.1
TIME_OF_ORIGIN_EXIT	Date	Time of leaving the origin TP. Can be NULL.
ETA	Date	Estimated time of arrival.

Column Name	Data Type SQL Server Oracle	Description
TIME_AT_DESTINATION	Date	Time of arrival to destination. Once set, this column does not change. A non-null value indicates to N4 that an AGV has entered or exited the PB or any destination for that matter. Therefore it is very important to populate these fields in a timely manner. Can be NULL.
PARALLEL_BUFFER_ENTRY	Date	Time of AGV last entry into a parallel buffer. A non-null value indicates to N4 that an AGV has entered or exited the Parallel Buffer. Therefore it is very important to populate these fields in a timely manner. Can be NULL.
PARALLEL_BUFFER_EXIT	Date	Time of AGV last exiting a parallel buffer. A non-null value indicates to N4 that an AGV has entered or exited the Parallel Buffer. Therefore it is very important to populate these fields in a timely manner. Can be NULL.
CREATED	Date	Creation time of record
UPDATED	Date	Time of last update

Regarding Command Completion - ECS should complete the AGV Recharge Command when the AGV arrives in the TP such that N4 can manage the state of the AGV - for example dispatch to other moves and/or evict a charging AGV

13.4.4.2 ASC_COMMANDS

Column Name	Data Type SQL Server Oracle	Description
COMMAND_GKEY	Number(18)	Command sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 863
ORDER_GKEY	Number(18)	Order sequence. Indexed field. Can be NULL if an unsolicited command is created, until N4 response. Example: 738

Column Name	Data Type SQL Server Oracle	Description
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each ASC_COMMANDS update. Default = 1 Example: 1
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each ASC_ORDERS update. Can be NULL if an unsolicited command is created, until N4 response. Example: 1
CHE_ID	String(7)	ASC ID
STATUS	String(10)	ASC order status Values - ENTERED, WORKING, COMPLETE, REJECTED, CANCELED, ABORTED. Example: ENTERED
ECS_PROBLEM_TYPE	String(25)	Values for rejection: ASC_NOT_AVAILABLE, VALIDATION_ERROR, ECS_RESTART Value for abort: ABORTED_BY_OPERATOR, BLOCKMAP_ERROR, EXECUTION_FAILED, REEFER_ON_POWER, ECS_RESTART If the value is ABORTED_BY_OPERATOR, the column CRANE_ABORT_CODE also will have a value. Can be NULL if there is not ECS problem. Example: VALIDATION_ERROR
ECS_PROBLEM_DESCRIPTION	String(100)	Free text description for ECS_PROBLEM_TYPE. Can be NULL when ECS_PROBLEM_TYPE = NULL or when ECS_PROBLEM_TYPE = ABORTED_BY_OPERATOR

Column Name	Data Type SQL Server Oracle	Description
CRANE_ABORT_CODE	Number(12)	ECS_PROBLEM_TYPE doesn't have to be ABORTED_BY_OPERATOR for this field to be populated. That's the decision of the ECS. No filtering in N4 either: Rather, ECS should populate field if the order is aborted Specify the exact values for ABORT_EXECUTION and ABORT_DOOR_DIRECTION Indicate that special processing when ABORT_DOOR_DIRECTION Indicate that custom values can be provided which then can be processed by customized code in the N4 ASC Abort Handler Example: 01
CRANE_ABORT_DESCRIPTION	String(100)	This is a deprecated column. Original was used when ECS_PROBLEM_TYPE was ABORTED_BY_OPERATOR, in which case this field contained free text description of the error passed to ECS by the QC system. Always NULL.
ORDER_PROGRESS	String(25)	Progress status of order currently being executed. ASC state during different stages of an order Values: UNDEFINED, PLANNED, DRIVING_ONLY, DRIVING_TO_PICK, PICKING, LIFTING_AFTER_PICK, DRIVING_TO_DEPOSIT, DEPOSITING, LIFTING_AFTER_DEPOSIT. Example: UNDEFINED
ACTUAL_DESTINATION	See Container and equipment locations	Filled in by ECS. If a response to an ASC order, matches the value of PLANNED_DESTINATION in ASC_ORDERS, unless ECS selects a different destination. In a case of an unsolicited ASC command: - has the value assigned by ECS Can be NULL when the ECS does not yet know the destination.

Column Name	Data Type SQL Server Oracle	Description
ACTUAL_DESTINATION_2	See Container and equipment locations	Filled in by ECS. If a response to an ASC order, matches the value of PLANNED_DESTINATION_2 in ASC_ORDERS, unless ECS selects a different destination. In a case of an unsolicited ASC command: - has the value assigned by ECS Can be NULL when the ECS does not yet know the destination. Only applies to the second container of a twin move.
ESTIMATED_LIFT_TIME	Date	Time when ECS estimates the lift to happen. Field value is unused by N4. Can be NULL.
ESTIMATED_COMPLETION_TIME	Date	Time when ECS estimates the order to be completed. Field value is consumed by N4, but never used. It is recommended not to set this value. Can be NULL.
LIFT_TIME	Date	Time when the spreader lifted the container. Can be NULL.
SET_TIME	Date	Time when the spreader set down the container. Can be NULL.
TRANSFER_POINT_ENTRY	Date	Time the ASC enters the Transfer Zone while performing an inbound or outbound container move. Should be NULL for intra-stack moves. Should only be set once by the ECS Can be NULL.
TRANSFER_POINT_EXIT	Date	Time the ASC exits the Transfer Zone while performing an inbound or outbound container move. Should be NULL for intra-stack moves. Should only be set once by the ECS Can be NULL.
MEASURED_WEIGHT_KG	Number(14)	Container weight measured by ECS in kilograms. Can be NULL. Example: 25250

Column Name	Data Type SQL Server Oracle	Description
MEASURED_WEIGHT_KG_2	Number(14)	Second container of a twin move's weight measured by ECS in kilograms. Can be NULL. Example: 25250
CONTAINER_ID	String(12)	Used only when - the command is initiated by ECS - a container is involved - ECS knows the container Null if there is no container in spreader. UNKNOWN if there is container in spreader, but ECS cannot calculate it due to insufficient data.
CONTAINER_ID_2	String(12)	Used only when - the command is initiated by ECS - a container is involved - ECS knows the container - there is a second container for a twin move Null if there is no container in spreader. UNKNOWN if there is container in spreader, but ECS cannot calculate it due to insufficient data.
ORIGIN	See Container and equipment locations	Location where ASC should get container represented by CONTAINER_ID, including cases when ASC command starts with a container in the spreader. Used only when the command is initiated by ECS.
ORIGIN_2	See Container and equipment locations	Location where ASC should get container represented by CONTAINER_ID_2, including cases when ASC command starts with a container in the spreader. Used only when the command is initiated by ECS. Only applies to twin moves.
CREATED	Date	Creation time of record
UPDATED	Date	Time of last update

13.4.4.3 QC_COMMANDS

Column Name	Data Type SQL Server Oracle	Description
COMMAND_GKEY	Number(18)	Command sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 863
ORDER_GKEY	Number(18)	Order sequence. Indexed field. Can be NULL when a command is created, until the ECS response. Example: 738
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each QC_COMMANDS update. Default=1.
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each QC_ORDERS update. Can be NULL when a command is created, until ECS response.
QC_ID	String(7)	Quay crane ID
REFERENCE_ID	String(32)	Reference ID generated by the QC system. REFERENCE_ID must be unique for the container move. Note the difference with TRANSACTION_REF. E.g. if discharging Twins, the two QC_COMMAND records will have different REFERENCE_ID values. However, both QC_COMMAND records will have the same TRANSACTION_REF value. Example: #QC31-12345678
PRESUMED_CONTAINER_ID	String(12)	For load moves ID of container which according to information received earlier from N4 is presumed to have been lifted from AGV. Can be NULL if not a load move. Example: CSQU1234567
WORK_QUEUE	String(60)	Work queue which is currently being operated. Matches value earlier received from N4 in QC_WORK_QUEUE_STATUS.

Column Name	Data Type SQL Server Oracle	Description
STATUS	String(10)	Order status Values: WORKING, COMPLETE Example: WORKING
CONTAINER_ORIG_LOCATION	See Container and equipment locations	The very first container location at QC. Once set, does not change.
CONTAINER_LAST_LOCATION	See Container and equipment locations	Last container location at QC.
CONTAINER_CURR_LOCATION	See Container and equipment locations	Current container location at QC.
CONTAINER_LENGTH	Number(12)	20, 40, 45 Example: 20
TRANSACTION_ID	Number(12)	DEPRECATED as of N4 version 3.4.8, 3.5, and above. See TRANSACTION_REF.
MEASURED_WEIGHT_KG	Number(14)	Container weight measured by ECS in kilograms. Can be NULL when more than one container has been lifted by the QC. Example: 25250
TRANSACTION_REF	String(32)	TRANSACTION_REF is unique for each QC move. This means that the TRANSACTION_REF must match for all QC_COMMAND records representing the containers a QC moved at the same time. For example, if the QC lifts a single container, the TRANSACTION_REF is unique across all QC_COMMAND records currently in the DB table. If the QC lift involves multiple containers - such as a twin, quad or tandem - the TRANSACTION_REF must have the same value for all QC_COMMAND records pertaining to that QC move. Note: This is different from the REFERENCE_ID. For example, for a Twin move, the two QC_COMMAND records will have different REFERENCE_ID values, but both QC_COMMAND records will have the same TRANSACTION_REF value. Example: ABC321
CREATED	Date	Record creation time
UPDATED	Date	Time of last update

13.4.4.4 TEST_COMMANDS

This table is not used for production systems and can be used for testing.

Column Name	Data Type SQL Server Oracle	Description
COMMAND_GKEY	Number(18)	Command sequence. Primary key. Indexed field - no duplicates. Automatically generated by database. Example: 863
ORDER_GKEY	Number(18)	Order sequence. Indexed field Can be NULL. Example: 738
ORDER_VERSION	Number(14)	Synchronization point number. Should increment with each TEST_ORDERS update. Default=1 Example: 1
COMMAND_VERSION	Number(14)	Synchronization point number. Should increment with each TEST_COMMANDS update. Default=1 Can be NULL. Example: 1
STATUS	String(20)	Order status. Values - ENTERED, WORKING, COMPLETE, REJECTED, CANCELED, ABORTED Example: ENTERED
PROBLEM_DESCRIPTION	String(25)	Free text description for an ECS problem. Can be NULL when there is no ECS problem.
CREATED	Date	Record created time
UPDATED	Date	Time of last update

13.4.5 ESC Status

13.4.5.1 AGV_STATUS

Used to display the status of all AGVs. Each device has a reserved row within the table that it uses to update its status. This table is updated by ECS periodically or whenever an action occurs. On average this is once every 3 seconds. All processes using this data should check the UPDATED field to determine if data is being updated. If communication to AGV is lost or ECS is stopped then this table is not updated.

Location columns will be updated even if the AGV is offline (OPERATIONAL_STATUS == OFFLINE)

Column Name	Data Type SQL Server Oracle	Description
CHE_ID	String(7)	AGV ID Primary key. Indexed field - no duplicates. Example: AGV03
OPERATIONAL_STATUS	String(20)	Additional details of action being carried out by AGV Values: OFFLINE, AUTOMATIC AGVs will carry out N4 orders only in AUTOMATIC state.
TECHNICAL_STATUS	String(10) See Technical status	
TECHNICAL_DETAILS	String(80)	Detailed description of the fault status. Is NULL when the TECHNICAL_STATUS is GREEN. Example: Flat rear left tire
LIFT_CAPABILITY	String(2)	The LIFT_CAPABILITY field describes the real-time capability of the Lift-AGV to use Racks or not. YES: lift tables present and in working order, OUT_OF_ORDER: AGV is in working order and lift tables present but lift capability is not working, NONOT_PRESENT: No capability as a Lift-AGV Example: YES
STARTUP_DELAY	Number(12)	Time in seconds after which AGV can start performing an order. Currently unused by N4. ECS should wait to set the Operational Status to AUTOMATIC until AGV is booted and ready to accept order rather than relying on posting a value to STARTUP_DELAY and expecting N4 to wait. Can be NULL. Example: 25
ORDER_GKEY	Number(18)	Order currently being executed Set to zero when the AGV has no order. Can be NULL. Example: 863

Column Name	Data Type SQL Server Oracle	Description
COMMAND_GKEY	Number(18)	Command currently being executed Set to zero when the AGV has no order. Can be NULL. Example: 738
LOCATION	See Container and equipment locations	Current location of AGV. Always provided. If the location is not a known location, the value is "UNDEFINED".
NEXT_LOCATION	See Container and equipment locations	Next location of AGV during an order execution. Always a known location. Is NULL when there is no order.
LOCATION_X	Number(14)	X coordinate of AGV location. Updated every 15 meters. Can be NULL.
LOCATION_Y	Number(14)	Y coordinate of AGV location. Updated every 15 meters. Can be NULL.
ORIENTATION	Number(12)	Current heading of AGV in degrees. Can be NULL. Example: 270
REFERENCE_ID_1	String(32)	Reference ID generated by the QC system during container transfers. Can be NULL. Example: #QC31-12345678
REFERENCE ID_2	String(32)	Reference ID generated by the QC system during container transfers. Can be NULL. Example: #QC31-12345679
CONTAINER_ID_1	String(12)	Current container 1. Indexed field. Can be NULL. Example: MSKU1234567
CONTAINER_ID_2	String(12)	Current container 2. Indexed field. Can be NULL. Example: MSKU8901234
FUEL_TYPE	String(125)	BATTERY, FUEL

Column Name	Data Type SQL Server Oracle	Description
BATTERY_STATE	String(6)	GREEN: No need to replace battery YELLOW: Battery exchange allowed at a time determined by TOS RED: Battery should be replaced immediately When an AGV goes to battery status RED, it should not receive any new productive moves and the next order should be to a battery exchange station. If no exchange station is available, it is allowed to send a park order for the AGV if it is undesirable to leave the AGV in the current location, for example on a quay lane or on a highway.
BATTERY_STATION_FACTORS	String(125)	List of candidate battery exchange positions for this AGV with preference value for each position in the range between 1 and 100 per station. Each <position name> is a value for a recharge transfer point in "Container and equipment locations". Preferences values are calculated using the number of available batteries at the position and the position availability. Format: <position name>:<preference> <position name>:<preference> ... Values in list are separated by spaces. Should be NULL if ECS provides charging state in TRANSFER_POINT_STATUS for transfer points which can charge. Should not be populated when a terminal is using Charging Stations. Example: RECH.BES01.P101:100
REMAINING_FUEL	Number(12)	Battery charge / fuel level percentage. Can be NULL. Example: 55
RUNNING_HOURS	Number(12)	Current running hours of the AGV. Can be NULL. Example: 888

Column Name	Data Type SQL Server Oracle	Description
IS_CHARGING	String(5)	YES/NO (default). Used when the battery of the AGV is being charged. Example: NO This column was added in N4 3.2.
UPDATED	Date	Time of last update.

13.4.5.2 ASC_STATUS

This table is used to display the status of all the container handling devices. Each device has a reserved row within the table that it uses to update its status. This table is updated by ECS periodically or whenever an action occurs. On average this is once every 3 seconds. All processes using this data should check the UPDATED field to determine if data is being updated. If communication to ASC is lost or ECS is stopped then this table is not updated.

Column Name	Data Type	Description
CHE_ID	String(7)	ID of ASC being issued the transport order Primary key. Indexed field - no duplicates Example: RMG03
WORK_STATUS	String(25) See Work Status	Work status of ASC See Work Status
OPERATIONAL_STATUS	String(20)	Additional details of action being carried out by ASC. It is not necessary to use MAINTENANCE_MODE. There is no functional difference between OPERATIONAL_STATUS = OFFLINE and MAINTENANCE_MODE for ASCs Values: OFFLINE, AUTOMATIC, MAINTENANCE_MODE ASCs only accept N4 orders in AUTOMATIC state.
TECHNICAL_STATUS	String(10) See Technical status	See Technical status
TECHNICAL_DETAILS	String(80)	Detailed description of the fault status. Is NULL when TECHNICAL_STATUS is GREEN. Example: Failed to lift container 3 times

Column Name	Data Type	Description
ORDER_GKEY	Number(18)	Order currently being executed. Is NULL when WORK_STATUS is IDLE. Example: 863
COMMAND_GKEY	Number(18)	Command currently being executed. Is NULL when WORK_STATUS is IDLE. Example: 863
LOCATION	String(50) See Container and equipment locations	Current ASC location
CONTAINER_ID	String(12)	Current container. UNKNOWN if there is container in spreader, but ECS cannot calculate it due to insufficient data. Indexed field. Is NULL when there is no container on the spreader. Example: MSKU1234567
CONTAINER_ID_2	String(12)	Current container that is the second container in a twin move. UNKNOWN if there is container in spreader, but ECS cannot calculate it due to insufficient data. Indexed field. Is NULL when there is no container on the spreader. This field is only used for ASC twin lift moves. Example: MSKU1234568
IS_TWIN_CAPABLE	String(8)	Indicates whether an ASC is twin capable and can be one of three values: • YES: The spreader is capable of hoisting two (end-to-end) adjacent containers at the same time (twin containers). • NO: The spreader is capable of hoisting only a single container. It cannot ever hoist two containers at the same time. • OFFLINE: The spreader is capable of hoisting two (end-to-end) adjacent containers at the same time, but can only hoist one container right now (e.g. due to a malfunction or manual setting).
UPDATED	Date	Time of last update

13.4.5.3 QC_STATUS

Describes technical status and location of QC on the quay and also IDs of containers at QC locations. N4 consumes the TECHNICAL_STATUS, QC_BOLLARD, and QC_BOLLARD_OFFSET_CM columns and are used to display the status and last known position of the QC within N4.

Names for QC container locations correspond to the naming system described in QC-related locations

Column Name	Data Type	Description
QC_ID	String(7)	QC/BFC ID
TECHNICAL_STATUS	String(10) See Technical status	For QC - purely informational (unlike for AGVs and ASCs). Does not affect QC functionality. Optional field.
QC_BOLLARD	String(7)	Name of bollard closest to QC.
QC_BOLLARD_OFFSET_CM	Number(14)	Bollard offset in centimeters.
UPDATED	Date	Time of last update.

13.4.5.4 DRIVE_TIME_STATUS

Used for the current calculated AGV driving times between locations of the following type:

- Transfer points in ASC transfer zones (waterside, landside, cantilever)
- Rail crane transfer points
- Parallel buffers
- Maintenance locations
- Battery exchange stations
- X-Ray stations

Transfer point names are provided with slot, without tier. Therefore their format is TYPE. <blockname>.<lane>.<slot>, where TYPE is WSTP, LSTP etc. The slot value corresponds to the 40 ft position.

How to indicate to N4 that the ECS wants to block PBs? That is, the ECS wants to tell N4 that AGVs should not be sent to certain PBs?

The answer is to set the drive times to and from these PBs to -1. Blocking PBs with AREA_STATUS is not supported within N4

The calculation of drive time approximations for trips involving a QC transfer point, see the explanation below together with the description of the table WORK_QUEUE_BUFFER_STATUS.

Each row represents a directed path from one transfer point to another. This table is updated by ECS periodically or whenever an action occurs.

For the purpose of drive time approximation the location in front of the rack (WS TP) and the position in the rack are treated as one location.

Column Name	Data Type	Description
FROM_TP_ID	String(50) See Container and equipment locations	ID of the from transfer point Composite Primary key - no duplicates Example: WSTP.W001.2.1
TO_TP_ID	String(50) See Container and equipment locations	ID of the to transfer point Composite Primary key - no duplicates Example: PB.M001
DRIVE_TIME	Number(12)	Drive time in seconds. Routes that are invalid should have a time of -1. Example: 342
UPDATED	Date	Time of last update

13.4.5.5 WORK_QUEUE_BUFFER_STATUS

Provides information about what parallel buffers are used for work queues, which makes it possible for Sparcs to calculate drive times from waterside transfer points to QC transfer points (and vice versa) for AGV orders related to the work queues.

The approach to a QC will happen via one of the buffers that the ECS Topology management assigns to the work queue. So there will be variations in the drive times, caused by the actually chosen buffer (on top of other variations caused by for instance traffic). In order to reflect the variation, ECS selects two representative buffers per work queue: one representing a close by entry buffer and one representing a far entry buffer. These buffers and the drive time approximations for the trip from the buffers to the QC are maintained by ECS in the table WORK_QUEUE_BUFFER_STATUS. Similarly for the exit from a QC.

The QC_WORK_QUEUE_STATUS.WQ_STATUS field will be set to 'EMPTY' when a WQ is deactivated. The related records in WQ_BUFFER_STATUS should be deleted after the QC_WORK_QUEUE_STATUS.WQ_STATUS is set to 'EMPTY'.

Column Name	Data Type	Description
QC_ID	String(7)	QC ID Composite primary key - no duplicates Example: QC1
WORK_QUEUE	String(60)	Work queue name Composite primary key - no duplicates Example: QC1-01
ENTRY_BUFFER_START	String(50) See Container and equipment locations	Start of the range of buffers for the work queue in question for approaching the QC. Locations specified must be valid N4 yard locations (i.e. N4 bins). Specifying invalid N4 yard locations causes exceptions and slow performance. Example: PB.M001

Column Name	Data Type	Description
ENTRY_BUFFER_START_TIME	Number(12)	Drive time approximation for the close representative entry buffer to the QC. This is the minimum drive time within the set of Parallel Buffers allocated. Example: 38
ENTRY_BUFFER_END	String(50) See Container and equipment locations	End of the range of buffers for the work queue in question for approaching the QC. Locations specified must be valid N4 yard locations (i.e. N4 bins). Specifying invalid N4 yard locations causes exceptions and slow performance. Example: PB.M003
ENTRY_BUFFER_END_TIME	Number(12)	Drive time approximation for the far representative entry buffer to the QC. This is the maximum drive times within the set of Parallel Buffers allocated. Example: 62
EXIT_BUFFER_START	String(50) See Container and equipment locations	Start of the range of buffers for the work queue in question for leaving the QC. Example: PB.M005
EXIT_BUFFER_START_TIME	Number(12)	Drive time approximation for the QC to the close representative exit buffer. Example: 28
EXIT_BUFFER_END	String(50) See Container and equipment locations	End of the range of buffers for the work queue in question for leaving the QC. Example: PB.M008
EXIT_BUFFER_END_TIME	Number(12)	Drive time approximation for the QC to the far representative exit buffer. Example: 48
UPDATED	Date	Time of last update



ECS will provide these entries for the current work queue and for the next work queue, if the start of the next work queue is within a certain horizon. The implication of this is that in the event of a short next work queue and the current work queue approaching completion, there may be orders for work queues for which there is no corresponding entry in the WORK_QUEUE_BUFFER_STATUS yet.

Calculation of the drive times involving a QC: explanation by example.

Trip waterside TP for instance AW21-5 -> QC for work queue A, approximation of the lower bound drive time

Look up work queue A in the WORK_QUEUE_BUFFER_STATUS table.

Find the ENTRY BUFFER START in the WORK_QUEUE_BUFFER_STATUS table: let's say this is Parallel Buffer 102, with entry buffer time X.

Find the entry " Parallel Buffer 102" -> AW21-5 in the DRIVE_TIME_STATUS table, let's say the drive time is Y.

Approximation: $X+Y$

For the upper bound drive time: take ENTRY BUFFER END instead of ENTRY BUFFER START and follow the recipe above.

Trips QC to QC:

Typically on a trip from a QC to a QC ECS may direct the AGV first to an exit buffer, from the exit buffer to an entry buffer and from there to the QC.

For instance QC51 -> QC56 for work queue A at QC51 to work queue B for QC56

In this case the following figures are relevant:

The lower bound approximation for the part QC51 to exit buffer start belonging to work queue A:
EXIT_BUFFER_START_TIME, (A1)

The upper bound approximation for the part QC51 to exit buffer start belonging to work queue A:
EXIT_BUFFER_END_TIME (A2)

The lower bound approximation for the part entry buffer start to QC56 belonging to work queue B:
ENTRY_BUFFER_START_TIME (B1)

The lower bound approximation for the part entry buffer start to QC56 belonging to work queue B:
ENTRY_BUFFER_END_TIME (B2)

The drive time approximations for EXIT_BUFFER_START for work queue A to ENTRY_BUFFER_START for work queue B. (AB1)

The drive time approximations for EXIT_BUFFER_START for work queue A to ENTRY_BUFFER_END for work queue B. (AB2)

The drive time approximations for EXIT_BUFFER_END for work queue A to ENTRY_BUFFER_START for work queue B. (AB3)

The drive time approximations for EXIT_BUFFER_END for work queue A to ENTRY_BUFFER_END for work queue B. (AB4)

So there are four resulting drive time approximations:

Via ENTRY_BUFFER_START and EXIT_BUFFER_START (the most optimistic) : $A1 + AB1 + B1$

Via ENTRY_BUFFER_START and EXIT_BUFFER_END: $A1 + AB2 + B2$

Via ENTRY_BUFFER_END and EXIT_BUFFER_START: $A2 + AB3 + B1$

Via ENTRY_BUFFER_END and EXIT_BUFFER_END (the most pessimistic) : $A2 + AB4 + B2$

13.4.5.6 AREA_STATUS

This table is used to show the availability of an area in ASC block or in a transfer zone. Each record in the table corresponds to a 20-ft slot, unless it is a transfer point, in which case a 40-ft slot is used. The table is populated only with entries that are managed. For example, the table may only contain the current status of the cells in the reefer bays

within the ASC block since these are the only locations that are blocked by ECS. It does not need to insert records for all cells within the ASC block.

Below are the example rows for a WS TZ – W003. Each record represents a single TP. If the user wants to block or unblock a range of TPs, then each record for every TP in the blocked or unblocked range should be modified so the STATUS is 0 or 1, respectively.

```

WSTP.W003.A 2016-10-18 16:18:40.990 1
WSTP.W003.B 2016-10-18 16:18:41.053 1
WSTP.W003.C 2016-10-18 16:18:41.140 1
WSTP.W003.D 2016-10-18 16:18:40.513 1
WSTP.W003.E 2016-10-18 16:18:40.567 1
WSTP.W003.F 2016-10-18 16:18:40.597 1
WSRACK.W003.B.03.A 2016-10-18 16:18:43.327 1
WSRACK.W003.C.03.A 2016-10-18 16:18:41.337 1
WSRACK.W003.D.03.A 2016-10-18 16:18:43.130 1
WSRACK.W003.E.03.A 2016-10-18 16:18:38.177 1

```

With regard to 40' positions for yard areas in the ASC stacks, one record represents each stack. Can 40' slots be provided? Or only the 20' names? The answer is each record in the table corresponds to a 20-ft slot, unless it is a transfer point, in which case a 40-ft slot is used. So for an 40' location in an ASC block two records will be required.

What if ECS doesn't specify a record in the AREA_STATUS table? The answer is if the ECS doesn't specify a record, then N4 will use whatever stack status has been provided by the user (or if no stack status value comes from N4, then the stack is not blocked).

Can the ECS delete records from the AREA_STATUS table after it inserted them? The answer is no, the ECS may not delete records from the AREA_STATUS table once it's inserted them. N4 will not track deletes from this table. Suppose a record for stack A existed in the AREA_STATUS table and that record previously indicated that stack A was blocked (temp blocked in N4). If the ECS or user deletes the record for stack A, N4 will continue to indicate that stack A is temp blocked. The ECS should always update any existing record in AREA_STATUS to indicate that a stack is block or not. The ECS does not need to provide a status for each and every stack in the yard. Rather the absence of a record for any stack has no effect in N4. If the ECS inserts a record for stack A, then it should continue to update that record in the case that stack is blocked or not in the future.

Column Name	Data Type	Description
AREA	String(50) See Container and equipment locations	Area in ASC block or ASC transfer point Composite primary key - no duplicates
STATUS	Number(12)	0 – Unavailable 1 – Available 2 – Available but occupied by Reefer mechanic (only applies to reefers) Example: 1
UPDATED	Date	Time of last update

13.4.5.7 TRANSFER_POINT_STATUS

The ECS informs N4 about the charging status of transfer points as well as the vehicles allowed to use the transfer point. TRANSFER_POINT_ID are values provided for the stack (the slot without the tier), with the format: TYPE.<blockname>.<lane>.<slot>, where TYPE is WSTP, LSTP etc. The slot value corresponds to the 40 ft position.

The ASSIGNABLE and CLAIM_OWNER columns provide visibility of the ECS transfer point assignment decisions.

Column Name	Data Type	Description
TRANSFER_POINT_ID	String(50) See Container and equipment locations	ID of the transfer point. Always corresponds to the 40 ft location. Primary key, Indexed field - no duplicates Example: WS-02-01
ASSIGNABLE	String(5)	YES(default)/NO Can be NULL. Example: YES
CLAIM_OWNER	String(7)	AGV ID if an AGV is using the TP. Can be NULL. Example: AGV301
LANE_TYPE	String(3)	AGV, UTR, OTR, TOS Type of vehicles which are allowed to use the TP. If "TOS", N4 sets allowed vehicle types internally. Example: AGV
IS_CHARGING	String(5)	YES(default)/NO. YES if the transfer point currently has AGV charging capability. This column was added in N4 3.2.
UPDATED	Date	Time of last update.

13.4.6 Notification and Startup

13.4.6.1 TRIGGER_EVENT_STATUS

TRIGGER_EVENT table is updated when either side inserts or updates (deletions currently are not used since purging of old records in the database tables is performed manually) a record in a different, non-trigger event related table.

There is one record for inserts and one record for updates for every table.

The value of the column NAME is equal to the concatenation of the related table name and "_INSERT" or "_UPDATE".

The records in TRIGGER_EVENT_STATUS are prepopulated with all the possible values prior to startup of N4 and ECS, so after the startup only updates are performed in this table.

A unique index on the column NAME is used to improve performance and reduce possibility of deadlocks.

Example (for SQL Server):

```
create unique index ux_trigger_event_status on trigger_event_status (name)
```


Column Name	Data Type	Description
NAME	String(50) See Container and equipment locations	Event name. Example: ASC_ORDERS_INSERT
PARAM	String(5)	Reserved for possible future use. Can be NULL.
SEQ	Navis DDL creates the correct data type which depends on Database Vendor (Oracle, SQL Server, etc.)	Incremental sequence of the last event. Example: 1003
UPDATED	Date	Time of last update.

13.4.6.2 TRIGGER_STATUS_HANDLED_SPARCS (N4), TRIGGER_STATUS_HANDLED_ECS (ECS)

In TRIGGER_STATUS_HANDLED tables N4 and ECS keep information about last handled events.

TRIGGER_STATUS_HANDLED_SPARCS is updated by N4, TRIGGER_EVENT_STATUS_HANDLED_ECS is updated by ECS.

Both tables have the same format.

Column Name	Data Type	Description
NAME	String(50) See Container and equipment locations	Event name. Example: ASC_ORDERS
SEQ	String(7)	Incremental sequence of the last event. Example: 1003
UPDATED	Date	Time of last update.

13.4.6.3 SPARCS_APPLICATION_STATUS (N4), ECS_APPLICATION_STATUS (ECS)

Both applications publish their status. There is a keep-alive mechanism so that either application can conclude whether the other application is up and responsive.

The heartbeat update rate is configurable in the application which sends the update. The recommended values are between 15 seconds for production and 60 seconds for testing.

Column Name	Data Type	Description
SYSTEM_ID	String(10)	System name Example: N4
START_TIME	Date	Time the application was started, to be updated on each application start

Column Name	Data Type	Description
REFERENCE_TIME	Date	To determine a possible difference in the actual time and the time of the system (like with historical data sets).
STATUS	String(10)	Status of the application. STATUS should not be changed when the ECS is shutting down - leave it as UP or STARTING. N4 determines the status of the ECS software using the heartbeat and the STATUS Values: Starting, Up Example: Up
INTERFACE_VERSION	String(10)	Identifier of the interface or message specification implemented by the ECS Vendor to identify their application version. This column was added in N4 3.2.
UPDATED	Date	Heartbeat update.

13.4.7 Datatypes

13.4.7.1 Locations

A 50 character string is used for container and equipment locations. The below table specifies the maximum number of characters for the location elements

String(50)

Location Element	Maximum Number of Characters
<blockname>	6
<column>	4
<row>	4
<lane>	4
<slot>	4
<tier>	2
<agvid>	7
<ascid>	6
<parallel buffer name>	5
<vessel_visit>	20
<vessel_bay>	3
<vessel_row>	3
<vessel_tier>	3
<qcid>	6

Location Element	Maximum Number of Characters
<qc_lane>	1
<platform_position>	2
<spreader_position>	1
<location_on_agv>	1
<door_direction>	1
<trolley>	1
<hoist_position>	2

Location	Format	Example	Notes
Yard	YARD.<blockname>.<column>.<row>.<tier>	YARD.M009.K.05.1	When used for an ASC location, <tier> is not provided.
Waterside transfer point	WSTP.<blockname>.<lane>.<slot>.<tier>	WSTP.W009.B.02.1 - ASC order WSTP.W009.B - AGV order	<blockname> is the transfer zone block name, not the ASC block name. <tier> is optional. Slot values match the values used for the actual yard. For waterside transfer points N4 does not have to provide slot and tier values in destination columns in AGV orders. ECS selects slots based on availability, requested door direction and container orientation. N4 does provide slot and tier values in ASC orders.
Waterside racks	WSRACK.<blockname>.<lane>.<slot>.<tier>	WSRACK.W009.B.02.1 For landside racks the prefix is CLRACK, otherwise the format is the same as for cantilever transfer points For landside racks the prefix is LSRACK, otherwise the format is the same as for waterside racks.	<blockname> is the transfer zone block name, not the ASC block name. <tier> is optional. Slots have the same values and meaning as for waterside transfer points. Similarly to waterside transfer points, N4 does not have to provide slot and tier values in destination columns in AGV_ORDERS.
Landside Transfer Point			For landside transfer points the prefix is LSTP, otherwise the format is the same as for waterside transfer points. In ASC orders to and from landside transfer points <slot> is always the value used for a 40-ft container and <tier> is always 1.

Location	Format	Example	Notes
CLTP.<blockname>.<lane>.<slot>.<tier>		CLTP.C009.Z.41.1 where Z is the lane (for an ASC order). CLTP.C009.Z.41 (for an AGV order).	<blockname> is the cantilever transfer zone block name, not the ASC block name. <tier> is optional for direct exchanges and AGV Orders. For cantilever transfer points the slot names are represented by the corresponding columns in the adjacent ASC block. For the cantilever transfer point below (in green) 41 and 42 will be the slots for each 20-ft slot. Cantilever transfer points in the same cantilever block will have the same row (and, correspondingly, the same lane) value. For cantilever transfer points N4 does not have to provide tier values in destination columns in AGV orders. N4 does provide tier values in ASC orders. Reminder: The 40 ft location name value is always used for AGV orders since they occupy a 40 ft location.
Parallel Buffer	PB.<parallel buffer name>	PB.M001	
AGV	AGV.<agvid>.<location_on_agv>.<door_direction>		<location_on_agv> can be F ("front"), R ("rear"), C ("center") or U ("unknown"). <door_direction> can be F ("front"), R ("rear"), or U ("unknown").
ASC Spreader	ASCSPREADER.<ascid>		
ASC Parking Locations	YARD.<blockname>.LS-PARK YARD.<blockname>.WS-PARK YARD.<blockname>.LS-MAINT YARD.<blockname>.WS-MAINT		A park position is a position where the ASC can be sent to by an automatic order. For an ASC to go the the maintenance position, the crane needs to be driven there by a mechanic.
Vessel location	VESSEL.<vessel_visit>.<vessel_bay>.<vessel_row>.<vessel_tier> VESSEL VESSEL.<vessel_visit> VESSEL.<vessel_visit>.<vessel_bay>.<vessel_row> VESSEL.<vessel_visit>.<vessel_bay>.<vessel_row>.<vessel_tier>		In exceptional cases it is possible that ECS will provide less information.
QC TP	QCTP.<qcid>.<qc_lane>		<qc_lane> is mandatory for ECS unless used for AGVs on the way to QC which have not yet passed a parallel buffer.
QC Spreader	QCSPREADER.<qcid>.<trolley>.<hoist_position>.<spreader_position>		
QC Platform	QCPLATFORM.<qcid>.<platform_position>.<spreader_position>		<platform_position> can be: WS: waterside platform LS: landside platform
QC Stack	QCSTACK.<qcid>.<spreader_position>		Used for the portal area of the QCs and for the backreach of quay cranes for moves or not to/from AGV (like at the barge quay of APMT).
Rail locations			RCTP.<blockname>.<lane>.<slot>.<tier> RCRACK.<blockname>.<lane>.<slot>.<tier> N4 does not have to provide slot and tier values in destination columns in AGV orders.
	MAINT.<blockname>.<lane> - maintenance TPs	MAINT.<MAIN1>.<T01>	

Location	Format	Example	Notes
	RECH.<blockname>.<lane> - recharge TPs	RECH.<BES01>.<P101>	
	X-RAY.<blockname>.<lane> - x-ray TPs	X-RAY.<XRAY>.<01>	
	OUT.NOT_ON_TERMINAL	OUT.NOT_ON_TERMINAL	AGV location when the AGV leaves the terminal
UNDEFINED			used only for AGV_STATUS.LOCATION when an AGV is moving (or stopped) between known locations.
Null			Null - used when there is physically no location. The current usages are: - for AGV_STATUS.NEXT_LOCATION when the AGV is idle - for ASC_COMMANDS.ACTUAL_DESTINATION when ECS creates an unsolicited ASC command and does not know the destination - for ASC_ORDERS.PLANNED_DESTINATION when ECS creates an unsolicited ASC command and does not know the destination - as a response to the ASC command.
UNKNOWN			– used in rare cases when N4 or ECS cannot provide location. For example, if a QC lifted a container from an AGV, and ECS cannot identify the AGV, the container location will be reported as AGV.UNKNOWN

QC Location Elements

QC Location Element	Values
<trolley>	M for main P for portal
<spreader_position>	L: twin position in the lower bollard direction (for twinned and quad containers) C: center position (for single containers) H: twin position in the higher bollard direction (for twinned and quad containers)
<hoist_position>	WS: waterside position, dual hoist (for tandem and quad containers) C: center position, single hoist (for single containers) LS: landside position, dual hoist (for tandem and quad containers)

13.4.7.2 Example diagram for ASC block, waterside transfer point and waterside locations

In N4-ECS interface, columns in an ASC block run along the width of containers and rows run along the length of containers. To illustrate this statement, on the diagram above transfer point lanes, rack lanes and ASC block columns are represented by letters - A to E for transfer point and rack lanes; and A to K for ASC block columns. Transfer point and rack slots and ASC block rows are represented by numbers.

Field	Example
Container length	610, 1219, 1372
Container height	244, 259, 279, 290
Date/Time	25-MAY-2005 12:00:05

13.4.7.3 Work Status

String(25)

Values	Meaning
UNDEFINED	ASC in the OFFLINE state
IDLE	ASC does not have an order
JOB_EXECUTION	ASC is performing an order
WAITING_FOR_OTHER_ASC	to continue performing the current order ASC is waiting for the other ASC to free the area the ASC needs
WAITING_FOR_RCC	Waiting for Remote Control Centre
SUSPENDED	User has put ASC on hold to be resumed or aborted later

13.4.7.4 Technical status

String(10)

GREEN	Can be used for any tasks
YELLOW	Can be used for any tasks, but has a minor malfunction
ORANGE	Can finish the current order, but should not be used for container-related orders after that. Repositioning orders are allowed.
RED	Cannot be used, needs repair

13.4.7.5 Orientation

String(12)

HIGHROW, LOWROW, Highbollard, Lowbollard, TRACKSIDE, YARDSIDE, UNKNOWN, NA.

Used for door direction and container and equipment orientation at waterside and locations.

If a container does not have doors, NA is used. NA also can be used when the system does not care about orientation.

UNKNOWN means that the container does have door direction but the system does not know it.

HIGHROW and LOWROW are used for specifying door orientation in ASC areas.

HIGHROW means that the door in the ASC block the door should face the side of the block with the higher row numbers.

LOWROW means that the door in the ASC block the door should face the side of the block with the lower row numbers.

Example – if the outmost row on one side of an ASC block has value 1 and the outmost row on the other side of the same block has value 90 and the door should face the row 90, the door direction value is HIGHROW, for the opposite door direction it will be LOWROW.

HIGHBOLLARD and LOWBOLLARD are used for orientation along the quay. Highbollards and lowbollards are defined by comparing bollard names.

Example - three bollards - B010, B011, B012.

For B011 the LOWBOLLARD direction will be in the direction of B010 and the Highbollard - in the direction of B012.

TRACKSIDE and YARDSIDE are used for container door direction on rail TPs which are perpendicular to the rail.

TRACKSIDE is the side of the TP which is next to the tracks and YARDSIDE is the opposite side.

13.4.7.6 ASC origin or destination vehicle type

Number(I2)

0	NONE
1	AGV
2	UTR
3	OTR
4	20ft
5	30ft
6	40/45ft
7	2x20ft
8	20+40/ 45ft
9	40/45+ 20ft

For OTRs values from 4 to 9 are received by N4 from the gate system and transmitted to ECS without any changes.

Default == NONE.

13.4.7.7 Position on truck chassis

Number(I2)

Value from 1 to 5.

For OTRS this value is received from the gate system and transmitted to ECS without any changes.

For UTRs N4 will use values from 1 to 3 - 2 for 40ft containers, 1 for 20ft containers in the position next to the URT's cabin, 3 for 20 ft containers in the position next to the ASC block.

If there is no truck involved, the column value is null.

AGV_ORDERS

this will drive the move kinds allowed for gate moves.

UTR includes yard, rail load, rail discharge

AGV is yard move, vessel discharge, vessel load

Defined in TransferPointStatusUpdateUtil.java

The COMMAND always contains the final location of the AGV, and container when the AGV is laden, in the FINAL_LOCATION_1 (FINAL_LOCATION_2 when handling twins).

Part 14

Crane Automation API Technical Specification

This document represents **only** a high-level description of the **Crane Automation API**. For a detailed description of the individual attributes used in each message, refer to the schema definition in the distribution package located on the Navis N4 host at `..\Program Files\Navis\sparc4\webapps\apex\WEB-INF\`.

The *N4_CraneAutomationSystem(CAS)_SDK.zip* is a separate package that contains this document, as well as relevant artifacts, available for download from the Navis Collaboration Center.

In This Section

Crane Automation API.	486
Crane Automation System (CAS).	486
Communication protocol.	486
Web service hosted by N4.	487
Web service hosted by CAS.	488
Common Functionality.	490
Sample messages.	490
Sample client implementation (Java).	490
WSDL Definitions.	491
Interface Specific Details.	492

14.1 Crane Automation API

The **Crane Automation API** is a generic web service API that is independent of any system provider or solution. You can use the Crane Automation API to customize the interactions between N4 and the Quay Crane and the Rail Crane.

14.2 Crane Automation System (CAS)

The Crane Automation API provides an interface between the **Crane Automation System (CAS)** and N4. The following table displays the interactions between CAS and N4.

Requests and Responses in the Quay and Rail Crane Automation API

Interface	Requester	Request Content	Responder	Response Content
Carrier Visit Update	N4	Vessel Visit details (SNX)	CAS	Acknowledgment/ messages
Container List	CAS	Carrier Visit	N4	CA Unit
Onboard Container List Update	N4	CA Unit CA Position (current)	CAS	Acknowledgment/ messages
Crane Work List Update	N4	CA Unit CA Position (planned) CA Work Instruction	CAS	Acknowledgment/ messages
Carrier Geometry	CAS	Carrier Visit	N4	Vessel Geometry (SNX)
Unit Capture	CAS	CA Unit	N4	Acknowledgment/ messages
Unit Position Update	CAS	CA Unit	N4	Acknowledgment/ messages
Ready to Transfer	N4	CA Unit	CAS	Acknowledgment/ messages

This table lists all the messages available via the CAS API. However, you can decide which messages to use for your site. For example, you may use only the message to support the OCR operation.

14.3 Communication protocol

The CAS API uses SOAP-based web services to interact with N4. These web services can be grouped into three categories:

- CAS requests information and N4 responds with information:
 - Carrier Geometry
 - Container List
- CAS sends information to N4 based on some event occurring in CAS.
 - Unit Capture
 - Unit Position Update

- N4 sends information to CAS based on some event occurring in N4.
 - Carrier Visit Update
 - Onboard Container List Update
 - Crane Work List Update
 - Ready to Transfer

N4 provides a basic Groovy implementation for the web service communication for each of these requests. This ensures that you can extend and customize the implementation according to your requirements.

14.4 Web service hosted by N4

When CAS initiates the communication, either in the form of a request for information or sending a message to N4, the CAS client calls the web service hosted by N4.

To understand how the N4 external web services are created, see the *N4 Compatible External Web Service* section in the *Navis N4: SDK*.

WSDL definition

N4 uses a Groovy-based generic web service to process the requests from CAS. Since this is a generic web service, the Web Services Description Language (WSDL) definition (see [genericWebService.wsdl](#)) includes a single operation to handle the client call from CAS. This approach reduces the time spent on creating boilerplate and duplicate code for different message types and enhances maintainability.

The single operation from the WSDL translates to the following call in the code:

```
GenericInvokeResponseWsType response = port.genericInvoke(scope, xmlPayload);
```

In the code snippet above:

- The `response` object contains messages and/or XML response from N4.
- The `scope` attribute contains the scope information required by N4.
- The `xmlPayload` attribute contains the XML sent by the client in string form and contains the request/message.

XML payload

The XML sent by the client as a payload contains the following information:

- The type of the request/message.
- The Groovy script that handles the request/message on the N4 side.
- The parameters in the form of name-value pairs.

Sample Carrier Geometry request

The following is an example of the XML payload for the Carrier Geometry request:

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='RAIL' />
  </parameters>
</groovy>
```

The elements and attributes included in the above message are:

- `This issue was resolved with Groovy code developed for the customer.`: Indicates that the request/message is handled using Groovy code.
- `<class-location=" code-extension">`: Indicates that the request is handled by a code extension in N4.
- `<class-name="DefaultN4InboundCasRequestHandler">`: Indicates that the name of the code extension that handles the request/message is `DefaultN4InboundCasRequestHandler`.

You can use the **Administration > System > Code Extensions > Code Extensions view** to see the other available system-seeded code extensions related to the CAS API:

- `DefaultN4InboundCasRequestHandler`: To handle the requests from CAS.
- `DefaultN4InboundCasMessageHandler`: To handle informational messages from CAS.
- `Default25InboundCasMessageHandler`: To handle requests for information from CAS.



Manual sites should use this code extension in place of the `DefaultN4InboundCasMessageHandler` code extension.

Sample Carrier Geometry response

As described above, the `response` object contains the messages and/or payload. The sample client code provided in a later section in this document shows how to extract the messages and the response payload from the response object returned by N4.

The following is an example of the response payload for the sample request above:

```
<payload>
  <parameters>
    <parameter id='requestType' value='carrierGeometryQuery' />
    <parameter id='visitType' value='VESSEL' />
    <parameter id='visitId' value='12311' />
  </parameters>
  <additional-info>
    <field id='aField' value='aValue' />
  </additional-info>
  <!--actual data element(s) -->
</payload>
```

The `actual data element(s)` are specific to the type of interface.

The elements and attributes included in the above message are:

- `<parameters>`: The parameters are sent by CAS as part of the original request. They are applicable only when CAS requests information from N4. The messages that are initiated based on some event happening in N4 do not contain this element.
- `<additional-info>`: This element is used to pass any additional arbitrary information from N4 to CAS. This is the information that is not part of the SNX.
- `Actual data element(s)`: This section contains the actual data and it might be either in strict SNX format provided by N4 or in CAS-specific data types.

14.5 Web service hosted by CAS

When N4 initiates the communication, N4 becomes the client calling the web service hosted by CAS.

WSDL definition

CAS uses a basic web service to process the requests from N4. The WSDL definition (see [basicWebService.wsdl](#)) includes a single operation to handle the client calls from N4. The basic web service is similar to the generic web service but is even simpler as the parameters and the response are in the string format.

The signature of the operation is:

```
String response = port.basicInvoke(scope, xmlPayload);
```

In the code snippet above:

- The `response` object can be an arbitrary string or an XML document that is parsed on the N4 side.
- The `scope` attribute contains the mandatory scope information required by N4 in string format.
- The `xmlPayload` attribute contains the XML sent by N4 (in string form) and contains the message.

By default, N4 sends these messages to an endpoint named `CAS-Outbound` defined in the **Integration Services** view in N4. The **Integration Type** for the endpoint is Web Service.

Starting N4 release 2.6, you can send outbound messages to CAS to multiple external web service endpoints by specifying the **Group** as `CAS-Outbound` for each outbound service.

If you do not define any entry with **Group** as `CAS-Outbound`, the system falls back to the earlier mechanism of finding an integration service with **Name** = `CAS-Outbound`. To modify this behavior, you can override the `DefaultN4OutboundCasMessageHandler` code extension. For more details on how to override a code extension, see the **Code Extensions** view topic in the N4 online help.

To create an outbound web service in N4:

1. In N4, go to **Administration > Settings > Integration Services**.
2. Add a new service with the following details:

- **Type: Web Service**
- **Name: CAS-Outbound**

Starting N4 release 2.6, you can define multiple endpoints by specifying the **Group** as `CAS-Outbound` for each outbound service in the group.

- **URL**
- **Login** credentials

3. Click **Save**.

Sample XML payload

The following is an example of the XML payload for the Carrier Visit Update request:

```
<payload >
  <additional-info>
    <field id='fieldChanges' value='null-20120915220000PDT, Inbound-Arrived' />
    <field id='visitType' value='vesselVisit' />
  </additional-info>
  <vessel-visit id="BOPL" visit-phase="ARRIVED" vessel-id="BOPL" vessel-name="BOPL
Name" service-id="BOPL" ETA="2012-09-09T00:00:00" ETD="2012-09-16T00:00:00"
ATA="2012-09-09T00:00:00" operator-id="JASON" is-common-carrier="N" is-dray-off="N" is-
no-client-access="N" published-eta="2012-09-09T00:00:00" published-
etd="2012-09-16T00:00:00" out-call-number="1" out-voy-nbr="101" in-call-number="1" in-
voy-nbr="101" facility="FCY111">
```

```

    <lines>
      <line id="JASON" in-voy-nbr="101" out-voy-nbr="101" />
    </lines>
  </vessel-visit>
</payload>

```

The elements and attributes included in the above message are:

- `<additional-info>` This element is used to pass additional information that is not part of the SNX to CAS. For example, in the above sample, changes to the, 'vessel visit' field, which triggered this message, are also sent as part of the message.
- `<ship-geometry>` This element contains the actual data. This is different depending on the type of message (see the sample messages/xsd in distribution for different types). It might be in strict SNX format, provided by N4, or CAS types. In this example, it is a vessel visit SNX.

When CAS sends a message in response to an N4 request/update, the default implementation in N4 logs the message but does not process the message any further. You must enhance the reference implementation to perform any additional processing in N4, based on the response from CAS.

14.6 Common Functionality

The common functionality used by the code extensions defined in sections above is provided in the classes `CasHelper` and `CasMessageHelper`, both included in system-seeded code extensions of type library. Code extension `CasHelper` is for the business related functionality while `CasMessageHelper` contains various messages used by the code extensions. Each of the following system-seeded code extensions refer to these classes:

- `DefaultN4InboundCasRequestHandler`
- `DefaultN4InboundCasMessageHandler`
- `DefaultN4OutboundCasMessageHandler` (*only refers to CasHelper*)
- `Default25InboundCasMessageHandler`

If you want to change the functionality that's common to all of these code extensions, create new code extensions based on `CasHelper` and `CasMessageHelper`, then update all references to `CasHelper` and `CasMessageHelper` in the code extensions above (`DefaultN4InboundCasRequestHandler`, `DefaultN4InboundCaseMessageHandler`, etc) to the names of your new code extensions. See the online help for the Code Extensions view in N4 for more details about creating code extensions.

14.7 Sample messages

You can access sample messages from the *N4_CraneAutomationSystem(CAS)_SDK.zip* available on the Navis Customer Portal. The sample messages are included under the [artifacts/samples](#) folder. The steps to perform any specific actions for a message, for example set up a general notice for Carrier Visit Update message, are included in the [readme.txt](#) in the distribution.

14.8 Sample client implementation (Java)

A sample implementation of the generic web service, N4 client to request information or to send messages (like Unit Capture) is provided in the *N4_CraneAutomationSystem(CAS)_SDK.zip*, available from the Navis Customer Portal.

14.9 WSDL Definitions

The *N4_CraneAutomationSystem(CAS)_SDK.zip* also includes the WSDL definitions for the various web service. You can also access these WSDL files from an N4 instance using the URL formats specified below.

- Generic web service hosted by N4:

<http://<serverName>:<port>/apex/services/argoservice?wsdl>

- Basic web service hosted by CAS:

<http://<serverName>:<port>/apex/services/argobasicservice?wsdl>

Interface Specific Details

In This Section

Carrier Geometry Request.	493
Carrier Visit Update Message.	504
Container List Request.	505
Crane Work List Update Message.	512
Load Swap Request Message.	514
Onboard Unit Update Message.	516
QC Updates via CAS.	518
Rail Messages in CAS API.	520
Ready To Transfer Message.	524
Unit Capture Messages.	525
Unit Position Update Message.	532
Miscellaneous Notes.	535

14.9.1 Carrier Geometry Request

14.9.1.1 Carrier Geometry Request

This message is a request made by CAS to N4; N4 responds by sending the vessel or rail geometry to CAS.

The CAS sends a request with:

- Visit Type ("VESSEL" or "RAIL")
- Visit ID (mandatory for ship geometry request, optional for rail geometry request)

The response from N4 contains:

- Visit Type (VESSEL or RAIL)
- Visit ID
- Messages (Success/Warnings/Errors)
for ship geometry request
- Ship model (Ship Model SNX)
for rail geometry request
- Railcar platform positions as well as some attributes for the railcar and railcar visit

Optional Parameter(s):

None, but you can add more by replacing the system-provided Groovy with custom Groovy.

14.9.1.2 Sample Request Inputs

carrierGeometryRequest for single railcar

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='visitId' value='KCS01' />
    <parameter id='yardOriginId' value='ORIGIN' />
  </parameters>
</groovy>
```

carrierGeometryRequest for a train visit

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='trainVisitId' value='ASPROS01' />
    <parameter id='yardOriginId' value='ORIGIN' />
  </parameters>
</groovy>
```

carrierGeometryRequest for single track

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
```

```

    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='trackName' value='TR' />
    <parameter id='yardOriginId' value='ORIGIN' />
  </parameters>
</groovy>

```

carrierGeometryRequest for rail with yardOriginId provided as an input parameter

```

<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='visitId' value='KCS01' />
    <parameter id='yardOriginId' value='ORIGIN' />
  </parameters>
</groovy>

```

carrierGeometryRequest for all active railcar visits

```

<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='yardOriginId' value='ORIGIN' />
  </parameters>
</groovy>

```

carrierGeometryRequest for a vessel

```

<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='carrierGeometryRequest' />
    <parameter id='visitType' value='VESSEL' />
    <parameter id='visitId' value='CASTOR01' />
  </parameters>
</groovy>

```

14.9.1.3 Sample Response Message for Rail Request (from N4)

```

<?xml version="1.0" encoding="UTF-8"?>
<payload>
  <parameters>
    <parameter id="requestType" value="carrierGeometryRequest" />
    <parameter id="visitType" value="RAIL" />
    <parameter id="trainVisitId" value="ASPROS01" />
    <parameter id="yardOriginId" value="ORIGIN" />
  </parameters>
  <railcars>
    <railCar id="RAILIN01" track="A" sequence="1" spotting-status="NOTSPOTTED"
      flipped="Y" segment-break-at-low-end="N" inspection-status="NOT_INSPECTED" is-
      damaged="N" inbound-train-visit-id="AGVTRIN01" outbound-train-visit-id="AGVTRIN01" />
    <railCar id="RCRA1" track="TRA" sequence="1" spotting-status="TP" flipped="N"
      segment-break-at-low-end="N" inspection-status="NOT_INSPECTED" is-damaged="N" transfer-
      point="" trackXCoord="7440" inbound-train-visit-id="TVA" outbound-train-visit-id="TVA">
      <platform label="1">

```

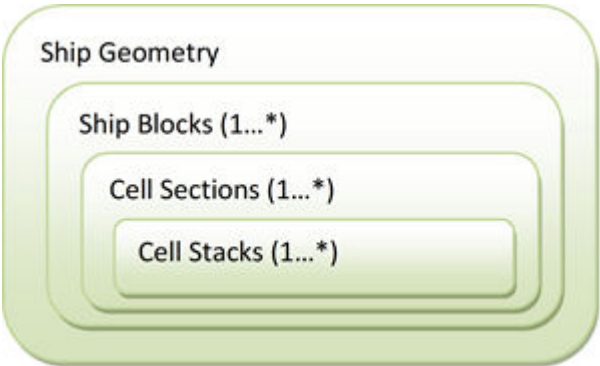
```

        <centroid cartesian-X="5910.64" cartesian-Y="0.0"
latitude="37.793559738471664" longitude="-122.2732872562487" />
        <railSlot>
            <slot-position loc-type="RAILCAR" block="RCRA1" row="1"
column="1" />
            <geoPos cartesian-X="5910.64" cartesian-Y="0.0"
latitude="37.793559738471664" longitude="-122.2732872562487" />
        </railSlot>
        <railSlot>
            <slot-position loc-type="RAILCAR" block="RCRA1" row="1"
column="2" />
            <geoPos cartesian-X="5910.64" cartesian-Y="0.0"
latitude="37.793559738471664" longitude="-122.2732872562487" />
        </railSlot>
    </platform>
    <platform label="2">
        <centroid cartesian-X="6071.64" cartesian-Y="0.0"
latitude="37.79355526410135" longitude="-122.27326983026032" />
        <railSlot>
            <slot-position loc-type="RAILCAR" block="RCRA1" row="2"
column="1" />
            <geoPos cartesian-X="6071.639999999999" cartesian-Y="0.0"
latitude="37.79355526410135" longitude="-122.27326983026032" />
        </railSlot>
        <railSlot>
            <slot-position loc-type="RAILCAR" block="RCRA1" row="2"
column="2" />
            <geoPos cartesian-X="6071.639999999999" cartesian-Y="0.0"
latitude="37.79355526410135" longitude="-122.27326983026032" />
        </railSlot>
    </platform>
    <platform label="3">
        <centroid cartesian-X="6232.64" cartesian-Y="0.0"
latitude="37.79355078972847" longitude="-122.27325240427193" />
        <railSlot>
            <slot-position loc-type="RAILCAR" block="RCRA1" row="3"
column="1" />
            <geoPos cartesian-X="6232.639999999999" cartesian-Y="0.0"
latitude="37.79355078972847" longitude="-122.27325240427193" />
        </railSlot>
        <railSlot>
            <slot-position loc-type="RAILCAR" block="RCRA1" row="3"
column="2" />
            <geoPos cartesian-X="6232.639999999999" cartesian-Y="0.0"
latitude="37.79355078972847" longitude="-122.27325240427193" />
        </railSlot>
    </platform>
</railCar>
</railcars>
</payload>

```

14.9.1.4 Ship Bin Model Structure

The ship bin model is partially composed of CellStacks owned by CellSections owned by ShipBlocks owned by one ShipBinModel object. See the diagram below:



Ship Block (side view)

Example 1: The ship block below groups all the cell sections above deck



Example 2: The ship block below groups all the cell sections below deck:



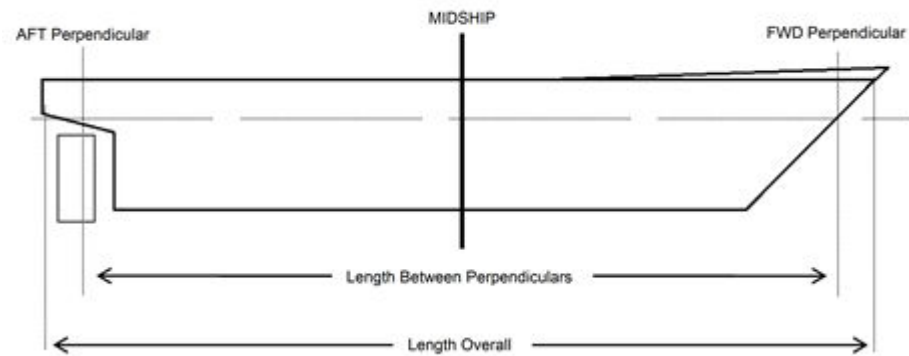
Cell Section (front view)



Cell Stack (front view)



Ship Model Overview



14.9.1.5 Message Description

Elements table

The following table displays the elements of the CAS API message.

Name	Parent Element	Description	Multiplicity
<parameters>	N/A	List of parameters included in the request message	1
<parameter>	<parameters>	Specific parameter of the request message	3...*
<additional-info>	N/A	Any additional information which is not part of SNX but which the CAS is interested in	1...*
<ship-geometry>	N/A	Container for ship geometry	1
<ship-model>	<ship-geometry>	Container for ship model	1

Name	Parent Element	Description	Multiplicity
<bin-name-table>	<ship-model>	Naming table for blocks, sections and stack. It allows customized naming conventions for rows, columns and tiers	1...*
<bin-name>	<bin-name-table>	Specific item within a bin-name-table	1...*
<ship-block>	<ship-geometry>	Describes an assembly of cell sections, i.e. all the cell sections above deck	1...*
<cell-section>	<ship-geometry>	Describes a section/bay on the vessel by grouping cell stacks	1...*
<cell-stack>	<ship-geometry>	Defines a stack on the vessel by grouping a vertical assembly of cells	1...*

Attributes tables

This section displays the attributes for the various elements in the CAS API message.

Element <parameter>

Name	Description	Sample Data	Data Type	Required?
id	Parameter id	requestType	String	Required
value	Parameter value	carrierGeometryRequest	String	Required

Element <ship-model>

Name	Description	Sample Data	Data Type	Required?
name	Ship bin model name	C10	String	Optional

Name	Description	Sample Data	Data Type	Required?
lcg-ref	Reference of longitudinal center of gravity: - MIDSHIPS : The midpoint between the aft and the forward perpendicular - AFT_P : It is the line perpendicular to the waterplane and passing through the centerline of the rudder pintles - FWD_P : A vertical line through the intersection of the design waterline and the forward end of a ship	MIDSHIPS AFT_PERPENDICULAR FWD_PERPENDICULAR	String	Optional
lcg-mode	Mode of longitudinal center of gravity KNOWN required for automation projects	KNOWN ESTIMATED	String	Optional
tcg-mode	Mode of transverse center of gravity KNOWN required for automation projects	KNOWN ESTIMATED	String	Optional

Element <bin-name-table>

Name	Description	Sample Data	Data Type	Required?
table-index	Table index	1	Long	Optional
table-name	Table name	C1020	String	Optional

Element <bin-name>

Name	Description	Sample Data	Data Type	Required?
logical-position	Logical position on a section, cell, stack, tier...	1	String	Optional
user-name	Name defined by the user for a certain logic position	01	String	Optional

Element <ship-block>

Name	Description	Sample Data	Data Type	Required?
name	Ship block name	A	String	Required
parent-name	Parent element containing this element	C10	String	Required
conv-id-row-alt	Link with naming convention table	2	Long	Optional
conv-id-row-std	Link with naming convention table	1	Long	Optional
conv-id-tier	Link with naming convention table		Long	Optional

Element <cell-section>

Name	Description	Sample Data	Data Type	Required?
name	Section name	B001	String	Required
parent-name	Parent element containing this element	B	String	Required
csn-conv-id-tier	Specifies the naming table that defines the tier labels for cell sections	6	Long	Optional
row-index	Row index	1	Long	Required
row-paired-from	40' extending to previous row	0	Long	Optional
row-paired-into	40' extending to next row. 20' occupies position 3, 40' occupies position 3 and 4	2	Long	Optional
conv-id-col	Logical index on the naming conv table	3	Long	Optional
lcg-40	LCG 40 alternate (40' containers)	11040.0 (cm)	Double	Optional
lcg-20	LCG 20 alternate (20' containers)	10840.0 (cm)	Double	Optional
lcg-45	LCG 45 alternate (45' containers)	9600.0 (cm)	Double	Optional
basic-lengths-allowed	Only containers of lengths specified will be allowed be stowed in this section	[40, 45]	String	Optional

Element <cell-stack>

Name	Description	Sample Data	Data Type	Required?
name	Stack name	B00100	String	Required
parent-name	Parent element containing this element	B001	String	Required
z-index-min	Min tier The bottom-most slot on the ship is tier 1. Some below-deck ship stacks do not go all the way to the bottom of the vessel's hull due to the ship's flare (or hull's curve). i.e., often center stacks have a min index of 1, but outboard ones might have a min index of 5, for example.	5	Long	Optional
z-index-max	Max tier	8	Long	Optional
col-index	Logical index Starboard is odd Port is even	0	Long	Required
lcg-40	LCG 40 alternate (40' containers)	11040.0 (cm)	Double	Optional
lcg-20	LCG 20 alternate (20' containers)	10840.0 (cm)	Double	Optional
lcg-45	LCG 45 alternate (45' containers)	9600.0 (cm)	Double	Optional
tcg	Transverse center of gravity	32767.0 (cm)	Double	Optional

Sample Response Message for Vessel Request (from N4)

```
<payload xmlns="http://www.w3schools.com"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-outbound-ws-cas-
response-type.xsd">
  <parameters>
    <parameter id='requestType' value='carrierGeometryQuery' />
    <parameter id='visitType' value='VESSEL' />
    <parameter id='visitId' value='12311' />
  </parameters>
  <!--Any additional information can be passed in the following section which is not
part of SNX but which the CAS is interested in-->
```

```

<additional-info>
  <field id='someField' value='someFieldValue' />
</additional-info>
<ship-geometry>
  <ship-model name="C10" lcg-ref="MIDSHIPS" lcg-mode="KNOWN" tcg-mode="KNOWN">
    <bin-name-table table-index="1" table-name="C1020">
      <bin-name logical-position="1" user-name="01" />
      <bin-name logical-position="3" user-name="05" />
      <bin-name logical-position="5" user-name="09" />
      <bin-name logical-position="6" user-name="11" />
      <bin-name logical-position="7" user-name="13" />
      .
      .
      .
      <bin-name logical-position="27" user-name="53" />
      <bin-name logical-position="29" user-name="57" />
      <bin-name logical-position="30" user-name="59" />
      <bin-name logical-position="31" user-name="61" />
    </bin-name-table>

    <bin-name-table table-index="4" table-name="C10">
      <bin-name logical-position="0" user-name="00" />
      <bin-name logical-position="1" user-name="01" />
      <bin-name logical-position="2" user-name="02" />
      <bin-name logical-position="3" user-name="03" />
      <bin-name logical-position="4" user-name="04" />
      .
      .
      .
      <bin-name logical-position="13" user-name="13" />
      <bin-name logical-position="14" user-name="14" />
      <bin-name logical-position="15" user-name="15" />
      <bin-name logical-position="16" user-name="16" />
      <bin-name logical-position="17" user-name="" />
      <bin-name logical-position="18" user-name="&quot;;" />
    </bin-name-table>
  </ship-model>
  <ship-block name="B" parent-name="C10" conv-id-row-alt="2" conv-id-row-std="1" />
  <ship-block name="A" parent-name="C10" conv-id-row-alt="2" conv-id-row-std="1" />
  <cell-section name="B001" parent-name="B" csn-conv-id-tier="6" row-index="1" row-
paired-from="0" row-paired-into="2" conv-id-col="3" lcg-40="11040.0" basic-lengths-
allowed="[40, 45]" />
  <cell-section name="A001" parent-name="A" csn-conv-id-tier="6" row-index="1" row-
paired-from="0" row-paired-into="2" conv-id-col="3" lcg-20="10840.0" lcg-40="11040.0"
lcg-45="11040.0" basic-lengths-allowed="[20]" />
  <cell-section name="B005" parent-name="B" csn-conv-id-tier="6" row-index="3" row-
paired-from="0" row-paired-into="4" conv-id-col="3" lcg-40="9600.0" basic-lengths-
allowed="[40, 45]" />
  <cell-section name="A005" parent-name="A" csn-conv-id-tier="6" row-index="3" row-
paired-from="0" row-paired-into="4" conv-id-col="3" lcg-40="9600.0" lcg-45="9600.0"
basic-lengths-allowed="[40, 45]" />
  .
  .
  .
  <cell-section name="A059" parent-name="A" csn-conv-id-tier="6" row-index="30" row-

```

```

paired-from="29" row-paired-into="0" conv-id-col="4" lcg-20="-11743.0"
lcg-40="-11360.0" lcg-45="-11360.0" basic-lengths-allowed="[20]" />
  <cell-section name="A061" parent-name="A" csn-conv-id-tier="6" row-index="31" row-
paired-from="0" row-paired-into="32" conv-id-col="4" lcg-40="-12800.0"
lcg-45="-12800.0" basic-lengths-allowed="[40, 45]" />
  <cell-stack name="B00100" parent-name="B001" z-index-min="5" z-index-max="8" col-
index="0" lcg-40="11040.0" tcg="32767.0" />
  <cell-stack name="B00101" parent-name="B001" z-index-min="5" z-index-max="8" col-
index="1" lcg-40="11040.0" tcg="131.0" />
  <cell-stack name="B00102" parent-name="B001" z-index-min="5" z-index-max="8" col-
index="2" lcg-40="11040.0" tcg="-131.0" />
  <cell-stack name="B00103" parent-name="B001" z-index-min="6" z-index-max="8" col-
index="3" lcg-40="11040.0" tcg="392.0" />
  <cell-stack name="B00104" parent-name="B001" z-index-min="6" z-index-max="8" col-
index="4" lcg-40="11040.0" tcg="-392.0" />
    .
    .
    .
  <cell-stack name="A06113" parent-name="A061" z-index-min="1" z-index-max="5" col-
index="13" lcg-40="-12800.0" lcg-45="-12800.0" tcg="1597.0" />
  <cell-stack name="A06114" parent-name="A061" z-index-min="1" z-index-max="5" col-
index="14" lcg-40="-12800.0" lcg-45="-12800.0" tcg="-1597.0" />
</ship-geometry>
</payload>

```

14.9.1.6 20' Bays Paired for 40' Containers: No Container Lengths Allowed

When you have paired slots in your vessel bays for handling 40' containers, the response message may not contain a line item for both bay cell sections.

This occurs in instances, such as with overhung slots, where those slots have no container capacity but 40' containers extend into them from another bay. With these paired slots, the second slot is configured with no *container lengths allowed*. Because of *no container lengths allowed* being selected for it, it is excluded from the response message.

This example shows how the expected behavior is displayed in a response message:

```

<cell-section name="A075" parent-name="A" csn-conv-id-tier="4" row-index="38" row-
paired-from="0" row-paired-
into="37" conv-id-col="3" lcg-20="1287.0" lcg-40="1670.0" lcg-45="1670.0" />
<cell-section name="A079" parent-name="A" csn-conv-id-tier="4" row-index="40" row-
paired-from="0" row-paired-
into="39" conv-id-col="3" lcg-40="219.0" />

```

In the Ship Editor, bay A079 is configured as a 20' bay that allows 40' containers and is linked to the adjacent bay A077.

Bay A077 is configured as *no container lengths allowed*, but is the bay where 40' containers will extend into from bay A079.

This leads to cell-section A077 not being displayed in the message.

14.9.2 Carrier Visit Update Message

14.9.2.1 Carrier Visit Update Message

When you update a vessel visit or a train visit to one of the following visit phases, N4 sends the Carrier Visit Update message to CAS:

- Arrived
- Working
- Completed
- Departed

Prerequisites

- [Set up a General Notice in N4 \(on page 504\)](#) that invokes the code extension to send the Carrier Visit Update message

The request from N4 includes the following information:

- Visit Type
- Visit Details (Vessel/Train Visit SNX)
- Cranes ("CR #1, CR #2, ...")

14.9.2.2 Sample Message (from N4)

```
<?xml version="1.0"?> -<payload xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-outbound-ws-cas-message-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://www.w3schools.com">
  -<additional-info>
    <field value="carrierVisitUpdate" id="requestType"/>
    <field value="VESSEL" id="visitType"/>
    <field value="C1,C2,C3,C4" id="craneId"/>
    <!-- A comma separated list of cranes -->
    <field value="null-20120915220000PDT, Inbound-Arrived" id="fieldChanges"/>
  </additional-info>
  -<vessel-visit id="BOPL" facility="FCY111" in-voy-nbr="101" in-call-number="1" out-voy-nbr="101" out-call-number="1" published-etd="2012-09-16T00:00:00" published-eta="2012-09-09T00:00:00" is-no-client-access="N" is-dray-off="N" is-common-carrier="N" operator-id="JASON" ATA="2012-09-09T00:00:00" ETD="2012-09-16T00:00:00" ETA="2012-09-09T00:00:00" service-id="BOPL" vessel-name="BOPL Name" vessel-id="BOPL" visit-phase="ARRIVED">
    -<lines>
      <line id="JASON" in-voy-nbr="101" out-voy-nbr="101"/>
    </lines>
  </vessel-visit>
</payload>
```

14.9.2.3 Carrier Visit Update message prerequisite

Before you can send a Carrier Visit Update message, you must set up a general notice in N4.

To create a general notice:

1. In N4, go to **Operations > Reports > General Notices**.
2. Create a general notice for either a vessel visit or a train visit:
 - Vessel visit:
Entity: Vessel Visit
Event: PHASE_VV
 - Train visit:
Entity: Train Visit
Event: PHASE_RV
3. From the filter criteria, select:
Visit Phase = Arrived, Working, Complete, Departed
4. From **Action to Perform**, select **Execute Code Extension**.
5. From the **Code Extension**, select `DefaultN4OutboundCasMessageHandler`.

The `DefaultN4OutboundCasMessageHandler` is a system-seeded outbound message handler code extension that invokes the web service configured for the **CAS-Outbound** integration service endpoint.

To create a customized outbound message handler code extension, create a new code extension of type `GENERAL_NOTICES_CODE_EXTENSION` before you create the general notice above. For more information on how to create a code extension, see the **Code Extensions** view topic in the online help.

14.9.3 Container List Request

14.9.3.1 Container List Request

You can use the Container List request to receive the list of on-board units for a vessel visit or a train visit, and the crane work lists.

The Container List request expects the following information in the response:

- **On-board container list:** List of all the units arriving on the vessel.
- **Crane work lists:** List of all the work instructions in all the work queues for the vessel visit segregated by the crane (point of work).
- **Two Booleans:** Indicate whether the updates to the container list or the crane work list should be pushed to CAS.

Required Parameters:

Name	Value
<code>requestType</code>	containerListRequest
<code>visitType</code>	VESSEL RAIL
<code>visitId</code>	[Vessel Visit ID] [Train Visit ID]

Optional Parameters:

Name	Value	Additional Details
onBoard	"Y" or "N"	If "Y", the vessel/rail on-board units are sent as part of the message.
craneId	comma-separated crane IDs or "ALL"	In case of "ALL", the units for all crane work lists are sent. If a value is not specified, a warning is returned.
sendOnBoardUnitUpdates	"Y" or "N"	If "Y", the onBoardUnitUpdate message is sent to CAS if there are changes in the unit data.
sendCraneWorkListUpdates	"Y" or "N"	If "Y", the craneWorkListUpdate message is sent to CAS if there are changes in the work queues.  The sendOnBoardUnitUpdates and sendCraneWorkListUpdates parameters are persisted per vessel visit and as a result should only be set to 'Y' if required; otherwise, the parameters should not be included in the message.

Name	Value	Additional Details
cas-transaction-reference	a unique ID for automated container's operations	 Required for fully-automated sites, but otherwise optional. The <code>cas-transaction-reference</code> parameter was introduced so that QC multi lifts for loads and discharges are performed successfully. For loads and discharges, <code>cas-transaction-reference</code> is saved in EC events, which represent the history of container movements. Maximum length of <code>cas-transaction-reference</code> is 255 characters; however, in practice it should never exceed 20 characters. The <code>cas-transaction-reference</code> parameter should be unique for multi-lift operations so that all related EC events are accurately identified and processed. It is required for the <code>unitPositionUpdate</code> and the <code>unitCapture</code> messages to have an identical <code>cas-transaction-reference</code> for the same container so that the lift, set, and identify operations for a particular unit are accurately tied together.

You can add more parameters to the Container List request by replacing the system-seeded Groovy code extension with a custom code extension.

Container List Request and Train Visits

Unlike the vessel visit, the rail work instructions are not planned for a particular Rail crane (RMG). During the operations, depending on the CHE Range, any crane can be assigned to a rail work instruction. Therefore, the Container List request returns all the work instructions for all the work queues for a train visit and does not segregated by crane. To be consistent with the vessel operations, the Container List request contains a `Crane Team UI` element; however, the value for this is always set as **ALL** for a train visit.

14.9.3.2 Sample ContainerListRequest Message (to N4) for vessel

```
<?xml version="1.0"?> -<groovy class-name="DefaultN4InboundCasRequestHandler" class-
location="code-extension" xsi:schemaLocation="http://www.w3schools.com ../schema/n4-
inbound-ws-message-or-request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.w3schools.com">
  -<parameters>
    <parameter value="containerListRequest" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
```

```

        <parameter value="ACT11954" id="visitId"/>
        <parameter value="Y" id="onBoard"/>
        <parameter value="C1,C2" id="craneId"/> <!-- A comma separated list of
cranes or "All" for all cranes working on the visit)-->
        <parameter value="Y" id="sendOnBoardUnitUpdates"/>
        <parameter value="Y" id="sendCraneWorkListUpdates"/>
    </parameters> </groovy>

```

14.9.3.3 Sample ContainerListRequest Message (to N4) for rail

```

<?xml version="1.0"?>
<groovy class-name="DefaultN4InboundCasRequestHandler" class-location="code-extension"
xsi:schemaLocation="http://www.w3schools.com ../../schema/n4-inbound-ws-message-or-
request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://
www.w3schools.com">
    -<parameters>
        <parameter id="requestType" value="containerListRequest" />
        <parameter id="visitId" value="TVA" />
        <parameter id="onBoard" value="Y" />
        <parameter id="visitType" value="TRAIN" />
    </parameters>
</groovy>

```

14.9.3.4 Sample ContainerListRequest Response (from N4) For Vessel

```

<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="1" status-id="INFO">
    <messages>
        <message message-id="INFO" message-severity="INFO" message-text="[class-
name:DefaultN4InboundCasRequestHandler]" message-detail="[class-
name:DefaultN4InboundCasRequestHandler]" />
        <message message-id="GROOVY_RESULT" message-severity="INFO" message-text="Result" />
    </messages>
    <payload>
        <parameters>
            <parameter id="craneId" value="All" />
            <parameter id="requestType" value="containerListRequest" />
            <parameter id="visitId" value="7961208" />
            <parameter id="onBoard" value="Y" />
            <parameter id="visitType" value="VESSEL" />
        </parameters>
        <onboard-units>
            <unit id="MSKU4799797" cas-unit-reference="" cas-transaction-reference="" gross-
weight="11100.0" yard-measured-weight="" iso-code="9510" length-mm="13716" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
                <current-position loc-type="VESSEL" location="7961208" block="A" row="90"
column="22" tier="90" />
                <flags>
                    <permission id="RTO_CB" />
                    <permission id="RTO_PD" />
                    <permission id="RTO_PP" />

```



```

    </flags>
  </unit>
  <unit id="MSKU4799293" cas-unit-reference="" cas-transaction-reference="" gross-
weight="11200.0" yard-measured-weight="" iso-code="9510" length-mm="13716" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
    <current-position loc-type="VESSEL" location="7961208" block="A" row="90"
column="22" tier="88" />
    <flags>
      <permission id="RTO_CB" />
      <permission id="RTO_PD" />
      <permission id="RTO_PP" />
    </flags>
  </unit>
  <unit id="MSKU4792236" cas-unit-reference="" cas-transaction-reference="" gross-
weight="11680.0" yard-measured-weight="" iso-code="9510" length-mm="13716" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
    <current-position loc-type="VESSEL" location="7961208" block="A" row="90"
column="22" tier="86" />
    <flags>
      <permission id="RTO_CB" />
      <permission id="RTO_PD" />
      <permission id="RTO_PP" />
    </flags>
  </unit>
  <unit id="MSKU4800497" cas-unit-reference="" cas-transaction-reference="" gross-
weight="15800.0" yard-measured-weight="" iso-code="9510" length-mm="13716" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
    <current-position loc-type="VESSEL" location="7961208" block="A" row="90"
column="22" tier="84" />
    <flags>
      <permission id="RTO_CB" />
      <permission id="RTO_PD" />
      <permission id="RTO_PP" />
    </flags>
  </unit>
  </onboard-units>
  <crane-work-lists>
    <crane id="SQC58">
      <work-queue name="796-disch-21A" deck="A" row="22" vessel-lcg="10150"
configuration="CONFIGURED" bollard="A46" bollard-offset-cm="258">
        <units>
          <unit id="MSKU9675968" cas-unit-reference="" cas-transaction-reference="" gross-
weight="6757.0" yard-measured-weight="" iso-code="4510" length-mm="12192" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
            <current-position loc-type="VESSEL" location="7961208" block="A" row="22"
column="05" tier="86" />
            <flags>
              <permission id="PREAN_DELIVER_PRM" />
              <permission id="RTO_PP" />
              <permission id="RTO_CB" />
              <permission id="RTO_PD" />
              <permission id="LINE_LOAD" />
              <permission id="LINE_TRUCK_PICKUP" />
              <permission id="LINE_RAIL_PICKUP" />
              <permission id="LINE_LANDSIDE_PICKUP" />
              <permission id="LINE_BARGE_PICKUP" />
            </flags>
          </unit>
        </units>
      </work-queue>
    </crane>
  </crane-work-lists>

```

```

    <permission id="LINE_DISCHARGE" />
    <permission id="ETT_DRAY_IN_PERMISSION" />
    <permission id="RTO_CB" />
    <permission id="IDO_PERMISSION" />
  </flags>
  <work-instruction sequence="79" move-kind="DSCH" crane-lane="0" door-
direction="U" lift-type="SINGLE" associated-unit-id-1="" associated-unit-id-2=""
associated-unit-id-3="" bay-center-offset-cm="0">
    <planned-position loc-type="YARD" location="APM" block="" row="" column=""
tier="0" />
  </work-instruction>
</unit>
<unit id="MRKU3762894" cas-unit-reference="" cas-transaction-reference="" gross-
weight="7022.0" yard-measured-weight="" iso-code="4510" length-mm="12192" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
  <current-position loc-type="VESSEL" location="7961208" block="A" row="22"
column="12" tier="86" />
  <flags>
    <permission id="PREAN_DELIVER_PRM" />
    <permission id="RTO_PP" />
    <permission id="RTO_CB" />
    <permission id="RTO_PD" />
    <permission id="LINE_LOAD" />
    <permission id="LINE_TRUCK_PICKUP" />
    <permission id="LINE_RAIL_PICKUP" />
    <permission id="LINE_LANDSIDE_PICKUP" />
    <permission id="LINE_BARGE_PICKUP" />
    <permission id="LINE_DISCHARGE" />
    <permission id="ETT_DRAY_IN_PERMISSION" />
    <permission id="RTO_CB" />
    <permission id="IDO_PERMISSION" />
  </flags>
  <work-instruction sequence="71" move-kind="DSCH" crane-lane="0" door-
direction="U" lift-type="SINGLE" associated-unit-id-1="" associated-unit-id-2=""
associated-unit-id-3="" bay-center-offset-cm="0">
    <planned-position loc-type="YARD" location="APM" block="" row="" column=""
tier="0" />
  </work-instruction>
</unit>
<unit id="PONU7432640" cas-unit-reference="" cas-transaction-reference="" gross-
weight="7126.0" yard-measured-weight="" iso-code="4510" length-mm="12192" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
  <current-position loc-type="VESSEL" location="7961208" block="A" row="22"
column="14" tier="86" />
  <flags>
    <permission id="PREAN_DELIVER_PRM" />
    <permission id="RTO_PP" />
    <permission id="RTO_CB" />
    <permission id="RTO_PD" />
    <permission id="LINE_LOAD" />
    <permission id="LINE_TRUCK_PICKUP" />
    <permission id="LINE_RAIL_PICKUP" />
    <permission id="LINE_LANDSIDE_PICKUP" />
    <permission id="LINE_BARGE_PICKUP" />
    <permission id="LINE_DISCHARGE" />
    <permission id="ETT_DRAY_IN_PERMISSION" />

```

```

    <permission id="RTO_CB" />
    <permission id="IDO_PERMISSION" />
  </flags>
  <work-instruction sequence="70" move-kind="DSCH" crane-lane="0" door-
direction="U" lift-type="SINGLE" associated-unit-id-1="" associated-unit-id-2=""
associated-unit-id-3="" bay-center-offset-cm="0">
    <planned-position loc-type="YARD" location="APM" block="" row="" column=""
tier="0" />
  </work-instruction>
</unit>
<unit id="PONU7615980" cas-unit-reference="" cas-transaction-reference="" gross-
weight="6686.0" yard-measured-weight="" iso-code="4510" length-mm="12192" height-
mm="2895" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN">
  <current-position loc-type="VESSEL" location="7961208" block="A" row="22"
column="10" tier="86" />
  <flags>
    <permission id="PREAN_DELIVER_PRM" />
    <permission id="RTO_PP" />
    <permission id="RTO_CB" />
    <permission id="RTO_PD" />
    <permission id="LINE_LOAD" />
    <permission id="LINE_TRUCK_PICKUP" />
    <permission id="LINE_RAIL_PICKUP" />
    <permission id="LINE_LANDSIDE_PICKUP" />
    <permission id="LINE_BARGE_PICKUP" />
    <permission id="LINE_DISCHARGE" />
    <permission id="ETT_DRAY_IN_PERMISSION" />
    <permission id="RTO_CB" />
    <permission id="IDO_PERMISSION" />
  </flags>
  <work-instruction sequence="72" move-kind="DSCH" crane-lane="0" door-
direction="U" lift-type="SINGLE" associated-unit-id-1="" associated-unit-id-2=""
associated-unit-id-3="" bay-center-offset-cm="0">
    <planned-position loc-type="YARD" location="APM" block="" row="" column=""
tier="0" />
  </work-instruction>
</unit>
  <crane-work-lists>
</payload>
</argo-response>

```

14.9.3.5 Sample ContainerListRequest Response (from N4) For Rail

```

<?xml version="1.0" encoding="UTF-8"?>
<payload>
  <parameters>
    <parameter id="requestType" value="containerListRequest" />
    <parameter id="visitId" value="TVA" />
    <parameter id="onBoard" value="Y" />
    <parameter id="visitType" value="TRAIN" />
  </parameters>
<onboard-units>
<unit id="DISC000001" cas-unit-reference="" cas-transaction-reference="" gross-
weight="25000.0" yard-measured-weight="" iso-code="2200" length-mm="6068" height-

```

```
mm="2591" width-mm="2438" is-hazardous="N" tank-rail-type="UNKNOWN" incompatibility-
reason="NONE" rail-cone-status="NO_CONES">
  <current-position loc-type="RAILCAR" location="RCRA1" block="RCRA1" row="1"
column="1" tier="B" />
</unit>
</onboard-units>
</payload>
```

14.9.4 Crane Work List Update Message

14.9.4.1 Crane Work List Update Message

This message originates from N4 and is triggered when:

- Work Instructions are added, removed or re-sequenced from currently working or the next queue
- The current work queue is completed.
- The work queue is assigned to a different crane
- The work queue is activated/deactivated

The request from N4 contains:

- Visit Type
- Visit ID
- Crane ID
- Work Queues (1-many)
 - Units (1-many)
 - Unit (CA Unit Details)
 - Planned Position (CA Unit Position)
 - Work Instruction (CA Unit Work Instruction)

To get accurate crane work list information, plan the vessel using Quay Commander.

This message payload contains additional attributes and the CAS crane work list XML containing the current work queue and the next work queue.

14.9.4.2 Sample Message(from N4)

```
<?xml version="1.0"?>
-<payload xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-outbound-ws-cas-
message-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://
www.w3schools.com">
  -<additional-info>
    <field value="craneWorkListUpdate" id="requestType"/>
    <field value="VESSEL" id="visitType"/>
    <field value="12311" id="visitId"/>
    <field value="C1" id="craneId"/>
  </additional-info> <!--For each update current and next work queue information is
sent-->
  <crane id="C1">
    <!--Current Work Queue-->
```

```

<work-queue vessel-lcg="currentWqVesselLcg" row="currentWqRow" deck="currentWqDeck"
name="currentWqName">
  -<units>
    <unit id="MATU4516367" width-mm="2438" height-mm="2895" length-mm="13716" iso-
code="4200" yard-measured-weight="" gross-weight="4535.93">
      <seals seal-4="" seal-3="" seal-2="" seal-1="" />
      <current-position location="ACT11954" loc-type="VESSEL" block="A" row="05"
column="07" tier="82"/>
      <work-instruction door-direction="" crane-lane="" associated-unit-id-3=""
associated-unit-id-2="" associated-unit-id-1="" lift-type="" move-kind="" sequence="">
        <planned-position location="YRD111" loc-type="YARD" block="C02" row="13"
column="01" tier="B" />
      </work-instruction>
    </unit>
  </units>
</work-queue>
<!--Next Work Queue-->
<work-queue vessel-lcg="nextWqVesselLcg" row="nextWqRow" deck="nextWqDeck"
name="nextWqName">
  <units>
    <unit id="MATU4516368" width-mm="2438" height-mm="2895" length-mm="13716" iso-
code="4200" yard-measured-weight="" gross-weight="4535.93">
      <seals seal-4="" seal-3="" seal-2="" seal-1="" />
      <current-position location="ACT11954" loc-type="VESSEL" block="A" row="07"
column="07" tier="82"/>
      <work-instruction door-direction="" crane-lane="" associated-unit-id-3=""
associated-unit-id-2="" associated-unit-id-1="" lift-type="" move-kind="" sequence="">
        <planned-position location="YRD111" loc-type="YARD" block="C03" row="13"
column="01" tier="B"/>
      </work-instruction>
    </unit>
  </units>
</work-queue>
</crane>
</payload>

```

14.9.4.3 Prerequisites for this message

The prerequisites for this message to be sent are:

1. Create a custom event in N4. (See instructions below.)
2. Call a system-seeded library, 'CraneWorkListUpdate', from the entity lifecycle interceptors of Work Instruction and Work Queue entities. See sample groovy NavisSampleCraneWorkListInterceptor in the N4_CraneAutomationSystem(CAS)_SDK.zip under artifacts\sample\containerworklistupdate \NavisSampleCraneWorkListInterceptor.xml. If an entity lifecycle interceptor does not exist for an entity, it needs to be created.
3. Create a general notice in N4 for the event created in step 1. (See instructions, below)

14.9.4.4 Create Custom Event

To create a custom event:

1. Create an Event Type with ID of CRANE_WORK_LIST_UPDATE.
2. Choose 'Applies to' entity as Vessel Visit, and make it Notifiable.
3. Save the event type.

14.9.4.5 Create a General Notice

To create a general notice in N4:

1. In N4, go to Operations > Reports > General Notices.
2. Create a general notice for Vessel Visit Entity with event 'CRANE_WORK_LIST_UPDATE'.
3. Do not choose any filter criteria.
4. For action to perform, choose 'Execute Code Extension' and choose the code extension "DefaultN4OutboundCasMessageHandler" from the drop-down list. DefaultN4OutboundCasMessageHandler is the system-provided outbound message handler code extension that invokes the web service on the 'CAS-Outbound' named integration service endpoint.

If you need a customized outbound message handler code extension, then create a new code extension of type GENERAL_NOTICES_CODE_EXTENSION before creating the general notice. See the online help for the Code Extensions view in N4 for more details.

14.9.4.6 For Rail Operations

In case of Rail, the functionality is slightly different. As a result of a Work Instruction or Work Queue update, all the Work Queues are sent, and not just the current and the next active queue, as in case of Vessel.

14.9.4.7 Configuration to enable Crane Work List Update (CWLU) message

To enable the Crane Work List Update (CWLU) message, some extra configuration is required:

1. A custom notifiable Event Type 'RAIL_CRANE_WORK_LIST_UPDATE' needs to be created with TrainVisit as serviceable entity.
2. A general notice needs to be created for the event type created above with the action of 'Execute Code Extension' with code extension being 'DefaultN4OutboundCasMessageHandler'

14.9.5 Load Swap Request Message

14.9.5.1 Load Swap Request

This request is used if you are using the ECS Load Swap feature instead of the N4 load swap functionality.

This is a licensed feature and the configuration for the feature is included in the N4 Automation Guide and is also available in the NCC.

The CAS sends a request that may include:

- Request Type - what the request is for. This will be `loadSwapRequest`.
- Request ID - the unique ID of the load swap.
- Visit Type - such as `VESSEL`.
- Visit ID - the unique ID of the visit.
- Crane ID - ID for the crane involved in the swap.
- unitXml - this contains the unit ID for the original container and the container it is being swapped with.

The response from N4 contains:

- Message Code
- Status - whether the load swap was successful.
- Reason - related to the status, such as `Request processed successfully`.

14.9.5.2 Sample Requests

loadSwapRequest for single successful load move swap

```
<groovy class-name="DefaultN4InboundCasMessageHandler" class-location="code-extension">
  <parameters>
    <parameter id='requestType' value='loadSwapRequest' />
    <parameter id='requestId' value='SwapNo1234' />
  </parameters>
  <response>
    <message code='NCAS-5000' status='SUCCESS' reason='Request processed
successfully' />
  </response>
</groovy>
```

loadSwapRequest for unsuccessful single load move swap

```
<groovy class-name="DefaultN4InboundCasMessageHandler" class-location="code-extension">
<parameters>
  <parameter id='requestType' value='loadSwapRequest' />
  <parameter id='requestId' value='SwapNo1234' />
</parameters>
<response>
  <message code='NCAS-xxxx' status='ERROR' reason='Specific reason determined by N4
will be included here' />
</response>
</groovy>
```

loadSwapRequest for swapping two single load moves

The swap request needs to specify either one or two pairs of units and corresponding work instruction gkeys, embedded as an XML string in the unitsXml parameter. As this parameter value is an XML nested inside another XML, it needs to be escaped.

```
<groovy class-name="DefaultN4InboundCasMessageHandler" class-location="code-extension">
  <parameters>
    <parameter id='requestType' value='loadSwapRequest'/>
    <parameter id='requestId' value='SwapNo1234'/>
    <parameter id='visitType' value='VESSEL'/>
  </parameters>
</groovy>
```

```

    <parameter id='visitId' value='VSL'/>
    <parameter id='craneId' value='QC03'/>
    <parameter id='unitXml' value='&lt;units&gt;
      &lt;unit id=&apos;NCCU1286092&apos; wi-gkey=&apos;1&apos;
      swap-with-id=&apos;MOAU0215223&apos; swap-with-wi-gkey=&apos;2&apos; /
&gt;
      &lt;/units&gt;'/>
  </parameters>
</groovy>

```

The human-readable (unescaped XML) value of the unitXml parameter in the above message is:

```

<units>
  <unit id='NCCU1286092' wi-gkey='1' swap-with-id='MOAU0215223' swap-with-wi-
gkey='2' />
</units>

```

loadSwapRequest for swapping four single load moves

```

<groovy class-name="DefaultN4InboundCasMessageHandler" class-location="code-extension">
  <parameters>
    <parameter id='requestType' value='loadSwapRequest'/>
    <parameter id='requestId' value='SwapNo1234'/>
    <parameter id='visitType' value='VESSEL'/>
    <parameter id='visitId' value='VSL'/>
    <parameter id='craneId' value='QC03'/>
    <parameter id='unitXml' value='&lt;units&gt;
      &lt;unit id=&apos;NCCU1286092&apos; wi-gkey=&apos;1&apos;
      swap-with-id=&apos;MOAU0215223&apos; swap-with-wi-gkey=&apos;2&apos; /
&gt;
      &lt;unit id=&apos;MOAU0734358&apos; wi-gkey=&apos;3&apos;
      swap-with-id=&apos;CLHU3906454&apos; swap-with-wi-gkey=&apos;4&apos; /
&gt;
      &lt;/units&gt;'/>
  </parameters>
</groovy>

```

The human-readable (unescaped XML value of the unitXml parameter in the above message is:

```

<units>
  <unit id='NCCU1286092' wi-gkey='1' swap-with-id='MOAU0215223' swap-with-wi-
gkey='2' />
  <unit id='MOAU0734358' wi-gkey='3' swap-with-id='CLHU3906454' swap-with-wi-
gkey='4' />
</units>

```

14.9.6 Onboard Unit Update Message

14.9.6.1 Onboard Unit Update Message

After the initial onboard unit list has been requested, the onboard unit list can change, such as when new BAPLIEs are received. N4 sends updates to the CAS as follows:

- If an onboard unit whose current inbound carrier is this carrier visit is changed to a different carrier visit, an update is sent to the CAS indicating that the container should be removed from the onboard list (action = remove).
- If an onboard unit whose current inbound carrier is this carrier visit has some other attribute updated, an update is

sent to the CAS indicating that the container should be updated (action = update). Note that updates will only be sent when attributes that are of interest to the CAS are updated. These attributes include:

- Equipment type, length, height, weight
- Current position
- If an inbound unit whose current inbound carrier is another carrier visit is changed to this carrier visit then an update is sent to the CAS indicating that the container should be added to the onboard list (action = add).



The request from N4 is implemented by an entity lifecycle interceptor on the Unit. The pre-conditions for triggering the Onboard Unit update are:

1. If the CAS has requested that Onboard Unit Updates in the Container List request.
2. Carrier Visit Phase is "Working".

The request from N4 contains:

- Visit Type
- Visit ID
- Onboard Unit
 - Action (add/update/remove)
 - Unit (CA Unit Details)
 - Current Position (CA Unit Position)

This message originates from N4 and is triggered by changes to the following attributes of the unit.

- Unit Last known position (UnitFacilityVisit entity)
- Unit actual inbound visit (Unit entity)
- Unit equipment type or dimensions (Container entity)

This message payload contains additional attributes and the CAS unit XML.

14.9.6.2 Sample Message(from N4)

```
<?xml version="1.0"?> -<payload xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-outbound-ws-cas-message-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://www.w3schools.com">
  -<additional-info>
    <field value="onBoardUnitUpdate" id="requestType"/>
    <field value="VESSEL" id="visitType"/>
    <field value="12311" id="visitId"/>
    <field value="Add" id="action"/> <!--The value can be 'Add', 'Update' or 'Remove'-->
  </additional-info>
  -<unit id="AACU8511954" width-mm="2438" height-mm="2438" length-mm="6068" iso-code="2000" yard-measured-weight="" gross-weight="1900.0"> <current-position location="ACT11954" loc-type="VESSEL" block="A" row="05" column="07" tier="82"/>
    <seals seal-4="" seal-3="" seal-2="" seal-1=""/>
  </unit>
</payload>
```

14.9.6.3 Prerequisites for this Message

The prerequisites for this message to be sent are:

1. Create a custom event in N4 (see instructions below).
2. Call a system-seeded library, 'OnboardUnitUpdate', from the entity lifecycle interceptors of the 3 entities above. See sample in the N4_CraneAutomationSystem(CAS)_SDK.zip under artifacts\sample\onboardunitupdate\NavisSampleUnitInterceptor.xml, or attached to this Confluence page. If an entity lifecycle interceptor does not exist for an entity, it needs to be created.
3. In N4, go to Operations > Reports > General Notices.
4. Create a general notice in N4 for the event created in step 1. (See instructions for creating a general notice, below.)

14.9.6.4 Create Custom Event

To create a custom event:

1. Create an Event Type with ID of UNIT_ON_BOARD_UPDATE.
2. Choose 'Applies to' entity as Unit, and make it Notifiable and 'Can be Bulk-applied'.
3. Save the event type.

14.9.6.5 Create a General Notice

To create a general notice in N4:

1. In N4, go to Operations > Reports > General Notices.
2. Create a general notice for Unit entity with event 'UNIT_ON_BOARD_UPDATE'.
3. Do not choose any filter criteria.
4. For action to perform, choose 'Execute Code Extension' and add choose the code extension "DefaultN4OutboundCasMessageHandler" from the drop-down list. DefaultN4OutboundCasMessageHandler is the system-provided outbound message handler code extension that invokes the web service on the 'CAS-Outbound' named integration service endpoint.

If you need a customized outbound message handler code extension, then a new code extension of type GENERAL_NOTICES_CODE_EXTENSION before creating the general notice. See the online help for the Code Extensions view in N4 for more details.

14.9.7 QC Updates via CAS

14.9.7.1 Update QC technical status and location via CAS

QC technical status can be received via CAS. Valid technical status values are:

- GREEN
- YELLOW
- ORANGE
- RED

Sample Message (to N4)

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasMessageHandler'>
  <parameters>
    <parameter id='requestType' value="craneTecnicalStatusUpdate"/>
    <parameter id='craneId' value='QC102'/>
    <parameter id='technicalStatus' value='GREEN'/>
  </parameters>
</groovy>
```

Sample Response (from N4)

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="1" status-id="INFO">
  <messages>
    <message message-id="INFO" message-severity="INFO" message-text="[class-
name:DefaultN4InboundCasMessageHandler]" message-detail="[class-
name:DefaultN4InboundCasMessageHandler]" />
    <message message-id="GROOVY_RESULT" message-severity="INFO" message-text="Result:
&lt;payload&gt;&#xA; &lt;parameters&gt;&#xA; &lt;parameter id='technicalStatus'
value='GREEN' /&gt;&#xA; &lt;parameter id='craneId' value='QC102' /&gt;&#xA;
&lt;parameter id='requestType' value='craneTechnicalStatusUpdate' /&gt;&#xA; &lt;/
parameters&gt;&#xA;&lt;/payload&gt;" message-detail="Result: &lt;payload&gt;&#xA;
&lt;parameters&gt;&#xA; &lt;parameter id='technicalStatus' value='GREEN' /&gt;&#xA;
&lt;parameter id='craneId' value='QC102' /&gt;&#xA; &lt;parameter id='requestType'
value='craneTechnicalStatusUpdate' /&gt;&#xA; &lt;/parameters&gt;&#xA;&lt;/
payload&gt;" />
  </messages>
  <payload>
    <parameters>
      <parameter id="technicalStatus" value="GREEN" />
      <parameter id="craneId" value="QC102" />
      <parameter id="requestType" value="craneTechnicalStatusUpdate" />
    </parameters>
  </payload>
</argo-response>
```

14.9.7.2 Update QC position via CAS

QC position (bollard + offset) can be received via CAS.

Valid bollard value is the name of an existing bollard.

Valid bollard offset is any positive or negative integer. Bollard offset is in centimeters.

Sample Message (to N4)

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasMessageHandler'>
  <parameters>
    <parameter id='requestType' value="cranePositionUpdate"/>
    <parameter id='craneId' value='QC102'/>
    <parameter id='bollardName' value='B16'/>
    <parameter id='bollardOffsetCm' value='654'/>
  </parameters>
```

</groovy>

Sample Response (from N4)

```
<?xml version="1.0" encoding="UTF-8"?>
<argo-response status="1" status-id="INFO">
  <messages>
    <message message-id="INFO" message-severity="INFO" message-text="[class-
name:DefaultN4InboundCasMessageHandler]" message-detail="[class-
name:DefaultN4InboundCasMessageHandler]" />
    <message message-id="GROOVY_RESULT" message-severity="INFO" message-text="Result:
    &lt;payload&gt;&#xA; &lt;parameters&gt;&#xA; &lt;parameters&gt;&#xA; &lt;parameter
id='bollardOffsetCm' value='654' /&gt;&#xA; &lt;parameter id='bollardName'
value='B16' /&gt;&#xA; &lt;parameter id='craneId' value='QC102' /&gt;&#xA;
&lt;parameter id='requestType' value='craneTechnicalStatusUpdate' /&gt;&#xA; &lt;/
parameters&gt;&#xA;&lt;/payload&gt;" message-detail="Result: &lt;payload&gt;&#xA;
&lt;parameters&gt;&#xA; &lt;parameter id='bollardOffsetCm' value='654' /&gt;&#xA;
&lt;parameter id='bollardName' value='B16' /&gt;&#xA; &lt;parameter id='craneId'
value='QC102' /&gt;&#xA; &lt;parameter id='requestType' value='cranePositionUpdate' /
&gt;&#xA; &lt;/parameters&gt;&#xA;&lt;/payload&gt;" />
  </messages>
  <payload>
    <parameters>
      <parameter id='requestType' value="cranePositionUpdate"/>
      <parameter id='craneId' value='QC102'/>
      <parameter id='bollardName' value='B16'/>
      <parameter id='bollardOffsetCm' value='654'/>
    </parameters>
  </payload>
</argo-response>
```

14.9.8 Rail Messages in CAS API

14.9.8.1 IBCMA Rail Cone Status Update in CAS Interface

IBCMA will send rail cone status updates for units. N4 will internally update units using received rail cone status. This functionality has been deprecated and is scheduled to be removed in future releases.

XML tag for rail cone status is "rail-cone-status".

Valid rail cone status values are:

- UNKNOWN
- NO_CONES
- UNLOCKED_CONES
- LOCKED_CONES

Sample Message (from IBCMA)

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasMessageHandler'>
  <parameters>
```

```

    <parameter id='requestType' value="railConeStatusUpdate"/>
    <parameter id='visitType' value="RAIL"/>
    <parameter id='unitXml' value='&lt;units&gt;&lt;unit cas-unit-
reference=&apos;&apos; id=&apos;CTRU632101&apos; rail-cone-status=&apos;NO_CONES&apos;
&gt; &lt;/unit&gt; &lt;/units&gt;'/>
  </parameters>
</groovy>

```

14.9.8.2 IBCMA Rail Cone Status Update in CAS By Platform

IBCMA will send rail cone status updates for all units on a particular platform. N4 will internally update units using received rail cone status.

XML tag for rail cone status is "cone-status".

Valid rail cone status values are:

- UNKNOWN
- NO_CONES or No Cones
- UNLOCKED_CONES or Unlocked
- LOCKED_CONES or Locked

Sample Message (from IBCMA)

```

<groovy class-location='code-extension' class-name='DefaultN4InboundCasMessageHandler'>
  <parameters>
    <parameter id='requestType' value='platformRailConeStatusUpdate' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='railcars' value='&lt;railcars&gt;
&lt;railcar id=&apos;RCAR0001&apos; &gt;
&lt;platform label=&apos;A&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;B&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;C&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;D&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;E&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;/railcar&gt;
&lt;railcar id=&apos;RCAR0002&apos; &gt;
&lt;platform label=&apos;A&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;B&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;C&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;D&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;platform label=&apos;E&apos; cone-status=&apos;UNKNOWN&apos; /&gt;
&lt;/railcar&gt;
&lt;/railcars&gt;' />
  </parameters>
</groovy>

```

14.9.8.3 Rail Spotting interface in CAS API

To use a third-party application to spot railcars, you should send the spotting information to N4 via the CAS API.

A xml response is given which include Railcar Geometries (1-n of CA Railcar Geometry) for all railcars on the track specified in the request.

Complete Sample Request Message

The following code segment represents a request Sample which is complete and has all data:

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='railcarSpottingRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='railcarId' value='RCRA1' />
    <parameter id='trackName' value='TRA' />
    <parameter id='sequenceOnTrack' value='1' />
    <parameter id='platformSequence' value='1' />
    <parameter id='platformLabel' value='A' />
    <parameter id='transferPoint' value='T004001' />
    <parameter id='latitude' value='1.321321' />
    <parameter id='longitude' value='37.6654654' />
    <parameter id='meterMark' value='100' />
    <parameter id='yardOriginId' value='AAF' />
  </parameters>
</groovy>
```

Request Sample for spotting a railcar by GPS

The following code sample represents a request Sample for spotting a railcar by GPS. Notice that transferPoint and MeterMark are not required.

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='railcarSpottingRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='railcarId' value='RCRA1' />
    <parameter id='trackName' value='TRA' />
    <parameter id='sequenceOnTrack' value='1' />
    <parameter id='platformSequence' value='1' />
    <parameter id='platformLabel' value='A' />
    <parameter id='latitude' value='-122.26716' />
    <parameter id='longitude' value='37.796949999999995' />
  </parameters>
</groovy>
```

Request Sample for spotting a railcar by TP

The following code segment represents a request Sample for spotting a railcar by TP. Notice that Lon/Lat and MeterMark are not required.

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='railcarSpottingRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='railcarId' value='RCRA1' />
    <parameter id='trackName' value='TRA' />
    <parameter id='sequenceOnTrack' value='1' />
    <parameter id='platformSequence' value='1' />
    <parameter id='platformLabel' value='A' />
    <parameter id='transferPoint' value='T004001' />
  </parameters>
</groovy>
```

```
</parameters>
</groovy>
```

Request Sample for spotting a railcar by Meter-mark

The following code segment represents a request Sample for spotting a railcar by Meter-mark. Notice that Lon/Lat and transferPoint are not required.

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasRequestHandler'>
  <parameters>
    <parameter id='requestType' value='railcarSpottingRequest' />
    <parameter id='visitType' value='RAIL' />
    <parameter id='railcarId' value='RCRA1' />
    <parameter id='trackName' value='TRA' />
    <parameter id='sequenceOnTrack' value='1' />
    <parameter id='platformSequence' value='1' />
    <parameter id='platformLabel' value='A' />
    <parameter id='meterMark' value='100' />
  </parameters>
</groovy>
```

Response Sample

```
<?xml version="1.0" encoding="UTF-8"?>
<payload>
  <parameters>
    <parameter id="platformSequence" value="1" />
    <parameter id="sequenceOnTrack" value="1" />
    <parameter id="transferPoint" value="T004001" />
    <parameter id="railcarId" value="RCRA1" />
    <parameter id="requestType" value="railcarSpottingRequest" />
    <parameter id="trackName" value="TRA" />
    <parameter id="meterMark" value="null" />
    <parameter id="longitude" value="null" />
    <parameter id="latitude" value="null" />
    <parameter id="visitType" value="RAIL" />
  </parameters>
  <railCarBins>
    <railCar id="RCRA1" track="TRA" sequence="1" spotting-status="TP" flipped="N"
segment-break-at-low-end="N" inspection-status="NOT_INSPECTED" is-damaged="N" transfer-
point="" trackXCoord="5305" inbound-train-visit-id="TVA" outbound-train-visit-id="TVA">
      <platform label="1">
        <centroid cartesian-X="3910.6400000000001" cartesian-Y="0.0"
latitude="37.79361532049753" longitude="-122.27350372815404" />
        <railSlot>
          <slot-position loc-type="RAILCAR" block="RCRA1" row="1" column="1" />
          <geoPos cartesian-X="3910.6400000000001" cartesian-Y="0.0"
latitude="37.79361532049753" longitude="-122.27350372815404" />
        </railSlot>
        <railSlot>
          <slot-position loc-type="RAILCAR" block="RCRA1" row="1" column="2" />
          <geoPos cartesian-X="3910.6400000000001" cartesian-Y="0.0"
latitude="37.79361532049753" longitude="-122.27350372815404" />
        </railSlot>
      </platform>
    </railCar>
  </railCarBins>
```

```

    </platform>
  </railCar>
  <railCar id="RCRA1" track="TRA" sequence="1" spotting-status="TP" flipped="N"
segment-break-at-low-end="N" inspection-status="NOT_INSPECTED" is-damaged="N" transfer-
point="" trackXCoord="5305" inbound-train-visit-id="TVA" outbound-train-visit-id="TVA">
    <platform label="1">
      <centroid cartesian-X="3910.640000000001" cartesian-Y="0.0"
latitude="37.79361532049753" longitude="-122.27350372815404" />
      <railSlot>
        <slot-position loc-type="RAILCAR" block="RCRA1" row="1" column="1" />
        <geoPos cartesian-X="3910.640000000001" cartesian-Y="0.0"
latitude="37.79361532049753" longitude="-122.27350372815404" />
      </railSlot>
      <railSlot>
        <slot-position loc-type="RAILCAR" block="RCRA1" row="1" column="2" />
        <geoPos cartesian-X="3910.640000000001" cartesian-Y="0.0"
latitude="37.79361532049753" longitude="-122.27350372815404" />
      </railSlot>
    </platform>
  </railCar>
</railCarBins>
</payload>

```

14.9.9 Ready To Transfer Message

14.9.9.1 Sample Message (from N4)

```

<?xml version="1.0"?>
<payload xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-outbound-ws-cas-
message-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://
www.w3schools.com">
  <additional-info>
    <field value="readyToTransferMessage" id="requestType"/>
    <field value="VESSEL" id="visitType"/>
    <field value="12311" id="visitId"/>
    <field value="C01" id="craneId"/>
    <field value="Y" id="readyToTransfer"/>
    <!--The value can be 'Y' or 'N' -->
  </additional-info>
  <units>
    <unit id="AACU8511954" cas-transaction-reference="CTR12345" cas-unit-
reference="CUR12345">
      <planned-position location="ACT11954" loc-type="VESSEL" block="A" row="05"
column="07" tier="82"/>
    </unit>
  </units>
</payload>

```

In response to the Unit Capture Image message, the same unit XML from CAS is echoed back.

In response to the Unit Capture Identify message, fully constructed CAS unit XML is sent as part of this message.

14.9.9.2 Sample Message Sent In Response Unit Capture Identify

```
<payload>
<additional-info>
  <field id='craneId' value='QC03' />
  <field id='visitId' value='KAT002' />
  <field id='requestType' value='readyToTransferMessage' />
  <field id='readyToTransfer' value='Y' />
  <field id='visitType' value='VESSEL' />
</additional-info>
<units>
  <unit id='CTR1208003' cas-unit-reference='QC03_CTR1208003_REF' cas-transaction-
reference='QC03_CTR1208002_TRAN_REF' gross-weight='14000.0' iso-code='2210' length-
mm='6058' height-mm='2591' width-mm='2438' is-hazardous='false' incompatibility-
reason='NONE'>
    <current-position loc-type='VESSEL' location='KAT002' block='A' row='55'
column='07' tier='82' />
    <work-instruction sequence='2' move-kind='DSCH' crane-lane='0' door-direction='W'
lift-type='TWIN' associated-unit-id-1='CTR1208002' associated-unit-id-2='' associated-
unit-id-3='' bay-center-offset-cm='0'>
      <planned-position loc-type='YARD' location='VICT' block='B01' row='06' column='F'
tier='1' />
    </work-instruction>
  </unit>
  <unit id='CTR1208002' cas-unit-reference='QC03_CTR1208002_REF' cas-transaction-
reference='QC03_CTR1208002_TRAN_REF' gross-weight='14000.0' iso-code='2210' length-
mm='6058' height-mm='2591' width-mm='2438' is-hazardous='false' incompatibility-
reason='NONE'>
    <current-position loc-type='VESSEL' location='KAT002' block='A' row='53'
column='07' tier='82' />
    <work-instruction sequence='1' move-kind='DSCH' crane-lane='0' door-direction='W'
lift-type='TWIN' associated-unit-id-1='CTR1208003' associated-unit-id-2='' associated-
unit-id-3='' bay-center-offset-cm='0'>
      <planned-position loc-type='YARD' location='VICT' block='B01' row='06' column='C'
tier='2' />
    </work-instruction>
  </unit>
</units>
<ready-to-transfer>
  <field id='CTR1208002' ready-to-transfer='Y' />
  <field id='CTR1208003' ready-to-transfer='Y' />
</ready-to-transfer>
</payload>
```

The message is sent to the outbound web service defined by the 'CAS-Outbound' named integration service endpoint.

14.9.10 Unit Capture Messages

14.9.10.1 Rail Inspection - Rail Load

A series of unit capture messages is used to record container information captured with an OCR. The same basic message is used for communicating that an container image has been captured or identified by the CAS, as well as to

request that the container be created or updated in N4, depending on whether the action specified is "Image", "Identify", "Create" or "Update". These messages are used as follows:

- **Unit Capture Image Message** - The first unit capture message sent to N4, this message passes the image of a container captured using an OCR from the CAS to N4. When this message is sent, the image hasn't been processed, so the actual container ID is not yet known.
- **Unit Capture Identify Message** - After the Unit Capture Image Message is sent, the CAS processes the image to identify the container ID in the image. Once the container number is identified, it's sent to N4 using the Unit Capture Identify Message. If the container exists in N4, then N4 executes the business logic to load or discharge. If the container does not exist in N4, and error is sent.
- **Unit Capture Create Message** - If a container identified in a Unit Capture Identify Message does not exist in N4, the CAS should send a Unit Capture Create Message, which creates the container in N4.
- **Unit Capture Update Message** - This message can be sent from the CAS to N4 after the container has been identified or created (that is, the container ID exists in N4).
- **Unit Capture CannotIdentify Message** - (Versions 3.02.4 and above) This message can be sent from the CAS to N4 when the CAS is unable to identify the container due to some reason. No workflow processing happens as part of this message but a new system provided service event (CHE_ERROR_CONTAINER_COULD_NOT_BE_IDENTIFIED) is generated. A general notice can be created for this event to create an EcAlarm to notify the users.

In case of Unit Capture messages of type 'Identify', 'Create' and 'Update', additional attributes/elements as defined in the schema definition (casTypes.xsd) can be sent as part of the message to update the Unit information in N4. Most of these attributes are self-explanatory but three attributes which require special mention are "is-placarded", "is-sealed" and "is-bundle". These attributes are intended to be used when the OCR system cannot decipher the observed placards, seals and bundled equipment for the unit. If the OCR does have the actual placard, seal and bundle information, it needs to be sent instead of these attributes. The valid values are 'true'/'false' (or 'Y'/'N'). The absence of attribute in the message defaults to false.

As of version 3.02.4, the functionality for identification of bundles has been enhanced such that any equipment ID which is part of the bundle can be sent as unit ID in the identification message. Previously, only the primary equipment ID (the master in the bundle) was accepted as equipment ID during identification. For example, there are 4 containers in a bundle (A,B,C and D) of which A is the master and the rest are attached to it as payloads. In that case, if the OCR sends the identification message with container A, it would be processed normally. However, if any of the containers B,C, or D are sent as the container ID in the message, then the following happens:

1. While validating the unit, CAS API will see that there is no active unit for that container ID (B, C, and D would be retired units in N4).
2. It checks to see if it is part of the bundle, if no, an error is registered (same as earlier), but if it is a bundle it would go through steps 3 to 5.
3. If it is part of the bundle, API tries to find the attached unit and get the active UFV (unit facility visit) corresponding to attached unit (which will be UFV for container A; the master in the bundle).
4. If it finds one, then the identification message will be sent to the workflow for the active UFV if it is on discharge list.
5. If the active UFV (the master) is not in discharge list, the normal flow takes place, that is, a warning is registered and discharge is still allowed to happen.

The request from CAS contains:

- Visit Type
- Visit ID

A special case is double banking, when one QC handles two barges which belong to two different vessel visits. In that case OCR system does not know from which barge the containers are coming from and it will provide two comma-separated visit IDs.

- Crane ID
- Action (Image/Identify/Create/Update/CannotIdentify)
- Units (1- 4 of CAS Unit Details)

The response from N4 contains:

- Visit Type
- Visit ID
In the mentioned above case of double banking (when two vessel visit IDs are provided) N4 will try to find the actual visit using unit ID if the first unit in Units.
If N4 could not find the actual visit, it will return the first actually existing visit using for the two provided visit IDs.
- Crane ID
- Action
- Units (1- 4 of CAS Unit Details)
- Messages (Success/Warnings/Errors)

Messages:

- Error: No Unit Reference (all actions)
- Error: Identified Unit not specified (Identify, Create and Update actions)
- Error: Unit Reference not matched for Updated Unit (Update action only)
- Error: When unit contains hold for UNIT_LOAD event on performing Load operation (all actions)
- Warning: Identified unit not planned for load (Identify action only, in cases where unit identified after lift from ground, HT or platform. See discharge use cases for unit not planned for discharge)
- Warning: Identified unit not planned for discharge (Identify action only, identified after set to ground or HT, See load use cases for unit not planned for load)
- Warning: Identified Unit not found in N4 (Identify action only)
- Warning: Identified unit not planned for discharge (identified after lift from vessel, train or platform)
- Success: Identified Unit found in N4 (Identify action only)
- Success: Unit Created (Create action only)
- Success: Updated Unit with matching Unit Reference (Update action only)

Required Parameters:

- Parameter name='requestType' value='unitCaptureMessage'
- Parameter name='visitType' value = 'VESSEL'
- Parameter name='visitId' value= the visit ID of the vessel
- Parameter name='craneId' value= the crane id.
- Parameter name='action' value= one of the actions above.
- Parameter name='unitXml' value= the unit XML (see sample messages)

Optional Parameters:

- Parameter name='sendReadyToTransferMessage' value = 'N' (Applicable for version 2.6 and above. It is applicable for 'Image' and 'Identify' messages only. If the value is 'N' 'readyToTransfer' message will not be sent to CAS as part of the 'unitCapture' processing. If it is not defined or the value is not 'N', the 'readytoTransfer' message will be sent).

- Parameter name = 'cas-transaction-reference' value= a unique ID for unknown containers in automated operations (Required for fully-automated sites, but otherwise optional)

Customers can add more parameters by replacing the system-provided Groovy with custom Groovy.

14.9.10.2 Image Messages

Unit XML example used in samples below

The max number of <unit> elements in the <units> element can be 4 (for quad lift)

```
<units>
  <unit cas-unit-reference='CUR12345' cas-transaction-reference='CTR12345'
id='UNIT1234567' />
  <unit cas-unit-reference='CUR12346' cas-transaction-reference='CTR12345'
id='UNIT1234568' />
</units>
```

Sample Unit Capture Image Message (to N4)

```
<?xml version="1.0"?> -<groovy class-name="DefaultN4InboundCasMessageHandler" class-
location="code-extension" xsi:schemaLocation="http://www.w3schools.com ../../schema/n4-
inbound-ws-message-or-request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.w3schools.com">
  -<parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Cranel" id="craneId"/>
    <parameter value="Image" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters>
</groovy>
```

Sample Unit Capture Image Response Message (from N4)

```
<?xml version="1.0"?> -<payload xsi:schemaLocation="http://www.w3schools.com ../../
schema/N4-outbound-ws-cas-response-type.xsd" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns="http://www.w3schools.com">
  -<parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Cranel" id="craneId"/>
    <parameter value="Image" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters>
  -<additional-info>
    <field value="aFieldValue" id="aField"/>
  </additional-info>
  -<units-response>
    -<unit-response status="SUCCESS" cas-transaction-reference="CTR12345" cas-unit-
reference="CUR12345"> <message text="Success: Image action processed"/>
  </unit-response>
  </units-response>
</payload>
```

```

</unit-response>
-<unit-response status="SEVERE" cas-transaction-reference="CTR12345" cas-unit-
reference=""> <message text="Error: Required unit attribute cas-unit-reference is
missing"/>
</unit-response>
</units-response>
</payload>

```

14.9.10.3 Identify Messages

Sample Unit Capture Identify Message (to N4)

```

<?xml version="1.0"?> -<groovy class-name="DefaultN4InboundCasMessageHandler" class-
location="code-extension" xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-
inbound-ws-message-or-request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.w3schools.com">
-<parameters>
  <parameter value="unitCaptureMessage" id="requestType"/>
  <parameter value="VESSEL" id="visitType"/>
  <parameter value="12311" id="visitId"/>
  <parameter value="Crane1" id="craneId"/>
  <parameter value="Identify" id="action"/>
  <parameter value="see the Unit Xml example above" id="unitXml"/>
</parameters>
</groovy>

```

Sample Unit Capture Identify Response Message (from N4)

```

<?xml version="1.0"?> -<payload xsi:schemaLocation="http://www.w3schools.com ../../
schema/N4-outbound-ws-cas-response-type.xsd" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns="http://www.w3schools.com">
-<parameters>
  <parameter value="unitCaptureMessage" id="requestType"/>
  <parameter value="VESSEL" id="visitType"/>
  <parameter value="12311" id="visitId"/>
  <parameter value="Crane1" id="craneId"/>
  <parameter value="Identify" id="action"/>
  <parameter value="see the Unit Xml example above" id="unitXml"/>
</parameters>
-<additional-info>
  <field value="aFieldValue" id="aField"/>
</additional-info>
-<units-response>
  -<unit-response id="UNIT1234567" status="SUCCESS" cas-transaction-
reference="CTR12345" cas-unit-reference="CUR12345"> <message text="Success: Identified
Unit found in N4"/>
  </unit-response>
  -<unit-response id="UNIT1234568" status="WARNING" cas-transaction-
reference="CTR12345" cas-unit-reference="CUR12346"> <message text="Warning: Identified
unit not planned for discharge"/>
  </unit-response>
</units-response>
</payload>

```

14.9.10.4 Create Messages

Sample Unit Capture Create Message (to N4)

```
<?xml version="1.0"?> -<groovy class-name="DefaultN4InboundCasMessageHandler" class-
location="code-extension" xsi:schemaLocation="http://www.w3schools.com ../../schema/n4-
inbound-ws-message-or-request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.w3schools.com">
<!-- Unit Xml Sample human readable(The special characters have to be escaped when
sent as part of the message): <units> <unit cas-unit-reference='CUR12345' cas-
transaction-reference='CTR12345' id='UNIT1234567' iso-code='2000' length-mm='6068'
height-mm='2438' width-mm='2438'/> <unit cas-unit-reference='CUR12346' cas-transaction-
reference='CTR12345' iso-code='2000' length-mm='6068' height-mm='2438' width-
mm='2438'/> </units> The max number of <unit> elements in the <units> element can be 4
(quad lift) -->
  -<parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Crane1" id="craneId"/>
    <parameter value="Create" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters> </groovy>
```

Sample Unit Capture Create Response (from N4)

```
<?xml version="1.0"?> -<payload xsi:schemaLocation="http://www.w3schools.com ../../
schema/N4-outbound-ws-cas-response-type.xsd" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns="http://www.w3schools.com">
  -<parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Crane1" id="craneId"/>
    <parameter value="Create" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters>
  -<additional-info>
    <field value="aFieldValue" id="aField"/>
  </additional-info>
  -<units-response>
    -<unit-response id="UNIT1234567" status="SUCCESS" cas-transaction-
reference="CTR12345" cas-unit-reference="CUR12345"> <message text="Success: Unit
Created"/>
    </unit-response>
    -<unit-response status="SEVERE" cas-transaction-reference="CTR12345" cas-unit-
reference="CUR12346"> <message text="Error: Required unit attribute id is missing"/>
    </unit-response>
  </units-response> </payload>
```

14.9.10.5 Update Messages

Unit Capture Update Message (to N4)

```
<?xml version="1.0"?> -<groovy class-name="DefaultN4InboundCasMessageHandler" class-
location="code-extension" xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-
inbound-ws-message-or-request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.w3schools.com">
  -<parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Crane1" id="craneId"/>
    <parameter value="Update" id="action"/>
    <parameter value="see the Unit Xml example commented out above" id="unitXml"/>
  </parameters>
</groovy>
```

Unit Capture Update Message Response (from N4)

```
<?xml version="1.0"?> -<payload xsi:schemaLocation="http://www.w3schools.com ../../
schema/N4-outbound-ws-cas-response-type.xsd" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns="http://www.w3schools.com">
  -<parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Crane123" id="craneId"/>
    <parameter value="Update" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters>
  -<additional-info>
    <field value="aFieldValue" id="aField"/>
  </additional-info>
  -<units-response>
    -<unit-response id="UNIT1234567" status="SUCCESS" cas-transaction-
reference="CTR12345" cas-unit-reference="CUR12345"> <message text="Success: Updated
Unit with matching Unit Reference and Transaction Reference"/>
    </unit-response>
    -<unit-response id="UNIT1234568" status="SEVERE" cas-transaction-
reference="CTR12345" cas-unit-reference="CUR12346"> <message text="Error: Unit
Reference not matched for Updated Unit "/>
    </unit-response>
  </units-response>
</payload>
```

14.9.10.6 Cannot Identify Messages

Unit Capture CannotIdentify Message (to N4)

```
<?xml version="1.0"?> -<groovy class-name="DefaultN4InboundCasMessageHandler" class-
```

```
location="code-extension" xsi:schemaLocation="http://www.w3schools.com ../../schema/N4-
inbound-ws-message-or-request-type.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://www.w3schools.com">
  <parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Cranel" id="craneId"/>
    <parameter value="CannotIdentify" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters>
</groovy>
```

Unit Capture CannotIdentify Response Message (from N4)

```
<?xml version="1.0"?> <payload xsi:schemaLocation="http://www.w3schools.com ../../
schema/N4-outbound-ws-cas-response-type.xsd" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns="http://www.w3schools.com">
  <parameters>
    <parameter value="unitCaptureMessage" id="requestType"/>
    <parameter value="VESSEL" id="visitType"/>
    <parameter value="12311" id="visitId"/>
    <parameter value="Cranel" id="craneId"/>
    <parameter value="CannotIdentify" id="action"/>
    <parameter value="see the Unit Xml example above" id="unitXml"/>
  </parameters>
  <additional-info>
    <field value="aFieldValue" id="aField"/>
  </additional-info>
  <units-response>
    <unit-response status="SUCCESS" cas-transaction-reference="CTR12345" cas-unit-
reference="CUR12345"> <message code="NCAS-2002" text="Unit capture handled
successfully[Action=CannotIdentify]" />
    </unit-response>
    <unit-response status="SEVERE" cas-transaction-reference="CTR12345" cas-unit-
reference=""> <message code="NCAS-2002" text="Unit capture handled
successfully[Action=CannotIdentify]" />
    </unit-response>
  </units-response>
</payload>
```

14.9.11 Unit Position Update Message

14.9.11.1 Unit Position Update Message

This message is available for release 3.1 or later.

The CAS uses this message to update N4 when the container has been lifted from its current position or set in a new position.

- The CAS sends a Unit Position Update message with the eci-updated-position when the container is lifted to the spreader from the vessel (for a discharge or shift-on-board) or ground/horizontal transport (for a load).
- If the unit has been identified, N4 updates the work instruction's move stage as described in the following table to

indicate the container has been lifted. This allows the horizontal transport (AGV or UTR), if any, to be dispatched another job.

For a...	the WI state updates to...
load	Put Underway
discharge on an AGV	Carry Underway
discharge at the Apron or Backreach	Carry Ready
discharge that has not yet landed	Fetch Underway

- If the unit has not been identified N4 captures the move event. When the container is identified later, N4 creates copies of the move events with the identified unit ID. When N4 reads the identification event, the unit's work instruction state updates.
If the container was lifted from horizontal transport, the horizontal transport controller (operator or ECS) indicates that the container has been lifted. This releases the horizontal transport for dispatch to another job.
- The CAS will send another Unit Position Update message with the set position when the container is landed in the planned position (on the vessel, on the train, on the ground, or on horizontal transport) to complete the move.
- Set position updates to the platform also update the unit position, if the move uses a platform, but the move stage remains Fetch Underway (for a discharge) or Put Underway (for a load).

The request from CAS contains:

- Visit Type
- Visit ID
- Crane ID
- Action (Lift/Set)
- Current Positions (0-4 of CA Unit Position)



The Action parameter specifies whether the included positions are lift or set positions.

The response from N4 contains:

- Visit Type
- Visit ID
- Action
- Current Positions (0-4 of CA Unit Position)
- Messages (Success/Warnings/Errors)

Messages:

- Error: No Unit Reference
- Error: Unit Reference not matched for identified unit
- Warning: Identified unit not planned for load (set to vessel, train or platform)
- Warning: Identified unit not planned for discharge (on set to ground, HT or platform)
- Warning: Set recorded for un-identified unit
- Warning: Lift and set positions for identified unit are the same (work instruction state rolled back)
- Success: Position updated for identified unit

- Success: Lift recorded for un-identified unit

Required Parameters:

Parameter name='requestType' value='unitPositionUpdateMessage'

Parameter name='visitType' value = 'VESSEL'

Parameter name='visitId' value= the visit ID of the vessel

Parameter name='craneId' value= the crane id.

Parameter name='action' value= one of the actions above.

Parameter name='unitXml' value= the unit XML (see sample messages)

Optional Parameters:

None, but customers can add more by replacing the system-provided Groovy with custom Groovy.

14.9.11.2 Sample Unit Position Update Message Request (from CAS to N4)

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasMessageHandler'>
  <parameters>
    <parameter id='requestType' value="unitPositionUpdateMessage"/>
    <parameter id='visitType' value='VESSEL'/>
    <parameter id='visitId' value='CLAD01'/>
    <parameter id='craneId' value='QC102'/>
    <parameter id='action' value='Lift'/>
    <parameter id='unitXml' value='&lt;units&gt;&lt;unit cas-unit-
reference=&apos;QC102_TEST0000002_REF&apos; cas-transaction-reference=&apos;200&apos;
&gt;
&lt;eci-updated-position loc-type=&apos;S&apos; location=&apos;YRD1111&apos;
block=&apos;QC102&apos; row=&apos;C&apos; column=&apos;C&apos; tier=&apos;&apos; /&gt;
&lt;/unit&gt;
&lt;/units&gt;'/>
  </parameters>
</groovy>
```

-----OR-----

```
<groovy class-location='code-extension' class-name='DefaultN4InboundCasMessageHandler'>
  <parameters>
    <parameter id='requestType' value="unitPositionUpdateMessage"/>
    <parameter id='visitType' value='VESSEL'/>
    <parameter id='visitId' value='CLAD01'/>
    <parameter id='craneId' value='QC102'/>
    <parameter id='action' value='Set'/>
    <parameter id='unitXml' value='&lt;units&gt;&lt;unit cas-unit-
reference=&apos;QC102_TEST0000002_REF&apos; cas-transaction-reference=&apos;200&apos;
&gt;
&lt;/unit&gt;
&lt;/units&gt;'/>
  </parameters>
</groovy>
```

```
<eci-updated-position loc-type='P' location='YRD1111'
block='QC102' row='WS' column='C' tier='' />
</unit>
</units>' />
</parameters>
</groovy>
```

14.9.11.3 Sample Unit Position Update Message Response (From N4 to CAS)

```
<payload>
  <parameters>
    <parameter id='unitXml' value='<units><unit cas-unit-
reference='QC102_TEST0000002_REF' cas-transaction-reference='200'
  <eci-updated-position loc-type='P' location='YRD1111'
block='QC102' row='WS' column='C' tier='' /
  </unit>' />
    <parameter id='craneId' value='QC102' />
    <parameter id='action' value='Set' />
    <parameter id='requestType' value='unitPositionUpdateMessage' />
    <parameter id='visitId' value='CLAD01' />
    <parameter id='visitType' value='VESSEL' />
  </parameters>
  <units-response>
    <unit-response cas-unit-reference='QC102_TEST0000002_REF' cas-transaction-
reference='200' id='' status='SUCCESS' ready-to-transfer='U'>
      <message code='NCAS-2003' text='Unit position update handled successfully
[Action=Set]' />
    </unit-response>
  </units-response>
</payload>
```

14.9.12 Miscellaneous Notes

This section provides important usage details about attributes and other sub-class elements.

14.9.12.1 Position Notes

In earlier versions, 'current position' and 'planned position' elements in the CAS API had three attributes: 'loc-type', 'loc-id', and 'slot', where loc-type represented the type of position (Vessel, Yard etc), loc-id represented the identifier for the type (VesselVisit id, Yard id etc) and slot contained the information about the actual position. The issue with 'slot' was that it was cryptic, and N4-specific, and hence required extra N4 configuration knowledge to decode it into the more user-friendly concepts of block, row, column, and tier, which could then be used by external systems.

In the GA version, the decoding of information in the 'slot' is done by the reference implementation of the CAS API. A new type, 'tCasPosition', has been added to the schema definition. With this change, the current and planned positions have more granular information, with well-defined individual attributes for block, row, column, and tier. This new xsd element replaces the 'tPosition' element, which only had a single attribute ('slot') for defining the current/planned position of the unit.

14.9.12.2 Error/Warning/Info message codes and text returned by N4

For the Error/Warning/Info messages originating from N4, the message code and text will be returned to CAS by N4 along with the original request sent by CAS. The codes will be have format of 'NCAS-nnnn', where NCAS stands for 'Navis Crane Automation System' and the 'nnnn' is a 4 digit number which falls in the following buckets based on the severity of message.

Error/Severe	0001 – 1000, 3001-4000
Warning	1001 - 2000
Information	2001 - 3000

A special error code [NCAS-0000] is reserved for errors thrown by business tier; for these kinds of errors the message text will be different based on the error.

Message code [NCAS-5000] is reserved for successful messages.

The following table details the message codes and their text in reference implementation:

Message Code	Message Text
ERRORS	
NCAS-0000	Business error occurred. Please see the messages section
NCAS-0002	Required parameter [visitType] is missing
NCAS-0003	Required parameter [visitId] is missing
NCAS-0004	Invalid visit type: Valid request types are [VESSEL] and [TRAIN]
NCAS-0005	No carrier visit found
NCAS-0006	Carrier geometry is not available
NCAS-0007	Invalid request type. Valid request types are [carrierGeometryRequest] and [containerListRequest]
NCAS-0008	Invalid request type. Valid request types are "[unitCaptureMessage], [railConeStatusUpdate], [unitPositionUpdateMessage] and [craneTechnicalStatusUpdate] Note: This message is used only release 3.0 and later. Not available in 2.5.
NCAS-0009	No Unit Xml was sent as part of message
NCAS-0010	The Unit Xml sent as part of the message is invalid
NCAS-0011	Invalid capture type. Valid types are [Image], [Identify], [Create] and [Update]
NCAS-0012	Invalid position update type. Valid types are [Lift], [Set]
NCAS-0013	Required unit attribute [id] is missing
NCAS-0014	Required unit attribute [cas-unit-reference] is missing
NCAS-0015	Required parameter [cranelId] is missing

Message Code	Message Text
NCAS-0016	Unit attribute [iso-code] for [action=Create] is missing
NCAS-0017	Invalid crane Id
NCAS-0018	The unit reference in the message is different than the existing unit reference id of the unit
NCAS-0019	The transaction reference in the message is different than the existing transaction reference id of the unit
NCAS-0020	The unit position update message is missing the required attribute [loc-type] in current-position node
NCAS-0021	The unit position update message is missing the required attribute [location] in current-position node
NCAS-0022	The unit position update message is missing the required attribute [slot] in current-position node
NCAS-0023	The unit position update message is missing the required current-position node
NCAS-0024	Invalid request type. Valid request type is [unitCaptureMessage] NOTE: For 2.5 release only
NCAS-0025	Equipment specified as bundled equipment not found
NCAS-0026	Identified unit is not planned for load or discharge
NCAS-0027	Cannot load as position is empty for TBD unit
NCAS-0028	Currently OCR data is not being accepted from this crane
NCAS-0029	The logged in scope is not Yard scope
NCAS-0033	Invalid rail cone status
NCAS-0034	Both label and sequence missing. At least platform sequence or platform label should be in the request.
NCAS-0035	Railcar id is required to update rail-cone status
NCAS-0036	No unit found on-board railcar-visit
NCAS-0037	INVALID PLATFORM SEQUENCE
NCAS-3001	Invalid technical status. Allowed technical status values are Green, Yellow, Orange and Red
NCAS-3101	Invalid offset value
NCAS-3102	Could not find bollard
NCAS-3103	Could not fetch yard bin model
NCAS-3121	Could not find railcar
NCAS-3122	Could not find railcar visit
NCAS-3201	Unable to find work instruction
NCAS-3202	Not enough information to spot a railcar.
WARNINGS	
NCAS-1001	Either [onBoard] parameter should be [Y] or [cranelid] parameter needs to be defined
NCAS-1002	Identified unit is not planned for load or discharge

Message Code	Message Text
NCAS-1003	Identified unit is not planned for load
NCAS-1004	Identified unit is not planned for discharge
NCAS-1005	Unit attribute [iso-code] for [action=Create] is missing
NCAS-1006	Carrier mismatch for Load
NCAS-1007	Carrier mismatch for Discharge
NCAS-1008	Identified unit is not planned and is not valid for TBD load
NCAS-1009	Unit not processed
NCAS-1010	Invalid iso-code
INFORMATION	
NCAS-2001	Unit successfully processed
NCAS-2002	Unit capture handled successfully
NCAS-2003	Unit position update handled successfully

Here is a sample error response from N4 for a container list request which has a invalid visit id:

```
<payload>
<parameters>
  <parameter id='requestType' value='containerListRequest' />
  <parameter id='sendOnBoardUnitUpdates' value='Y' />
  <parameter id='sendCraneWorkListUpdates' value='Y' />
  <parameter id='craneId' value='All' />
  <parameter id='visitId' value='someVisitId' />
  <parameter id='onBoard' value='Y' />
  <parameter id='visitType' value='VESSEL' />
</parameters>
<error code='NCAS-0005' message='No carrier visit found [visitId=someVisitId]' />
</payload>
```

Index

'properties' element.....	56	AGV Exceptions.....	419
'units' element.....	56	AGV Interaction.....	403
For accessories (Equipment hold or permission).....	49	AGV Laden Abort.....	422
2		AGV Rejection.....	420
20' Bays Paired for 40' Containers: No Container		AGV State Correction.....	430
Lengths Allowed.....	503	AGV Unladen Abort.....	421
A		AGV_COMMANDS.....	455
Abort due to Invalid command (ABORT_04).....	374	AGV_CONTAINER_UPDATE.....	447
Abort Interactions.....	299, 367	AGV_ORDERS.....	436
Abort on Door direction (ABORT_03).....	374	AGV_STATUS.....	465
Abort on No Truck (ABORT_06).....	381	Allowed actions.....	50
Abort on Pickup (ABORT_02).....	372	Alternative means of identifying units.....	56
Abort on Set Down (ABORT_01).....	368	API Specifications.....	88
Abort on Truck Mismatch (ABORT_05).....	376	Area Status.....	433
Abort_01 Interaction Diagram.....	301	Area Status Update.....	332
Abort_02 Interaction Diagram.....	302	AREA_STATUS.....	474
Abort_03 Interaction Diagram.....	303	Argo Basic Service Client Example for .NET	
Abort_04 Interaction Diagram.....	304	(ArgoServiceTester).....	93
Abstract AGV Interaction.....	404	Argo Generic and Basic Web Service API.....	83
Accessing the Common Webservice API.....	246	Argo Generic and Basic Web Service API	
ACCS Area Status Table (AREA_STATUS).....	332	Implementation.....	84
ACCS ASC Status Table Layout (STATUS).....	313	Argo Generic Web Service API Format.....	84
ACCS Command Queue Table		ArgoServiceTester skeleton application.....	94
(ASC_COMMAND_QUEUE).....	310	Arriving and Departing Units.....	203
Active Orders and Commands.....	397	ASC Automated Equipment Database Interface	
ActiveMQ Configuration.....	222	(ECN4).....	291
ActiveMQ Failover Configuration.....	227	ASC Exceptions.....	424
ActiveMQ Failover in a NutShell.....	228	ASC Interaction.....	409
ActiveMQ integration with .Net.....	226	ASC Laden Abort.....	426
Add JMS queues to the facilities.....	216	ASC origin or destination vehicle type.....	483
Add Web Reference to N4 Web service in .NET		ASC Rejection.....	425
Application.....	113	ASC Status.....	306
AGV Container Movement Interaction.....	406	ASC Unladen Abort.....	425
AGV Delays.....	423	ASC_COMMANDS.....	458
AGV Divert.....	430	ASC_ORDERS.....	441
AGV Drive Orders.....	408	ASC_STATUS.....	469
		Assumptions.....	293
		Attributes tables.....	498
		Automated Gates.....	245
		Automated Yard Crane Database Interface.....	336

Automation Database Interface.....	394	Configure N4 Tomcat node to Weblogic JMS server. .	221
Available Services.....	79	Configure provider specific parameters.....	217
B		Configure the Universal Query for the First Time. .	31
Basic Authentication.....	20	Configure Tomcat Security.....	32
Basic Scan Validation.....	270	Configuring a Recap Query.....	24
Bay Types.....	346	Configuring an SNX Query.....	23
BERTH_STATUS.....	454	Configuring Notice Requests for the Outbound Queue	219
Billing REST Endpoints.....	170		
booking-information.....	191	Configuring the Default Query.....	22
Boolean Parameters in Universal Query.....	36	Configuring the Query.....	22
C		Constructing the URL.....	26
Calling Groovy Plug-ins from the Customize Window.	238	Container Door Direction.....	346
.....		Container List Request.....	505, 505
Calling the Argo Basic Webservice.....	99	Container vs. Embedded Equipment Element. . .	206
cancel-appointment.....	262	Corrective Actions.....	427
Cancelations.....	386	Crane Automation API.....	486
Cancellations.....	304, 428	Crane Automation API Technical Specification. .	485
cancel-transaction.....	260	Crane Automation System (CAS).....	486
cancel-truck-visit.....	260	Crane ECS Area Status Table (AREA_STATUS). 358	
cancel-truck-visit-appointment.....	253	Crane ECS Command Queue Table	
Cannot Identify Messages.....	531	(YC_COMMAND_QUEUE).....	348
Carrier Geometry Request.....	493, 493	Crane ECS Status Table Layout (STATUS).....	355
Carrier Visit Update Message.....	504, 504	Crane Move Order.....	389
Carrier Visit Update message prerequisite.....	504	Crane Work List Update Message.....	512, 512
Cell Section (front view).....	496	Create a General Notice.....	514, 518
Cell Stack (front view).....	497	Create a N4 external webservice in MS .NET (2008).	129
CHE Current Position API.....	167		
Check Privileges API.....	165	Create a N4 external webservice in MS .NET (2010).	131
Client Web Services Integration for .NET.....	111		
Code Extensions Executed in the Workflow.....	238	Create a Simple Filter.....	33
Code Extensions to Override Gate Tasks.....	239	Create a Unit on the Fly.....	51
Code Snippets/Attachments.....	121	Create an N4 external web-service in C# with Visual	
Common Aspects.....	137	Web Developer Express 2008.....	129
Common Functionality.....	490	Create an N4 external web-service in C# with Visual	
Common Webservice API.....	246	Web Developer Express 2010.....	131
Communication protocol.....	486	Create an N4 User.....	32
Complete Sample Request Message.....	522	Create Custom Event.....	514, 518
Configuration and Security Aspects.....	56	Create Messages.....	530
Configuration to enable Crane Work List Update		Create the Groovy code.....	154
(CWLUI) message.....	514	create-appointment.....	261, 261
Configure JMS Queues.....	224	create-container-notice.....	192
		create-truck-visit.....	259
		create-truck-visit-appointment.....	252

create-truck-visit-appointment(update truck position and door direction).....	252	Elements table.....	497
Creating a REST API with Groovy.....	152	Enable the Groovy message log.....	153
D		Enable Universal Query.....	32
Data Entry Assist.....	244	Equipment Availability.....	402
Data Import Mode.....	184	Equipment Failure.....	402
Data Import Mode vs. Production Mode.....	184	Equipment Interaction.....	401
Data Modeling and Design.....	338	Equipment Status.....	402
Database Interface Definition.....	435	Error cases.....	126
Datatypes.....	436, 478	Error handling.....	157
Debugging and Error Handling.....	220	Error Handling.....	85
Default Workflows.....	363	Error/Warning/Info message codes and text returned by N4.....	536
delete-container-notice.....	193	ESC Status.....	465
Deletion.....	207	Event Processing Via JMS for N4.....	215
Deploying the Groovy REST API.....	158	Event Recording for ICU Updates.....	57
Development Guide.....	87	Events.....	56
Development Platforms and Web Service Toolkits.....	79	Example.....	123, 125, 238, 240, 68
Disabling XPSCache Cluster Loading for Data Migration.....	209	Example .Net Gate Integration.....	121
Dispatched Moves Completed by Remote Operator.....	385	Example diagram for ASC block, waterside transfer point and waterside locations.....	481
Door Direction Changes.....	306	Example Groovy.....	155
DRIVE_TIME_STATUS.....	471	Example Request.....	278
Driver Interaction.....	267	Examples.....	185
Dropoff Details.....	268	Exception/Transaction Handling.....	72, 71, 71, 71
E		Exceptions.....	419
ECN4 Start Up and Recovery Process.....	306	Executing the Query.....	28
ECN4 Transfer Point Status Table (TRANSFER_POINT_STATUS).....	315	Export SNX File – Ship Bin Model.....	289
ECN4 Transport Order Table (ASC_TRANSPORT_ORDERS).....	307	Expose User SNX schema to Web service.....	122
ECS Commands.....	455	F	
ECS Initiated Crane Movement Order.....	390	Fail Unexpected Container.....	271
Either required and/or optional predicates.....	39	Failover Setup.....	228
Element <bin-name>.....	499	Feature Summary.....	293
Element <bin-name-table>.....	499	flags.....	193
Element <cell-section>.....	500	For bills of lading.....	49
Element <cell-stack>.....	501	For chassis (Equipment hold or permission).....	49
Element <parameter>.....	498	For containers (Equipment hold or permission).....	49
Element <ship-block>.....	500	For export bookings.....	49
Element <ship-model>.....	498	For Rail Operations.....	514
Element <truck-visit>.....	248	For units we need to use a unit-identity tag to define the correct target.....	49

Functional Columns.	348	Interface Specific Details.	492
G		Intra Stack Move.	321
Gate API Message Naming Conventions.	247	Introduction. 17, 147, 179, 278, 292, 337	
Gate API Specification.	243	Invocation.	147
Gate API Specification Overview.	244	Invoking N4 RESTful API.	166
Gate Workflow Customization.	232	Invoking Outbound WebService from N4 which is compliant with N4 basic service.	146
Gate Workflow Customization Options.	233	Invoking the Groovy REST API.	158
General API Structure.	246	Invoking the web service call.	148
General Process Flow.	245	Invoking Web Services Programmatically.	90
General Status.	86	J	
Giving roles to the n4api user.	33	Java Client Integration Example.	105
Groovy Code in the Customize Window.	237	JMS Configuration for Inbound and Outbound Queues.	216
Groovy plug-ins available for pre and post inbound reefer message processing.	70	JMS Failover Scenarios.	229
H		JSON Processing.	154
Holds Permissions Update (HPU) API.	46	K	
How Client Invokes the Generic Service.	106	Keep things Tidy.	204
How to Locate the Generic ArgoWebservice in java code.	106	Key Generation.	398
How to look up for static code values.	42	L	
How to test it.	143	Landside OTR Deliver 2 Imports.	326
HPU API.	47	Landside OTR Deliver Import.	325
HPU format.	48	Landside OTR Receive 2 Exports.	324
HTTPS.	20	Landside OTR Receive Export.	323
I		Landside OTR Receive Export & Deliver Import.	327
IBCMA Rail Cone Status Update in CAS By Platform.	521	Landside OTR Receive Export & Deliver Import w/ Remain on Truck Container.	329
IBCMA Rail Cone Status Update in CAS Interface.	520	Landside UTR Inbound Move (Wheeled Area to ASC).	330
ICU API.	55	Landside UTR Outbound Move (ASC to Wheeled Area).	331
ICU Examples.	57	Lane Monitor View.	273
Identify Messages.	529	Levels of Gate Automation.	244
Image Messages.	528	Limiting Access to the Query Url.	30
Import Process.	206	list-transactions.	252
Import Ship File.	287	Load Swap Request.	514
Import-Export Ship Bin Model.	286	Load Swap Request Message.	514
Inbound Messages from Reefer Monitoring System to N4.	63	Location Status.	432
Including schema extension fields.	275	Locations.	478
Incremental Container Update (ICU) API.	54	Logical business entities.	48
Informational Columns.	352		
Initial data conversion via SNX.	205		

LogUserMessage.....	236	Overview of the N4 REST API Service.....	151
M		P	
Marrying the Chassis to a Container.....	187	Parameters.....	27
Message content.....	137	Performance and Scalability.....	335
Message content - Common Aspects.....	130	Pickup Selection.....	268
Message Description.....	497	Position Notes.....	535
Message Structure.....	56	Position on truck chassis.....	483
Miscellaneous Notes.....	535	Prepare ActiveMQ on multiple machines Setup..	229
Monitor JMS Queues.....	223	Preparing ActiveMQ for the same machine setup	228
Multiple Reefer Update Message.....	68	Prerequisites for this message.....	513
N		Prerequisites for this Message.....	518
N4 and ECS Application Status (N4_APPLICATION_STATUS, ECS_APPLICATION_STATUS).....	360	Primary Interaction.....	295
N4 Compatible External Web Service.....	128	print-ticket.....	265
N4 JMS Restarting Connection.....	228	Process API's Provided by N4.....	250
N4 Orders.....	436	Process API's Provided by Other Systems.....	264
N4 Passthrough for EVP.....	169	Process Pre-lodged Dropoffs.....	270
N4 REST API.....	150	Process Pre-lodged Pickups.....	271
N4 Setup and Environment.....	112	Process Truck API Workflow.....	269
N4 Status.....	447	process-truck.....	253
N4 Web Services.....	77	Production Mode.....	184
N4 Web Services: Introduction.....	78	Providing time stamps.....	397
need-assistance.....	261	Pull Bare Cassette from LSTZ.....	332
Non-Functional Specification.....	334	Purge Yard Crane database tables.....	392
Notification and Startup.....	476	Q	
Notification Mechanism.....	398	QC Container Movement Interaction.....	413
notify-arrival.....	259	QC Interaction.....	412
notify-resume.....	265	QC Updates via CAS.....	518
O		QC_COMMANDS.....	463
Offline Operation of ASC and Inventory Synchronization.....	304	QC_ORDERS.....	444
Offline Operation of Crane and Inventory Synchronization.....	387	QC_STATUS.....	471
Onboard Unit Update Message.....	516, 516	QC_WORK_QUEUE_STATUS.....	453
open-gate-arm.....	265	Query All.....	66
Optional boolean predicates.....	37	Query for Reefers but exclude non move history, chargeable unit event, and guarantee table records..	66
Orientation.....	482		66
Outbound Messages from N4 to Reefer Monitoring System.....	73	Query for Reefers IDs only.....	66
Overview.....	395	Query results for a single column.....	26
Overview of Gate Workflow Execution.....	234	Query results for three columns.....	25
		Query results for two columns.....	25
		Querying Reefers.....	64

Querying specific reefer(s)..... 64

R

Rack Suitability..... 403
 Rail Inspection - Rail Load..... 525
 Rail Messages in CAS API..... 520
 Rail Spotting interface in CAS API..... 521
 read-preadvise..... 251
 Ready To Transfer Message..... 524
 record-scan (truck + equipment)..... 251
 record-scan (truck only)..... 251
 Reefer Messaging Flow..... 62
 Reefer Monitoring API..... 61
 Reefer Monitoring XML API..... 60
 ReeferMonitoringWebServiceHandlerImplPostInvoke..... 71
 ReeferMonitoringWebServiceHandlerImplPostInvokeInTx..... 71
 ReeferMonitoringWebServiceHandlerImplPreInvoke..... 70
 ReeferMonitoringWebServiceHandlerImplPreInvokeInTx..... 71
 Reference Entities..... 181
 Refreshing an access token after it expires..... 160
 RejectConditionTrue..... 237
 Related Topics..... 92
 Re-organization of the SNX unit element..... 210
 Report Generation Web Service..... 139
 reprint-most-recent-ticket..... 270
 Request Sample for spotting a railcar by GPS... 522
 Request Sample for spotting a railcar by Meter-mark..... 523
 Request Sample for spotting a railcar by TP... 522
 request-pickup-appointments..... 259
 request-stage-details..... 252
 request-stage-details (visit/tran key)..... 252
 request-trouble-transactions..... 252
 request-visit-details..... 252
 Required boolean predicates..... 38
 Requirements..... 62
 Response Format Configuration Settings..... 248
 Response Sample..... 523
 Responsibilities..... 397
 REST Authentication API..... 161

REST Protocol..... 18
 Restart the JMS connections..... 218
 Road Truck Chassis Positions..... 305, 346
 Running the C# Client..... 118

S

Sample .NET C# Client Accessing ICU on N4... 115
 Sample client implementation (Java)..... 490
 Sample ContainerListRequest Message (to N4) for rail..... 508
 Sample ContainerListRequest Message (to N4) for vessel..... 507
 Sample ContainerListRequest Response (from N4) For Rail..... 511
 Sample ContainerListRequest Response (from N4) For Vessel..... 508
 Sample Java Client..... 143
 Sample Message (from IBCMA)..... 520, 521
 Sample Message (from N4)..... 504, 524
 Sample Message (to N4)..... 519, 519
 Sample Message Sent In Response Unit Capture Identify..... 525
 Sample Message(from N4)..... 512, 517
 Sample messages..... 490
 Sample Request..... 64
 Sample Request Inputs..... 493
 Sample Requests..... 515
 Sample Response..... 64
 Sample Response (from N4)..... 519, 520
 Sample Response Message for Rail Request (from N4)..... 494
 Sample Response Message for Vessel Request (from N4)..... 501
 Sample Source Code of a Standalone (Java) Webservice Client..... 107
 Sample Unit Capture Create Message (to N4)... 530
 Sample Unit Capture Create Response (from N4) 530
 Sample Unit Capture Identify Message (to N4)... 529
 Sample Unit Capture Identify Response Message (from N4)..... 529
 Sample Unit Capture Image Message (to N4)... 528
 Sample Unit Capture Image Response Message (from N4)..... 528
 Sample Unit Position Update Message Request (from CAS to N4)..... 534

Sample Unit Position Update Message Response (From N4 to CAS).....	535	Status and Error Reporting.....	138
Security.....	19, 53	Status Messages.....	86
Security - Authentication.....	120	Steps to run Java standalone Webservice Client.....	109
Security and Session Management.....	81	Steps to set up an outbound service in N4:.....	147
Security for the REST API.....	160	Structure of an SNX document.....	179
Security Setup.....	82	submit-multiple-transactions.....	260
Self-help Gates.....	245	submit-transaction.....	259
Semi-automation.....	244	Summary.....	234
Service Order API.....	173	Summary of Customization Options.....	234
Service Rules.....	48	Support for Custom API messages.....	274
Set up API access to N4.....	152	Supported Entities.....	181
SetNextStage.....	237	Supported Queries.....	140
Setup.....	278	Supported Reefer Details.....	62
Setup ActiveMQ.....	223	Synchronization Points.....	399
Setup and Environment for .NET.....	112	System Interactions.....	294
Ship Bin Model.....	276	T	
Ship Bin Model Request.....	277	Task Filters.....	234
Ship Bin Model Structure.....	495	Task to Override the Next Stage Determination.....	237
Ship Block (side view).....	496	Tasks that Create Custom Error Conditions.....	236
Ship Model Overview.....	497	Technical Requirements.....	335
Signaling Activities with SNX.....	202	Technical status.....	482
Simple Unit movements within the facility.....	203	Test the API in your browser.....	33
Single Reefer Update Message.....	66	TEST_COMMANDS.....	465
SNX Export.....	124	TEST_ORDERS.....	446
SNX Import.....	123	Testing Reefer Monitoring Web Services.....	73
SNX vessel-visit import (Create/update vessel visit using SNX via Web Services).....	124	Testing the Generic Services.....	88
snx-export.....	194	The <equipment> message.....	199
Solicit Appointment Selection.....	271	The <equipment-damages> message.....	199
Solicit Further Details.....	271	The <unit-assign-to-booking> message.....	200
Solicit Pickup Selection.....	271	The <unit-cancel-preadvise> message.....	200
Some notes.....	185	The <unit-hazards> message.....	198
SPARCS_APPLICATION_STATUS (N4), ECS_APPLICATION_STATUS (ECS).....	477	The <unit-property-update> message.....	198
Special Considerations: <unit> element.....	183	The <unit-renumber> message.....	199
Special Considerations: other Elements.....	189	The <unit-return-to-storage> message.....	199, 200
stage-done.....	260	The <unit-routing> message.....	197
Standards Compliance.....	79	The 'is-correction-only' attribute.....	203
Start and use Microsoft Visual Studio .NET to create your application.....	113	Timestamps.....	180
Startup and Recovery Process.....	399	Topology Information.....	82
Static Code Lookup.....	41	TOS Crane Transport Order Table (YC_TRANSPORT_ORDERS).....	339
		TOS Initiated Crane Movement Order.....	389
		Total Automation.....	244

Transfer Point Status	433	update-appointment	252, 262
Transfer Point Status Update	315	update-truck-visit (scale-weight-kg)	252
Transfer Point Status Use Cases	321	update-truck-visit-appointment	252
TRANSFER_POINT_STATUS	475	Updating Reefers	66
Transformation of Customer's XML Data Into SNX	207	Use Cases for the REST Authentication API	162
Transport Order Data Flow	295	user	190
Transport Order primary interaction	364	User Interface Considerations	333
TRIGGER_EVENT_STATUS	476	Using ActiveMQ as a JMS provider (Add Link) ..	217
TRIGGER_STATUS_HANDLED_SPARCS (N4), TRIGGER_STATUS_HANDLED_ECS (ECS) ..	477	Using MassUnitType	127
Triggering Groovy via Web Services	92	Using SNX	177
Trouble Processing	271	Using the Universal Query	21
Troubleshooting	28	V	
Troubleshooting ActiveMQ	225	Verification of the webservice from N4	130, 137
Truck Status (TRUCK_STATUS)	361	Vessel Load Swapping	417
truck-arrival	265	Vessel Work Information	418
truck-departure	266	vessel-visit, train-visit	190
trucking-company	190	View Bin Plan	289
U		W	
UI Filter Screen	37	Waterside Inbound Move	321
Unit Capture CannotIdentify Message (to N4) ..	531	Waterside Inter Stack Move	322
Unit Capture CannotIdentify Response Message (from N4)	532	Waterside Outbound Move	321
Unit Capture Messages	525	Web service hosted by CAS	488
Unit Capture Update Message (to N4)	531	Web service hosted by N4	487
Unit Capture Update Message Response (from N4) ..	531	Weight measurements	397
Unit Categories	43	What does it do	140
Unit Dray Statuses	42	What is SNX?	178
Unit Freight Kinds	42	Work Instruction Move Types	42
Unit Hazards and IMDG Classes	43	Work Status	482
Unit Position Update Message	532, 532	WORK_INSTRUCTION_STATUS	448
Unit updates and transactions on Units with SNX	196	WORK_QUEUE_BUFFER_STATUS	472
Unit XML example used in samples below	528	Working Entities	182
Universal Query	40	WSDL Definitions	491
Universal Query API	16	X	
Universal Query Java Integration Example	34	XML Payload Formats	247
Unsolicited Moves	428	XML Request Format	247
Update Messages	531	XML Response Format Configuration	248
Update Process & Return Error Messages	51	XML Response Payload Format	248
Update QC position via CAS	519	XML Schema	142
Update QC technical status and location via CAS	518	XML Troubleshooting Hints for First-time SNX/XML Customers	208